



Siemens EDA

Questa® ADMS™ User's Manual

Software Version 2026.1

Revision Number 33

Unpublished work. © 2026 Siemens

This Documentation contains trade secrets or otherwise confidential information owned by Siemens Industry Software Inc. or its affiliates (collectively, "Siemens"), or its licensors. Access to and use of this Documentation is strictly limited as set forth in Customer's applicable agreement(s) with Siemens. This Documentation may not be copied, distributed, or otherwise disclosed by Customer without the express written permission of Siemens, and may not be used in any way not expressly authorized by Siemens.

This Documentation is for information and instruction purposes. Siemens reserves the right to make changes in specifications and other information contained in this Documentation without prior notice, and the reader should, in all cases, consult Siemens to determine whether any changes have been made.

No representation or other affirmation of fact contained in this publication shall be deemed to be a warranty or give rise to any liability of Siemens whatsoever.

If you have a signed license agreement with Siemens for the product with which this Documentation will be used, your use of this Documentation is subject to the scope of license and the software protection and security provisions of that agreement. If you do not have such a signed license agreement, your use is subject to the Siemens Universal Customer Agreement, which may be viewed at <https://www.sw.siemens.com/en-US/sw-terms/base/uca/>, as supplemented by the product specific terms which may be viewed at <https://www.sw.siemens.com/en-US/sw-terms/supplements/>.

SIEMENS MAKES NO WARRANTY OF ANY KIND WITH REGARD TO THIS DOCUMENTATION INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, AND NON-INFRINGEMENT OF INTELLECTUAL PROPERTY. SIEMENS SHALL NOT BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, CONSEQUENTIAL OR PUNITIVE DAMAGES, LOST DATA OR PROFITS, EVEN IF SUCH DAMAGES WERE FORESEEABLE, ARISING OUT OF OR RELATED TO THIS DOCUMENTATION OR THE INFORMATION CONTAINED IN IT, EVEN IF SIEMENS HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

TRADEMARKS: The trademarks, logos, and service marks (collectively, "Marks") used herein are the property of Siemens or other parties. No one is permitted to use these Marks without the prior written consent of Siemens or the owner of the Marks, as applicable. The use herein of third party Marks is not an attempt to indicate Siemens as a source of a product, but is intended to indicate a product from, or associated with, a particular third party. A list of Siemens' Marks may be viewed at: <https://www.sw.siemens.com/en-US/trademarks/>. The registered trademark Linux® is used pursuant to a sublicense from LMI, the exclusive licensee of Linus Torvalds, owner of the mark on a world-wide basis.

About Siemens Digital Industries Software

Siemens Digital Industries Software is a global leader in the growing field of product lifecycle management (PLM), manufacturing operations management (MOM), and electronic design automation (EDA) software, hardware, and services. Siemens works with more than 100,000 customers, leading the digitalization of their planning and manufacturing processes. At Siemens Digital Industries Software, we blur the boundaries between industry domains by integrating the virtual and physical, hardware and software, design and manufacturing worlds. With the rapid pace of innovation, digitalization is no longer tomorrow's idea. We take what the future promises tomorrow and make it real for our customers today. Where today meets tomorrow. Our culture encourages creativity, welcomes fresh thinking and focuses on growth, so our people, our business, and our customers can achieve their full potential.

Support Center: <https://support.sw.siemens.com>

Send Feedback on Documentation: https://support.sw.siemens.com/doc_feedback_form

Revision History ISO-26262

Revision	Changes	Status/ Date
33	Modifications to improve the readability and comprehension of the content. Approved by Jonathan Ashworth. All technical enhancements, changes, and fixes listed in the Solido Simulation Suite Release Notes for this product are reflected in this document. Approved by Sathishkumar Balasubramanian.	Released February 2026
32	Modifications to improve the readability and comprehension of the content. Approved by Jonathan Ashworth. All technical enhancements, changes, and fixes listed in the Solido Simulation Suite Release Notes for this product are reflected in this document. Approved by Sathishkumar Balasubramanian.	Released September 2025
31	Modifications to improve the readability and comprehension of the content. Approved by Jonathan Ashworth. All technical enhancements, changes, and fixes listed in the Solido Simulation Suite Release Notes for this product are reflected in this document. Approved by Sathishkumar Balasubramanian.	Released June 2025
30	Modifications to improve the readability and comprehension of the content. Approved by Jonathan Ashworth. All technical enhancements, changes, and fixes listed in the Solido Simulation Suite Release Notes for this product are reflected in this document. Approved by Sathishkumar Balasubramanian.	Released February 2025

Author: In-house procedures and working practices require multiple authors for documents. All associated authors for each topic within this document are tracked within the Siemens documentation source. For specific topic authors, contact the Siemens Digital Industries Software documentation department.

Revision History: Released documents include a revision history of up to four revisions. For earlier revision history, refer to earlier releases of documentation on Support Center.

Table of Contents

Introduction.....	1-1
Questa ADMS Overview.....	1-1
How To Use Questa ADMS User Documentation.....	1-3
AMS Languages Support.....	1-6
Standards Supported.....	1-6
 Environment Settings.....	 2-1
System Initialization File (modelsim.ini).....	2-1
Library Path Variables.....	2-3
Setting Up Libraries for Group Use.....	2-4
SPICE Subcircuit Generation Variables.....	2-6
Questa SIM Simulation Control Variables.....	2-7
Simulator Control Variables.....	2-9
Setting VectorPatternString.....	2-17
VHDL Compiler Control Variables.....	2-18
VHDL-AMS Compiler Control Variables.....	2-20
Verilog-AMS Compiler Control Variables.....	2-21
Eldo Initialization File (eldo.ini).....	2-21
Questa ADMS GUI Preferences.....	2-22
Changing GUI Preferences.....	2-22
GUI Preference Variables.....	2-25
Optional Environment Variables.....	2-27
VA_INCLUDE_PATH.....	2-28
QSIM_INI.....	2-29
AMS_VCO_MODE.....	2-30
QSIM_LOCATION_MAP.....	2-30
LD_LIBRARY_PATH.....	2-30
AMS_ADMS_LICENSE_QUEUE.....	2-31
Files Used by Questa ADMS.....	2-32
Location Maps.....	2-37
 Questa ADMS GUI Overview.....	 3-1
Questa ADMS Application Interface.....	3-2
Window Zooming.....	3-4
Undocking and Docking Windows.....	3-5
Moving Windows.....	3-6
Organizing Column Information.....	3-7
Quick Access Toolbars.....	3-7
GUI Icons and Their Meanings.....	3-8
Status Bar.....	3-9
Scaling Fonts.....	3-10
Find and Filter Functions.....	3-10

Filtering and Sorting Objects in the Objects Window.....	3-13
Wildcard Modes when Searching.....	3-14
Filtering Objects by Type in the Structure Window.....	3-15
Editing GUI Window Colors.....	3-16
Editing GUI Window Font Type and Size.....	3-17
Customizing the Simulator GUI Layout.....	3-17
Assigning Layouts to Modes.....	3-18
Customizing the Main Application Window Title.....	3-19
Bookmarks.....	3-19
Save/Reuse of Questa ADMS GUI Configuration.....	3-19

Design Preparation..... 4-1

Working with SPICE Netlists.....	4-1
Declaring and Instantiating Language Descriptions.....	4-1
Declaring and Instantiating HDL Models.....	4-2
.ckt or .cir File.....	4-3
Defining Parameter or Generic Values.....	4-9
Instantiating VHDL/VHDL-AMS Entities with Composite Ports in SPICE.....	4-13
Attaching Ports of Composite Types in SPICE.....	4-14
Requesting Plots of Mixed-Signal Simulation Results in a SPICE Netlist.....	4-15
Instantiating a SPICE Subcircuit from VHDL, VHDL-AMS or Verilog.....	4-17
DSPF Parasitics on AMS Ports.....	4-18
Including a Spectre Netlist.....	4-18
Merging of Instances in Parallel.....	4-19
Instantiating Subcircuits From Multiple Libraries.....	4-20
Working with VHDL and Verilog.....	4-21
Mapping Verilog Vectors to VHDL.....	4-21
Rules for Instantiating VHDL-AMS in VHDL Models.....	4-22
VHDL and VHDL-AMS Configuration Declarations.....	4-24
Bus Connection Examples.....	4-26
Access to Simulation Parameter Values.....	4-28
Verilog Instantiating VHDL-AMS.....	4-29
Simulator Resolution Limit (Verilog).....	4-30
Working with VHDL-AMS.....	4-31
VHDL-AMS Instantiating SPICE.....	4-32
VHDL-AMS Access to SPICE Parameter Values.....	4-36
Support of Records in VHDL-AMS.....	4-37
Composite Types in VHDL-AMS.....	4-37
Rules for VHDL-AMS Instantiating VHDL.....	4-38
Util Package for VHDL-AMS.....	4-41
Return Types of VHDL-AMS Functions.....	4-42
Working with Verilog-AMS.....	4-43
Verilog-AMS Instantiating SPICE Models and Subcircuits.....	4-43
Verilog-AMS Instantiating SPICE Primitives.....	4-44
Verilog-AMS Access to SPICE Parameter Values.....	4-46
Rules for Verilog-AMS Instantiating Verilog.....	4-47
Verilog-AMS Instantiating VHDL-AMS.....	4-51
Verilog-AMS Branches.....	4-52

Verilog-AMS Variables.....	4-60
Formulating Verilog Hierarchical Names in Mixed-Language Simulations.....	4-61
Using wreal.....	4-64
Opening, Closing, and Writing to Files During Simulation.....	4-66
AC Noise Results for Verilog-AMS Noise Functions.....	4-68
Working With Verilog-A.....	4-69
Using Verilog-A v1.0 Code in Questa ADMS.....	4-70
Mapping Ports Using the Interface Matcher.....	4-71
Associating Design Units with vamatch.....	4-74
Associating Design Units Using the .BIND Command.....	4-76
Connecting Nets.....	4-78
Naming Conventions.....	4-79
Writing Object Names.....	4-80
Writing Hierarchical Path Names.....	4-82
Obtaining a Hierarchical Path Name.....	4-86
References to Verilog Escaped Identifiers.....	4-87
Extended Identifiers.....	4-88
Instance Names of Digital Models in SPICE Subcircuits.....	4-89
Case-sensitivity for VHDL-AMS Declarations.....	4-90
Design Libraries.....	5-1
Overview of Design Libraries.....	5-1
Creating a Design Library.....	5-3
Specifying a Library at Compilation.....	5-4
Changing the Working Library.....	5-4
Editing Library Mappings.....	5-5
Viewing Library Contents.....	5-5
Deleting Library Contents.....	5-7
Refreshing Libraries.....	5-7
Maintaining Libraries Using Make Files.....	5-8
Compilation.....	6-1
Compilation Commands.....	6-1
Normal Compilation Example: SPICE Instantiating VHDL-AMS.....	6-3
Special Compilation Examples.....	6-4
Special Compilation Case: VHDL Instantiating VHDL-AMS.....	6-4
Special Compilation Case: VHDL Instantiating SPICE.....	6-6
Special Compilation Case: Verilog Instantiating SPICE.....	6-9
Using Previously Compiled Design Units.....	6-10
Sharing VHDL Packages.....	6-11
Importing Design Units Manually.....	6-11
Using Libraries of Design Units Compiled on Other Platforms.....	6-12
Lock Mechanism.....	6-13
Simulation.....	7-1
Interactive Simulation.....	7-1

Running an Interactive Simulation.....	7-1
Examining Design Files in the Source Window.....	7-5
Accessing Language Templates.....	7-7
Running Commands from the Transcript Window.....	7-8
Changing Values in the Locals Window.....	7-10
Batch Mode Simulation.....	7-10
Passing Options Directly to Eldo and ADiT.....	7-11
Multiple-Top-Design Simulation.....	7-12
Multiple-Run Analyses.....	7-13
Viewing Simulation Results.....	7-19
Which Waveform Viewer Should I Use?.....	7-20
Adding Signals to the Solido Waveform Analyzer Wave Window.....	7-20
Waveform Database File Formats.....	7-22
Removing Items from the Solido Waveform Analyzer Wave Window.....	7-23
Adding Items to the Log File.....	7-24
Incremental Saving Mode.....	7-25
Automating Simulation Processes.....	7-25
Loading a Specific Design Unit.....	7-26
Questa ADMS Command File.....	7-26
Startup Files (do Files).....	7-27
The modelsim.tcl File.....	7-28
Saving and Reloading Formats and Content.....	7-28
Manually Saving the Transcript File.....	7-29
Reusing Solido Waveform Analyzer Configurations.....	7-30
Methods for Setting Simulation Parameters.....	7-32
Simulation Save and Restart.....	7-33
Save/Restart Commands.....	7-33
Saving a Simulation at Regular Intervals.....	7-36
Saving and Restarting a Simulation Interactively.....	7-38
Checkpoint the Simulation From the Test Bench.....	7-40
Using Simulation Parameters to Restore Checkpoints.....	7-48
Simulator Command Time Units.....	7-50
Optimizing Simulation.....	7-50
Controlling Simulation Accuracy.....	7-51
Accuracy Control Options for Analog Simulation.....	7-55
Crossing Time Accuracy Options Summary.....	7-57
Performance/Accuracy Trade-off Options.....	7-58
Analog Solver Delayed Start.....	7-61
Speeding Up Elaboration of SPICE Subcircuits.....	7-63
Improving Performance and Reducing Memory Consumption.....	7-63
Multi-Threading.....	7-64
Using Questa ADMS Premier.....	7-65
VHDL-AMS Attribute 'MGC_FOLLOW.....	7-66
Preoptimizing Pure Digital Subregions.....	7-66
Simulation Time Reporting.....	7-67
Controlling HDL Domain Errors.....	7-68
Monte Carlo Analysis.....	7-68

Verification.....	8-1
Interactive Debug.....	8-1
Setting File-Line Breakpoints.....	8-1
Adding and Removing Breakpoints.....	8-2
Modifying File-Line Breakpoints.....	8-3
Saving and Loading Breakpoints.....	8-4
Setting Break Severity.....	8-4
Interrogating Nets in the Contributor Window.....	8-6
Visualizing Current Flow in the CurrentFlow Window.....	8-8
Interactive Forcing and Loading.....	8-10
Non-Convergence Errors.....	8-14
Recompiling Design Units in the Source Window.....	8-15
Diagnosis Mode.....	8-16
Verbose Error Message Reporting.....	8-19
Limiting Message Display.....	8-20
Viewing Messages in the Message Viewer.....	8-21
Simulation Statistics.....	8-23
Viewing Statistics in Interactive Mode.....	8-23
Statistics File.....	8-27
Example: Analyzing Simulation Issues Using the Statistics File.....	8-28
Integration with Questa SIM Verification Methodology.....	8-30
Coverage Windows.....	8-31
Code Coverage in Interactive Mode.....	8-32
Creating Coverage Reports.....	8-36
Predefined Test Attribute Data.....	8-37
Questa SIM Verification Management Windows.....	8-39
Capturing VHDL-AMS Assertion Data in Interactive Mode.....	8-40
Capturing VHDL-AMS Assertion Data in Batch Mode.....	8-46
Naming Conventions for Assertions.....	8-49
Capturing SPICE Extractions in a UCDB.....	8-50
Capturing Multiple-Run Simulation Data in a UCDB.....	8-56
Plotting VHDL-AMS Assertions.....	8-58
Plotting SPICE SOA Assertions.....	8-59
SystemVerilog Assertions.....	8-60
SystemVerilog bind Construct.....	8-66
Access to Analog Objects from SystemVerilog.....	8-68
get_potential.....	8-70
get_flow.....	8-72
Universal Verification Methodology.....	8-73
 Boundary Elements.....	 9-1
What is a Boundary Element?.....	9-1
Signal Boundary Model.....	9-1
What is a Mixed-Signal Net?.....	9-3
How Mixed-Signal Nets Behave in Questa ADMS.....	9-4
Unidirectional Mixed-Signal Nets.....	9-5
Bidirectional Mixed-Signal Nets.....	9-6
Pure Analog Interconnect.....	9-7

Built-in or Custom Boundary Elements.....	9-8
Writing Custom VHDL-AMS Boundary Elements.....	9-9
Boundary Element Insertion.....	9-10
Inserting Boundary Elements.....	9-10
Controlling Boundary Element Assignment.....	9-11
Inserting Boundary Elements Using Connect Rules.....	9-15
Consideration of Continuous Disciplines.....	9-20
Cross-Domain Hierarchical References.....	9-22
Built-in A2D and D2A Boundary Elements.....	9-23
Real A2D Converter Tolerance.....	9-24
Special Supply Boundary Elements.....	9-25
Logical States with Built-In D2A Boundary Elements.....	9-26
Digital-to-Digital (Logic-Real) Boundary Elements.....	9-28
Viewing Mixed-Signal Nets in Questa ADMS.....	9-36
Viewing Boundary Elements in the Structure Window.....	9-37
Plotting Mixed-Signal Nets.....	9-38
Naming Conventions for Boundary Elements.....	9-40
Boundary Elements Log File.....	9-42
Boundary Elements Examples.....	9-43
Boundary Elements Example: Cross Interface.....	9-43
Boundary Elements Example: Use of the .MODEL Command.....	9-44
Boundary Elements Examples: Net Spy.....	9-46
 Using Power Aware Simulation in Questa ADMS	 10-1
Questa ADMS Power Aware Simulation (PAsim).....	10-1
Connecting Power to Analog Power Ports.....	10-3
Example: Connecting UPF Power to Power Supply Pins in a SPICE Subcircuit.....	10-4
Example: Connecting UPF Power to Power Supply Ports of a Verilog-AMS Module...	10-6
Example: Connecting UPF Power to Power Supply Ports in a VHDL-AMS Entity.....	10-7
Power Connect Elements.....	10-8
Power-Aware Signal Connect Elements.....	10-10
Plotting Nets of Type supply_net_type.....	10-13
Mixed-Signal Testbench with 2 Power Domains.....	10-14
 Using Questa ADMS with the Questa Verification Run Manager (VRM)	 11-1
Questa Verification Run Manager.....	11-1
Running Questa ADMS Simulations Under the VRM.....	11-1
Running Multiple Run Questa ADMS Simulations Under the VRM.....	11-2
 Examples.....	 12-1
Summary of the Questa ADMS Examples.....	12-2
12-Bit Analog to Digital Converter.....	12-3
Mixed-Language Ring Oscillator.....	12-6
C Code Encapsulation.....	12-9
SPICE Inverter in a Top Verilog Design.....	12-11
VHDL-AMS Description of a Phase Locked Loop.....	12-12

Verilog-AMS Description of an Integrator.....	12-15
Verilog-AMS Description of Phase Lock Loop.....	12-16
Creating and Compiling a connectrules File.....	12-19
SystemC and VHDL Description of a Flip-Flop Latch with a Top-Level SPICE Netlist.	12-22
Questa ADMS-ADiT Examples.....	12-24
Inverter Chain with Verilog-on-Top.....	12-24
Questa ADMS-ADiT with SPICE on Top.....	12-25
ADiT SPICE and Verilog Description of an Inverter Chain.....	12-27
SPICE and Verilog Inverter Chain Example Source Files.....	12-31
8-bit Adder Using AND, OR, and XOR VHDL VITAL Gates.....	12-34
8-bit Adder Using AND, OR, and XOR Verilog Primitives.....	12-37
Boundary Elements Insertion in a Mixed-Language Design.....	12-38
Verilog-AMS Amplifier with a Resolved wreal Input and an Electrical Output.....	12-41
Modeling Examples.....	12-43

Net Spy..... 13-1

Introduction to Net Spy.....	13-1
Net Spy Implementation in Questa ADMS.....	13-2
Coverage and Limitations.....	13-3
Net Spy for Digital and Analog Objects.....	13-4
init_signal_spy.....	13-5
init_signal_spy Example.....	13-8
\$init_signal_spy Example.....	13-9
Verilog-AMS Model Example.....	13-11
init_signal_spy Example with Top SPICE.....	13-12
enable_signal_spy.....	13-18
disable_signal_spy.....	13-20
signal_force.....	13-22
signal_force Example.....	13-24
signal_release.....	13-28
Net Spy for Analog Objects.....	13-29
init_terminal_short.....	13-30
init_terminal_short Example.....	13-34
\$init_terminal_short Example.....	13-36
init_terminal_reference.....	13-39
init_terminal_reference Example.....	13-43
\$init_terminal_reference Example.....	13-45
init_terminal_contribution.....	13-48
init_terminal_contribution Example.....	13-51
\$init_terminal_contribution Example.....	13-53
get_terminal_id.....	13-56
get_terminal_across_value.....	13-58
get_terminal_across_value Example.....	13-59

Value Change Dump (VCD) Files 14-1

Overview of VCD Files.....	14-1
Creating an Extended VCD File.....	14-1

Extended VCD File Creation Example.....	14-3
Creating a Four-State VCD File.....	14-4
Creating a VCD File.....	14-4
Re-simulation.....	14-6
Four-State VCD File Example.....	14-6
C Code Encapsulation.....	15-1
C Code Conventions.....	15-1
Use of C Functions in VHDL-AMS.....	15-2
C Code Encapsulation Procedure in Questa ADMS.....	15-3
C Template in Questa ADMS.....	15-4
Correspondence Between C Type and VHDL-AMS Type.....	15-7
Predefined Macros in Questa ADMS.....	15-7
MathWorks Integration.....	16-1
Mathworks Integration Overview.....	16-1
MATLAB Testbench Configuration.....	16-3
MATLAB Component Configuration.....	16-5
Simulink Cosimulation Configuration.....	16-7
Dialog and Field Reference.....	17-1
Assertions Window.....	17-2
Contributor Window.....	17-7
CurrentFlow Window.....	17-9
Design Options Dialog Box.....	17-12
Eldo Commands Dialog Box.....	17-13
File Breakpoint Dialog Box.....	17-18
Force Selected Signal Dialog Box.....	17-20
Interface Matcher.....	17-23
Supply Connections Dialog Box.....	17-26
Library Window.....	17-28
Locals Window.....	17-30
Modify Breakpoints Dialog Box.....	17-32
Modulated Steady State Dialog Box.....	17-34
Objects Window.....	17-36
Preferences Dialog Box.....	17-39
Processes Window.....	17-42
Questa ADMS > Questa Import Library Dialog Box.....	17-44
Questa > Questa ADMS Import Library Dialog Box.....	17-47
Restart Dialog Box.....	17-49
Reuse Previous Configuration Dialog Box.....	17-50
Rundata Window.....	17-51
Runtime Options Dialog Box.....	17-52
Source Window.....	17-55
Start Simulation Dialog Box.....	17-58

Stat Analog Kernel Window.....	17-68
Stat Mixed-Signal Activity Window.....	17-70
Stat Summary Window.....	17-72
Structure Window.....	17-74
Transcript Window.....	17-77
Wave Window (Solido Waveform Analyzer).....	17-80
Toolbars.....	17-81
Standard Toolbar.....	17-83
Compile Toolbar.....	17-85
Process Toolbar.....	17-86
Simulate Toolbar.....	17-87
Source Toolbar.....	17-89
AC Tool Toolbar.....	17-90
Predefined VHDL-AMS Packages Supplied with Questa ADMS.....	A-1
TEXTIO Packages.....	A-5
Standard TEXTIO Package.....	A-5
File Declaration.....	A-6
Using STD_INPUT and STD_OUTPUT.....	A-6
Example of How to Make Output Data Available Before Simulation Ends.....	A-7
Writing Strings and Aggregates.....	A-8
IEEE STD_LOGIC_TEXTIO Package.....	A-10
Multiple Data Types.....	A-11
ASPDETECT.....	A-14
absdelta.....	A-15
LOOK_UP_TABLE_V1X.....	A-16
VHDL-AMS Subset Definition.....	B-1
Design Entities and Configurations.....	B-2
Subprograms and Packages.....	B-7
Types and Natures.....	B-11
Declarations.....	B-14
Specifications.....	B-21
Names.....	B-24
Expressions.....	B-25
Sequential Statements.....	B-26
Concurrent Statements.....	B-30
Identifiers.....	B-36
Predefined Attributes.....	B-37
Simultaneous Statements.....	B-38
Verilog-AMS Subset Definition.....	C-1
Verilog-AMS Characteristics in Questa ADMS.....	C-1
Verilog-AMS Extensions.....	C-3
Reserved Keywords.....	C-8
Table-Based Interpolation and Lookup System Functions.....	C-8

Overview of the \$table Functions.....	C-8
\$table_model.....	C-11
\$table1f.....	C-12
\$table1a.....	C-14
\$table2f.....	C-15
\$table2a.....	C-16
\$table3f.....	C-18
\$table3a.....	C-19

C Template and Reference Tables.....D-1

General C Template in Questa ADMS.....	D-2
Correspondence Between C Type and VHDL-AMS Type.....	D-4
Predefined Macros.....	D-5
macro.h File.....	D-9
Encapsulated C Function Example.....	D-12

Tips and Techniques.....E-1

Zero-delay Loop Issues.....	E-1
Structural vs. Behavioral Descriptions.....	E-2
Using Intermediate Quantities.....	E-5
Handling of VHDL-AMS Assertions.....	E-6
Reduction of Analog Output File Size.....	E-7
Plotting Waveforms.....	E-7
.ez.do File Limitations.....	E-7
Solido Waveform Analyzer Error Handling.....	E-8
Plot of Electro Mechanical Force (EMF) Type Waveforms.....	E-9
Net Activity Profiler.....	E-10
Workarounds.....	E-10
VHDL String Representation.....	E-11
UNIX Limitation When Instantiating a SPICE Entity.....	E-11
Simulation Display Time Limitation.....	E-12
HMIN Option Convergence.....	E-12
Differences Between HDL-A and Questa ADMS Using across'Dot.....	E-13
Computing Huge Values for Quantities.....	E-13
Different Results are Generated With Two Equivalent Equations.....	E-13
Difference in Default Quantities.....	E-13
Model Writing Style Effect on Results.....	E-14
Case-sensitivity for Verilog Modules.....	E-14
Different Results for Equations Containing Functions.....	E-14
Laplace Transform Function.....	E-14
Converters Limitation.....	E-15
Specifying the Number of Warnings Displayed.....	E-15
File Declarations in TEXTIO.....	E-16
Aggregate Operand.....	E-18
ABS Operator.....	E-18
VHDL-AMS Parent Instantiating a Verilog-AMS Child.....	E-18
Verilog Parent Instantiating a VHDL-AMS Child.....	E-18

Unnamed Ports in Modules Instantiated From Verilog-AMS.....	E-19
Questa SIM in Restore Mode With Anti-Hacking Linux OS.....	E-19
FFT Analysis on Pure Digital Signals.....	E-20
Miscellaneous Workarounds.....	E-20
Macros for Simulator Version Identification.....	E-21
Improved Diagnostics for Certain Erroneous Models.....	F-1
Overview of Singularities.....	F-1
No DC Path to Ground.....	F-2
Voltage Loop.....	F-4
Inconsistent Equations.....	F-5
Multiple Solutions.....	F-6
Numerical Noise.....	F-7
Interface Association File (.assoc).....	G-1
.assoc File.....	G-2
Statistics File Reference.....	H-1
Definition Parameters.....	H-2
Elaboration Parameters.....	H-4
Simulation Parameters.....	H-6
Statistics File Example.....	H-8
Third-Party Information.....	xvii

List of Figures

Figure 3-1: Questa ADMS Main Window.....	3-2
Figure 3-2: Questa ADMS Graphical User Interface — Unzoomed.....	3-4
Figure 3-3: Graphical User Interface — Zoomed Objects Window.....	3-5
Figure 3-4: Undocked Objects Window.....	3-6
Figure 3-5: GUI: Double Bar.....	3-6
Figure 3-6: Toolbar Manipulation.....	3-7
Figure 3-7: Status Bar.....	3-9
Figure 3-8: Search Bar in Find Mode.....	3-10
Figure 3-9: Search Bar in Filter Mode.....	3-10
Figure 3-10: Example of a String Not Found.....	3-11
Figure 3-11: Filtering Objects in the Objects Window.....	3-14
Figure 3-12: Filter Objects Dialog Box.....	3-16
Figure 3-13: Configure Window Layouts Dialog.....	3-18
Figure 4-1: Assigning a Continuous Discipline to a SPICE Pin.....	4-73
Figure 4-2: Vector or Scalar Level Mapping Dialog.....	4-75
Figure 4-3: A Mixed-Signal Net Hierarchy.....	4-84
Figure 4-4: Adding an Object to the Wave Window from the Objects Window.....	4-86
Figure 4-5: Object Pathname in Transcript Window.....	4-87
Figure 5-1: Multiple Package Names in Library Window.....	5-6
Figure 7-1: SPICE-On-Top Design Structure Example.....	7-2
Figure 7-2: VHDL-AMS-On-Top Design Structure Example.....	7-3
Figure 7-3: Verilog-On-Top Structure Example.....	7-3
Figure 7-4: Viewing Source Code in the Source Window.....	7-4
Figure 7-5: Source Window with Language Templates Pane.....	7-8
Figure 7-6: Transcript Window — Command Help As You Type.....	7-9
Figure 7-7: Multiple Top Design Units (Structure Window).....	7-13
Figure 7-8: Run Popup.....	7-14
Figure 7-9: Save/Restart Simulation Results.....	7-38
Figure 7-10: Save/Restart—GUI Simulation Results.....	7-40
Figure 7-11: Save/Restart—VHDL-AMS Event-Driven Simulation Results.....	7-45
Figure 7-12: Save/Restart—Verilog(-AMS) Event-Driven Simulation Results.....	7-48
Figure 8-1: Interrogating Nets in the Contributor Window.....	8-6
Figure 8-2: Setting Current Threshold.....	8-7
Figure 8-3: Current Contributions in the CurrentFlow Window.....	8-9
Figure 8-4: Diagnostics Messages Reported in the Message Viewer.....	8-18
Figure 8-5: Stat Summary Window.....	8-24
Figure 8-6: Stat Mixed-Signal Activity Window.....	8-25
Figure 8-7: Stat Mixed-Signal Activity Window - Eldo A2D Events.....	8-25
Figure 8-8: Object Pathname.....	8-26
Figure 8-9: Go to Global Chart Button.....	8-26
Figure 8-10: Stat Analog Kernels Window.....	8-26

Figure 8-11: Selecting Code Coverage Analysis Type.....	8-33
Figure 8-12: Files and Coverage Analysis Windows.....	8-33
Figure 8-13: Coverage Statistics in the Source Window.....	8-35
Figure 8-14: Coverage Report File - Textual.....	8-37
Figure 8-15: Example Design Loaded - Capturing Coverage Data.....	8-41
Figure 8-16: Plotting Assertions.....	8-43
Figure 8-17: Coverage Report Assertion Results.....	8-44
Figure 8-18: Branch Coverage Report.....	8-45
Figure 8-19: Annotated VHDL-AMS Source in the Source Window.....	8-46
Figure 8-20: Code Coverage Analysis - "top".....	8-53
Figure 8-21: Source Window - "top".....	8-53
Figure 8-22: Code Coverage Analysis - "rdiv".....	8-54
Figure 8-23: Source Window - "rdiv".....	8-54
Figure 8-24: .EXTRACT Assertions.....	8-55
Figure 8-25: Extract Information in the .chi File.....	8-55
Figure 8-26: Multiple-Run Results in the Verification Management Browser.....	8-58
Figure 8-27: Simulation Results.....	8-63
Figure 8-28: Assertion Passed at Time 32ns.....	8-64
Figure 8-29: Assertion Passed at Time 133ns.....	8-64
Figure 8-30: Assertion Fails.....	8-65
Figure 8-31: Assertion Passes Resulting From a "Droop" of Less Than 25ns.....	8-65
Figure 8-32: Structure of Bound Instance inot_p.....	8-67
Figure 8-33: UVM Components.....	8-73
Figure 8-34: Typical UVM Flow Incorporating a DUT Wrapper.....	8-74
Figure 8-35: DUT Wrapper.....	8-75
Figure 9-1: Signal Boundary Model.....	9-2
Figure 9-2: A Mixed Signal Net Descending Through a Hierarchy.....	9-3
Figure 9-3: A Mixed Signal Net.....	9-4
Figure 9-4: Unidirectional Mixed-Signal Net with All Ports of Mode IN.....	9-5
Figure 9-5: Unidirectional Mixed-Signal Net with All Ports of Mode OUT.....	9-6
Figure 9-6: Bidirectional Mixed-Signal Net.....	9-7
Figure 9-7: Converters Example Design.....	9-13
Figure 9-8: Controlling Analog Input Value Tolerance for an A2D of Mode REAL.....	9-24
Figure 9-9: Controlling Input Net Direction Tolerance for an A2D of Mode REAL.....	9-25
Figure 9-10: Objects Window Showing Mixed-Signal Nets.....	9-36
Figure 9-11: Objects Window Showing Boundary Elements.....	9-36
Figure 9-12: Ports.....	9-37
Figure 9-13: Changing Preference Value.....	9-38
Figure 10-1: Digital Instance to be Replaced.....	10-3
Figure 10-2: SPICE Subcircuit Replacing Digital Instance.....	10-4
Figure 10-3: Mixed-Signal Testbench With 2 Power Domains.....	10-14
Figure 10-4: INV2.....	10-15
Figure 10-5: Power Domain Switching as Instructed by the Testbench.....	10-21
Figure 11-1: VRM Results Window.....	11-3
Figure 11-2: VRM Cockpit Window.....	11-3

Figure 12-1: ADC12 Example.....	12-3
Figure 12-2: Analog and Digital Results.....	12-6
Figure 12-3: Analog Subcircuit “invana”.....	12-7
Figure 12-4: Analog and Digital Results.....	12-8
Figure 12-5: Phase Locked Loop Block Diagram.....	12-13
Figure 12-6: CTRL Waveform.....	12-14
Figure 12-7: REF and VCO Digital Waveforms.....	12-14
Figure 12-8: REF and VCO Digital waveforms—Zoom 1.....	12-14
Figure 12-9: REF and VCO Digital waveforms—Zoom 2.....	12-14
Figure 12-10: Verilog-AMS Phase Locked Loop Example.....	12-16
Figure 12-11: Results Using .DEFHOOK Command.....	12-19
Figure 12-12: Results Using Connect Rules.....	12-22
Figure 12-13: SystemC Example Simulation Results.....	12-24
Figure 12-14: Inverter Chain.....	12-24
Figure 12-15: Partitioning Before Simulation.....	12-29
Figure 12-16: Partitioning After Simulation.....	12-30
Figure 12-17: Simulation Results.....	12-31
Figure 12-18: Structure and Objects Windows (VHDL) Without Propagation Delay.....	12-35
Figure 12-19: Simulation Results—tpd=0ns.....	12-35
Figure 12-20: Structure and Objects Windows (VHDL) With Propagation Delay.....	12-36
Figure 12-21: Simulation Results—tpd=3ns.....	12-36
Figure 12-22: Simulation Results—tpd=3ns (zoom).....	12-36
Figure 12-23: Structure and Objects Windows (Verilog).....	12-37
Figure 12-24: Simulation Results—tpd=0ns (Verilog).....	12-38
Figure 12-25: Simulation Results—tpd=3ns (Verilog).....	12-38
Figure 12-26: Simulation Results.....	12-41
Figure 12-27: Simulation Results for in_voltage and out_voltage.....	12-43
Figure 13-1: init_signal_spy Example.....	13-11
Figure 13-2: init_signal_spy Example with Top SPICE.....	13-15
Figure 13-3: Example of signal_force.....	13-27
Figure 13-4: init_terminal_short Example.....	13-36
Figure 13-5: init_terminal_reference Example.....	13-45
Figure 13-6: init_terminal_contribution Example.....	13-53
Figure 13-7: init_terminal_contribution Example.....	13-55
Figure 13-8: get_terminal_across_value Example.....	13-61
Figure 16-1: Possible EDA Simulator Link Simulation Configurations.....	16-2
Figure 16-2: MATLAB Testbench Configuration.....	16-3
Figure 16-3: MATLAB Testbench Configuration Setup Flow.....	16-4
Figure 16-4: MATLAB Component Configuration.....	16-5
Figure 16-5: MATLAB Component Configuration Setup Flow.....	16-6
Figure 16-6: Simulink Cosimulation Configuration.....	16-8
Figure 16-7: Simulink Cosimulation Configuration Setup Flow.....	16-8
Figure 17-1: Assertions Window.....	17-2
Figure 17-2: Contributor Window.....	17-7
Figure 17-3: CurrentFlow Window.....	17-9

Figure 17-4: Design Options Dialog Box.....	17-12
Figure 17-5: Eldo Commands Dialog Box - All Options.....	17-14
Figure 17-6: File Breakpoint Dialog Box	17-18
Figure 17-7: Force Selected Signal Dialog Box - Signal Mode.....	17-20
Figure 17-8: Force Selected Signal Dialog Box - Net Mode.....	17-21
Figure 17-9: Interface Matcher Selection Window.....	17-23
Figure 17-10: Interface Matcher Dialog Box.....	17-23
Figure 17-11: Supply Connections Dialog Box.....	17-26
Figure 17-12: Library Window.....	17-28
Figure 17-13: Multiple Packages in the Library Window.....	17-28
Figure 17-14: Locals Window.....	17-30
Figure 17-15: Modify Breakpoints Dialog Box.....	17-32
Figure 17-16: Modulated Steady State Dialog Box.....	17-34
Figure 17-17: Objects Window.....	17-36
Figure 17-18: Preferences Dialog Box - By Window Tab.....	17-39
Figure 17-19: Preferences Dialog Box - By Name Tab.....	17-40
Figure 17-20: Processes Window - Active Processes.....	17-42
Figure 17-21: Processes Window - Processes in Region.....	17-42
Figure 17-22: Questa ADMS > Questa Import Library Dialog Box Box.....	17-44
Figure 17-23: Questa ADMS Import Library Dialog Box Box.....	17-47
Figure 17-24: Restart Dialog Box.....	17-49
Figure 17-25: Reuse Previous Configuration Dialog Box.....	17-50
Figure 17-26: Rundata Window.....	17-51
Figure 17-27: Runtime Options Dialog Box—Message Severity Tab.....	17-52
Figure 17-28: Runtime Options Dialog Box—Control Variables Tab.....	17-53
Figure 17-29: Source Window.....	17-56
Figure 17-30: Start Simulation Dialog Box—Design Tab.....	17-59
Figure 17-31: Start Simulation Dialog Box—VHDL Tab.....	17-60
Figure 17-32: Start Simulation Dialog Box—Verilog Tab.....	17-61
Figure 17-33: Start Simulation Dialog Box—SDF Tab.....	17-62
Figure 17-34: Stat Analog Kernels Window.....	17-68
Figure 17-35: Stat Mixed-Signal Activity Window.....	17-70
Figure 17-36: Go to Global Chart Button.....	17-71
Figure 17-37: Stat Summary Window.....	17-72
Figure 17-38: Structure Window.....	17-74
Figure 17-39: Transcript Window.....	17-77
Figure 17-40: Solido Waveform Analyzer Wave Window.....	17-80
Figure E-1: Plot of EMF Type Waveform.....	E-9

List of Tables

Table 1-1: Conventions for Command Syntax.....	1-4
Table 2-1: Library Path Variables.....	2-3
Table 2-2: Questa SIM Simulation Control Variables.....	2-7
Table 2-3: Simulator Control Variables.....	2-9
Table 2-4: Questa ADMS Temporary Files.....	2-32
Table 2-5: Questa ADMS Standard Output Files.....	2-34
Table 3-1: Questa ADMS Windows.....	3-3
Table 3-2: Design Object Icons.....	3-8
Table 3-3: Design Unit Icons in Libraries.....	3-8
Table 3-4: Information Displayed in Status Bar.....	3-9
Table 3-5: Graphic Elements of Search Bar in Find Mode.....	3-12
Table 3-6: Graphic Elements of Search Bar in Filter Mode.....	3-13
Table 4-1: SPICE Primitives Supported in Verilog-AMS.....	4-45
Table 4-2: Verilog/VHDL Equivalent Ports.....	4-50
Table 4-3: wreal Resolution Functions.....	4-65
Table 4-4: Supported Verilog-AMS Features.....	4-70
Table 4-5: .BIND Configurations.....	4-77
Table 7-1: Design Object Icons.....	7-3
Table 7-2: Simulate > Run Menu Options.....	7-5
Table 7-3: Source Window File Icons.....	7-7
Table 7-4: Supported Parameters.....	7-51
Table 7-5: Macros For Use With TUNING Parameter.....	7-52
Table 8-1: Coverage Windows.....	8-31
Table 8-2: Code Coverage Icons.....	8-34
Table 8-3: Coverage Icons in the Source Window.....	8-35
Table 8-4: Mixed-Signal Simulation Predefined Fields in UCDB Test Attribute Record.....	8-38
Table 8-5: SystemVerilog System Functions for Accessing Analog Objects.....	8-69
Table 9-1: Directional Qualifiers.....	9-17
Table 9-2: Effective Real Values in Compatible Mode.....	9-30
Table 9-3: Effective Logic Values in Compatible Mode.....	9-31
Table 9-4: Effective Real Values in Strict Mode.....	9-32
Table 9-5: Effective Logic Values in Strict Mode.....	9-33
Table 9-6: Effective Real Values in Relaxed Mode.....	9-33
Table 9-7: Effective Logic Values in Relaxed Mode.....	9-34
Table 9-8: Logic to Real Conversion for D2D Boundary Elements of Type MODE=REAL.....	9-35
Table 9-9: Real to Logic Conversion for D2D Boundary Elements of Type MODE=REAL.....	9-35
Table 9-10: Character Replacement in Names Containing Extended or Escape Identifiers.....	9-41
Table 10-1: P2E Connect Element Input Value.....	10-9
Table 10-2: E2P Connect Element Primary Power Value.....	10-10
Table 10-3: Built-in Power Connect Elements in the mgc_ams Library.....	10-11
Table 12-1: 12-Bit Analog to Digital Converter Example Files.....	12-4

Table 12-2: Ring Oscillator Example Files.....	12-7
Table 12-3: C Code Encapsulation Examples.....	12-9
Table 12-4: SPICE Inverter Example Files.....	12-11
Table 12-5: VHDL-AMS Phase Locked Loop Example Files.....	12-13
Table 12-6: Verilog-AMS Integrator Example Files.....	12-15
Table 12-7: Verilog-AMS Phase Locked Loop Example Files.....	12-16
Table 12-8: SystemC Flip-Flop Latch Example Files.....	12-23
Table 12-9: Inverter Chain Example Files.....	12-25
Table 12-10: Inverter Chain With SPICE On Top Example Files.....	12-26
Table 12-11: SPICE and Verilog Inverter Chain Example Source Files.....	12-32
Table 13-1: VHDL-AMS Procedure and Equivalent Verilog-AMS System Task.....	13-2
Table 13-2: Supported Data Types.....	13-3
Table 16-1: MATLAB Testbench Configuration with Questa SIM.....	16-4
Table 16-2: MATLAB Testbench Configuration with Questa ADMS.....	16-5
Table 16-3: MATLAB Component Configuration with Questa SIM.....	16-6
Table 16-4: MATLAB Component Configuration with Questa ADMS.....	16-7
Table 16-5: Simulink Cosimulation Configuration with Questa SIM.....	16-9
Table 16-6: Simulink Cosimulation Configuration with Questa ADMS.....	16-9
Table 17-1: Assertions Window Contents.....	17-2
Table 17-2: Contributor Window Contents.....	17-7
Table 17-3: CurrentFlow Window Actions.....	17-9
Table 17-4: Design Options Dialog Box Contents.....	17-12
Table 17-5: Eldo Commands Dialog Box Contents.....	17-15
Table 17-6: File Breakpoint Dialog Box Contents.....	17-18
Table 17-7: Force Selected Signal Dialog Box Contents.....	17-21
Table 17-8: Interface Matcher Dialog Box.....	17-24
Table 17-9: Supply Connections Dialog Box.....	17-26
Table 17-10: Library Window Contents.....	17-28
Table 17-11: Locals Window Contents.....	17-30
Table 17-12: Modify Breakpoints Dialog Box Contents.....	17-32
Table 17-13: Modulated Steady State Dialog Box Contents.....	17-34
Table 17-14: Objects Window Contents.....	17-36
Table 17-15: Preferences Dialog Box - By Window Tab Contents.....	17-40
Table 17-16: Preferences Dialog Box - By Name Tab Contents.....	17-40
Table 17-17: Processes Window Contents.....	17-43
Table 17-18: Questa ADMS > Questa Import Library Dialog Box Contents.....	17-44
Table 17-19: Questa > Questa ADMS Import Library Dialog Box Contents.....	17-47
Table 17-20: Restart Dialog Box Contents.....	17-49
Table 17-21: Reuse Previous Configuration Dialog Box Contents.....	17-50
Table 17-22: Rundata Window Contents.....	17-51
Table 17-23: Runtime Options Dialog Box Fields.....	17-53
Table 17-24: Design Tab Contents.....	17-62
Table 17-25: VHDL Tab Contents.....	17-63
Table 17-26: Verilog Tab Contents.....	17-64
Table 17-27: Libraries Tab Contents.....	17-65

Table 17-28: Start Simulation Dialog Box - SDF Tab.....	17-65
Table 17-29: Stat Analog Kernel Window Contents.....	17-68
Table 17-30: Stat Mixed-Signal Activity Window Contents.....	17-70
Table 17-31: Stat Summary Window Contents.....	17-73
Table 17-32: Structure Window Contents.....	17-74
Table 17-33: Transcript Window Menu Contents.....	17-78
Table 17-34: Standard Toolbar Buttons.....	17-83
Table 17-35: Compile Toolbar Buttons.....	17-85
Table 17-36: Process Toolbar Buttons.....	17-86
Table 17-37: Simulate Toolbar Buttons.....	17-87
Table 17-38: Source Toolbar Buttons.....	17-89
Table 17-39: AC Tool Toolbar Buttons.....	17-90
Table A-1: Predefined VHDL-AMS Packages.....	A-1
Table A-2: Data and Data Type.....	A-11
Table D-1: Case of scalar object.....	D-4
Table D-2: Case of 1-dimensional array object.....	D-4
Table D-3: Case of CONSTANT SCALAR Parameters of Mode IN.....	D-5
Table D-4: Case of VARIABLE SCALAR Parameters of Mode IN.....	D-5
Table D-5: Case of SIGNAL SCALAR Parameters of Mode IN.....	D-5
Table D-6: Case of VARIABLE SCALAR Parameters of Mode OUT/INOUT.....	D-6
Table D-7: Case of SIGNAL SCALAR Parameters of Mode OUT/INOUT.....	D-6
Table D-8: Case of CONSTANT 1-DIMENSION ARRAY Parameters of Mode IN.....	D-6
Table D-9: Case of VARIABLE 1-DIMENSION ARRAY Parameters of Mode IN.....	D-7
Table D-10: Case of SIGNAL 1-DIMENSION ARRAY Parameters of Mode IN.....	D-7
Table D-11: Case of VARIABLE 1-DIMENSION ARRAY Parameters of Mode OUT/INOUT.....	D-7
Table D-12: Case of SIGNAL 1-DIMENSION ARRAY Parameters of Mode OUT/INOUT.....	D-8
Table E-1: Memory Usage With Vector Optimization.....	E-3
Table E-2: Memory Usage Without Vector Optimization.....	E-4

1. Introduction

This chapter provides a basic overview of Questa ADMS, and describes concepts relating to the tool and this manual.

Questa ADMS Overview.....	1-1
How To Use Questa ADMS User Documentation.....	1-3
AMS Languages Support.....	1-6
Standards Supported.....	1-6

Questa ADMS Overview

The Questa ADMS simulation system extends the familiar Questa SIM verification platform with analog and mixed-signal standard languages. The Questa SIM, Eldo[®] (and Eldo RF), and ADiT[™] simulation engines are incorporated to provide a single-kernel architecture that enables you to efficiently simulate mixed-language designs within one consistent environment.

Questa ADMS is language neutral; multiple languages may be combined in one simulation to describe the design. You can combine VHDL-AMS, Verilog-AMS, VHDL, Verilog, SystemVerilog, SPICE and SystemC anywhere and at any level in the design. The testbench can be SPICE, an analog or mixed-signal language, or a digital language.

Digital blocks compiled and simulated by Questa SIM can be used in Questa ADMS without any modification. SPICE subcircuits can be used anywhere in the design hierarchy.

Questa SIM Integration

Note

From release 2026.1, the Questa SIM version integrated in Questa ADMS is frozen at 2025.3.

The Questa SIM software is integrated within Questa ADMS in a way that enables you to use:

- Dynamically linked debugging and design visualization extensions, to help to pinpoint problems in mixed-signal designs.
- The Questa SIM power-aware flow and digital optimizer.
- The SystemVerilog OVM for complex system verification.
- The integrated TCL scripting language, enabling batch control of the simulation and waveform display.

Questa ADMS uses gcc 4.5.1, but will use gcc 4.5.0 (as Questa SIM does) when compiling SystemC.

Refer to the following manuals for further information on Questa SIM functionality:

- *Questa SIM User's Manual*
- *Questa SIM GUI Reference Manual*
- *Questa SIM Command Reference Manual*
- *Questa SIM Verification Management User's Manual*
- *Power Aware Simulation User's Manual*

Eldo Integration

Questa ADMS accesses the Eldo simulator in order to simulate the following within a mixed-signal, or pure analog, simulation:

- A design that includes SPICE subcircuits as components.
- A SPICE netlist that includes VHDL-AMS and VHDL design entities or Verilog-AMS and Verilog modules as components.
- A pure SPICE netlist.

See the [Eldo User's Manual](#) and [Eldo Reference Manual](#) for detailed information on Eldo.

Questa ADMS supports the Eldo Premier simulator, which provides an increase in performance and capacity without sacrificing accuracy compared to Eldo classic, allowing much larger circuits to be simulated.

For more information, see [Eldo Premier](#) in *Eldo User's Manual*.

ADiT Integration

Questa ADMS supports ADiT (Analog Digital Turbo Simulator) for Fast-SPICE simulation. ADiT can simulate large circuits with simulation speed 10x~ 100x faster than that of SPICE.

See the [ADiT User's Manual](#) for detailed information on ADiT.

Questa ADMS RF

Many digital communication systems include tightly integrated RF, analog mixed-signal and DSP functions. RF carriers severely slow classical mixed-signal transient simulation. Questa ADMS RF specifically targets this challenge by combining the capability of Questa ADMS with the modulated steady-state (.MODSST) analysis of Eldo RF, see [Eldo RF and Questa ADMS](#) in the *Eldo RF User's Manual*.

Note

A Questa ADMS RF license is required to use the RF algorithms in Questa ADMS.

For an introduction to Questa ADMS RF, using an example for which the tool brings real benefits in terms of performance and usability, refer to the tutorials [AGC Loop Tutorial for Questa ADMS RF](#) and [ZigBee Chain Tutorial for Questa ADMS RF](#) in the *Eldo RF Tutorials Manual*.

Integration with MathWorks

Questa ADMS supports MathWorks Simulink models in a Questa ADMS hierarchy. Integration between Questa ADMS/Questa SIM and MATLAB®/Simulink® is possible using HDL Verifier™.

MATLAB, Simulink, and Real-Time Workshop are registered trademarks of *The MathWorks, Inc.*

See the [MathWorks Integration](#) chapter for more information.

Related Topics

[AMS Languages Support](#)

[How To Use Questa ADMS User Documentation](#)

[Questa ADMS GUI Overview](#)

How To Use Questa ADMS User Documentation

Questa ADMS documentation provides instructions, examples, modeling resources, descriptions of dialog boxes, and descriptions of commands. Questa ADMS documentation uses command syntax conventions.

Note

For summary information, see also [Questa ADMS Quick Reference.pdf](#) and [VHDL-AMS Quick Reference.pdf](#).

To become familiar with Questa ADMS, work through the basic tutorials in [Getting Started with Questa ADMS](#).

Take a tour of the Questa ADMS GUI and learn how to set display preferences in the [Questa ADMS GUI Overview](#) chapter.

The following chapters in this manual describe the basic usage flow for simulation using Questa ADMS:

1. [Design Preparation](#) — Prepare your digital and analog source files and netlists.
2. [Design Libraries](#) — Create a design library in which to store simulation results.
3. [Compilation](#) — Compile source files into the design library.
4. [Simulation](#) — Run simulations and analyze the results.
5. [Verification](#) — Debug your design interactively, save and restart simulations, and perform verification.

For detailed descriptions of all GUI dialog boxes, refer to the [Dialog and Field Reference](#) chapter. Full and detailed syntax descriptions of all of the commands available in Questa ADMS are provided in the separate manual: [Questa ADMS Command Reference](#).

You access the documentation system using any of the following methods:

- **solidodocs** shell command from a Linux shell window.
- **Start > Solido Sim <release> > Documentation** in Windows.
- **Help > Help and Manuals** menu within the GUI products.

Command Syntax Conventions

Table 1-1 describes the conventions used in this manual for Questa ADMS command syntax:

Table 1-1: Conventions for Command Syntax

Syntax Notation	Description
< >	Angled brackets surrounding a syntax item indicate a user-defined argument; do not enter the brackets in commands.
[]	Square brackets generally indicate an optional item; if the brackets surround several words, all must be entered as a group; the brackets are not entered.
{ }	Braces indicate that the enclosed expression contains one or more spaces yet should be treated as a single argument, or that the expression contains square brackets for an index; in both cases, the braces are entered.
...	An ellipsis indicates items that may appear more than once; the ellipsis itself does not appear in commands.
	The vertical bar (or pipe) indicates a choice between items on either side of it; do not include the bar in the command.
#	Comments included with commands are preceded by the number sign (#); useful for adding comments to DO files (macros).
monospaced type	Monospaced type is used in command examples.
code in red	Code shown in red means that the syntax is part of the Language Reference Manual but it is not supported in Questa ADMS.
code in amber	Code shown in amber means that the syntax is partially supported in Questa ADMS, but that there is some deviation from the Language Reference Manual.

Modeling Resources

The following resources are available for mixed-signal behavioral modeling:

- Modeling Cookbooks for VHDL-AMS, Verilog-AMS and Verilog-A

The AMS Modeling Cookbooks are a ready source of modeling tips and techniques for end-users who must develop their own customized, efficient and accurate mixed-signal behavioral models.

The models used as examples cover a variety of communications and multimedia applications. They can be used out-of-the-box for system-level design, architectural exploration, system-level functional verification, and for enhancing the simulation speed of complex mixed-signal systems.

Refer to the appropriate documentation for the language of interest:

- [AMS Modeling Cookbook for VHDL-AMS.pdf](#) and [AMS Modeling Cookbook Source for VHDL-AMS.pdf](#)
- [AMS Modeling Cookbook for Verilog-AMS.pdf](#) and [AMS Modeling Cookbook Source for Verilog-AMS.pdf](#)
- [AMS Modeling Cookbook for Verilog-A.pdf](#) and [AMS Modeling Cookbook Source for Verilog-A.pdf](#)
- Modeling Examples for VHDL-AMS, Verilog-AMS, and Verilog-A; with access to hundreds of parameterized analog/mixed-signal models

The Modeling Examples are a set of behavioral models, with graded levels of complexity, for communications and multimedia applications. The models are organized into categories or sub-libraries. The categories covered by the modeling examples are: A/D, D/A, Amplifiers/Comparators, PLL, Delta-Sigma, Filters, Control, Functions, DC-DC, Modulators/Demodulators, Digital, Sources, SerDes, and RF.

Refer to the appropriate documentation for the language of interest:

- [Modeling Examples for VHDL-AMS.pdf](#)
- [Modeling Examples for Verilog-AMS.pdf](#)
- [Modeling Examples for Verilog-A.pdf](#)

The AMS Modeling Cookbooks and Modeling Examples packages are provided as source code. To use these packages you must compile the source files provided and set the path to the Cookbooks and Modeling examples libraries using the variable name AMSLib.

Related Topics

[System Initialization File \(modelsim.ini\)](#)

[Design Libraries](#)

AMS Languages Support

VHDL-AMS represents complex models directly, combining differential equations, algebraic constraints and logical controls. You may use conventional macro modeling alone or in combination with direct modeling. The Verilog-AMS language enables you to create modules that encapsulate high-level behavioral descriptions as well as structural descriptions of systems and components.

Descriptions in VHDL-AMS mixing analog and event-driven elements are many times more efficient than SPICE-style macro models. Additionally, with Questa ADMS, you can include SPICE primitives and subcircuits anywhere in the hierarchy; VHDL/Verilog may also be included.

VHDL-AMS

VHDL-AMS models can also describe non-electrical systems. High-level designs often include non-electrical components, and even when they don't, adequate analysis and testing may demand a complex simulated environment. For example, an electronic controller with both digital logic and analog control elements may need to be modeled as part of an automobile braking system including mechanical and fluidic elements. System simulation yields another bonus: the designer may be able to trade off a small change in the electronic portion of the control system against a large and cost-saving change in the fluidics.

Verilog-AMS

Verilog-AMS is a mixed-signal-language solution allowing the designer to create and use modules that encapsulate high-level behavioral descriptions as well as structural descriptions of systems and components. Verilog-AMS provides the capability to integrate system and circuit-level aspects of the design, allowing the designer of analog and mixed-signal systems and integrated circuits more productivity throughout the entire mixed-signal design process.

Verilog-A, which is a subset of Verilog-AMS, is a language for analog circuits and systems; it provides a dimension of design and simulation capability for analog electronic design. The language also enables descriptions directly using mathematical relationships, thus easily allowing system descriptions other than electrical.

Related Topics

Standards Supported

Standards Supported

Questa ADMS supports Verilog, VHDL, and System C standards.

Verilog Standards

Anything that the Verilog part of Questa SIM accepts is also accepted in Questa ADMS, except for the boundary between Verilog (Questa SIM) and VHDL-AMS. In the current version, the generic mapping of

an instantiated module is ignored. Questa SIM Verilog is based on the IEEE standards 1364-1995 and 1364-2005. Questa SIM Verilog also supports a partial implementation of SystemVerilog P1800-2012, and this is supported by Questa ADMS only when instantiated by Questa SIM. VCD (Value Change Dump) files are supported; see [Value Change Dump \(VCD\) Files](#).

Questa ADMS supports a subset of Accellera¹ Verilog-AMS 2.4; see [Verilog-AMS Subset Definition](#) for more information.

VHDL Standards

Apart from the features described in the [VHDL-AMS Subset Definition](#) appendix, Questa ADMS supports the VHDL language as defined by IEEE standards 1076-1987, 1076-1993, 1076-2002, 1076-2008 and the VHDL-AMS language as defined by the IEEE standard 1076.1-1999. A design developed with Questa ADMS can be used on any system that supports these standards.

Tip

Further IEEE documentation, including the *1076.1-1999 IEEE Standard VHDL Analog and Mixed-Signal Extensions Language Reference Manual*, is available from the IEEE website (<http://standards.ieee.org>).

The default VHDL language version for Questa SIM is VHDL-2002.

Additionally, all of VHDL 1076-2008 and Verilog 1364-2005 that is accepted by Questa SIM is also accepted in Questa ADMS. This means that the following are also supported:

- SDF 1.0 through 4.0
- VITAL 2.2b
- VITAL'95 - IEEE 1076.4-1995
- VITAL 2000 - IEEE 1076.4-2000

Descriptions can be mixed at any level between digital (Verilog and VHDL) and analog/mixed-signal (VHDL-AMS, Verilog-AMS and SPICE).

See [VHDL Input Formats](#).

System C

Questa ADMS supports the SystemC language based on the Open SystemC Initiative (OSCI) SystemC proof-of-concept SystemC simulator through Questa SIM.

SystemC can only be used in Questa ADMS if:

- It is encapsulated inside a Questa SIM VHDL/Verilog wrapper.

¹ Accellera Organization Inc.

- It does not instantiate directly, or indirectly in its sub-hierarchy, any non-Questas SIM instance.

It is recommended that you obtain the OSCI functional specification, or the latest version of the *IEEE SystemC Language Reference Manual* (IEEE Std 1666-2011). These can be obtained from the IEEE website (<http://standards.ieee.org>).

Related Topics

Design Preparation

2. Environment Settings

This chapter describes the variables and preferences used to control the Questa ADMS environment.

Note

Ensure the *SYMPHONY_HOME* environment variable is not defined when using Questa ADMS.

System Initialization File (modelsim.ini).....	2-1
Eldo Initialization File (eldo.ini).....	2-21
Questa ADMS GUI Preferences.....	2-22
Optional Environment Variables.....	2-27
Files Used by Questa ADMS.....	2-32
Location Maps.....	2-37

System Initialization File (modelsim.ini)

Questa ADMS uses the same initialization file as Questa SIM (*modelsim.ini*) to store library mappings and variable definitions.

Note

From release 2025.3, MODELSIM is deprecated, specify QSIM_INI.

This file is automatically created when libraries are mapped. The contents of a *modelsim.ini* file can be read and edited using a text editor. Comments within the file are preceded with a semicolon (;).

The *modelsim.ini* file is located in the directory specified by the **QSIM_INI** environment variable. This can be considered a global file rather than a local file. If no QSIM_INI environment variable exists, or no *modelsim.ini* file exists in that location, Questa ADMS will look for it in the following locations:

- *\$(QSIM_WD)/modelsim.ini*

Note

From release 2025.3, MGC_WD is deprecated, and a warning will be generated. Instead, specify QSIM_WD, or the environment variable DISABLE_QUESTA_DEPRECATED_WARNING=1 to suppress the warning.

- *./modelsim.ini*
- *\$(AMS_MODEL_TECH)/modelsim.ini*
- *\$(AMS_MODEL_TECH)/../modelsim.ini*
- *\$(MGC_HOME)/lib/modelsim.ini*

The path to `./modelsim.ini` is set even though the file doesn't exist.

Note
`$AMS_MODEL_Tech` is a run-time variable and is equivalent to `$MGC_AMS_HOME/libs`. The value of this variable will override the path.

Library Path Variables.....2-3

Setting Up Libraries for Group Use.....2-4

SPICE Subcircuit Generation Variables.....2-6

Questa SIM Simulation Control Variables.....2-7

Simulator Control Variables.....2-9

Setting VectorPatternString.....2-17

VHDL Compiler Control Variables.....2-18

VHDL-AMS Compiler Control Variables.....2-20

Verilog-AMS Compiler Control Variables.....2-21

Library Path Variables

Library path variables are listed under the [Library] heading in the *modelsim.ini* file.

To set the library path variables, edit the initialization file (*modelsim.ini*) directly with any text editor using the syntax:

```
<variable> = <value>
```

Table 2-1: Library Path Variables

Variable Name	Description	Default Location
Questa SIM Libraries Remapped by Questa ADMS		
STD	Sets the path to the VHDL STD library	<i>\$AMS_MODEL_TECH/std</i>
IEEE	Sets the path to the library containing IEEE and Synopsys arithmetic packages	<i>\$AMS_MODEL_TECH/ieee</i>
VITAL2000	Sets path to the library containing Vital2000 packages	<i>\$AMS_MODEL_TECH/vital2000</i>
SYNOPSIS	Sets the path to the library containing Synopsys packages.	<i>\$AMS_MODEL_TECH/synopsys</i>
MODELSIM_LIB	Sets the path to the library containing the Util package used by Net Spy	<i>\$AMS_MODEL_TECH/modelsim_lib</i>
Questa ADMS Libraries		
VITAL2.2B	Sets the path to the VITAL2.2b library	<i>\$AMS_MODEL_TECH/vital2.2b</i>
MGC_AMS	Sets the path to internal declaration packages	<i>\$AMS_MODEL_TECH/MGC_AMS</i>
IEEE_ENV	Sets the path to the IEEE_ENV library containing standard VHDL packages for multiple energy domain support (IEEE 1076.1.1-2004, MATERIAL_CONSTANTS)	<i>\$AMS_MODEL_TECH/IEEE_ENV</i>
ARITHMETIC	Sets the path to the library containing arithmetic packages	<i>\$AMS_MODEL_TECH/ARITHMETIC</i>

Table 2-1: Library Path Variables (continued)

Variable Name	Description	Default Location
VERILOG	Sets the path to the Verilog library	<code>\$AMS_MODEL_TECH/verilog</code>
AMS_LIB	Sets the path to AMS modeling cookbooks and modeling examples libraries	

`$AMS_MODEL_TECH` is a run-time variable and is equivalent to `$MGC_AMS_HOME/libs`. The value of this variable will override the path.

Setting Up Libraries for Group Use

You can use the *modelsim.ini* file to set up a hierarchy of library mappings.

Procedure

Add an "others" clause in the [Library] section of the *modelsim.ini* file:

```
others = <other_modelsim_ini>
```

Results

You can specify only one "others" clause in the library section of a given *modelsim.ini* file. However, the file referred to by the others clause may itself contain an others clause, so you can use this feature to chain a set of hierarchical *.ini* files.

The "others" clause only instructs the tool to look in the specified *modelsim.ini* file for a library. It does not load any other part of the specified file.

If there are two libraries with the same name mapped to two different locations - one in the current *modelsim.ini* file and the other specified by the "others" clause - the mapping specified in the current *modelsim.ini* file will take effect.

See also:

- [Using Design Libraries](#)
- [Modeling Resources](#)
- [Optional Environment Variables](#)

Examples

```
[Library]
asic_lib = /cae/asic_lib
work = my_work
others = $MGC_AMS_HOME/libs/modelsim.ini
```

SPICE Subcircuit Generation Variables

The [vaspi] section of the *modelsim.ini* file contains two variables that control default vaspi behavior; UseNewFlow and UseCktName.

- UseNewFlow

Sets the default vaspi behavior.

- 0

Equivalent to the -arch option. The default.

- 1

Equivalent to the -noarch option

- UseCktName

This variable only applies when the UseNewFlow variable is set to 1, or if the vaspi -noarch command is being run.

- 0

Equivalent to the -noarch -digname option. The default.

- 1

Equivalent to the -noarch -cktname option.

See also [vaspi](#) in *Questa ADMS Command Reference*.

Questa SIM Simulation Control Variables

Variables are required for compatibility between Questa ADMS and Questa SIM. These variables can be found under the heading [vsim] in the *modelsim.ini* file.

Table 2-2: Questa SIM Simulation Control Variables

Variable Name	Description	Default Value
DatasetSeparator ¹	Specifies the dataset separator for fully rooted contexts. This must not be the same character as defined by the PathSeparator variable.	/ in Questa ADMS. In Questa SIM, the default is :
PathSeparator ¹	Specifies the character used to separate hierarchical object names. This must not be the same character defined by the DatasetSeparator variable. This is also the default for the SignalSpyPathSeparator variable. Normally "/" is used for VHDL and "." for Verilog.	: in Questa ADMS. In Questa SIM, the default is /
SignalSpyPathSeparator	Specifies the character used to separate hierarchical object names in arguments to the Net Spy procedures. This must not be the same character as defined by the DatasetSeparator variable.	PathSeparator
Resolution ²	Simulator resolution. This can be set to fs, ps, ns, us, ms, or sec with an optional prefix of 1, 10, or 100.	ps in Questa ADMS. The Questa SIM default is ns.
VoptFlow ¹	Specifies that Questa SIM performs built-in optimizations on your design to maximize simulator performance. These optimizations yield performance improvements over non-optimized runs. Value Range: 0, 1	1 (optimized mode)
CreateDirForFileAccess ³	Controls whether the Verilog system task \$fopen or vpi_mcd_open() will create a non-existent directory when opening a file in append (a), or write (w) modes. Value Range: 0, 1.	0 (non-existent directories are not created)

¹ This variable is changed automatically when using the vasetinimode command.

² A different resolution can be specified by invoking Questa ADMS with the argument -t, for example: `vasim ... -t [<multiplier>]<time_unit>`.

³ Digital parts of Verilog-AMS or VHDL-AMS descriptions are supported, but pure VHDL descriptions compiled using the vcom command are not supported.

Example

The following example shows the modified settings of the [vsim] section of the *modelsim.ini* file for compatibility with Questa ADMS:

```
[vsim]
; vopt flow
; Set to turn on automatic optimization of a design.
; Default is on
VoptFlow = 1
.
.
; Simulator resolution
; Set to fs, ps, ns, us, ms, or sec with optional prefix of 1, 10, or 100.
Resolution = ps
.
.
; Specify whether paths in simulator commands should be described
; in VHDL or Verilog format.
; For VHDL, PathSeparator = /
; For Verilog, PathSeparator = .
; Must not be the same character as DatasetSeparator.
PathSeparator = :

; Specify the dataset separator for fully rooted contexts.
; The default is ':'. For example: sim:/top
; Must not be the same character as PathSeparator.
DatasetSeparator = /
```

Related Topics

[Simulator Command Time Units](#)

[Simulator Resolution Limit \(Verilog\)](#)

Simulator Control Variables

Simulator control variables define simulation options, such as the use of extended identifiers and the simulation output format. These variables are under the [vasim] heading in the *modelsim.ini* file.

To set these variables, edit the initialization file, *modelsim.ini*, directly with any text editor using the syntax:

```
<variable> = <value>
```

Comments within the file are preceded with a semicolon (;).

Table 2-3: Simulator Control Variables

Variable Name	Description	Default Value
Update Simulation Time Variables		
UpdateSimulationTimeDelayGraph	Specifies the rate (ms) at which the simulation time is updated during a simulation in graphical mode	1000 (every second of elapsed time)
UpdateSimulationTimeDelayBatch ¹	Specifies the rate (ms) at which the simulation time is updated during a simulation. Set to 0 to disable this mechanism.	10000 (every ten seconds of elapsed time)
Message Variables		
MaxNbOfDisplayedMsg	Specifies the number of times a warning message is displayed during a simulation. See Limiting Message Display .	5
VerboseMode ²	Controls the number of notes and messages that are written to the Transcript Window . When enabled, all messages are displayed. When disabled only important messages are displayed. Value Range: 0, 1	0 (disabled)
Diagmode	Analog diagnosis detects and reports possible design errors and constructs that could slow down simulation. It can be enabled to	ELAB

¹ The progress of simulation time in batch mode, as controlled by UpdateSimulationTimeDelayBatch, is only displayed in the Transcript Window when VerboseMode = 1.

² Warnings and error messages are always displayed and all notes and messages are written to the .errm.log file in the output directory whatever value this variable is set to.

Table 2-3: Simulator Control Variables (continued)

Variable Name	Description	Default Value
	generate the data during Eldo parsing, elaboration or simulation. When diagmode is set to elab, only diagnosis data from Eldo parsing and elaboration are collected. Value Range: ELAB, NORUN, RUN, NO. See Diagnosis Mode .	
VerboseAnalog	When enabled, all convergence information coming from the analog solvers are output (this is the same behavior as Eldo standalone. When disabled, only a subset of convergence messages are output.	0 (disabled)
Incremental Saving Variables		
JwdbIncrementalSaving	Enables incremental saving. If disabled, all incremental saving variables are ignored, however, the command isavewdb and the vasim -isaving argument can be used to override this setting. Value Range: 0, 1	1 (enabled)
JwdbIncrementalSavingOnMemory	When enabled, the waveform data stored in the JWDB is saved when the size of the JWDB reaches the value specified by the JwdbSpillThreshold variable. Value Range: 0, 1	1 (saved)
JwdbSpillThreshold ³	Defines the size in bytes of the JWDB that triggers a save operation	104800000 (100MB)
JwdbIncrementalSavingFlushDelay	Questa ADMS flushes all data in the JWDB server at a time period defined by this variable, in seconds. This does not mean that Questa ADMS saves the waveform data to the disk every defined number of seconds. The waveform data will be available in the JWDB server and therefore waveforms in	10

³ You should take into account that the data will be compressed when saved to disk; in other words, the size of the saved data should be smaller than the size specified on this variable.

Table 2-3: Simulator Control Variables (continued)

Variable Name	Description	Default Value
	the Solido Waveform Analyzer wave viewer can be updated.	
JwdbIncrementalSavingOnBreak	When set to yes, the data in the JWDB is saved each time you specify a break, either through the GUI or in batch mode. Note that in this sense, executing a do file is not considered as performing a break. Value Range: yes, no	no
SaveJwdbAtBreak	When set to 1, the data in the JWDB is saved each time you specify a break, either through the GUI or in batch mode. Note that in this sense, executing a do file is not considered as performing a break. Value Range: 0, 1	0 (off)
Previous Session Results Management Variables		
AutoSaveInPreviousSession	When set to 1, the simulation results already present in the existing JWDB file for this design are saved in the PreviousSession folder of JWDB before storing the waves of the new simulation.	0 (off)
AlwaysAutoSaveInPreviousSessionAtRestart	When set to 1, the simulation results already present in the existing JWDB file for this design are saved in the PreviousSession folder of JWDB before storing the waves of the new simulation requested by the Tcl restart command, by the Restart button in the Simulate Toolbar , or by selecting File > Restart from the main window menu. Value Range: 0, 1	0 (off). In this case, the behavior is controlled by the AutoSaveInPreviousSession variable.
Display at TStop Variables		
DisplayOutputSummaryStatsAtEachTStop	If set to 1, then each time the simulation time becomes a multiple of TStop, the output summarystats command is run. Value Range: 0, 1	0 (off)

Table 2-3: Simulator Control Variables (continued)

Variable Name	Description	Default Value
DisplayOutputPostprocessingAtTStop	If set to 1, then when the simulation reaches TStop, the output postprocessing command is run and the simulation finishes. Note that in order to run the Verilog-AMS final_step block, this variable must be set to 1. Value Range: 0, 1	0 (off)
Statistics File Variables		
BoundaryStatListNb	The number of the most active boundary elements to be listed in the Simulation section of the statistics file. See Simulation Statistics .	10
StatisticsFileMaximumSize	Specifies the maximum size of a single statistics file (refer to Simulation Statistics). If this size is reached, a new statistics file is created and a message appears in the transcript. The names of these files are of the format: <code><stat_file_name>.<n></code> where <code><n></code> = 1, 2, 3, and so on.	50000000 (50 MB)
Miscellaneous Variables		
IterationLimitDeltaDebug	Questa ADMS automatically adds a watch to suspicious nets when the count of delta cycles approaches within this value of the maximum permitted number of delta cycles at a given simulation time (see vasim -iteration_limit).	10
ExtendedId	Enables or disables the use of extended identifiers. Value Range: 0, 1	1 (enable)
OutputResults	Specifies the output format of the simulation. Value Range: jwdb, cou	jwdb

Table 2-3: Simulator Control Variables (continued)

Variable Name	Description	Default Value
UniqueHierarchy	Use unique hierarchy management when creating curves in the database. Value Range: 0, 1	1 (on)
VectorPatternString	Used when an HDL instance is being replaced by an equivalent SPICE subcircuit or behavioral model (mapping by name). The .BIND , vaspi and vamatch commands allow mapping by name. See Setting VectorPatternString .	@name@_@idx@ @name@[@idx@] @name@<@idx@> @name@\\<@idx@\\>
ShowVasimShellCommand ⁴	Displays the vasim commands and options that were used to launch the simulator. Enables a shell vasim command line in the Transcript Window . Value Range: 0, 1.	0 (disabled)
AutoNoExec	Enables execution of commands available from a UNIX or Windows shell as if they were Tcl commands. Value Range: 0, 1	1 (enable)
GuiForceUniqueWaveform	Forces Solido Waveform Analyzer to check the uniqueness of each added waveform. Value Range: 0, 1	1 (enable)
DockEZwaveInQuestaAdms	Docks the Wave window (Solido Waveform Analyzer) in the Questa ADMS main application window. Value Range: 0, 1	0 (undocked)
ListWindowUpdate	Updates the List Window either at the end of (0), or during (1) a simulation.	0 (at the end)
MaxInstantiationDepth	Sets the maximum number of hierarchical levels accepted before recursivity instantiations detection.	70
CheckParameterExistence	Used to ignore or generate errors when instance parameters that have not been defined in corresponding module specifications, and are not known	0 (ignore)

⁴ This can be useful in cases where Questa ADMS is started by another software layer and you want to see all options that were set at the time.

Table 2-3: Simulator Control Variables (continued)

Variable Name	Description	Default Value
	by Questa ADMS, are found. These parameters are automatically generated by some netlisters. As Questa ADMS is not able to map them to any parameter of the module, they are qualified as "meaningless". Value Range: 0, 1	
StopAndSaveSimulationInCaseOfMemoryShortage	Controls how to save the current simulation results if there is a shortage of memory or disk space during simulation. Value Range: - enable Results are saved using the checkpoint mechanism - disable Results will not be saved using the checkpoint mechanism - warnings_only Results are saved using the checkpoint mechanism only when the memory checker returns a warning - errors_only Results are saved using the checkpoint mechanism only when the memory checker returns an error	enable
CheckpointWithWDBInCaseOfMemoryShortage	When enabled, waveform data will be recorded to a JWDB file when an automatic checkpoint is saved in the event of a memory shortage. When restarting the simulation, the waves will start at the time of the restore and not at time 0.	0
CheckpointWithWDB	When enabled, waveform data will be recorded to a JWDB file when a checkpoint is requested	1

Table 2-3: Simulator Control Variables (continued)

Variable Name	Description	Default Value
	in the design, or when using the checkpoint and .SAVE commands. When restarting the simulation, the waves will start at the time of the restore and not at time 0.	
WaveBoundariesAs	Defines how boundary elements are displayed within Solido Waveform Analyzer. This variable is set by the vasetinimode command. - analog V(...) - digital If more than one digital element exists⁵, they are all grouped together under a bus that is named as the wave. - Both Analog and digital representations can be displayed at the same time.	analog
GenerateFormat	Controls the format of the old-style VHDL for ... generate statement region name for each iteration if the OldVhdlForGenNames variable is set to 1. Two values are allowed: %s__%d %s(%d) In both cases, %s represents the generate statement label and the %d represents the generate parameter value at a particular iteration (this is the position	%s(%d)

⁵ For instance one that is Std_logic and one that is Real, or two that are a result of different converter insertions.

Table 2-3: Simulator Control Variables (continued)

Variable Name	Description	Default Value
	number if the generate parameter is of an enumeration type).	
OldVhdlForGenNames	Instructs the simulator to use a previous style of naming (pre-2010.1) for VHDL for... generate statement iteration names in the design hierarchy (1) or to use the current version (0). The previous style is controlled by the value of the GenerateFormat value.	0
VerboseOnHugeD2AInteractions	Specifies the number of times a warning should be generated when huge digital signal values are encountered during simulation. With this value set to 0, a message is displayed only the first time a huge digital signal becomes huge; if set to 1, a warning is displayed each time that same signal becomes huge.	0
AssocAsAWhole	Controls the port association file interpretation when associating a vector to a SPICE scalar pin. - When set to 1, the mapping in the association file is done as a whole for $P_digital \Rightarrow P_SPICE$ or $P_digital \Rightarrow (P_SPICE)$. However, if $P_digital$ is composite, then the SPICE port is connected to the full vector. - When set to 0, the mapping in the association file is done as a whole for $P_digital \Rightarrow P_SPICE$. For $P_digital \Rightarrow (P_SPICE)$, when $P_digital$ is composite, P_SPICE is mapped to its first element.	1

Related Topics

Incremental Saving Mode

Setting VectorPatternString

You can use the variable `VectorPatternString` to specify a pattern or rule by which a series of scalar elements can be made to correspond to the series of elements of an HDL vector or array.

In `VectorPatternString`, specify the form of the scalar element using the keywords `@name@` and `@idx@`.

- `@name@` represents the name of the vector and is mandatory
- `@idx@` represents the index of the scalar element and is also mandatory

The low index scalar element is always associated with the low index of the vector element.

The default setting is:

```
VectorPatternString = @name@_@idx@ @name@[@idx@] @name@<@idx@> @name@\\<@idx@\\>
```

Example

For example, this definition allows elements of the port vector `toto` to be associated with the following scalars:

- `toto_1 toto_2 toto_3 ...` (because these names match the first pattern in the definition), or
- `toto[8] toto[9] ...` (because these match the second pattern), or
- `toto<3> toto<4> toto<5> toto<6> ...` (because these match the third pattern).

To make `toto` also match scalars `toto_data1`, `toto_data2`, `toto_data3`, add the pattern `@name@_data@idx@` to the `VectorPatternString` list, that is:

```
VectorPatternString = @name@_@idx@ @name@[@idx@] @name@<@idx@>  
@name@\\<@idx@\\> @name@_data@idx@
```

Note

When specifying `VectorPatternString`, items are entered as a white-space-separated list.

Related Topics

Simulator Control Variables

VHDL Compiler Control Variables

The [vcom] section of the *modelsim.ini* file contains variables that control VHDL compilation.

- CreateSource

To instantiate a Questa SIM entity into Questa ADMS, use:

```
CreateSource = 1
```

The switch must be set before compilation of the VHDL design entity that is to be instantiated. During compilation, the switch compiles the VHDL entity into a format that can be instantiated in Questa ADMS. If the switch is not set, an error is generated if you try to instantiate the Questa SIM entity into Questa ADMS. The switch is set by default when using the version of Questa SIM that is installed during the AMS installation:

- Quiet

When enabled, Loading... messages are reported in the [Transcript Window](#). This variable is disabled by default.

- Quiet = 0

"Loading..." messages are disabled. This is equivalent to using the -quiet argument for the vacom/vcom/vlog commands

- Quiet=1

"Loading..." messages are reported. This is the same as the Questa SIM default (verbose mode).

- VHDL93

Specifies the VHDL language version during compilation. The VHDL-2002 standard is the Questa SIM default and this is compatible with Questa ADMS:

```
VHDL93 = 2002
```

- PreserveCase

When enabled, this variable preserves the case of VHDL and VHDL-AMS declarations (as they are written by the end-user) wherever they are displayed in Questa ADMS. This only affects how the identifiers are displayed, it does not make the language case-sensitive.

- PreserveCase = 1

VHDL and VHDL-AMS declarations are displayed in the case used in the declaration. This is the default.

- PreserveCase = 0

VHDL and VHDL-AMS declarations are displayed in lower case.

See [Case-sensitivity for VHDL-AMS Declarations](#).

See also vcom and vlog in the *Questa SIM Command Reference Manual*.

Example

The example below shows the modified settings of the [vcom] section of the *modelsim.ini* file for compatibility with Questa ADMS:

```
[vcom]
VHDL93 variable selects language version as the default.
Default is VHDL-2002.
; Value of 0 or 1987 for VHDL-1987.
; Value of 1 or 1993 for VHDL-1993.
; default or value of 2 or 2002 for VHDL-2002.
; Value of 3 or 2008 for VHDL-2008.
VHDL93 = 2002
.
.
.
; Turn on creation of VHDL package and entity source file (_parsed.vhd).
; Default is off (0).
CreateSource = 1
.
.
.
; Turn off "Loading..." messages. Default is messages on.
; Quiet = 1
.
.
.
```

VHDL-AMS Compiler Control Variables

The [vacom] section of the *modelsim.ini* file contains variables that control VHDL-AMS compilation.

- AmsCompileOption_F_MeansFileNotForce

When enabled, the argument `vacom -fi |file` can be replaced with `vacom -f`. This is to allow consistency with Questa SIM.

- AmsCompileOption_F_MeansFileNotForce = 1

The argument use with `vacom` to include a file containing command line arguments is `-f`.

- AmsCompileOption_F_MeansFileNotForce = 0

The argument use with `vacom` to include a file containing command line arguments is `-fi` or `-file`. This is the default.

Note

When setting this variable, be sure to edit the correct section of the *modelsim.ini* file, under the heading [vacom], as this variable is also available for `valog`.

See also **`vacom`** in *Questa ADMS Command Reference*.

Verilog-AMS Compiler Control Variables

The [valog] section of the *modelsim.ini* file contains variables that control Verilog-AMS compilation.

- AmsCompileOption_F_MeansFileNotForce

When enabled, the argument valog -fi |file can be replaced with valog -f. This is to allow consistency with Questa SIM.

- AmsCompileOption_F_MeansFileNotForce = 1

The argument use with valog to include a file containing command line arguments is -f.

- AmsCompileOption_F_MeansFileNotForce = 0

The argument use with valog to include a file containing command line arguments is -fi or -file. This is the default.

Note

When setting this variable, be sure to edit the correct section of the *modelsim.ini* file, under the heading [valog], as this variable is also available for vacom.

See also [valog](#) in *Questa ADMS Command Reference*.

Eldo Initialization File (eldo.ini)

Questa ADMS reads the Eldo initialization file, *eldo.ini*, which will be interpreted and loaded at the very beginning of each simulation run. This file stores information such as environment variable definitions, command line arguments, and netlist commands.

When an environment variable is set by an *eldo.ini* file, a warning is reported in the transcript:

```
** Warning: (vasim - 1428) One or more environment variables were set by
the eldo.ini file. This only affects the current simulation kernel (and not
Questa-ADMS user interface).
```

When the QSIM_INI variable is set by an *eldo.ini* file, an additional warning is produced:

```
# ** Warning: (vasim - 1427) The QSIM_INI environment variable (that points
to the modelsim.ini file to read) was set by the eldo.ini file. This may
change the path of the libraries used by this simulation, and the value of
some preferences set by the modelsim.ini file.
```

Note

From release 2025.3, MODELSIM is deprecated, specify QSIM_INI.

Related Topics

[System Initialization File \(modelsim.ini\)](#)

Questa ADMS GUI Preferences

The Questa ADMS GUI is programmed using Tcl/Tk. It is highly customizable; you can control everything from window size, position, and color to the text of window prompts, default output filenames, and so on. You can also add buttons and menus that run custom Tcl code.

The variable values save automatically when you exit Questa ADMS. Some of the variables are modified by actions you take with menus or windows (e.g., resizing a window changes its geometry variable). Or, you can edit the variables directly either from the **ADMS >** prompt or by selecting the menu item **Tools > Preferences** to display the [Preferences Dialog Box](#).

Most GUI preferences are stored as Tcl variables in the *.modelsim* file in your home directory on Unix/Linux platforms.

Changing GUI Preferences.....	2-22
GUI Preference Variables.....	2-25

Changing GUI Preferences

Tcl preference variables can be edited either using the Preferences dialog box or by using the set command in the Transcript window.

Note

Do not modify these preferences directly in the *.modelsim* file. You must select **Tools > Edit Preferences** and use the [Preferences Dialog Box](#) or enter a Tcl set command in the Transcript window.

Editing Tcl Preference Variables Using the Preferences Dialog Box.....	2-22
Editing Tcl Preference Variables using the set Command.....	2-23
Restoring the Default GUI Settings.....	2-24
Saved GUI Preferences.....	2-24

Editing Tcl Preference Variables Using the Preferences Dialog Box

You can edit a Tcl preference variable.

The following procedure describes how to edit a Tcl preference variable value using the [Preferences Dialog Box](#).

Procedure

1. Choose **Tools > Edit Preferences**; the Preferences dialog box is displayed. Select the By Name tab.
Every Tcl variable is listed with its description and current value. The variables are organized into groups in the Preference Item column, which can be expanded in a tree structure. You can also use the **Expand All** and **Collapse All** buttons to navigate the list.
2. Expand the tree in the Preference Item column to locate and select the required variable, or click **Find** to search for it.
3. Click **Change Value...**
4. Enter a new value for the variable in the pop-up dialog box and click **OK**.
5. Click **OK** to confirm, or **Apply** at the bottom of the Preferences dialog box to save the change when you exit Questa ADMS.

Editing Tcl Preference Variables using the set Command

You can use the Tcl set command from the Transcript window to view and change preference variables.

For example the following command will return the current value for the PrefReuse(GuiConfig) variable:

```
set PrefReuse(GuiConfig)
```

This returns:

```
# yes
```

Indicating that the current value of the variable is "yes".

To change a variable, simply supply a new value using this command. For example, the following command will change the variable value to "ask".

```
set PrefReuse(GuiConfig) ask
```

Related Topics

[Transcript Window](#)

Restoring the Default GUI Settings

You can return the GUI to its original state.

Procedure

To return the GUI to its original state, select **Tools > Edit Preferences**, select the **By Name** tab, then click **Reset Defaults**.

Saved GUI Preferences

Questa ADMS GUI preferences are saved automatically when you exit the tool.

They are stored by default in the *.modelsim* file in your HOME directory on UNIX/Linux platforms. If you prefer to store GUI preferences to a different location, set the MODELSIM_PREFERENCES environment variable to define it. Setting this variable causes Questa ADMS to use the specified path and file instead of your HOME directory.

This variable may contain a relative pathname, in which case, the file is relative to the working directory at the time the tool is started.

The file does not need to exist before setting the variable as Questa ADMS will initialize it. If the file does exist, ensure that the correct read/write access is set. If the file is read-only, Questa ADMS will not update or otherwise modify the file.

Related Topics

[GUI Preference Variables](#)

[Questa ADMS GUI Preferences](#)

[Changing GUI Preferences](#)

GUI Preference Variables

Preference variables control the Questa ADMS GUI.

Note

Do not modify these preferences directly in the *modelsim* file. You must use the **Tools > Edit Preferences** dialog box or enter a Tcl set command in the Transcript window. See [Changing GUI Preferences](#).

GUI Save and Reuse Preference Variables

- **PrefReuse(GuiConfig)**
 Enables and disables the GUI save/reuse feature. When enabled, windows visibility, window size and position, and commands previously run are saved and can be reloaded in the next session. This variable is disabled by default.
 - **yes**
 The save/reuse feature is enabled. When enabled, large designs may take noticeably longer to load in Questa ADMS.
 - **ask**
 When the design is reloaded, the [Reuse Previous Configuration Dialog Box](#) prompts you to confirm what should be reused from the previous simulation.
 - **no**
 All other PrefReuse variables will be ignored and Questa ADMS will not load the configuration from the previous session. This is the default setting.
- **PrefReuse(GuiStructureConfig)**
 Preserves the view of the hierarchy in the [Structure Window](#).
 - **1 (enabled)**
 When enabled, the tree expansion in the Structure window from the previous session will be reloaded.
 - **0 (disabled)**
 When disabled, the previous design context, that is, the active region in the Structure window, and the contents of the [Source Window](#), [Objects Window](#), [Locals Window](#), and List Window will not be reloaded.
- **PrefReuse(WaveConfig)**
 Controls how Solido Waveform Analyzer configurations are saved and reloaded.
 - **tcl**

The configuration of the Solido Waveform Analyzer window from the previous session is saved and loaded from the file `.ez.do`. This is the default setting.

Note

The `.ez.do` file should only be used in designs that do not contain RF domain analyses. Solido Waveform Analyzer configuration files are re-generated at the end of the session therefore, any modifications made to the file will be lost.

- `swd`

The configuration of the Solido Waveform Analyzer window from the previous session is saved and loaded from the file `.swd`.

- `no`

The configuration of the Solido Waveform Analyzer window from the previous session is saved but not reloaded in the current session.

Structure Window GUI Preferences

- `PrefStructure(ShowUserDefinedConverters)`

Makes user-defined boundary elements visible in the Structure window. This is used in conjunction with the **Contributor Window** to interrogate nets.

- 1

User-defined boundary elements are displayed in the Structure window and the Contributor window.

- 0

User-defined boundary elements are only displayed in the Contributor window.

- `PrefStructure(MaxSpiceDevices)`

Limits the number of devices displayed in a SPICE subcircuit.

- -1

All devices are displayed.

- 0

No devices are displayed.

- $N > 0$

Specifies the maximum number of devices displayed. Default is 1000.

Note

Does not apply to devices in Verilog-A or Verilog-AMS.

Waveform Viewer Mode Variable

- `viewermode ezwave|questa|both`

Specifies the destination waveform viewer for plot requests of digital HDL objects that can optionally be displayed in Solido Waveform Analyzer.

- ezwave
All objects are plotted in Solido Waveform Analyzer even if Questa viewer is opened. Any digital HDL objects that are outside the WDB subset are plotted in the Questa viewer. This is the default.
- questa
All objects are plotted in Questa SIM viewer.
- both
Objects are plotted in both the Solido Waveform Analyzer and Questa SIM viewers.

Solido Waveform Analyzer Waveform List Preferences

- PrefWave (HideEZwaveWaveformList)
Specifies whether the Solido Waveform Analyzer Waveform List is hidden.
 - 1
The Solido Waveform Analyzer Waveform List is hidden.
 - 0
The Solido Waveform Analyzer Waveform List display option follows the Solido Waveform Analyzer setting of wfChooserVisible. This is the default.

Related Topics

- [Save/Reuse of Questa ADMS GUI Configuration](#)
- [Viewing Boundary Elements in the Structure Window](#)

Optional Environment Variables

Environment variables are referenced and set according to operating system conventions in order to prepare the Questa ADMS environment prior to compilation and simulation. This section describes how to define your own values for environment variables.

VA_INCLUDE_PATH..... 2-28

QSIM_INI..... 2-29

AMS_VCO_MODE..... 2-30

QSIM_LOCATION_MAP..... 2-30

LD_LIBRARY_PATH..... 2-30

AMS_ADMS_LICENSE_QUEUE..... 2-31

VA_INCLUDE_PATH

The `VA_INCLUDE_PATH` environment variable is used to locate the Verilog-AMS files specified on the Verilog-AMS ``include` compiler directive. `VA_INCLUDE_PATH` is a mandatory runtime variable and is hidden from the environment.

You can access it as follows:

```
$MGC_AMS_HOME/com/ams_runtime_setup.ksh VA_INCLUDE_PATH
```

Its default value is `$MGC_AMS_HOME/include/veriloga`.

If you define your own value of `VA_INCLUDE_PATH` in the shell environment according to the shell syntax (C-shell or SH) it will override the default AMS runtime value. You can check the value at any time with the above command. See [Managing Environment Variables](#) in *Solido Simulation Suite Installation Guide*.

The form of the Verilog-AMS ``include` directive is:

```
`include "<include_filename>"
```

where `include_filename` is the name of the file to be included in the source file. You can specify a full path, relative path, or simple filename. For example:

```
'include "libs/capacitor/cap.inc"
'include "/home/comp/ahdlinclude/count.va"
'include "/macro.va"
```

The `VA_INCLUDE_PATH` environment variable, if it exists, must contain a series of path names of directories separated by ":". The directories need not all exist. A relative path (without a leading "/") is relative to the directory containing the main Verilog-A file, whether it appears as an `include_filename` that is a file, or as an element of `VA_INCLUDE_PATH` that is a directory.

The search rules used to find Verilog-AMS file(s) are as follows:

- If `include_filename` is a simple filename or a relative filename, then the directory containing the source file is looked in first before looking inside the working directory.
- If `include_filename` is a simple filename and `VA_INCLUDE_PATH` exists, search the directories in the `VA_INCLUDE_PATH` environment variable, in their order of appearance, for a file with the given simple filename. Include the first matching file that is encountered.

If neither of the above conditions exist, an error message is returned during compilation.

See [VA_INCLUDE_PATH Variable](#) in the *Eldo Verilog A User's Manual*.

QSIM_INI

You can use the QSIM_INI variable to specify a different initialization file to the one in the location from which Questa ADMS was launched, or referenced in a command file. This enables you to have a “global” library map that is used even when you are in a different directory.

Note

From release 2025.3, MODELSIM is deprecated, specify QSIM_INI.

Specify the full path including the filename to the *modelsim.ini* file, for example:

```
setenv QSIM_INI /home/shark/my_modelsim.ini
```

You can use environment variables in your initialization file. Use a dollar sign (\$) before the environment variable name, for example:

```
[Library]
work = $HOME/work_lib
test_lib = ./$TESTNUM/work
```

It is possible to maintain different versions of the *modelsim.ini* file with different names. The QSIM_INI environment variable can be set to point to whichever file is required.

Specify the full path to a QSIM_INI variable or the library files to compile or run a simulation from your working directory if the QSIM_INI variable or library files are located in a different directory. For example:

```
valib $PWD/testlib
valog *.vla
setenv QSIM_INI $PWD/modelsim.ini
cd test
vasim -cmd compressor.cir
```

Note

If the library is specified as a relative path instead of an absolute path and you change to a different working directory, then the library may not be found.

Related Topics

System Initialization File (modelsim.ini)

AMS_VCO_MODE

AMS_VCO_MODE specifies whether Questa ADMS is running on a 32-bit or 64-bit version of Linux, and accepts the values 32 or 64. This variable is evaluated against the Questa SIM variable MTI_VCO_MODE, set for Questa SIM running within the ADMS environment.

This evaluation is made according to the following rules:

- If AMS_VCO_MODE exists, its value is used, regardless of the setting of the MTI_VCO_MODE variable set for Questa SIM.
- If it does not exist, and if MTI_VCO_MODE exists, ADMS will use the MTI_VCO_MODE value.
- If neither environment variable is specified, a default value of 32 is used.

AMS_VCO_MODE and MTI_VCO_MODE are set by default when the AMS tree is installed.

Note

Questa SIM is considered to be running within the ADMS environment when the environment variable MGC_AMS_HOME is defined.

QSIM_LOCATION_MAP

QSIM_LOCATION_MAP sets the path to the location map table file.

Note

From release 2025.3, MGC_LOCATION_MAP is deprecated, and a warning will be generated. Instead, specify QSIM_LOCATION_MAP, or the environment variable DISABLE_QUESTA_DEPRECATION_WARNING=1 to suppress the warning.

This file defines easily reallocated “soft” paths which are used to find source files when the following commands are used:

vamap, vamatch, varefresh, vasetlib, vasim, vasp, vacom, vadir, valib, valog, vmake/vmake

In Questa ADMS, location maps are handled in the same way as Questa SIM. For a full, detailed description of location map support, see [Location Maps](#) and Location Mapping in the *Questa SIM User's Manual*.

LD_LIBRARY_PATH

In C Code encapsulation, the LD_LIBRARY_PATH variable must be correctly set.

In the provided examples, it is done using the following command:

```
LD_LIBRARY_PATH=`/bin/sh $MGC_AMS_HOME/com/ams_runtime_setup.ksh  
LD_LIBRARY_PATH`  
  
export LD_LIBRARY_PATH
```

Refer to [Managing Environment Variables](#) in *Solido Simulation Suite Installation Guide* for more information.

Related Topics

[C Code Encapsulation](#)

[C Code Encapsulation](#)

[System Initialization File \(modelsim.ini\)](#)

AMS_ADMS_LICENSE_QUEUE

If you start a Questa ADMS simulation with the `vasim` command, then specifying the `AMS_ADMS_LICENSE_QUEUE` environment variable will automatically launch the `vasim -queue` argument.

This environment variable also applies to `vasim` commands executed through the GUI or in a *do* file.

Refer to [vasim](#) in *Questa ADMS Command Reference* for more information.

Files Used by Questa ADMS

Questa ADMS generates temporary files during library management, compilation, and simulation.

Temporary Files

Questa ADMS generates several temporary files during the different stages of the flow of the tool: library management, compilation phase, and simulation. A list of these files is provided below, together with details of where they can be located. Under normal circumstances they are deleted before simulation terminates. However, if Questa ADMS terminates abnormally some of them may persist. If you receive an error message that mentions a file with one of these extensions, you should search for and delete the files, then rerun the associated command.

Table 2-4: Questa ADMS Temporary Files

File Extension	Description
LIB/_OS/\$AMS_VCO/<unit_name>.c	Created during compilation phase
(where LIB is the current working library)	
\$TmpDir/info, \$TmpDir/del, \$TmpDir/lib	Created by the vadel command
\$TmpDir/del, \$TmpDir/map	Created by the vmap command
\$TmpDir/2, \$TmpDir/my.vhd, \$TmpDir/res.vhd, \$TmpDir/ent.ms, \$TmpDir/extendedName, \$TmpDir/ams, \$TmpDir/dir, \$TmpDir/ms2, \$TmpDir/msent, \$TmpDir/adms, \$TmpDir/msarch, \$TmpDir/foreign, \$TmpDir/info, \$TmpDir/ms_vmap, \$TmpDir/extended, \$TmpDir/du, \$TmpDir/ms, \$TmpDir/comp	Created by the vacom command
\$TmpDir/link, \$TmpDir/vdir, \$TmpDir/ms_vmap, \$TmpDir/2, \$TmpDir/all, \$TmpDir/ecg, \$TmpDir/ecg2, \$TmpDir/comp, \$TmpDir/comperr, \$TmpDir/ver, \$TmpDir/ver2	Created by the valog command
\$TmpDir	Created by the vadir command
/var/tmp/vaspi	Created by the vaspi command
\$TmpDir/du, \$TmpDir/info	Created by the vasim command
\$TmpDir/\$AMS_USER<hour> <year><day><month>	Created during simulation, contains transcript information
<CmdFileName>.alt ¹	Internal files created during simulation in the command file directory

¹ <CmdFileName> is the name of the simulation command file without the extension.

Table 2-4: Questa ADMS Temporary Files (continued)

File Extension	Description
libfile_eldo_XXXX.cir	
<CmdFileName>.asu	
<CmdFileName>.h3	
<CmdFileName>.cml	
<CmdFileName>.rmos	
<CmdFileName>.net	
<CmdFileName>.mfta	
<CmdFileName>.ostat	
.h23i9k1	Multiple run files created during simulation in the command file directory
#mrun_map#XX	
.eldo_env	
.eldo_env.tmp	
<CmdFileName>.id	WDB output files created during simulation in the command file directory
<CmdFileName>.lock	

Files created by the script commands generate temporary files in the *\$TmpDir* directory. If you receive an error message that mentions a file with the *\$TmpDir* prefix, you should search for and delete the files, then re-run the corresponding script command.

$\$TmpDir = \{TMP-\{tmp-/tmp\}/vaxxx.\$. \$i \text{ and } i = [1, 2, \dots]\}$

which means that the *TmpDir* variable is equal to:

- *TMP/vaxxx.\\$. \$i*, if *TMP* variable is defined on the end-user's machine
- else *tmp/vaxxx.\\$. \$i*, if *tmp* is defined on the end-user's machine
- else */tmp/vaxxx.\\$. \$i*, if */tmp* exists on the end-user's machine

where *vaxxx* is the Questa ADMS command used, *\$\$* is a process number, and *\$i* is an index: 1, 2, 3, etc.

Standard Output Files

Table 2-5 lists the output files generated by Questa ADMS.

Table 2-5: Questa ADMS Standard Output Files

File or Directory	Description
<working_library>/__index	General library management directories
<working_library>/MS	
<working_library>/_CG	
<working_library>/_DAT	
\$ent_file_comm	Created by the vacom command. \$ent_file_comm is equal to \$library_path/_CG/_parsed_\$e_comment.vhd
\$ent_file_adms	Created by the vacom command. \$ent_file_adms is equal to \$library_path/_CG/_parsed_\$e.vhd
<working_library>/<kind>/unit_name.vif	Created during compilation. <kind> can be the following: _ENTI, _ARCH, _PACK, _BODY, _CONF, _MODU, _PRIM, _VAMSDISC, _VAMSNAT, _VAMSCONM, _VAMSCONS
<working_library>/<kind>/another_unit_name.vif.obs	Created during compilation. <kind> can be the following: _ENTI, _ARCH, _PACK, _BODY, _CONF, _MODU, _PRIM, _VAMSDISC, _VAMSNAT, _VAMSCONM, _VAMSCONS
<working_library>/__TOP__	Created during compilation.
<working_library>/_CG/<unit_name>.ecg	Created during compilation phase. For further information see Design Libraries
<working_library>/_CG/__internal__connect_specification_<unit_name>.ecg	
<working_library>/_CG/<unit_name>.info	
<working_library>/_CG/<unit_name>.ms	
<CmdFileName>.ali ²	Created during simulation in the command file directory
<CmdFileName>.conv	Boundary Elements Log File , created during simulation in the command file directory
<CmdFileName>.wdb	Created during simulation in the command file directory

² <CmdFileName> is the name of the simulation command file without the extension.

Table 2-5: Questa ADMS Standard Output Files (continued)

File or Directory	Description
<CmdFileName>.cou	Created during simulation in the command file directory
.transcript	Created during simulation in <path>/<transcript>
<CmdFileName>.errm.log	Created during simulation in command file directory
<CmdFileName>.dbg	Created when the vasim argument -debuginfo is specified. This file should be sent to a support engineer in the event of a problem.
<CmdFileName_date_time>.stat	Created when the vasim argument -stat is specified. This file should be sent to a support engineer in the event of a problem. See Statistics File Reference .
<CmdFileName>.meas	WDB output files created during simulation in the command file directory. See Eldo Input and Output Files and the .NOISETRAN command in the <i>Eldo Reference Manual</i> .
<CmdFileName>.hmp	
<CmdFileName>.ext	
<CmdFileName>.trX	HSPICE output files created during simulation in the command file directory
<CmdFileName>.acX	
<CmdFileName>.swX	
<CmdFileName>.csdf	CSDF output file created during simulation in the command file directory (see the -gwl argument in the <i>Eldo Reference Manual</i>)
<CmdFileName>.ez.do	WDB output Tcl file created during simulation in the command file directory. See Reusing Solido Waveform Analyzer Configurations . ³
<CmdFileName>.swd	WDB output file created during simulation in the command file directory. See Reusing Solido Waveform Analyzer Configurations .
<CmdFileName>.psf	PSF output file created during simulation in the command file directory. See the -gwl argument in the <i>Eldo Reference Manual</i> .
<CmdFileName>.spi3	SPICE3 output file created during simulation in the command file directory. See SPI3ASC , SPI3BIN , SPI3NOCOMPLEX , SPICEDC , and SPIOUT in the <i>Eldo Reference Manual</i> .
<CmdFileName>.bkn	Optimizer files created during simulation in the command file directory

³ The.ez.do file should only be used in designs that do not contain RF domain analyses. See “.ez.do FileLimitations”.

Table 2-5: Questa ADMS Standard Output Files (continued)

File or Directory	Description
<CmdFileName>.opz	
<CmdFileName>.opz2	
<CmdFileName>.imp	
<CmdFileName>.opsX	
<CmdFileName>.probeopX	OP files created during simulation in the command file directory (see the -probeop2 argument in the <i>Eldo Reference Manual</i>)
<CmdFileName>.opX	
<CmdFileName>.aex	Extract file created during simulation in the command file directory (see Eldo Input and Output Files in the <i>Eldo User's Manual</i>)
<CmdFileName>.pz	Pole-Zero post-processor output file created during simulation in the command file directory (see the .PZ command in the <i>Eldo Reference Manual</i>)
iic	See Save-Generated Files .
cml	
ms	
adms	
wdb	
ez.do	An alternative to swd. Will not be generated if the simulation is launched in batch mode (-c option) ³
swd	Will not be generated if the simulation is launched in batch mode (-c option)
.reuse	Save/reuse files generated at the end of the session in the command file directory (see section Save/Reuse of Questa ADMS GUI Configuration)
<CmdFileName>.reuse	
<CmdFileName>.exp.reuse	
<CmdFileName>.part.reuse	
<CmdFileName>.BDY	ADiT output file containing a list of boundary nodes between Eldo and ADiT
<CmdFileName>.BOX	ADiT white box netlist file
<CmdFileName>.DOT	ADiT white-box dot statements file
<CmdFileName>.ICO	ADiT node operation biases file

vams_ms-stacktrace File

If an unexpected problem is detected by Questa ADMS (for example, a message that reports an “Internal Error”), the file *vams_ms-stacktrace.dump* or *vams_ms-stacktrace.vstf* will be generated in the output directory and the simulation will terminate. Include the *vams_ms-stacktrace* file along with the model causing the error when submitting your issue to Siemens Digital Industries Software support.

Location Maps

You can use a location map file to replace prefixes of physical pathnames by environment variables.

Create a location map file called *mgc_location_map* in the current directory or your home directory. This file should be used to define a mapping between physical pathname prefixes and environment variables. Logical pathnames are listed, followed by one or more equivalent physical pathnames. Logical pathnames must be prefixed with \$ and physical pathnames must begin with /. Physical pathnames are equivalent if they refer to the same physical directory (they may have different pathnames on different systems).

For example, an entry in a location map file might look like:

```
$project1
/home/my_directory/project1
```

If the **QSIM_LOCATION_MAP** environment variable is set, the location map file is opened on invocation. If QSIM_LOCATION_MAP is not set, Questa ADMS will look for a file named *mgc_location_map* in the following locations, in order:

- The current directory
- Your home directory
- The directory containing the AMS binaries
- The AMS installation directory

Note

From release 2025.3, MGC_LOCATION_MAP is deprecated, and a warning will be generated. Instead, specify QSIM_LOCATION_MAP, or the environment variable DISABLE_QUESTA_DEPRECATION_WARNING=1 to suppress the warning.

When writing paths in output files, Questa ADMS reverses them and writes variables defined in the location map when it can be applied.

In Questa ADMS, location maps are handled in the same way as Questa SIM. For a full, detailed description of location map support, see Location Mapping in the *Questa SIM User's Manual*.

How Location Mapping Works

When a pathname is stored, an attempt is made to map the physical pathname to a path relative to a logical pathname. This is done by searching the location map file for the first physical pathname that is a prefix to the pathname in question. The logical pathname is then substituted for the prefix.

For example if the following mapping exists in the *mgc_location_map* file:

```
$SRC/test.vhd  
/usr/vhdl/src/test.vhd
```

If a mapping can be made to a logical pathname, then this is the pathname that is saved.

A typical mapping would be the path to a source file entry for a design unit in a library.

For mapping from a logical pathname back to the physical pathname, an environment variable must be set for each logical pathname (with the same name). Questa ADMS reads the location map file when a tool is invoked. If the environment variables corresponding to logical pathnames have not been set in your shell, Questa ADMS sets the variables to the first physical pathname following the logical pathname in the location map. For example, if you don't set the SRC environment variable, Questa ADMS will automatically set it to:

```
/home/vhdl/src
```

The same makefile generated by *vmake* or *vamake* cannot be used in two different physical locations. These commands do not take the location map into account when referring to the original source files.

It is not possible to provide a location map file directly using the Eldo command:

```
.OPTION use_location_map=<file>
```

If this command is given, a warning is issued with the instruction to use the `QSIM_LOCATION_MAP` environment variable instead.

Refer to Location Mapping in *Questa SIM User's Manual* for a full, detailed description of location map support.

Related Topics

[MGC_LOCATION_MAP](#)

3. Questa ADMS GUI Overview

This chapter provides an overview of the Questa ADMS graphical user interface (GUI).

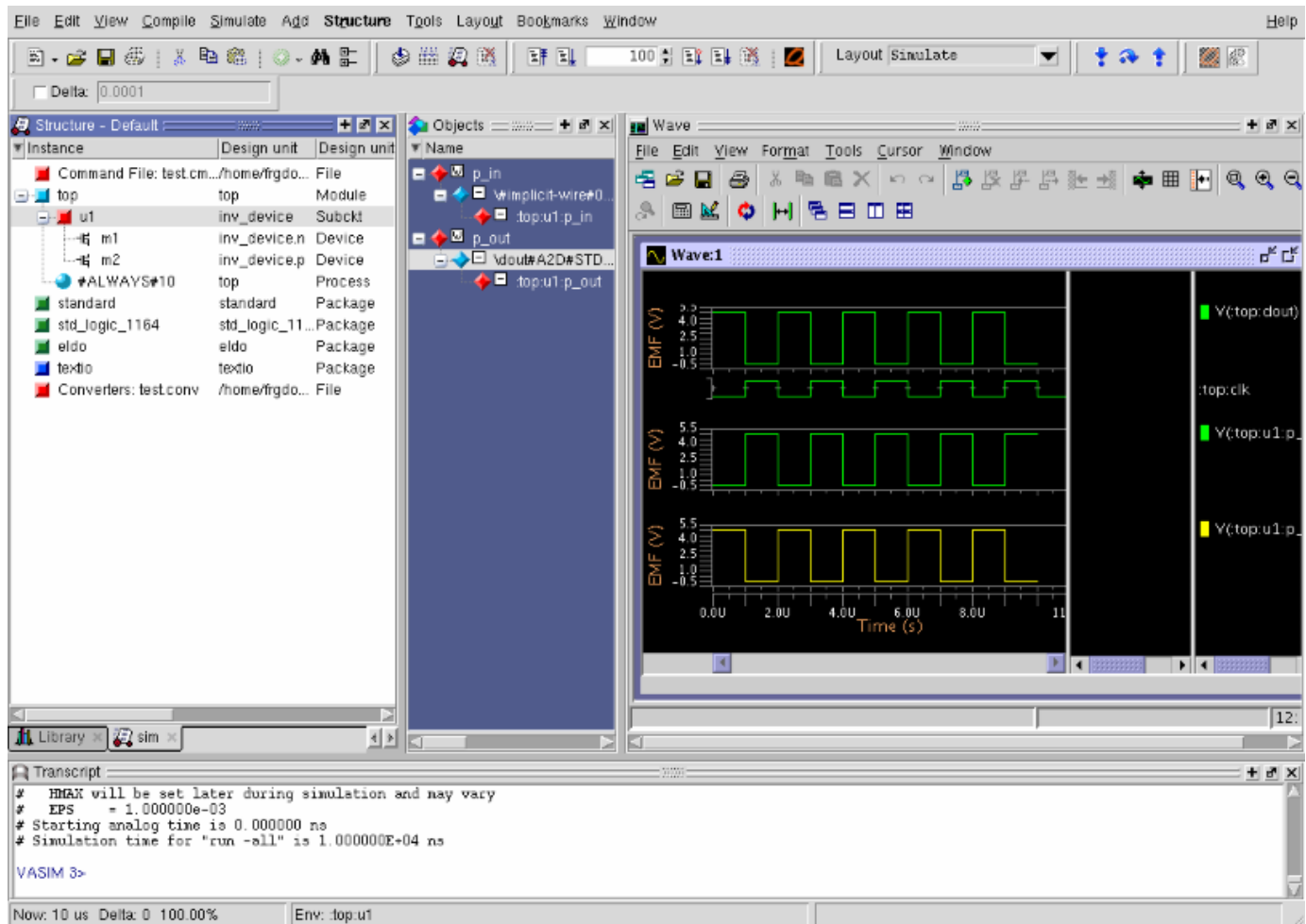
For a complete description of the dialog boxes and windows, see [Dialog and Field Reference](#).

- Questa ADMS Application Interface..... 3-2
- Window Zooming..... 3-4
- Undocking and Docking Windows..... 3-5
- Moving Windows..... 3-6
- Organizing Column Information..... 3-7
- Quick Access Toolbars..... 3-7
- GUI Icons and Their Meanings..... 3-8
- Status Bar..... 3-9
- Scaling Fonts..... 3-10
- Find and Filter Functions..... 3-10
- Filtering and Sorting Objects in the Objects Window..... 3-13
- Wildcard Modes when Searching..... 3-14
- Filtering Objects by Type in the Structure Window..... 3-15
- Editing GUI Window Colors..... 3-16
- Editing GUI Window Font Type and Size..... 3-17
- Customizing the Simulator GUI Layout..... 3-17
- Bookmarks..... 3-19
- Save/Reuse of Questa ADMS GUI Configuration..... 3-19

Questa ADMS Application Interface

The Questa ADMS application interface collects all functional windows together in an intuitive working environment.

Figure 3-1: Questa ADMS Main Window



Description

Windows can be positioned at various places within the main window or they can be dragged out ("undocked") of the parent window altogether.

The user interface is based on Tcl/Tk, which means that you have the capability to build your own simulation environment. Preference variables and configuration commands give you control over the use and placement of windows, menus, menu options, and buttons. Refer to [System Initialization File \(modelsim.ini\)](#) and Tcl and DO Files in the *Questa SIM User's Manual* for more information.

Objects

Table 3-1: Questa ADMS Windows

Window	Icon	Description
Library Window		Displays the libraries in the current project and their contents
Structure Window		Displays a hierarchical view of the structure of the design. An entry is created for each object in the design.
Objects Window		Shows the names and current values of items in the region currently selected in the Structure Window
Processes Window		Displays a list of processes that are scheduled to run during the current simulation cycle or that are declared in the region currently selected in the Structure Window
Locals Window		Displays the names of variables, generics and constants within the current active processes or current block, as well as their current values
Source Window		Used to view, edit and, optionally, compile and simulate VHDL-AMS, Verilog-AMS, SPICE, VHDL or Verilog source code. This window is updated according to what is selected in the Structure Window or Processes Window .
Transcript Window		Keeps a running history of commands and messages and provides a command line interface
Wave Window (Solido Waveform Analyzer)		Solido Waveform Analyzer viewer capable of handling analog and digital waves at the same time
Wave Window (Questa SIM)		Questa SIM viewer for inspecting digital waves
List Window		Questa SIM viewer for inspecting the simulation values of selected digital nets in a tabular format
Rundata Window		Displays results of the <code>.chi</code> file

Related Topics

[Window Zooming](#)

[Undocking and Docking Windows](#)

[GUI Icons and Their Meanings](#)

Window Zooming

You can zoom windows.

You can expand windows to fill the entire main window by clicking the zoom  button in a window header. When you do this, all docked windows maximize to fill the main window and are accessible using tabs at the bottom of the window.

To restore the windows to their original sizes and positions, click the unzoom  button in a window header.

Figure 3-2 shows the Questa ADMS GUI in the default unzoomed mode, with all windows tiled in the main window area. **Figure 3-3** shows the Questa ADMS GUI in zoomed mode, with the **Objects Window** occupying the main window area.

Figure 3-2: Questa ADMS Graphical User Interface — Unzoomed

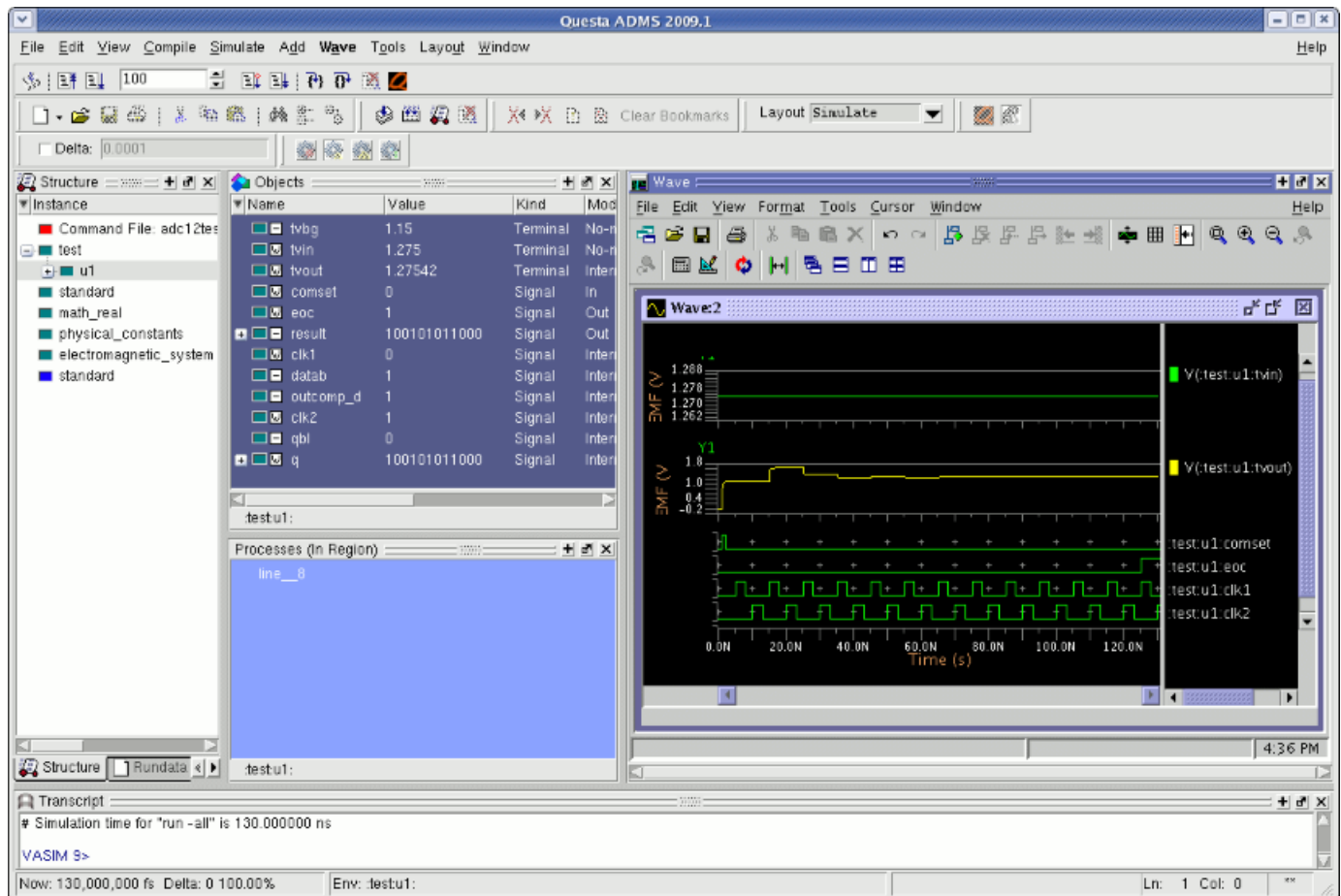
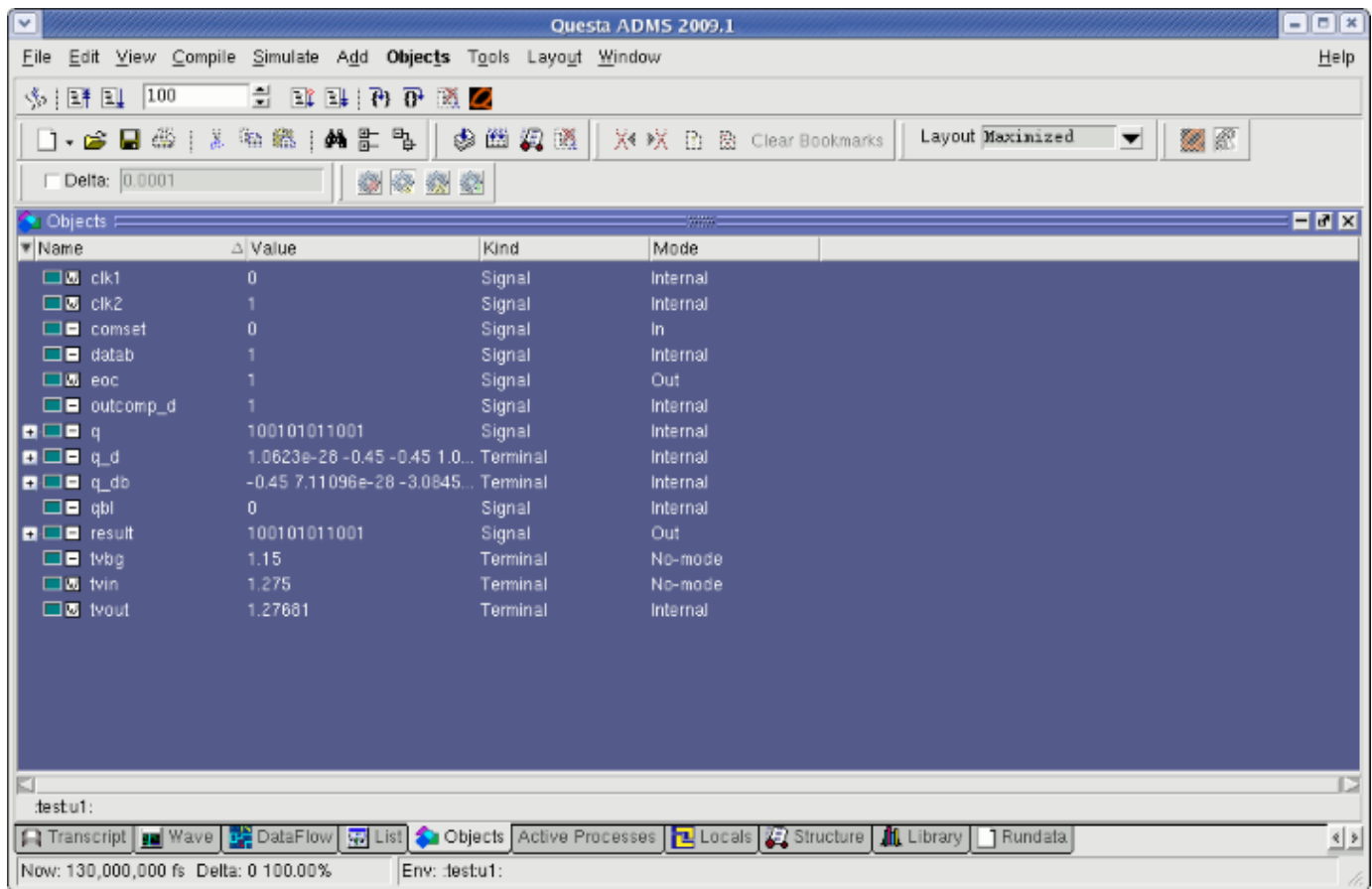


Figure 3-3: Graphical User Interface — Zoomed Objects Window



Related Topics

[Undocking and Docking Windows](#)
[GUI Icons and Their Meanings](#)

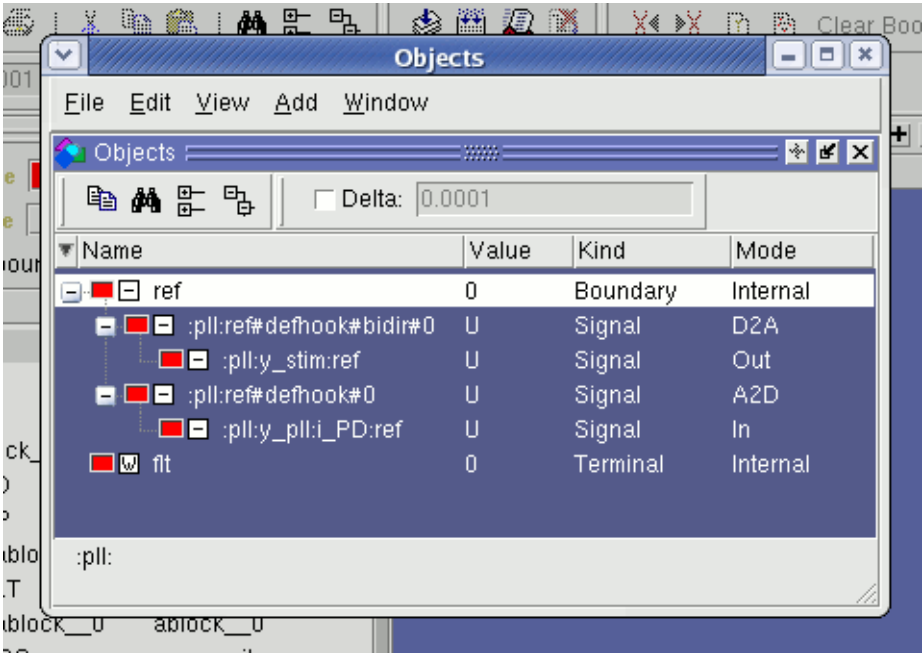
Undocking and Docking Windows

Windows can be docked and undocked from the main application window.

When undocked, some windows offer access to otherwise-unavailable functionality. [Figure 3-4](#) shows an example of an undocked [Objects Window](#).

You can undock and dock a window using the undock and dock buttons,  and  respectively.

Figure 3-4: Undocked Objects Window



To redock a floating window, either click on the double-bar handle in the window header and drag it back into the main window, or click the dock button.

Related Topics

- Window Zooming
- Moving Windows

Moving Windows

The double bar in the window header enables you to change the window's position. You can move a window within the main window or to an undocked position outside the main window.

Figure 3-5: GUI: Double Bar



Click-and drag the handle in the middle of a double bar (your mouse pointer will change to a four-headed arrow when it is in the correct location). As you move the mouse to various parts of the main window, an outline will show you valid locations into which to drop the window. If you drag the window outside of the main window and let go of the mouse button, the window becomes undocked.

Related Topics

- Window Zooming

Undocking and Docking Windows

Organizing Column Information

You can organize information displayed in columns, in various ways.

Many windows display information in columns, and you can perform a number of operations on these:

- Click and drag on a column heading to rearrange columns.
- Click and drag on a border between column names to increase/decrease column size.
- Sort columns by clicking once on the column heading to sort in ascending order; clicking twice to sort in descending order; and clicking three times to sort in default order.
- Hide or show columns by either right-clicking a column heading and selecting an object from the context menu or by clicking the column-list dropdown arrow and selecting an object.

Related Topics

[Window Zooming](#)

[Undocking and Docking Windows](#)

Quick Access Toolbars

Toolbar buttons provide access to commonly used commands and functions.

Toolbars can be docked and undocked (moved to or from the main toolbar area) by clicking and dragging on the toolbar handle at the left-edge of a toolbar.

Figure 3-6: Toolbar Manipulation



You can also hide/show the various toolbars. To hide or show a toolbar, right-click on a blank spot of the main toolbar area and select a toolbar from the list.

To reset toolbars to their original state, right-click on a blank spot of the main toolbar area and select **Reset**.

Related Topics

[Window Zooming](#)

[Undocking and Docking Windows](#)

[GUI Preference Variables](#)

GUI Icons and Their Meanings

The color and shape of icons indicate the language and type of a design object.

[Table 3-2](#) and [Table 3-3](#) describe the icons used in the Questa ADMS GUI:

Table 3-2: Design Object Icons

Icon	Design Unit
	VHDL-AMS
	Verilog-AMS
	VHDL
	Verilog
	SPICE
	SPICE Black-Box
	Eldo SPICE Device
	MODSST Algorithm
	SPICE and Fast-SPICE
	Fast-SPICE
	Fast-SPICE Black-Box
	Foreign SPICE

Table 3-3: Design Unit Icons in Libraries

Icon	Library Design Unit
	Library
	VHDL-AMS Entity

Table 3-3: Design Unit Icons in Libraries
(continued)

Icon	Library Design Unit
	Verilog-A Module
	SPICE Subckt
	VHDL Entity
	VHDL Architecture
	Verilog Module

Related Topics

[Library Window](#)

[Objects Window](#)

[Source Window](#)

[Structure Window](#)

Status Bar

Fields in the status bar at the bottom of the main window provide information about the current simulation.

Figure 3-7: Status Bar

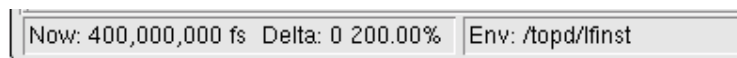


Table 3-4: Information Displayed in Status Bar

Field	Description
Now	The current simulation time
Delta	The current simulation iteration number
Env	The name of the current context (the object selected in the active Structure Window)

Related Topics

[Simulation](#)

Scaling Fonts

You can adjust font settings to accommodate the aspect ratios of widescreen and double-screen displays.

It may be necessary to adjust font settings to accommodate the aspect ratios of widescreen and double-screen displays or to handle launching Questa ADMS from an X-session. You should only need to set the font scaling once unless you change display resolution or the hardware (monitor or graphics card). The font scaling setting applies to Windows and UNIX operating systems. On UNIX systems, the font scaling setting is stored based in the \$DISPLAY environment variable.

Prerequisites

- A ruler is required to perform this setup.

Procedure

1. With the **Transcript Window** active, select **Transcript > Adjust Font Scaling**. The Adjust Scaling dialog box is displayed.
2. Instructions on the dialog box explain how to measure and enter the required values, but in summary, place the ruler on the screen and then read off (and enter) a pixel value against an arbitrary point on the ruler.
3. Click **OK**, and then restart Questa ADMS. The tool fonts are scaled according to your measurements.

Related Topics

[Editing GUI Window Font Type and Size](#)

Find and Filter Functions

Find and/or filter capabilities are available for most windows.

The filtering function is denoted by a Contains field label ([Figure 3-9](#)).

Figure 3-8: Search Bar in Find Mode



[Figure 3-8](#) shows an example of searching for a string mat in the **Structure Window** (find mode).

Figure 3-9: Search Bar in Filter Mode



Figure 3-9 shows an example of searching for any occurrences of the number 5 in the **Structure Window** (filter mode).

For windows that support both find and filter modes, you can toggle between the two modes by using any of the following methods:

- Use the **Ctrl+M** hotkey
- Click the **Find** or **Contains** words in the toolbar
- Select the mode from the Find Options popup menu (accessed by clicking the magnifying glass icon at the left of the field)

The last selected mode is remembered between sessions.

The search bar appears at the bottom of the active window when you do any one of the following:

- Choose **Edit > Find** in the menu bar
- Click the **Find** icon in the toolbar 
- Right-click anywhere in the active window and select **Find** from the popup menu
- Press **Ctrl+f** on Windows or **Control s** on your UNIX or Linux

All of the above actions are toggles; repeat the action and the search bar will close.

There is a simple history mechanism to allow saving search strings for later use. The keyboard shortcuts to support this are:

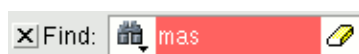
- **Ctrl+S** - save current search text into history list
- **Ctrl+P** - retrieve previous search text
- **Ctrl+N** - retrieve next search text

Other hotkey actions include:

- Esc key - closes the search bar
- Enter (Windows) or Return (UNIX or Linux) key - initiates a "Find Next" action
- **Ctrl+T** - toggles search while typing (default is on)

As you enter text in the data entry field it will turn red as soon as no matches are found, as shown in **Figure 3-10**.

Figure 3-10: Example of a String Not Found



The graphic elements associated with the search bar are shown in [Table 3-5](#). Note that all graphic elements do not appear for all window types.

Table 3-5: Graphic Elements of Search Bar in Find Mode

Button	Name	Action
	Close Search Bar	Closes the search bar
	Find Options	Opens the Find Options popup menu
	Search entry field	Allows entry of search parameters
	Clear Entry Field	Clears the entry field. This button does not appear until something has been entered into the search entry field.
	Execute Search	Initiates the search
	Toggle Search Direction	Toggles search direction upward or downward through the active window
	Bookmark All Matches	Places a blue flag (bookmark) at every occurrence of the search item in the Source Window ; highlights every occurrence in other windows. To remove bookmarks from the Source Window , select Source > Clear Bookmarks .
☐ Aa	Match Case	The search must match the case of the text entered in the search entry field
☐ {a}	Exact (whole word)	Searches for whole words that match those entered in the search entry field
☐ Σ	Regular Expression	Searches for a regular expression
☒ 	Wrap Search	If selected, searches are continued beyond the end of the file so that the whole file is searched regardless of the cursor's position. If not selected, searches are stopped when the end (forward searches) or start (backward searches) of the file is reached.

Filter mode enables you to filter information in the [Objects Window](#), [Processes Window](#) and [Structure Window](#):

By entering a string in the Contains field, you can filter the view of the selected window down to the specific information for which you searching.

Table 3-6: Graphic Elements of Search Bar in Filter Mode

Button	Name	Description
	Filter Regular Expression	Sets the wildcard mode: Glob Style, Regular Expression, or Exact Match. See Wildcard Modes when Searching .
	Clear Filter	Clears the field and removes the filter from the active window

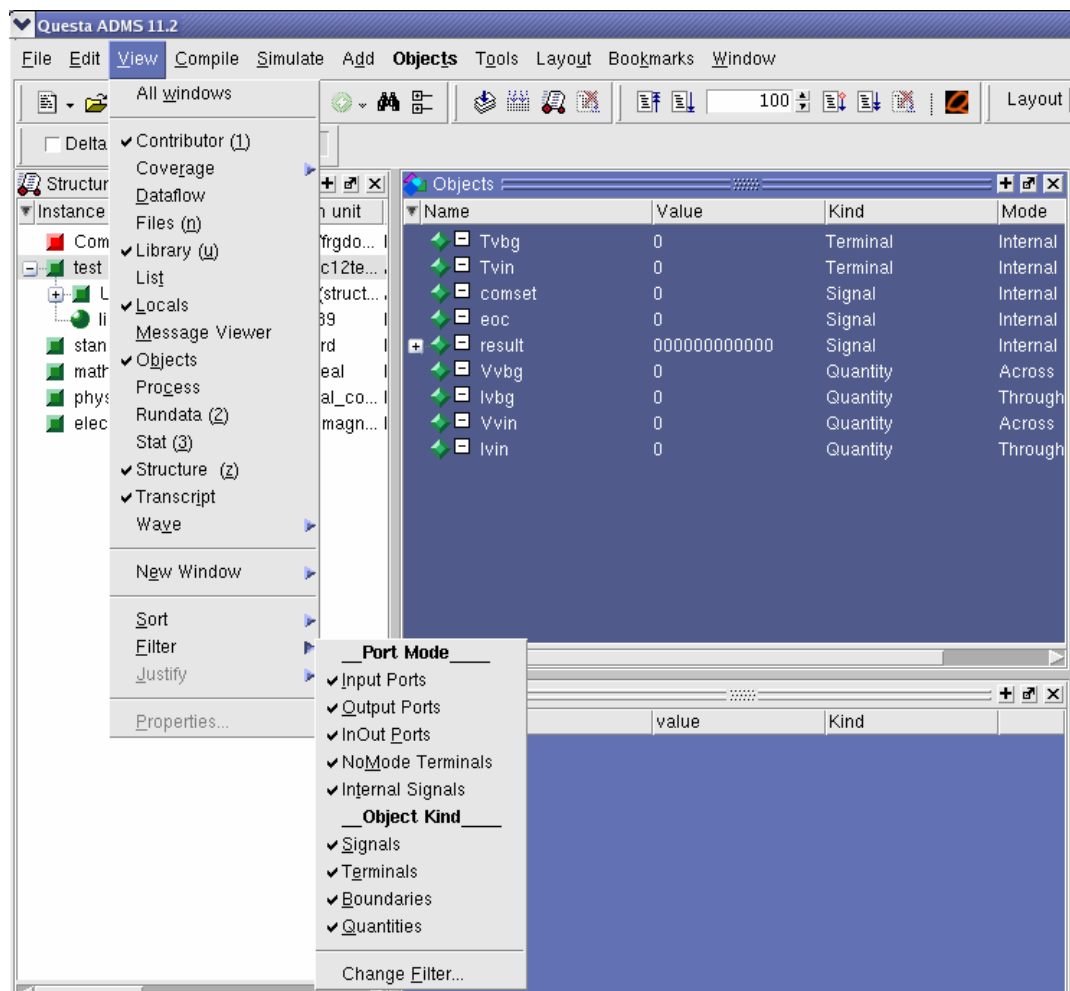
The Contains field is case-insensitive; if you need to search for case-sensitive strings use a regular expression and prepend the string with (?c).

Filtering and Sorting Objects in the Objects Window

Analog or mixed-signal objects in the Objects window can be filtered by type.

With the [Objects Window](#) selected, check or uncheck the options under the **View > Filter >** menu to control what is visible.

Figure 3-11: Filtering Objects in the Objects Window



You can also use **View > Sort >** to sort analog or mixed-signal objects in ascending or descending order.

Note
Digital objects cannot be filtered or sorted in the Objects window using these menu items.

Related Topics

[Wildcard Modes when Searching](#)

Wildcard Modes when Searching

A dropdown menu on the search bar enables you to set the wildcard mode.

- Glob Style
Enables you to use the following special wildcard characters:

- * - matches any sequence of characters in the string
- ? - matches any single character in the string
- [<chars>] - matches any character in the set <chars>
- \<x> - matches the single character <x>, which enables you to match on any special characters (*, ?, [,], and \)

For more information refer to the Tcl documentation: select **Help > Tcl Man Pages** then navigate to **Tcl Commands > string > string match**.

- Regular Expression

This wildcard mode enables you to use wildcard characters based on Tcl regular expressions.

- Exact Match

No characters have a special meaning; in other words, wildcard features are disabled.

The Find Options menu displays the search options available to you, and hot keys if available.

Related Topics

Find and Filter Functions

Filtering Objects by Type in the Structure Window

You can specify which type of objects to display in the Structure window.

Tip

You can further filter objects by name. Refer to [Find and Filter Functions](#) for more information.

Procedure

1. Right-click in the Structure window, then choose **Show > Change Filter**.

Alternatively, choose **View > Filter > Change Filter** or **Structure > Show > Change Filter** from the main window.

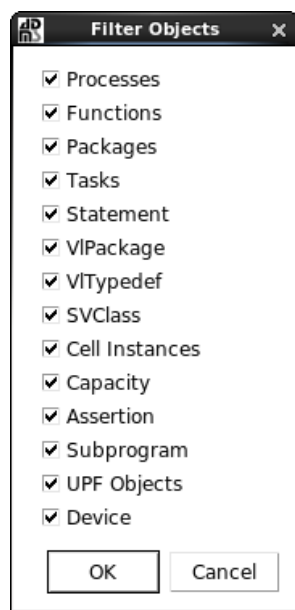
The Filter Objects dialog box opens. See [Figure 3-12](#).

2. Select the type of objects to display in the Structure window, then click **OK**.

Refer to Filtering Structure Window Objects in the *Questa SIM GUI Reference Manual* for information about the available object types. Objects of type Device are SPICE devices in SPICE subcircuits.

Examples

Figure 3-12: Filter Objects Dialog Box



Related Topics

Structure Window

Editing GUI Window Colors

You can change the colors of text and page elements.

Procedure

1. Choose **Tools > Edit Preferences**; the **Preferences Dialog Box** is displayed. Select the **By Window** tab.
2. Select a window category from the Window List column. The pane to the right changes to show the selected categories current color scheme. The editable text and page elements are listed with a sample of the selected window and a color palette from which new colors can be chosen. For example:
 - a. Select Objects Window from the Window List.
 - b. Select background or foreground from the Objects Window Color Scheme list.
 - c. Select a color from the palette.
3. Click **OK** to confirm, or **Apply** to choose another category to edit.

Related Topics

[Editing GUI Window Font Type and Size](#)
[Changing GUI Preferences](#)

Editing GUI Window Font Type and Size

You can change the font type and/or size of all GUI windows.

Procedure

1. Choose **Tools > Edit Preferences**; the **Preferences Dialog Box** is displayed. Select the **By Window** tab.
2. Select the font type to change from the list in the **Fonts** pane:
 - **fixedFont**
All text in the **Source Window** and Notepad display, and in all text entry fields or boxes
 - **footerFont**
All footer text that appears in footer of main window and all undocked windows
 - **menuFont**
All menu text
 - **textFont**
Transcript Window text and text in list boxes
 - **treeFont**
All text that appears in any window that displays a hierarchical tree

The current font is shown in the field next to the list.
3. Click **Choose** and select new font settings from the pop-up dialog box.

Related Topics

[Editing GUI Window Colors](#)
[Changing GUI Preferences](#)

Customizing the Simulator GUI Layout

You can customize the layout of windows and toolbars.

Questa ADMS has 2 default layouts that correspond to different modes of operation:

- NoDesign

A design is not yet loaded.

- Simulate

A design is loaded.

As you load and unload designs, Questa ADMS switches between the layouts. The default layouts can be modified, or you can create new custom GUI layouts, by selecting **Layout > Save Layout As**.

Specify a new name for the layout or use an existing name to overwrite that layout. The layout is saved to the `.modelsim` file.

Assigning Layouts to Modes..... 3-18

Customizing the Main Application Window Title..... 3-19

Assigning Layouts to Modes

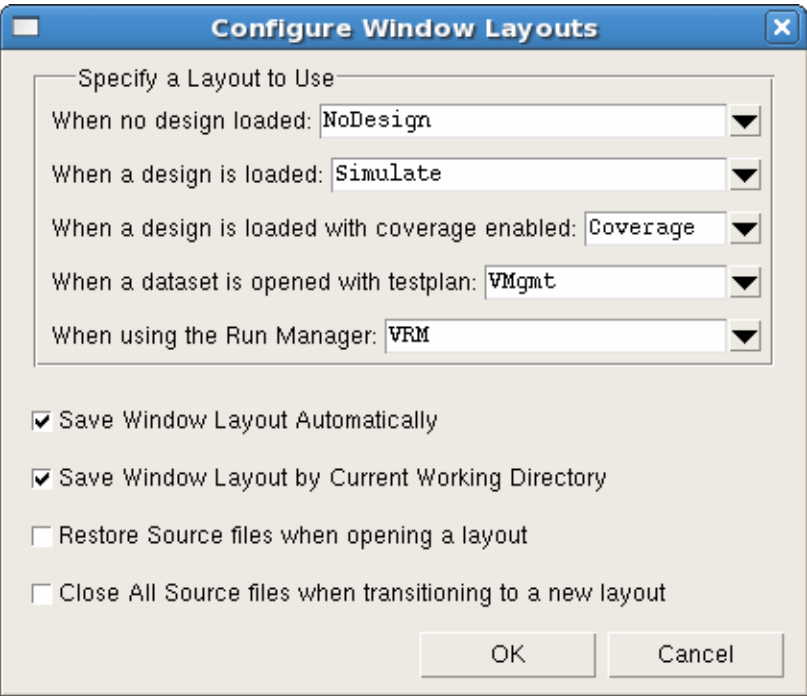
You can specify which layout appears in each mode.

This topic details how to specify which layout appears in each mode (no design loaded, design loaded).

Procedure

1. From the Questa ADMS main window, select **Layout > Configure**. The Configure Window Layouts dialog box is displayed:

Figure 3-13: Configure Window Layouts Dialog



2. Select a layout for each mode.
3. Click **OK**. The layout assignment is saved to the `.modelsim` file.

By default, any changes you make to a layout are saved automatically for the current design when you exit the tool or when you change modes. You can reset the layouts for all modes to their original defaults by selecting **Layout > Reset**. This command does not delete custom layouts.

Customizing the Main Application Window Title

You can add a title to the main application window using the `vasim -title` option.

This can be useful to identify which GUI belongs to which `run_script`.

Refer to [vasim](#) in *Questa ADMS Command Reference* for more information.

Related Topics

[Questa ADMS GUI Overview](#)
[Changing GUI Preferences](#)

Bookmarks

You can create bookmarks that allow you to return to a specific view or place in your design. You can save and automatically restore bookmarks. The windows that allow bookmarking include the Structure, Objects, Process, Source, and Files windows.

Refer to Bookmarks in the *Questa SIM GUI Reference Manual* for more information.

Save/Reuse of Questa ADMS GUI Configuration

You can enable an automatic save/reuse configuration mechanism. This mechanism saves the GUI configuration for the currently loaded design when the session is ended.

The saved configuration will be used the next time that design is loaded.

This behavior is enabled by setting the GUI preference variable `PrefReuse(GuiConfig)` to "yes":

```
set PrefReuse(GuiConfig) yes
```

The following items will be reused when the design is reloaded:

- Position and contents of all open windows.

- Commands run from the Transcript window are available using the up- and down-arrow keys on the keyboard or the history command.
- View of the hierarchical tree in the **Structure Window**.
- Configuration of the **Wave Window (Solido Waveform Analyzer)**; the waves will only be plotted after invoking a run command.

The save/reuse files *file_name.reuse* are written to the same directory as the output files (*.wdb,.dbg,.chi*). These files are only used when the GUI preference variable PrefReuse(GuiConfig) is set to either yes or ask. When the variable is set to no the save/reuse files are ignored and Questa ADMS will load the design as if it were loaded for the first time.

Related Topics

[Changing GUI Preferences](#)
[GUI Preference Variables](#)

4. Design Preparation

You can write and combine models into a design hierarchy, for simulation in Questa ADMS.

Working with SPICE Netlists.....	4-1
Working with VHDL and Verilog.....	4-21
Working with VHDL-AMS.....	4-31
Working with Verilog-AMS.....	4-43
Working With Verilog-A.....	4-69
Mapping Ports Using the Interface Matcher.....	4-71
Associating Design Units with vamatch.....	4-74
Associating Design Units Using the .BIND Command.....	4-76
Connecting Nets.....	4-78
Naming Conventions.....	4-79

Working with SPICE Netlists

The SPICE parts of a mixed-signal design are described within circuit netlist files, and are elaborated when the simulation session begins.

Declaring and Instantiating Language Descriptions.....	4-1
Declaring and Instantiating HDL Models.....	4-2
.ckt or .cir File.....	4-3
Defining Parameter or Generic Values.....	4-9
Instantiating VHDL/VHDL-AMS Entities with Composite Ports in SPICE.....	4-13
Attaching Ports of Composite Types in SPICE.....	4-14
Requesting Plots of Mixed-Signal Simulation Results in a SPICE Netlist.....	4-15
Instantiating a SPICE Subcircuit from VHDL, VHDL-AMS or Verilog.....	4-17
DSPF Parasitics on AMS Ports.....	4-18
Including a Spectre Netlist.....	4-18
Merging of Instances in Parallel.....	4-19
Instantiating Subcircuits From Multiple Libraries.....	4-20

Declaring and Instantiating Language Descriptions

To use a language description in a SPICE design, you must declare it using an extension of the standard .MODEL SPICE command. This declaration should be made in the top-level netlist or in the Questa ADMS Command File. You can then use the language description via an extension of the macromodel instantiation syntax in the Eldo language.

For example, the VHDL-AMS Entity/Architecture opamp(dominant_pole) is declared in the top-level SPICE configuration:

```
.MODEL opamp(dominant_pole) MACRO  
+ LANG=vhdlams LIB=admslib
```

The VHDL-AMS Entity/Architecture opamp(dominant_pole) is then instantiated in the top-level SPICE configuration:

```
Y1 opamp(dominant_pole) PORT: 4 0 2 40 70
```

Related Topics

[.MODEL](#)

[Y](#)

[.ckt or .cir File](#)

[Questa ADMS Command File](#)

Declaring and Instantiating HDL Models

Digital and mixed-signal models written in the supported hardware description languages may be instantiated in a SPICE description, using extended versions of the SPICE .MODEL command and macromodel instantiation syntax (Y).

When you use the Y syntax to instantiate a digital model from a SPICE subcircuit, you must ensure that the instance name is supported by Questa SIM. Refer to [Instance Names of Digital Models in SPICE Subcircuits](#) for more information.

Related Topics

[.MODEL](#)

[Y](#)

.ckt or .cir File

Circuit netlist files (.ckt or .cir) describe the SPICE parts of a mixed-signal design.

You can use SPICE netlist description statements inside the netlist to incorporate any analog device, source, or macromodel.

Format

Refer to [Netlist Structure](#) and [General Aspects of the Language Syntax](#) in *Eldo User's Manual*.

Parameters

Object Declarations

Statements declaring objects, for example:

- .MODEL
Used to provide device parameters, or to give the data to find a behavioral instance.
- .PARAM
Used to declare and give values to SPICE parameters.
- .SUBCKT
Used to declare a new subcircuit.

Device Instantiations

Statements that instantiate SPICE devices or HDL instances. A specific letter is used for a given device, for example:

- R
Used for a resistor.
- X
Used for a subcircuit.
- Y
Used for an HDL instance.

File Inclusion

Statements providing links to other files which contain parts of the description, for example:

- **.INCLUDE**
Inserts the contents of another file into an input netlist. Can only be used to include a SPICE file where the top level design is SPICE.
- **.LIB**
Inserts model or subcircuit definitions from a library file, into an input netlist.

Refer to [Supported Eldo Simulator Commands](#) in *Questa ADMS Command Reference for more information*.

Related Topics

.MODEL

Y

[Diode Models \[Eldo Reference Manual\]](#)

[BJT Models \[Eldo Reference Manual\]](#)

[Sources \[Eldo Reference Manual\]](#)

[Analog Macromodels \[Eldo Reference Manual\]](#)

[Switched Capacitor Macromodels \[Eldo Reference Manual\]](#)

[Magnetic Macromodels \[Eldo Reference Manual\]](#)

[JFET and MESFET Models \[Eldo Reference Manual\]](#)

[MOSFET Models \[Eldo Reference Manual\]](#)

[Digital Macromodels \[Eldo Reference Manual\]](#)

.MODEL

Declares a language description or HDL model for use in a SPICE design.

Note

This extended SPICE .MODEL command includes a component name separate from the behavioral model name and a provision for providing actual values for model parameters.

Syntax

Language Descriptions

```
.MODEL eldo_model_name macro lang=language
+      [ lib=logical_lib_name ]
+      [ mod=behavioral_model_name ]
+      [ generic|param ':'
+      generic_name=value
+      { generic_name=value } ]
```

HDL Models

```
.MODEL component_name macro lang=language
+      [ lib=logical_lib_name ]
+      [ mod=behavioral_model_name ]
+      [ generic|param ':'
+      generic_name=value
+      { generic_name=value } ]
```

Parameters

Language Descriptions

- eldo_model_name
Any legal Eldo component name, as for all other Eldo .MODEL cards. This name is independent of the module or entity name, which allows several models with the same language name being present in different libraries.
- lang=language
Language name of the corresponding HDL model. language can be vhdlams, verilogams, vhdl, or verilog.
- logical_lib_name
Specifies a library. If no library is specified, the default work design library will be used.
- generic | param

Used indiscriminately for specifying parameters. Generic or parameter values are associated by name only. Specify the parameter name as the name that appears in the behavioral description. Refer to [Y](#) for examples of allowed generic types.

For non-numerical values (different from real or integer), the value must be provided as a string (between “ ”) with the same format as if specified when instantiating the model. The values that are provided in the declaration of the model (using this syntax) may be overridden at the instantiation.

The standard Eldo syntax is also accepted:

```
.MODEL behavioral_model_name MACRO LANG=language [LIB=logical_lib_name]
```

HDL Models

- **component_name**
Any legal Eldo component. This name is independent of the module or entity name.
- **lang=language**
Language name of the corresponding HDL model. language can be: vhdlams, verillogams, vhdl or verillog.
- **lib=logical_lib_name**
Optional. Specifies the library in which the behavioral model is contained. If no library is specified, the default work design library will be used.
- **mod=behavioral_model_name**
Optional. If it is omitted, then component_name is used as the model name.
 - **behavioral_model_name**
The name of a VHDL or VHDL-AMS design entity: <entity> or <entity>(<architecture>); or the name of a VHDL-AMS configuration; or the name of a Verilog or Verilog-AMS module.
- **generic | param**
A list of named associations of parameters or generics of the HDL model to actual values. Refer to [Defining Parameter or Generic Values](#) for more information.

The standard Eldo syntax is also accepted:

```
.MODEL behavioral_model_name MACRO LANG=language [LIB=logical_lib_name]
```

Refer to [.MODEL](#) in *Eldo Reference Manual* for more information.

Related Topics

[Declaring and Instantiating Language Descriptions](#)
[Y](#)

Instantiates a language description or an HDL model inside a SPICE description.

Syntax

Language Descriptions

```
Yinstance name
+ GENERIC: param=value {param=value} ...
+ PORT: act_port_name {act_port_name} ...
```

HDL Models

```
Y<instance_name> model_name
+ [ generic|param ':' generic_name=value { generic_name=value } ]
+ port: actual_signal_name {actual_signal_name}
```

Parameters

Language Descriptions

- instance
User-defined name for the instance.
- name
Name of the design entity declared using [.MODEL](#).
- param=value
Mapping by name for the generics. The following are examples of allowed generic types.
 - Real values can follow the parameter name, for example:

```
Yinst name GENERIC: g=0.0
```

- For other scalar types, values have to be placed between " ":

```
Yinst name GENERIC: g=" 0"
Yinst name GENERIC: g=" TRUE"
Yinst name GENERIC: g=" 10 ns"
```


- String notation is allowed for the predefined types `String`, `Bit_vector`, `Std_logic_vector`, and `Std_ulogic_vector`:

```
Yinst name GENERIC: g=" 0100"
```

- Literal values for elements of vectors:

```
Yinst name GENERIC: g=" (1.0, 2.0, 3.0)"
```

- `act_port_name`
Positional mapping for the ports.

HDL Models

- `Y<instance_name>`
The Eldo instance name of the behavioral instantiated model (including the Y letter).
- `model_name`
The HDL model name, as defined in the [.MODEL](#) declaration.
- `generic_name=value`
A list of named associations of parameters or generics of the HDL model to actual values. A value supplied here will override a value supplied for the same parameter or generic on the [.MODEL](#) declaration for the HDL model. Refer to [Defining Parameter or Generic Values](#) for more information.
- `actual_signal_name`
Maps actual signals to the ports of the behavioral model by positional association. Named association is not supported. If the port is a vector type then the actual signal name is replaced by a list of actual signals enclosed in single quotes.

Related Topics

[Declaring and Instantiating Language Descriptions](#)
[Instance Names of Digital Models in SPICE Subcircuits](#)

Defining Parameter or Generic Values

You can specify values for generics or parameters of an HDL model using the `.MODEL` statement or in the component instantiation (or both).

If the parameter is integer or real valued then the parameter value is simply a number. However, parameters in HDL models sometimes have composite types. This topic describes how to format values for parameters of the following types:

- [Time and Enumeration Type Values](#)

- [Values For VHDL/VHDL-AMS Vectors of Scalar or Complex Types](#)
- [Values For Verilog Vectors of Scalar Types](#)
- [Values For VHDL/VHDL-AMS Vectors of Complex Types](#)

Time and Enumeration Type Values

For scalar types other than type Real, the Verilog/Verilog-A or VHDL/VHDL-AMS value must be enclosed within quotation marks "".

For VHDL/VHDL-AMS predefined scalar types:

```
Y<instance_name> component_name
+ GENERIC: g="'0'"           ! Bit
+ GENERIC: g="TRUE"          ! Boolean
+ GENERIC: g="'a'"           ! Character
+ GENERIC: g="0"              ! Integer
+ GENERIC: g=0.0              ! Real
+ GENERIC: g="10ns"          ! Time
```

For Verilog/Verilog-A predefined scalar types:

```
Y<instance_name> component_name
+ GENERIC: g="0"              ! Integer
+ GENERIC: g=0.0              ! Real
```

VHDL/VHDL-AMS user-defined scalar types

- VHDL-AMS Enumeration type declaration:

```
type Light_color is (red, yellow, green)
```

In the SPICE description:

```
Y<instance_name> model_name
+ GENERIC: g="green"          ! Light_color
```

- VHDL-AMS based universal_integer type :

```
type My_int is range 0 to Integer'High
```

In the SPICE description:

```
Y<instance_name> model_name
+ GENERIC: g="3"           ! My_int
```

- VHDL-AMS based universal_real type :

```
type My_real is range 0.0 to Real'High
```

In the SPICE description:

```
Y<instance_name> model_name
+ GENERIC: g="3.0"         ! My_real
```

Values For VHDL/VHDL-AMS Vectors of Scalar or Complex Types

Composite elements are restricted to complex (Records of two Real values) vectors (one-dimensional arrays) of scalar and complex values.

For composite types based on scalar types (vectors of scalar elements and COMPLEX type), a simplified aggregate notation will be used:

- It is a positional aggregate, with a possible OTHERS clause at the end of the aggregate. If the OTHERS clause is used, it means that the size of the formal generic is known (for example a record object or an array with a dimension).
- Elements of the aggregate are literal values (no possible expressions)
- The string notation is allowed for the following predefined types:
 - String
 - Bit_vector
 - Std_logic_vector
 - Std_ulogic_vector

Example

- For VHDL-AMS Vector type s:

```
type Real_vector is array (natural range <>) of Real;
type String is array (natural range <>) of Character;
type Bit_vector is array (natural range <>) of Bit;
type Complex is record
    Re, Im : Real;
end record;
```

- In the SPICE description
 - For type Real_vector:

```
Y<instance_name> model_name
+ GENERIC: g="(1.0, 2.0, 3.0, 0.0)" ! Real_vector
```

For type Real_vector(1 to 4):

```
Y<instance_name> model_name
+ GENERIC: g="(1.0, others => 0.0)" ! Real_vector(1 to 4)
```

- For type String:

```
Y<instance_name> model_name
+ GENERIC: g="('1', 'x', 'X')" ! String
```

or:

```
Y<instance_name> model_name
+ GENERIC: g="123" ! String
```

- For type Bit-vector:

```
Y<instance_name> model_name
+ GENERIC: g="01010101" ! Bit_vector
```

- For type Bit-vector(1 to 1):

```
Y<instance_name> model_name
+ GENERIC: g="(others => '0')" ! Bit_vector(1 to 1)
```

- For type Complex:

```
Y<instance_name> model_name
+ GENERIC: g="(others => 0.0)" ! Complex
```

or:

```
Y<instance_name> model_name
+ GENERIC: g="(0.0, 1.0)" ! Complex
```

Values For Verilog Vectors of Scalar Types

In the SPICE description, for type String:

```
Y<instance_name> model_name
+ GENERIC: g="M123" ! String
```

Note

"Aggregate notation" for string cannot be used for Verilog.

Values For VHDL/VHDL-AMS Vectors of Complex Types

In this case, it is an aggregate of aggregates, for example:

```
Y<instance_name> model_name
+ GENERIC: g="(others => (others => 0.0))"
                                     ! Complex_vector(1 to 3)
Y<instance_name> model_name
+ GENERIC: g="((others => 0.0), (others => 0.0))"
                                     ! Complex_vector(1 to 2)
Y<instance_name> model_name
+ GENERIC: g="((1.0, 0.0), (0.0, 1.0))"
                                     ! Complex_vector
Y<instance_name> model_name
+ GENERIC: g="((1.0, 0.0), others => (0.0, 1.0))"
                                     ! Complex_vector(1 to 3)
```

Related Topics

[Attaching Ports of Composite Types in SPICE](#)
[Types and Natures](#)

Instantiating VHDL/VHDL-AMS Entities with Composite Ports in SPICE

VHDL/VHDL-AMS entities can be associated with composite ports in SPICE.

Association can be made in the following ways:

- Association of element-by-element, or "one-for-one". One SPICE node is associated for each field of the record or element of the array.
- Association "as-a-whole". A composite will be associated as a whole, rather than element-by-element if the SPICE node used with a composite signal port appears in a .DECLARE COMPOSITE command.
- Resolved composites (arrays or records)

- Questa ADMS supports resolved composite signals. The resolution function of a resolved signal is shared between VHDL-AMS and VHDL.
- A SPICE node associated with a resolved composite signal must appear in a `.DECLARE COMPOSITE` command, that is, only association “as-a-whole” is allowed.

Refer to [.DECLARE COMPOSITE](#) in *Questa ADMS Command Reference* for more information.

Composite Boundaries and Boundary Elements

- One boundary element is inserted for each record field or array element.
“One-for-one” is the default mode for `vacom –target_entity`, or for `vacom` with the “component” form of digital interface definition.
- One boundary element for the entire composite “as-a-whole”.
- This requires an `.assoc` file, created by `vaspi –interactive`.

Related Topics

[.assoc File](#)

Attaching Ports of Composite Types in SPICE

Composite elements are restricted to Records and Vectors (one-dimensional arrays) of scalar elements.

Vectors of Scalar Elements

To assemble SPICE scalar elements into a composite VHDL-AMS element, use the following syntax:

```
yXXX e(a) port: a (b) (c d) (e f g h i j) k
```

In this instantiation, a possible declaration of `e` is:

```
entity e is
  generic(...);
  port(a : Real;
        b : Bit_vector;
        c : String;
        d : Bit_vector;
        e : Bit);
end;
```

Here, `(b)` represents a `Bit_vector` of one Bit (from 0 to 0), `(c d)` represents a String of two Characters (from 0 to 1) and `(e f g h i j)` represents a `Bit_vector` of six Bits (from 0 to 5).

Complex Vectors

The following statement is used in the SPICE description to associate a complex vector port:

```
yXXX e(a) port: ((a b) (c d))          ! connected to formal f
```

This is equivalent to:

```
yXXX : use entity e(a)
      port map(f(1) => (Re => a, Im => b),
               f(2) => (Re => c, Im => d));
```

Related Topics

[Requesting Plots of Mixed-Signal Simulation Results in a SPICE Netlist
Types and Natures](#)

Requesting Plots of Mixed-Signal Simulation Results in a SPICE Netlist

You can use the .PLOT and .PROBE commands inside a .cir or .cmd file to plot specific results from analog or mixed-signal simulations.

They have the same effect as the [add wave](#) and [add log](#) commands respectively.

Use the following key-letters to specify the type of mixed-signal object:

- V

A reference quantity associated to a VHDL-AMS terminal or Verilog-AMS node. For example, to plot the reference of the terminal, t:

```
.PLOT TRAN V(t)
```

To plot the reference of the terminal, t, AC phase:

```
.PLOT AC Vp(t)
```

- S

A VHDL-AMS analog quantity or a Verilog-AMS branch quantity. For example, to plot AC magnitude in dB of the quantity, q:

```
.PLOT AC Sdb(q)
```

Only analog quantities are plotted, no digital objects are plotted.

Additionally, to plot internal states for the macromodel x1.y1:

```
.PLOT TRAN S(x1.y1->*)
```

- SG

A VHDL, VHDL-AMS, Verilog or Verilog-AMS signal. Plotted behavioral nets may reference devices and nodes inside subcircuits. For example, to plot the signal of node n2 in subcircuit Xnot1:

```
.PLOT TRAN SG(Xnot1.n2)
```

Tip

You can also plot variables from a netlist using `.PLOT TRAN VAR()`

To plot currents, use `.PLOT I()`. Plots of a Y, X, or an Eldo device with port names are specified in the same way as for `.PLOT V()`.

The `.PLOT` and `.PROBE` commands plot the specified object if it is a terminal, a signal, a quantity, a branch or a variable. However, non-terminal objects (signals, quantities, branches or variables) are not plotted when the wildcard character '*' is used. This is to avoid adding large numbers of unwanted waveforms to the Solido Waveform Analyzer database.

You can use `.OPTION LIMPROBE` to limit the number of waveforms in the Solido Waveform Analyzer database. Refer to [limprobe](#) in *Questa ADMS Command Reference* for more information.

To plot signals and terminals, you must specify the command `.PLOT SG(regex)` in addition to `.PLOT V(regex)`.

Refer to [.PLOT](#) in *Eldo Reference Manual* for more information.

Hierarchical Object Names

The `.PLOT` and `.PROBE` commands used inside a `.cir` or `.cmd` file have the same effect as the [add wave](#) and [add log](#) commands respectively. The following syntactical differences exist when using `.PLOT` or `.PROBE` as opposed to `add wave` or `add log`:

- You must omit the first level of the hierarchical name as this is the name of your command file (or `.cir` file if SPICE is on top)
- You must use "." rather than ":" as the hierarchical separator.

Example

The following add wave command plots the net exp_on:

```
add wave :line_scan_camera_eldonet:y_exposure1:exp_on
```

The equivalent .PLOT command is:

```
.PLOT TRAN SG(y_exposure1.exp_on)
```

Tip

When a net is added to the Wave window, the add wave command including the full net name is written to the Transcript window.

Related Topics

[add wave \[Questa ADMS Command Reference\]](#)

[add log \[Questa ADMS Command Reference\]](#)

[Obtaining a Hierarchical Path Name](#)

Instantiating a SPICE Subcircuit from VHDL, VHDL-AMS or Verilog

The `vaspi` command prepares a SPICE subcircuit for instantiation from a digital context.

It creates an association between a SPICE subcircuit and a digital model (that describes the same behavior at the digital level).

To make associations, use the following syntax:

```
vaspi [<options>] <digital_unit> <subckt_name>[@<file_name>]
```

Refer to [vaspi](#) in *Questa ADMS Command Reference* for more information.

An instantiated SPICE subcircuit can be hierarchical, but any [.LIB](#) or [.SUBCKT](#) lib commands used to instantiate lower-level subcircuits must be included in the `.cmd` file or in the body of the SPICE subcircuit. All other SPICE commands and option statements (`.TRAN`, `.MODEL`, `.OPTIONS`, `.INCLUDE`, and so on) must be placed in the command file (`.cmd`).

Note that when a node is named VDD, VSS, VEE, VCC, or GND, and this node is not connected to a power supply, a warning message will be reported.

Related Topics

[VHDL-AMS Instantiating SPICE](#)

DSPF Parasitics on AMS Ports

Questa ADMS supports the import of Detailed Standard Parasitic Format (DSPF) files. DSPF files contain extracted parasitic information for SPICE nodes and analog nodes that are ports of VHDL-AMS and Verilog-AMS units.

You can perform backannotation from a level 0 DSPF file generated by Calibre. Refer to [DSPF Backannotation](#) in *Eldo User's Manual* for more information.

Note

For Questa ADMS-Premier and Questa ADMS-ADiT, the .DSPF_INCLUDE option must be set to DEV=MIXED. Refer to [.DSPF_INCLUDE](#) in *Eldo Reference Manual* for more information.

Including a Spectre Netlist

You can specify a Spectre netlist to provide a subcircuit definition.

You can use a Spectre netlist file to provide a subcircuit definition for instantiation in a VHDL-AMS entity. Use the `vacom` command with the `-spectre` argument to indicate that the subcircuit definition provided with `-spice` is a Spectre file:

```
vacom [-spectre <subckt_name>[@<spectrefile>]] | [-spice <subckt_name>
[@<spiceectrefile>]]
```

You can use a Spectre netlist file to provide a subcircuit definition for instantiation in a Verilog-AMS entity. Use the `vaspi` command with the `-spectre` argument to indicate that the subcircuit definition given is a Spectre netlist.

```
vaspi [-spectre] <subckt_name>[@<spectrefile>] | [-subckt_file
<spectrefile>]
```

You can use a Spectre netlist file to provide a subcircuit definition when specifying a model name on the `vamatch` command:

```
vamatch [<model_name_1>] [<model_name_2>]
```

Where models 1 or 2 can be:

```
-hdl <[library.]du> |
-spice <subcktname[@spicefile]> |
-spectre <subcktname[@spectrefile]>
```

An option on the Interface Matcher selection window, enables you to select instances from a Spectre netlist.

Limitations

In the Structure window, devices names will be displayed with the translated (eldo) format.

The waves window displays the original (Spectre) format for current plots while the voltage plots have the translated (eldo) format.

Merging of Instances in Parallel

By default, analog instances connected in parallel are merged into a single instance. The parallel instances must have the same set of parameters, and follow each other in the netlist.

In the following example, resistors r1, r2, r3, r4, r5 and r6 are merged into a single resistor instance. instances x1, x2, x3 and x4 of the one-level-hierarchy subcircuit are also merged into a single subcircuit instance:

```
v1 1 0 pwl 1n 10 2n 2 5n 5 10n 20
r1 1 2 1
r2 1 2 1
r3 1 2 1
r4 1 2 1
r5 1 2 1
r6 1 2 1
r7 2 0 2
x1 2 0 foo
x2 2 0 foo
x3 2 0 foo
x4 2 0 foo

.subckt foo a b
r1 a b 1
.ends

.tran 0.1n 20n
.plot tran i(r3)
.end
```

A warning message is reported when merging has occurred, and the number of devices merged is reported in the `.chi` file.

Note

HDL models and subcircuits containing HDL model instances are not merged unless `.OPTION CATMX` is set in the netlist.

The default behavior can be disabled with the SPICE command `.OPTION NOCATMX`. This command cancels the effect of any `.OPTION CATMX` command that precedes it in the command file.

Refer to [Merging Instances in Parallel](#) in *Eldo User's Manual* for more information.

Note

Merging is disabled for a [Monte Carlo Analysis](#) or any analysis that includes a `.DSPF_INCLUDE` command.

Instantiating Subcircuits From Multiple Libraries

You can use the SPICE `.SCOPE` command to support the instantiation of multiple `.MODEL/.SUBCKT` definitions, from different libraries. You can use a key to associate a different library with a set of instances.

Refer to `.SCOPE` in *Eldo Reference Manual* for more information.

Examples

- In this example, `mylib1` and `mylib2` both contain a definition of subcircuit "foo". Each library is given a different key, which will be used to select the version of foo during instantiation:

```
.lib key=dk1 mylib1 foo
.lib key=dk2 mylib2 foo
```

The following instantiation statements refer to different versions of the subcircuit foo, as the `.SCOPE` commands associate each instance with a different library key:

```
X1 a b foo
X2 a b foo

.scope key=dk1 inst=X1
.scope key=dk2 inst=X2
```

- Use .SCOPE to specify the same library key for multiple instances:

```
.scope key=dk3 inst=(x3,x4)
```

Working with VHDL and Verilog

Special considerations are needed when using VHDL in Questa ADMS.

Mapping Verilog Vectors to VHDL.....4-21

Rules for Instantiating VHDL-AMS in VHDL Models.....4-22

VHDL and VHDL-AMS Configuration Declarations.....4-24

Bus Connection Examples.....4-26

Access to Simulation Parameter Values.....4-28

Verilog Instantiating VHDL-AMS.....4-29

Simulator Resolution Limit (Verilog).....4-30

Mapping Verilog Vectors to VHDL

Verilog vectors cannot be mapped directly to the VHDL type Std_ulogic_vector when encapsulating Verilog vectors in HDL models.

Instead of associating the Std_logic_vector port directly with the VHDL-AMS Std_ulogic_vector, you must use an intermediate Std_logic_vector signal declaration and map the Verilog port to it before assigning it to the Std_ulogic_vector port using a type conversion:

```
Std_ulogic_vector_port <=
Std_ulogic_vector(Std_logic_vector_intermediate_signal);
```

This is done using a signal assignment, because it is not possible to have a function call or a type conversion within a port association in VHDL-AMS. Alternatively, you could change all occurrences of Std_ulogic_vector to Std_logic_vector within the VHDL-AMS wrapper.

Rules for Instantiating VHDL-AMS in VHDL Models

VHDL-AMS instances must follow certain rules in order for them to be instantiated in VHDL models.

This topic describes the types of generic parameters allowed in the instantiated interface. There is no constraint on the architecture.

In this topic, VHDL-AMS syntax shown in

red

is not supported in Questa ADMS. Syntax shown in

amber

is supported with limitations.

GENERIC Interface List

In general, any physical or enumerated types are supported. For other types, if you are using a type allowed in Questa ADMS but not in the foreign interface rule, the default value is used. No error message will be reported.

```
interface_constant_declaration ::=
    [ $ 4.3.2 ]
    [ constant ] identifier_list : [ in ] subtype_indication
    [ := static_expression ]
```

Limitations

subtype_indication can be any of the following types:

- In the STD library:
 - STD.STANDARD.BOOLEAN
 - STD.STANDARD.BIT
 - STD.STANDARD.BIT_VECTOR
 - STD.STANDARD.CHARACTER
 - STD.STANDARD.STRING
 - STD.STANDARD.INTEGER
 - STD.STANDARD.REAL

- STD.STANDARD.REAL_VECTOR
- STD.STANDARD.TIME
- In the IEEE library
 - IEEE.STD_LOGIC_1164.STD_LOGIC
 - IEEE.STD_LOGIC_1164.STD_LOGIC_VECTOR

For the static_expression of the default value of the generic, no functions are allowed.

PORT Interface List

In general any physical or enumerated types are supported. For other types, if you are using a type allowed in Questa ADMS but not in the foreign interface rule, no error message will be reported, and the default value is used.

```
interface_signal_declaration ::=
    [§ 4.3.2]
    [ signal ] identifier_list : [ mode] subtype_indication [ bus ]
    [ := static_expression ]
    mode ::= in | out | inout | buffer | linkage
```

Limitations

On the PORT interface list, the supported modes are IN, OUT, INOUT and BUFFER.

The kinds of port supported are non-resolved signals and resolved signals.

The subtype_indication can be any of the supported port data types:

- In the STD library:
 - STD.STANDARD.BOOLEAN
 - STD.STANDARD.BIT
 - STD.STANDARD.BIT_VECTOR
 - STD.STANDARD.CHARACTER
 - STD.STANDARD.STRING
 - STD.STANDARD.INTEGER
 - STD.STANDARD.REAL
 - STD.STANDARD.REAL_VECTOR
 - STD.STANDARD.TIME
- In the IEEE library:

- IEEE.STD_LOGIC_1164.STD_LOGIC
- IEEE.STD_LOGIC_1164.STD_LOGIC_VECTOR

Related Topics

VHDL and VHDL-AMS Configuration Declarations

VHDL and VHDL-AMS Configuration Declarations

Both VHDL `configuration_declarations` and VHDL-AMS `configuration_declarations` are supported in Questa ADMS. A VHDL configuration is compiled with `vcom`, and a VHDL-AMS configuration with `vacom`.

A `configuration_declaration` will be called simply a "configuration" in the following paragraphs. There are several rules governing the coding of configurations that must be followed in Questa ADMS.

The architectures named after for (that is, in a `block_configuration`) must all be VHDL compiled with `vcom` in a VHDL configuration, and they must all be VHDL-AMS compiled with `vacom` in a VHDL-AMS configuration. An architecture created with `vaspi` that encapsulates a SPICE subcircuit is a VHDL-AMS architecture.

A VHDL configuration can use a VHDL-AMS `entity_aspect`, and vice-versa. An `entity_aspect` appears after the reserved word `use` in a `component_configuration`. The `entity_aspect` chooses either an entity/architecture pair or a nested configuration. The entity/architecture or the configuration will be bound to the component instances specified by the `instantiation_list` and `component_name` immediately preceding the `use` statement. The `instantiation_list` is commonly `all`, meaning all instances of the named component.

If a VHDL configuration uses a VHDL-AMS `entity_aspect` or a VHDL-AMS configuration uses a VHDL `entity_aspect` then the following `block_configuration` is not allowed. This means that the configuration declaration cannot descend further into the design hierarchy. However, the effect of descending into the design hierarchy can be achieved by choosing a nested configuration for the `entity_aspect`.

A special syntax is required for the `entity_aspect` of a configuration when it specifies a nested configuration from the other language. You must use the reserved word `entity` in place of the reserved word `configuration`.

Syntax

The following shows a portion of the syntax for a configuration declaration (refer to the *VHDL Language Reference Manual* for the complete version). Syntax shown in

amber

is supported with limitations.

```
configuration_declaration ::=
    configuration identifier of entity_name is
        block_configuration
    end;

block_configuration ::=
    for architecture_name
        { component_configuration }
    end for ;

component_configuration ::=
    for instantiation_list : component_name
        [ use entity_aspect ; ]
        [ block_configuration ]
    end for ;

entity_aspect ::=
    entity entity_name [ ( architecture_identifier ) ]
    | configuration configuration_name
    | entity configuration_name
```

Limitations on Configuration Declarations

In addition to the limitations described in [Configuration Declarations](#), the following limitations also apply when instantiating VHDL-AMS configurations from VHDL:

- block_configuration is not allowed if the language of the library unit is different from the language of the enclosing configuration.
- entity configuration_name is used if the configuration_name specifies a configuration from the other language.

When a VHDL-AMS configuration is a sub-element of another configuration declaration, it is not possible to share it across the VHDL-AMS to VHDL boundary. To obtain the effect of a single configuration for a mixed-signal DUT that crosses the boundary, write two configurations and instantiate the second at the lowest level of the first.

Example

To instantiate the VHDL-AMS configuration C, use the following syntax in VHDL:

```
inst : entity C port map (...);
```

or

```
inst : entity C(config) port map (...);
```

Instantiating a VHDL configuration from a SPICE description does not require the use of the configuration keyword, so you can simply use:

```
.MODEL C macro lang=vhdlams
y1 C generic: ... port: ...
```

or

```
.MODEL C(config) macro lang=vhdlams
y1 C(config) generic: ... port: ...
```

Related Topics

[Support of Records in VHDL-AMS](#)

[vacom \[Questa ADMS Command Reference\]](#)

Bus Connection Examples

There is a particular syntax to connect a STD_LOGIC bus to SPICE.

The syntax to connect a STD_LOGIC bus to SPICE is illustrated in the following examples. In both cases, boundary elements are automatically inserted as required.

SPICE On Top

This example shows how to connect a STD_LOGIC bus WSELECT to SPICE, where SPICE is at the top-level:

- Excerpt from top level SPICE netlist:

```
Y_WALSHRC WALSHRC(RTL)
+ port : ( WSELECT[3] WSELECT[2] WSELECT[1] WSELECT[0] )
+ RESET_RX CORR3C WCLK CLKSEL WALSH_OUT
```

- Entity of lower-level WALSHRC module:

```
entity WALSHRC is
  port (
    wselect: in std_logic_vector(3 downto 0);
    reset: in std_logic;
    clkfast: in std_logic;
```

```

        wclock: in std_logic;
        clkselect: in std_logic;
        wout: out std_logic    );
end WALSHRC

```

VHDL On Top

This example shows how to connect a STD_LOGIC bus to SPICE where VHDL is at the top level instantiating a SPICE subcircuit:

- Excerpt from top level VHDL netlist:

```

architecture test of trx_tb is
    signal wselect: std_logic_vector(3 downto 0);
    signal Xclk, rstTX, rstRX, sync_frame, d_txip: std_logic;
    signal spread_data, d_rxop, error_occur: std_logic;
    signal txdataclk, rxdataclk, pll_clk: std_logic;
    signal d_rxopX, error_occurX, txdataclkX: std_logic;
    signal rxdataclkX, spread_dataX: std_logic;
    begin
        DUTtrx_le: entity trx_lib.trx_le_ent(trx_le_ckt)
            port map (d_rxop, d_txip, error_occur, pll_clk, rstRX,
                    rstTX, rxdataclk, spread_data, spread_data, sync_frame,
                    txdataclk, wselect, Xclk);
    end test;

```

- Excerpt from subcircuit:

```

.SUBCKT trx_le_ckt d_rxop d_txip error_occur pll_clk rstRX
+ rstTX rxdataclk spread_data_in spread_data_out sync_frame
+ txdataclk wselect[0] wselect[1] wselect[2] wselect[3] Xclk
* devices:
...
.ENDS

```

Related Topics

[Boundary Elements](#)

[Working with SPICE Netlists](#)

Access to Simulation Parameter Values

You can access the values of simulation parameters from within VHDL/VHDL-AMS descriptions and Verilog/Verilog-AMS descriptions.

VHDL/VHDL-AMS

The following function returns the value of the simulation parameter *<param_name>* in a VHDL or VHDL-AMS description:

```
simpam ("<param_name>"[, <expression>])
```

The simpam function returns *<expression>* if the simulation parameter has not been assigned a value.

Verilog/Verilog-AMS

The following system function returns the value of a real-valued or integer-valued simulation parameter *<param_name>* in a Verilog, Verilog-AMS, or System-Verilog description:

```
$simpam ("<param_name>"[, <expression>])
```

The \$simpam system function returns *<expression>* if the simulation parameter has not been assigned a value.

Refer to [Verilog-AMS Extensions](#) for a list of built-in simulation parameters.

The following system function returns the value of a string-valued simulation parameter *<param_name>* in a Verilog, Verilog-AMS, or System-Verilog description:

```
$simpam$str ("<param_name>")
```

The size of the return value of a \$simpam\$str call is defined at elaboration. If the value of the simulation parameter increases in length during simulation, the value returned by \$simpam\$str will be truncated.

Refer to section 9.15 in *Verilog-AMS Language Reference Manual Version 2.4* for more information about the \$simpam and \$simpam\$str system functions.

Related Topics

[Methods for Setting Simulation Parameters](#)

[Using Simulation Parameters to Restore Checkpoints](#)

[simpam get \[Questa ADMS Command Reference\]](#)

Verilog Instantiating VHDL-AMS

There is a particular syntax for instantiating a VHDL-AMS module in Verilog.

The syntax for instantiating a VHDL-AMS module in Verilog is:

```
\[lib_name.]entity(arch) [#(generic)] instance_name (port_mapping)
```

The backslash (\) defines that an extended name is defined. In this example the backslash is required because the architecture name is defined. If you specify only the entity name, the backslash is not required.

- lib_name
(Optional) The name of the library where the VHDL-AMS design unit has been compiled.
- entity(arch)
The VHDL-AMS entity and architecture names respectively.
- #(generic)
(Optional) Assigns values to the generics in the VHDL-AMS design unit, this can be done by name or by position.
- instance_name
The name of the instance.
- (port_mapping)
Defines the mapping of the VHDL-AMS ports to the Verilog ports, this can be done by name or by position.

Example

In this example, a VHDL-AMS design unit is instantiated twice inside a Verilog module. The first instantiation uses association by position, the second instantiation uses association by name.

The Verilog module is called TOP_VERILOG:

```
module TOP_VERILOG;
wire a1, b1;
wire a2, b2;
\\Association by position
\work.entity_vhdlams(arch) #(2) instposition (a1, b1);
\\Association by name
\work.entity_vhdlams(arch) #(.XX(2)) instname (.a(a2), .b(b2));
endmodule
```

The VHDL-AMS entity and architecture are called `entity_vhdlams` and `arch` respectively:

```
library IEEE;
use IEEE.ELECTRICAL_SYSTEMS.ALL;

entity entity_vhdlams is
generic (XX: integer := 1);
port (terminal a, b: electrical);
end entity_vhdlams;

architecture arch of entity_vhdlams is
constant XX2: integer:= 2* XX;
quantity qaca across qtha through a;
quantity qacb across qthb through b;
begin
    testing: process
    begin
        REPORT "EXPECTED VALUE 2 - GENERIC XX = " & integer'image(XX)
            SEVERITY note;
        REPORT "EXPECTED VALUE 4 - GENERIC XX2 = " & integer'image(XX2)
            SEVERITY note;
        wait;
    end process testing;
    qtha == qacb + 0.5;
    qaca - qthb == qtha;
end arch;
```

Related Topics

Working with VHDL-AMS

Simulator Resolution Limit (Verilog)

The simulator resolution limit defaults to the smallest time unit that you specify among all of the ``timescale` directives in the design.

A ``timescale` directive takes effect where it appears in a source file and applies to all source files specified with the same `valog` or `vlog` command. A design may have multiple ``timescale` directives; modules that are compiled separately may have different timescale directives.

Here is an example of a ``timescale` directive:

```
`timescale 1 ns / 100 ps
```

The first number is the time units; the second number is the time precision, which is the rounding factor for the specified time unit.

Example

Verilog modules Dig1 and Dig2 have the following `timescale directives:

```
module Dig1: `timescale 10ns / 100ps
module Dig2: `timescale 1ns / 10ps
```

The Verilog module Dig3 has no `timescale directive in effect.

Verilog-AMS modules Ana1 and Ana2 have the following `timescale directives:

```
module Ana1: `timescale 10ps / 10ps
module Ana2: `timescale 1ns / 1ps
```

The Verilog-AMS module Ana3 has no `timescale directive in effect.

The smallest defined resolution on the digital side is 10ps, and the smallest defined resolution on the analog side is 1ps.

Therefore, 1ps is applied to the modules that have no `timescale directive in effect (Dig3 and Ana3):

```
`timescale 1ps/1ps
```

Related Topics

[Questa SIM Simulation Control Variables](#)
[valog \[Questa ADMS Command Reference\]](#)

Working with VHDL-AMS

This section describe the necessary considerations when working with VHDL-AMS.

See also [VHDL-AMS Subset Definition](#) for details about language support limitations in Questa ADMS.

The default integration method used by Questa ADMS when resolving VHDL-AMS descriptions is trapezoidal (trap). To use the backward-Euler method instead, specify the following option in the *.cmd* or *.cir* file:

```
.OPTION BE
```

Refer to [Integration Methods](#) in *Eldo User's Manual* for more information.

VHDL-AMS Instantiating SPICE.....	4-32
VHDL-AMS Access to SPICE Parameter Values.....	4-36
Support of Records in VHDL-AMS.....	4-37
Composite Types in VHDL-AMS.....	4-37
Rules for VHDL-AMS Instantiating VHDL.....	4-38
Util Package for VHDL-AMS.....	4-41
Return Types of VHDL-AMS Functions.....	4-42

VHDL-AMS Instantiating SPICE

You can instantiate SPICE from a VHDL-AMS description.

There are two methods for instantiating SPICE from a VHDL-AMS description:

- Associate the SPICE subcircuit `subckt_name` to an entity (Ent) using the **vaspi** command. This method will enable the use of digital vectors in the entity.

```
vaspi Ent <subckt_name>[@<subckt_file_path_name>]
```

- Declare a VHDL-AMS component equivalent to the SPICE subcircuit and associate to it the three attributes. This method is only available from VHDL-AMS (the VHDL-AMS block can be instantiated from VHDL or Verilog). Using this method, digital vectors are not allowed in the interface between VHDL-AMS and SPICE.

VHDL-AMS Instantiating SPICE Using vaspi	4-32
VHDL-AMS Instantiating SPICE Using a Component Declaration.....	4-34

VHDL-AMS Instantiating SPICE Using **vaspi**

The **vaspi** command prepares a SPICE subcircuit for instantiation from a digital context. It creates an association between a SPICE subcircuit and a digital model (that describes the same behavior at the digital level).

The syntax of the **vaspi** command is as follows:

```
vaspi [<options>] <digital_unit> <subckt_name>[@<file_name>]
```

The following example demonstrates how to compile a VHDL-AMS-on-top design which instantiates a SPICE subcircuit.

Prerequisites

- The subcircuit declaration UA741 is contained in the netlist *child-spice.ckt*:

```
.SUBCKT UA741 2 3 6 4 7
* Pins:      2    Input (minus), 3    Input (plus)
*           6    Output
*           4    V- Power Supply, 7    V+ Power Supply
.MODEL NPN NPN BF = 160 RB=100 CJS=2P TF=0.3N ...
R1  1  4    1K
Q1  9  3 10 NPN
...
.ENDS
```

- The VHDL-AMS source file *ua741_adms.vhd* contains a VHDL-AMS entity called *ua741_adms*:

```
library ieee;
use ieee.electrical_systems.all;
entity ua741_adms is
  port (
    terminal t2, t3, t6, t4, t7 : electrical);
end ua741_adms;
```

The VHDL-AMS ports listed match the SPICE subcircuit pins in number and order. The GENERICS in the VHDL-AMS entity declaration are of type STD.REAL in order to map to SPICE subcircuit parameters.

Procedure

1. Compile the VHDL-AMS source file *ua741_adms.vhd* using the **vacom** command:

```
vacom ua741_adms.vhd
```

2. Use the **vaspi** command to create a SPICE architecture for the *ua741_adms* VHDL-AMS entity:

```
vaspi ua741_adms UA741@child-spice.ckt
```

3. Compile the top-level VHDL-AMS entity and architecture contained in *top-vhdlams.vhd*:

```
vacom top-vhdlams.vhd
```

The VHDL-AMS entity *ua741_adms* is now instantiated in this top-level VHDL-AMS entity and the new SPICE VHDL-AMS architecture can be accessed.

Results

Alternatively, you can use the `-interactive` argument to invoke the **Interface Matcher**:

```
vaspi -interactive [<options>]
[<digital_unit> <subckt_name>[@<file_name>]]
```

As a result of this process, an interface association file `<digital_unit>-<subckt_name>.assoc` is created, which is used at elaboration time to build the hierarchy. It lists the mapping of every pin of the SPICE unit with the corresponding port of the digital unit.

Refer to **Mapping Ports Using the Interface Matcher** for more information.

Related Topics

Interface Matcher

Interface Association File (.assoc)

VHDL-AMS Subset Definition

VHDL-AMS Instantiating SPICE Using a Component Declaration

The component declaration method of instantiation maps a VHDL-AMS component to a SPICE subcircuit using three proprietary attributes.

The attributes are declared in the `MGC_AMS` library, Eldo package. The attributes have the following contents:

- `Eldo_device`
This is always the enumerated value `Eldo_subckt`.
- `Eldo_subckt_name`
The Eldo name of the subcircuit. If the subcircuit is declared within another subcircuit, the name *must* be the Eldo hierarchical name of the subcircuit. For example, `<subckt1_name>.<subckt2_name>`.
- `Eldo_file_name`
The name of the file where the subcircuit is described. It may be a relative or absolute UNIX pathname. The name may contain environment variables. For example, `$eldopath/circuits.ckt`.

Restrictions and Limitations

- The attributes `Eldo_device`, `Eldo_subckt_name`, and `Eldo_file_name` cannot be used in the architecture name. If they are, an error message will be generated.

- When declaring a component, all SPICE parameters must correspond to VHDL-AMS generics of type STD.REAL.

Note

It is not mandatory to declare all the SPICE parameters in the VHDL-AMS component (as is the case for components in VHDL-AMS design entities).

- When declaring a component, all SPICE pins must correspond to VHDL-AMS port terminals of a scalar nature. The nature itself may be any kind of nature. For example: IEEE.ELECTRICAL_SYSTEMS.ELECTRICAL.
- When declaring a component, all the SPICE pins must be declared (there can be no open ports).

Prerequisites

- A subcircuit description is contained in the file *circuits.ckt*:

```
.SUBCKT stim inp outp p1=1
```

The subcircuit uses the following parameters:

- p1 is declared in the header of the subcircuit.
- p2 is declared somewhere within the visibility scope of this subcircuit; either within *circuits.ckt*, in the command file (.cmd) or in the top-most netlist file (.cir).

Procedure

1. To prepare the subcircuit for instantiation, declare a component within a VHDL-AMS architecture:

```
library ieee;
use ieee.electrical_systems.electrical;
-- for the proprietary attributes
library MGC_AMS;
use MGC_AMS.ELDO.all;

architecture ... of ... is
  component ELDO_stim is
    generic (p1 : Real; p2 : Real);
    port (terminal in1, in2 : Electrical);
  end component ELDO_stim;

  attribute Eldo_device
    of ELDO_stim : component is Eldo_subckt;
  attribute Eldo_subckt_name
    of ELDO_stim : component is "stim";
  attribute Eldo_file_name
    of ELDO_stim : component is "circuits.ckt";
  ...
end architecture ...;
```

2. Instantiate the subcircuit (encapsulated into a VHDL-AMS component) in the same way as other components in a VHDL-AMS description.

Related Topics

[VHDL-AMS Access to SPICE Parameter Values](#)

[VHDL-AMS Subset Definition](#)

VHDL-AMS Access to SPICE Parameter Values

The VHDL-AMS package ELDO_PARAMETERS enables SPICE parameters defined with the .PARAM command to be used within a VHDL-AMS description.

The following example accesses the parameter R that is defined in the top-level SPICE netlist from within the VHDL-AMS description:

```
entity resistance is
    port(a, b Electrical);
end entity resistance;
library MGC_AMS;
use MGC_AMS.eldo_parameters.all;
architecture spice_param of resistance is
    quantity Vab across Iab through a to b;
    constant R : Real := param("R");
    -- R has been defined in the top ELDO cmd or cir file
    begin
        Iab == Vab / R;
    end architecture spice_param;
```

Only numerical values are supported by this package. String values cannot be referenced, for example, the following parameters are illegal:

```
.param p4="alma"
.param p5(a,b)=sin(a+b)
```

If an illegal SPICE parameter is used, or the referenced SPICE parameter is not declared in the command file, the following error message will be returned:

```
No eldo parameter called 'p4' exists in $param system function call or its
value is not a scalar numeric value.
```

Any parameter declared in a .PARAM command in the SPICE context can be swept using either the .STEP or the .DATA command. Each swept value is available in successive runs and nested sweeps are allowed. See [Multiple-Run Analyses](#).

Support of Records in VHDL-AMS

Questa ADMS supports record objects within VHDL-AMS and at the VHDL/VHDL-AMS boundary.

Constants and generic constants, variables and signals can be of a record type, although the following restrictions apply to the types of the field of a record in VHDL-AMS:

- Elements of records can be scalars, arrays or records, however, the elements of a nested array cannot themselves be records.
- A record type may contain a maximum of 4095 elements.
- A record type cannot contain an array of records.
- A record type cannot contain globally static elements (locally static elements are allowed).
- Resolved records may only be associated as a whole. Partial sub-element associations are not accepted.
- Record nature declarations are not supported.
- In addition, when a record is shared across the VHDL/VHDL-AMS boundary, the following rules apply:
 - A record may be of resolved or unresolved record type.
 - The array of record type is allowed, but the records cannot be of a resolved subtype.
 - The record type or subtype for generic association at the Questa SIM interface is not supported.

Composite Types in VHDL-AMS

Composite types are aggregates of other types.

In Questa ADMS, composite types with up to three levels are supported:

- A record of arrays
- An array of records
- A record of records
- An array of arrays

The lowest level in all cases is an element of a scalar type. The elements of a resolved composite type must be scalar.

Related Topics

Types and Natures

Rules for VHDL-AMS Instantiating VHDL

When `configuration_declaration`, `package_declaration`, and `package_body` are defined in VHDL, they can be used in VHDL-AMS.

This section describes the restrictions on the interface when instantiating VHDL design units from VHDL-AMS.

Design Unit

```
primary_unit ::=                                [§ 11.1]
    entity_declaration
  | configuration_declaration
  | package_declaration

secondary_unit ::=                             [§ 11.1]
    architecture_body
  | package_body
```

Instantiating a VHDL configuration in VHDL-AMS requires a different configuration notation syntax. The configuration is seen as a design entity called C(config) where C is the name of the configuration.

```
instantiated_unit ::=                           [§ 9.6]
    [ component ] component_name
  | entity entity_name [ ( architecture_identifier ) ]
  | configuration configuration_name
```

The packages used in both simulators must comply with the rules described in the [VHDL-AMS Subset Definition](#).

GENERIC Interface List

This topic describes the types of generic parameters allowed in the instantiated interface.

```
interface_constant_declaration ::=              [§ 4.3.2]
    [ constant ] identifier_list :
    [ in ] subtype_indication [ := static_expression ]
```

The `subtype_indication` can be any of the following generic parameter types:

- STD.STANDARD.BOOLEAN
- STD.STANDARD.BIT
- STD.STANDARD.BIT_VECTOR

- STD.STANDARD.CHARACTER
- STD.STANDARD.STRING
- STD.STANDARD.INTEGER
- STD.STANDARD.REAL
- STD.STANDARD.REAL_VECTOR
- STD.STANDARD.TIME
- IEEE.STD_LOGIC_1164.STD_LOGIC
- IEEE.STD_LOGIC_1164.STD_ULOGIC
- IEEE.STD_LOGIC_1164.STD_LOGIC_VECTOR
- IEEE.STD_LOGIC_1164.STD_ULOGIC_VECTOR

In general, any enumerated type and any physical type are supported. For other types, according to the basic VHDL-AMS subset, the default value will always be used. It is not possible to change the default value at the instantiation from Questa ADMS.

For the static_expression of the default value of the generic, no functions are allowed.

PORT Interface List

```
interface_signal_declaration ::=                                [§ 4.3.2]
  [ signal ] identifier_list : [ mode ]
    subtype_indication [ bus ] [ := static_expression ]
mode ::= in | out | inout | buffer | linkage                [§ 4.3.2]
```

On the PORT interface list three aspects exist:

1. The mode of the PORT

The supported modes are IN, OUT, INOUT and BUFFER.

2. The kind of the PORT

Non-resolved or resolved signals are supported.

3. The type of the PORT

The following STD.STANDARD and basic IEEE. STD_LOGIC_1164 types are supported:

- STD.STANDARD.BOOLEAN
- STD.STANDARD.BIT
- STD.STANDARD.BIT_VECTOR
- STD.STANDARD.CHARACTER
- STD.STANDARD.STRING

- STD.STANDARD.INTEGER
- STD.STANDARD.REAL
- STD.STANDARD.REAL_VECTOR
- STD.STANDARD.TIME
- IEEE.STD_LOGIC_1164.STD_LOGIC
- IEEE.STD_LOGIC_1164.STD_ULONGIC
- IEEE.STD_LOGIC_1164.STD_LOGIC_VECTOR
- IEEE.STD_LOGIC_1164.STD_ULONGIC_VECTOR
- In general, any enumerated type and any physical type are supported.

GENERIC MAP Aspect and PORT MAP Aspect

Syntax in red is not supported for GENERIC MAP aspect and PORT MAP Aspect.

```
association_element ::=
  [formal_part =>] actual_part
formal_part ::=
  formal_designator
  | partial_function_name (formal_designator)
  | type_mark (formal_designator)
```

Port Map Aspect Example

- Design Unit

```
ENTITY sar IS
  PORT (clk, comset, datab: IN Bit;
        eoc: OUT Bit;
        q: OUT bit_vector(11 DOWNT0 0));
END ENTITY sar;

ARCHITECTURE modelsim OF sar IS
  ...
END ARCHITECTURE modelsim;
```

- Instantiation

```
ENTITY adc12 IS
  PORT (
    TERMINAL Tvb, Tvin: Electrical;
    SIGNAL comset: IN std_logic;
    SIGNAL eoc: OUT std_logic;
    SIGNAL result: OUT Bit_vector
      (11 DOWNT0 0));
END ENTITY adc12;
```



```

ARCHITECTURE structural_mixed_ms OF adc12 IS
...
SIGNAL clk1, datab: std_logic;
SIGNAL clk2, qbl: std_logic;
SIGNAL q:bit_vector(result'Range);
BEGIN
...
    x1: ENTITY sar(modelsim)
        PORT MAP (clk1, comset, datab,
                  eoc, q);
...

```

Related Topics

12-Bit Analog to Digital Converter

Util Package for VHDL-AMS

The “Util” package, included in Questa ADMS, contains a number of VHDL-AMS utilities.

The package is part of the *modelsim_lib* library, which is located in the Questa ADMS installation tree, and is mapped in the *modelsim.ini* file. To access the utilities subprograms in the package, you must add the following lines to VHDL-AMS code:

```

library modelsim_lib;
use modelsim_lib.util.all;

```

The utilities subprograms that can be accessed from the Util package are:

- `get_resolution`
Returns the current simulator resolution as a real number. For example, 1 femto second corresponds to 1.0×10^{-12} . By default the Questa ADMS simulator resolution is 1 picosecond. When the following command is run:

```
resval := get_resolution;
```

The value returned to `resval` will be 1.0×10^{-12} . The simulator resolution can be changed when Questa ADMS is invoked by specifying the argument `-t` on the **vasim** command; for example:

```
vasim ... -t [<multiplier>]<time_unit>
```

- `init_signal_spy`
Sets the value of the VHDL-AMS/VHDL signal or Verilog register/net (called the `src_object`) onto an existing VHDL-AMS/ VHDL signal or Verilog register/net (called the `dest_object`). This enables you to

reference signals, registers, or nets at any level of hierarchy from within a VHDL-AMS architecture (for example, a testbench).

- `init_terminal_short`

Sets the value of `terminal_1` to the value of `terminal_2` creating a short circuit. The terminals can be anywhere in the design (coming from either a VHDL-AMS terminal or a SPICE node). The two terminals will share a single KCL function, the sum of the through quantities is the result of the quantities on each of the terminals. The resulting voltage will be the reference on each of the terminals. This is equivalent to a `.connect` command in Eldo.

- `init_terminal_reference`

The procedure `init_terminal_reference()`, sets the value of the voltage at the destination terminal (`dest`), to the value of the voltage at the source terminal (`source`). If the source terminal is called `sT` and the destination terminal is called `dT`, then this is equivalent to the Eldo command:

```
Exx dT 0 sT 0
```

for a Voltage Controlled Voltage Source (VCVS).

- `init_terminal_contribution`

Sets the value of the voltage at the destination terminal to the value of the current (or contribution) of source terminal. If the source is not a terminal, the resulting value is always 0.0 and in this case, it has no meaning. If the source terminal is called `sT` and the destination terminal is called `dT`, then this is equivalent to the Eldo command:

```
Hxx dT 0 sT gain
```

for a Current Controlled Voltage Source (CCVS). The two terminals that are given as parameters of this procedure can be any terminal in the design (coming either from VHDL-AMS, SPICE or Verilog-AMS).

Related Topics

[init_signal_spy](#)

[init_terminal_short](#)

[init_terminal_reference](#)

[init_terminal_contribution](#)

[System Initialization File \(modelsim.ini\)](#)

Return Types of VHDL-AMS Functions

Various return types of VHDL-AMS functions are supported.

The return types of VHDL-AMS functions cannot be of type `line`. A function can return a scalar, a record or an array, but the elements of an array cannot be an array. The following return types are supported:

- A record of a scalar
- An array of a scalar
- A record of an array of a scalar
- An array of a record of a scalar
- A record of an array of an array of a scalar

The following return type is not supported:

- An array of an array of a scalar.

Related Topics

[VHDL-AMS Subset Definition](#)

Working with Verilog-AMS

You need to consider certain things when working with Verilog-AMS.

The following sections describe the necessary considerations when working with Verilog-AMS.

[Verilog-AMS Instantiating SPICE Models and Subcircuits.....](#) 4-43

[Verilog-AMS Instantiating SPICE Primitives.....](#) 4-44

[Verilog-AMS Access to SPICE Parameter Values.....](#) 4-46

[Rules for Verilog-AMS Instantiating Verilog.....](#) 4-47

[Verilog-AMS Instantiating VHDL-AMS.....](#) 4-51

[Verilog-AMS Branches.....](#) 4-52

[Verilog-AMS Variables.....](#) 4-60

[Formulating Verilog Hierarchical Names in Mixed-Language Simulations.....](#) 4-61

[Using wreal.....](#) 4-64

[Opening, Closing, and Writing to Files During Simulation.....](#) 4-66

[AC Noise Results for Verilog-AMS Noise Functions.....](#) 4-68

Verilog-AMS Instantiating SPICE Models and Subcircuits

The model or subcircuit name is used to reference a Verilog-AMS module when instantiating SPICE models or subcircuits within it.

When instantiating SPICE models or subcircuits within a Verilog-AMS module, the model or subcircuit name is used to reference it, as follows:

```
modelname m1 (x, y z);
subcktname s1 (p1, p2);
```

The name of the model or subcircuit corresponds to the name of the design unit instantiated. Parameter values can only be specified by name association.

Examples

- Here is an instantiation of a SPICE model, named rmodel:

```
.MODEL rmodel res tc1=0.001 tc2=0.005
```

To instantiate the SPICE model rmodel, the following is defined in the Verilog-AMS module:

```
rmodel m1 (a1, a2);
```

- Here is a SPICE subcircuit, named opamp:

```
.SUBCKT opamp 2 3 6 4 7
.MODEL NPN NPN BF=160 RB=100 CJS=2P TF=0.3N ...
R1 1 4 1K
Q1 9 3 10 NPN
...
.ENDS
```

To instantiate the SPICE subcircuit opamp, the following is defined in the Verilog-AMS module:

```
opamp s1 (a1, a2, a3, a3, a4, a5);
```

Where a1, a2, a3, a4 and a5 should be Verilog-AMS nets, not variables.

Related Topics

Verilog-AMS Instantiating SPICE Primitives

Verilog-AMS Instantiating SPICE Primitives

SPICE primitives can be instantiated in Verilog-AMS.

SPICE primitives are instantiated in Verilog-AMS using the following syntax:

```
primitive #(parameter) instancename (connectivity);
```

Table 4-1 lists the SPICE primitives supported in Verilog-AMS:

Table 4-1: SPICE Primitives Supported in Verilog-AMS

Primitive name	Port name	Parameter name
resistor	p, n	r, tc1, tc2
capacitor	p, n	c, ic
inductor	p, n	l, ic
vdc	p, n	dc, mag, phase
vexp	p, n	dc, mag, phase, val0, val1, td0, tau0, td1, tau1
vpulse	p, n	dc, mag, phase, val0, val1, td, rise, fall, width, period
vpwl	p, n	dc, mag, phase, wave
vsine	p, n	dc, mag, phase, offset, amp1, freq, td, damp, sinephase, ammodindex, ammodfreq, ammodphase, fmmindex, fmmfreq
idc	p, n	dc, mag, phase
iexp	p, n	dc, mag, phase, val0, val1, td0, tau0, td1, tau1
ipulse	p, n	dc, mag, phase, val0, val1, td, rise, fall, width, period
ipwl	p, n	dc, mag, phase, wave
isine	p, n	dc, mag, phase, offset, amp1, freq, td, damp, sinephase, ammodindex, ammodfreq, ammodphase, fmmindex, fmmfreq
diode ¹	a, c	area
bjt ¹	c, b, e, s	area
mosfet ¹	d, g, s, b	w, l, ad, as, pd, ps, nrd, nrs
jfet ¹	d, g, s	area
mesfet ¹	d, g, s	area
vcvs	p, n, ps, ns	gain

¹ The names diode, bjt, jfet, mesfet, and mosfet are never used from within Verilog-AMS HDL because these components require a model. Therefore, the model name is used in Verilog-AMS HDL instead of the primitive name.

Table 4-1: SPICE Primitives Supported in Verilog-AMS (continued)

Primitive name	Port name	Parameter name
vccs	sink, src, ps, ns	gm
tline	1, b1, t2, b2	z0, td, f, nl

Examples

- To instantiate the SPICE primitive resistor, with a value of 10k, the following is defined in the Verilog-AMS module:

```
resistor #(.r(10k)) R1 (a, b);
```

- To define a SPICE capacitor in the subcircuit c_bypass in order for it to be instantiated in Verilog-AMS:

```
.SUBCKT c_bypass coup1 coup2
c1 1 0 100n
.ENDS
```

To instantiate the SPICE subcircuit c_bypass, define the following in the Verilog-AMS module:

```
c_bypass s1 (a1, a2);
```

Related Topics

Verilog-AMS Instantiating SPICE Models and Subcircuits

Verilog-AMS Access to SPICE Parameter Values

The \$param system function call returns the value of a parameter declared in SPICE.

The .PARAM command may appear in the .cmd file or in the SPICE hierarchy above the Verilog-AMS instance. \$param() can be used anywhere a Verilog-AMS parameter is allowed. The single parameter of \$param() must be a constant string literal denoting the name of the parameter.

Any parameter declared in a .PARAM command in the SPICE context can be swept using either the .STEP or the .DATA command. Each swept value is available in successive runs using the \$param() call from Verilog-AMS.

Refer to [.PARAM](#) in *Eldo Reference Manual* for more information.

Here is example using \$param() to initialize a Verilog-AMS parameter and to drive a voltage:

```
module foo;
    parameter real vcutoff = $param ("cutoff");
    ...
    analog begin
        A <+ $param ("vmax");
    ...
endmodule
```

The parameter must have a numerical value. An attempt to reference a parameter with a string or function value will generate an error message.

Here are examples of parameters that can be accessed:

```
.param p1=12
.param p2=12e-2
.param p3=cos(3.14)
.param p4='sin(2.0)*1.65'
```

The following parameters cannot be accessed:

```
.param p5="alma"
.param p6(a,b)=sin(a+b)
```

An attempt to reference an undefined parameter or a parameter with an illegal value generates the following error message:

```
(adms-nnnn) No Eldo parameter called 'p4' exists in $param system function
call or this value is not a scalar numeric value.
```

Related Topics

[Verilog-AMS Subset Definition](#)
[Multiple-Run Analyses](#)

Rules for Verilog-AMS Instantiating Verilog

There are restrictions on the interface when instantiating Verilog modules from Verilog-AMS.

In this section, syntax shown in

red

is not supported in Questa ADMS. Syntax shown in

amber

is supported with limitations.

Refer to Mapping Data Types and Mixed-Language Simulation in the *Questa SIM User's Manual* for more information.

Module Declaration Syntax

```
module_declaration ::=
    module | macromodule module_identifier [ list_of_ports ] :
        [ module_items ]
    endmodule
```

The keyword `macromodule` is not supported.

Only the modules are visible in the Questa SIM and Questa ADMS environment. There is no limitation on the `list_of_ports` part.

Module Items

```
module_item ::=
    module_item_declaration
    | parameter_override
    | module_instantiation

module_item_declaration ::=
    parameter_declaration
    | input_declaration
    | output_declaration
    | inout_declaration
    | ...
```

The `module_item_declaration` contains:

- The parameters (similar to VHDL generics)
- The input, output, and inout declaration
- All of the other declarations and statements.

Note

Parameter names are case insensitive. For example, `Vmax` and `VMAX` are synonymous.

Limitation

Modules instantiated from Verilog-AMS must not have unnamed ports. For example, putting the slice in the port list makes the port “unnamed”. The following error message will be reported:

```
...
Compiling Module Declaration verilogsams_not_allowed_in_adms
-----
In file a.v line      1:
  module verilogsams_not_allowed_in_adms ( port_vector[7:0] );
                                     ^
[Error] Indexed and sliced port reference not yet supported
Verilog-ams subset : Unsupported
Error: (valog) Compilation: Analysis failed.
```

Parameter Declaration

```
parameter_declaration ::=
    parameter list_of_param_assignments ;
list_of_param_assignments ::=
    declarator_init { , declarator_init }
declarator_init ::=
    parameter_identifier = constant_expression
```

The type of the Verilog parameter is determined by its initial value.

Verilog Ports

Verilog ports are part of the module declaration item:

```
module_item_declaration ::=
    ...
    | input_declaration
    | output_declaration
    | inout_declaration
    | ...

input_declaration ::=
    input [ range ] list_of_port_identifiers ;
output_declaration ::=
    output [ range ] list_of_port_identifiers ;
list_of_port_identifiers ::=
    port_identifier { , port_identifier }
range ::=
    '[' constant_expression : constant_expression ']'
```

The following table shows a mapping between equivalent Verilog and VHDL ports:

Table 4-2: Verilog/VHDL Equivalent Ports

Verilog Port	Equivalent to:
	std_logic VHDL Port
input p1;	p1 : in std_logic;
output [7:0] p2;	p2 : out std_logic_vector (7 downto 0);
output [4:7] p3;	p3 : out std_logic_vector(4 to 7);
input [n-1:0] p4;	p4 : in std_logic_vector;

Instantiated Unit

A Verilog instance works in a similar way to a VHDL instance except that the optional architecture name is forbidden, because Verilog modules do not have architectures. The red syntax in the following example is not supported:

```
instantiated_unit ::=
[ component ] component_name
| entity entity_name [ ( architecture_identifier ) ]
| configuration configuration_name
```

Note

Verilog names on the border between Questa SIM and Questa ADMS (that are visible from Questa ADMS) *must* be in lowercase.

Example

Using component instantiation:

- Verilog module

```
'timescale 1ns / 1ns

module verilog_nand (inp1, inp2, outp);
  parameter del = 2;
  input  inp1;
  input  inp2;
  output outp;
  nand #del nandi (outp, inp1, inp2);
endmodule // verilog_nand
```

- Instantiation in Verilog-AMS

```

...
COMPONENT verilog_nand
  PORT (inp1   : IN   std_logic;
        inp2   : IN   std_logic;
        outp   : OUT  std_logic);
END COMPONENT;

SIGNAL out1, ... , out5 : std_logic;
SIGNAL control : std_logic;

BEGIN
  ...
  not1 : verilog_nand PORT MAP
          (control, out5, out1);
  ...

```

Hierarchical names cannot be used in defparam statements. For more information on referencing digital objects, refer to the system task [init_signal_spy](#).

Related Topics

[Verilog-AMS Instantiating VHDL-AMS](#)

Verilog-AMS Instantiating VHDL-AMS

A VHDL-AMS design unit can be referenced in a Verilog-AMS description, as though the design unit is a module of the same name (in uppercase according to Questa ADMS). You can reference a design unit either by an entity name or by a configuration name.

Port associations may be named or positional. The same port names and port positions as appearing in the entity have to be used. The named port association is case sensitive and the VHDL-AMS identifier's leading and trailing backslashes are removed before comparison. Generic associations are provided via the list of parameter values for the module instance. The values are listed in the same order as the generics appear in the entity.

An entity name is not case sensitive in Verilog-AMS instantiations. The entity default architecture is selected from the work design library unless specified otherwise.

```
entityname u1 (a, b, c);
```

Verilog-AMS does not have the concept of architectures or libraries, so the escaped identifier is employed to provide an extended form of instantiation, for example:

```
\mylib.entityname(archi) u1 (a, b, c);
```

Refers to architecture "archi" of design unit named "entityname" within library "mylib". The current instantiation is called u1. Positional port association is applied and the IO port interconnections are specified as (a, b, c).

Refer to Mixed-Language Simulation in the *Questa SIM User's Manual* for more information.

Limitations

The following limitations apply when instantiating VHDL-AMS from Verilog-AMS.

- Only entities or entity/architecture pairs can be instantiated. It is not possible to instantiate a VHDL-AMS or VHDL configuration.
- When instantiating a VHDL-AMS design entity, quantity mapping is not supported.

There are some general use limitations concerning the Verilog-AMS implementation as detailed below:

- Mapping a Verilog register to a Verilog-AMS implicit port

When instantiating Verilog-AMS from Verilog it is not possible to have a Verilog register (or an element of a register) associated to a Verilog-AMS implicit port. As a workaround, declare the implicit port as an input. Explicit analog port declarations are allowed in Verilog-AMS if connected to a Verilog register.

- Hierarchical names

Use of hierarchical names in Verilog-AMS is limited to Verilog hierarchy. All modules and signals need to refer to Verilog objects.

- None of the Verilog APIs (PLI, VPI, VPI 2) are supported by Verilog-AMS.

This restriction is only within Verilog-AMS. Within Questa ADMS, any Verilog API is supported from Verilog, and any Verilog or SystemVerilog API is supported by SystemVerilog.

Verilog-AMS Branches

A branch is a path between two nets from which it derives a discipline. Branches can be explicitly or implicitly declared.

Explicitly declared branches are referred to as named branches. Unnamed branches are created by applying an access function to either a net or a pair of nets. If the access function is applied to a single net, then the branch is formed between that net and the global reference node (ground).

Named branches can be declared with vector terminals. If one of the terminals of a branch is a vector net, then the other terminal shall either be a scalar net or a vector net of the same size. In the latter case, the branch is referred to as a vector branch.

- When both terminals are vectors, the scalar branches that make up the vector branch connect to the corresponding scalar nets of the vector terminals.

- When one terminal is a vector and the other is a scalar, a singular scalar branch connects to each scalar net in the vector terminal and each terminal of the vector branch connects to the scalar terminal.

Accessing Branch, Net and Port Functions.....	4-54
Plotting Verilog-AMS Branches.....	4-56
Viewing and Searching for Verilog-AMS Branches.....	4-59

Accessing Branch, Net and Port Functions

The following examples demonstrate how access functions applied to branches, nets, and ports.

b1 refers to a branch, n1 and n2 represent either nets or ports, and p1 represents a port. These branches, nets, and ports are assumed to belong to the electrical discipline, where V is the name of the access function for the voltage (potential) and I is the name of the access function for the current (flow).

- To access the voltage across branch b1:

```
V(b1)
```

- To access the voltage of n1 (a net or a port) relative to ground:

```
V(n1)
```

- To access the voltage difference between n1 and n2 (nets or ports):

```
V(n1, n2)
```

The following will produce an error:

```
V(n1, n1)
```

- To access the current flowing in branch b1:

```
I(b1)
```

- To access the current flowing in the unnamed branch from n1 to ground:

```
I(n1)
```

- To access the current flowing in the unnamed branch between n1 and n2:

```
I(n1, n2)
```

The following will produce an error:

```
I(n1, n1)
```

- To access the current flow into the module through port p1:

$I(<p1>)$

Plotting Verilog-AMS Branches

You can plot a Verilog-AMS named branch, implicit branch, vector named branch, an element implicit branch, or the reverse of an implicit branch.

Both SPICE syntax and **add wave** or **add log** syntax for inclusion in do files are provided:

- To plot a named branch declared in the Verilog-AMS description:

```
branch (a,b) br;
```

- SPICE syntax:

```
.PLOT... V(<path>.br)
.PLOT... I(<path>.br)
```

- Syntax for do file:

```
add wave :<path>:br
add wave -i :<path>:br
```

- To plot an implicit branch declared in the Verilog-AMS description:

```
electrical a, b;... V(a, b) <+...;
```

or

```
I(a, b) <+...;
```

- SPICE syntax:

```
.PLOT ... V(<path>.a,<path>.b)
.PLOT ... I(<path>.a,<path>.b)
```

- Syntax for do file:

```
add wave :<path>:(a,b)
add wave -i :<path>:(a,b)
```

Note

In order to request an implicit branch plot within a SPICE description, you need to provide the full path for the two nets of the branch, which corresponds to the existing syntax to plot the voltage

between two nets. The path is factorized and put outside of the brackets () before putting the name into JWDB and Solido Waveform Analyzer.

- To plot a vector named branch declared in the Verilog-AMS description:

```
electrical [0:4] a, [0:4] b;
branch (a,b) br;
```

- SPICE syntax:

```
.PLOT ... V(<path>.br[2])
.PLOT ... I(<path>.br[2])
```

Note that square brackets [] are required for vector elements in SPICE syntax.

- Syntax for do file:

```
add wave {:<path>:br[2]}
add wave -i {:<path>:br[2]}
```

Note that brackets () or square brackets [] are both accepted for vector elements in do files.

- To plot an element implicit branch declared in the Verilog-AMS description:

```
electrical [0:4] a, [0:4] b;
V(a[1], b[3]) <+...;
```

or

```
I(a[1],b[3]) <+ ...;
```

- SPICE syntax:

```
.PLOT ... V(<path>.a[1], <path>.b[3])
.PLOT ... I(<path>.a[1], <path>.b[3])
```

- Syntax for do file:

```
add wave {:<path>:(a[1],b[3])}
add wave -i {:<path>:(a[1],b[3])}
```

- To plot the reverse of an implicit branch declared in the Verilog-AMS description:

```
electrical a, b;
... V(a, b) <+...;
```

or

```
I(a, b) <+...;
```

- SPICE syntax:

```
.PLOT ... V(<path>.b, <path>.a)
.PLOT ... I(<path>.b, <path>.a)
```

- Syntax for do file:

```
add wave :<path>:(b,a)
add wave -i :<path>:(b,a)
```

Reverse branches correspond to the same branch with the reverse order, but with the opposite values. This reverse notation is only accepted for implicit branches. It is only possible to read the potential or flow of reverse branches. They can only be plotted, probed, listed or examined.

Limitation

Questa ADMS does not support the SPICE syntax which allows the voltage across two nets to be plotted wherever they are in the hierarchy, when not defining a branch:

```
.PLOT ... V(<path1>.a, <path2>.b)
```

For instance:

```
.PLOT ... V(a.b.c.d, a.b.e)
```

In this case, the wave is present in Solido Waveform Analyzer and saved in the *.ez.do* file as:

```
add wave :top:a:b:(c:d,e)
```

This syntax for a potential that does not correspond to a branch potential, may be part of a *.ez.do* file, but because ADMS does not accept it, an error message will be reported. However, if the plot is present in the *.cir* or *.cmd* file, the execution will be successful.

Viewing and Searching for Verilog-AMS Branches

When you run a simulation, branches that are part of Verilog-AMS descriptions are listed in the Objects Window and recorded in the *.wdb* file. Both implicit and declared branches are listed. This information can be used during debug to trace the behavior of a Verilog-AMS module.

Implicit branches are named inside brackets '()' and with no spaces, for example:

```
(<net1>, <net2>)
```

The voltage of the branch or the current throwing them can be plotted or logged in the same way as nets. The Tcl commands are also the same as for plotting voltage or current of an analog net.

Note

It is not possible to display, log (probe) or wave (plot) current or voltages of branches that are not used inside the Verilog-AMS description.

Branches are considered as terminals for any filter search (find, add wave/add log). When wildcards are used for branches, only elaborated branches can be found, for example:

- * returns (net1,net2)
- *1*2* returns (net1,net2)
- *2*1* returns nothing
- (net2,net1) returns (net2,net1) that corresponds to the opposite of (net1,net2)

Port Branches

Port branches are notated in Verilog-AMS descriptions as follows:

```
I(<p>)
```

where p is a port.

This corresponds to contribute or probe current through the port p, and is not really a branch in the electrical sense of a path between two nodes. It is therefore not represented in the [Objects Window](#).

When plotting or probing of the port branch in Verilog-AMS, a branch can exist across only one port p (that corresponds to a branch between the port p and ground):

```
I(p) <+ ...:
```

In this case, plotting or probing the current of this branch is done as follows:

```
.PLOT ... I(<path>.p, 0)
add wave -i :<path>:(p)
```

Related Topics

[Verilog-AMS Variables](#)

Verilog-AMS Variables

Verilog-AMS variables are displayed in the Objects window. Variables are defined as either integer or real.

Integer Variables

An integer declaration declares one or more variables of type integer. These variables can hold values ranging from -2^{31} to $2^{31}-1$. Arrays of integers can be declared using a range which defines the upper and lower indices of the array. Both indices shall be constant expressions and shall evaluate to a positive integer, a negative integer, or zero (0). Integers are initialized at the start of a simulation depending on how they are used. Integer variables whose values are assigned in an analog context default to an initial value of zero (0). Integer variables whose values are assigned in a digital context default to an initial value of x.

A real declaration declares one or more variables of type real. The real variables are stored as 64-bit quantities, as described by IEEE standard 754-1985, for double-precision floating-point numbers. Real variables are initialized to zero (0) at the start of a simulation.

Verilog-AMS variables are selected using the filter signals or nets in the add wave or find commands.

See also [Objects Window](#).

[Plotting Verilog-AMS Variables.....4-60](#)

Plotting Verilog-AMS Variables

A variable can be plotted using add wave or probed using add log in the same way as any other object.

```
add wave <variable_name>
```

Variables that are computed on the analog side will be displayed as quantities (continuous waveforms) and variables computed on the digital side will be plotted as digital signals (sampled waves).

Procedure

To plot a Verilog-AMS variable using SPICE .PLOT command syntax:

```
.PLOT [ANALYSIS] VAR(<variable_name>) [(LOW, HIGH)] [(VERSUS)] {OVN [(LOW, HIGH)]} [WHEN condition] [UNIT=NAME] [(SCATTERED)] [STEP=value]
```

Examples

```
.plot tran var(y1.Volt_a_int)
.probe tran var(y1.Volt_b_int)
```

Wildcards are supported:

```
.plot tran var(y1.*)
.plot tran var(y1.inst.*)
.probe tran var(y1.Volt*)
```

Plotting of an array of variables, or an element of an array of variables:

```
.plot tran var(y1.r1)
.plot tran var(y1.r1[1])
```

Related Topics

[add wave \[Questa ADMS Command Reference\]](#)

[add log \[Questa ADMS Command Reference\]](#)

[Objects Window](#)

Formulating Verilog Hierarchical Names in Mixed-Language Simulations

In SystemVerilog, Verilog, or Verilog-AMS, you can use a hierarchical name in the initial value expression of a parameter (parameter and localparam) or in parameter associations of an instantiation statement, to refer various objects.

You can use it to the following objects:

- Parameters of Verilog-AMS, SPICE, Verilog or System Verilog regions. The following types are supported:
 - Verilog-AMS parameters: integer, real, string and vectors of integers or reals.
 - Verilog parameters: integer, real and string are supported.
 - SPICE parameters: real

- Generics of VHDL-AMS or VHDL regions.
Types of integer, real, time, physical and enumeration are converted into integer or real. Strings are handled by Verilog-AMS strings.
- Indexed parameters, or a slice of a parameter (or a generic)
- Verilog variables
- A Verilog digital object such as a wire or a register (including SystemVerilog digital signals that are part of the Verilog subset).
- A VHDL-AMS terminal, a Verilog-AMS analog net, or a SPICE node.

defparam statements can be used across layers that are Verilog, SystemVerilog, Verilog-AMS or SPICE, but not across VHDL-AMS.

Limitations

- The resolution will fail for SPICE parameters which are optimized (optimization may be disabled using the SpiceParameterOpt variable, set in the *modelsim.ini* file).
- In port associations of an instantiation statement: the expression as actual cannot contain a hierarchical reference to a parameter
- Only backward parameter references are supported
- Backward hierarchical references are supported in Eldo-VerilogA when the hierarchical reference points to a Verilog-A parameter (but not a SPICE parameter). However, in this context the hierarchical references cannot cross SPICE regions.
- The following Verilog constructs are not supported as targets of hierarchical references:
 - Functions/Tasks
 - Scopes
 - Class objects (references to class data-members of the supported Verilog types are allowed)
- Hierarchical references are not supported in the following contexts (*Verilog-AMS Language Reference Manual 2.4 section 6.7*):
 - In the initial value expression of an analog variable
 - In declared ranges
 - On the right hand side of a defparam statement
 - In an analog block
 - In "for genvar" loop bounds

Note

Using .COMPOSITE with a hierarchical name to define the size of a Verilog hierarchical reference gives incorrect results when the subcircuit contains only one device. In this case, you must provide the .COMPOSITE command inside the subcircuit definition.

Digital-to-Digital References

In a Verilog or SystemVerilog description, the following types of Verilog or SystemVerilog objects can be referenced hierarchically:

- Parameters
- Variables
- Digital signals

These objects are accessible in cases where there is a SPICE, Verilog, SystemVerilog, Verilog-AMS, VHDL or VHDL-AMS layer between the context and the reference localizations.

Digital-to-Analog References

If a SPICE or Verilog-AMS object is referenced from a Verilog or SystemVerilog description, only analog or digital nets can be referenced hierarchically. Such a reference is not allowed if there is a VHDL-AMS layer between the context and the reference localizations. Intervening SPICE and Verilog-AMS layers are supported.

The following rules apply if the hierarchical name refers to an analog net:

- In order to manage the reading or the driving of this object from Verilog or SystemVerilog, a boundary element will be automatically inserted to interface with the analog object represented by this name. The digital side of this boundary element will follow these rules:
 - If the analog net is already a boundary net, the digital net will be used for managing the read or write from the digital side.
 - If the analog net is a pure analog net (not a boundary net), a logic digital net will be created on the digital side that will be connected to the digital part of the inserted boundary element.

Boundary elements are selected according to the **.DEFHOOK** statements or according to the Verilog-AMS **resolveto** statements if they are provided.

- Hierarchical names can be used for reading a value or forcing, assigning or releasing a net in the design (using native Verilog **assign**, **force** and **release** statements).

Any Verilog testbench can directly reference an analog or mixed-signal description from a digital description if the names are the same or if the name associations have been provided in the **Interface Association File (.assoc)** (ports only) when switching a digital portion of the design to an analog or mixed one. When an analog port is given an associated digital name in an **.assoc** file, either names can be used

Analog-to-Analog References

The following usage rules and semantic restrictions shall be applied to analog identifiers referenced hierarchically from a mixed-signal module:

- Potential and flow access for named branches can be done hierarchically.

- Hierarchical reference of an implicit net is allowed when the referenced net is first coerced to a specific discipline.
- The following limitations apply:
 - Analog variables cannot be accessed hierarchically
 - It is not possible to access the flow of an unnamed hierarchical branch
 - Port branches cannot be hierarchically referenced
 - It is not possible to have potential and flow contributions to named or unnamed branches hierarchically
 - Hierarchical notation cannot be used to assign to a variable
 - In an analog context, reading an out of module parameter is not supported. However, it is possible to write them using defparam statements.

When the flow of a named branch is referenced hierarchically from a Verilog-AMS description, you may encounter an error during elaboration, if some optimizations stop branches from elaborating. Use the **valog** -nooptbr option to disable optimizations on a named branch:

```
valog -nooptbr <model_name:branch_name>
```

Related Topics

Naming Conventions

Writing Hierarchical Path Names

Using wreal

Nets and ports of type wreal, and arrays of wreal can be declared in both Verilog-AMS and SystemVerilog.

Like wires, a wreal net may have multiple drivers. Unlike wires, the drivers of wreal nets are real numbers rather than logic levels. In addition to real values, wreal can take on two additional states: wrealXstate and wrealZstate. wrealZstate represents disconnection, wrealXstate indicates that the value is unknown.

For details on how to set and reference the values refer to [wreal Compatibility with VHDL](#).

Variable ports of type real can be declared only in SystemVerilog, not in Verilog-AMS. A wreal and a real can connect through a port connection.

Note

It is not possible to save or checkpoint a simulation that contains wreal declarations.

By default, Questa ADMS initializes driven and undriven wreal nets to 0.0. If you specify the vasim argument -disablewrealinitzero, Questa ADMS uses the Questa SIM initial wreal values: wrealZstate for

driven wreal nets and wrealXstate for undriven wreal nets. You can combine -disablewrealinitzero with -wrealinitdrivingvalzero or -wrealdefaultzero to initialize only driven or undriven wreal nets (respectively) to 0.0.

wreal Resolution Function

To compute the effective value, wreal drivers on a network are passed through a resolution function. Six resolution functions are supported, as described in [Table 4-3](#).

Table 4-3: wreal Resolution Functions

wreal Resolution Function	Description
default	Single active driver only, support for Z state
4state	Similar to Verilog 4-State resolution for digital nets
sum	Resolves to a summation of all the driver values
avg	Resolves to the average of all the driver values
min	Resolves to the smallest value of all the driver values
max	Resolves to the greatest value of all the driver values

The following **vasim** (or vsim) argument is used to select the resolution function to use for wreal when multiple drivers are connected to the same net:

```
vasim -wreal_resolution default | 4state | sum | avg | min | max
```

wreal Compatibility with VHDL

A package wreal_pkg is available as part of the MGC_AMS library, which interprets X and Z values of a Real signal coming from Verilog-AMS and sets these values to be compatible with Verilog-AMS wreal values.

```
-- Part of MGC_AMS library

package wreal_pkg is
    function wreal_resolution
        (arg : Real_vector) return Real;
    subtype wreal is wreal_resolution Real;
    type wreal_vector is
        array(Natural range <>) of wreal;

    function is_real_X (arg : Real) return Real;
    function is_real_Z (arg : Real) return Real;
    procedure set_real_X (arg : OUT Real);
    procedure set_real_Z (arg : OUT Real);
end package wreal_pkg;
```

wreal is equivalent to the Verilog corresponding time, using one of the 6 existing resolution functions, according the option provided at simulation.

The functions `is_real_<>` return TRUE if the real number is X or Z.

The procedures `set_real_<>` are putting the values X or Z to the value of the object passed to them.

Specifying Digital Net Type (wire/wreal)

When a Verilog-AMS unit is compiled using `valog`, its analog nets can be connected to single bit digital nets (equivalent to `wire`) or to a real-type digital net (`wreal`). In order for Verilog-AMS to be instantiated in VHDL it is necessary to specify the digital connection type at compilation, using:

```
valog [ -target_type wire|wreal ]
```

This defines the digital net that corresponds to all the data ports of the Verilog-AMS unit. The default is `wire`.

If the mixed-signal unit is seen from the digital side by mixing `wire` and `wreal` types, or if the supply nets attributes are not present, a digital unit representing the Verilog-AMS one can be provided using:

```
valog [ -target_entity <unit_name> ]
```

If present, it overrides `-target_type`.

Related Topics

[valog \[Questa ADMS Command Reference\]](#)

[Verilog-AMS Amplifier with a Resolved wreal Input and an Electrical Output](#)

[Verilog-AMS Subset Definition](#)

Opening, Closing, and Writing to Files During Simulation

You can use functions in Verilog-AMS and Verilog-A models to manipulate files during simulation.

In Verilog-AMS and Verilog-A models, you can open, close and write into files during the simulation using the `$fopen`, `$fwrite`, `$fdisplay`, `$fstrobe`, and `$fclose` system functions. If required, you can use an environment variable to specify the path to the required file.

Examples

- If the environment variable MYDIR is defined as */home/my_design*, the full path name used will be */home/mydesign/myfile.txt*:

```
module top;
  real v;
  integer file_descriptor;

  analog
  begin
    @(initial_step)
    begin
      file_descriptor = $fopen("$MYDIR/myfile.txt");
    end

    v = v + 1.0;
    $fwrite(file_descriptor, "%d", v);

    @(final_step)
    begin
      $fclose(file_descriptor);
    end
  end
endmodule
```

This will open the file */home/my_design/myfile.txt*

- If you specify a directory that does not exist, then it will be created, provided that the opening mode is write ("w" mode) or append ("a" mode). The directory is not created in read mode ("r").

For example, in the following, if the directory */home/my_design* exists, but */home/my_design/files* does not exist, and the file is opened for writing or appending, the directory "files" will be created before opening the file:

```
module top;
  real v;
  integer file_descriptor;

  analog
  begin
    @(initial_step)
    begin
      file_descriptor = $fopen("/home/my_design/files/myfile.txt");
    end

    v = v + 1.0;
    $fwrite(file_descriptor, "%d", v);

    @(final_step)
```

```

begin
    $fclose(file_descriptor);
end

end
endmodule

```

Note

In Verilog-AMS designs, environment variables can also be used in digital parts of the module. In addition, if a directory does not exist, it will be created, provided that the opening mode is write ("w" mode) or append ("a" mode). Be aware that this behavior is not implemented in pure Verilog models.

Related Topics

[Verilog-AMS Subset Definition](#)

[Location Maps](#)

AC Noise Results for Verilog-AMS Noise Functions

AC noise analysis (.NOISE) results are sorted by Verilog-AMS noise function label.

The Verilog-AMS noise functions `white_noise`, `flicker_noise`, `noise_table`, and `noise_table_log` each have an optional name argument, which you can use to give the noise source a label. Refer to section 4.6.4 in *Verilog-AMS Language Reference Manual Version 2.4* for more information.

Questa ADMS sorts AC noise analysis (.NOISE) results by the labels that you specify in the Verilog-AMS noise functions. For example, the AC noise analysis results for a Verilog-AMS module containing the noise contributions:

```

V(out) <+ white_noise(<white1>, "device_a")
        + white_noise(<white2>, "device_b")
        + white_noise(<white3>, "device_b")
        + flicker_noise(<flicker1>, 2.0, "device_a")

```

will be sorted as follows:

device_a	<noise_a>
device_b	<noise_b>
TOTAL	<total_noise>

where:

- <noise_a> is the sum of the contributions for <white1> and <flicker1> at the noise analysis frequency.

- `<noise_b>` is the sum of the contributions for `<white2>` and `<white3>` at the noise analysis frequency.
- `<total_noise>` is the sum of `<noise_a>` and `<noise_b>`.

Working With Verilog-A

The following sections describe the necessary considerations when working with Verilog-A.

Using Verilog-A v1.0 Code in Questa ADMS..... 4-70

Using Verilog-A v1.0 Code in Questa ADMS

You can use Verilog-A v1.0 Code in Questa ADMS.

The syntax of the analog subset changed between Verilog-A LRM1.0 specifications and Verilog-AMS LRM2.1 specifications.

If you are using Verilog-A v1.0 code in Questa ADMS simulations, you may need to use the argument -1.0 during compilation.

Table 4-4 reflects these changes and shows which features are supported in this release of Questa ADMS. In the Questa ADMS Status column, -1.0 indicates that the feature is interpreted according to LRM 1.0 specification (upward compatibility) if the option -1.0 (or equivalent option -lrm10) is passed to the valog command. This option allows Questa ADMS to handle existing designs that are not compliant with Verilog-AMS 2.1 (because of changes between v1.0 and v2.1).

Table 4-4: Supported Verilog-AMS Features

Feature	Verilog-A v1.0	Verilog-AMS v2.1	Change type	Questa ADMS Status
Analog time	\$realtime	\$abstime	new	-1.0
Distribution functions	\$dist functions() Integer based functions	\$rdist functions() real value equivalents to \$dist functions()	new	-1.0
Empty discipline	predefined as type wire type	not defined	default definition	-1.0: limited to scalar nets
Implicit nodes	'default_nodetype discipline identifier default: wire	default: empty discipline no domain type	default definition	-1.0: 'default_nodetype supported but no default (needs empty discipline)
initial step	default = TRAN	default = ALL	default definition	default value is ALL with or without option -1.0
final step	default = TRAN	default = ALL	default definition	default value is ALL with or without option -1.0
Analog ground	no definition	now a declaration statement	definition	-1.0
\$realtime	\$realtime: timescale =1 sec	\$realtime: timescale=`timescal	definition	-1.0

Table 4-4: Supported Verilog-AMS Features (continued)

Feature	Verilog-A v1.0	Verilog-AMS v2.1	Change type	Questa ADMS Status
		e def=1n, see \$abstime		
Discontinuity function	discontinuity(x)	\$discontinuity(x)	syntax	-1.0
Array setting	aa[0 : 1] = {2.1 = (1), 4.5 = (2)}	aa[0 : 1] = {2.1; 4.5}	syntax	Not supported
Limiting exponential function	\$limexp(expression)	limexp(expression)	syntax	-1.0
Port branch access	I(a,a)	I(<a>)	syntax	-1.0
Timestep control	bound_step(const expression)	\$bound_step(expr)	syntax	-1.0
Continuous waveform delay	delay()	absdelay()	syntax	-1.0
User-defined analog functions	function	analog function	syntax	-1.0
'default nodetype	'default nodetype	'default discipline	Obsolete	-1.0
Forever statement	forever	N/A	Obsolete	Not supported

Related Topics

[Verilog-AMS Characteristics in Questa ADMS](#)

[Working with Verilog-AMS](#)

Mapping Ports Using the Interface Matcher

Use the Interface Matcher to make interface associations between a SPICE subcircuit and a digital design unit, by mapping analog pins to digital ports.

To make interface associations between any two design units that describe the same behavior at the digital level, refer to [Associating Design Units with vamatch](#).

Procedure

1. Enter the **vaspi** -interactive command to open the **Interface Matcher**:

```
vaspi -interactive [<options>] <digital_unit>
<subckt_name>[@<file_name>]
```

If you provide unit names and options on the command line, the corresponding fields are populated in the Interface Matcher Unit Selection window. If you did not specify unit names and options, select them as follows:

- a. In the File field, specify the SPICE netlist file that contains the required SPICE subcircuit.
If you are using a Spectre netlist file to provide a subcircuit definition, select the SPECTRE option.
- b. In the Subcircuit field, select the required SPICE subcircuit contained within the selected SPICE netlist file.
- c. In the Source library field, select the required library.
- d. In the Model field, select the required HDL model from those contained within the selected library.

Tip

You can search for a SPICE subcircuit within the SPICE file or a HDL model within a source library by clicking the Find button; this displays the Find window.

- e. In the Work library field, select the working library. This library is where the model containing the SPICE description will be created.
If the digital Source library and Work library are the same, you will be asked to confirm the overwrite when you click OK.
2. Click OK. The Interface Matcher dialog box is displayed. The current pin/port mapping associations are listed with analog pins (SPICE PINS) on the left, and digital ports (BEHAVIORAL PORTS) on the right. Pins and ports that are displayed adjacent to one another on the same line will be associated.
 3. Click the Automatic Mapping button  to associate pins and ports with similar names. The association lists will be reordered based on the matching.
 4. If required, adjust the mapping manually. Move the analog pins to line up with the required digital ports using the arrow buttons next to each list, or by dragging/dropping the pin or port to the required position using the right mouse button.
To move multiple pins/ports in one step, hold down the left mouse button, and drag the mouse over the required range of pins/ports.
 5. If pins or ports could not be mapped automatically, they will be listed in the SPICE GLOBAL ASSOCIATIONS and BEHAVIORAL GLOBAL ASSOCIATIONS lists at the bottom of the dialog box. Use the arrow buttons to move them into the Associations lists and associate them manually.

6. Optionally, to associate a Spice pin with a global Spice or HDL net, click on a Spice port in the SPICE GLOBAL ASSOCIATIONS list. This opens the [Supply Connections Dialog Box](#). Enter information about the global net in the Global net field.
7. To create an association for an array, select consecutive analog pins from the association list, then click on the Analog Association button:



This will associate the selected pins so they can be mapped to a digital array. The association is symbolized with a "+" next to the first pin in the hierarchy.

8. To map several pins/ports to a single pin/port, create an interconnection. To do this, select consecutive pins or ports in either the analog or digital association lists, then click on the Interconnection button:



This will make an interconnection to connect all the elements to a single pin/port. This is symbolized with a "*" next to the first pin/port in the hierarchy.

9. To split an analog association or an interconnection, select all of the pins/ports in the association or interconnection and then click the Split button:



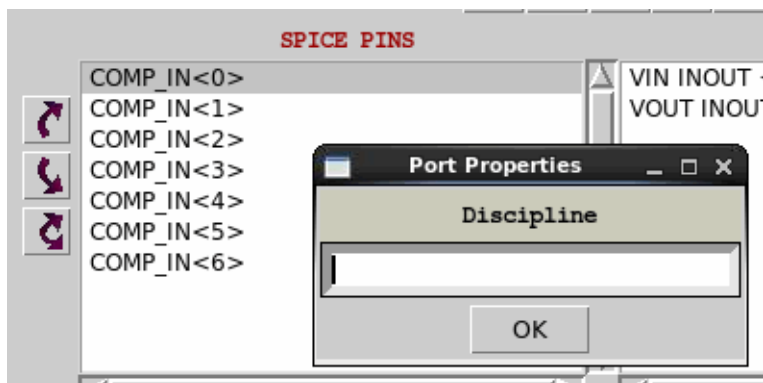
The pins/ports will then return to their original states.

Note

It is not possible to map an analog association/interconnection with a digital interconnection. If this occurs, the first pin of the associations/interconnections will be highlighted in red.

10. Optionally, double-click on a SPICE pin to assign a continuous discipline to it:

Figure 4-1: Assigning a Continuous Discipline to a SPICE Pin



11. Optionally, to add one or more supply connections, double-click on a pin in the SPICE GLOBAL ASSOCIATIONS list or the SPICE PINS list. This opens the [Supply Connections Dialog Box](#). The association between the supply Spice pin and the Power Domain Connection is defined using the information in the UPF section of this dialog. Choose the appropriate items in the dialog box and

click **OK**. The relative supply Spice port from SPICE GLOBAL ASSOCIATIONS list is moved to the SPICE PINS list and has a UPF supply port created in the BEHAVIORAL PORTS list with the information given in the Supply Connections dialog box. The Spice pin and the behavioral port are represented on the same row.

If the Power domain connection name filled by user is the name of an existing digital port which is still in the BEHAVIORAL GLOBAL ASSOCIATIONS list, a digital port is not created and this port which is moved up in the BEHAVIORAL PORTS list in front of the Spice supply net, at the same level.

If a Spice pin from the SPICE PINS list is double-clicked, the Supply Connections dialog box is displayed with its corresponding supply connection, but no value or selection can be modified. Click **RESET** to remove all the information filled relative to the port. The Spice port is moved from SPICE PINS list to SPICE GLOBAL ASSOCIATIONS list and the UPF port is removed from the BEHAVIORAL PORTS list. The Spice port is highlighted and the Supply Connections dialog box remains open ready for power domain connection specification.

12. Once mapping is complete, click **Save**. The **Interface Association File (.assoc)** will be generated, and the vasp command is re-run automatically using this new file.

By default, the association file is saved to the location from which the Interface Matcher was launched. The saved location is displayed in the terminal window when generation is complete. If a previously generated association file exists with the same name, the new file will overwrite the existing file. Use the **Save as** button to specify a different name and location of the association file under which it is to be saved.

If any internal errors occur during the Interface Matcher wizard session, an error file named *tcl_script.error* is generated, and saved in the same directory from which the Interface Matcher was launched.

Related Topics

[Associating Design Units with vamatch](#)

[Interface Matcher](#)

[vaspi \[Questa ADMS Command Reference\]](#)

[Interface Association File \(.assoc\)](#)

Associating Design Units with vamatch

You can create associations between design units that describe the same behavior.

The **vamatch** command is used when loading a design, in order to create associations between design units that describe the same behavior. An **Interface Association File (.assoc)** is created as a result of this process, which can then be used with the **.BIND** command. The syntax is:

```
vamatch [options] [model_name1] [model_name2]
```

If any of the units that are associated by vamatch have already been compiled on the Questa SIM side, they will be automatically available to Questa ADMS without the need to recompile.

Refer to Mapping data types (Mixed-language Simulation) in the *Questa SIM User's Manual* for more information.

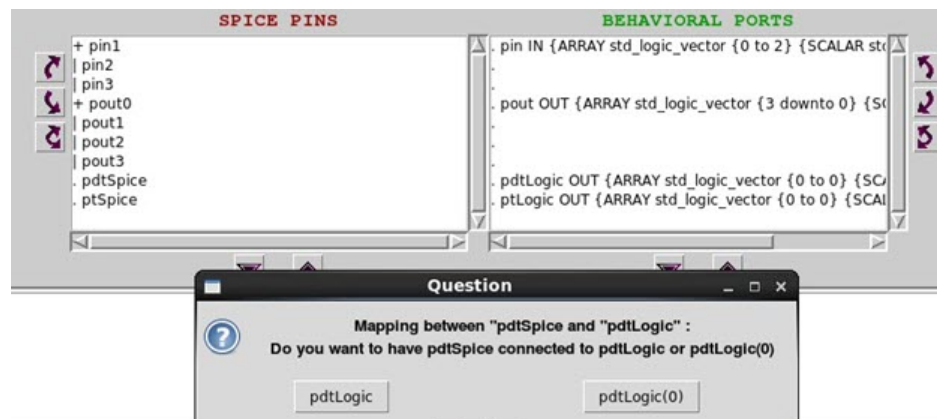
The options available on the **Interface Matcher** are similar to those that appear with the `vaspi -interactive` command; this process is described in **Mapping Ports Using the Interface Matcher**.

You can specify whether the association file generated by the **vamatch** command maps the design unit ports at the vector level or scalar level through the `modelsim.ini` file variable `AssocAsAWhole`. For example, if the following criteria apply:

- `AssocAsAWhole` variable is zero.
- Digital vector size is one.
- Number of analog scalars is one.

Then the following dialog box is displayed, which enables you to specify vector or scalar design unit mapping during the elaboration phase.

Figure 4-2: Vector or Scalar Level Mapping Dialog



Example

The following is an example of an association file generated by `vamatch` (and `vaspi -interactive`) when two VHDL-AMS `real_vector` (0 to 2) port signals (`sig_in` and `sig_out`) are connected to SPICE nets (`IN_0`, `IN_1`, `IN_2` and `OUT_0`):

```
Unit 1: VHDL common:ENT_VR
Unit 2: SPICE
SPI_6@subckt.spi
[Port association]
sig_in => ( IN_0 , IN_1, IN_2 )
    // need scalar converters between Real and Electrical. Sig_in(0) is
    // connected to IN_0, sig_in(1) to IN_1, and sig_in(2) to IN_2
sig_out => ( OUT_0 )
    // need composite converters between Real_vector and Electrical.
```

```
// The whole composite signal sig_out is connected to OUT_0.
[Unit 1 special]
[Unit 2 special]
```

vamatch can manage unconstrained arrays, provided the unconstrained dimension is the top one.

When too many ports are mapped in front of a port of a composite type, vamatch stops the generation of the association file, and reports an error.

When too few ports are mapped in front of a port of a composite type, vamatch proceeds with the generation of the association file, but leaves spaces in the associations, for example:

```
rr => ( ( IN_0 , IN_1 ) , )
```

Related Topics

[Interface Association File \(.assoc\)](#)

[Associating Design Units Using the .BIND Command](#)

[vaspi \[Questa ADMS Command Reference\]](#)

[vamatch \[Questa ADMS Command Reference\]](#)

Associating Design Units Using the .BIND Command

You can replace a SPICE subcircuit or behavioral model (VHDL, VHDL-AMS, Verilog, or Verilog-AMS) with an equivalent SPICE subcircuit or behavioral model.

A SPICE subcircuit or behavioral model (VHDL, VHDL-AMS, Verilog, or Verilog-AMS) can be replaced with an equivalent SPICE subcircuit or behavioral model, using the .BIND command. This allows you substitute one model or subcircuit for another without modifying the description.

The substituted model or subcircuit must be defined in the .cir file (or referenced in a file included with the .include command).

The .BINDSCOPE command can be used to specify the hierarchical path (in full or part of) to the instance(s) that will be replaced. In this case the path specified with the .BIND command must be relative to the path in the .BINDSCOPE command.

Table 4-5 shows the supported substitutions which can be made using the .BIND command.

Table 4-5: .BIND Configurations

	With:				
Replace:	SPICE	ADiT	Foreign SPICE	Y model (VHDL & Verilog)	Y model (VHDL-AMS & Verilog-AMS)
SPICE	Yes	Yes	Yes	Yes	Yes
ADiT	Yes	Yes	No	Yes	Yes
Foreign SPICE	Yes	No	No	Yes	Yes
Y model (VHDL & Verilog)	Yes	Yes	Yes	Yes	Yes
Y model (VHDL-AMS & Verilog-AMS)	Yes	Yes	Yes	Yes	Yes

Post-processing commands such as .PLOT or **add wave** do not require modification following a .BIND substitution. If the plot of a net (signal, terminal, quantity or node) is requested on an instance that has been replaced, the net name specified on the .PLOT or **add wave** command may not exist in the substituted model/subcircuit. In this case, Questa ADMS searches for the net name in the original instance. If found, the corresponding net in the new model/subcircuit will be plotted in Solido Waveform Analyzer with the name specified on the .PLOT or **add wave** command.

Examples

- A SPICE description is contained in the netlist *top.cir*. To substitute all instances of subcircuit inverter that are part of the sub-instance called X1.X2 by the Verilog model inverter compiled in the working library, a new file called *new-top.cir* can be written as follows:

```
* file new-top.cir
.INCLUDE top.cir
* substitution model declaration
.MODEL new_inverter macro lang=verilogams
+      mod=inverter param: delay=10ns
* substitution command
.BIND inst=x1.x2.*
+      from_subckt=inverter to_model=new_inverter
```

- This example is similar to the previous example, but this time the .BINDSCOPE command is used in conjunction with the .BIND command to define the path to the instances that will be replaced:

```
* file new-top.cir
.INCLUDE top.cir
* substitution model declaration
.MODEL new_inverter macro lang=verilogams
```

```
+      mod=inverter param: delay=10ns
* substitution command
.BINDSCOPE PATH=x1.x2
.BIND inst=*
+      from_subckt=inverter to_model=new_inverter
```

- In this example, a SPICE subcircuit with 8 data inputs at the interface (data_in<0>, data_in<1>, ..., data_in<7>) is replaced with a VHDL model that has an 8-bit port vector called data_in. Association by name will be used, therefore, the form of the SPICE nodes and the VHDL port vector have to be identical. The form of the SPICE node and VHDL port vector must have been defined by the VectorPatternString variable (see [Simulator Control Variables](#)):

```
set VectorPatternString(1) "@name@<@idx@>"
```

The index 0 of the vector data_in will be mapped to the node data_in<0>. The index 1 of the vector data_in will be mapped to the node data_in<1> etc. The .BIND command statement to make the substitution is:

```
.BIND inst=x1.x2
+      from_subckt=data_module to_model=data_module
+      default_mapping= by_name
```

Limitations

- You cannot use the .BIND command to make substitutions if the parent of the instance being replaced is VHDL or Verilog.
- A HDL model must be instantiated as a Y-element in a SPICE subcircuit in order to be replaced with another SPICE subcircuit.
- .BIND commands are ignored if they attempt to substitute models in portions of code where that model is not supported. Substitution will only be made in the parts of the code that support the instantiation of the model.
- .BIND statements ignore .MODEL parameters when substituting a Verilog-A HDL model with a SPICE subcircuit.
- It is not possible to replace an ADiT SPICE model with a foreign SPICE model using the .BIND command.

Related Topics

[Tutorial 3: Replacing Design Units Using the .BIND Command \[Getting Started with Questa ADMS\]](#)

[.BIND \[Questa ADMS Command Reference\]](#)

[.BINDSCOPE \[Questa ADMS Command Reference\]](#)

Connecting Nets

Questa ADMS supports two ways to connect nets in your design.

SPICE

You can use the Eldo **.CONNECT** command to connect two nets in a SPICE region. Either of the two nets may be analog or digital. If one of the nets is analog and the other net is digital, Questa ADMS will insert a boundary element. Questa ADMS will also insert a boundary element if one of the nets is digital of type logic and the other net is digital of type real or wreal.

When using **.CONNECT** to connect two digital nets of different types, Questa ADMS will issue the following error if one of the nets is not of type logic, real, or wreal:

```
(vasim-4169) Cannot connect signals with different types through .CONNECT
SPICE command
```

Questa ADMS does not support **.CONNECT** for VHDL digital objects of user-defined type.

Questa ADMS supports **.CONNECT** for Verilog variables. However, the simulation results may be incorrect if you use **.CONNECT** with a Verilog variable that is not driven by a concurrent statement. Questa ADMS will issue the following error if you use **.CONNECT** with any Verilog variable, but the simulation will continue:

```
(vasim-4170) Assuming Verilog variable <name> has concurrent statement when
referencing it through SPICE description. It can lead to incorrect results
if this assumption is not correct.
```

VHDL-AMS and Verilog-AMS

You can use the **init_terminal_short** procedure to connect two nets in VHDL-AMS and Verilog-AMS regions. At least one of the two nets must be analog.

Naming Conventions

This section describes the conventions for writing names and searching for objects within Questa ADMS.

Writing Object Names.....	4-80
Writing Hierarchical Path Names.....	4-82
Obtaining a Hierarchical Path Name.....	4-86
References to Verilog Escaped Identifiers.....	4-87
Extended Identifiers.....	4-88
Instance Names of Digital Models in SPICE Subcircuits.....	4-89
Case-sensitivity for VHDL-AMS Declarations.....	4-90

Writing Object Names

There are four types of object name.

- Simple Name

The name of an object in the current context (currently selected instance in the Structure window). For example:

```
clk or data[3:0]
```

- Relative Name

A name that is relative to the current context, in the current or specified dataset. It does not start with a path separator, but can include a dataset name or a hierarchical path. For example:

```
u1/data or view:clk
```

- Fully-Rooted Name

A name that starts with a path separator and includes a hierarchical path to an object. If the top-level design unit name is not specified, the first top-level instance in the design is assumed. For example:

```
/top/u1/clk
```

- Absolute Name

An explicitly-specified hierarchical name containing a dataset name and a fully rooted name, for example:

```
sim:/top/u1/clk
```

Object Name Syntax

```
[<datasetName><datasetSeparator>][<pathSeparator>][<hierarchicalPath>]  
<objectName>[<elementSelection>]
```

Parameters

- datasetName

The logical name of the .wdb file in which the object exists. The currently active simulation is named "sim".

If you do not specify a dataset name as part of the name, the current dataset is used. The current dataset is determined by the dataset currently selected in the **Structure Window** or by the last dataset specified in an environment.

- datasetSeparator

The character used to terminate the dataset name. The default is a forward slash (/) but a different character (other than backslash (\)) may be specified using the environment variable DatasetSeparator (see [Questa SIM Simulation Control Variables](#)) in the *modelsim.ini* file. This character must be different than the pathSeparator character.

- pathSeparator

The character used to separate hierarchical object names. The default is a colon (:) but a different character (other than backslash (\)) may be specified via the [PathSeparator](#) variable in the *modelsim.ini* file. This character must be different than the datasetSeparator.

- hierarchicalPath

A set of hierarchical instance names separated by a path separator and ending in a path separator prior to the objectName. For example:

```
/top/proc/clock
```

- objectName

The name of the required object.

- elementSelection

Indicates some combination of the following:

- Array indexing

Single array elements are specified using either parentheses "()" or square brackets "[]" around a single number.

- Array slicing

Slices (or part-selects) of arrays are specified using either parentheses "()" or square brackets "[]" around a range specification. A range is two numbers separated by one of the following: to, downto, or :

- Record field selection

A record field is specified using a period (.) followed by the name of the field.

- C++ class, structure, and union member selection

A class, structure, or union member is specified using the record field selection syntax, described above.

Examples

- To specify the object clk in the current context, use:

```
clk
```

- To specify the object clk in the top-level design unit, use:

```
/top/clk
```

- To specify the object clk, two levels down from the top-level design unit, use:

```
/top/block1/u2/clk
```

- To specify the object clk, two levels down from the current context, use:

```
block1/u2/clk
```

- To specify an index of an array object, use:

```
array_sig[4]
```

- To specify a slice of an array object in VHDL or SystemC, use:

```
{array_sig(1 to 10)}
```

- To specify a slice of an array object in Verilog or SystemC, use:

```
{mysignal[31:0]}
```

- To specify a field of a record, a C++ class or structure member, or a C++ base class, use:

```
record_sig.field
```

Note

VHDL-AMS unit names (VHDL-AMS entity, architecture, package, and configuration names) and Verilog-AMS module names must not exceed 200 characters in length. All other identifiers (constants, generics etc.) must not exceed 1024 characters in length.

Related Topics

[Writing Hierarchical Path Names](#)

[Obtaining a Hierarchical Path Name](#)

Writing Hierarchical Path Names

A net begins at a locally declared root node and descends through the design hierarchy from instance to instance, through a tree of associations. You can use a hierarchical path to reference a specific instance.

The net name is the name of the simple name of the net:

```
net_name ::=
[library_name:]package_name.signal_name | [hierarchical_path.]simple_name

hierarchical_path ::= instance_label{.instance_label}

root_name ::= signal_name|terminal_name|verilog_name| node_name
```

If the hierarchical path is not used, the object with the given `simple_name` must be in the testbench, or be locally declared. Port and interface pin names are never used for `simple_name`.

A hierarchical path is a dot-separated path to an instance, that starts inside the testbench, and descends through the design hierarchy. It is always an absolute, or rooted path.

`instance_label` can be:

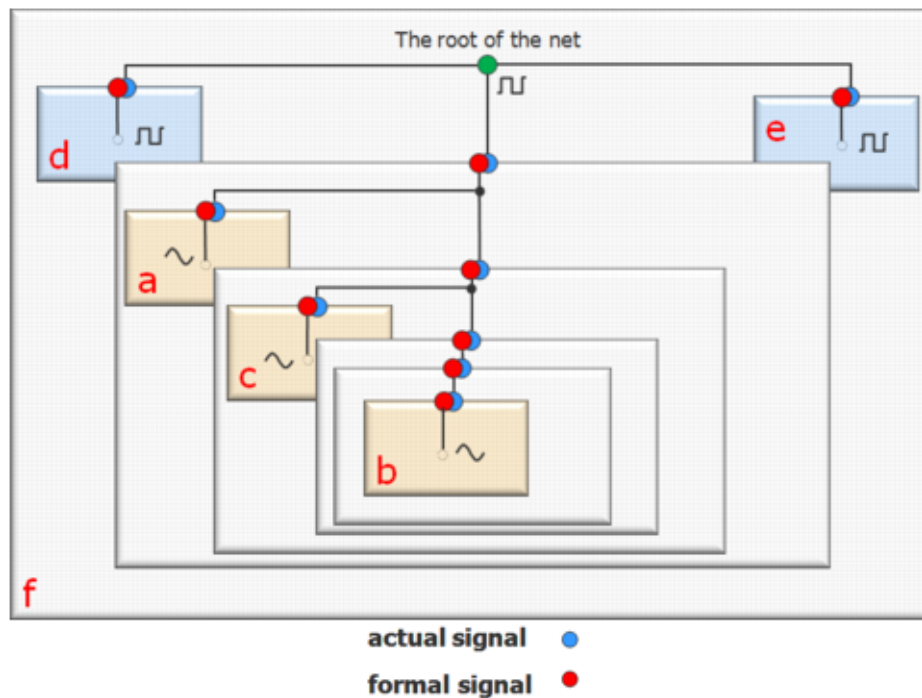
- For Verilog or Verilog-AMS: `proj_a uut (...)`
- For VHDL or VHDL-AMS: `recvr: entity cdma...`
- For SPICE: `yu2 rxamp ...`

For example, a hierarchical path might be `proj_a.recvr.yu2`.

In Eldo and in Eldo/ADMS, you can use an alternative notation for instance names. You can separate the `x` or `y` in the instance name from the rest of the characters with the quote (") character. With this notation, the `x` or `y` will only indicate the form of the instantiation: `x` for an Eldo subcircuit and `y` for a behavioral instance. The instance name is the string of characters after the quote. The hierarchical path does not include the quote sign or the subcircuit type

The hierarchical path to the top of the hierarchy has no `instance_label`, therefore the top design entity does not form part of the name. The name starts one level below. The path to the top of the hierarchy is the null string (the empty string).

Figure 4-3: A Mixed-Signal Net Hierarchy



When VHDL-AMS names are specified in Eldo commands, they must follow the correct naming convention for the type of hierarchy they reside in. If a VHDL-AMS object is declared within a VHDL-AMS scope, meaning that from the top of the design to the object, only VHDL-AMS hierarchy has been encountered, then VHDL-AMS objects declared within a package (which is compiled in a library) are named as:

```
library_name : package_name .simple_name
```

If the object is declared within a block or a design entity, use a hierarchical path:

```
instance_label{.instance_label} .instance_label .simple_name
```

Where instance_label can be the name of an instance, a block, or a SPICE subcircuit.

If the VHDL-AMS object is entirely under a SPICE scope, in order to refer to an item declared in a package, the following SPICE syntax must be used:

```
logical_library_name:package_name.simple_name
```

For example, the following syntax is used to refer to a terminal named vdd declared in the package pkg that is compiled in the library named work:

```
work:pkg.vdd
```

The following Eldo commands (placed in the .cmd file) connect the global terminals in a package called power_supply_main with SPICE vdd and ground:

```
.connect work:power_supply_main.vdd vdd
.connect work:power_supply_main.vss 0
```

Note that when a node is named VDD, VSS, VEE, VCC, or GND, and this node is not connected to a power supply, a warning message will be reported.

Absolute and Relative Paths

You can use absolute and relative paths in order to reference objects. The absolute path provides the full hierarchical path to the object. The relative path provides a path to the object that is relative to the current location.

Absolute path syntax:

```
: { <instance_name> : } <object_name>
```

Relative path syntax:

```
{ <instance_name> : } <object_name>
```

<instance_name> is used to specify a region of the design. Additionally, ".." can be used to reference the parent region.

<object_name> is used to specify a digital or analog net.

Examples

In order to reference the following net:

```
:top:instance1:instance2:instance3:net:
```

You can use an absolute path:

```
:top:instance1:instance2:instance3:net
```

or:

```
:top:instance1:instance2_other:...:instance2:instance3:net
```

If the calling region is :top:instance1:instance2, then the relative path is:

```
instance3:net
```

If the calling region is :top:instance1:instance2_other then the relative path (from instance2_other) is:

```
...:instance2:instance3:net
```

Related Topics

Obtaining a Hierarchical Path Name

Obtaining a Hierarchical Path Name

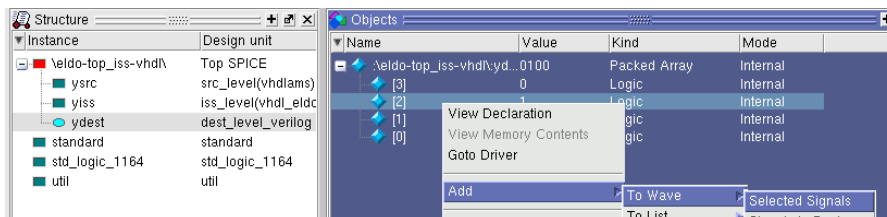
You can find the path name to an object in a compiled design.

When you have loaded a compiled design into Questa ADMS (interactive mode) you can quickly find the path name to an object, by adding it to the Wave window. The correct path of the object appears in the [Transcript Window](#), and is written to the transcript file.

Procedure

1. In the [Structure Window](#), select the required instance, and in the [Objects Window](#), select the required object.
If these windows are not already visible, select **View > Structure** and **View > Objects** to open them.
2. Right-click on the selected object and select **Add > To Wave > Selected Signals**.

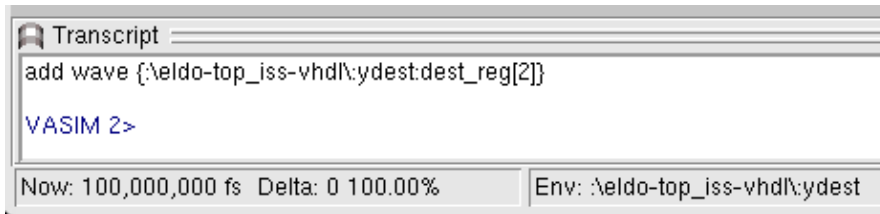
Figure 4-4: Adding an Object to the Wave Window from the Objects Window



- Look at the message reported in the **Transcript Window** to see the path of the object. This is also written to the transcript file:

```
add wave {:\eldo-top_iss-vhdl\ydest:dest_reg[2]}
```

Figure 4-5: Object Pathname in Transcript Window



Related Topics

[Formulating Verilog Hierarchical Names in Mixed-Language Simulations](#)
[Writing Hierarchical Path Names](#)

References to Verilog Escaped Identifiers

You need to use a special terminator to refer to Verilog escaped identifiers when using Eldo commands.

To refer to Verilog escaped identifiers when using Eldo commands (such as .PLOT or .PROBE) you must terminate the identifier with the "\" character instead of a space.

For example, the Verilog escaped identifier `instance-1` must be referred to as `\instance-1\`

use

```
.PLOT tran V(\instance-1\.net1)
```

instead of

```
.PLOT tran V(instance-1 .net1)
```

In addition, if the "\" character appears in the Verilog escaped identifier name, then it must be doubled-up. For example, the Verilog escaped identifier `instance\1` must be referred to as `\instance\\1\`.

Note

The hierarchical character "." is not allowed in node names unless the node name is enclosed by escape characters "\\".

Related Topics

Naming Conventions

Extended Identifiers

Questa ADMS supports extended identifiers by default.

The following graphic characters are not supported in extended identifiers:

```
@, (space), "
```

If a SPICE description contains identifiers using the “\” character, and if Questa ADMS does not use extended identifiers, it is possible to retract the default behavior by using the following command:

```
.OPTION noadmsbs
```

Alternatively, set the ExtendedId simulator setup variable (see [Simulator Control Variables](#)) to 0.

In normal use, Eldo allows the backslash character “\” to be used as a character, and the name is parsed in uppercase. When extended identifiers are enabled, the backslash character “\” cannot be used in SPICE subcircuits as a character. An error will be generated if it is. In addition, in this case, any label between backslashes is not parsed in uppercase.

Examples

With extended identifiers enabled, the following are valid examples of usage in Eldo syntax.

- Model Declaration:

```
.MODEL \#Test\\bench\\(\#Archi\\tecture\  
+ macro lang=vhdlams
```

- Instantiating a Model:

```
y1 \#Test\\bench\\(\#Archi\\tecture\  
+ generic: \#Gene\\ric$1\=1.0 ... port: ...
```

- Declaration of a User-defined Converter:

```
.MODEL mod_name hook  
+ lib: \#Test\\bench\\(\#Archi\\tecture\  
+ \#Gene\\ric$1\=1.0 ...
```


- Explicit Insertion of Converters:

```
.hook \#To\\to\\.tata\\.Net$1\\ ...
```

- Plotting:

```
.PLOT <analysis> v(\#To\\to\\.tata\\.Net$1\\ )
```

Related Topics

Viewing Simulation Results

Instance Names of Digital Models in SPICE Subcircuits

When you instantiate a digital model from a SPICE subcircuit, you must ensure that the instance name uses the identifier syntax supported by Questa SIM.

If you use the *Y*<identifier> syntax to instantiate a digital model from a SPICE subcircuit, the instance name must not contain any of the following characters:

```
<>[]&%-+:/.~'!
```

For example, the instance name Y12-bit_adc is not supported. However, the instance name Y12bit_adc is supported.

You can use the *Y*"<identifier> syntax to instantiate a model without including "Y" at the beginning of the instance name. If you use the *Y*"<identifier> syntax to instantiate a digital model from a SPICE subcircuit, you must ensure that the instance name uses the following syntax:

- If the instance name is not a VHDL-style extended identifier, the instance name must begin with a letter or "_" and must not contain any of the characters listed above. For example, the instance name 12bit_adc is not supported because it begins with a digit.
- If the instance name is a VHDL-style extended identifier, there are no additional restrictions. For example, the instance name \12-bit_adc\ is supported.

Related Topics

Extended Identifiers

Declaring and Instantiating HDL Models

Y

Case-sensitivity for VHDL-AMS Declarations

The case of VHDL-AMS declarations is not always preserved wherever they are displayed in Questa ADMS.

The case of VHDL-AMS declarations (as written by the end-user) is preserved wherever they are displayed in Questa ADMS, with some exceptions:

- In the Structure and Wave windows, the case used is preserved for all declarations: objects (signals, constants, variables, terminals), processes and blocks. The unit names (entities, architectures, packages, configurations) are displayed in lower case. For example:

```
sim/:simple:HelloWorld:myVar
```

- When using the find and examine commands, units will be in lower case, but objects will use the same case used in the declaration, for example:

```
:simple:naturalSignal
```

- In VHDL reports, all data will be in lower case.
- In VHDL attributes 'simple_name and 'instance_name, all data will be in lower case. The following is an example of a 'instance_name attribute:

```
:simple(toto):helloworld:myvar
```

This preservation of case is controlled using the VHDL compiler variable PreserveCase (see [VHDL Compiler Control Variables](#)), or can be specified during compilation using vacom -lower and vacom -preserve.

Related Topics

[vacom \[Questa ADMS Command Reference\]](#)

5. Design Libraries

You can create, maintain, and use design libraries. A design library is special repository in which the compiled source code for the design will be stored.

Overview of Design Libraries.....5-1

Creating a Design Library..... 5-3

Specifying a Library at Compilation..... 5-4

Changing the Working Library..... 5-4

Editing Library Mappings..... 5-5

Viewing Library Contents..... 5-5

Deleting Library Contents.....5-7

Refreshing Libraries.....5-7

Maintaining Libraries Using Make Files.....5-8

Overview of Design Libraries

A design library is special repository in which the compiled source code for the design will be stored.

The first step in the Questa ADMS simulation process is to create a design library — a special repository in which the compiled source code for the design will be stored, ready to be loaded into Questa ADMS during the elaboration phase, prior to simulation.

Design libraries can be used in two ways:

- As a local working library that contains the compiled version of your design
 - As a resource library.
- The contents of your working library will change as you update your design and recompile. Only one library can be the working library.
- A resource library is typically static and serves as a parts source for your design. You can create your own resource libraries or they may be supplied by another design team or a third party.

For example, you may compile your gate-level design and test bench into a working library, and reference gate-level models located in a separate resource library, from your design.

Using Design Libraries

Before you can run the compiler, you must:

- Designate one library as the **Working Library**.
- You may create a new library using the vlib and vmap commands, or designate any existing library as the working library by using the **vasetlib** command.

If a Questa SIM design library already exists, the compiled objects inside it can be used in a mixed-signal simulation in Questa ADMS.

- Define a logical library name for any required [Resource Libraries](#) referred to in the source.

Use the [vmap/vmap](#) command to establish a logical library name (mapped to the appropriate library path name) for each user-defined library.

Working Library

The working library (logical name *work*) is the library into which design units are placed after compilation. Only one library can be set as the working library at any one time. Unless otherwise specified, the last library created is assumed to be the working library.

The path name of the working library is stored in the [System Initialization File \(modelsim.ini\)](#). This file is read when you start Questa ADMS or run one of the Questa ADMS [System Commands](#).

The library name *work* has special attributes within Questa ADMS; it is predefined in the compiler and need not be declared explicitly as would be necessary for end-user libraries described in [Creating a Design Library](#).

Caution

It is recommended that library names ending in *work* or *WORK* should be avoided, because of the potential confusion of this library name with the working library.

Resource Libraries

A resource library contains design units that can be referenced within the design unit being compiled. Any number of libraries (including the working library itself) can be resource libraries during compilation. Resource libraries may consist of resource packages supplied with Questa ADMS.

A library clause giving the logical name of the library must immediately precede any design unit that refers to a resource library. References to *work* do not require a library clause.

Note

The proprietary library `IEEE_PROPOSED` is no longer supported. If you have existing models that use this library, change the library name "IEEE_PROPOSED" to "IEEE" everywhere it appears, then recompile the affected units.

Related Topics

[Predefined VHDL-AMS Packages Supplied with Questa ADMS](#)

[Creating a Design Library](#)

[Viewing Library Contents](#)

Creating a Design Library

You can create a design library and a mapping between the logical library name and a directory.

Use the following procedure to create a working design library and create a mapping between the logical library name and a directory. Alternatively, if you already have the GUI launched, you can use the [Library Window](#) to create a new library; see [Editing Library Mappings](#).

Design libraries must be created using the vlib and vmap commands. You cannot create them using the standard UNIX mkdir command.

Procedure

1. To create a design library called <library_name>, enter the following command into a UNIX shell or the [Transcript Window](#):

```
vlib <library_name>
```

Design libraries can have any legal name allowed by the operating system, with one exception; extended identifiers are not supported. Library names are case insensitive.

If the specified library name already exists, an error message is returned and the existing library is untouched.

Note

It is also possible to use the legacy command valib to create a library. The valib command automatically performs a vmap mapping to associate the logical name to a physical design library directory if possible, whereas vlib does not perform such a mapping. Therefore, when using valib to create a library, there is no need to manually assign a logical name.

2. Before the compiler can place the compiled design units into this new library, you must assign a logical name to it. To set the mapping between a logical library name and a directory, use the following :

```
vmap <logical_name> <path>
```

For example: vmap work MYLIB will modify the *modelsim.ini* file to contain the logical-to-physical name assignment: work = MYLIB.

You can verify this mapping by repeating the vmap command, but without specifying the physical name:

```
vmap work
```

The *modelsim.ini* file is read and the message “work” maps to directory MYLIB is returned.

Related Topics

[Overview of Design Libraries](#)

[Viewing Library Contents](#)

[valib/vlib \[Questa ADMS Command Reference\]](#)

[vamap/vmap \[Questa ADMS Command Reference\]](#)

[Tutorial 1: Simulating a Mixed-Signal Design in Interactive Mode \[Getting Started with Questa ADMS\]](#)

Specifying a Library at Compilation

During compilation, use the `-work` option to specify a library in which to store the compilation results. This can be useful when you want to store the results in a different location than the current working library.

```
vacom -work <library_name> <file_name>
```

This will override the `WORK=` library setting in your [System Initialization File \(modelsim.ini\)](#) and any references to it from the design source files.

If you use this method, you can use the `-lib` option to specify the library when starting the simulator:

```
vasim -lib <library_name>
```

Related Topics

[vacom \[Questa ADMS Command Reference\]](#)

[vasim \[Questa ADMS Command Reference\]](#)

[vamap/vmap \[Questa ADMS Command Reference\]](#)

Changing the Working Library

You can define which library you want to be the default working library.

You can define which library you want to be the default working library (work) using the [`vasetlib`](#) command:

```
vasetlib <library_name>
```

`<library_name>` specifies the library path name (relative or absolute) of the default working library.

This command is strictly equivalent to a `vmap work <path>` command (see [vamap/vmap](#) in the *Questa ADMS Command Reference*).

Related Topics

[Predefined VHDL-AMS Packages Supplied with Questa ADMS](#)

[Creating a Design Library](#)

[Refreshing Libraries](#)

[Specifying a Library at Compilation](#)

Editing Library Mappings

You can associate a logical name with a library using the Library window and Edit Library Mapping dialog box.

The Library window is opened by default when you invoke the simulator and load a design.

Procedure

1. Click on the [Library Window](#) to activate the context-sensitive menus.
2. Choose **Library** > **Edit** to display the Edit Library Mapping dialog box.
3. Enter the required logical name in the Library Mapping Name field, and the physical name in the Library Pathname field.
4. Click **OK**.

Related Topics

[Creating a Design Library](#)

[Overview of Design Libraries](#)

[Viewing Library Contents](#)

Viewing Library Contents

You can use the `vadir` command to view the contents of a library. You can also display the compilation date of a design unit.

See also `vdir` in the *Questa SIM Command Reference Manual*.

Procedure

1. Enter the following command to view the contents of a library:

```
vadir [-lib <library_path>]
```

If you do not specify `<library_path>`, Questa ADMS returns the contents of the WORK design library.

2. (Optional) Enter the following command to display the compilation date of a design unit:

```
vadir -l -r [-lib <library_path>] <design_unit>
```

Questa ADMS returns the following:

```
ENTITY      TOP      (Mon Jan 29 17:04:11 2001)
  ARCH      ARCH/TOP  (Mon Jan 29 17:04:12 2001)
ENTITY      OSCMOS    (Mon Jan 29 17:04:04 2001)
  ARCH      DIGITAL_MODELSIM/OSCMOS MS (Mon Jan 29 17:04:04
2001)
```

Where the terms in the last line are:

- ARCH
The type of design unit.
- DIGITAL_MODELSIM/OSCMOS
The architecture/Entity name.
- MS
Simulated by Questa SIM.
- (Mon Jan 29 17:04:04 2001)
The compilation date.

Results

You can also view and edit library contents in the Library window. In a library, two packages with the same name may exist, one VHDL and one VHDL-AMS. The VHDL-AMS version may contain analog declarations that cannot be used in a digital context. A VHDL model will use the digital version of the package and a VHDL-AMS model will use the analog version.

Figure 5-1: Multiple Package Names in Library Window

Name	Type	Path
WORK	Library	/home/fbaray/ADMS/2011/libwindow/adms/l...
lib1	Library	lib1
absislessthan	AMS Connect Module	/home/fbaray/ADMS/2011/libwindow/adms/l...
absislessthan_vhd	AMS Entity	/home/fbaray/ADMS/2011/libwindow/adms/l...
vams	AMS Architecture	
foo	AMS Package	/home/fbaray/ADMS/2011/libwindow/adms/...
foo	Package	/home/fbaray/ADMS/2011/libwindow/adms/...

Related Topics

[vadir/vdir \[Questa ADMS Command Reference\]](#)

[Creating a Design Library](#)

Refreshing Libraries

Library Window

Deleting Library Contents

You can delete a unit from a library.

To delete a design unit from a library, use the **vadel/vdel** command:

```
vadel [-help] [-lib <path>] <design_unit> [ <arch_name>... ]
```

If no library is specified with [-lib <path>], the contents of the work design library are deleted by default.

Library contents can also be viewed, edited and deleted using the [Library Window](#).

Related Topics

[Creating a Design Library](#)

[vadel/vdel \[Questa ADMS Command Reference\]](#)

[Refreshing Libraries](#)

Refreshing Libraries

When moving to a new release of Questa ADMS, you must update libraries by recompiling or refreshing them.

Use the varefresh command to refresh design libraries that were created with a previous release:

```
varefresh {library_name}
```

where library_name is the logical or physical name of the library to update. Multiple libraries can be specified. If library_name is not included in the command then the work design library is updated.

If the specified libraries are dependant on other libraries, you must also update those libraries. If two libraries are dependant on each other, they must be updated at the same time by specifying them on the same command.

If you created libraries on a Windows or Linux platform, then they must be refreshed on the same platform. For example, if you created a library using Windows, it must be refreshed on Windows. An error valib-1053 is displayed if, for example you created a library on Windows and then try to refresh that library on Linux.

Related Topics

[varefresh \[Questa ADMS Command Reference\]](#)

[Overview of Design Libraries](#)

Maintaining Libraries Using Make Files

You can maintain libraries using GNU make.

The **vamake/vmake** command enables you to use a GNU make program to maintain your libraries. Run vamake/vmake from the UNIX command prompt on a compiled design library:

```
vmake <library_name>
```

The utility outputs a makefile. You can then run `ams_make` to reconstruct the library. `ams_make` recompiles only the design units (and their dependencies) that have changed.

If you add new design units or delete old ones, you should re-run `vamake/vmake` to generate a new makefile.

Related Topics

[Viewing Library Contents](#)

[vamake/vmake \[Questa ADMS Command Reference\]](#)

6. Compilation

Hierarchical behavioral model source code is compiled within the Questa ADMS environment.

Compilation Commands.....	6-1
Normal Compilation Example: SPICE Instantiating VHDL-AMS.....	6-3
Special Compilation Examples.....	6-4
Using Previously Compiled Design Units.....	6-10
Lock Mechanism.....	6-13

Compilation Commands

Source code in each simulation language should be compiled using the appropriate compiler command:

- For VHDL use `vcom`:

```
vcom [options] <filename> [<filename> ...]
```

The order of compilation is important; you must compile any entities or configurations before compiling an architecture that references them.

- For Verilog, use `vlog`:

```
vlog [options] <filename> [<filename> ...]
```

- For SystemVerilog, use `vlog` with the `-sv` option:

```
vlog -sv [options] <filename> [<filename> ...]
```

- For Verilog-AMS use **valog**:

```
valog [options] <filename> [<filename> ...]
```

- For VHDL-AMS use **vacom**:

```
vacom [options] <filename> [<filename> ...] [-ms <ms_options>]
```

- For SPICE use `vaspi`:

SPICE compilation is not normally necessary, but there are exceptions, as described below.

Exceptions

For the following instantiation scenarios, special care is required during compilation:

- VHDL parent instantiating a VHDL-AMS child

The VHDL component declaration must contain digital ports with the appropriate modes (in, out, inout) and types. Use the component form in your component declaration.

Refer to [Special Compilation Case: VHDL Instantiating VHDL-AMS](#).

- Verilog-AMS parent or SPICE parent instantiating a SPICE child.

A SPICE model or subcircuit can be directly instantiated in a Verilog-AMS description. The model or subcircuit name is used to reference it, as follows:

```
modelname m1 (x, y z);
subcktname s1 (p1, p2);
```

When instantiating SPICE from Verilog-AMS or SPICE, you can use `vaspi`, but it is not necessary. For “raw” SPICE (.MODEL or .SUBCKT) use direct instantiation.

Refer to [Verilog-AMS Instantiating SPICE Models and Subcircuits](#).

- VHDL instantiating SPICE.

Refer to [Special Compilation Case: VHDL Instantiating SPICE](#).

- Verilog or SystemVerilog instantiating SPICE. See the example in [Special Compilation Case: Verilog Instantiating SPICE](#).

Limitations

- When a VHDL-AMS parent instantiates a Verilog-AMS child, the following VHDL-AMS generic objects can NOT be mapped to Verilog-AMS parameters:
 - INTEGER_VECTOR
 - REAL_VECTOR
 - REAL or INTEGER aggregates from the IEEE library.
- When VHDL-AMS instantiates Verilog-AMS, VHDL-AMS terminal arrays cannot be associated with a Verilog-AMS analog or digital wire array.
- It is not possible to have Verilog instantiating Verilog-AMS instantiating Verilog. However, it is possible to have Verilog-AMS instantiating Verilog instantiating Verilog-AMS.

VHDL Input Formats

VHDL input is assumed to be in VHDL-2002 format by default (not VHDL 93). This implies some differences in the code accepted by the VHDL part of VHDL-AMS and the VHDL implemented in Questa SIM by default:

- For digital design entities precompiled by Questa SIM, the default VHDL definition is 1076-2002 (VHDL-2002).

You can use the -93 option of the vcom or vacom compiler commands to force the use of the 1076-93 (VHDL-93) syntax.

- For VHDL-AMS design entities compiled by Questa ADMS, the default definition is 1076.1-99 (VHDL-AMS-99) therefore the digital parts compiled on the Questa ADMS side are based on 1076-93 (VHDL-93).

Related Topics

[vacom \[Questa ADMS Command Reference\]](#)

[Special Compilation Examples](#)

[VHDL-AMS Subset Definition](#)

Normal Compilation Example: SPICE Instantiating VHDL-AMS

You can compile a SPICE-on-top design instantiating a VHDL-AMS child instance.

This example shows how to compile a SPICE-on-top design that instantiates a VHDL-AMS child instance.

Procedure

- child-vhdlams.vhd* contains a VHDL-AMS OP-AMP design unit, with two available architectures, *ideal* and *dominant_pole*:

```
library ieee;
use IEEE.electrical_systems.all;
entity opamp is
    generic ( Avd0 : real := 100.0, -- gain/dB
              fp1  : real := 5.0   -- pole
            );
    port (terminal
          m,          -- input minus terminal
          p,          -- input plus terminal
          outp,        -- output terminal
          m_supply,    -- V- power supply
          p_supply : electrical -- V+ power supply
        );
end entity opamp;
-- available architectures: ideal, dominant_pole
-- source code for architectures is not shown
```

Compile the VHDL-AMS entity and architecture contained in *child-vhdlams.vhd* into the current work design library using the **vacom** command:

```
vacom child-vhdlams.vhd
```

2. The VHDL-AMS OP-AMP design unit is declared in the SPICE netlist, *top-spice.cir*:

```
*RC BAND PASS WITH VHDL-AMS OP AMP model
.MODEL opamp(dominant_pole) macro lang=vhdlams
R1 1 3 12.952K
R22 3 0 846.01
R2 4 2 322.2K
C1 3 2 100N
C2 3 4 100N
Y1 opamp(dominant_pole)
+ GENERIC: Avd0=106.0
+ PORT: 4 0 2 40 70
VEE 40 0 DC -15
VCC 70 0 DC 15
V0 1 0 AC 1
.AC DEC 100 10 1000
.PLOT AC VDB(2)
.PLOT AC VP(2)
.END
```

Related Topics

Special Compilation Examples

Special Compilation Examples

This section contains a set of examples where the type of instantiation cannot be performed using the normal compilation commands alone. Extra steps are required in order for successful compilation to take place.

Special Compilation Case: VHDL Instantiating VHDL-AMS..... 6-4

Special Compilation Case: VHDL Instantiating SPICE..... 6-6

Special Compilation Case: Verilog Instantiating SPICE..... 6-9

Special Compilation Case: VHDL Instantiating VHDL-AMS

For a VHDL parent to instantiate a VHDL-AMS child, you must supply a digital interface definition. You can supply a digital interface definition in a component declaration. Alternatively, you can specifying a VHDL “target entity” in the compilation command, with which to associate the VHDL-AMS unit.

Supplying a digital interface definition is the recommended method.

- Component Declaration

Prior to compilation, put a VHDL component declaration in the architecture to define the digital interface. You must use the component form in your component instantiation. The component declaration port list should contain digital ports with the appropriate modes (in, out, inout) and types.

Then, you can compile as normal, using the **vacom** and vcom commands.

- Target Entity Instantiation

This method enables you to replace an existing VHDL child instance with a VHDL-AMS one, without editing the original source code. When compiling the VHDL-AMS instance, you specify a VHDL target entity describing its interface.

Replacing a VHDL Target Entity..... 6-5

Modifying VHDL-AMS Analog Port Mode and Type Definitions.....6-6

Replacing a VHDL Target Entity

You can associate a VHDL target entity with a VHDL-AMS entity and architecture in order for it to be instantiated from a VHDL parent instance.

In a VHDL design consisting of a parent and child model, suppose that you want to replace a precompiled VHDL child model with a VHDL-AMS model.

Procedure

1. A target entity is a standard VHDL entity declaration containing the digital interface. If the target entity does not already exist, create it and compile it using the vcom command:

```
vcom child-vhdl.vhd
```

2. Use the **vacom** command with the -target_entity option to compile the VHDL-AMS design unit that will be instantiated from VHDL:

```
vacom -target_entity child-vhdlams.vhd
```

The VHDL-AMS entity is compiled into the working library and associated with the VHDL entity of the same name. The VHDL entity provides a digital interface for use when instantiating from VHDL.

3. Compile the VHDL entity and architecture using the vcom command:

```
vcom top-vhdl.vhd
```

Related Topics

VHDL-AMS Subset Definition

Modifying VHDL-AMS Analog Port Mode and Type Definitions

Digital entities contain ports with a mode and a type. In analog entities, ports have no mode; they are defined on nature. When VHDL-AMS entities contain analog ports, you should use the `vacom` options `-target_type` and `-target_mode` to define their type and mode.

When you instantiate VHDL-AMS entities in VHDL, a corresponding digital entity is created by using the `vacom` command with the `-ams` option.

Procedure

1. Compile the VHDL-AMS description:

```
vacom -ams analog_entity.vhd
```

This generates a VHDL file for each entity, called:

```
<lib>/_adms/<ENTITY_NAME>.vhd
```

2. Modify the port mode (direction) and type of each generated VHDL entity as required.

The name of the entity cannot be modified.

3. Compile the modified VHDL files using `vcom`:

```
vcom <lib>/_adms/<ENTITY_NAME>.vhd
```

4. Compile the analog entity again, with the `-target_entity` option:

```
vacom -target_entity analog_entity.vhd
```

The compiled analog entity now uses the modified digital interface.

Related Topics

VHDL-AMS Subset Definition

Special Compilation Case: VHDL Instantiating SPICE

You must prepare a SPICE subcircuit for instantiation in a VHDL model using the `vaspi` command. You must also associate a VHDL target entity with the SPICE subcircuit.

The following syntax is used to do this:

```
vaspi <target_entity> <subckt_name>@<subckt_file_name>
```

The SPICE subcircuit can be hierarchical, but all subcircuit definitions used must be made available, either by defining them within the top-most subcircuit, or in the command file (.cmd). All other SPICE command and option statements (.TRAN, .MODEL, .OPTIONS, .INCLUDE, etc.) should be included in the command file (.cmd).

The SPICE pins can correspond to scalar VHDL-AMS port terminals of any kind in the VHDL-AMS entity declaration. However, the generics in the VHDL-AMS entity declaration must be of type STD.REAL.

Prerequisites

- The file *child-spice.ckt* contains the SPICE child declaration:

```
.SUBCKT UA741 2 3 6 4 7
* Pins:      2   Input (minus), 3   Input (plus)
*           6   Output
*           4   V- Power Supply, 7   V+ Power Supply
.MODEL NPN NPN BF = 160 RB=100 CJS=2P TF=0.3N ...
R1 1 4 1K
Q1 9 3 10 NPN
...
.ENDS
```

- The file *ua741_vhdl.vhd* contains a digital entity in VHDL for the SPICE child using the same subcircuit name ua741:

```
library ieee;
use ieee.std_logic_1164.all;
entity ua741 is
  port (
    s2 : in  std_logic;           -- Input (minus)
    s3 : in  std_logic;           -- Input (plus)
    s6 : out std_logic;           -- Output
    s4 : in  std_logic;           -- V- Power Supply
    s7 : in  std_logic);          -- V+ Power Supply
end ua741;
```

The VHDL ports listed match the SPICE subcircuit pins in number and order.

Procedure

1. Create a design library called spicelib:

```
valib spicelib
```

2. Compile the ua741 VHDL entity contained in *ua741_vhdl.vhd* into the new design library spicelib:

```
vcom ua741_vhdl.vhd
```

3. Create a SPICE architecture for the pre-compiled VHDL entity ua741. The SPICE subcircuit name is UA741, and is contained in file *child-spice.ckt*:

```
vaspi ua741 UA741@child-spice.ckt
```

Note

The port names of a module compiled and then associated with a SPICE subcircuit using `vaspi` must be legal VHDL identifiers and may not be VHDL reserved words. In particular, `in` and `out`, commonly used as identifiers in Verilog modules, are not allowed.

4. Create a design library called admslib in which the top-VHDL will be compiled:

```
valib admslib
```

5. The VHDL top design unit, *top-vhdl.vhd* uses ua741:

```
library ieee;
use ieee.std_logic_1164.all;
library spicelib;           -- spicelib contains ua741
entity top is end top;
architecture vhd1 of top is
    signal inpm : std_logic := '0';      -- Input (minus)
    signal inpp : std_logic := '0';      -- Input (plus)
    signal outp : std_logic;             -- Output
    signal vpls : std_logic := '1';      -- V- Power
    signal vmin : std_logic := '0';      -- V+ Power
begin -- vhd1
    Xua741: entity spicelib.ua741 port map (s2 => inpm, s3 => inpp, s6
=> outp, s4 => vpls, s7 => vmin);
end vhd1;
```

6. Compile the top-level VHDL entity contained in *filetop-vhdl.vhd* into the design library admslib:

```
vcom top-vhdl.vhd
```

Related Topics

Special Compilation Examples

Special Compilation Case: Verilog Instantiating SPICE

You can compile a Verilog-on-top design that instantiates a SPICE subcircuit.

This example shows how to compile a Verilog-on-top design that instantiates a SPICE subcircuit.

A SPICE subcircuit can be hierarchical, but all subcircuit definitions used must be defined within the top-most subcircuit, or in the command file (.cmd) used for simulation. All SPICE command and option statements (.TRAN, .MODEL, .OPTION, .INCLUDE, etc.) should be included in the command file (.cmd).

Prerequisites

- A Verilog module ua741 is declared in the source file *ua741_verilog.v*:

```
`timescale 1ns/1ns
module ua741(s2, s3, s6, s4, s7);
    input  s2, s3; // Inputs (minus, plus)
    output s6;     // Output
    input  s4,s7;  // V-, V+ Power Supplies
endmodule // ua741
```

- top-verilog.v* uses the new design unit:

```
`timescale 1ns/1ns
module top_verilog();
    reg inpm, inpp, vpls, vmin;
    wire outp;
    initial begin
        inpm=1'b0;
        inpp=1'b0;
        vpls=1'b1;
        vmin=1'b0;
    end
    \spicelib.ua741 Xua741(inpm, inpp, outp, vpls, vmin);
endmodule // top_verilog
```

The Verilog parent module lists the ports and parameters of the SPICE subcircuit. The Verilog ports listed need to match the SPICE subcircuit pins in number and order.

Procedure

1. Create a design library called spicelib:

```
valib spicelib
```

2. Compile the ua741 Verilog parent module contained in *ua741_verilog.v* into the design library:

```
vlog ua741_verilog.v
```

3. Use the vasp command to create a SPICE architecture and attach it to the ua741 Verilog module. The SPICE subcircuit name is UA741, and is contained in *child-spice.ckt*:

```
vaspi ua741 UA741@child-spice.ckt
```

Note

The port names of a module compiled and then associated with a SPICE subcircuit using vasp must be legal VHDL identifiers and may not be VHDL reserved words. In particular, in and out, commonly used as identifiers in Verilog modules, are not allowed.

The SPICE subcircuit can now be used in the Verilog parent design by simply instantiating the new module inside of *top-verilog.v*.

4. Create a design library called admslib:

```
valib admslib
```

5. Compile the top-level Verilog module contained in *top-verilog.v* into the admslib design library:

```
vlog top-verilog.v
```

Related Topics

[Instantiating a SPICE Subcircuit from VHDL, VHDL-AMS or Verilog](#)
[Special Compilation Examples](#)

Using Previously Compiled Design Units

Pure digital design units located in pre-existing Questa SIM design libraries can be instantiated in a mixed-signal design.

This can happen provided that the compiler variable `CreateSource=1` was set in the `modelsim.ini` file when the digital design unit was compiled with the `vlog` or `vcom` digital compiler commands. The digital design unit will be available upon elaboration in Questa ADMS.

If you want to use design units that are located within a write-locked library, you must set the environment variable:

```
ADMS_RONLY_LIB_IMPORTS = <directory>
```

This must be a writable directory. Additionally, you cannot use a single `ADMS_RONLY_LIB_IMPORTS` directory for 2 write-locked libraries that have the same simple name.

The `valog -ms`, or `vacom -ms` commands force the digital design unit to be recompiled at compile time rather than at elaboration time.

If the digital design unit is modified after the mixed-signal design has been compiled, the digital design unit will no longer be available to Questa ADMS.

Sharing VHDL Packages.....	6-11
Importing Design Units Manually.....	6-11
Using Libraries of Design Units Compiled on Other Platforms.....	6-12

Sharing VHDL Packages

VHDL packages can be shared.

When a VHDL package has been compiled by the `vcom` command, it is available to all VHDL and VHDL-AMS design units that reference the package. If a VHDL-AMS design (compiled with the **`vacom`** command) contains a reference to a digital VHDL package, The digital VHDL package will be automatically recompiled for VHDL-AMS the first time a reference to the package is encountered during elaboration.

This step requires that the library is not write-locked. If it is, then the results are written to the work library instead. If the library is write-locked, ensure that the work library is read and write enabled.

If the package includes items that are not part of the **VHDL-AMS Subset Definition** then warnings will be issued, and the unsupported declarations will be ignored.

A VHDL-AMS package (containing items like terminals) must be compiled with the **`vacom`** command. It cannot be shared with VHDL design units.

Importing Design Units Manually

You can import pre-compiled digital design entities into Questa ADMS for use in mixed-signal simulation.

The **Questa > Questa ADMS Import Library Dialog Box** is used to select and transfer design units from Questa SIM into Questa ADMS.

Note

Digital design units that are to be imported for use in a mixed-signal design must have been compiled with the CreateSource switch set to 1 in the *modelsim.ini* file. Refer to **VHDL Compiler Control Variables** for more information.

Procedure

1. Enter the command **import_ms** into the command line prompt or the **Transcript Window**. This displays the **Questa > Questa ADMS Import Library Dialog Box**.
2. Select the Questa ADMS library in the Library field. The list of design units is populated with all the items present in the associated Questa SIM library.
3. Select items from the list of design units and use the transfer buttons (>, >>, <, <<) to move them into the items to be transferred list on the right. > and < will move only the selected items between the lists; >> and << will move all items from one list to the other.
4. Click **Compile**.
The items in the items to be transferred list are recompiled for mixed-signal use, and transferred to Questa ADMS.
5. The dialog box remains open, so it is possible to select another library and continue. Otherwise, click **Close**.

Results

You can choose to run **import_ms** in batch mode using the -c option, specifying the path to the required design units.

Using Libraries of Design Units Compiled on Other Platforms

A Questa ADMS library compiled on a 64-bit platform may access a Questa ADMS library that was created on either a 64-bit platform or a 32-bit platform.

The following limitations apply when accessing a library that was compiled on a different a platform:

- The physical location of the library must be the same for end-users on all platforms, regardless of whether the library was created on a 64-bit or a 32-bit platform.
- You must have write access to the library. This is because *vasim* automatically recompiles the required design units on-the-fly before starting the simulation.

A write-locked library created on a 32-bit platform can be extended for use on a 64-bit platform by unlocking the library, executing **valib -update** on the 64-bit platform, and then relocking the library.

Related Topics

[vasim \[Questa ADMS Command Reference\]](#)

[valib/vlib \[Questa ADMS Command Reference\]](#)

[Importing Design Units Manually](#)

[Design Libraries](#)

[import_ms \[Questa ADMS Command Reference\]](#)

Lock Mechanism

Questa ADMS automatically creates a `_lock` file in a library to prevent simultaneous compilation updates to the library. The file is removed at the end of compilation.

If the compilation commands terminate abnormally, then the lock files may not be removed.

Unlock the library by manually removing the `<library>/__adms_lock` and/or `<library>/__adms_import_lock` files.

7. Simulation

The simulation process includes preparing a design, compiling it into a design library, and preparing it for simulation.

Interactive Simulation.....	7-1
Batch Mode Simulation.....	7-10
Viewing Simulation Results.....	7-19
Automating Simulation Processes.....	7-25
Methods for Setting Simulation Parameters.....	7-32
Simulation Save and Restart.....	7-33
Simulator Command Time Units.....	7-50
Optimizing Simulation.....	7-50
Monte Carlo Analysis.....	7-68

Interactive Simulation

The simulator can be run in interactive mode. The design is loaded into Questa ADMS and the graphical user interface is used to view design hierarchy and source code, run simulations, interrogate the design and view the results.

Running an Interactive Simulation.....	7-1
Examining Design Files in the Source Window.....	7-5
Accessing Language Templates.....	7-7
Running Commands from the Transcript Window.....	7-8
Changing Values in the Locals Window.....	7-10

Running an Interactive Simulation

You can load a design into the Questa ADMS simulator, interact with the design using the GUI, then run a simulation.

Prerequisites

- Ensure that all required simulation options such as the use of extended identifiers and simulation output format have been correctly defined in the *modelsim.ini* file; see [Simulator Control Variables](#).
- Your design files should be compiled into a design library, see [Creating a Design Library](#) and [Compilation Commands](#).
- For HDL-on-top designs, you must create a [Questa ADMS Command File](#) that contains the required simulation control commands. Command files can be written using a simple text editor, or can be created when loading the design using the [Start Simulation Dialog Box](#). For SPICE-on-top designs, the required commands can be included in the netlist circuit (.cir) or subcircuit (.ckt) file.

Procedure

1. Enter the **vasim** command at the command line prompt:

```
vasim
```

The **Start Simulation Dialog Box** is displayed.

2. Specify the Top Design to simulate, either Compiled-HDL (for VHDL-AMS, Verilog-AMS, VHDL or Verilog) or Spice (for SPICE on-top designs).
3. In the Outpath field, specify the directory to which simulation data should be output, or leave it blank to output to the current working directory.
4. In the Library field, specify the Questa ADMS library containing your compiled design files.
5. If your design uses HDL-on-top, select the top-level design unit(s) in the Design Unit Selection frame and specify the required command file in the Command File / Eldo Design field. You can also click **New** to create a new file using the **Eldo Commands Dialog Box**.

Note

Command files are required when your mixed-signal design has HDL-on-top. For SPICE-on-top designs, simulation control commands can be included in the netlist circuit (.cir) or subcircuit (.ckt) file.

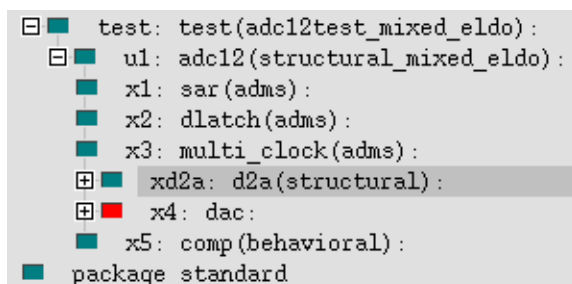
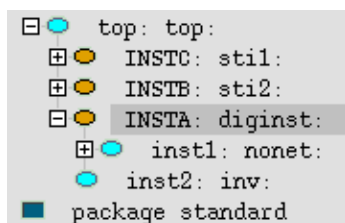
6. Optionally, you can specify digital-specific Questa SIM options in the User Questa SIM Arguments field.
7. Click **Load**. When all the required design units are compiled and the design is loaded ready for simulation, it is said to be elaborated. The **Structure Window** displays the hierarchy of your design, depending on which modeling language establishes the top level. The following figures show examples of design hierarchies as they appear in the **Structure Window**:

Figure 7-1: SPICE-On-Top Design Structure Example

```

top_spice: Top SPICE
├─ y1: top_level(struct):
│   ├── y2: exposure(time):
│   ├── y3: enable_exp(lsc):
│   └─ x1: analog_block:
│       ├── ydefhook#inl1_0#top_spice: d2a(ams):
│       ├── ydefhook#inl2_1#top_spice: d2a(ams):
│       ├── ydefhook#inp_2#top_spice: d2a(ams):
│       ├── ydefhook#inr_3#top_spice: d2a(ams):
│       └─ package standard

```

Figure 7-2: VHDL-AMS-On-Top Design Structure Example**Figure 7-3: Verilog-On-Top Structure Example**

The language of each design unit in the hierarchy is identified by the shape and color of its icon, as described in [Table 7-1](#).

Table 7-1: Design Object Icons

Icon	Design Unit	Icon	Design Unit
	VHDL-AMS		Eldo SPICE Device
	Verilog-AMS		MODSST Algorithm
	VHDL		SPICE and Fast-SPICE
	Verilog		Fast-SPICE
	SPICE		Fast-SPICE Black-Box
	SPICE Black-Box		Foreign SPICE

- Double-click on an instance in the Structure window to display the source code in the Source window:

Figure 7-4: Viewing Source Code in the Source Window

```

15      GENERIC MAP(10 ns, 3 ns)
16      PORT MAP(clk1, clk2);
17      x4: ENTITY dac(behavioral)
18      PORT MAP (q, Tvbg, Tvout);
19      x5: ENTITY comp(behavioral)
20      PORT MAP (Tvout, Tvin, outcomp_d);
21
22  END ARCHITECTURE structural_mixed_ms;
23
24  -----
25  ARCHITECTURE adc12test_mixed_ms OF test is
26      TERMINAL Tvbg, Tvin: Electrical;
27      SIGNAL comset: Bit;
28      SIGNAL eoc: Bit;
29      SIGNAL result: bit_vector(11 DOWNT0 0);
30      QUANTITY Vvbg ACROSS Ivbg THROUGH Tvbg;
31      QUANTITY Vvin ACROSS Ivin THROUGH Tvvin;
32  BEGIN
33
34      U1: ENTITY work.adc12(structural_mixed_ms)
35      PORT MAP (Tvbg, Tvin, comset, eoc, res);
36
37      Vvbg == 1.15;
38      Vvin == 1.275;
39      comset <= '0', '1' AFTER 1 ns, '0' AFTER 2 ns;
40
41  END ARCHITECTURE adc12test_mixed_ms;
42

```

9. (Optional) Your command file or netlist may already contain **add wave** or **.PLOT** commands, to select signals for inspection after simulation is run. If it does not, or to add further signals to the **Wave Window (Solido Waveform Analyzer)** manually, right-click the required signals in the **Objects Window** and choose **Add Wave**.
10. Choose **Simulate > Run > Run All** to run the simulation.
11. To stop the simulation, choose **Simulate > Break** from the main menu or click the **Break** button on the **Simulate Toolbar**.
 You can stop the simulation at any time, either to end the session completely, or break the session, allowing changes to the simulation configuration to be made before resuming.
12. To continue a simulation after a break, choose **Simulate > Run > Continue**.
13. To start a new simulation session and load a new design, choose **Simulate > Start Simulation**.
14. Choose **Simulate > End Simulation** to stop the current simulation and quit the simulation session. You will be asked to confirm this operation.

Table 7-2 describes other options on the **Simulate > Run** menu used to start, continue and restart simulations in different ways.

Table 7-2: Simulate > Run Menu Options

Menu Item	Action
Simulate > Run > Run	Runs the simulation for one run length. Run length is specified using the field on the Simulate Toolbar .
Simulate > Run > Run-All	Runs the simulation for the time defined either by the .TRAN command or TIME'High for digital designs if no .TRAN command is specified.
Simulate > Run > Continue	Continues the simulation after a break has been performed.
Simulate > Run > Run-Next	Runs the simulation until the next digital event time.
Simulate > Run > Run-Next BeginOfAsp	Runs the simulation until the beginning of the next analog solution point: DC, AC or Transient. This action may be associated with a hint of a quantity using the Force Selected Signal Dialog Box .
Simulate > Run > Run-Next Asp	Runs the simulation to the end of the current analog solution point, or the next if the current one is already finished.
Simulate > Run > Run-Quiescent	Runs the simulation until the end of the quiescent point computation.
Simulate > Run > Step	Steps to the next statement.
Simulate > Run > Step-Over	Performs VHDL procedures and functions, and Verilog/Verilog-AMS tasks and functions.
Run > Restart	Reloads the design elements and resets the simulation time to zero. Only design elements that have changed are reloaded; you specify whether to maintain various objects such as logged signals or breakpoints.

Related Topics

[Examining Design Files in the Source Window](#)

[run \[Questa ADMS Command Reference\]](#)

[Basic Mixed-Signal Simulation \[Getting Started with Questa ADMS\]](#)

[Adding Signals to the Solido Waveform Analyzer Wave Window](#)

Examining Design Files in the Source Window

You can use the Source window to view and edit source code, command files, subcircuits (.ckt), and netlist (.cir) files within your design.

Procedure

Do one of the following:

If you want to...	Do the following...
View source code in the Source window.	Either: - Double-click on an instance in the Structure window. - Right-click on an instance in the Structure window and choose View Declaration, or choose Structure > View Declaration from the main menu. - Enter the following command in Transcript window: <pre>view source</pre> - Choose File > Open from the Source menu. The pathname of the source file is indicated in the header of the Source window. Each file you open or create resides within a separate tab in the Source window. Files displayed in the Source window are identified by icons as described in Table 7-3.
Copy and paste text between the Source window and the Transcript window.	Use the buttons on the Standard Toolbar .
Place a marker to indicate the current execution line.	Enter the following command in the Transcript window: <pre>see</pre> Questa ADMS displays 10 lines of the source file and places a marker to indicate the current execution line. You can specify how many lines to display. Refer to <i>see</i> in the <i>Questa SIM Command Reference Manual</i> for more information.
View a process.	Select an item in the Structure window, then select the process in the Process window. An arrow next to the line numbers indicates the selected process. You can use the <code>vacom -nodebug</code> argument to keep the design structure hidden.
Make a file editable.	Right-click in the Source window and uncheck Read Only. By default, files open in read only mode. To change the default behavior, set the <code>PrefSource(ReadOnly)</code> variable to 0. Refer to

If you want to...	Do the following...
	Changing GUI Preferences for more information about setting preference variables.

Results

Table 7-3: Source Window File Icons

Icon	File Type
	ASCII Text File
	SPICE File
	VHDL-AMS Source File
	Verilog-AMS Source File

Related Topics

[Simulation Save and Restart](#)

[Simulate Toolbar](#)

[Running an Interactive Simulation](#)

[run \[Questa ADMS Command Reference\]](#)

[step \[Questa ADMS Command Reference\]](#)

Accessing Language Templates

Language templates are provided to help you write correct HDL code. They are a collection of HDL templates that are intended as an interactive “reference” for creating small sections of code for new designs, test benches, language constructs, or logic blocks.

Procedure

Do one of the following:

If you want to...	Do the following...
Access language templates if the source window is docked.	Choose Source > Show Language Templates .
Access language templates if the source window is undocked.	Either: Choose the  icon on the Source Window toolbar.

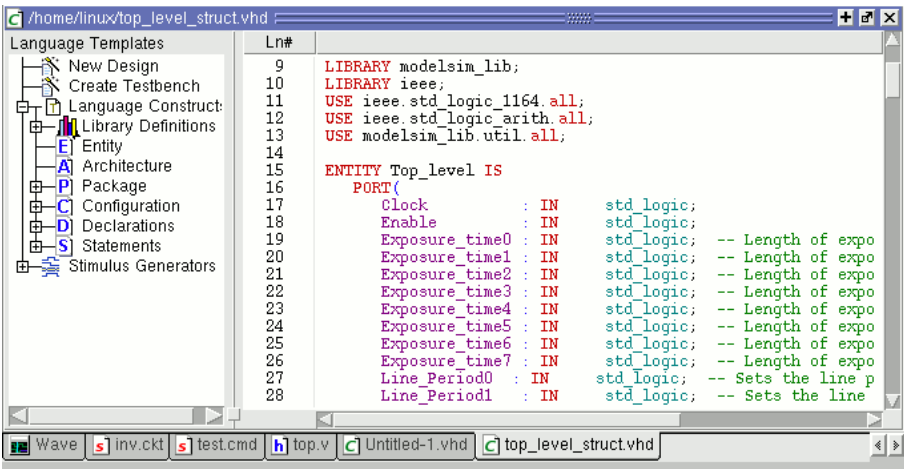
If you want to...	Do the following...
	Choose View > Show language templates.

Results

The Language Templates panel will be displayed on the left hand side of the Source window.

Examples

Figure 7-5: Source Window with Language Templates Pane



Related Topics

[Source Window](#)

Running Commands from the Transcript Window

The Transcript window maintains a running history of commands that are invoked and messages that occur as you work with Questa ADMS. When a simulation is running, the Transcript displays a command prompt, allowing you to enter command line commands from within the GUI.

Procedure

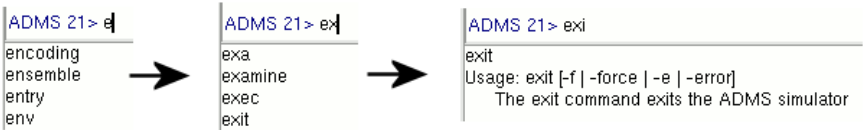
Do one of the following:

If you want to...	Do the following...
Run a command for the first time.	Enter the command at the prompt in the Transcript window. A dropdown box appears that lists the available commands matching the string typed so far. Use the Up and Down keys or the mouse to select the required command. Once a unique command

If you want to...	Do the following...
	is complete, the command usage is presented in the dropdown box. Press the Tab key to complete the command name. You can disable command completion by choosing Help > Command Completion .
Run a previously run command.	Press the Up and Down keys to display the commands that have previously been run in the Transcript window.
Place the cursor at the beginning or end of the command line.	Press Home to place the cursor at the beginning of the command line. Press End to place the cursor at the end of the command line.
Clear the Transcript window.	Enter the following command in the Transcript window: <code>.main clear</code> This command clears the Transcript window only. The contents of the <i>transcript</i> file are not affected by this command. Tip By default, the Transcript window retains the last 5000 lines of output from the transcript. To change this default, choose Transcript > Saved Lines. Setting this variable to 0 instructs Questa ADMS to retain all lines of the transcript.
Disable creation of the transcript file.	Enter the following command in the Transcript window immediately after Questa ADMS starts: <code>transcript file ""</code>

Examples

Figure 7-6: Transcript Window — Command Help As You Type



Related Topics

Manually Saving the Transcript File

Changing Values in the Locals Window

You can change the name or value of a variable, constant, or generic of a behavioral model displayed in the Locals window.

If you want to change the value of any constant, VHDL-AMS models using such constants must be compiled using **vacom** -constants.

Changing the value of a parameter or generic will not modify any design elements that depended on that parameter or generic during elaboration (for example, sizes of arrays).

Note

It is not possible to change values of constants or variables of a Verilog-A module computed by ADiT.

Changes can only be applied to generics of VHDL, Verilog, VHDL-AMS, Verilog-AMS behavioral models, but not to SPICE parameters (defined using **.PARAM**, or passed at instantiation).

Procedure

1. If the **Locals Window** is not already visible, choose **View > Locals** or use the command view locals to display it.
2. In the **Structure Window**, select a design object with local variables defined, then either:
 - Right-click and select **Change**
 - Choose **Locals > Change**

The change dialog box is displayed.

3. If required, specify a new Variable Name. In this case, Variable Name also signifies VHDL-AMS constants and generics.
4. If required, specify a new (valid) Value for the variable and Click **OK**.

An array value must be specified as a string (not enclosed in quotation marks).

Related Topics

[Locals Window](#)

[vacom \[Questa ADMS Command Reference\]](#)

Batch Mode Simulation

To start simulating in batch mode, use the **vasim** command with the **-c** option.

The interactive GUI will not start, therefore you must provide all the instructions for simulation on the command line. You can include simulator commands in the SPICE netlist file, or in a dedicated command file. Refer to [Automating Simulation Processes](#).

The following example invokes a design with SPICE-on-top, specifying the name of the netlist file using the -cmd option.

```
vasim -c -cmd file.cir
```

Progress messages will be reported as the simulation runs. For long simulations, elapsed time and elapsed CPU time will be reported.

Tip
To stop a running simulation in batch mode, use the keyboard shortcut CTRL+C.

[Passing Options Directly to Eldo and ADiT.....](#) 7-11

[Multiple-Top-Design Simulation.....](#) 7-12

[Multiple-Run Analyses.....](#) 7-13

Passing Options Directly to Eldo and ADiT

You can specify Eldo or ADiT options directly on the Questa ADMS command line.

You can specify Eldo or ADiT options directly on the Questa ADMS command line, using either the vasim command line option:

```
vasim -eldoopt <option> | -aditopt <option>
```

Or by specifying the Eldo option:

```
.OPTION ELDOOPT <option> | ADITOPT <option>
```

Related Topics

- [vasim \[Questa ADMS Command Reference\]](#)
- [Multiple-Run Analyses](#)
- [Simulation Save and Restart](#)

Multiple-Top-Design Simulation

It is possible to test a design unit using one or more design units as stimuli. When simulating multiple-top designs, you can specify two or more top design units.

The first top design unit can be the top of any legal ADMS hierarchy; the second and subsequent tops must be pure digital design units.

For example:

- You can specify multiple-top-design entities in this order: Verilog-AMS, VHDL, Verilog.
- However the following order cannot be simulated: Verilog-AMS, VHDL-AMS, Verilog.

The syntax to invoke the simulator and launch a multiple-top design simulation is:

```
vasim [-lib <library_pathname>] adms_ms_du {[MS_library.]MS_du}
```

where:

- -lib <library> is the name of the library where the first top design unit is compiled. If omitted, Questa ADMS uses the current working library.
- adms_ms_du is the name of the first top-level design unit. This can be a Verilog, SystemVerilog, Verilog-AMS, VHDL, VHDL-AMS, or SPICE design unit. For VHDL or VHDL-AMS entity, if architecture is specified, it must be enclosed within quotation marks, for example:

```
"entity(architecture)"
```

- MS_library is the name of the library containing the digital design unit MS_du. If the optional MS_library is specified, it must be enclosed within quotation marks, for example:

```
vasim ... "library.entity(architecture)" "library.module"
```

Note

If MS_library is omitted, Questa ADMS will look in the current working library.

Multiple top design units are displayed in the [Structure Window](#). The Structure window hierarchy shown in [Figure 7-7](#) shows a multiple-top design that contains a top VHDL-AMS unit called *top(vhdlams)*, a top VHDL design unit called *vhdl_top(pure_digital)*, and a top Verilog design unit called *verilog_top*. The design was loaded using the following command:

```
vasim -cmd vhdlams-top.cir top(vhdlams) vhdl_top(pure_digital) verilog_top
```

Figure 7-7: Multiple Top Design Units (Structure Window)

```

■ Command File: vhd1ams-top.cir
☐ ■ top: top(vhd1ams):
    ■ ycontrol: econtrol(behavioral):
    ■ ynot1: nand_bit(behavioral):
    ■ ynot2: inverter_bit(behavioral):
    ■ ynot3: inverter_bit(behavioral):
    ■ ynot4: inverter_bit(behavioral):
    ■ ynot5: inverter_bit(behavioral):
    ■ vhd1_top: vhd1_top(pure_digital):
    ☐ ● verilog_top: verilog_top:
        ■ package standard
        ■ package std_logic_1164
        ■ package standard
        ■ package std_logic_1164

```

Related Topics

[vasim \[Questa ADMS Command Reference\]](#)

[Running an Interactive Simulation](#)

Multiple-Run Analyses

You can use Multiple-run analyses to perform multiple simulation runs, with changing values of the selected parameters.

Overview of Multiple-Run Analyses.....	7-13
.ALTER Blocks.....	7-17
Re-Running Specific .ALTER Blocks.....	7-17

Overview of Multiple-Run Analyses

Multiple-run analyses enable the automatic execution of multiple simulation runs (a “sweep”), each with a different value for the selected parameters.

Any parameter declared in a **.PARAM** command in the SPICE context can be swept. To execute a sweep, use one of the following sweep commands: **.STEP**, **.DATA**, **.MC**, **.TEMP**, and **.DC**.

Place sweep commands in the command file (*.cmd*) for HDL-on-top designs, or in the top-level netlist (*.cir*) file, for SPICE-on-top designs. Commands may be contained within a file specified with a **.INCLUDE** command from either of these contexts, and **.INCLUDE** commands may be nested to any depth.

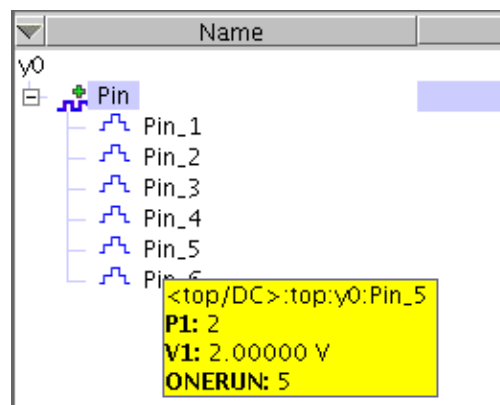
A sweep command calls out a set of parameters declared elsewhere in the design, and generates an ordered set of parameter value vectors containing a value for each parameter, according to the rules specific to that command (see [Running Parametric Analyses](#) in the *Eldo User's Manual*).

If the parameter to be swept is declared in a nested context, then the parameter name in the command must be preceded by a path name in the format described in [Writing Hierarchical Path Names](#).

A swept parameter can be used anywhere a SPICE parameter is allowed; for example, as the parameter of a SPICE primitive, or as an actual value associated with a generic parameter of a Verilog module instantiated from SPICE. Only real-valued parameters are allowed for high-level language actual parameters.

Waveform data is output in the form of compound waveforms, which can be treated as a unit in the Solido Waveform Analyzer viewer for display, waveform calculations and measurements. To see the run information for a particular waveform, hover the mouse over the waveform in the Solido Waveform Analyzer Waveform List.

Figure 7-8: Run Popup



A DC sweep analysis (.DC command) corresponds to a succession of quiescent computations, where the quiescent state is calculated for each incremental step. By observing the circuit response, you can identify some hysteresis effects. DC sweep analysis can be performed on the following: circuit component, global circuit temperature (TEMP), global circuit parameter (PARAM), voltage or current source, or be a data-driven sweep (DATA).

Commands

Use the [onRunDone](#) command to specify a number of commands to be performed at the end of each run of a multiple-run simulation. Once set, the commands specified with onRunDone are performed each time a simulation run completes. Specifying onRunDone without arguments will list the currently registered commands.

Note

The onRunDone registered commands are only performed if the simulation is a multiple-run simulation.

Use the [onRunDCDone](#) command to define a block of commands in a do file to be executed after each DC simulation in mprun, multi_run and simple contexts. Used with at least one command specified as an argument, onRunDCDone registers the specified command(s), overwriting any previously registered commands. If no arguments are specified, the command lists the currently registered onRunDCDone

command(s). If the simulation is only a .dc simulation, onRunDCDone will be executed immediately after onRunDone.

Use the **onRunStart** command to define a block of commands in a do file to be executed at the beginning of every run of a multiple-run simulation, or in a single run for reuse of do-files across testbenches. Used with at least one command specified as an argument, onRunStart registers the specified command(s), overwriting any previously registered commands. If no arguments are specified, the command lists the currently registered onRunStart command(s).

During a simulation, use the following Tcl commands to return information about a multiple-run simulation:

- **multipleRun**—Returns 1 if the simulation is a multiple-run simulation
- **oneRun**—Returns the value specified by `vasim -onerun`, or 0 if the `-onerun` argument was not used
- **alterName**—Returns the name specified by `vasim -alter`, or an empty string if `-alter` is not present.

The Verilog system function `$get_run_id()` returns a unique identifier corresponding to the current simulation run in a multiple-run simulation. It returns 0 if the simulation is not a multiple-run simulation.

Note

`get_run_id` is only available in a Verilog or SystemVerilog module and in the digital part of a Verilog-AMS module. It is not available in the analog part of a Verilog-AMS module.

Distributed Simulations

You can use the Eldo `.MPRUN` command to submit batches of runs as sub-processes on one multi-processor machine, or on many machines. Refer to [Running Distributed Multiple Runs](#) in *Eldo User's Manual* and `.MPRUN` in *Eldo Reference Manual* for more information.

The location of the standard output of each sub-process is as follows:

- If `.MPRUN LOGFILE=YES` (default), Questa ADMS saves the standard output of sub-process number X in the file `<netlist_name>.partX.sim.log` in the output directory.
- If `.MPRUN LOGFILE=NO`, Questa ADMS does not save the standard output of the sub-processes. The standard output of sub-processes running on the local host is interspersed with the standard output of the main process. The standard output of sub-processes running other hosts is unavailable.

If you use the LSF load balancer, Questa ADMS saves the standard output for sub-process number X in the file `<netlist_name>.partX.log` in the output directory. The standard output includes status information about the LSF job. The `.MPRUN LOGFILE` parameter has no effect if you use the LSF load balancer.

Examples

- In this example of multiple-run analysis using HDL-on-top, a global parameter VAL is stepped. This step will be applied to every instance of the subcircuit inv. The command file contains the following commands:

```
* .cmd Command file
...
.PARAM VAL=1u
.STEP PARAM VAL=1u 5u 1u
...
```

The following is the SPICE subcircuit for the instance inv, referencing the parameters that are being stepped in each independent simulation run:

```
.SUBCKT inv INP OUTP
M1 OUTP INP Vcc Vcc PCH L=0.25u w=val M=1
M2 OUTP INP Vcc Vcc NCH L=0.25u w=val M=1
.ENDS inv
```

- In this example, each time a simulation run completes, coverage data is saved to a separate UCDB file, named *file<n>.ucdb*:

```
set i 1
onRunDone { coverage save "file$i" ; set i [expr $i+1] }
run -all
```

Tip

Use the vcover merge command to merge the separate files into one UCDB.

Refer to [Capturing Multiple-Run Simulation Data in a UCDB](#) to see a different method for capturing this data.

- In this example, the simulator is instructed to create a coverage report file containing assertion and code coverage data for each run at the end of each simulation run:

```
set i 1
onRunDone {coverage report -assert -code s -details -file
"report_$i.txt" ; set i [expr $i+1] }
run -all
quit
```

Related Topics

[Automating Simulation Processes](#)

[Capturing Multiple-Run Simulation Data in a UCDB](#)

Simulation Save and Restart

.ALTER Blocks

You can re-run a simulation with a modified netlist. Use the .ALTER command to specify blocks containing different parameters and data.

When the simulation is run, a netlist containing .ALTER blocks is interpreted as follows:

- 1. The simulator reads the netlist up to the first .ALTER statement and performs the analyses. This is the first (nominal) simulation run.
- 2. The block of statements between the first .ALTER and .END (or another .ALTER) statements is read, and the netlist is modified using these statements.
- 3. The simulation is re-run with the modified netlist.
- 4. If subsequent .ALTER blocks exist, the nominal netlist is modified using these statements and the simulation is re-run. The initial netlist is always used as the reference to be altered, not the result of the previous .ALTER command.

If there is an error in a .ALTER block, it will be skipped and the next .ALTER block will be used.

Tip

You can use a do file to define a set of commands to be executed on each run of a .ALTER session. See the [onRunStart](#) command in the *Questa ADMS Command Reference*.

See [Altering Netlist Data For Simulation Reruns](#) in the *Eldo User’s Manual* and [.ALTER](#) in the *Eldo Reference Manual*.

Related Topics

Re-Running Specific .ALTER Blocks

Re-Running Specific .ALTER Blocks

You can run a specific .ALTER block without first running the main netlist.

Procedure

Do one of the following:

If you want to...	Do the following...
Run a specific .ALTER block without first running the main netlist.	Use the following <code>vasim</code> command argument: <code>vasim ... -alter <alter_label> ...</code>

If you want to...	Do the following...
	Where <alter_label> is the corresponding name or index of the required .ALTER block. Specifying an index of 0 executes the nominal run. Only one .ALTER block can be specified with <code>vasim -alter <alter_label></code>.
Select which scheduled run to execute, if control commands in the .cmd file or in the top-level SPICE file schedule multiple simulation runs.	Use the following <code>vasim</code> command argument: <pre>vasim ... -onerun <single_run_index> ...</pre> Where <single_run_index> specifies the index of the specific single simulation run.
select one .ALTER block for execution and execute a specific run, if the netlist contains .ALTER blocks that schedule multiple runs.	Use the following <code>vasim</code> command arguments: <pre>vasim ... -alter <alter_label> -onerun <single_run_index> ...</pre>

Note

To see the ONERUN single run index and the alter information for a particular waveform, hover the mouse over the waveform in the Solido Waveform Analyzer Waveform List.

Examples

The following netlist contains 2 .ALTER blocks:

```
v1 1 0 p2
r1 1 2 p
c1 2 0 1 ic=0
.plot tran v(2)
.tran 1 10 uic
.param p =1
.param p2=3
.step param p 1 10 1

! first alter run
.alter one
.step param p 1 10 1
.step param p2 10 20 1

! second alter run
.alter two
.step param p 1 10 2
.end
```

- To run the 3rd run of the first .STEP, use:

```
vasim -onerun 3 # param p=3, p2=3
```

or

```
vasim -alter 0 -onerun 3
```

- To run the 4th run of the second .STEP nested in 5th run of the first .STEP and defined in .ALTER block "one":

```
vasim -alter "one" -onerun 48 # param p=5, p2=13
```

- To run the 2nd run of the .STEP defined in .ALTER block 'two':

```
vasim -alter "two" -onerun 2 # param p=3, p2=3
```

- To run only .ALTER block "two"

```
vasim -alter "two"
```

Related Topics

[.ALTER Blocks](#)

Viewing Simulation Results

Simulation results are viewed as waveforms using the Wave window (Solido Waveform Analyzer) or the Questa SIM Wave window.

- [Wave Window \(Solido Waveform Analyzer\)](#)

[Wave Window \(Solido Waveform Analyzer\)](#)

Analog and digital waves can be viewed at the same time, in the Solido Waveform Analyzer Wave Viewer, displayed as a window inside Questa ADMS. Choose **View > Wave > Questa ADMS** to view it.

- [Questa SIM Wave Window](#)

To view waveforms using the Questa SIM Wave window in Questa ADMS, choose **View > Wave > QuestaSim**.

[Which Waveform Viewer Should I Use?.....](#) 7-19

[Adding Signals to the Solido Waveform Analyzer Wave Window.....](#) 7-20

[Waveform Database File Formats.....](#) 7-22

Removing Items from the Solido Waveform Analyzer Wave Window.....	7-23
Adding Items to the Log File.....	7-24
Incremental Saving Mode.....	7-25

Which Waveform Viewer Should I Use?

One viewer may provide better tools for analyzing a given type of simulation result.

Some waveforms can be usefully represented in both the Solido Waveform Analyzer and Questa SIM wave windows, such as time sequences of bits, real numbers, or enumeration values. However, in some cases, one viewer will provide better tools for analyzing a given type of simulation result. For example:

- The Solido Waveform Analyzer Wave window can represent a function of frequency but the Questa SIM Wave window cannot.
- The Questa SIM Wave window can represent a generalized record but Solido Waveform Analyzer cannot.
- The Solido Waveform Analyzer Wave window can generate an eye diagram.
- The Questa SIM Wave window has more sophisticated mechanisms for displaying the results of assertions.
- The Questa SIM Wave window does not have an analog waveform calculator.
- It is not possible to view nets inside SystemC or SystemVerilog using the [Wave Window \(Solido Waveform Analyzer\)](#) even when the waves appear in the JWDB database (they are empty). You must use the Questa SIM Wave window when running Questa ADMS.

Do this before running the simulation by using the command `ms wlfopen` before specifying any `add wave` or `add log` commands.

When SystemC or SystemVerilog nets are connected to VHDL, VHDL-AMS, Verilog, Verilog-AMS, or SPICE nets, then those connected nets can be displayed using Solido Waveform Analyzer.

Related Topics

[Adding Signals to the Solido Waveform Analyzer Wave Window](#)

[Removing Items from the Solido Waveform Analyzer Wave Window](#)

[Adding Items to the Log File](#)

Adding Signals to the Solido Waveform Analyzer Wave Window

Before running a simulation, you can add the signals you want to inspect to the Wave window (Solido Waveform Analyzer) so that their resulting waveforms are displayed when the simulation runs. Additionally, you can start generating a waveform for a signal at any time during simulation.

Use the following procedure to add items to the Wave window.

Prerequisites

If the **Wave Window (Solido Waveform Analyzer)** is not currently visible, choose **View > Wave > Questa ADMS** to open it.

Procedure

1. Select the required design unit in the **Structure Window**. The objects within it will be displayed in the **Objects Window**.
2. Select the required signals in the **Objects Window**. Use SHIFT+Select to select multiple items.
3. Right-click on the required signals in the **Objects Window** and choose **Add Wave**.

The equivalent add wave command is written to the **Transcript Window**.

Results

If an AC analysis is specified in the netlist, the **AC Tool Toolbar** is displayed in the **Objects Window**. This toolbar enables you to specify which kind of plot you want to generate.

Waveform data from the simulation is stored in a database file. Databases created during previous simulation runs can be reloaded into Solido Waveform Analyzer automatically.

You can also use any of the following methods to add signals to the Wave window:

- From the main menu, choose **Add > To Wave > Selected Signals**.
Choosing **Add > To Wave > Signals in region** adds all items in the region that is selected in the Structure window, and choosing **Add > To Wave > Signals in design** will add all items in the design.
- Drag and drop items from the **Objects Window** into the Wave window.
- Enter the add wave command directly into the **Transcript Window**:

```
add wave <item_name> <item_name>
```

You can use wildcards to add multiple items. For example, to add all the items in the currently selected region, enter:

```
add wave *
```

Or to add all the items in the design, enter:

```
add wave :*
```

Use the `-s` argument to plot the digital part of a boundary net, for example:

```
add wave -s :test:b1
```

You can use the **limprobe** command to limit the number of waveforms in the Wave window.

Tip

It is possible to add an array of array to the **Wave Window (Solido Waveform Analyzer)**. Clicking the top level array in the **Objects Window** expands the branch, displaying the elements beneath it. You can add the whole array of array, one specific array or a scalar element of an array of array.

See also **FSDB** in *Eldo Reference Manual*.

Related Topics

Removing Items from the Solido Waveform Analyzer Wave Window

add wave [Questa ADMS Command Reference]

add log [Questa ADMS Command Reference]

savewdb [Questa ADMS Command Reference]

Waveform Database File Formats

Waveform Database File Formats

The Solido Waveform Analyzer viewer uses the Joint Waveform DataBase (JWDB) format, which is a true mixed-signal waveform database.

This database can hold many different waveform types, including:

- Analog (float, double or complex)
- Histogram
- Spectral
- Scatter
- Verilog
- Standard logic
- VHDL char
- Buses and records
- Bit
- Boolean
- String

- Integer (16, 32, or 64 bits)
- User-defined enumerated types.

X-values can either be 64-bit integers or double-precision floating-point numbers. The JWDB can contain signals from the time and frequency domains.

JWDB is also a multi-run database. Waveforms and buses are stored, managed, and analyzed as compound waveforms. In addition to compound waveforms, JWDB has hierarchies which allow waveforms to be placed in folders for further data management.

FSDB Files

To use the FSDB output format, enable it for each design, using the following command:

```
.OPTION FSDB
```

It is not possible to set this as a default output format.

See also [FSDB](#) in *Eldo Reference Manual*.

Related Topics

[Removing Items from the Solido Waveform Analyzer Wave Window](#)

[add wave \[Questa ADMS Command Reference\]](#)

[add log \[Questa ADMS Command Reference\]](#)

[savewdb \[Questa ADMS Command Reference\]](#)

Removing Items from the Solido Waveform Analyzer Wave Window

You can remove a wave from the Wave window (Solido Waveform Analyzer) during simulation.

Procedure

1. Select the required signal in the [Objects Window](#).
2. To remove the signal from the [Wave Window \(Solido Waveform Analyzer\)](#), use any of the following methods:

- Choose **Add > Stop > Selected Net**.

The Wave window will stop displaying the signal's values, and any subsequent values will not be logged in the database. The values which have already been displayed and logged will remain in the database.

Alternatively, use the command:

```
stop <wave_name>
```

Refer to peer link_stop in the *Questa SIM Command Reference Manual* for more information.

- Choose **Add > Delete > Selected Net**.

The Wave window will stop displaying the signal’s values, or logging any subsequent values in the database. Any previously displayed and logged elements will be removed from the database.

Alternatively, use the command:

```
stop -rm <wave_name>
```

- To stop displaying values in the Wave window, but continue logging them in the database, choose **Add > To Log > Selected Signals** or use the command:

```
add log [options] <object_name>
```

Related Topics

[Adding Signals to the Solido Waveform Analyzer Wave Window](#)
[add log \[Questa ADMS Command Reference\]](#)

Adding Items to the Log File

You can add items to the log file.

Procedure

Do one of the following:

If you want to...	Do the following...
Add items to the log file.	Enter the following into the Transcript Window : add log <object_name> <object_name>
Add all the items in the current region	Enter the following into the Transcript window: add log *
Add all the items in the design	Enter the following into the Transcript window: add log -r :*

Related Topics

[add log \[Questa ADMS Command Reference\]](#)

Incremental Saving Mode

Waveforms are interactively and incrementally accessible by the Solido Waveform Analyzer (wave) window during the simulation. Incremental saving enables you to save waveform data to a *.wdb* file when the Joint Waveform DataBase (JWDB) reaches a specified threshold.

Incremental means that the waves are updated only when the simulation stops (at the end of each run command or break action). Using incremental saving mode, you can specify that the data in the JWDB database is saved at a specified threshold.

The **vasim** argument *-isaving* loads Questa ADMS in incremental saving mode and uses the variables set in the *modelsim.ini* file. To load Questa ADMS without incremental saving mode, use the *-noisaving* argument. Both of these arguments override the value of the *JwdbIncrementalSaving* variable.

Several variables are set in the *modelsim.ini* file to manage incremental saving. You can modify these for your own requirements.

If incremental saving is required during a simulation, either in batch mode or when a **run** -all is performed in the GUI, the incremental saving variables must be set before you invoke Questa ADMS. Changes to the variables made during simulation are not taken into account.

If Questa ADMS is invoked without incremental saving specified, the **isavewdb** command can be run from the Questa ADMS **Transcript Window**.

Related Topics

[Simulator Control Variables](#)

Automating Simulation Processes

You can automate load and command processes when starting a simulation.

Loading a Specific Design Unit.....	7-26
Questa ADMS Command File.....	7-26
Startup Files (do Files).....	7-27
The modelsim.tcl File.....	7-28
Saving and Reloading Formats and Content.....	7-28
Manually Saving the Transcript File.....	7-29
Reusing Solido Waveform Analyzer Configurations.....	7-30

Loading a Specific Design Unit

You can specify the name of a design when you invoke the simulator at the command line.

For example, use the following syntax to invoke the simulator with a design loaded that has SPICE as top-level:

```
vasim -cmd <spicetop.cir>
```

To invoke the simulator with a design loaded that has HDL as top-level:

```
vasim <HDL_entity_or_module>
```

For VHDL or VHDL-AMS-on-top, the most recently compiled architecture will be used. To load a specific architecture, you can explicitly provide the architecture name after the entity name:

```
vasim <entity> <architecture>
```

To invoke the simulator and load a configuration file that contains the design to be simulated:

```
vasim <configuration_file>
```

When loading a configuration file, you cannot specify architecture name. Hence, the last compiled architecture will be used.

The `vasim` command supports `vsim` arguments. For backward compatibility, you can also specify `vsim` arguments after the `vasim -ms` argument. You must use the `vasim -ms` argument if you want Questa ADMS and Questa SIM to run different do files. Refer to [Startup Files \(do Files\)](#) for more information.

Related Topics

[Running an Interactive Simulation](#)

[Viewing Simulation Results](#)

[vasim \[Questa ADMS Command Reference\]](#)

Questa ADMS Command File

A command file can contain some or all of the simulation commands required during a simulation.

It is loaded when you invoke the simulator using the `-cmd` option or by selecting it (or creating a new one) when loading a design with the [Start Simulation Dialog Box](#).

Command files may use the optional file extension `.cmd`, but this is not mandatory.

Command files are required when your mixed-signal design has HDL-on-top. For SPICE-on-top designs, simulation control commands can be included in the netlist circuit (`.cir`) or subcircuit (`.ckt`) file. For both file types, the use of the file extension is not mandatory.

Command files can be written using a simple text editor, or can be created when loading a design using the [Start Simulation Dialog Box](#).

When a design is loaded, you can open the command file in the [Source Window](#) by double-clicking on it in the [Structure Window](#). It will open as read-only. To make it editable from within the Source window, select Source from the main menu and uncheck the Read Only option.

Related Topics

[Running an Interactive Simulation](#)

[Viewing Simulation Results](#)

[Startup Files \(do Files\)](#)

Startup Files (do Files)

Startup files, or “do” files, are simple text files that the end-user creates.

This file contains a batch of commands that Questa ADMS reads and runs in sequence as if the end-user entered them individually at the command line. A user can run a do file any time.

Startup files may be specified using the `-do` argument when launching the simulator, for example:

```
vasim -cmd top_spice.cir -do adms_cmd.do
```

Multiple startup files can be specified; each filename must be preceded with the argument `-do`. Commands are run in the order that they appear in the startup files.

```
vasim -cmd top_spice.cir -do adms_cmd.do -do adms2_cmd.do -do adms3_cmd.do
```

You can include a Questa SIM do file using `vasim -ms -do <dofile_questa>`. If multiple do files are specified using both `vasim -do <dofile>` and `vasim -ms -do <dofile_questa>`, the Questa SIM do file(s) will be run first.

Alternatively, specify the required commands on the command line. They must be enclosed within double quotes and separated by semi-colons. For example:

```
vasim -cmd mydesign.cir -do "add wave <object_name>; run -all; quit -f"
```

Commands can be specified on multiple -do arguments, for example:

```
vasim -cmd mydesign.cir -do "add wave <object_name>" -do "run -all; quit -f"
```

If your do file contains a vasim command which specifies further do files, they will be ignored. Use the **do** command to run the startup script instead.

Tip

If you have a set of default Tcl commands that you want to run for every simulation, you can put them in a file called *modelsim.tcl* in the home directory and Questa ADMS will load it on invocation.

Refer to DO File in the *Questa SIM User's Manual* for more information.

Related Topics

[Running an Interactive Simulation](#)

[vasim \[Questa ADMS Command Reference\]](#)

[Supported Eldo Simulator Commands \[Questa ADMS Command Reference\]](#)

[do \[Questa ADMS Command Reference\]](#)

[The modelsim.tcl File](#)

The modelsim.tcl File

Questa ADMS loads modelsim.tcl on invocation.

If you have a set of default Tcl commands that you want to run for every simulation, you can put them in this file. Questa ADMS searches for the file as follows:

- use MODELSIM_TCL environment variable if it exists. If MODELSIM_TCL is a list of files, each file is loaded in the order that it appears in the list
- else use ./modelsim.tcl if it exists
- else use \$(HOME)/modelsim.tcl if it exists.

Saving and Reloading Formats and Content

You can use the write format restart command to create a do file. When invoked with the do command in subsequent simulation runs, this do file recreates all debug windows.

Transcript commands (without comments) are also saved to restore the previous setup.

The syntax is:

```
write format restart <filename>
```

Set the ShutdownFile variable to this do filename to run the write format restart command upon exit.

Related Topics

[do \[Questa ADMS Command Reference\]](#)

[Startup Files \(do Files\)](#)

[Manually Saving the Transcript File](#)

Manually Saving the Transcript File

You can save the transcript file.

Variable settings determine the filename used for saving the transcript. Set the PrefMain(file) variable in the *.modelsim* file or the TranscriptFile variable in the *modelsim.ini* file, to log the transcript output to a specific file. By default the TranscriptFile variable in *modelsim.ini* is set to "transcript". Setting either variable ensures that the transcript contents are always saved and no explicit saving is necessary. However, if you want to manually save a copy of the transcript to another location, use the following procedure.

Procedure

1. Choose **File > Save Transcript As...** from either the Transcript Window menu or the main menu.
2. Specify a file name for the transcript file and navigate to the location in which to save it before clicking Save.

Results

The file name and location are stored in the Tcl variable PrefMain(saveFile). Subsequent saves to this file can be made by selecting Save or clicking the  icon.

Tip

Saved transcript files can be used as macros (do files). Refer to the [do](#) command for more information.

Related Topics

[Running Commands from the Transcript Window](#)

Reusing Solido Waveform Analyzer Configurations

When a simulation completes, the current state of the Wave window (Solido Waveform Analyzer) is automatically saved to a `.ez.do` file (default) or a `.swd` file. You can reload the file in a subsequent session.

This mechanism is enabled by default. You can control it by modifying the GUI preference variable `PrefReuse(WaveConfig)`.

If the variable is set to no, the Solido Waveform Analyzer configuration is saved but not reloaded.

The configuration is stored under the base name of the command file with the extension `.ez.do` or `.swd`.

A saved configuration file is automatically loaded when:

- The simulation is restarted and `PrefReuse(WaveConfig)` is set to either `tcl` or `swd`, or the command `usewaveconfig` is specified.
- The Questa ADMS GUI configuration is saved and reloaded and the GUI preference variable `PrefReuse(GuiConfig)` is set to "yes" (see [Save/Reuse of Questa ADMS GUI Configuration](#)).

Three commands are available to load and save Solido Waveform Analyzer configurations interactively from Questa ADMS. For example, you can save and restore Solido Waveform Analyzer window placement and geometry. The commands are:

- `savewavewindow`
Saves the organization of all windows open in Solido Waveform Analyzer
- `savewaveconfig`
Saves the organization of the active window only
- `usewaveconfig`
Loads a saved Solido Waveform Analyzer configuration to display the saved waves

These commands are only available through the Transcript window.

If you use the `usewaveconfig -append` argument, any previously opened windows will not be removed from the display. Instead, the saved configuration is opened in a new window. All current `add wave/log` commands are considered (they are in the database) but you must select them to see them. The `usewaveconfig` command remains active until you exit the GUI or load a new design.

Note

When automatic save/use Solido Waveform Analyzer configuration is disabled, you can request the previous Solido Waveform Analyzer configuration for simulation when restarting a simulation. If the Solido Waveform Analyzer configuration is maintained for the new run, all the wave commands that were not part of this configuration are automatically considered as log commands.

Related Topics

[savewaveconfig \[Questa ADMS Command Reference\]](#)
[savewavewindow \[Questa ADMS Command Reference\]](#)
[usewaveconfig \[Questa ADMS Command Reference\]](#)
[view \[Questa ADMS Command Reference\]](#)
[.ez.do File Limitations](#)
[Wave Window \(Solido Waveform Analyzer\)](#)

Methods for Setting Simulation Parameters

You can set simulation parameters interactively, on the command line, and in SPICE netlists.

You can assign the value *<value>* to the simulation parameter *<param_name>* using any of the following methods:

- The `simparam set` simulator command:

```
simparam set <param_name> <value>
```

This method has the highest precedence.

- The `vasim -simparam` argument:

```
vasim -simparam <param_name>=<value>
```

- The `eldo -simparam` argument:

```
vasim -eldoopt "-simparam <param_name>=<value>"
```

- The `.SIMPARAM` SPICE command:

```
.SIMPARAM <param_name>=<value>
```

This method has the lowest precedence.

An error will be reported if the new value is not of the expected type. If the severity level is set to warning, note, or suppress messages, simulation will continue using the default value.

Note

You cannot assign a new value to any of the built-in simulation parameters. Refer to [Verilog-AMS Extensions](#) for a list of the built-in simulation parameters.

Related Topics

[Access to Simulation Parameter Values](#)

[Passing Options Directly to Eldo and ADiT](#)

[Using Simulation Parameters to Restore Checkpoints](#)

[simparam set \[Questa ADMS Command Reference\]](#)

[simparam get \[Questa ADMS Command Reference\]](#)

[vasim \[Questa ADMS Command Reference\]](#)

Simulation Save and Restart

The save and restart feature saves a snapshot of an ongoing simulation and restarts it later.

It can be useful in the following scenarios:

- For long simulations.
To use the processor for another job, of higher priority, you can stop a long simulation and restart later.
- To use a common startup procedure for many different simulation runs.
Mixed-signal simulation can have a long initialization time. You may want to debug the initialization once, and then ignore the initialization in subsequent tests.
- To create a regular backup of the simulation state in order that the simulation can be restarted in the event of a network issue, a memory or disk space shortage, or any other reason that unexpectedly forced the simulation to stop.

The simulation may be restarted from the saved point any number of times. A transient simulation can be saved at multiple time points in a simulation using the **.TSAVE** command. The simulation can be restarted from one of multiple time points in a subsequent simulation run.

Save/restart behavior can either be set up in advance, by placing the required commands and options in the command file, by writing code in an HDL description, or it can be run from the Transcript window during the simulation.

You can only restart a saved transient simulation state within a release-compatible simulator. Simulators are compatible if they share a full release number (such as 2010.1) regardless of subsequent lettered releases. For example, a simulation state saved from the 2009.2 simulator can be restarted in the 2009.2b simulator, but not 2010.1. If you attempt to restart a saved simulation from an incompatible release, the simulation is stopped once an incompatibility is detected and an error message is reported.

The save/restart feature is not available with Questa ADMS RF (.MODSST only) when the circuit contains digital parts. The **.ADMS_START** command does not work with **.SST** analysis when it is initialized with a pre-transient phase (supported with **.MODSST**).

Save/Restart Commands.....	7-33
Saving a Simulation at Regular Intervals.....	7-36
Saving and Restarting a Simulation Interactively.....	7-38
Checkpoint the Simulation From the Test Bench.....	7-40
Using Simulation Parameters to Restore Checkpoints.....	7-48

Save/Restart Commands

You can control simulation save and restart behavior.

The following commands control simulation save and restart behavior:

- **.SAVE**

Saves simulation information at specific times during simulation to a *.iic* directory. For example, the following command instructs the simulator to schedule a save at 50ms:

```
.SAVE TIME=50ms
```

To save a simulation run at multiple time points, use the **.TSAVE** command.

- **.TSAVE**

Saves simulation information at a specified time point to a *.iic* directory. For example, if a number of saves are required, an instruction for each can be included in the command file:

```
.TSAVE NOREPLACE TIME=3ms
.TSAVE REPLACE TIME=10ms
.TSAVE NOREPLACE TIME=20ms
.TSAVE NOREPLACE TIME=24ms
```

In this case, the options REPLACE or NOREPLACE specify whether to keep or overwrite the previously saved data.

Use FILE="fileBaseName" to specify the prefix of the generated checkpoint directory. The following example generates directories *spice_top_2000.iic* and *spice_top_4000.iic*:

```
.TSAVE NOREPLACE TIME=2ns FILE="spice_top"
.TSAVE NOREPLACE TIME=4ns FILE="spice_top"
```

- **checkpoint**

Saves the state of the simulation at the current time:

```
checkpoint -noreplace my_saving
```

Can be run from the **Transcript Window** or a script.

- checkpoint("fileBaseName", replace) and \$checkpoint("fileBaseName", replace)

Used inside an HDL description, to save the simulation state at the current time, to a directory named *"fileBaseName"_timepoint.iic*. Specify the replace option to remove all previously saved checkpoint directories in the output directory and replace them with the checkpoint directory specified on the command.

Example in VHDL-AMS:

```
PROCESS (condition)
BEGIN
    checkpoint("my_saving", true);
END PROCESS;
```

Example in Verilog-AMS:

```
always @ (posedge condition)
begin
    $checkpoint("fileBaseName", replace);
end
```

- **.RESTART**

Restarts a simulation run with information previously saved using either the .save or .tsave command. For example:

```
.RESTART my_saving TIME=25ms
```

- **restore**

Restores the simulation from the time point specified in a checkpoint file. For example:

```
restore -time=longest my_saving
```

The .SAVE END command saves the complete simulation environment at the end of a transient analysis run. The .SAVE TIME=VAL command saves the simulation status at a specific time point of a transient analysis run. The .RESTART command restarts the transient simulation from either:

- The time the last simulation was ended, if the last save corresponded to the end point.
- The specific time point, if the last save corresponded to a specific time point.

Save-Generated Files

A checkpoint generates a directory that contains files that allow Questa ADMS to restart.

The directory is named with the name specified in the .SAVE/.TSAVE command. If no name is given, Questa ADMS uses the name of the command file.

You must not edit the files within this directory. Removing, renaming, or editing these files will cause an error to be reported if the simulation is restored. Only the directories can be renamed.

Changes After a Save

After a simulation state has been saved, Questa ADMS applies or does not apply changes according to the following:

- **Forbidden Changes**

Any change to the configuration of the design is not allowed. For instance, it is not possible to switch an instance from a digital description to an analog description, or vice versa. HDL descriptions must remain identical.

- Allowed Changes

You can change the parameters of discrete components (such as resistors and capacitors) or sources (such as PWL, SIN) between the save and the restart. Any commands that modify the state of HDL descriptions are allowed after the restart. Use the **change** command to change the values of parameters, constants, or variables. Use the **force** command to change the values of signals or terminals.

Related Topics

[Saving a Simulation at Regular Intervals](#)

[Saving and Restarting a Simulation Interactively](#)

[Checkpoint the Simulation From the Test Bench](#)

[Simulation Save and Restart](#)

Saving a Simulation at Regular Intervals

You can automatically save results at regular intervals during the simulation.

The following example shows how to automatically save results at regular intervals during the simulation. It uses a simple voltage divider made of resistors.

- The top voltage value is dependent upon a parameter, so that it can easily be changed.
- The simulation is required to run for 10ms, with a save every 2ms.
- All checkpoints must be kept. To do this, you use the .SAVE command with the REPEAT SEQ argument.

The netlist for this design is contained within the file *netlist.cir*:

```
*
.PARAM my_value=2.0
V0 GND 0 DC 0.0
V1 TOP GND PWL (0 0.0 3m my_value 5m {2*my_value} 10m {0.5*my_value})
R0 TOP TOP_1 1K
R1 TOP_1 TOP_2 1K
R2 TOP_2 GND 2K
.PLOT TRAN V(TOP) V(TOP_1)
.TRAN 10m 10m UIC
.SAVE TIME=2ms REPEAT SEQ
.END
```

Procedure

1. Create a working library and load the netlist into Questa ADMS:

```
valib $PWD/MYLIB
vasim -cmd netlist.cir
```

2. Run the simulation until the end:

```
run -all
```

During simulation, the following messages report that saves are being performed:

```
Save file netlist.iic.1/iic created at time 2.000000e-03
Save file netlist.iic.2/iic created at time 4.000000e-03
Save file netlist.iic.3/iic created at time 6.000000e-03
Save file netlist.iic.4/iic created at time 8.000000e-03
Save file netlist.iic.5/iic created at time 1.000000e-02
```

At the end of the simulation, all checkpoint files are kept.

3. Edit the netlist to modify the .PARAM statement from 2.0 to 5.0, and add node TOP_2 to be saved. Also, replace the .SAVE command by a .restart command at time 8 ms:

```
.PARAM my_value=5.0
.PLOT TRAN V(TOP) V(TOP_1) V(TOP_2)
.RESTART TIME=8ms
```

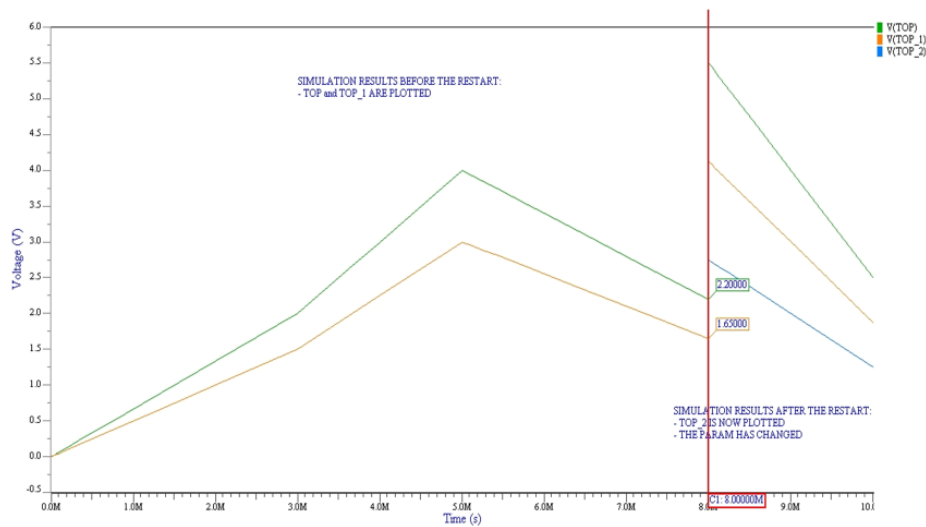
Note the messages in the [Transcript Window](#), stating that the simulation start time is now 8 ms, that the run time is still 10 ms, and that the restart was successful.

4. Run the simulation again, and when it completes, observe the results in the [Wave Window \(Solido Waveform Analyzer\)](#):

```
swave netlist.wdb
```

Until the time of the restart (before the cursor in [Figure 7-9](#)), the results correspond to those from the first simulation. This is why TOP_2 does not have any value, because it was not saved at this time. After the restart, the new simulation results are available. Note the significant change in the value of TOP and TOP_1, corresponding to the update of the parameter my_value from 2.0 to 5.0.

Figure 7-9: Save/Restart Simulation Results



Related Topics

Saving and Restarting a Simulation Interactively

Saving and Restarting a Simulation Interactively

If no preparation has been made to save the simulation, but the need arises mid-simulation, you can perform a save during simulation.

This situation can occur if a simulation takes longer than expected to complete, or when you want to alter simulation parameters.

Procedure

1. Create a working library and load the netlist into Questa ADMS:

```
valib $PWD/MYLIB
vasim -cmd netlist.cir
```

2. Run the simulation for some time less than the time specified on the .TRAN command:

```
run 7 ms
```

3. Schedule the save by entering the checkpoint command in the [Transcript Window](#):

```
checkpoint
```

A message similar to the following is reported:

```
# ** Note: (vasim - 1406) Checkpoint scheduled at time 7e+06 ns.
# (Checkpoint files will be generated during simulation or at the end
of session)
```

Note

The checkpoint command only schedules the save; the save itself is performed on the next run command or when the simulation session ends.

4. Run the simulation to the end:

```
run -all
```

Note the messages sent to the Transcript window:

```
# ** Note: (eldo - 21) All previously saved checkpoint files
will be removed from the output directory and replaced with
'netlist_7e+09.iic'.
# Save file netlist_7e+09.iic/iic created at time 7.000000e-03
```

5. Use the restore command to restore the checkpoint:

```
restore -time=7e-3
```

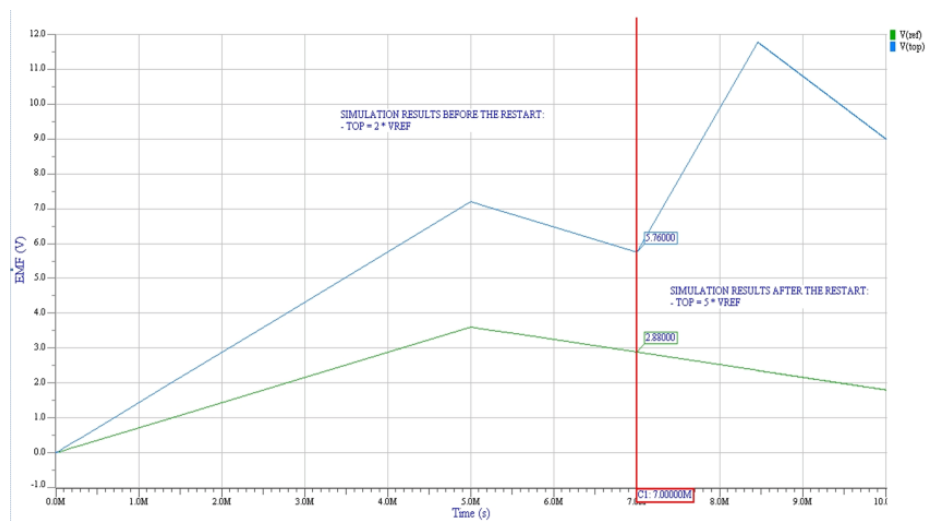
Note the messages in the Transcript window, stating that the start time is 7ms, that the run time is 10ms, and that the restore was successful:

```
# Starting analog time is 7.000000E+06 ns
# Simulation time for "run -all" is 1.000000E+07 ns
# ** Note: (vasim - 1100) Restore at time 7e+06 ns completed
```

6. Run the simulation to the end again, and then observe the results in the [Wave Window \(Solido Waveform Analyzer\)](#):

```
run -all
swave netlist.wdb
```

Figure 7-10: Save/Restart—GUI Simulation Results



Examples

This example contains a voltage-controlled voltage source (VCVS) with a gain declared as a parameter. The voltage value on node TOP is defined as the voltage value on node REF multiplied by the gain. The netlist is contained in the file *netlist.cir*:

```
.PARAM my_value=2.0
V0 GND 0 DC 0.0
V1 REF GND PWL (0 0.0 5m 3.6 10m 1.8)
E0 TOP GND VCVS REF GND my_value
.TRAN 10m 10m UIC
.PLOT TRAN V(TOP) V(REF)
.END
```

Related Topics

- Simulation Save and Restart
- Saving a Simulation at Regular Intervals
- Checkpoint the Simulation From the Test Bench

Checkpoint the Simulation From the Test Bench

You can save the simulation state when a specific event occurs.

checkpoint() Command Examples.....	7-41
Saving Simulation State in a VHDL/VHDL-AMS Process.....	7-41
Saving Simulation State in a Verilog/Verilog-AMS Task.....	7-45

checkpoint() Command Examples

The checkpoint() and \$checkpoint() functions save the simulation state.

You can use the checkpoint() and \$checkpoint() command to save the simulation state when the design under simulation has reached a functional state, or when a certain operation completes. In this case, the save instruction should be placed in a VHDL/VHDL-AMS process or in a Verilog/Verilog-AMS task. For example:

- In this example, the save request occurs in a VHDL/VHDL-AMS process that waits for a specified time. The simulation is saved into a file named *"basename"_timepoint.iic* after 50ms. The final WAIT statement, stops the process after one save. Without this final WAIT statement, the simulation would be saved every 50 ms.

```
PROCESS
BEGIN
WAIT FOR 50 ms;
checkpoint("basename", TRUE);
WAIT;
END PROCESS;
```

- In this example, the save request occurs in a VHDL/VHDL-AMS process that monitors the value of the signal my_condition, and saves the simulation into a file named *"basename"_timepoint.iic*, every time its value becomes 1.

```
PROCESS (my_condition)
BEGIN
IF (my_condition'EVENT AND my_condition = '1') THEN
    checkpoint("basename", 1);
END IF;
END PROCESS;
```

- In this example, the save request occurs in a Verilog/Verilog-AMS task. The task monitors the value of the signal my_condition, then saves the simulation into a file named *"basename"_timepoint.iic* every time its value becomes 22.

```
integer my_condition;
always @ (my_condition) begin
    if (my_condition == 22)
        $checkpoint("basename",1);
end
```

Saving Simulation State in a VHDL/VHDL-AMS Process

You can save the simulation state in a VHDL/VHDL-AMS process when a condition is reached.

The following example uses VHDL-AMS source code for a current-controlled voltage regulator. In this model, the regulator has a typical output of 3.6 V and maximum output current of 6 mA. A checkpoint is made as soon as the output voltage reaches the threshold specified for the generic `ldo_low` (here, 2.2 V).

Prerequisites

- The VHDL-AMS source code is contained in the file `code.vhdl`:

```

LIBRARY ieee;
USE ieee.electrical_systems.ALL, ieee.thermal_systems.ALL, ieee.std_logic_1164.ALL;
LIBRARY mgc_ams;
USE mgc_ams.checkpoint.all;

ENTITY REGULATOR IS
  GENERIC ( ldo_output : REAL := 3.6; -- typical regulator output value
    ldo_low : REAL := 2.2 ); -- The threshold that triggers the checkpoint
  PORT ( TERMINAL OUTPUT : ELECTRICAL; -- The regulator's output
    TERMINAL VDD : ELECTRICAL; -- The regulator's voltage supply
    TERMINAL GND : ELECTRICAL; -- The regulator's voltage reference
    SIGNAL ENABLE : IN STD_LOGIC); -- The regulator's enable signal
END ENTITY REGULATOR;

ARCHITECTURE BHV OF REGULATOR IS
  SIGNAL is_ready : STD_LOGIC := '0'; -- An internal signal that triggers the checkpoint
  SIGNAL ldo_value : REAL := 0.0; -- An internal signal for the output value
  QUANTITY my_voltage ACROSS OUTPUT TO GND; -- Output voltage quantity
  QUANTITY my_current THROUGH VDD TO OUTPUT; -- Output current quantity
BEGIN
  ldo_value <= ldo_output; -- Sets the output voltage value according to the generic provided
  IF ENABLE = '0' OR my_voltage'ABOVE(ldo_value) USE
  my_current == 0.0;
  ELSIF my_voltage'ABOVE(ldo_value-10.0e-3) USE
  my_current == 6.0e-3 * (ldo_value-my_voltage) / 10.0e-3;
  ELSE
  my_current == 6.0e-3;
  END USE;
  BREAK ON ENABLE, ldo_value;
  BREAK ON my_voltage'ABOVE(ldo_value), my_voltage'ABOVE(ldo_value-10.0e-3);
  -- Triggers the checkpoint when the output reaches the threshold
  -- NOTE: a 2 ms delay is inserted to make sure that the output is stable before saving
  is_ready <= '1' AFTER 2 ms WHEN my_voltage'ABOVE(ldo_low) ELSE '0';
  -- Process monitoring "is_ready" in order to trigger the checkpoint
  -- The simulation will be saved in a file with base name "my_saved_file"
  -- All previously saved checkpoints will be removed and replaced by this new checkpoint
  do_save: PROCESS (is_ready)
  BEGIN
  IF is_ready = '1' THEN
    REPORT "The regulator reached its minimum threshold, saving now"
    SEVERITY NOTE;
    checkpoint("my_saved_file", true);
  END IF;
  END PROCESS do_save;
END ARCHITECTURE BHV;

```

- The following command file, *netlist.cir* contains the commands to control the simulation:

```
.MODEL MY_LDO MACRO LANG=VHDLAMS LIB=MYLIB MOD=REGULATOR(BHV)
YI_LDO MY_LDO PORT: OUTPUT VDD GND ENABLE
V0 GND 0 DC 0.0
V1 ENABLE GND PWL (0 0.0 1m 3.6)
V2 VDD GND DC 4.8
C0 OUTPUT GND 2.2u
R0 OUTPUT GND 1000

.DEFH00K A2D_std_logic_default D2A_std_logic_default

.MODEL A2D_std_logic_default A2D MODE=std_logic VTH1=0.5 VTH2=0.7

.MODEL D2A_std_logic_default D2A MODE=std_logic VLO=0.0 VHI=3.6
+ TRISE=10ns TFALL=10ns

.PLOT TRAN V(OUTPUT)

.TRAN 5m 5m UIC
.END
```

Procedure

1. Create a library named MYLIB, and compile the VHDL-AMS source code in the file *code.vhdl* using the **vacom** command:

```
valib $PWD/MYLIB
vacom code.vhdlams
```

2. Load the design into Questa ADMS and run the simulation:

```
vasim -cmd netlist.cir
run -all
```

Note that the simulation start time is 0 ns and that the run time is 5 ms:

```
# Starting analog time is 0.0 ns
# Simulation time for "run -all" is 5.000000E+06 ns
```

When the output voltage reaches the expected threshold, the following message is reported:

```
# ** Note: The regulator reached its minimum threshold, saving now
# Time: 3,144,623,441,766 fs Iteration: 0 Instance:
netlist:yi_ldo Process: do_save
```

At the exact same time, the checkpoint is initiated:

```
# ** Note: (eldo - 21) All previously saved checkpoint files
will be removed from the output directory and replaced with
'my_saved_file_3.14462e+09.iic'.

# Save file my_saved_file_3.14462e+09.iic/iic created at time
3.144623e-03
```

3. In the command file *netlist.cir*, do the following:

- a. Modify the value of the load resistor from 1000Ω to 100Ω . This new load will produce an output current greater than the 6mA allowed by the regulator.
- b. Add a .PLOT command to instruct VDD to be plotted in addition to OUTPUT.
- c. Add the .RESTART command.

```
R0 OUTPUT GND 100
.PLOT TRAN V(OUTPUT) V(VDD)
.RESTART my_saved_file NEWEST
```

4. Run the simulation again:

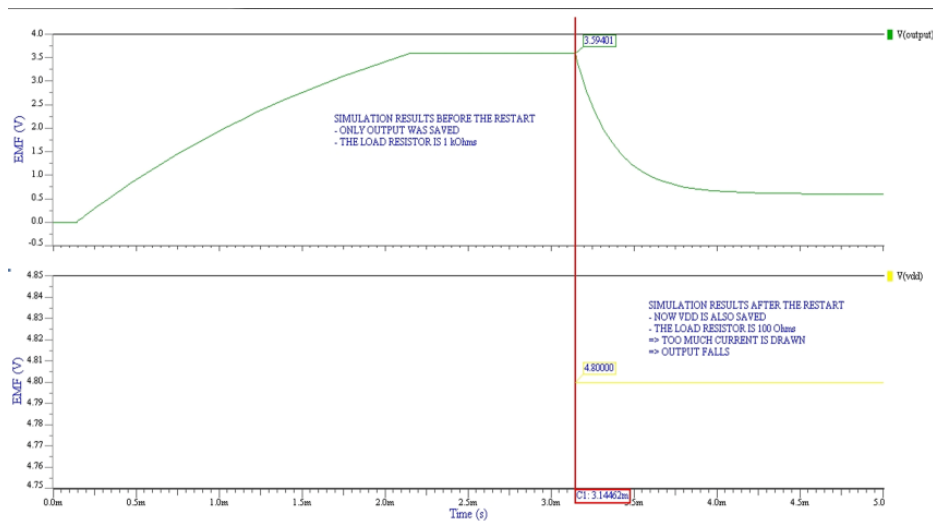
```
vasim -cmd netlist.cir
```

Results

Messages are reported to the [Transcript Window](#), stating that the simulation start time is now 3.144 ms, that the run time is still 5 ms, and that the restart was successful.

Observe the results displayed in the [Wave Window \(Solido Waveform Analyzer\)](#). Until the time of the restart (before the cursor shown in [Figure 7-11](#)), the results correspond to those from the first simulation. This is why VDD does not have any value, because it was not saved at this time. After the restart, the new simulation results are available. The output voltage falls because the load resistor requires more current than the regulator can provide.

Figure 7-11: Save/Restart—VHDL-AMS Event-Driven Simulation Results



Related Topics

[Simulation Save and Restart](#)

[Save/Restart Commands](#)

[Saving a Simulation at Regular Intervals](#)

[Saving and Restarting a Simulation Interactively](#)

Saving Simulation State in a Verilog/Verilog-AMS Task

You can save the simulation state in a Verilog/Verilog-AMS task when a condition is reached.

The following example uses a resistor described in Verilog-AMS, instantiated in a test bench that drives the voltage on one side, with the other side tied to the ground. Questa ADMS monitors the value on the voltage-driven top of the resistor, and counts the number of times it goes below a certain threshold. The simulation state is saved each time this number is a multiple of 8. Each save overwrites the previous one.

Prerequisites

- The Verilog-AMS source code of the resistor and the test bench is contained in the file *code.v*:

```
// include the library that contains the analog packages
`include "disciplines.vams"

// declare the resistor module
module resistor (p, n);
    parameter real r=100;    // resistance (Ohms)
    inout p, n;
    electrical p, n;

    analog
```

```

V(p,n) <+ r * I(p,n);
endmodule

// declare the test bench module
module testbench (p, n);
  inout p, n;
  electrical p, n;
  integer cnt;

  // instantiate the resistor within the test bench
  resistor my_res (p, n);

  initial begin
    cnt = 1;
  end

  // count the number of times the voltage goes below 1.25 V
  always @ (cross(V(p)-1.25)) begin
    if (V(p) < 1.25)
      cnt = cnt + 1;
  end

  // checkpoints the simulation state every 8 times, in a file named "my_vams_checkpoint"
  always @ (cnt) begin
    if (cnt % 8 == 0)
      $checkpoint("my_vams_checkpoint",1);
  end

endmodule

```

- The following command file, *netlist.cir* contains the commands to control the simulation:

```

.MODEL my_testbench macro lang=verilog lib=mylib mod=testbench
yi_bench my_testbench port: portp portn
v0 portn 0 dc 0.0
v1 portp portn sin (1.25 1.25 15K 0 600)
.PLOT tran v(portp)
.TRAN 2m 2m uic
.END

```

Procedure

1. Create a library named MYLIB, and compile the Verilog-AMS source file, using the **valog** command:

```

valib $PWD/MYLIB
valog code.v

```

2. Load the design into Questa ADMS using the **vasim** command and run the simulation:

```

vasim -cmd netlist.cir
run -all

```

Note that the simulation start time is 0ns, and that the run time is 2ms:

```
# Starting analog time is 0.000000 ns
# Simulation time for "run -all" is 2.000000E+06 ns
```

Each time the condition is met, the simulation state is saved. Messages report the checkpoints in the transcript:

```
# ** Note: (eldo - 21) All previously saved checkpoint files
will be removed from the output directory and replaced with
'my_vams_checkpoint_4.33334e+08.iic'.

# Save file my_vams_checkpoint_4.33334e+08.iic/iic created at time
4.333337e-04

# ** Note: (eldo - 21) All previously saved checkpoint files
will be removed from the output directory and replaced with
'my_vams_checkpoint_9e+08.iic'.

# Save file my_vams_checkpoint_9e+08.iic/iic created at time
9.000002e-04

# ** Note: (eldo - 21) All previously saved checkpoint files
will be removed from the output directory and replaced with
'my_vams_checkpoint_1.36667e+09.iic'.

# Save file my_vams_checkpoint_1.36667e+09.iic/iic created at time
1.366667e-03

# ** Note: (eldo - 21) All previously saved checkpoint files
will be removed from the output directory and replaced with
'my_vams_checkpoint_1.83334e+09.iic'.

# Save file my_vams_checkpoint_1.83334e+09.iic/iic created at time
1.833344e-03
```

3. Modify the command file *netlist.cir* as follows:

- a. Modify the amplitude from 1.25V to 5.25V.
- b. Modify the frequency from 15 kHz to 3 kHz of the voltage on top of the resistor.
- c. Add a .RESTART command.
- d. Change the simulation run time from 2 ms to 10 ms:

```
v1 portp portn sin (1.25 5.25 3K 0 600)
.RESTART my_vams_checkpoint NEWEST
```

4. When the design is loaded, use the GUI to plot the current through the resistor.

5. Run a second simulation:

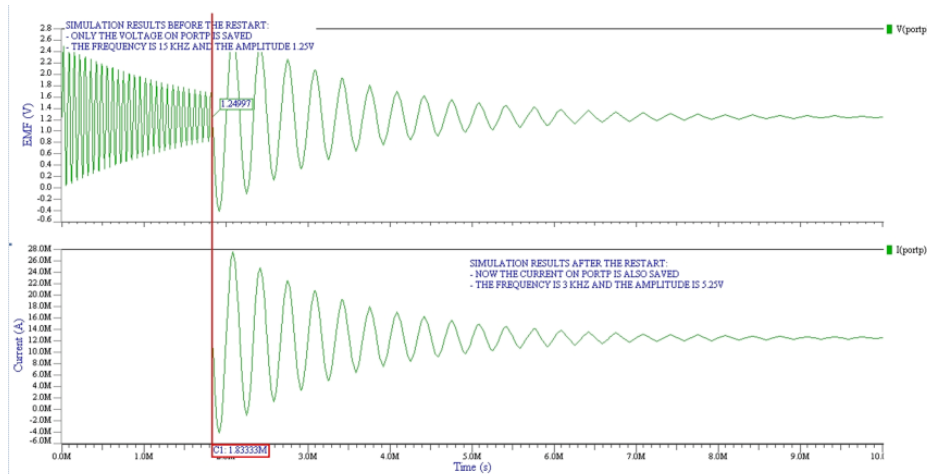
```
run -all
```

Messages are reported in the [Transcript Window](#), stating that the simulation start time is now 1.833 ms, that the run time is now 10 ms, and that the restart was successful.

Results

When the simulation ends, the results are displayed in the [Wave Window \(Solido Waveform Analyzer\)](#). Until the time of the restart (before the cursor), the results correspond to those from the first simulation. This is why the current on PORTP does not have any value, because it was not saved at this time. After the restart, the new simulation results are available. Notice the voltage changing in frequency and amplitude at this time.

Figure 7-12: Save/Restart—Verilog(-AMS) Event-Driven Simulation Results



Related Topics

[Simulation Save and Restart](#)

[Save/Restart Commands](#)

[Saving a Simulation at Regular Intervals](#)

[Saving and Restarting a Simulation Interactively](#)

Using Simulation Parameters to Restore Checkpoints

You can restore a checkpoint any number of times. Each time you restore a checkpoint, you can run a different set of tests according to the specified restore simulation parameter.

Once a restore value is defined in the source code, and set to the value of the specified simulation parameter, the value of the simulation parameter can be changed at any time, using the `simpam set` command.

VHDL/VHDL-AMS Example

The following example demonstrates how to create restore values that use simulation parameters:

```
PROCESS .....
    VARIABLE my_restore_int_variable: INTEGER;
    VARIABLE my_restore_real_variable: REAL;
BEGIN
    .....
    restore_int_variable = simpam("chkptint");
    .....
    CASE restore_int_variable is
        WHEN .....
        WHEN .....
    END CASE;
    .....
    restore_real_variable = simpam("chkptreal");
    IF (restore_real_variable > val)
        .....
    THEN
        .....
    END IF;
    .....
END PROCESS;
```

Verilog/Verilog-AMS Example

The following example demonstrates how to create restore values that use simulation parameters:

```
int restore_integer;
real restore_real;
always ..... begin
    .....
    restore_integer = $simpam("chkptint");
    case restore_integer
        ... :
        default :
    endcase
    .....
    restore_real = $simpam("chkptreal");
    if (restore_real > val)
        .....
end
```

Related Topics

- [Access to Simulation Parameter Values](#)
- [Methods for Setting Simulation Parameters](#)
- [simparam set \[Questa ADMS Command Reference\]](#)

Simulator Command Time Units

You can specify the simulation time unit for delays in all simulator commands that have time arguments.

For example:

```
force clk 1 50 ns, 1 100 ns -repeat 1 us
run 2 ms
```

Note that all the time units in a Questa ADMS command need not be the same.

Unless you specify otherwise, simulation time is always expressed using the resolution units that are specified by the UserTimeUnit variable in the vsim section of the *modelsim.ini* file.

By default, the specified time units are assumed to be relative to the current time unless the value is preceded by the character @, which signifies an absolute time specification.

Related Topics

- [Questa SIM Simulation Control Variables](#)

Optimizing Simulation

There are tips and techniques to help to speed up your simulation.

- [Controlling Simulation Accuracy.....](#) 7-51
- [Accuracy Control Options for Analog Simulation.....](#) 7-55
- [Crossing Time Accuracy Options Summary.....](#) 7-57
- [Performance/Accuracy Trade-off Options.....](#) 7-58
- [Analog Solver Delayed Start.....](#) 7-61
- [Speeding Up Elaboration of SPICE Subcircuits.....](#) 7-63
- [Improving Performance and Reducing Memory Consumption.....](#) 7-63
- [Multi-Threading.....](#) 7-64
- [Using Questa ADMS Premier.....](#) 7-65
- [VHDL-AMS Attribute 'MGC_FOLLOW.....](#) 7-66
- [Preoptimizing Pure Digital Subregions.....](#) 7-66
- [Simulation Time Reporting.....](#) 7-67

Controlling Simulation Accuracy

You can read and modify the value of precision parameters to control simulation accuracy either globally for a given time period, or for a specified instance.

Adjusting the accuracy applied to parts of the design that are not of interest to the particular set of tests you are running, can reduce simulation time significantly.

Table 7-4: Supported Parameters

Parameter	Accessed by Global Tasks and Functions	Accessed by Instances or Models
TUNING	Yes	Yes
ABSTOL	Yes	Yes
RELTOL	Yes	Yes
VNTOL	Yes	Yes
EPS	Yes	Yes
RELTRUNC	Yes	Yes
HMIN	Yes	No
HMAX	Yes	No
CHGTOL	Yes	No
FLUXTOL	Yes	No
ITOL	Yes	No
NGTOL	Yes	No
OUT_RESOL	Yes	No
FREQSMP	Yes	No
PCS	Yes	No

All of these have values that are numerical and greater than zero, except for TUNING. [Table 7-4](#) lists the macros that you can use to control the TUNING parameter.

Table 7-5: Macros For Use With TUNING Parameter

Macro	Numerical Value
TUNING_FAST	1
TUNING_STANDARD	2
TUNING_ACCURATE	3
TUNING_HIGH	4
TUNING_VHIGH	5

In SystemVerilog or Verilog compiled with vlog (but not Verilog compiled with valog), then the numerical values must be used.

For VHDL and VHDL-AMS, the following predefined values for the TUNING parameter are available in the `mgc_ams.eldo_parameters` package (see `$MGC_AMS_HOME/src/mgc_ams.vhd`):

```
constant TUNING_FAST      : real := 1.0;
constant TUNING_STANDARD : real := 2.0;
constant TUNING_ACCURATE : real:= 3.0;
constant TUNING_HIGH     : real:= 4.0;
constant TUNING_VHIGH    : real:= 5.0;
```

HDL Access

Precision parameters apply to analog simulation only, but you can access them through HDL descriptions in VHDL, VHDL-AMS, Verilog, Verilog-AMS and System-Verilog.

Access From Verilog, Verilog-AMS and SystemVerilog System Functions

- To read global precision parameter values:

```
$get_global_accuracy ("<parameter>")
```

Returns the value of precision parameter named `<parameter>` set for the whole design, (the global accuracy in effect at the moment of the call). Used alone, this will return a numerical value. You can make this more comprehensive by using:

```
$display("The TUNING is $g", $get_global_accuracy("TUNING"));
```

This will return the following:

```
# The TUNING is 2
```

- Use the following syntax inside structured procedures beginning with `initial` or `always`, or inside an analog event control statement in the analog block, to set global precision parameter values:

```
$set_global_accuracy("<parameter>", <expr_value> [, <expr_delay>]);
```

This sets the value of precision parameter named `<parameter>` to the value `<expr_value>` for the whole design. It will take effect from the time of the call until the time of the call plus `<expr_delay>` (or till the end of the simulation if no delay is specified). For example:

```
@(initial_step)
  $set_global_accuracy("EPS", 1e-2, 100n);
```

This will set EPS to 1e-2 from time 0 to 100ns.

Caution

If you write the following directly in an analog block, the EPS parameter would be continually set with a delay of 100ns and would never expire:

```
$set_global_accuracy("EPS", 1e-2, 100n);
```

For TUNING, use the macros as described in [Table 7-5](#).

- To read precision parameter values for an instance:

```
$get_instance_accuracy ("<instance_name>", "<parameter>")
```

Returns the value of precision parameter named `<parameter>` for an instance of hierarchical name `<instance_name>`.

- Use the following syntax inside structured procedures beginning with `initial` or `always`, or inside an analog event control statement in the analog block, to set the precision parameter value for an analog or mixed-signal instance:

```
$set_instance_accuracy ("<instance_name>", "<parameter>", <expr_value> [,
0|1])
```

Sets the value of precision parameter named `<parameter>` to `<expr_value>` for the instance specified by its hierarchical name `<instance_name>`. The value set takes effect immediately after the call, and continues until the end of the simulation.

- To set precision parameter values for all instances of an analog or mixed-signal model:

```
$set_model_accuracy ("<model>", "<parameter>", <expr_value>)
```

Sets the value of precision parameter named <parameter> to <expr_value> for all instances of modules, entities or SPICE subcircuits named <model>.

Note

When setting values for accuracy parameters, the instance or model must be analog or mixed-signal; the accuracy parameters have no meaning for a digital model.

Access From VHDL and VHDL-AMS

- To read global precision parameter values:

```
v := get_global_accuracy ("<parameter>");
```

Returns the value of precision parameter named <parameter> set for the whole design, (the global accuracy in effect at the moment of the call). Used alone, this will return a numerical value.

- To set global precision parameter values:

```
set_global_accuracy ("<parameter>", <expr_value> [, <expr_delay>]);
```

This sets the value of precision parameter <parameter> to the value <expr_value> for the whole design. It will take effect from the time of the call until the time of the call plus <expr_delay> (or until the end of the simulation if no delay is specified).

- To read precision parameter values for an instance:

```
v := get_instance_accuracy ("<instance_name>", "<parameter>");
```

Returns the value of precision parameter named <parameter> for an instance of hierarchical name <instance_name>.

- To set a precision parameter value for an instance:

```
set_instance_accuracy ("<instance_name>", "<parameter>", <expr_value> [, <boolean_expr>]);
```

Sets the value of precision parameter named <parameter> to <expr_value> for the instance specified by its hierarchical name <instance_name>. The value set takes effect immediately after the call, and continues until the end of the simulation.

- To set precision parameter values for all instances of a specific model:

```
set_model_accuracy ("<model>", "<parameter>", <expr_value>);
```

Sets the value of precision parameter named <parameter> to <expr_value> for all instances of modules, entities or SPICE subcircuits named <model>. The value set takes effect immediately after the call, and continues until the end of the simulation.

Tcl Access

The Tcl commands `accuracy get` and `accuracy set` are also available to read and control accuracy. Refer to [accuracy get](#) and [accuracy set](#) in the *Questa ADMS Command Reference* for their full syntax descriptions.

Note

Use the `-recursive` option to set accuracy parameters for all the child instances of given instance.

Order of Precedence

If you use two or more different methods to set the accuracy of the same parameter, the following order of precedence will apply:

1. Tcl access
2. HDL (in order: instance, model, global)
3. SPICE (in order: instance, model, global)

Related Topics

[accuracy get \[Questa ADMS Command Reference\]](#)

[accuracy set \[Questa ADMS Command Reference\]](#)

Accuracy Control Options for Analog Simulation

You can use a set of relative and absolute accuracy parameters to control the accuracy of the simulation resolution for the analog solver.

A set of relative and absolute accuracy parameters EPS, VNTOL, ABSTOL, and RELTOL can be used to control the accuracy of the simulation resolution for the analog solver. Decreasing the value of each parameter increases the accuracy. These accuracy parameters are specified in the command file (.cmd).

- EPS

Controls the overall simulation accuracy. The maximum acceptable value of this parameter is 1.0×10^{-2} (error fixed to 1%). The command syntax is:

```
.OPTION EPS=val
```

The default value of EPS is determined by the highest voltage levels in the circuit, based upon the analysis of independent voltage sources. If all voltage levels are below 1.9V, then the default EPS is set to 1mV. If any voltage level is higher than 1.9V, then the default EPS is set to 5mV. Explicitly specifying an EPS value overrides this rule and sets EPS to the specified value regardless of the voltage sources present in the circuit. This voltage source rule is also ignored if EPS is set indirectly via .OPTION TUNING.

Note

If you don't set the EPS value, then the value inside the .chi file and the value reported in the transcript may differ, because the value in the transcript is the value used by Eldo during simulation.

- VNTOL

Controls the absolute accuracy of the ACROSS and FREE quantities during analog simulation. You should adjust the value of this option relative to the magnitudes of these quantities. The command syntax is:

```
.OPTION VNTOL=val
```

The default value is 1μv.

- ABSTOL

Controls the absolute accuracy of the THROUGH quantities during analog simulation. You should adjust the value of this option relative to the magnitudes of these quantities.

The default ABSTOL parameter value is VNTOL × ITOL. The command syntax is:

```
.OPTION ABSTOL=<val>
```

See [.OPTION ITOL](#) in the *Eldo Reference Manual*.

- RELTOL

Controls the relative accuracy during analog simulation and has an implicit influence on the time-step: the smaller the RELTOL parameter value, the smaller the time-step. The maximum acceptable value of this option is fixed to 1.0×10^{-2} (error fixed to 1%). The command syntax is:

```
.OPTION RELTOL=<val>
```

The default RELTOL parameter value is 1.0×10^{-3} .

Example

Suppose that P is an ACROSS quantity and Q is a THROUGH quantity. For P, convergence is reached when:

$$|P(i) - P(i-1)| < \text{RELTOL} * |\max(|P(i)|, |P(i-1)|)| + \text{VNTOL}$$

$P(i)$ is the value of the ACROSS quantity at the current iteration and $P(i-1)$ is the value of the ACROSS quantity at the previous iteration.

For Q , convergence is reached when:

$$|Q(i) - Q(i-1)| < \text{RELTOL} * |\max(|Q(i), Q(i-1)|)| + \text{ABSTOL}$$

$Q(i)$ is the value of the THROUGH quantity at the current iteration and $Q(i-1)$ is the value of the THROUGH quantity at the previous iteration.

If the magnitude of the ACROSS quantity P is $\approx 10.0 \times 10^3$, and if RELTOL is fixed to 1.0×10^{-3} , then the VNTOL option is neglected if it is set to 1.0×10^{-5} . The VNTOL option has to be adjusted depending on the magnitude of the P quantity, in order for it to be taken into account.

Note

When EPS is less than or equal to 1.0×10^{-4} , it is used only for time-step estimation. Accuracy then depends only on VNTOL and RELTOL. The value of EPS also affects values of RELTOL and VNTOL, unless the latter have been set via the .OPTION command.

See also **.OPTION** in *Eldo Reference Manual*.

Related Topics

[Controlling Simulation Accuracy](#)
[Performance/Accuracy Trade-off Options](#)

Crossing Time Accuracy Options Summary

You can use the CROSS operator to generate a monitored analog event. This enables you to detect threshold crossings in analog signals. The behavior of the CROSS operator is affected by several accuracy options.

The CROSS operator generates a monitored analog event to detect when an expression crosses zero (0) in the specified direction. The arguments of the CROSS operator control the simulator time steps to accurately resolve the crossing:

```
cross(expr [, dir [, time_tol [, expr_tol ] ] ] );
```

The dir argument is an integer which must evaluate to +1, 0 or -1 and represents the direction(s) of the crossings to be detected. A value of +1 causes only rising transitions to be detected, a value of -1 causes only falling transitions to be detected, and a value of 0 (the default) causes both rising and falling transitions to be detected. time_tol and expr_tol are optional real values that apply a tolerance to the time step and expression values respectively. If the tolerances are not defined the simulator will set the tolerances as detailed below.

The value of the **THTOLSPI** option corresponds to `expr_tol` in the CROSS operator definition. The `expr_tol` argument overrides THTOLSPI because information from Verilog-AMS has priority over SPICE options. If the option NOTHTOLSPI is specified then `expr_tol` is given a high value (1V). The value of the **TTHRESOL** option corresponds to `time_tol` in the CROSS operator definition. `time_tol` overrides TTHRESOL. The default value is $10 \times \text{HMIN}$ (where the default value of HMIN is 10^{-12} sec). The behavior is the same for both Verilog-A and Verilog-AMS.

Setting the value of `time_tol` to below HMIN (either by specifying `time_tol` directly or by setting it via TTHRESOL) causes a warning message to be reported, and `time_tol` is reset to the HMIN value.

THTOLSPI and TTHRESOL are equivalent to cross tolerances. They represent the maximum allowable error between the estimated crossing point and the true crossing point. The crossing event occurs when both tolerances are satisfied: when the simulation time is within $\pm \text{time_tol}$ of the “actual” crossing and $\text{abs}(\text{signal_level}) < \text{expr_tol}$.

If neither NOTHTOLSPI nor THTOLSPI is used, the default for `expr_tol` is defined according to the value of EPS. The EPS option controls the overall simulation accuracy. Based on the value of EPS, default values are given to a group of intermediate tolerance parameters internal to the simulator. The default value of EPS is 5.0×10^{-3} .

Note

If you set the value of EPS, the value inside the `.chi` file and inside the transcript will be the same. If you do not set the value of EPS, the value inside the `.chi` file and the transcript could be different because the value in the transcript is the value used by Eldo during the simulation. Refer to [Accuracy Control Options for Analog Simulation](#) for more information.

If both THTOLSPI and NOTHTOLSPI are specified, the last one in the order of the `.cmd` or `.cir` file is considered.

See also [Improving DC Convergence](#) in *Eldo User's Manual*.

Related Topics

[Performance/Accuracy Trade-off Options](#)
[Controlling Simulation Accuracy](#)

Performance/Accuracy Trade-off Options

During analog simulation, the time-step is varied depending on estimates of the truncation error or convergence. You can use a set of options to control the time-step.

You should specify the time-step options in the command file (`.cmd`).

You must know the system to be simulated in depth—particularly the time constants of the different models—to tune these options in an appropriate way in order to obtain results with the required accuracy.

ALTCROSS

```
.OPTION ALTCROSS
.OPTION NOALTCROSS
```

The ALTCROSS option enables an efficient algorithm for the calculation required by the VHDL-AMS 'above' attribute and the Verilog-A/Verilog-AMS @cross function. The improvement over the previous (legacy) default is, in some cases, highly significant, but for backwards compatibility you can use the legacy algorithm by specifying the NOALTCROSS option. The ALTCROSS option is enabled by default.

For example, suppose that the simulator chooses two successive solution points at times T_n and T_{n+1} while executing a given model. Now suppose that the model is altered so that a crossing event occurs at time T_c where:

$$T_n < T_c < T_{n+1}$$

No other change is made. Then the simulator will compute solution points for the altered model at times T_n , T_c , and $T_c + (T_{n+1} - T_n)$, and the crossing event has no further effect on the calculation of the time step.

When using the NOALTCROSS option, the timestep is reduced to $T_c - T_n$ following the calculation of T_c , and recovers exponentially after T_c .

HMIN

```
.OPTION HMIN = <value>
```

The HMIN option specifies the minimum internal time-step. The value of HMIN must be at least a magnitude smaller than the fastest time constants that need to be accurately represented by the model. The default value of HMIN is 1.0 picosecond.

The value of HMIN must be smaller than the step size required to obtain a proper local truncation error. The three integration methods available within Questa ADMS (Trapezoidal, Backward-Euler, and Gear) estimate the magnitude of the error. This estimate is used to adjust the time-step to provide an estimated local truncation error that is small enough according to your specification.

Note

If you increase the value of HMIN (without losing accuracy), it will save CPU time.

When the value of HMIN is important and the Modified Lock Step algorithm is used, exceptionally, analog/digital synchronization step conflict may occur during the simulation. Questa ADMS points out these convergence problems with the following error message:

```
#Error: Analog/digital synchronization step conflict.
Use .OPTION mixedstep=locked
```

These conflicts can be bypassed in the following ways:

- Decrease or remove the HMIN option with the Modified Lock Step algorithm.
- Switch to the Locked Step algorithm by using the option mixedstep=locked.

You can set this option in the following locations:

- In the command file (extension *.cmd*), if it exists.
- In the circuit file (extension *.cir*), if it exists.
- In a new command file, if no command file or circuit file already exists. To do this, click **New** in the **Start Simulation Dialog Box**, then click **More** in the Spice Commands dialog box and enter mixedstep=locked in the Additional Options text box.

ABSOLUTE_HMIN

```
.OPTION ABSOLUTE_HMIN = <value>
```

The ABSOLUTE_HMIN option specifies the absolute minimum internal timestep. This is similar to the HMIN option, except that the simulator will never try to further reduce the specified value if it cannot converge. If both ABSOLUTE_HMIN and HMIN are specified, the last option specified is used.

HMAX

```
.OPTION HMAX = <value>
```

The HMAX option specifies the maximum internal time-step. If an Eldo SIN source is used, the default value of HMAX is 1/10 of the wave period. If no Eldo SIN source is used, and the HMAX option has not been set, the value of HMAX is unspecified.

If the behavioral models compute SIN signals, for example, it is important that you set HMAX in the command file to be 1/10 of the wave period. This will ensure that the results have the required accuracy.

TTHRESOL

```
.OPTION TTHRESOL = <value>
```

For crossing time accuracy, you can use the TTHRESOL option to specify the crossing threshold time resolution. The value of TTHRESOL affects the Verilog-A CROSS operator and the VHDL-AMS 'ABOVE attribute. The default value of TTHRESOL is 10×HMIN.

THTOLSPI

```
.OPTION THTOLSPI = <value>
.OPTION NOTHTOLSPI
```

The THTOLSPI option specifies the accuracy of the value tolerance algorithm for crossing time accuracy. The default value of THTOLSPI is 1m.

Two crossing time accuracy algorithms are available: one with value tolerance (default), the other with time tolerance. The value tolerance algorithm is faster when computing threshold crossing points (for the VHDL-AMS 'ABOVE attribute and Verilog-AMS @CROSS operator). The value tolerance algorithm computes a crossing point with a Y value accuracy. For a lower than default accuracy, decrease the Y accuracy to 100u, for example.

Depending on the behavioral model description, the increased speed of the value tolerance algorithm may cause some issues. If this occurs, either adjust the value of THTOLSPI or use the time tolerance algorithm instead of the value tolerance algorithm. The NOTHTOLSPI option enables the time tolerance algorithm for crossing time accuracy. The [TTHRESOL](#) option specifies the crossing threshold time resolution.

NOROUND_ADMSTIME

```
.OPTION NOROUND_ADSMTIME
```

The default algorithm used in A2D conversion rounds the time-step in Questa ADMS simulations to ensure no events are missed, but may induce time-step changes in the output. Use .OPTION NOROUND_ADMSTIME in the netlist or add -no_round_admstime to the vasim command to disable the default algorithm behavior and reduce simulation time.

Related Topics

[Crossing Time Accuracy Options Summary](#)
[Controlling Simulation Accuracy](#)

Analog Solver Delayed Start

You can delay the analog solver computations.

It is possible to delay the analog solver computations until either a user-specified time or a call from the [analog_start](#) command, the VHDL/VHDL-AMS package analog_start, or the Verilog/Verilog-AMS system task \$analog_start. This is useful for designs that require extended digital initialization before the analog part is computed, thus saving simulation time. It is very useful when the analog part has no influence

during a long digital initialization. This command must be specified in the SPICE *.cmd* or *.cir* file using the following syntax:

```
.ADMS_START <time_value>|external
```

time_value corresponds to the delay before starting the analog solver computation, and can be specified as a parameter or as an expression. During the period that the analog solver does not perform any computations, if a value is requested for an analog point (Quantity, Terminal or A2D built-in boundary element), 0.0 is assumed.

When external is specified, Questa ADMS will wait for a call from the [analog_start](#) command, the VHDL/VHDL-AMS package analog_start, or the Verilog/Verilog-AMS system task \$analog_start before starting the analog solver computation. A DC computation is performed when the analog solver starts computing points.

Activate Analog Solver from VHDL/VHDL-AMS

The analog solver can be activated from within a VHDL description or the digital part of a VHDL-AMS description using the procedure analog_start. For example:

```
library Mgc_AMS;
use Mgc_ams.analog_start.all;
...
p1:process
  begin
    if (sig1=TRUE) then
      analog_start;
    end if;
    wait on sig1;
  end process p1;
```

Activate Analog Solver from Verilog/Verilog-AMS

The analog solver can be activated from within a Verilog description or the digital part of a Verilog-AMS description using the system task \$analog_start. For example:

```
initial
begin
  z = 0;
  #2000 $analog_start;
end
```

Related Topics

Accuracy Control Options for Analog Simulation

Speeding Up Elaboration of SPICE Subcircuits

The SPICE command `.OPTION ADMS_FAST_PARSE` instructs Questa ADMS to speed up the elaboration of subcircuits containing only SPICE instances.

Syntax:

```
.OPTION adms_fast_parse=<subckt_name>
```

`<subckt_name>` specifies the full path to the Eldo subcircuit instance.

The performance improvement and memory gain are proportional to the combined size of the hidden subcircuit instances.

You cannot have more than one `<subckt_name>` within the same `.OPTION` command, but this option can be specified several times within the same netlist, for example:

```
.OPTION adms_fast_parse=<subckt_name_1>
.OPTION adms_fast_parse=<subckt_name_2>
.OPTION adms_fast_parse=<subckt_name_3>
```

Alternatively you can use the `*` wildcard character to specify the option for multiple similar subcircuit names declared in the current hierarchical level:

```
.OPTION adms_fast_parse=foo*
```

Limitations

- Subcircuits that instantiate HDL model instances cannot be parsed using this option. It can only be applied on a SPICE subcircuit if its subhierarchy contains only SPICE objects.
- The `vasim` options `-g` and `-G` cannot affect a parameter located in SPICE model associated with `ADMS_FAST_PARSE` option.
- Parameters located in a SPICE model identified by `ADMS_FAST_PARSE` option are not visible.

Related Topics

[Accuracy Control Options for Analog Simulation](#)

Improving Performance and Reducing Memory Consumption

You can instruct Questa ADMS to speed-up the elaboration of subcircuits to improve performance and reduce memory consumption.

The SPICE command

```
.OPTION auto_fastparse
```

instructs Questa ADMS to speed-up the elaboration of subcircuits to improve performance and reduce memory consumption. This is the default. Optimization is then inhibited for a subcircuit instance and for its ancestors in the hierarchy if:

- the instance:
 - is the target of a .bind statement
 - is the target of a hierarchical reference
 - is the target of a SystemVerilog bind statement
 - is the target of a SystemVerilog defparam statement
 - is the target of a vasim command line switch +acc that mentions the model name
 - contains an HDL instance
 - is identified by a .PDMAIN spice command
- Questa ADMS has been invoked with FLI, in other words vasim -foreign

AUTO_FASTPARSE is equivalent to ADMS_FAST_PARSE=*, except that the excluded cases listed above are automatically eliminated.

Parameters of a subcircuit optimized with AUTO_FASTPARSE support modifications through the -g/-G switch. The subcircuit itself is visible in the Source window when selected in the Structure window.

To disable use the non-optimize

```
.option AUTO_FASTPARSE=0
```

Related Topics

Speeding Up Elaboration of SPICE Subcircuits

Multi-Threading

You can activate multi-threading for DC, transient, RF, and AC analog simulations to speed-up simulation.

Multi-threading works best for designs with a large number of SPICE devices, and designs containing analog portions of Verilog-A/Verilog-AMS.

Multi-threading is enabled by invoking ADMS using the following command:

```
vasim -eldoopt "-use_proc <n>"
```

Most of the SPICE analog devices and all the analog portions of Verilog-A/Verilog-AMS models can be evaluated in parallel (MOSFETs, resistors, capacitors, diodes, BJTs) and therefore Questa ADMS will load these in parallel.

All of the analog portions of the VHDL-AMS models will be in the same thread, so their evaluation is done in sequence rather than in parallel.

Pure digital HDL models or portions of designs partitioned to ADiT for simulation are not multi-threaded.

Note

It is not recommended to use more than 4 threads when running Questa ADMS-ADiT simulations. A significant performance degradation may occur.

See also [Multi-Threading Eldo Simulations](#) in *Eldo User's Manual*.

Using Questa ADMS Premier

Questa ADMS supports the Eldo Premier simulator, which provides an increase in performance and capacity without sacrificing accuracy compared to Eldo Classic. This enables simulation of much larger circuits.

Tip

Refer to [Eldo Premier](#) in *Eldo User's Manual* for more information.

Restrictions and Limitations

- Quantity ports in VHDL-AMS interface declarations are not supported.

Procedure

Enter the following command to invoke Questa ADMS using Eldo Premier:

```
vasim -premier
```

VHDL-AMS Attribute 'MGC_FOLLOW

Creates a signal that follows a scalar quantity. 'MGC_FOLLOW is twice as fast and simpler to use than 'ABOVE.

Syntax

```
Q'MGC_FOLLOW(DELTA[, TIME_TOL])
```

Parameters

- DELTA
A static expression of the same type as Q that evaluates to a positive value. This is the minimal value change which may trigger an event on the result signal.
- TIME_TOL
A static expression of type REAL that evaluates to a non-negative value. If omitted, it defaults to 0.0. This is the minimal time between two consecutive events on the result signal.

Description

The prefix to 'MGC_FOLLOW can be any scalar quantity denoted by the static name Q. The result type will be the subtype of Q. This signal is updated when:

- The difference between its value and the value of Q is greater than DELTA
- The sign of the slope of Q changes, with the restriction that the time between two consecutive events shall not be less than TIME_TOL
- There is an event on DOMAIN.

Preoptimizing Pure Digital Subregions

A block of pure digital code with a single Verilog module or VHDL architecture at the top can be optimized to reduce load time and simulation run time.

This works best with large blocks that contain many hierarchically instantiated instances. First compile all of the modules, entities and architectures with vlog or vcom. Then pass the top level module or architecture of the block to the vopt command:

```
vopt <digital_module_name> -pdu -o <optimized_module_name>
```

This creates a preoptimized version of the digital subregion, where `digital_module_name` is the name of a pure digital subregion instantiated in your design, and `optimized_module_name` is a name you specify for the preoptimized version of it. During elaboration of your design, wherever an instantiation of `digital_module_name` is encountered, it will automatically be replaced by the preoptimized version.

You can preoptimize any pure digital region, providing that all instantiated modules within it are also pure digital. Additionally, a preoptimized digital module may instantiate other preoptimized digital modules.

Refer to Preoptimizing Regions of Your Design in the *Questa SIM User's Manual* for details concerning the management of configuration design units, code coverage, visibility of nets in optimized regions and other relevant details.

Related Topics

[vopt \[Questa ADMS Command Reference\]](#)

Simulation Time Reporting

You can control the rate at which status information is reported to the transcript during simulation.

Status information recorded includes the elapsed CPU time, estimated total CPU time, percentage of simulation done, plus any data selected using the [STAT](#) option.

You can specify the interval between status reports using the simulator variable `UpdateSimulationTimeDelayBatch`. Declaring a value explicitly will override the setting in the *modelsim.ini* file. The default value is 10000, meaning that status information is reported every 10 seconds.

Alternatively, you can specify the number of times that status information is reported using the [simudiv](#) command, the [vasim](#) -simudiv argument, or the SPICE command `.OPTION .OPTION SIMUDIV`. The default value is 0, meaning that status information is reported in accordance with the value of the simulator variable `UpdateSimulationTimeDelayBatch`.

Note

You cannot use this feature in conjunction with `.OPTION TIMEDIV`.

If you use more than one method to specify the rate at which status information is reported, the following rules of precedence apply (from highest precedence to lowest precedence):

1. `simudiv [<value>]`
2. `vasim -simudiv <value>`
3. `.OPTION SIMUDIV=<value>` in a SPICE command file
4. `UpdateSimulationTimeDelayBatch = <value>` in *modelsim.ini*

To disable status information reporting, set `UpdateSimulationTimeDelayBatch` to 0.

Controlling HDL Domain Errors

The SPICE command `.OPTION AHDLDOMAINERROR` controls how Questa ADMS handles HDL mathematical domain errors.

This option handles out of range errors for the following mathematical functions coming from Verilog-A(MS) and VHDL-AMS: `ln`, `log`, `pow`, `sqrt`, `tan`, `asin`, `acos`, `acosh`, and `atanh`.

Syntax:

```
.OPTION ahldomainerror=none|error|erroriter
```

When set to `none`, the default, when an out of range error occurs the simulation continues using the nearest acceptable value.

Set to `error` to stop the simulation at the end of the accepted time point and generate an error message. Or set to `erroriter` to stop the simulation at the current iteration and generate an error message.

Monte Carlo Analysis

Monte Carlo analysis (`.MC` command) is a series of DC, AC, or TRAN analyses, where one or more circuit or model parameters follow a probability distribution. The distribution type can be uniform, Gaussian, or user-defined.

This kind of analysis can only be applied to elements that are on the SPICE description level; this is performed using the same `.MODEL` card syntax as in Eldo and can be applied to SPICE devices or SPICE parameters. Such SPICE parameters may be passed to a generic/parameter of a behavioral description, or can be accessible from a `$param` Verilog-AMS function call.

- For a Monte Carlo analysis on SPICE devices, the LOT, DEVX, and DEV variations can be used.
- For a Monte Carlo analysis on SPICE parameters, the LOT and DEVX variations can be used. The DEV variation can be used if the parameters are passed to SPICE devices. However, DEV variation cannot be used if the parameters are passed on a Y instance; an error is generated. To change this error to a warning, to enable you to continue the simulation, specify:

```
.OPTION ADMSMCDEVSEV=WARNING
```

By default, the ADMSMCDEVSEV option is set to error.

Refer to [Monte Carlo Analysis](#) in *Eldo User's Manual* for more information.

Example of Monte Carlo Analysis with VHDL-AMS Generics

To perform a Monte Carlo analysis on VHDL-AMS generics, you must perform the Monte Carlo analysis on some SPICE parameters, and pass the SPICE parameter values to the VHDL-AMS generics.

For example, the netlist below defines the parameter `res_val` with a value of 1k and a LOT specification of 15%. The VHDL-AMS model is declared on a `.MODEL` command, and is instantiated in the SPICE netlist. The SPICE parameter `res_val` is passed to the VHDL-AMS generic `r` on the instantiation line:

```
.PARAM res_val=1k LOT=15%
.MODEL resist_fix macro lang=vhdlams mod=resistor_fixed
Yres2 resist_fix load 0 PARAM: r=res_val
.OPTION CARLO_GAUSS
.MC 100 ALL
```

The Monte Carlo analysis will perform 100 runs with a Gaussian distribution. The VHDL-AMS design unit is shown below:

```
LIBRARY ieee;
USE ieee.electrical_systems.ALL;
LIBRARY MGC_AMS;
USE MGC_AMS.eldo_parameters.ALL;
  ENTITY resistor_fixed IS
    GENERIC (R : REAL := 1.0e3);
    PORT ( TERMINAL t1, t2 : ELECTRICAL);
  END ENTITY resistor_fixed;

  ARCHITECTURE simple_fixed OF resistor_fixed IS
    QUANTITY v ACROSS t1 TO t2;
    QUANTITY i THROUGH t1 TO t2;
    BEGIN
      v==i*R;
  END ARCHITECTURE simple_fixed;
```


8. Verification

You can debug a design under test using Questa ADMS.

Interactive Debug.....	8-1
Simulation Statistics.....	8-23
Integration with Questa SIM Verification Methodology.....	8-30
Universal Verification Methodology.....	8-73

Interactive Debug

You can debug a design under test in the Questa ADMS GUI.

Setting File-Line Breakpoints.....	8-1
Adding and Removing Breakpoints.....	8-2
Modifying File-Line Breakpoints.....	8-3
Saving and Loading Breakpoints.....	8-4
Setting Break Severity.....	8-4
Interrogating Nets in the Contributor Window.....	8-6
Visualizing Current Flow in the CurrentFlow Window.....	8-8
Interactive Forcing and Loading.....	8-10
Non-Convergence Errors.....	8-14
Recompiling Design Units in the Source Window.....	8-15
Diagnosis Mode.....	8-16
Verbose Error Message Reporting.....	8-19
Limiting Message Display.....	8-20
Viewing Messages in the Message Viewer.....	8-21

Setting File-Line Breakpoints

You can add, remove, enable or disable breakpoints on your executable source code in the Source window.

These debugging capabilities are available for all HDL languages, and the analog sequential parts of Verilog-AMS descriptions. The analog parts of VHDL-AMS and Verilog-AMS, are not supported.

Note

Debug mode for analog items is disabled by default. It should be enabled during compilation using the option `valog -analogdebug`.

The \$error and \$warning system tasks behave as a breakpoint when the BreakOnAssertion variable is set to the appropriate level.

Limitations When Debugging Analog Objects

When adding breakpoints inside analog blocks and functions, the following restrictions exist:

- Breakpoints may only be hit during a transient analysis.
- References to analog objects using conditional expressions are not supported (the use of bp -cond or when commands).
- Stepping into executable analog code may introduce some minor changes in the results introduced by numerical noise.
- These debugging capabilities are not available for multi-threaded simulations.

Related Topics

[Source Window](#)

Adding and Removing Breakpoints

You can add a breakpoint to a line of executable source code.

Note

Executable lines of code where breakpoints can be set are indicated in red. Executable lines of code that cannot have breakpoints set are indicated in blue.

Restrictions and Limitations

- Debugging in this way is not possible within a model that has been compiled with the -nodebug or -noanalogdebug options.

Procedure

Do one of the following:

If you want to...	Do the following...
Add a breakpoint to a line.	Click the line number in the Source window. A red breakpoint marker ● will appear.

If you want to...	Do the following...
	You can also use the bp command to add a breakpoint from the Transcript window. For example, enter the following command to set a breakpoint in the file <i>top.vhd</i> at line 147: <pre>bp top.vhd 147</pre>
Disable or enable a breakpoint.	Click the breakpoint marker.
Edit a breakpoint.	Right-click the breakpoint marker, then choose Edit Breakpoint. The File Breakpoint Dialog Box opens.
Edit all breakpoints set in the current design.	Right-click a breakpoint marker, then choose Edit All Breakpoints. The Modify Breakpoints Dialog Box opens.
Delete a breakpoint.	Right-click the breakpoint marker, then choose Remove Breakpoint .

Results

You can use the Modify Breakpoints dialog box to add, modify, disable, delete, load, and save breakpoints. New breakpoints are based on either:

- A signal or signal value.
- A file and line number.

Modifying File-Line Breakpoints

You can modify a breakpoint according to the line number in a source file.

Procedure

1. To modify a breakpoint according to the line number in a source file, do any one of the following to display the **Modify Breakpoints Dialog Box** which provides a list of all breakpoints in the design:
 - Choose **Tools > Breakpoints** from the Main menu
 - Right-click a breakpoint and select **Edit All Breakpoints** from the popup menu.
2. To modify a breakpoint, select a file-line breakpoint from the list.
3. Click Modify, which opens the **File Breakpoint Dialog Box**.
4. Fill out any of the following fields to modify the selected breakpoint:

- Breakpoint Label — Designates a label for the breakpoint.
- Instance Name — The full pathname to an instance that sets a SystemC breakpoint so it applies only to that specified instance.
- Breakpoint Condition — One or more conditions that determine whether the breakpoint is observed. You must enclose the condition expression within quotation marks (""). If the condition is true, the simulation stops at the breakpoint. If false, the simulation bypasses the breakpoint. A condition cannot refer to a VHDL variable (only a signal).
- Breakpoint Command—A string, enclosed in braces ({}), that specifies one or more commands to be run at the breakpoint. Use a semicolon (;) to separate multiple commands.

Tip

When filling in the fields in the File Breakpoint dialog box, you should use the same syntax and format as the -inst switch, the -cond switch, and the command string of the bp command.

5. Click **OK** to close the File Breakpoint dialog box.
6. Click **OK** to close the [Modify Breakpoints Dialog Box](#).

Saving and Loading Breakpoints

You can save and load breakpoints to and from a do file using the Save and Load buttons on the Modify Breakpoints dialog box.

- Choose **Tools > Breakpoints** to display the [Modify Breakpoints Dialog Box](#).
 - Click **Save** to save the breakpoints in the current design. You will be prompted to choose a filename and location for the do file.
 - Click **Load** to load a previously saved do file containing breakpoint information.

Related Topics

[Modify Breakpoints Dialog Box](#)

[Interactive Debug](#)

[Startup Files \(do Files\)](#)

[bp \[Questa ADMS Command Reference\]](#)

[bd \[Questa ADMS Command Reference\]](#)

[Modifying File-Line Breakpoints](#)

Setting Break Severity

You can change the severity level for the current simulation.

By default, simulation stops when a severity level of FAILURE is reached. You can change the severity level for the current simulation run using the [Runtime Options Dialog Box](#), accessed by selecting **Simulate > Runtime Options**.

You can also configure message logging using this dialog box, such that messaging will be disabled for certain levels of severity.

To change the default severity level, edit the BreakOnAssertion variable in the *modelsim.ini* file. Refer to BreakOnAssertion in the *Questa SIM User's Manual* for more information.

Controlling Message Severity Levels..... 8-5

Controlling Message Severity Levels

You can suppress or change the severity of notes, warnings, and errors.

You can suppress or change the severity of notes, warnings, and errors, using the following *vasim* options:

```
vasim [-suppress|-error|-fatal|-note|-warning]
      ["<tool>-<msgNum>|<msgNum>{ <tool>-<msgNum>|<msgNum>}" |
       <tool>-<msgNum>|<msgNum>{, <tool>-<msgNum>|<msgNum>}]
```

<tool> can be *vsim* for Questa SIM messages (default) or *vasim* for Questa ADMS messages.

You cannot change the severity of or suppress fatal or internal *vasim* messages.

You can specify a permanent default in the [msg_system] section of the *modelsim.ini* to change the severity of all warnings to errors (warning=error). This setting is overwritten when using *vasim -warning*.

Caution

Under rare circumstances, a Tcl script may misbehave if the severity of a message is changed. If a message is changed from "error" to "warning", and the warning occurs, the script may terminate in spite of the change. Conversely, if a message is changed from "warning" to "error", the script may not terminate as expected. In the former case, adding the command *onerror resume* to the Tcl script can be used as workaround.

Additionally, you can use the **suppress** command to prevent the display of a specific message or messages.

Related Topics

[suppress \[Questa ADMS Command Reference\]](#)

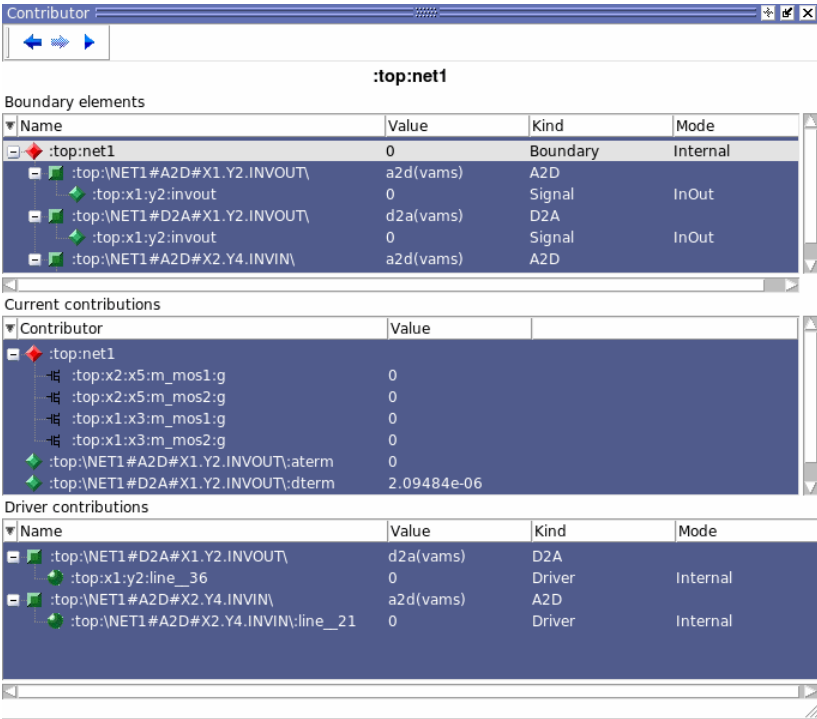
Interrogating Nets in the Contributor Window

You can use the Contributor Window along with the Structure, Objects, and CurrentFlow windows to visualize the connectivity of any net, and to read current and digital driver contribution values.

The Contributor window displays information about a selected net, threading through a hierarchy of instances. It shows how a net or port gets its value by displaying all the contributions associated with it.

The **Structure Window** shows the design’s element hierarchy. When you select an instance in the Structure window, the **Objects Window** displays the objects contained in the selected instance. If during debug the value of a network appears to be incorrect, you can investigate the problem by selecting readers or writers of the network in any instance in the Objects window. The **Contributor Window** then displays all of the members of the network. The contribution of each member to the final value is displayed in the Contributions panels.

Figure 8-1: Interrogating Nets in the Contributor Window



Note
To view data in the Contributor window, you must select an object in the Objects window. Current contributions will only be visible once simulation has begun.

The Contributor window will display the root net as follows:

- If it is a mixed-signal net the Boundary Elements pane lists each A2D and D2A boundary element. Boundary elements in this list can be expanded to show the resulting signal.

- The Current contributions pane will be populated with the current contributions to this net (once simulation has run). You can view a graphical representation of the current contributions in the [CurrentFlow Window](#).
- The Driver contributions pane will be populated with the names and values of each of the digital sources on the selected net.

The sources or drivers coming from [signal_force](#), [init_signal_spy](#) or the [force](#) command are also displayed in the Driver contributions pane. Contributions coming from [init_terminal_contribution](#), [init_terminal_short](#), [init_terminal_reference](#) or the [force](#) -source command are also displayed in the Current contributions pane.

If the selected object is a node used in an [init_terminal_short](#) or [.CONNECT](#) command, then the Shortcuts list is displayed to show the information.

Use the following procedure to interrogate nets in the [Contributor Window](#).

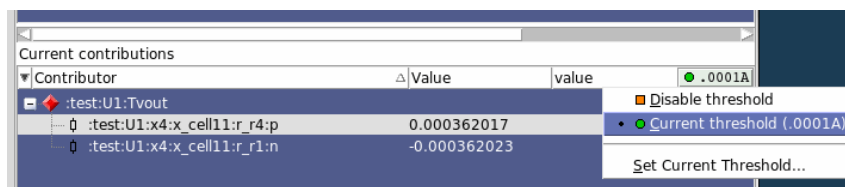
Restrictions and Limitations

Contributions are not shown if they are below the Current Threshold value set in the Contributions field in the [Contributor Window](#).

Procedure

1. When a simulation is paused at a breakpoint, select a region in the Structure window. This updates the Objects window with the list of objects.
2. In the Objects window, right-click on a net or a port and choose **View Contributor**. The Contributor window will display all the objects of that net.
3. To filter out contributions that are less than a specified threshold, click the button on the right of the Current contributions pane, choose **Set Current Threshold** and enter the required minimum current value.

Figure 8-2: Setting Current Threshold



4. Double-click on an item in the Contributor window. The corresponding instance and object in the Structure and Objects windows are highlighted.

If the selected element is an analog contribution coming from Net Spy, the [Source Window](#) opens at the line in which the Spy statement occurred.

5. Right-click on a driver in the Contributor window and select **Goto region**.

The instance of the relative boundary will be selected in the Structure window, and the Objects window and other windows will update according to this new selection.

Use the Go to this node  and scroll buttons   to navigate through the recently selected nodes. Right-click on the toolbar and select Contributor Tools to view these buttons.

Related Topics

Boundary Elements

Viewing Mixed-Signal Nets in Questa ADMS

Viewing Boundary Elements in the Structure Window

drivers [Questa ADMS Command Reference]

tracei [Questa ADMS Command Reference]

Contributor Window

CurrentFlow Window

Visualizing Current Flow in the CurrentFlow Window

The CurrentFlow window shows a graphical representation of the current contributions on a selected object.

Restrictions and Limitations

Contributions are not shown if they are below the Current Threshold value set in the Contributions field in the [Contributor Window](#).

Current values are always displayed in amps.

A negative current value on a pin indicates positive current flowing out.

In a big network it can take a long time to compute all the node current flow values. In this situation a busy icon is shown. It is not possible to interrupt this computation.

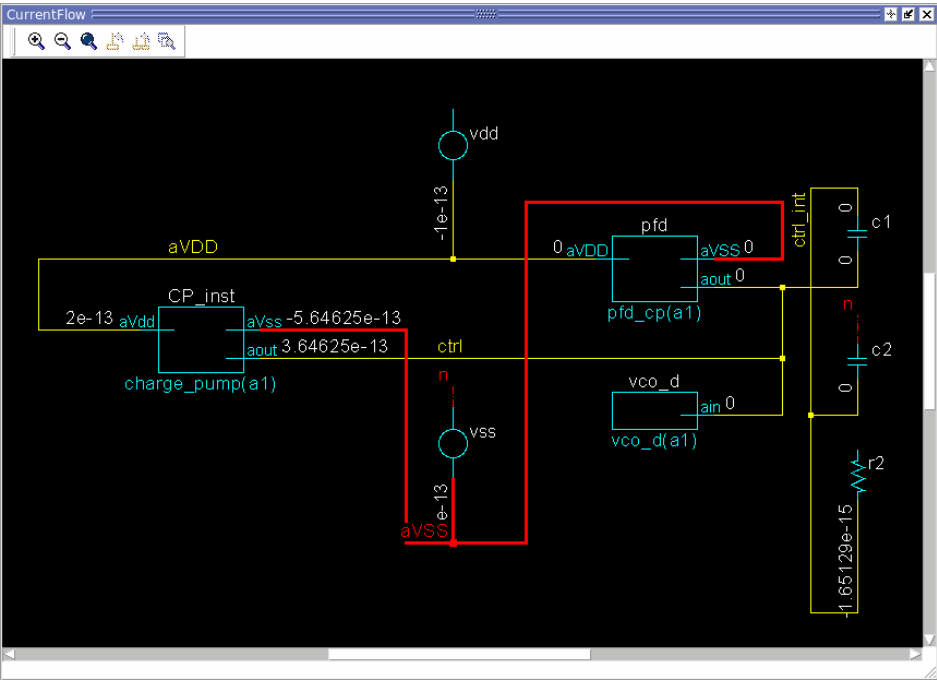
The current flow values may fail to update if the user re-runs the simulation by calling a new 'run' command.

Procedure

1. In the [Contributor Window](#), select an object in the Current Contributions pane, right-click and select View currentflow.

The [CurrentFlow Window](#) opens, showing a graphical representation of the current contributions on the selected object. Selected objects are highlighted in red.

Figure 8-3: Current Contributions in the CurrentFlow Window



2. Choose one of the following actions:

If you want to...	Do the following...
Highlight all current contributions for a specific pin.	Double-click on the red dashed line beside the pin.
Highlight a specific pin in the Objects Window and its corresponding region in the Structure Window.	Single-click on the pin in the CurrentFlow window.
Highlight a specific net in the Objects Window and its corresponding region in the Structure Window.	Single-click on the net in the CurrentFlow window.
Highlight a specific instance's corresponding region in the Structure Window.	Single-click on the instance in the CurrentFlow window.
Show the hierarchical path of an instance.	Hover over the instance in the CurrentFlow window.
Remove a specific net from the schematic.	1. Right-click on the net 2. Choose Remove Selected Net
Zoom in.	1. Right-click in the CurrentFlow Window 2. Choose Zoom In or click and drag to the lower-right

If you want to...	Do the following...
	or when the window is undocked click 
Zoom out.	1. Right-click in the CurrentFlow Window 2. Choose Zoom Out or click and drag to the upper-right or when the window is undocked click 
Zoom full (zoom to fit).	1. Right-click in the CurrentFlow Window. 2. Choose Zoom Full or click and drag to the lower-left or when the window is undocked click 
Clear the CurrentFlow window.	1. Right-click in the CurrentFlow Window. 2. Choose Clear .

Related Topics

- Boundary Elements
- [Viewing Mixed-Signal Nets in Questa ADMS](#)
- [Viewing Boundary Elements in the Structure Window](#)
- [drivers \[Questa ADMS Command Reference\]](#)
- [tracei \[Questa ADMS Command Reference\]](#)
- Contributor Window
- CurrentFlow Window

Interactive Forcing and Loading

A testbench consists of a driver, monitor, responder, and the DUT instance. The driver supplies stimulus; the monitor records or measures results; the responder loads the DUT, responds with suitable acknowledge signals and provides other non-stimulus context emulation.

The interface between testbench and DUT uses the ports of the DUT, but often also includes hierarchical references to elements of the DUT below that level.

[Forcing Signals.....](#) 8-11

[Forcing Analog Terminals.....](#) 8-12

[Loading the TestBench Dynamically.....](#) 8-12

Forcing Signals

You can apply a stimulus to a signal.

Use the following procedure to apply a stimulus to a signal.

1. Right-click on the required signal in the **Objects Window** and select **Modify > Force**

The **Force Selected Signal Dialog Box** will open and the Signal Name field will be populated with the name of the selected signal.

2. Specify a new value for the signal in the Value field. A value can be specified in radices other than decimal by using the form:

```
base#value
```

For example, to specify the hexadecimal value EE, enter:

```
16#EE
```

3. Specify the type of force to use:

- Freeze

Freezes the signal at the specified value until it is forced again or until it is unforced.

- Drive

Sets the signal to the specified value. The value remains until there is a subsequent driver transaction, or until the signal or net is forced again, or until it is unforced.

- Deposit

Sets the signal to the specified value. The value remains until there is a subsequent driver transaction, or until the signal or net is forced again, or until it is unforced.

4. Optionally, enter a value in the Delay field to delay the application of the force. This value is the time relative to the current time, or you can specify an absolute value by preceding the value with the @ character. If no time units are specified, the default is the resolution units selected at simulation startup.

A zero-delay force causes the change to occur in the current, rather than the next, simulation delta cycle.

5. Optionally, enter a value in the Repeat field to set the force to repeat. A repeating force command forces the value before other, non-repeating force commands that occur in the same step.
6. Optionally, enter a value in the Cancel field to cancel the force. This value is the time, relative to the current time. Cancellation occurs at the last simulation delta cycle of a time unit. A value of 0 cancels the force at the end of the current time period.
7. Click **OK** to apply the force and close the dialog box.

8. To remove the force, from an object, right-click on it in the Objects Window and select **Modify > Remove Force**.

Forcing Analog Terminals

You can apply analog stimulus (voltage source) to an analog terminal object.

Use the following procedure to apply analog stimulus (voltage source) to an analog terminal object:

1. Right-click on the required analog terminal in the **Objects Window** and select **Modify > Force**
The **Force Selected Signal Dialog Box** will open and the Net Name field will be populated with the name of the selected object.
2. In the Eldo Parameters field, enter the parameters of the source as an Eldo string, excluding the name of the source and the names of the terminals.
3. Click **OK** to apply the force and close the dialog box.

Note that a force on a terminal may only occur at the beginning of an ASP (see the run and run -next BeginofASP commands) and will be used as a hint to start iterations to compute the values at this ASP.

4. To remove the force, from an object, right-click on it in the **Objects** window and select **Modify > Remove Force**.

Loading the TestBench Dynamically

You can load the TestBench dynamically.

If the driver, monitor and responder are written in SPICE, you can make simple changes to the testbench during the course of preliminary testing. For example, if the driver consists of a periodic voltage source, the frequency of the source might be changed; if it is discovered that a load is required for proper operation of the DUT but the load was omitted from the responder, it can be patched in.

Because it is not possible to instantiate a subcircuit or HDL unit in the command file, the changes are limited to the addition of source or load primitives.

Adding SPICE Elements to an HDL TestBench

Use the Tcl commands **spice add** and **spice delete** to add or remove sources and loads to an HDL testbench on an ad-hoc basis.

SPICE elements are added to the root of the analog or mixed-signal net that you specify. You can use a hierarchical path name (relative or absolute) to reference the net. Wildcards or range specifications (slices) are not allowed. However, you can use a TCL loop on the result of a find command to add SPICE elements globally.

The <val> argument to the spice add command is used to specify the value of the added resistor (in Ohms) or capacitor (in Farads) using the SPICE syntax of the passive primitives R or C. This value expression must be constant (static); value changes, if any, are not taken into account. Refer to [Resistor](#) and [Capacitor](#) in the *Eldo Reference Manual*, for the supported syntaxes.

Only the value (and expressions for the value) are supported, AMS model parameters cannot be used. However, you can use a SPICE parameter in the expression for the value (with its full hierarchical path name) provided that the following option is specified:

```
.OPTION KEEP_PARAMS
```

Note

Expressions used in the spice add command cannot contain HDL parameters.

If several resistors or capacitors are added on a single net, they are combined. The resulting value used for simulation is the value of the equivalent element of the parallel addition of the elements.

Example

1. :top:X1:N1 is a port connected to the analog net :top:N1. Here, two resistors are added:

```
spice add resistor :top:X1:N1 10k
spice add resistor :top:N1 5k
```

2. A list is requested:

```
spice list resistor :top:N1
```

This returns:

```
{resistor :top:X1:N1:10k} {resistor :top:N1 5k}
```

3. Now, the first resistor is deleted:

```
spice delete resistor :top:X1:N1
```

4. All elements are listed:

```
spice list all :top:X1:N1
```

This returns:

```
{resistor :top:X1 5k}
```

The resistor or capacitor equivalent to the parallel addition is shown in the Current Contribution pane of the **Contributor Window**.

Related Topics

[spice add \[Questa ADMS Command Reference\]](#)

[spice delete \[Questa ADMS Command Reference\]](#)

Non-Convergence Errors

If a non-convergence error is reported at DC, transient simulation cannot begin. No command that advances simulation time or continues the search for the quiescent point is allowed.

If the transient simulation fails to converge, Questa ADMS reports error vasm-3810 in the transcript. For example:

```
Error: (vasim-3810) Analog solution failed to converge at simulation time
77,080 fs.
+ Simulation has stopped at time 77,063 (the last successful analog
solution).
```

Questa ADMS restores the state of the system just prior to the error. You can browse the design structure in the Questa ADMS GUI then apply a force, execute a change command, or insert a resistor or capacitor to obtain convergence. For example:

```
spice add resistor b 1
run -all
```

Questa ADMS continues the transient simulation.

PIVTOL Option

A model may have problems converging in cases where:

- The model handles values and coefficients that are not usual for electrical systems
- There is an important dispersion between the different quantities involved in the equations, the model may have problems converging. It may also provide different results when the order of the equations is changed.

You may have to change the value of the PIVTOL option to solve this:

```
.OPTION PIVTOL = <value>
```

The value set corresponds to the absolute minimum value that can be accepted in the matrix to be a pivot for the LU-Factorization algorithm. Decreasing the value to below the default value 1.0×10^{-16} may help the analog solver to converge to the correct result.

Refer to **PIVTOL** in *Eldo Reference Manual* for more information.

Related Topics

[Simulation Statistics](#)

[Interactive Forcing and Loading](#)

[spice add \[Questa ADMS Command Reference\]](#)

[Improving DC Convergence \(Eldo\) \[Eldo User's Manual\]](#)

Recompiling Design Units in the Source Window

You can recompile design units in the source window.

Use the following procedure to recompile VHDL-AMS and Verilog-AMS design units from within the [Source Window](#).

Procedure

1. Double-click on an instance in the [Structure Window](#) to make the source file visible in the Source window:
2. Choose **Compile > Compile** or click the **compile** button  on the toolbar. If the design unit has been previously compiled in the current library, Questa ADMS will automatically compile it using the same compilation command.
If the design unit has not been compiled into the current library—the Compile Source Files dialog box is displayed to request the appropriate compilation command and required library.
3. The Library field defaults to the current library. A different library can be selected from the Library dropdown list, or you can select **Browse** to choose a library that is not defined in the current *modelsim.ini* file.
4. Specify a compilation command in the Compilation command line field. The command must be written as it would be in the UNIX/Linux shell. Only the valog and vacom compilation commands can be specified.
5. Select the Restart Current Simulation checkbox if you want to restart the simulation after the design unit has been compiled and the whole design has been re-elaborated successfully. The option is enabled by default after a simulation has been run in the current session.

6. Click **Compile**.

Results

If the Do not update library option is enabled, the unit is compiled without updating the library. This option should only be used for checking that the source file can be compiled successfully, before editing and recompiling other files for this design, or until a new simulation is run. Units recompiled using this option are instantiated or referenced by other units, those units will become obsolete and the simulation will then fail. In these cases, you should not use this option if you want to re-run the simulation following compilation.

Related Topics

Source Window

[view \[Questa ADMS Command Reference\]](#)

Diagnosis Mode

Diagnosis mode reports structures or usages that may cause problems during simulation and are likely to be design errors. Diagnosis messages are generated during elaboration and simulation. They can be viewed in the transcript or in the Message Viewer, depending on the setting of the msgmode option.

Tip

Refer to [Diagnosing Convergence and Performance Issues](#) in the *Eldo User's Manual* for more information.

There are two kinds of diagnosis checks:

- Lint checks, which check for suspicious usage.
- Performance bottleneck checks which may help determine why a circuit does not behave as expected or takes more time than expected to simulate.

Lint Checks

Activate lint checks using any of the following methods:

- The **vasim** command line argument:

```
vasim -diagmode norun|run
```

- The SPICE command option:

```
.OPTION DIAGMODE = NORUN|RUN
```

- The simulator variable in the *modelsim.ini* file:

```
DIAGMODE = NORUN|RUN
```

These are listed in order of priority, highest to lowest.

The options, which are not case sensitive, are:

- norun

Diagnosis will be enabled during elaboration and DC analysis. No simulation run after DC analysis will be available.

- run

Diagnosis will be enabled during elaboration, DC and transient analysis. Simulation runs for other analyzes will not generate any relevant diagnosis data.

The types of lint checks carried out are:

- Connectivity:
 - Is any dipole pin not connected?
 - Is the drain or source of any MOS transistor not connected?
 - Is the collector or emitter of any bipolar transistor not connected?
- Values of SPICE components or devices:
 - Is any resistor value too small or negative?
 - Is any capacitor value too large or negative?
 - Are any MOS or BJT parameters beyond their limits?
 - Do any source parameters give sharp slews?
- Topological:
 - Are there any missing DC paths to ground ?
 - Are any gates floating ?
 - Are there any voltage loops ?
- Miscellaneous:
 - Are there a high number of waveforms to plot for a request made through probe, plot, add wave and add log ?
 - Are there any intrusive commands such as force or signal force ?

Lint Check Diagnosis Messages

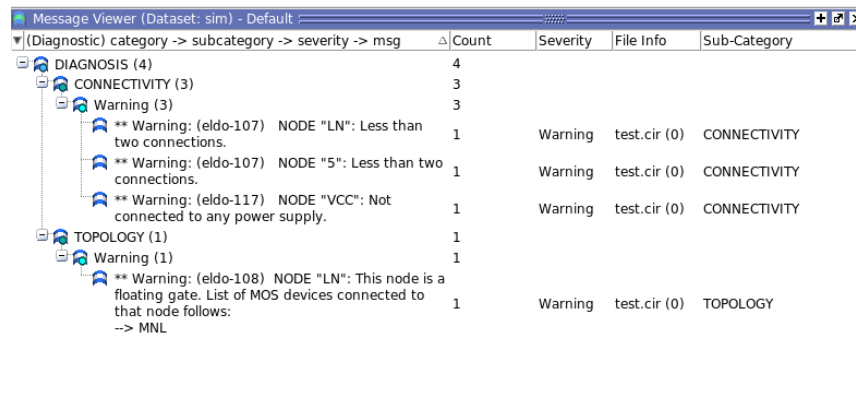
Diagnosis messages appear in the transcript and the Message Viewer window if reporting is enabled, see [Viewing Messages in the Message Viewer](#).

Message information reported includes the message ID, severity, the time and simulation stage (elaboration, DC, transient analysis) at which the message was reported, context information, such as the hierarchical name of the region where the object exists, the message simple name and the message type (net or device).

Messages can be sorted and filtered as required. Double-click on a message to view the source code that caused the message to be reported.

The **verror** command can be used to get a full description of a given error message. See [Verbose Error Message Reporting](#).

Figure 8-4: Diagnostics Messages Reported in the Message Viewer



(Diagnostic) category -> subcategory -> severity -> msg	Count	Severity	File Info	Sub-Category
DIAGNOSIS (4)	4			
CONNECTIVITY (3)	3			
Warning (3)	3			
** Warning: (eldo-107) NODE "LN": Less than two connections.	1	Warning	test.cir (0)	CONNECTIVITY
** Warning: (eldo-107) NODE "S": Less than two connections.	1	Warning	test.cir (0)	CONNECTIVITY
** Warning: (eldo-117) NODE "VCC": Not connected to any power supply.	1	Warning	test.cir (0)	CONNECTIVITY
TOPOLOGY (1)	1			
Warning (1)	1			
** Warning: (eldo-108) NODE "LN": This node is a floating gate. List of MOS devices connected to that node follows: --> MNL	1	Warning	test.cir (0)	TOPOLOGY

Refer to GUI Elements of the Message Viewer Window in *Questa SIM GUI Reference Manual* for more information.

Performance Bottleneck Checks

You can use the **vasim** -diagmode argument to help determine why a circuit does not behave as expected or takes more time than expected to simulate:

```
vasim -diagmode perf|conv|tstep
```

The options are:

- perf

Enables performance analysis. Activates the .FORMAT_STEP_LIMITERS command and returns the percentage of CPU time spent evaluating each type of device. This distribution is reported per Verilog-A model, and per type of MOS, BJT, Diode, and JFET model.

- conv

Enables the analysis of non-convergence issues. When Eldo stops because of a non-convergence a file is generated named *<netlist_name>.noconv*, used as an input for the convergence analysis. Not supported by Questa ADMS Premier.

- tstep

Enables the analysis of time step management. Use this if Eldo performs too many average Newton iterations or if it rejects too many time steps: if the ratio of accepted time steps compared to rejected time steps is lower than six then specify this argument to try to understand what is limiting the simulator. Not supported by Questa ADMS Premier.

You can use multiple -diagmode arguments to specify a combination of diagnosis modes. For example:

```
vasim -cmd top.cir -diagmode run -diagmode perf
```

Each -diagmode argument must specify only one option.

Related Topics

[Viewing Messages in the Message Viewer](#)
[vasim \[Questa ADMS Command Reference\]](#)
[Verbose Error Message Reporting](#)

Verbose Error Message Reporting

You can use the verror command to return a full description of a given error message.

Use the **verror** command to return a full description of a given error message.

```
verror [-fmt|-full] [<tool>-<msgNum>|<msgNum>] ...  
verror [-fmt|-full] [-kind <tool>] -all
```

For example, when an error message is written to the transcript, such as:

```
** Error (eldo-20108) This node is a floating gate. foo
```

To report more information about this message, enter:

```
verror eldo-20108
```

The following output will be reported:

Message # 20108:

The indicated node is connected to at least one MOS gate, but it is not connected to anything else but MOS gates, capacitors, or a reverse-biased diode, or it is not connected at all. You should probably change the netlist to include a gate bias. Alternatively, the `.OPTION FLOATGATE0` will automatically connect this and all similar floating gates to ground.

Convergence Messages

The simulator variable `VerboseAnalog` controls the display of the convergence progress information written to the Transcript Window. This variable is defined under the `[vasim]` heading in the `modelsim.ini` file.

```
VerboseAnalog = 1
```

When enabled, by being set to 1, all convergence information coming from the analog solvers are output. This is the same behavior as Eldo standalone. When disabled, by being set to 0, only a subset of convergence messages are output. By default, it is disabled.

Related Topics

[verror \[Questa ADMS Command Reference\]](#)

[Setting Break Severity](#)

[Limiting Message Display](#)

[Viewing Messages in the Message Viewer](#)

Limiting Message Display

Use the `maxmsg` command to control the number of times each message (with the same message ID) is displayed during simulation.

```
maxmsg [<value>]
```

If specified without any argument this command returns the current value. If set to zero, the limit is removed.

You can also use the following `vasim` argument to set this limit:

```
vasim -maxmsg <value>
```

Or by using the SPICE command:

```
.OPTION MAXMSG=<value>
```

Alternatively, set a default limit in the *modelsim.ini* file, using the variable `MaxNbOfDisplayedMsg`.

If more than one method is used to set this value, the following order of precedence applies:

1. The Tcl command `maxmsg`
2. The `vasim` argument `-maxmsg`
3. The SPICE command `.OPTION MAXMSG`
4. The *modelsim.ini* variable `MaxNbOfDisplayedMsg`.

Note

Questa SIM messages are not affected by this setting (those prefixed with `vsim`).

Related Topics

[maxmsg \[Questa ADMS Command Reference\]](#)

[Verbose Error Message Reporting](#)

[Viewing Messages in the Message Viewer](#)

Viewing Messages in the Message Viewer

You can view messages in the message viewer.

The Message Viewer is used to access, organize, and analyze elaboration and runtime messages written to the transcript during elaboration and simulation. These messages can result from calls to Verilog/Verilog-AMS display system tasks, VHDL/VHDL-AMS assert/report statements and PLI/FLI print function calls.

Note

Only messages with the following tool prefixes are displayed in the Message Viewer window: `eldo`, `eldo-premier`, `eldo-adit`, `vasim`, `vsim`.

Prerequisites

Messages are not reported to the Message Viewer by default. Use the following **vasim** arguments to control whether messages are displayed in the Transcript window, the Message Viewer or both. This overrides the default setting, set in the *modelsim.ini* file:

- `-displaymsgmode {tran | wlf | both}`

Controls where calls to display system tasks are reported.

- -msgmode {tran | wlf | both}

Controls where elaboration and runtime messages are reported.

- tran — (default) Outputs messages only to the transcript, therefore they are unavailable in the Message Viewer.
- wlf — Outputs messages only to the Message Viewer, and saves them to a *wlf* file. Messages are not available in the transcript.
- both — Outputs messages to both the transcript and to the Message Viewer, and saves them to a *wlf* file.

Note

You can also set the default behavior in the *modelsim.ini* file, using the variables `displaymsgmode` and `msgmode`:

```
displaymsgmode = {tran | wlf | both}
msgmode = {tran | wlf | both}
```

Procedure

1. To access the Message Viewer window, choose **View > Message Viewer** or use the following Tcl command:

```
view msgviewer
```

2. To display a detailed description of a message, right-click the message text and choose View Verbose Message.
3. To open the source file that caused the message, double-click the message text.
4. In the Region column, double-click the hierarchical reference to change the focus of the Structure and Objects windows.
5. In the File Info column, double-click the file name to open the source file and set a marker at the line number.

Results

For a full description of this window, refer to GUI Elements of the Message Viewer Window in the *Questa SIM GUI Reference Manual*.

Related Topics

Verbose Error Message Reporting

Diagnosis Mode

Simulation Statistics

To help you to understand how your design behaves, data about the current simulation can be collected, and output to a statistics file.

This statistics can be used to monitor simulation performance, determine the contribution of elements to circuit size, debug simulation slowdown, and determine which nodes and blocks should be treated for minimizing rejections.

In interactive mode, some statistical information can be viewed in the [Stat Summary Window](#), the [Stat Mixed-Signal Activity Window](#) and the [Stat Analog Kernel Window](#). Their contents are updated each time the simulation pauses.

Statistical data either in the stat file or graphically is not generated by default; you must use one of the following methods to request it:

- Launch Questa ADMS using `vasim -stat`
- Use the stat enable command during or at the end of a simulation. stat enable will start logging statistics from that point in the simulation onwards. Simulation data prior to the execution of the stat enable command will not be recorded.

Use the stat dump command at any point after a stat enable to write the data to an output statistics file. Multiple stat dump commands will append data for each time frame to the statistics file.

The stat windows are continuously updated. The stat dump command is required in order to write the data to the statistics file.

Note

Generating statistics has a negative impact on simulation performance; be aware that simulation time will be increased by approximately 20%.

See also [Statistics File Reference](#).

[Viewing Statistics in Interactive Mode.....](#) 8-23

[Statistics File.....](#) 8-27

[Example: Analyzing Simulation Issues Using the Statistics File.....](#) 8-28

Viewing Statistics in Interactive Mode

When statistics collection is enabled, some statistical data can be viewed in graphical form.

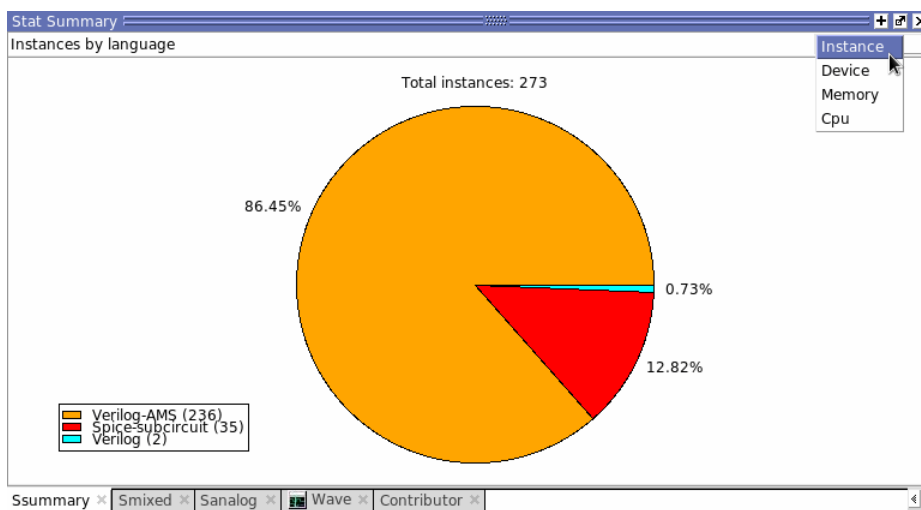
When statistics collection is enabled, some of it can be viewed in graphical form, using the [Stat Summary Window](#), the [Stat Mixed-Signal Activity Window](#) and the [Stat Analog Kernel Window](#).

The following procedure describes how to view statistical data using these windows.

Procedure

1. Start a Questa ADMS simulation with `vasim -stat` to enable the collection of statistical data. Run the simulation to completion.
2. Choose **View > Stat > Stat Summary**, to open the [Stat Summary Window](#). This window provides a summary of elaboration and simulation information in a pie chart format.

Figure 8-5: Stat Summary Window



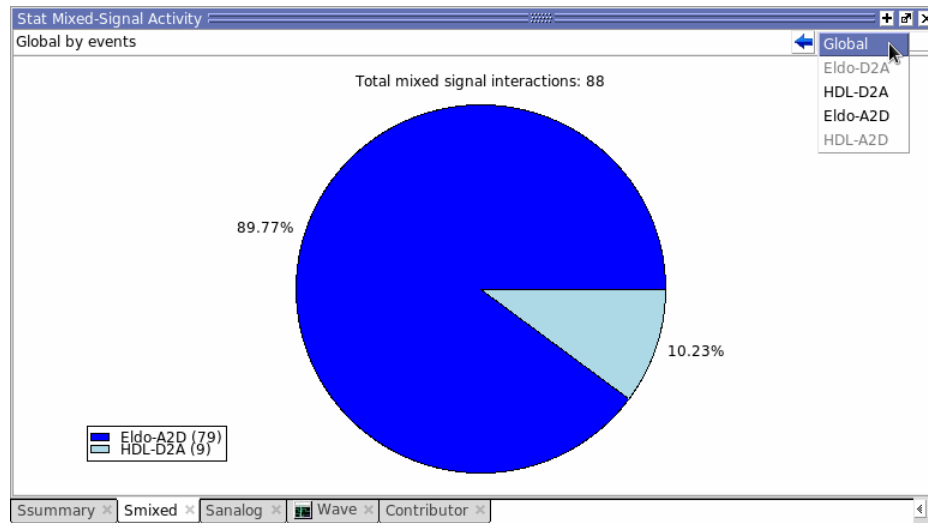
3. Click the label in the top right corner to switch between the following graphs, using the dropdown list:
 - **Instance** — A summary of instances by language
 - **Device** — A summary of SPICE devices by kind
 - **Memory** — A summary of memory allocation
 - **Cpu** — Cumulative CPU time by phase: elaboration, DC, AC, TRAN

The absolute values are shown in the key, and the total number of instances and devices is shown in the graph legend.

4. Choose **View > Stat > Stat Mixed-Signal Activity** to view the [Stat Mixed-Signal Activity Window](#). This window displays statistical information about the interactions between analog and digital. A global chart shows the distribution of the causes of mixed signal interactions:
 - D2A events (Eldo Built-in boundary elements)
 - D2A events (any breaks of HDL models)
 - A2D events (Eldo Built-in boundary elements)

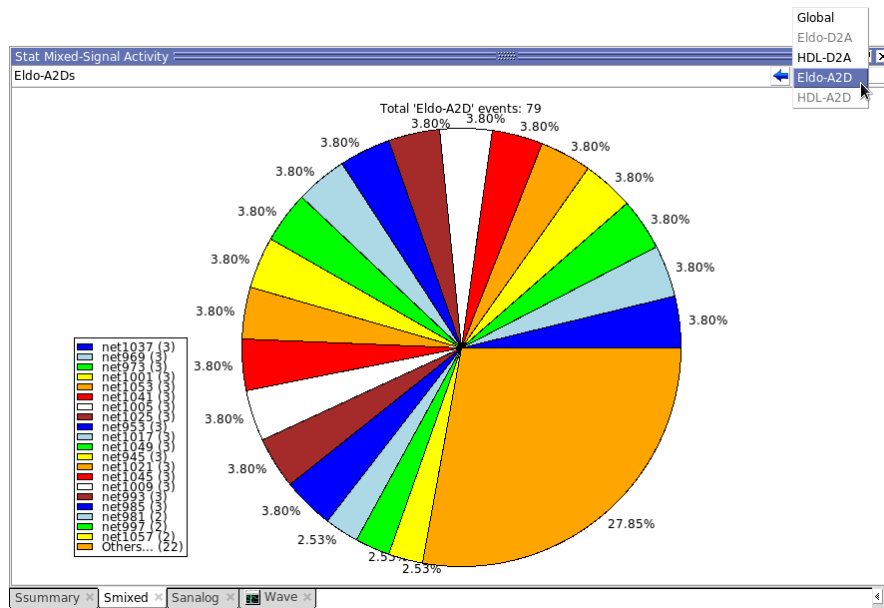
- A2D events (VHDL-AMS-Q'above and Verilog-Ams @Cross statements).

Figure 8-6: Stat Mixed-Signal Activity Window



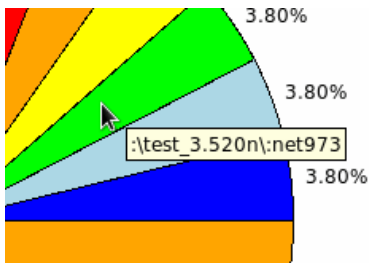
- Click on a segment in the global chart to display a node-related chart, showing the distribution of the 10 most active nodes in that category.

Figure 8-7: Stat Mixed-Signal Activity Window - Eldo A2D Events



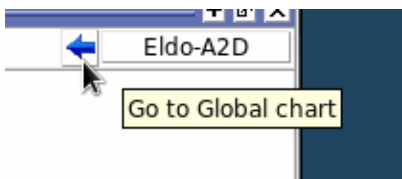
- Hover over a segment of the graph to view the full pathname to the object that caused the event.

Figure 8-8: Object Pathname



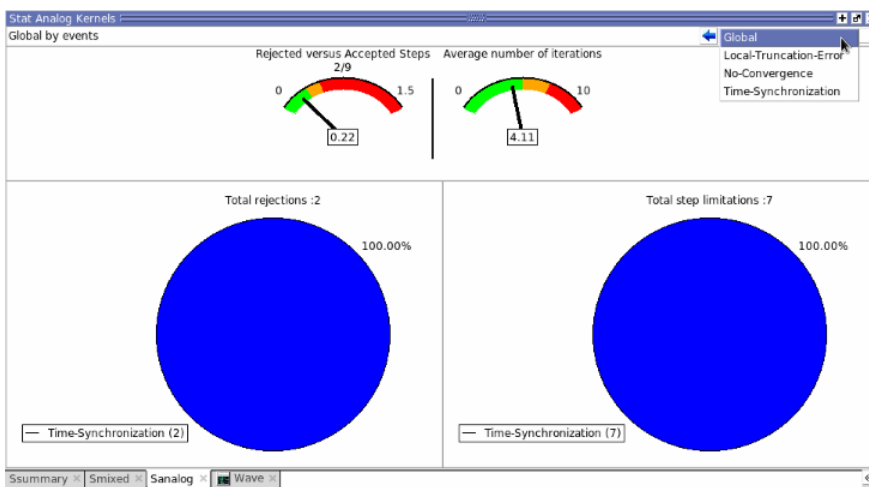
7. Click on the segment to select that node in the Objects window. The current region changes to that of the selected node.
8. Right-click on a segment and select **View Declaration** to view the source in the Source window.
To return to the global chart, select the blue arrow next to the graph selection button.

Figure 8-9: Go to Global Chart Button



9. Choose **View > Stat > Analog Kernels**, or use the command view sanalog. This chart provides information about analog kernel activity.

Figure 8-10: Stat Analog Kernels Window



The ratio between rejected and accepted steps, and the average number of Newton iterations per step. A high level of Newton iterations is an indicator that the requested precision is too high.

For designs with more than 1000 nodes, a counter showing the matrix density level is visible. A low level of matrix sparsity could be caused by too many parasitics (capacity coupling) or missing power supplies.

The two pie charts show the distribution of local truncation error, no convergence and time synchronization in all rejections and all step limitations.

10. Enter the stat reset command into the Transcript window to clear the simulation statistics recorded in the following sections of the statistics file:

- Simulation: Mixed-Signal activity
- Simulation: Digital kernel
- Summarized Simulation Information
- Mixed-Signal interaction

The related statistics data shown in the [Stat Summary Window](#) and the [Stat Mixed-Signal Activity Window](#) will be reset.

Related Topics

[Statistics File](#)

[Statistics File Reference](#)

[Simulation Statistics](#)

Statistics File

All generated statistical data is recorded in a statistics file.

When you have loaded a design into Questa ADMS and generated statistics, you can view the statistics file in the Questa ADMS GUI by selecting **View > Stat > Stat File** from the menu.

By default, the statistics file is saved to the current location (the directory from which vasim was run). You can specify a different location for the generated statistics file, using the vasim -outpath option.

The naming convention for this file is:

`<CmdFileName>_<date>_<time>.stat`

Where <CmdFileName> is the name of the simulation command file without the extension, <date> is of the form <year><month><day>, and <time> is of the form <hours><minutes><seconds>.

For example, if the following command is run on May 24 2012 at 9.23am:

```
vasim -stat -cmd top_spice.cir
```

The statistics file generated will be named: `top_spice_20120524_092325.stat`.

Alternatively, you can specify a unique filename for the statistics file generated, using the vasim -statfile option.

By default, a statistics file is limited to a maximum size of 50 MB, but you can adjust this limit by setting the `StatisticsFileMaximumSize` variable (see [Simulator Control Variables](#)) to the required limit. When the maximum size is reached, subsequent statistics files are created. The names of these files are of the format:

`<CmdFileName>_<date>_<time>.<n>`

Where `<n>` = 1, 2, 3, and so on, or `<file_name>.<n>` if you have used the `-statfile` option in order to specify a filename for the statistics file.

Example: Analyzing Simulation Issues Using the Statistics File

You can debug a design using the statistics file.

Prerequisites

Specify statistics file generation prior to running the simulation using `vasim -stat`.

Note

Generating a statistics file has a negative impact on simulation performance; be aware that simulation time will be increased by approximately 20%.

Procedure

1. Begin by reading the Requested simulation time: data.

- Analog only CPU time (crossing of Analog only and CPU time)/Total CPU time. If the simulation is ADMS with ADiT, this section is split into two "Eldo only" and "ADiT only".

If the ratio is important (usually, when there is a slowdown, the ratio becomes greater than 50%) then go to [2](#).

- Digital only CPU time (crossing of Analog only and CPU time)/Total CPU time. If the ratio is important (usually, when there is a slowdown, the ratio becomes greater than 50%) further analysis cannot be made from the statistics file because there is no profiler available within Questa ADMS to determine which digital signal, process or transaction is slowing down the simulation. However, you could use `watch -r *` so that every event is reported in the transcript, then filter that information to show the most active signal or the hierarchy of activity.

2. Step A1

Read the Simulation: Analog kernel data.

- The number of terms in the matrix gives an estimate of the complexity of the design and is directly linked to the simulation performances in terms of CPU time.
- The average number of iterations indicates the difficulty in calculating analog step points.
 - For Questa ADMS/Eldo, a number greater than 3 indicates difficulties to converge.

- For Questa ADMS/ADiT, this number relates only to the partition inside Eldo.
- Analyze the ratio of due to LTE, due to Newton, due to D2A, any, due to A2D, HDL analog events, due to A2D, built-in converters, due to Verilog-AMS final step stmt with respect to the total number of Rejected time steps.
 - If the due to LTE or due to Newton ratio is high, go to [4](#).
 - If the due to D2A, any, due to A2D, HDL analog events or due to A2D, built-in converters ratio is high, go to [3](#).

3. Step AD

Read the Mixed-signal interaction table, under Summarized Simulation Info.

Look to see which event is causing most rejection (Effective Rej. is worse than "simple" rejection):

- If the rejecting event is D2A (built-in), go to [3.a](#).
- If the rejecting event is A2D (built-in), go to [3.b](#).
- If the rejecting event is A2D(Q'above), A2D(@Cross), or D2A(Break), go to [3.c](#).

a. Step AD1

Read the Top 10 D2A boundary elements table, under [Simulation Parameters](#).

This is the list of interface signals sorted in descending number of events. The top ones are the most active D2A signals that you should monitor to see why they are causing simulation slowdown. Changing the design and creating an adequate model can avoid those synchronizations.

b. Step AD2

Read the Top 10 A2D boundary elements table, under [Simulation Parameters](#).

This is the list of interface signals sorted in descending number of events. The top ones are the most active A2D signals that you should monitor to see why they are causing simulation slowdown. Changing the design and creating an adequate model can avoid those synchronizations.

c. Step AD3

Read the Simulation: Reject, synchro, ... data

Look at the Rejection due to time-synchronization section. This may give you a clue as to which model instance is causing simulation slowdown, but it will not be obvious exactly which signal is causing the slowdown.

4. Step AA

Read the Simulation: Reject, synchro, ... data"

Look at the Rejection due to no-convergence, Rejection due to Local-Truncation-Error (LTE) and Step limitation due to Local-Truncation-Error sections. The node where most activity occurred should be

analyzed in detail by plotting the voltage and all the current contributions attached to it, using the following syntax:

```
.PLOT <analysis> in(node)
```

Related Topics

[Statistics File Reference](#)

[vasim \[Questa ADMS Command Reference\]](#)

Integration with Questa SIM Verification Methodology

Questa ADMS enables you to save coverage data to the Unified Coverage Database (UCDB) for VHDL-AMS assertions, SPICE Safe Operating Area (SOA), .EXTRACT commands, and Condition code coverage.

In addition to the existing Questa SIM coverage, Questa ADMS enables you to save coverage data to the Unified Coverage Database (UCDB) for the following items:

- VHDL-AMS assertions
- SPICE Safe Operating Area (SOA) and .EXTRACT commands
- Condition code coverage for VHDL-AMS if-use and case-use statements

Confirmation that one clause of an if-use or case-use conditional set of equations has been reached is reported in the UCDB as branch coverage.

Assertion and if-case, use-case definitions are written into the VHDL-AMS source code description. SPICE Safe Operating Area assertions are written in the SPICE netlist using .SETSOA statements; data will be captured in the UCDB database when there is a .CHECKSOA command in the design.

In order to capture coverage data during simulation:

1. Specify the type of code to be captured when compiling, using the command:

```
vacom +cover
```

2. Run the simulation in interactive mode, using the command:

```
vasim -coverage
```

Coverage and assertion data will be visible in the Code Coverage Analysis and Assertions windows.

3. Generate some reports using the command:

```
coverage report
```

4. Save the data using the command:

```
coverage save <file>.ucdb
```

For each simulation run, assertion and coverage data is saved to the UCDB. A single UCDB may contain several runs, each contributing to the coverage of different parts of a design.

You can open a previously saved `.ucdb` file in Questa ADMS directly using the **File > Open** menu item.

Coverage Windows.....	8-31
Code Coverage in Interactive Mode.....	8-32
Creating Coverage Reports.....	8-36
Predefined Test Attribute Data.....	8-37
Questa SIM Verification Management Windows.....	8-39
Capturing VHDL-AMS Assertion Data in Interactive Mode.....	8-40
Capturing VHDL-AMS Assertion Data in Batch Mode.....	8-46
Naming Conventions for Assertions.....	8-49
Capturing SPICE Extractions in a UCDB.....	8-50
Capturing Multiple-Run Simulation Data in a UCDB.....	8-56
Plotting VHDL-AMS Assertions.....	8-58
Plotting SPICE SOA Assertions.....	8-59
SystemVerilog Assertions.....	8-60
SystemVerilog bind Construct.....	8-66
Access to Analog Objects from SystemVerilog.....	8-68
get_potential.....	8-70
get_flow.....	8-72

Coverage Windows

Coverage data is displayed in several windows. The coverage data includes information about code coverage, instance coverage, and assertions.

Table 8-1: Coverage Windows

Window	Description
Code Coverage Analysis	Displays covered (executed), uncovered (missed), and/or excluded statements, branches, conditions, expressions, FSM (Finite State Machine) states and transitions, as well as signals

Table 8-1: Coverage Windows (continued)

Window	Description
	that have and have not toggled. See Code Coverage Analysis Window in the <i>Questa SIM GUI Reference Manual</i> .
Instance Coverage	Used to analyze coverage statistics for each instance in a flat, non-hierarchical view. The same code coverage statistics information is also available in the Structure Window . See Instance Coverage Window in the <i>Questa SIM GUI Reference Manual</i> .
Details	Used to view detailed results about coverage metrics from a simulation. See Coverage Details Window in the <i>Questa SIM GUI Reference Manual</i> .
Assertions	Lists all embedded and external assert directives that were successfully compiled and simulated during the current session. When assertions fire with failure messages, the Assertions window displays the name and failure count in red, both during simulation and in post-simulation mode.
Cover Directives	Displays accumulated cover directive statistics at the current simulation time, including percentages and a graph for each directive and instance. Refer to Viewing Cover Directives in the Cover Directives Window in the <i>Questa SIM User's Manual</i> .
Covergroups	Displays SystemVerilog covergroups, coverpoints, crosses and bins in the currently selected region (selected via the Structure Window). See Covergroups Window and Viewing Functional Coverage Statistics in the GUI, in the <i>Questa SIM User's Manual</i> .

See also Coverage and Verification Management in the UCDB, Code Coverage, and Condition and Expression Coverage in the *Questa SIM User's Manual*.

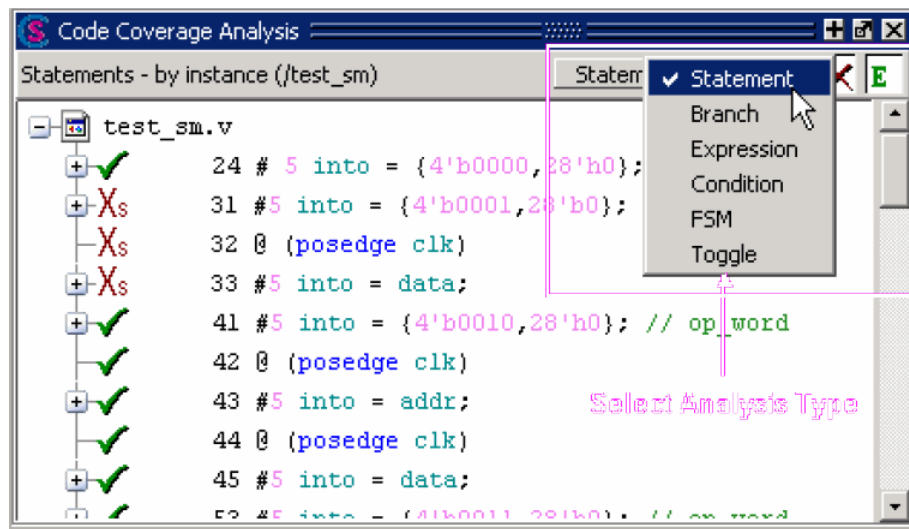
Code Coverage in Interactive Mode

Code coverage produces feedback on which executable statements, branches, conditions, and expressions in your source code have been run.

When you simulate a design with code coverage enabled, coverage data is displayed primarily in the Code Coverage Analysis, Instance Coverage, and Coverage Details windows. You can open and close coverage windows with the **View > Coverage** menu selection.

In the Coverage Analysis window you can select the analysis type to display by clicking the Analysis Type selector button.

Figure 8-11: Selecting Code Coverage Analysis Type

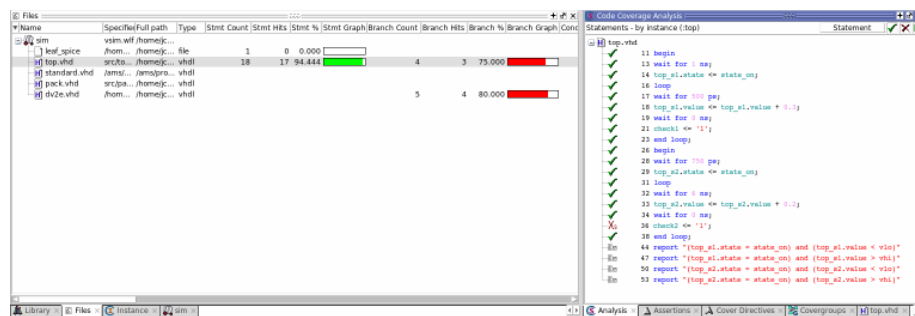
**Note**

For VHDL-AMS, only Branch analysis is supported. For SPICE, only Statement analysis is supported. For pure digital regions, all analysis types are supported

When you load a design with Code Coverage enabled, additional columns are visible in the Files and Structure windows to display coverage data.

When you select different files from the Files window, the Code Coverage Analysis window is updated to show coverage data for the selected file.

Figure 8-12: Files and Coverage Analysis Windows



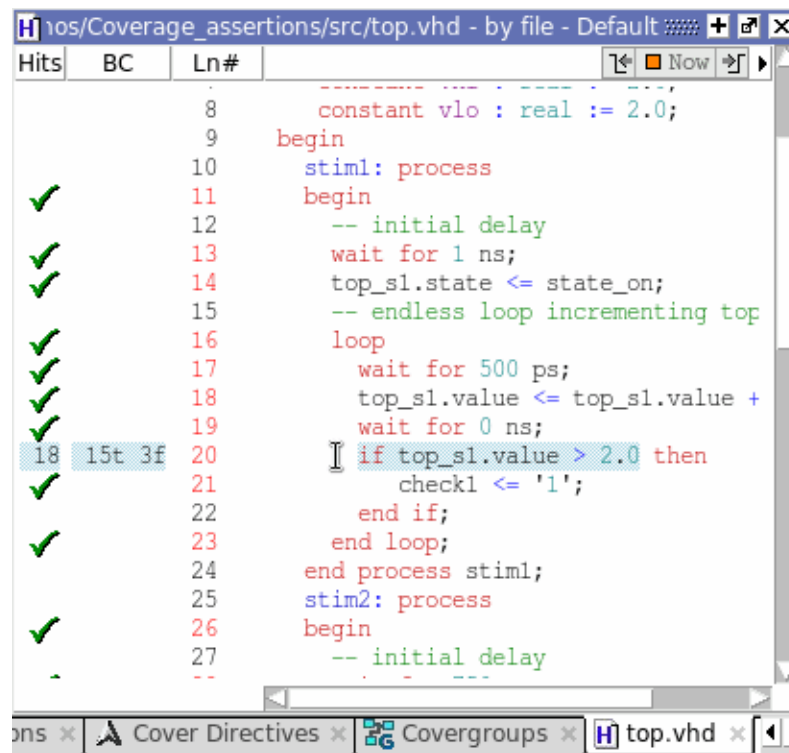
Each line in the Code Coverage Analysis window shows an icon to indicate whether elements in the line (statements, branches, conditions, or expressions) were executed, not executed, or excluded. [Table 8-2](#) describes the Code Coverage icons.

Table 8-2: Code Coverage Icons

Icon	Description/Indication
	All statements, branches, conditions, or expressions on a particular line have been executed
	Multiple kinds of coverage on the line were not executed
	True branch not executed (BC column)
	False branch not executed (BC column)
	Condition not executed (Hits column)
	Expression not executed (Hits column)
	Branch not executed (Hits column)
	Statement not executed (Hits column)
	Indicates a line of code to which active coverage exclusions have been applied. Every item on the line is excluded; none are hit.
	Some excluded items are hit
	Some items are excluded, and all items not excluded are hit
	Some items are excluded, and some items not excluded have missing coverage
	Auto exclusions have been applied to this line. Hover the cursor over the E _A and a tool tip balloon appears with the reason for exclusion,

Double-click any entry in the Code Coverage Analysis window to display that line in the Source window.

Figure 8-13: Coverage Statistics in the Source Window



The Hits and Branch Coverage are displayed in the Hits and BC columns. Hover your mouse over a line in the source code. This replaces the coverage icons in these columns with the coverage figures for that line of code. [Table 8-3](#) describes the various coverage icons.

Table 8-3: Coverage Icons in the Source Window

Icon	Description
green checkmark	Indicates a statement that has been executed
green E	Indicates a line that has been excluded from code coverage statistics
red X	An X in the Hits column indicates a missed (unexecuted) statement (X_S), branch (X_B), or condition (X_C). An X in the BC column indicates a missed true (X_T) or false (X_F) branch.

To display numbers for all lines instead of these icons, select **Tools > Code Coverage > Show Coverage Numbers**. Red numbers indicate missed coverage in that line of code. An ellipsis (...) is displayed whenever there are multiple statements on the line.

The Instance Coverage window displays coverage statistics for each instance in a flat, non-hierarchical view. Double-click any instance in the Instance Coverage window to see its source code displayed in the Source window.

Creating Coverage Reports

You can create textual reports on coverage statistics.

Procedure

To create textual reports on coverage statistics, do one of the following:

- Choose **Tools > Coverage Report > Text** from the Main window menu bar.
- Right-click any object in the Structure or Files windows and choose **Code Coverage > Code Coverage Reports**.
- Right-click any object in the Instance Coverage window and select **Code coverage reports**. You may also choose **Instance Coverage > Code Coverage Reports** from the Main window menu bar when the Instance Coverage window is active.

Results

The Coverage Text Report dialog box opens. You can select which files, instances or design units to report on, and control the collection of data for the report. By default, the report file is saved in the current directory, and named *report.txt*. The report is opened for viewing in the Notepad text viewer.

You can also use the coverage report command to produce textual report files.

See also Code Coverage in the *Questa SIM User's Manual*.

Examples

Figure 8-14: Coverage Report File - Textual

```

Coverage Report Summary Data by file

Total Coverage By File (code coverage only, filtered view): 58.5%

File: src/dv2e.vhd
  Enabled Coverage      Active      Hits      Misses % Covered
  -----
  Branches              5         4         1      80.0

File: src/leaf_spice
  Enabled Coverage      Active      Hits      Misses % Covered
  -----
  Stmts                 1         0         1       0.0

File: src/top.vhd
  Enabled Coverage      Active      Hits      Misses % Covered
  -----
  Stmts                 18        17         1      94.4
  Branches              4         3         1      75.0
  Toggle Bins           12         1        11       8.3

TOTAL ASSERTION COVERAGE: 64.2%  ASSERTIONS: 14
  
```

Predefined Test Attribute Data

In the UCDB, a set of test-related items describe the simulation context for each run: the command line arguments performed, directory paths to output files, the simulator options used, and the environment characteristics such as username or date.

Questa ADMS provides an additional set of predefined fields in the UCDB test attribute record, for data relevant to an analog or mixed-signal simulation. These additional fields are described in [Table 8-4](#).

Table 8-4: Mixed-Signal Simulation Predefined Fields in UCDB Test Attribute Record

Field/Attribute Name	Description
SIMTIME	The last simulation time. Time is 0 for a frequency-domain simulation.
TIMEUNIT	The digital time resolution.
CPUTIME	The CPU time. This value can also be returned using the simstat command.
TESTSTATUS	The status of the assertion test, determined by the UCDB merge status (merge fail) and the highest severity messages. For VHDL-AMS assertions: • 0 - OK • 1 - Warning • 2 - Error • 3 - Fatal (Failure) For SOA assertions: • 0 - OK • 1 - Error
TESTNAME	A unique name for the test record of each run of a multiple-run simulation. Default is run_<n>, where <n> is an arbitrary sequence number, to guarantee uniqueness. Optionally, add a prefix to this, using the following command: <pre>coverage save -testname <testname></pre> In this case, TESTNAME is: <pre><testname>_run_<n></pre>
LOGNAME	The full directory path to the transcript or log file.
OUTPATH	The full directory path to the location of the simulation output files.

Table 8-4: Mixed-Signal Simulation Predefined Fields in UCDB Test Attribute Record (continued)

Field/Attribute Name	Description
WDBNAME	The full directory path to the waveform database (.wdb) file
CHIFILENAME	The full directory path to the Eldo ASCII output (.chi) file, if generated.
STATFILENAME	The full directory path to the statistics file, if generated.
SIMTYPE	The simulation type that has generated the data. Lists all analyses performed during the simulation run.
VASIMARGS	The simulator command line arguments used to create the simulation run.
VASIM_RUN_PARAMS	The parameters and values used in the simulation run.
AEXFILENAME	The full directory path to the AEX results file
SOAFILENAME	The full directory path to the SOA results file
HIZFILENAME	The full directory path to the high impedance results file

Note: It is not possible to use the coverage attribute command to make modifications in the UCDB to the test-related items listed in this table.

The UCDB test attribute record is described in detail in *Understanding Stored Test Data in the UCDB* in the *Questa SIM User's Manual*.

See also *Coverage and Verification Management in the UCDB* in the *Questa SIM User's Manual*.

Related Topics

[Capturing VHDL-AMS Assertion Data in Interactive Mode](#)

[Capturing VHDL-AMS Assertion Data in Batch Mode](#)

[Capturing Multiple-Run Simulation Data in a UCDB](#)

Questa SIM Verification Management Windows

Questa Verification Management windows display information about UCDB tests for the purpose of managing the verification process.

The following Questa standard verification management windows are accessible from Questa ADMS:

- Verification Management Browser Window
- Verification Tracker Window

- Verification Results Analysis Window

They are accessed via the **View > Verification Management** menu.

Opening the Browser or Tracker windows is also possible using the view command:

```
view browser  
view tracker
```

Refer to Verification Management Browser Window and Verification Tracker Window in the *Questa SIM GUI Reference Manual*.

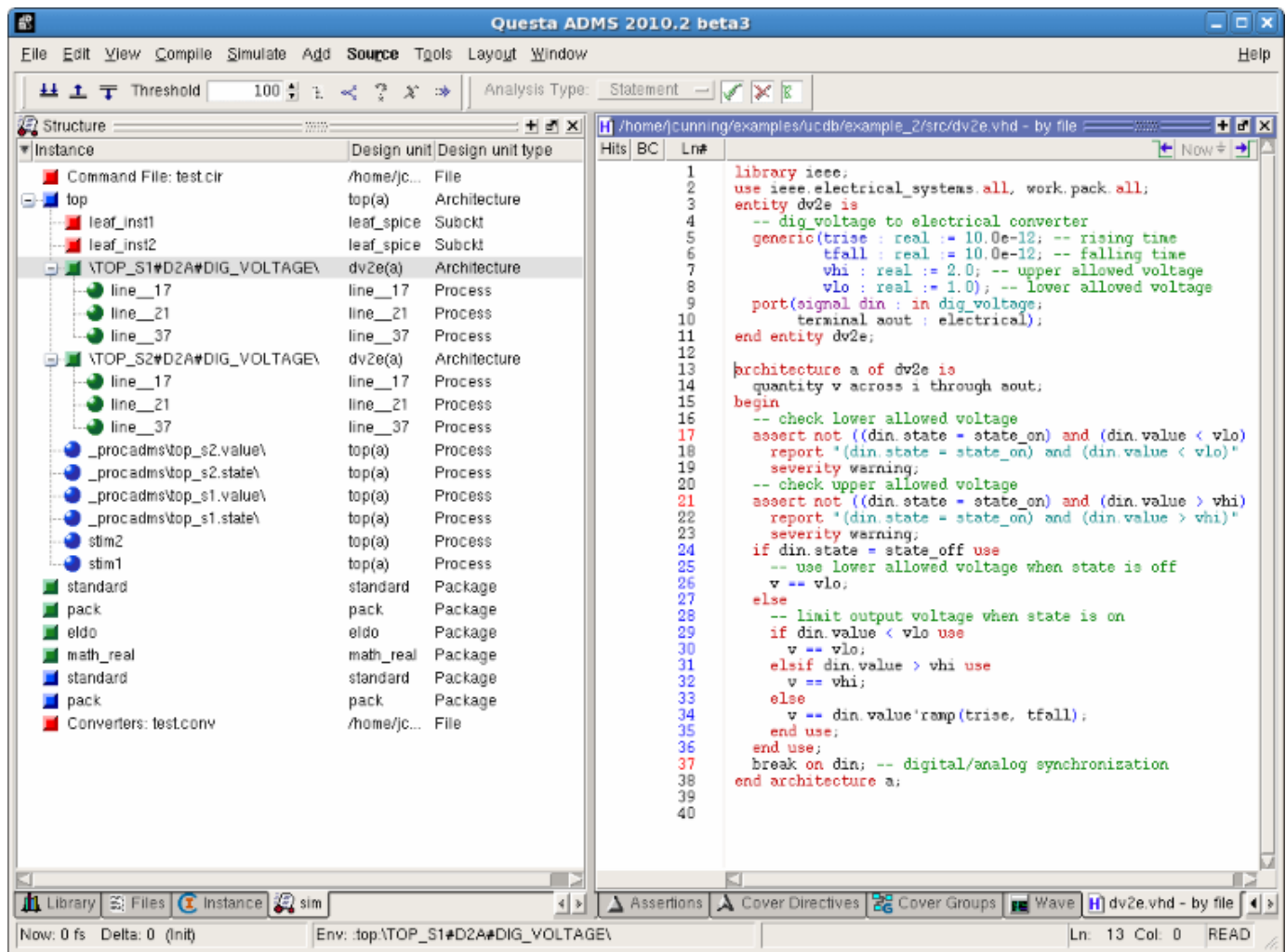
See also Questa Verification Management Overview in the *Questa SIM Verification Management User's Manual* and Coverage and Verification Management in the UCDB in the *Questa SIM User's Manual*.

Capturing VHDL-AMS Assertion Data in Interactive Mode

You can capture and view coverage data in Interactive mode in Questa ADMS.

This example demonstrates how to capture and view coverage data in Interactive mode in Questa ADMS. The design used in this example is VHDL-on-top, and instantiates two SPICE subcircuits beneath it. [Figure 8-15](#) shows the structure of the design, loaded into Questa ADMS.

Figure 8-15: Example Design Loaded - Capturing Coverage Data



The source file *dv2e.vhd* contains instructions to capture assertion data and an if-use clause (highlighted in red):

```
library ieee;
use ieee.electrical_systems.all, work.pack.all;
entity dv2e is
  -- dig_voltage to electrical converter
  generic(trise : real := 10.0e-12; -- rising time
          tfall : real := 10.0e-12; -- falling time
          vhi : real := 2.0; -- upper allowed voltage
          vlo : real := 1.0); -- lower allowed voltage
  port(signal din : in dig_voltage;
        terminal aout : electrical);
end entity dv2e;

architecture a of dv2e is
  quantity v across i through aout;
begin
```

```

-- check lower allowed voltage
assert not ((din.state = state_on) and (din.value < vlo))
    report "(din.state = state_on) and (din.value < vlo)"
    severity warning;
-- check upper allowed voltage
assert not ((din.state = state_on) and (din.value > vhi))
    report "(din.state = state_on) and (din.value > vhi)"
    severity warning;
if din.state = state_off use
    -- use lower allowed voltage when state is off
    v == vlo;
else
    -- limit output voltage when state is on
    if din.value < vlo use
        v == vlo;
    elsif din.value > vhi use
        v == vhi;
    else
        v == din.value'ramp(trise, tfall);
    end use;
end use;
break on din; -- digital/analog synchronization
end architecture a;

```

The SPICE subcircuit *leaf_spice.cir* contains an instruction to capture a safe operating area assertion:

```

.SUBCKT leaf_spice spice_in
.setsoa d r1 i = (-1.5m, +1.5m)
r1 spice_in 0 1k
.ENDS

```

The command file *test.cir* contains instructions for the simulation:

```

.MODEL dv2e hook dv2e(a)
.DEFH00K dv2e

.TRAN 10n 10n

.CHECKSOA

```

Procedure

1. Compile the VHDL-AMS code using `vacom +cover`:

```
vacom +cover src/dv2e.vhd
```


- Launch the simulator with `vasim -coverage`, and specify the command file `test.cir`:

```
vasim -coverage -cmd src/test.cir top -outpath .
```

- Add the VHDL-AMS assertions to the Wave window:

- By default the wildcard filter does not include assertions. therefore, in order to use a wildcard character to add waves to the Wave window, you must change the default setting of the WildcardFilter variable. Choose **Tools > Wildcard Filter** and uncheck the filters for Assertion and ImmediateAssert.

- Add the assertions to the Wave window using:

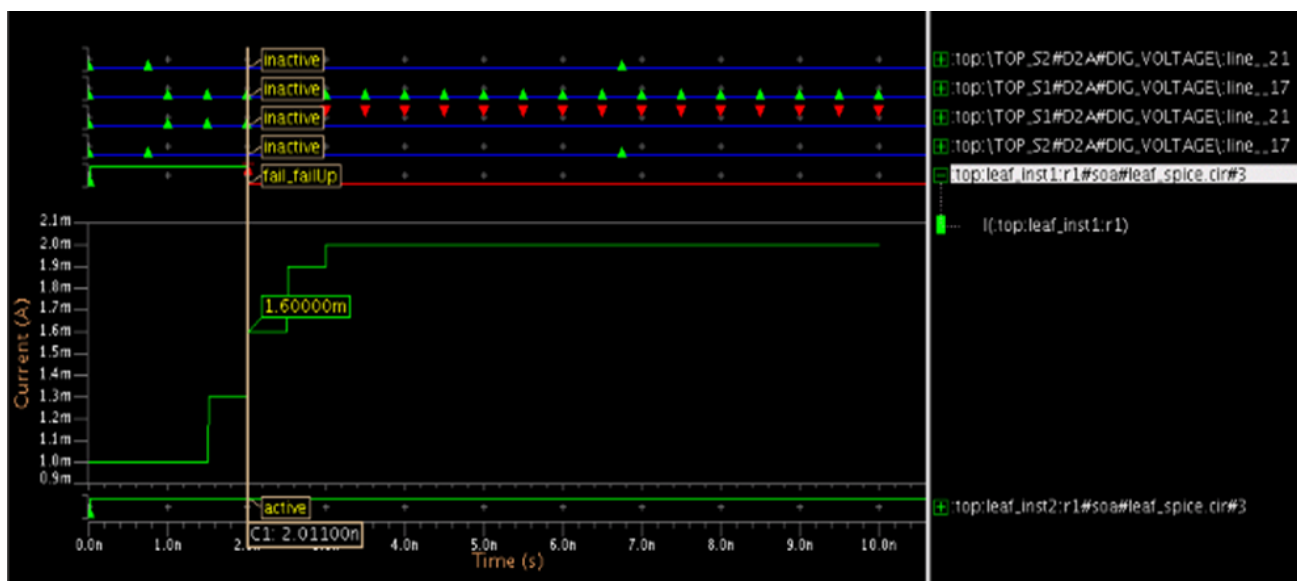
```
add wave -r :*line*
```

- Run the simulation:

```
run -all
```

If `.PLOT` commands have been specified in the source code to plot assertions, then the assertions will be displayed in the **Wave Window (Solido Waveform Analyzer)** during simulation.

Figure 8-16: Plotting Assertions



- Generate some reports:

- coverage report -assert -details

Generates a report showing the number of fail counts as shown in **Figure 8-17**:

Figure 8-17: Coverage Report Assertion Results

```

Transcript :
#
#
# VASIM 8> coverage report -assert -details
#
# ASSERTION RESULTS:
# -----
# Name                               File(Line)           Failure Pass
#                               Count      Count
# -----
# :top:leaf_inst1:R1#S0A#leaf_spice.cir#3
#                               /home/agardeux/UCDB/Demo/src/leaf_spice.cir(3)    94    -
# :top:leaf_inst2:R1#S0A#leaf_spice.cir#3
#                               /home/agardeux/UCDB/Demo/src/leaf_spice.cir(3)    0    -
# :top:\TOP_S1#D2A#DIG_VOLTAGE\line__17
#                               /home/agardeux/UCDB/Demo/src/dv2e.vhd(17)      0    -
# :top:\TOP_S1#D2A#DIG_VOLTAGE\line__21
#                               /home/agardeux/UCDB/Demo/src/dv2e.vhd(21)    15    -
# :top:\TOP_S2#D2A#DIG_VOLTAGE\line__17
#                               /home/agardeux/UCDB/Demo/src/dv2e.vhd(17)      0    -
# :top:\TOP_S2#D2A#DIG_VOLTAGE\line__21
#                               /home/agardeux/UCDB/Demo/src/dv2e.vhd(21)      0    -
#
#
#
#
#
#
#
#
#
#

```

- coverage report -code b -details -byinstance

Generates a branch coverage report as shown in [Figure 8-18](#):

Figure 8-18: Branch Coverage Report

```

Transcript
# Coverage Report Summary Data by file
#
# File: /home/agardeux/UCDB/Demo/src/dv2e.vhd
#   Enabled Coverage   Active   Hits   Misses % Covered
#   -----
#   Branches           5         4         1      80.0
#
#
# VASIM 12> coverage report -code b -details
# Coverage Report by file with details
#
# File: /home/agardeux/UCDB/Demo/src/dv2e.vhd
# Branch Coverage:
#   Enabled Coverage   Active   Hits   Misses % Covered
#   -----
#   Branches           5         4         1      80.0
#
# =====Branch Details=====
#
# Branch Coverage for file /home/agardeux/UCDB/Demo/src/dv2e.vhd --
#
# -----IF Branch-----
#   24                   0         470   Count coming in to IF
#   24                   0         21     if din.state = state_off use
#   29                   0         449     if din.value < vlo use
# Branch totals: 2 hits of 2 branches = 100.0%
#
# -----IF Branch-----
#   29                   0         449   Count coming in to IF
#   29                   0         ***0*** if din.value < vlo use
#   31                   0         206     elsif din.value > vhi use
#   34                   0         243     v == din.value'ramp(trise, tfall);
# Branch totals: 2 hits of 3 branches = 66.6%
#
#

```

6. After simulation has completed, save data to the UCDB:

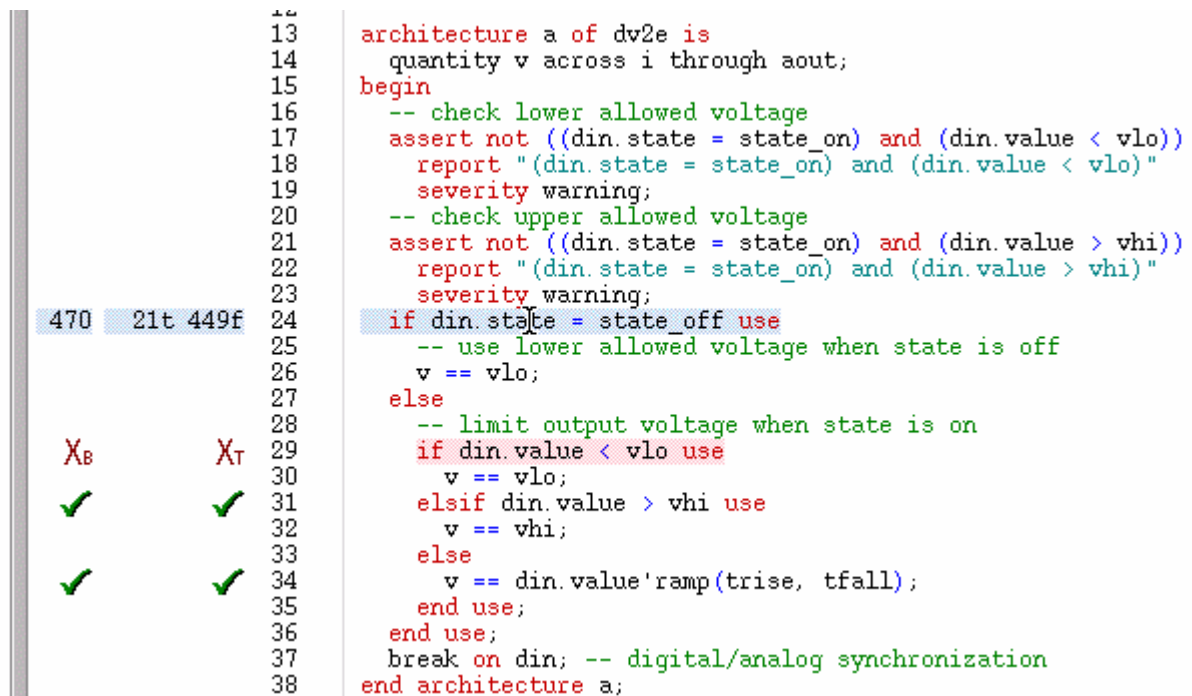
```
coverage save top.ucdb
```

Tip

You can break the simulation to save a *.ucdb* file, start the simulation again, and save another *.ucdb* file when the simulation is complete.

The VHDL-AMS source is annotated in the Source window, showing where the if-use branches are failing.

Figure 8-19: Annotated VHDL-AMS Source in the Source Window



7. (Optional) Open the saved UCDB file to view the coverage data at a later date, using:

```
vasim -viewcov top.ucdb
```

Coverage data is displayed in the windows described in [Coverage Windows](#). Choose **Layout > Coverage** to open a suggested workspace for coverage mode, or use the **View > Coverage** menu to control visibility of these windows.

Related Topics

[Capturing VHDL-AMS Assertion Data in Batch Mode](#)

[Plotting VHDL-AMS Assertions](#)

[Plotting SPICE SOA Assertions](#)

[Naming Conventions for Assertions](#)

[Assertions Window](#)

Capturing VHDL-AMS Assertion Data in Batch Mode

You can capture VHDL-AMS assertion data in batch mode.

Prerequisites

- The file `assertion_coverage.vhd` contains the following VHDL-AMS source code:

```
entity assertion_coverage is end;
architecture a of assertion_coverage is
    signal s : natural;
begin
    s <= (s + 1) mod 6 after 1 ns;
    assert s < 3 report "s >= 3" severity note;
    assert s < 6 report "s >= 6" severity note;
end architecture a;
```

Procedure

1. Create a library called lib:

```
valib lib
```

2. Compile the VHDL-AMS source code `assertion_coverage.vhd` into the library, specifying the coverage option:

```
vacom +cover assertion_coverage.vhd
```

3. Run the simulation in batch mode, specifying the coverage and coverage save options:

```
vasim -c -coverage assertion_coverage -do 'run 20 ns ; coverage save
assertion_coverage.ucdb ; exit'
```

4. Enter the following command to view the UCDB content:

```
vcover dump assertion_coverage.ucdb
```

Refer to `vcover dump` in the *Questa SIM Command Reference Manual* for more information.

Or, to print the results to a file, use:

```
vcover report -file <filename> assertion_coverage.ucdb
```

Refer to `vcover report` in the *Questa SIM Command Reference Manual* for more information.

Results

```

Assertion Scope -
Name           : :assertion_coverage:line__6
Type           : UCDB_ASSERT
Source type    : VHDL-AMS
File info      : name = <path>/assertion_coverage.vhd line = 6
Weight         : 1
Flags          : 0x00010000
Attribute: name = #ACTION# int = 1
Attribute: name = SEVERITY int = 0

Cover item -
  Name         : line__6
  Type         : UCDB_ASSERTBIN
  File info    : name = <path>/assertion_coverage.vhd line = 6
  Count        : 9
  Limit        : -1
  Enabled      : 1
  Has Action   : 1
  Flags        : 0x00010701

Assertion Scope -
Name           : :assertion_coverage:line__7
Type           : UCDB_ASSERT
Source type    : VHDL-AMS
File info      : name = <path>/assertion_coverage.vhd line = 7
Weight         : 1
Flags          : 0x00010000
Attribute: name = #ACTION# int = 1
Attribute: name = SEVERITY int = 0

Cover item -
  Name         : line__7
  Type         : UCDB_ASSERTBIN
  File info    : name = <path>/assertion_coverage.vhd line = 7
  Count        : 0
  Limit        : -1
  Enabled      : 1
  Has Action   : 1
  Flags        : 0x00010701

```

ASSERTBIN — The assertion failure count.

See also Coverage and Verification Management in the UCDB in the *Questa SIM User's Manual*.

Related Topics

[Capturing VHDL-AMS Assertion Data in Interactive Mode](#)

[Plotting VHDL-AMS Assertions](#)

[Naming Conventions for Assertions](#)

Assertions Window

Naming Conventions for Assertions

There are naming conventions for assertions.

VHDL-AMS assertions will be saved in the UCDB database with the following names:

- For sequential assertions:

```
<context_id>#<assertion_id>[_<token>]
```

- <context_id> is the enclosing process label or "*line__*" followed by its line number for an assertion in a process or the subprogram name for an assertion in a subprogram.
- <assertion_id> is the assertion label or "*immed__*" followed by its line number.
- <token> is the number of the assertion where multiple assertions occur at the same line.

- For concurrent assertions:

```
<assertion_id>[_<token>]
```

- <assertion_id> is the assertion label or "*line__*" followed by its line number.
- <token> is the number of the assertion where multiple assertions occur at the same line.

VHDL-AMS if-use and case-use will be saved in the UCDB database with the following name:

```
branch#<line_number>#<token>#
```

SOA assertions are identified as follows:

```
[<device>]_<soaLabelName>
```

where <device> is the device name when the SOA applies to a device and <soaLabelName> is the label if the SOA has a label.

The waveform name for SOA is the label of the <soaLabelName>.

<soaLabelName> must identify a unique SOA, otherwise the following error will be generated:

```
(eldo-xxx) label name of SOA and EXTRACT must be unique.
```

If no label is specified for SOA, the UCDB and waveform name will be:

```
<filename>(<lineNo>)
```

where <filename> designates the name of the file (with its extension) containing the .setsoa command and <lineNo> is the line number of that command.

If no analysis is specified on a SOA, it can be used for several analyses (specified in the netlist). and then assertion counters for such SOA stored into UCDB will represent the sum through all analyses.

See also Coverage and Verification Management in the UCDB and Code Coverage in the *Questa SIM User's Manual*.

Related Topics

[Capturing VHDL-AMS Assertion Data in Interactive Mode](#)

[Capturing VHDL-AMS Assertion Data in Batch Mode](#)

Capturing SPICE Extractions in a UCDB

You can use the SPICE command .EXTRACT to extract circuit information (such as waveform values) from the results of a simulation.

The .EXTRACT command has the following syntax:

```
.EXTRACT function(wave, condition, [min, max]) [objective_info]
```

In the following example, a simple voltage divider SPICE subcircuit is instantiated in a VHDL top design. The VHDL entity and architecture contained within *top.vhd* specifies two signals a and b.

A free running clock flips a between 1 and 0 every nanosecond, and measures the value of b. Two sequential assertion statements define the expected behavior of a on the rising and falling edges of b. When the value of b changes from 0 to 1, and from 1 to 0, the stability of a is monitored. If the value of a has remained the same for the length of time specified in each condition, then the assertion has passed. If the assertion fails, then a message is reported.

Prerequisites

- The VHDL file *top.vhd* contains the following commands:

```
entity top is end;

library ieee;
use ieee.std_logic_1164.all;
```



```

architecture a of top is
    signal a : std_logic := '0';
    signal b : std_logic := 'Z';
begin
    a <= not a after 1 ns;
    inst: entity work.rdiv port map(a, b);
    process(b)
    begin
        if rising_edge(b) then
            assert a'stable(100 ps) report "rising edge assertion failure";
        elsif falling_edge(b) then
            assert a'stable(60 ps) report "falling edge assertion failure";
        end if;
    end process;
end architecture a;

```

- The SPICE Netlist file *rdiv.cir* contains the following commands:

```

.SUBCKT rdiv a b
ra a 1 1
r1 1 b 1
rb b 0 1
.extract tran xup(v(a),5.0)
.extract tran xup(v(a),1.0)
.extract tran xup(v(a),2.0)
.extract tran label=max_va max(v(a)) ubound = 3.0
.extract tran label=min_va min(v(a)) ubound = 1.0
.extract tran label=val_va yval(v(a), 100n) lbound=1.0
.ENDS

```

The first three .EXTRACT commands record the time (x-axis value), each time the voltage at node a rises to a value of 5.0, 1.0 or 2.0.

The fourth and fifth .EXTRACT commands record the maximum and minimum values of the waveform, and specify the assertion that these values should fall between 1.0 and 3.0.

The final .EXTRACT command records the voltage at node a at 100ns, and specifies the condition that this value should not fall below 1.0.

- The VHDL file *rdiv.vhd* defines the digital ports for the subcircuit:

```

library ieee;
use ieee.std_logic_1164.all;
entity rdiv is
    port(signal din : in std_logic;
          signal dout : out std_logic := 'Z');
end entity rdiv;

```

- The command file *test.cmd* declares and inserts the required A2D and D2A boundary elements, and requests a transient analysis:

```
.MODEL A2D_default A2D MODE=std_logic VTH1=0.6 VTH2=0.8 TX=10us
.MODEL D2A_default D2A MODE=std_logic VLO=0.0 VHI=4.8 TRISE=200p
TFALL=100p

.DEFH00K A2D_default D2A_default

.TRAN 3.5n 3.5n
```

- The do file *dofile* instructs the simulator to run, to output the .EXTRACT results and to save the data to the specified UCDB:

```
run -all
output postprocessing
coverage save results.ucdb
```

Procedure

1. Enter the following commands to create a library and compile the source files:

```
valib mylib
vacom rdiv.vhd
vaspi rdiv rdiv@rdiv.cir
vcom top.vhd +cover
```

When compiling the digital top design unit *top.vhd*, the *+cover* option is specified, instructing the simulator to capture coverage data.

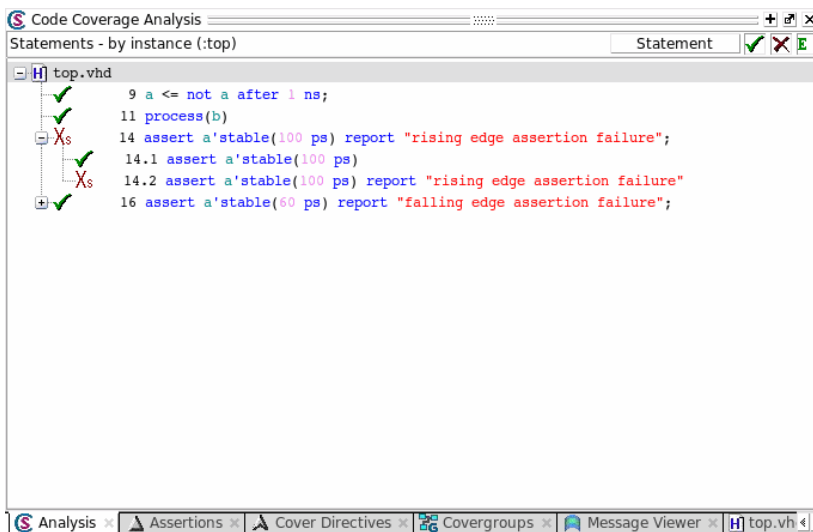
2. Run the simulation in Questa ADMS, using the *-coverage* option to specify coverage mode. In order to include assertions in the recorded coverage data, use the *-autoexclusionsdisable* option:

```
vasim -coverage -cmd test.cmd top -do dofile
-autoexclusionsdisable=assertions
```

The design opens in Questa ADMS, in the layout mode "Coverage". All the required coverage windows are displayed, and populated with the data from the simulation.

3. Select "top" in the Instance Coverage window. The Code Coverage Analysis window shows the statement coverage for that instance.

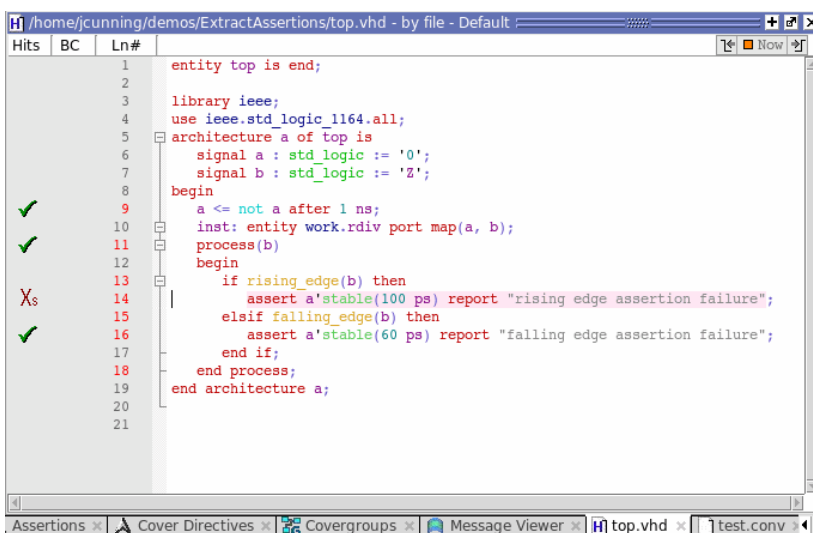
Figure 8-20: Code Coverage Analysis - "top"



Click on each item in the Code Coverage Analysis window to see the details in the Coverage Details window.

- Double-click an item in the Code Coverage Analysis window to open the source file to locate the assertion in the source file.

Figure 8-21: Source Window - "top"



The Hits column indicates where an assertion has passed, failed, or has not been evaluated.

Figure 8-22: Code Coverage Analysis - "rdiv"

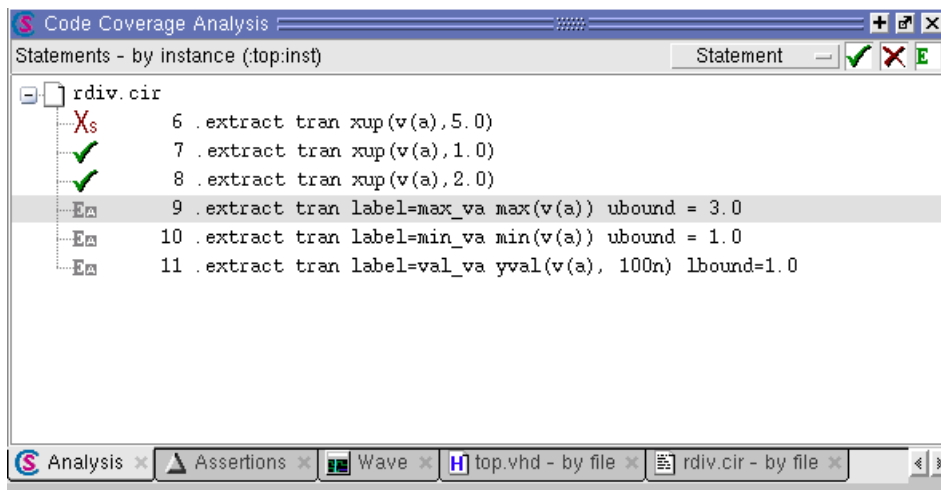
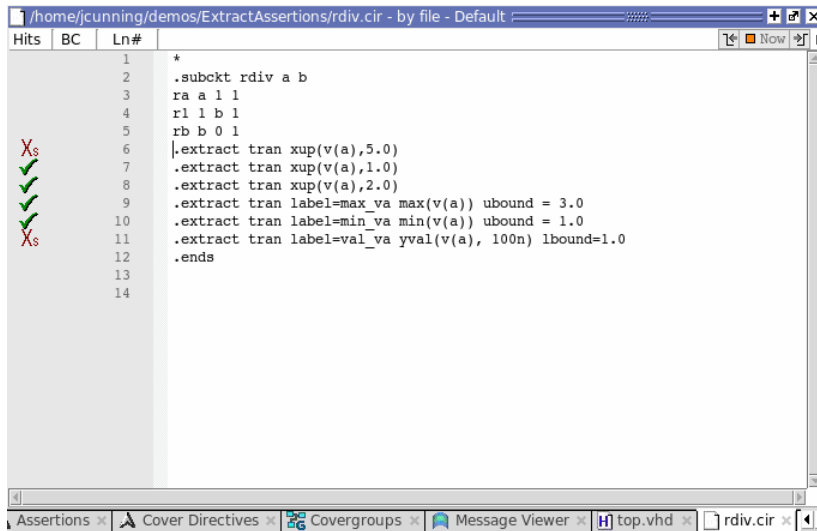


Figure 8-23: Source Window - "rdiv"



The first .EXTRACT statement was not evaluated, as the 'condition on' time is outside the simulation time.

- Each .EXTRACT instruction appears as an assertion in the Assertions window. Double-click an item in the list to open the source file.

Figure 8-24: .EXTRACT Assertions

Name	Assertion Type	Language	Assertion Expression	Enable	Failure Count	Pass Count
:top:line__11#immed__14	Immediate	VHDL	assert (<a_stable_0>)	on	0	0
:top:line__11#immed__16	Immediate	VHDL	assert (<a_stable_1>)	on	1	0
:top:inst:VAL_VA	Extract	SPICE	YVAL(V(A),100N)	on	0	0
:top:inst:MIN_VA	Extract	SPICE	MIN(V(A))	on	0	0
:top:inst:MAX_VA	Extract	SPICE	MAX(V(A))	on	1	0

Assertions Filter: NoFilter

As the output postprocessing command was included in the do file, the results of the extractions are listed in the .chi file, and written to the transcript.

Figure 8-25: Extract Information in the .chi File

```

212 0*
213 0****          EXTRACT INFORMATION
214 0****    TEMPERATURE = 2.7000E+01 Celsius
215
216 0*****
217
218
219
220 XUP(V(A),5.0) cannot be measured. Check your input netlist.
221 * INST XUP(V(A),1.0) = 1.0556E-09
222 * INST XUP(V(A),2.0) = 1.1111E-09
223 * INST.MAX_VA = 3.6000E+00 (upper boundary violated, limit is 3)
224 * INST.MIN_VA = -3.3741E-15
225 INST.VAL_VA cannot be measured. Check your input netlist.
226

```

Results

You can view coverage data related to .EXTRACT results in coverage mode (vasim -coverage):

- Coverage statements indicate if the .EXTRACT has been evaluated or not. For instance, a .EXTRACT statement is not evaluated if the 'condition on' time is outside the simulation time.
- Assertions represent the result of the objective_info constraints if objective_info is present:

```

objective_info:=
  equal=rvalue
  | {lbound=rvalue | ubound=rvalue}
  | lbound=rvalue ubound=rvalue

```

By default, extracts saved as assertions are excluded from statement coverage. You must specify the -autoexclusionsdisable option to enable this.

SPICE .EXTRACT results are saved in the hierarchical structure of the UCDB at the place that reflects their location in the design, and use the following naming convention:

```
<extractLabelName>
```

where <extractLabelName> is the label if the EXTRACT has a label. A label name is required to manage it as an assertion.

The waveform name for EXTRACT is <extractLabelName>.

<extractLabelName> must identify a unique EXTRACT command, otherwise the following error will be generated:

```
(eldo-xxx) label name of SOA and EXTRACT must be unique.
```

Related Topics

[Capturing Multiple-Run Simulation Data in a UCDB](#)

[Naming Conventions for Assertions](#)

[Assertions Window](#)

Capturing Multiple-Run Simulation Data in a UCDB

You can use the `vasim -coverage` and `coverage save` commands to save the results of several runs of a multiple-run simulation to a UCDB.

Prerequisites

- The SPICE netlist `test_step.cir` contains a simple circuit, the instructions for a multiple-run simulation, and the following .EXTRACT commands:

```
.param ampl=5

v1 1 0 sin 0 ampl
r1 1 0 1k

.TRAN 1n 1n

.extract label=extr1 max(i(r1))
.extract label=extr2 min(i(r1)) ubound=-7.5m
.extract label=extr3 xup(i(r1), 7m)

.step param ampl 5 10 1
```

Procedure

1. Load the design, specifying the -coverage argument:

```
vasim -coverage -cmd test_step.cir
```

2. Use the coverage save -onexit <ucdb_file> command to capture the results of each run in the specified UCDB file:

```
coverage save -onexit -testname mytest test_step.ucdb
```

This command must be executed before the simulation is run. If the coverage save command is run *after* a multiple-run simulation has completed, then only the data from the last run is saved to the UCDB.

In this example, the -testname argument specifies a prefix name for the series of runs. This name will be appended with `_run_<n>` to form the TESTNAME attribute, a unique name for each of the runs: `mytest_run_1`, to `mytest_run_6`. The merged UCDB file will be called `test_step.ucdb`.

If -testname is not used, the TESTNAME attribute will default to `run_<n>`.

Tip

To view the value of a test record attribute, use:

```
vcover attribute -name <attribute> <ucdbfile>
```

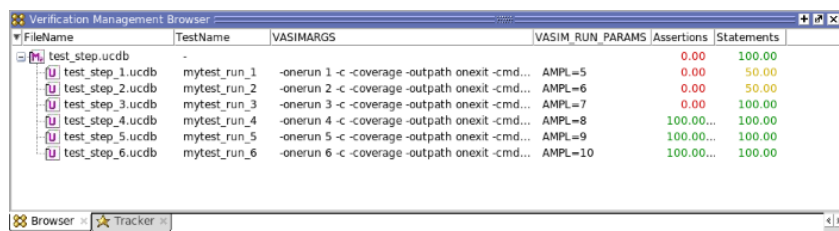
3. Perform the multiple-run simulation using the run -all command:

```
run -all
```

All of the UCDB files generated for each run of the sweep are merged into a single UCDB file, `test_step.ucdb`.

4. Choose **View > Verification Management > Browser**.
The Verification Management Browser Window opens. Refer to Verification Management Browser Window in the *Questa SIM GUI Reference Manual* for more information.
5. Right-click in the Verification Management Browser and choose **Add File** from the menu. Select the merged UCDB file, `test_step.ucdb`.
6. Expand the merged UCDB file to display the individual files containing all of the test records for the multiple-run simulation, in the Verification Management Browser.

Figure 8-26: Multiple-Run Results in the Verification Management Browser



FileName	TestName	VASIMARGS	VASIM_RUN_PARAMS	Assertions	Statements
test_step.ucdb				0.00	100.00
test_step_1.ucdb	mytest_run_1	-onerun 1 -c -coverage -outpath onexit -cmd...	AMPL=5	0.00	50.00
test_step_2.ucdb	mytest_run_2	-onerun 2 -c -coverage -outpath onexit -cmd...	AMPL=6	0.00	50.00
test_step_3.ucdb	mytest_run_3	-onerun 3 -c -coverage -outpath onexit -cmd...	AMPL=7	0.00	100.00
test_step_4.ucdb	mytest_run_4	-onerun 4 -c -coverage -outpath onexit -cmd...	AMPL=8	100.00...	100.00
test_step_5.ucdb	mytest_run_5	-onerun 5 -c -coverage -outpath onexit -cmd...	AMPL=9	100.00...	100.00
test_step_6.ucdb	mytest_run_6	-onerun 6 -c -coverage -outpath onexit -cmd...	AMPL=10	100.00...	100.00

The VASIMARGS and VASIM_RUN_PARAMS test record attributes show the vasim command arguments required to re-execute the run, and the parameters corresponding to that run.

Tip

Right-click on a column header in the Verification Management Browser, and choose Change Column Visibility to specify which attributes to view.

- 7. To re-run a test, right-click the required test in the Verification Management Browser and choose **Command Execution > Execute on Selected > TestReRun**.

You can use this to re-run a specific run of a multiple-run analysis, without first running the main netlist.

New output files prefixed by `[_alter_<altername>]_run_<n>` will be generated for that specific run.

A second elaboration phase will be required if the selected simulation run does not use the parameter values of the nominal netlist.

Results

Each run of the simulation generates its own UCDB file, and at the end of the sweep, they are merged into a single parent UCDB file.

You can view the merged UCDB file and the individual tests for each run in the Verification Management Browser window. You can then perform re-runs of the simulation from the Verification Management Browser window, or re-run specific runs of a multiple-run simulation without first running the main netlist.

Refer to Manage Test Data in UCDBs and Running Tests and Collecting Data in the *Questa SIM User’s Manual* for more information.

Related Topics

Multiple-Run Analyses

Plotting VHDL-AMS Assertions

You can plot VHDL-AMS assertions in the Solido Waveform Analyzer Wave window.

Tip

Refer to [Plotting Assertions](#) in *Solido Waveform Analyzer User's and Reference Manual* for more information.

Restrictions and Limitations

- It is not possible to drag and drop assertions to the Wave window.
- Waves plotted in Solido Waveform Analyzer show assertion states and events for all waves related to the assertion evaluation. VHDL-AMS assertions have only one state: INACTIVE, and two possible events: PASS or FAIL.
- Pure digital assertions, must be plotted in the Questa WLF viewer.

Procedure

Right-click the assertion in the [Assertions Window](#), then choose **Add Wave > Selected Objects**.

Results

The assertion is saved in the JWDB database and displayed in Solido Waveform Analyzer.

You can also use the [add wave](#) command to plot VHDL-AMS assertions. You must provide the exact assertion name when using add wave because immediate assertions are excluded by default when performing wildcard matches with simulator commands.

To change the default behavior of wildcard matches, choose **Tools > Wildcard Filter**, then uncheck Assertion and ImmediateAssert. You can also change this variable using the command line. Refer to Using the WildcardFilter Preference Variable in the *Questa SIM Command Reference Manual* for more information.

Related Topics

[Plotting SPICE SOA Assertions](#)
[Naming Conventions for Assertions](#)

Plotting SPICE SOA Assertions

You can plot SPICE SOA assertions.

To plot SPICE SOA assertions in the Solido Waveform Analyzer Wave window: Right-click on the SPICE SOA assertion in the [Assertions Window](#), and select **Add Wave > Selected Objects**.

SPICE SOA are saved in the JWDB database and displayed in Solido Waveform Analyzer.

You can also request plots of SPICE SOA assertions using the add wave, add log, .PLOT or .PROBE commands.

When using add wave or add log, the wave is named as listed in the UCDB section.

The .PLOT and .PROBE commands are used as follows:

- With the SOA_name, as defined in UCDB:

```
.PLOT soa ( [ <hierachical_path> ' .' ] <SOA_name> )
```

- For all SOA at a given hierarchy level:

```
.PLOT soa ( [ <hierachical_path> . ] '*' )
```

The wave name will be saved in the UCDB.

Refer to [Plotting Safe Operating Area Limits](#) in *Eldo User's Manual* for more information.

Related Topics

[Plotting VHDL-AMS Assertions](#)

[Naming Conventions for Assertions](#)

SystemVerilog Assertions

You can instantiate SystemVerilog modules that contain assertions inside Verilog-AMS descriptions. This enables you to verify the behavior of a design.

An assertion is a way to express a property, such as: "When the hardware signal p1 changes from 0 to 1, then the state variable of instance ab1 in the design is expected to change from value x to y." When the conditions stated in the assertion are met, the assertion has "passed". Assertion passes and failures are reported in the transcript.

Questa ADMS extends the Verilog-AMS language to include the assertion parts of System Verilog grammar constructs. This allows digital assertions to be written in Verilog-AMS modules. Once digital assertions are written in this way, you can access voltages and electrical objects within them.

SystemVerilog assertions can be declared within Verilog-AMS descriptions. These become System Verilog-AMS descriptions and should be compiled with:

```
valog -svams
```

Supported assertions are described in: "Assertion declarations" in the IEEE document "IEEE Standard for System Verilog—Unified Hardware Design, Specification, and Verification Language", IEEE STD 1800-2012, appendix A.2.10.

The exceptions to the standard SystemVerilog syntax are detailed in [SystemVerilog Assertion Declarations](#).

Access to Analog Objects Inside Assertions..... 8-61

Access to Analog Objects Inside Assertions

You can get access to analog objects inside assertions.

Analog elements are supported in the following parts of the assertion:

- In expressions you can use an analog access function applied to an object with a continuous discipline, for instance, $V(e)$, to express the voltage of the electrical object e .
- In expressions, analog events are supported, for example:

```
@(cross(V(e)-3.3)
```

The point where voltage e crosses 3.3v.

- Properties and sequences can have analog arguments, for example:

```
"property p(electrical e)"
```

Example

This example demonstrates the use of an assertion designed to detect floating nodes. For this assertion to pass, the following condition must be true: When $vdd1$ is powered down ($vdd1 < 4.5$), either $isolate$ is high or $vdd2$ should be powered down ($vdd2 < 4.5$).

For the purpose of determining if a supply is powered up or down, "droops" of less than 25 ns are ignored.

```
realtime t;
  (@(cross(V(vdd1)-4.5, -1)) (1, t=$realtime)) |->
    ((isolate || V(vdd2) < 4.5)
or
  @(cross(V(vdd2)-4.5, -1)) ($realtime t < 25 ns)));
```

Questa ADMS supports the following System Verilog-AMS construction to represent this property. The electrical ports of the property v1 and v2 are connected in assert to vdd1 and vdd2. @(cross(V(v1)-4.5)) represents an analog event, and V(v2) represents an access to a voltage:

```
property p1(electrical v1,electrical v2);
  real t;
  @(cross(V(v1)-4.5)) (1,t=$realtime)
  |-> ((isolate || V(v2)<4.5)
  or
  @(cross(V(v2)-4.5)) ($realtime-t<25));
endproperty

assert property (p1(vdd1,vdd2)) $display("e1 is raised");
```

The following module, that uses the property, is compiled with valog -svams. As stimuli, it contains two sinusoids with a small shift (less than 25ns) and a small difference in period. The isolate signal is initially 1, but changes to 0 after 50ns. Notice that in the \$realtime statement, the condition < 25ns of the property is written < 2.5 because of the time unit.

```
`include "disciplines.h"
`include "constants.h"

`timescale 10ns/1ps

module e1_a(v1,v2,isolate);
  input electrical v1;
  input electrical v2;
  input wire isolate;

  property p1(electrical pv1,electrical pv2);
    real t;
    @(cross(V(pv1)-4.5,-1)) (1,t=$realtime) |->
      ((isolate || V(pv2)<4.5)
      or
      (@(cross(V(pv2)-4.5,-1)) (($realtime-t)<2.5)));
  endproperty

  ap1: assert property (p1(v1,v2)) $display("p1 analog is raised at time
  %t %g",$realtime,$abstime);

endmodule // e1_a

module top_e1_a;
  electrical vdd1;
  electrical vdd2;
  parameter real amp=5.0;

  reg        isolate;
  initial
  begin
    isolate=1;
    #5 isolate=0;
```

```

end

analog
begin
    V(vdd1) <+ 5.0+sin(($abstime+0.195e-7)*1e7*`M_TWO_PI)*amp/
exp($abstime*5e6);
    V(vdd2) <+ 5.0+sin($abstime*1.25e7*`M_TWO_PI)*amp/
exp($abstime*5e6);
end

e1_a i1_1(vdd1,vdd2,isolate);
endmodule // top_e1_a

```

A simple netlist contains instructions for the simulation:

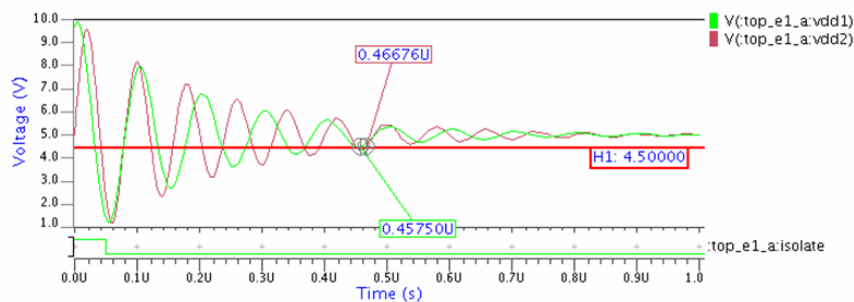
```

*
.tran 1ns 1000ns

```

Figure 8-27 shows the simulation results in Solido Waveform Analyzer. The vdd1 waveform is shown in green, the vdd2 waveform in red. In the second waveform row, the isolate signal is shown, changing from 1 to 0.

Figure 8-27: Simulation Results



The time points where the assertion is true (4 occurrences) and one where it is false (1 occurrence) are reported in the transcript:

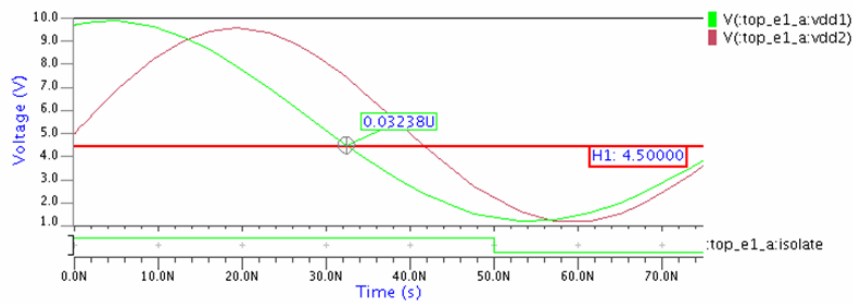
```

# Simulation time for "run -all" is 1.000000E+03 ns
# p1 analog is raised at time          32380 3.238e-08
# p1 analog is raised at time          133629 1.33629e-07
# p1 analog is raised at time          235774 2.35774e-07
# ** Error: Assertion error.
#   Time: 366989 ps Started: 339712 ps Scope:
top_e1_a.i1_1.__dig_submod.ap1 File: d.sva Line: 19
# p1 analog is raised at time          451019 4.51019e-07

```

Figure 8-28 shows the results at 32ns, the first reported time point.

Figure 8-28: Assertion Passed at Time 32ns

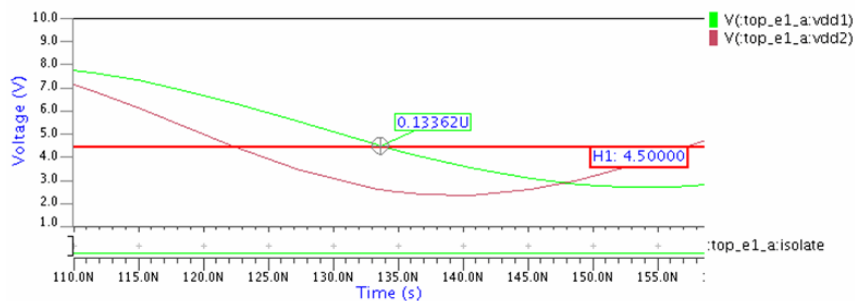


When vdd1 crosses 4.5V (descending), the isolate signal is 1. Therefore, the part of the property highlighted in red below, is true, and the assertion is passed:

```
property p1(electrical pv1,electrical pv2);
  real t;
  @(cross(V(pv1)-4.5,-1)) (1,t=$realtime) |->
    ((isolate || V(pv2)<4.5)
    or
    (@(cross(V(pv2)-4.5,-1)) (($realtime-t)<2.5)));
endproperty
```

Figure 8-29 shows the results at 133ns, the second reported time point.

Figure 8-29: Assertion Passed at Time 133ns

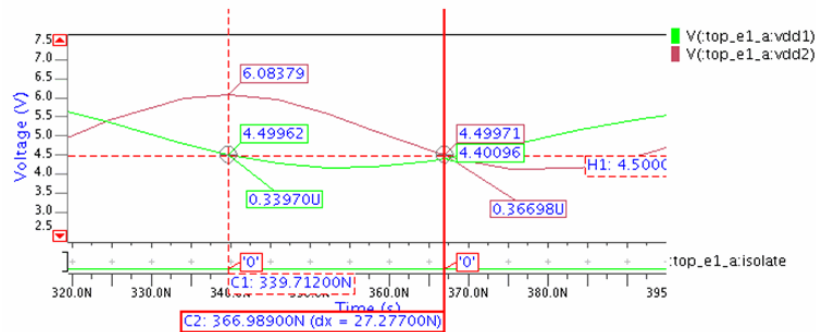


Here, when vdd1 crosses 4.5V, the isolate signal is 0, but vdd2 is below vdd1. So the property is still true, and the assertion is passed:

```
property p1(electrical pv1,electrical pv2);
  real t;
  @(cross(V(pv1)-4.5,-1)) (1,t=$realtime) |->
    ((isolate || V(pv2)<4.5)
    or
    (@(cross(V(pv2)-4.5,-1)) (($realtime-t)<2.5)));
endproperty
```

Figure 8-30 shows the results where the assertion fails.

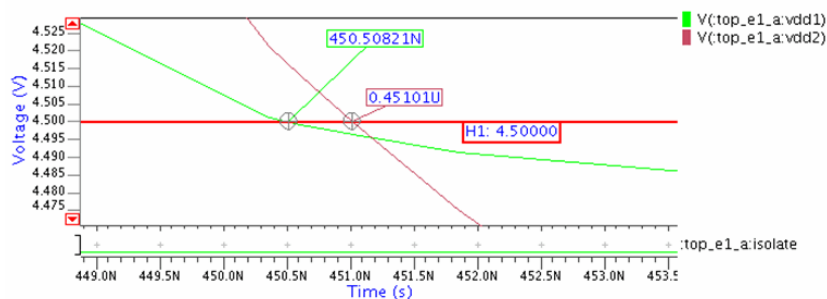
Figure 8-30: Assertion Fails



A vertical cursor C1 is placed at the time that evaluation of the first part of the assertion starts (shown in red below). A second cursor C2, is placed at the time the evaluation fails, which corresponds to the time the assertion below (highlighted in amber) is evaluated. At this time, the dx value, the simulation time between the two cursors, is 27ns, which is greater than the limit of 25ns.

```
property p1(electrical pv1,electrical pv2);
    real t;
    @(cross(V(pv1)-4.5,-1)) (1,t=$realttime) |->
        ((isolate || V(pv2)<4.5)
        or
        (@(cross(V(pv2)-4.5,-1)) (($realttime-t)<2.5)));
endproperty
```

Figure 8-31: Assertion Passes Resulting From a “Droop” of Less Than 25ns



At 450ns the vdd1 crosses 4.5V but vdd2 is still greater than 4.5V until 451ns, just 1 ns later. As this is less than the 25ns specified in the assertion, the red part of the property below is evaluated to true, and therefore the assertion is passed:

```
property p1(electrical pv1,electrical pv2);
    real t;
    @(cross(V(pv1)-4.5, -1)) (1,t=$realtime) | ->
        ((isolate || V(pv2)<4.5)
```

```

        or
        (@(cross(V(pv2)-4.5, -1)) (($realtime-t)<2.5)));
    endproperty

```

Related Topics

Verilog-AMS Extensions

SystemVerilog bind Construct

The SystemVerilog bind construct enables you to associate a digital module, interface, or program instance with another module or module instance. This enables you to attach verification criteria with minimal changes to design code.

The process to do this is as follows:

1. Create a digital module or instance containing assertions.
2. Designate a target module or instance, which can be digital, analog or mixed-signal.
3. Use the bind statement to create an instance of the assertion module inside the target module with the specified instance name and port connections. The syntax of the bind is as follows:

```

bind <target_module_scope> <assertion_module_name> <instance_name>
    <port_connections>

```

Refer to Using SystemVerilog bind Construct in Mixed-Language Designs in the *Questa SIM User's Manual* for more information.

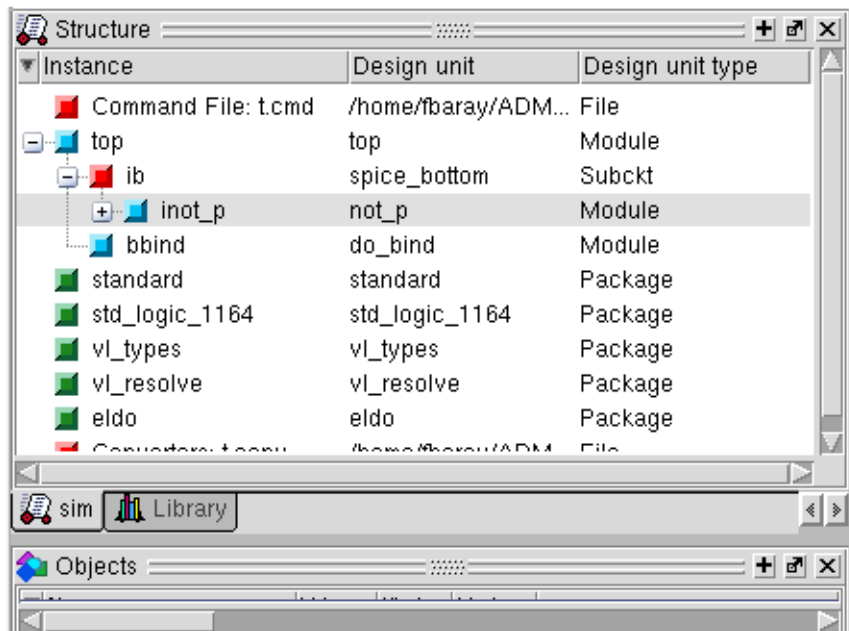
For example, in the bind instance (a SystemVerilog module do_bind), an instance of the design unit not_p called inot_p (with ports a, b) is added into the SPICE subcircuit ib within the module top:

```

module do_bind();
    bind top.ib not_p inot_p(a,b);
endmodule

```


Figure 8-32: Structure of Bound Instance inot_p



To instantiate a SPICE subcircuit directly inside a digital unit, it is necessary to use the `vaspi` command to create a link between a digital representation of the SPICE subcircuit and the real SPICE subcircuit. In this example, a Verilog digital wrapper named `bottom` is created:

```
module bottom(a,b);
  output reg a;
  output wire b;
endmodule // bottom
```

The `vaspi` command is used to link `bottom` with the `spice_bottom` subcircuit:

```
vaspi bottom spice_bottom@t.cmd
```

In the Structure window, the design unit name for the `ib` instance is `spice_bottom` (not `bottom`).

Then, in order to bind into the SPICE subcircuit, the name of the digital wrapper is used when writing the bind statement:

```
bind bottom not_p inot_p(a,b)
```

Rules

If, as a result of a SystemVerilog `bind` statement, an analog net is connected to a digital port, a boundary element will be inserted. Its direction will depend on the drivers and readers on that port, which in turn,

are dependent upon how the port of the bound instance is used inside the module. For instance, if an inout port is connected to an analog net, but it has no drivers, only an A2D converter will be inserted.

.DEFHOOK rules and analog and digital types/natures/disciplines can be used to decide which converter to insert.

If the bound instance name defined in `bind_directive` is an escaped name and the target scope is a SPICE region where escaped names are not supported but extended ones are, then the bound instance name is converted from escaped to extended.

- actuals of parameters may be:
 - literal values
 - simple name
 - hierarchical references

Actual names are resolved in the target scope context.

- actuals of ports may be:
 - simple name
 - hierarchical references

Actual names are resolved in the target scope context.

When the target scope unit is instantiated using a `vaspi` command (meaning a wrapper is used) and port associations are provided in an interface association (`.assoc`) file, then actual names are resolved in the same way as hierarchical references:

- Typically, actual names are resolved in the target scope
- If not found, names are resolved in the wrapper used.

The following limitations exist:

- The added instance must be a digital design unit. It cannot be analog or mixed-signal.
- The SPICE `.BIND` command cannot be applied on bound instances.

Related Topics

[.DEFHOOK \[Questa ADMS Command Reference\]](#)

[.BIND \[Questa ADMS Command Reference\]](#)

Access to Analog Objects from SystemVerilog

You can sample instantaneous values in the analog solver from SystemVerilog code.

You can access voltages and currents from Spice, branch and net potentials and flows from Verilog-AMS, and values of across, through, terminal reference, and terminal contribution quantities from VHDL-AMS.

Note

For Verilog-AMS discipline electrical, the potential is voltage, and the flow is current. For VHDL-AMS nature electrical, the across type is voltage, and the through type is current.

Two SystemVerilog system functions enable you to get the values by referring to a Spice pin or node, a Verilog-AMS net, named branch, or implicit branch, or a VHDL-AMS terminal or branch quantity using a SystemVerilog hierarchical name reference. The instantaneous value is obtained by linear interpolation between analog solution points. The system functions may be called in any sequential context.

Table 8-5 lists the two SystemVerilog system functions for accessing analog objects from SystemVerilog.

Table 8-5: SystemVerilog System Functions for Accessing Analog Objects

SystemVerilog system function	Description
get_potential	Returns a real value representing the voltage of a Spice pin or node, the potential of a Verilog-AMS net with continuous discipline, a named branch, or implicit branch, or a VHDL-AMS terminal reference or branch across quantity.
get_flow	Returns a real value representing the current through a Spice pin, the flow through a Verilog-AMS port with continuous discipline, a named branch, or implicit branch, or the value of a VHDL-AMS terminal port contribution or branch through quantity.

Related Topics

[get_potential](#)

[get_flow](#)

[get_terminal_across_value](#)

get_potential

Returns a real value representing the voltage of a SPICE pin or node, the potential of a Verilog-AMS net with continuous discipline, a named branch, an implicit branch, or a VHDL-AMS terminal or branch across quantity.

Usage

`$get_potential(name)`

Arguments

- *name*

name must be a SystemVerilog literal string or string variable that designates an object. It may designate a Spice, Verilog-AMS, or VHDL-AMS object and this object must be scalar. It can be specified using a SystemVerilog hierarchical name reference which may contain digital (SystemVerilog or VHDL) as well as analog (Spice, Verilog-AMS, VHDL-AMS) instances in the hierarchy. The object is found using SystemVerilog search rules.

An error will be issued if *name* designates a composite object or does not designate an object of the appropriate kind.

Return Values

- The voltage, potential, or reference or across quantity of the designated object.

Description

Returns a real value representing:

- The voltage of the Spice net or pin designated by *name*.
- The potential of a Verilog-AMS net with continuous discipline, a named branch, or implicit branch designated by *name*.
- The value of a VHDL-AMS terminal contribution or branch across quantity designated by *name*.

Examples

```
get_potential("name") // pin, port, node, net, branch, branch quantity
get_potential("top.dut.name") // pin, port, node, net, branch, branch
quantity
get_potential("top.dut.s.yi1.p,n") // implicit branch
get_potential("$root.top.dut.name") // rooted reference
```

Related Topics

[Access to Analog Objects from SystemVerilog](#)

get_flow

Returns a real value representing the current through a subcircuit Spice pin, the flow through a Verilog-AMS port with continuous discipline, a named branch, an implicit branch, or the value of a VHDL-AMS terminal port contribution or branch across quantity.

Usage

`$get_flow(name)`

Arguments

- *name*

name must be a SystemVerilog literal string or string variable that designates an object. It may designate a Spice, Verilog-AMS, or VHDL-AMS object and this object must be scalar. It can be specified using a SystemVerilog hierarchical name reference which may contain digital (SystemVerilog or VHDL) as well as analog (Spice, Verilog-AMS, VHDL-AMS) instances in the hierarchy. The object is found using SystemVerilog search rules.

name must be a literal, a constant or a variable with an initial value. If it is a variable, then the value may not be changed during simulation. An error will be issued if such an attempt is made.

An error will be issued if *name* designates a composite object or does not designate an object of the appropriate kind.

Return Values

- The current, flow, or contribution or through quantity of the designated object.

Port/pin electrical current flows into the device, subcircuit, module, or entity, and the values of Verilog-AMS branch flows and VHDL through quantities are as defined in the respective HDL.

Description

Returns a real value representing:

- The current through a subcircuit Spice pin designated by *name*.
- The flow through a Verilog-AMS port net with continuous discipline, a named branch, or implicit branch designated by *name*.
- The value of a VHDL-AMS terminal port contribution or branch through quantity designated by *name*.

Examples

```
get_flow("name") // pin, port net or terminal, branch, branch quantity
get_flow("top.dut.name") // pin, port, branch, branch quantity
get_flow("top.dut.s.yi1.p,n") // implicit branch
get_flow("$root.top.dut.name") // rooted reference
```

Related Topics

[Access to Analog Objects from SystemVerilog](#)

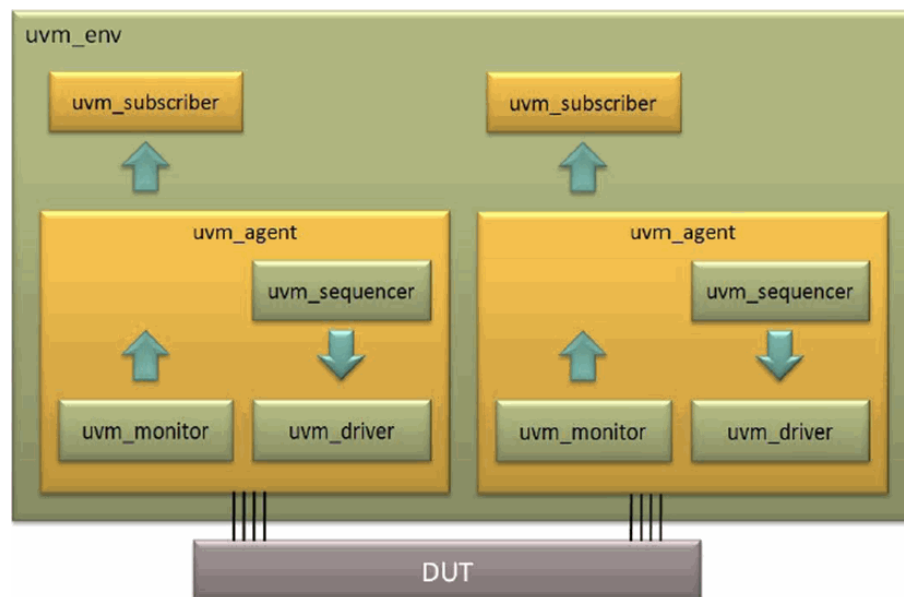
[get_terminal_across_value](#)

Universal Verification Methodology

Universal Verification Methodology (UVM) enables you to perform constrained-random verification, by building configurable, flexible testbenches for designs developed in any language.

Communication in UVM is at the transaction-level modeling (based on SystemC TLM). UVM puts emphasis on stimulus generation and in particular enables you to construct layered sequential stimulus.

Figure 8-33: UVM Components



The main differences between using OVM/UVM in a mixed-signal environment as opposed to a pure digital environment is in how the verification engineer must handle analog ports and analog measurements at the monitor/driver/responder level.

To support this methodology, a package of analog sources (OSRC) and probes (OPRB) is provided, which range in complexity from the simplest voltage measurement and waveform generator to complex waveform extractions.

Workflow

A design under test (DUT) is a candidate for using UVM with Questa ADMS in the following cases:

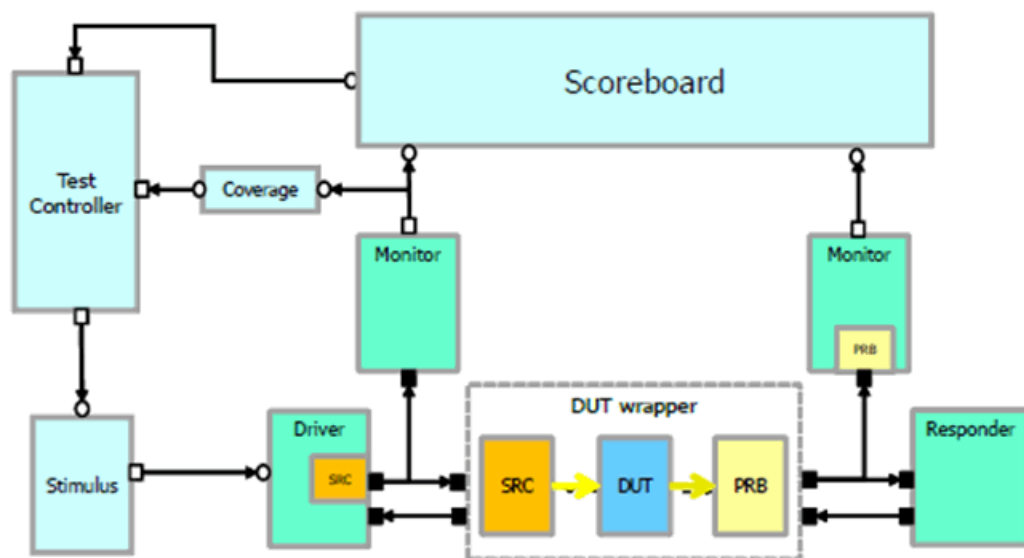
- The DUT has analog ports that need to be stimulated, loaded (with, for example, a resistor to ground) or probed for results.
- The DUT internally refers to global electrical nodes that require similar attention.

The following actions are necessary in order for the monitor and driver to interact with the analog/mixed-signal ports:

1. The DUT is wrapped in a Verilog-AMS wrapper DUT
2. The wrapper has the digital input ports of the original DUT, plus digital ports to specify analog stimulus and loads, probe controls, and digital outputs coding analog results
3. The wrapper instantiates the original DUT and the analog/mixed-signal code required to translate between the analog ports of the original DUT and the extended ports of the wrapper DUT.

A small collection of source (SRC) and probe (PRB) routines are provided to automate the exchange. A SRC is divided into a class that is instantiated in the driver, and a Verilog-AMS module that is instantiated in the wrapper. A PRB is divided into a class that is instantiated in the monitor, and a Verilog-AMS module that is instantiated in the wrapper. The two parts of the SRC or PRB communicate through the extended wrapper ports.

Figure 8-34: Typical UVM Flow Incorporating a DUT Wrapper



SRCs and PRBs range over the standard stimulus and measurement requirements familiar in the analog simulation world. The SRCs are conventional things you find in an analog simulator, for example, voltage or current sources, pulse generators, sine wave generators and modulators. The PRBs analyze the analog waveform over time in many different ways to make typical analog measurements, for example, peak over an interval, frequency, rise or fall time, settling time, Fourier transform, phase detection and jitter measurement.

In Questa ADMS, an intelligent probe may actually control the time step (mandatory for FFT) or the accuracy parameters over an interval. Alternatively, you can write a probe that doesn't affect the analog engine at all.

You should consider SRCs and PRBs to be a new OVM layer, lying at the bottom just over the DUT. However, they are below the transaction level and are not very well isolated from DUT details. Implicitly placed connect modules are not appropriate for this. Connect modules are useful in the DUT, where they implement, with a clean syntax, a coherent model of what is at the physical level an electrical equipotential point. That is not what is going on in the machinery of measurement and stimulus represented by SRCs and PRBs. By placing the SRC and PRB code in the driver and monitor (and in the DUT wrapper) a locality of reference is achieved, which would not be available if implicitly placed connect modules are used. Every PRB and SRC would require a unique connect module and unique discipline to indirectly trigger the insertion of the connect module.

Figure 8-35: DUT Wrapper



Digital connections are shown in black, analog/mixed-signal connections are shown in yellow.

Related Topics

Verification

9. Boundary Elements

In mixed-signal designs, boundary elements are required at analog-digital boundaries. Questa ADMS provides a set of built-in boundary elements that can be used to do this. Alternatively you can supply your own, written in VHDL-AMS or Verilog-AMS.

You can control where in the design boundary elements are added. The insertion of boundary elements is controlled using a set of boundary element commands.

What is a Boundary Element?	9-1
Built-in or Custom Boundary Elements	9-8
Boundary Element Insertion	9-10
Viewing Mixed-Signal Nets in Questa ADMS	9-36
Boundary Elements Examples	9-43

What is a Boundary Element?

A boundary element is an analog-to-digital, digital-to-analog, or bidirectional converter. Boundary elements are required at places in the design where connections between digital ports and analog terminals or nets exist.

Questa ADMS can insert a boundary element between an electrical node and a digital input or output automatically. You can either write and compile custom boundary elements in a VHDL-AMS or Verilog-AMS model, or you can use pre-defined boundary elements, provided with Questa ADMS as a library model.

A library model to be used as a signal boundary is configured by specifying values for its generic parameters in .MODEL commands in the command file (.cmd). Before simulation starts, you must choose a configured boundary element model from the selection offered using .DEFHOOK commands in the command file. The boundary element selected for a given net must conform in type and mode with the net (A2D, D2A, D2D or BIDIR).

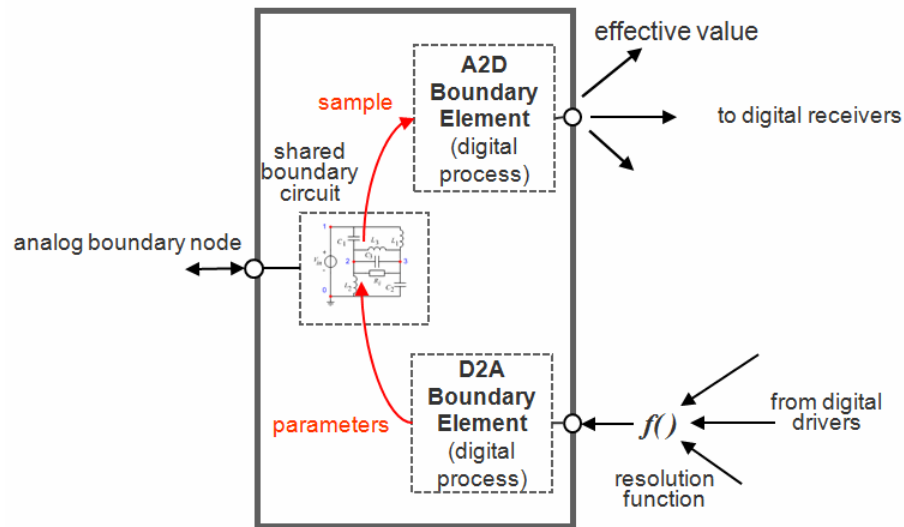
Signal Boundary Model	9-1
What is a Mixed-Signal Net?	9-3
How Mixed-Signal Nets Behave in Questa ADMS	9-4
Unidirectional Mixed-Signal Nets	9-5
Bidirectional Mixed-Signal Nets	9-6
Pure Analog Interconnect	9-7

Signal Boundary Model

A signal boundary model is an analog/mixed-signal model that contains the instructions the simulator uses to calculate the values of a mixed-signal net.

Figure 9-1 depicts a signal boundary model. This is an analog/mixed-signal model that contains the instructions the simulator uses (along with the resolution function) to calculate the values of a mixed-signal net. Its interface exhibits a single electrical port and one or two digital signal ports of identical type. It contains one or two boundary elements, each associated with one of the digital signal ports.

Figure 9-1: Signal Boundary Model



A boundary element is either an A2D boundary element or a D2A boundary element. The boundary model contains a boundary circuit that is shared by the elements, and also digital logic implementing the remainder of the A2D and/or the D2A boundary elements. The boundary node is a node of the boundary circuit that is shared with the analog circuit of the design through the electrical port of the boundary model. The mixed-signal simulator determines the boundary node voltage and the current passing through the boundary node when it solves the system of equations corresponding to the entire design. The boundary circuit may exhibit characteristics such as finite impedance, time delay, rise and fall slope or hysteresis.

The resolution function is either implicit or user-defined and associated with the digital type (or, in VHDL, the subtype) of the net. It combines the values of the digital drivers to yield a single resolved value with which the simulator must drive the digital input port associated with a D2A boundary element. If there is no resolution function associated with the net then there must be only one driver, and the resolved value is the value of the driver. If there are no drivers then the selected boundary model must not contain a D2A boundary element.

A boundary model must know what voltages correspond to digital '1', '0', and 'X'. There are three different kinds of boundary models corresponding to three different ways to supply the voltages.

- Use constant values. The literal values specify the high and low voltages
- Use global SPICE nodes. The voltages on the global spice nodes determine the high and low voltages
- Use UPF. The power specification in the UPF file specifies voltages for each power domain. The UPF voltages for the power domain determine the high and low voltages. See [UPF Calibrated Signal Boundaries](#).

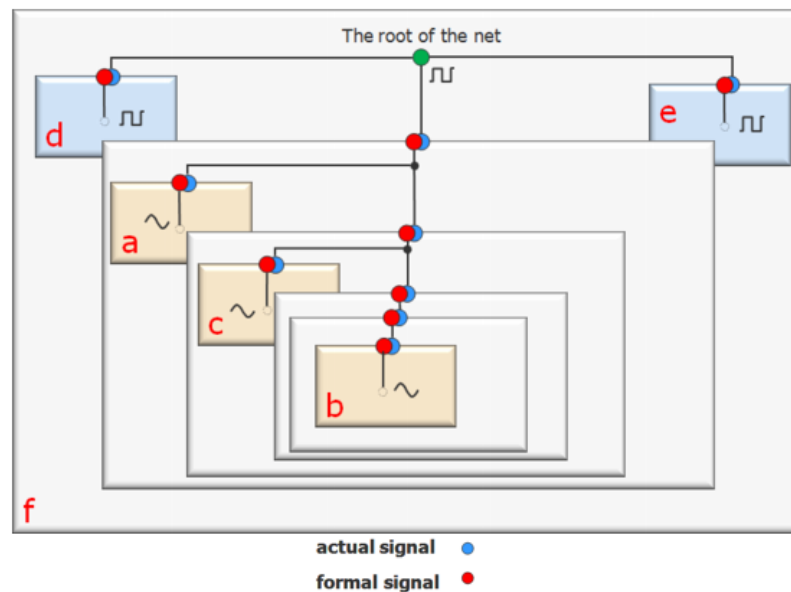
What is a Mixed-Signal Net?

A mixed-signal net contains a mixture of analog and digital interface elements.

A net begins at a locally declared root node and descends through the design hierarchy from instance to instance, through a tree of associations. When a net contains a mixture of analog and digital interface elements, Questa ADMS inserts boundary elements at the places where analog and digital meet. These “cross interfaces” come into existence in two ways:

- For digital-on-top designs, when SPICE attributes are added to a VHDL-AMS component declaration with signal ports.
- Within SPICE, when a net connects an analog pin to a digital pin.

Figure 9-2: A Mixed Signal Net Descending Through a Hierarchy



A net gets its value from its drivers and associated branches. A driver is a digital signal assignment statement. The assignment can be a root signal or to any of the port signals on the net. A branch is a relationship between two nets. It consists of the constitutive equations that define the voltage between the nets and the current flowing between the nets.

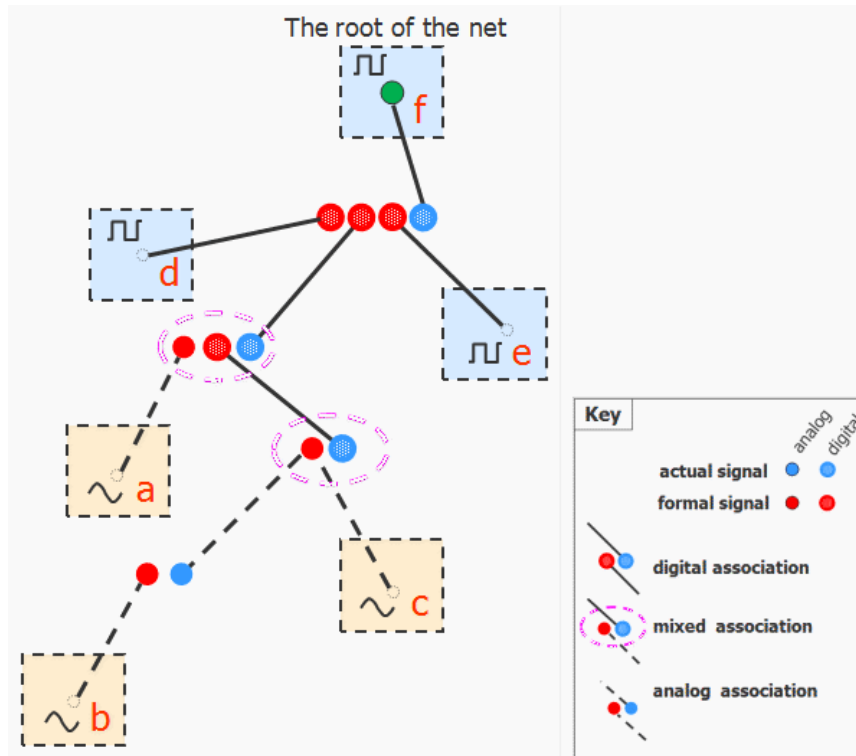
Related Topics

How Mixed-Signal Nets Behave in Questa ADMS Boundary Element Insertion

How Mixed-Signal Nets Behave in Questa ADMS

A mixed association is an association in which either the formal or actual part of the association is an electrical terminal, and in which the other part is a digital signal. A network with a mixed association is a mixed-signal net.

Figure 9-3: A Mixed Signal Net



At elaboration, every mixed-signal net is reorganized so that it has one common analog node, at which all of the electrical terminals or nodes on the net are electrically connected.

A mixed-signal net has a net type and a net mode.

- Net Type

The net type of a mixed signal net is the type of its digital ports.

- Net Mode

The net mode of a mixed signal net is determined by the modes of its mixed associations. Each mixed association has an association mode, A2D, D2A or BIDIR. If they are all of association mode D2A, then the net mode of the mixed-signal net is D2A; if they are all of association mode A2D, then the net mode of the mixed-signal net is A2D; otherwise the net mode of the mixed-signal net is BIDIR.

The signals of **Unidirectional Mixed-Signal Nets** will be automatically connected to the common analog node through a single boundary element—either A2D or D2A. The signals of **Bidirectional Mixed-Signal**

Nets will be connected using either two boundary elements (one D2A and one A2D) or a single bidirectional boundary element (BIDIR).

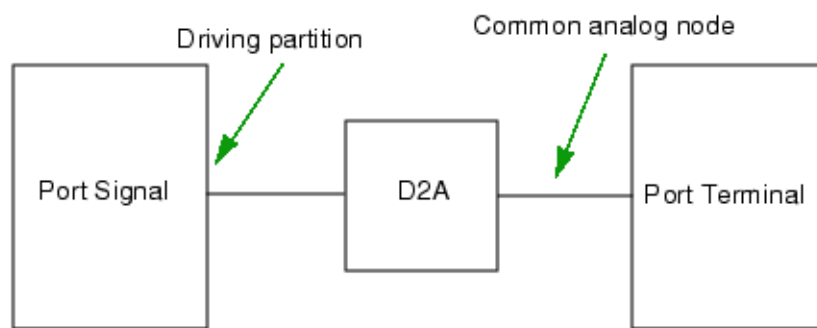
Unidirectional Mixed-Signal Nets

In VHDL or Verilog, assignments to the digital elements of a net are combined with a resolution function to resolve the value of the net. Questa ADMS extends the digital languages by inserting an A2D or D2A boundary element between the common analog node and the digital portion of a unidirectional net.

Unidirectional—Ports Mode IN

If the digital portion of a mixed-signal net is rooted at the top of the model hierarchy and the ports on the net are all of mode IN, the boundary element will be of type D2A.

Figure 9-4: Unidirectional Mixed-Signal Net with All Ports of Mode IN



Whenever a signal assignment targeting a signal on the net is performed, the digital net is updated, then the D2A boundary element responds by adjusting the driving circuit connected to the common analog node to reflect the new digital value. Reading references to signals on the net return the resolved value at the input to the D2A boundary element, not the value of the common analog node.

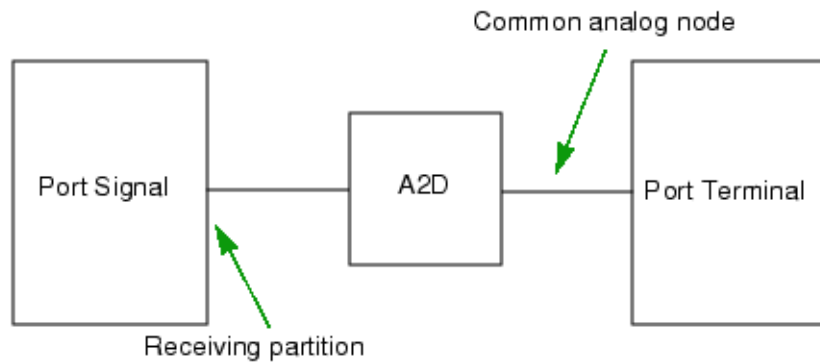
The initial value of the root signal of the net is propagated to the D port during initialization and remains constant throughout simulation. Even though all the analog portions are connected together at the common analog node, a reading reference to a signal on the net will return the initial value of the root signal. This is certainly a paradoxical result, and so the use of IN ports only should be avoided.

Unidirectional—Ports Mode OUT

Suppose that all of the ports are of mode OUT; the boundary element will be an A2D boundary element (see [Figure 9-5](#)).

The signal assignments will be combined with the output of the A2D to resolve the value of the net. Reading references to the signals will see the resolved value. The value of the common analog node contributes to the resolved value, but the resolved value has no effect on the common analog node.

Figure 9-5: Unidirectional Mixed-Signal Net with All Ports of Mode OUT



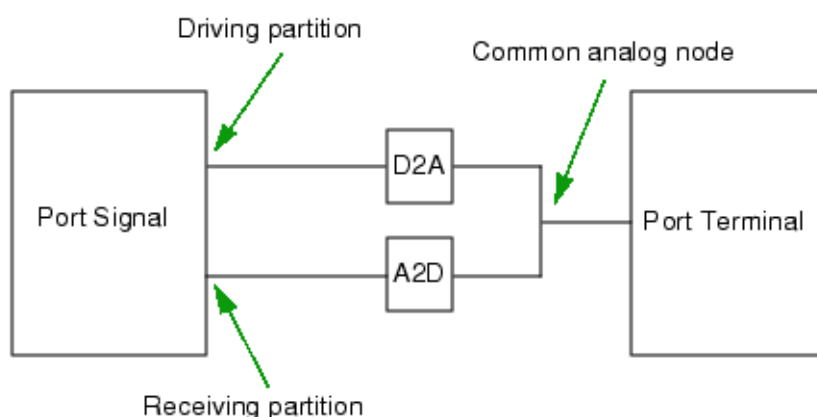
Bidirectional Mixed-Signal Nets

The digital portion of a bidirectional net is divided into two sections: a driving partition and a receiving partition. Assignments to the signals of a bidirectional net are combined by a resolution function to resolve the value of the driving partition.

Reading references to the signals of a bidirectional net see the value of the receiving partition. Questa ADMS inserts a boundary element between the common analog node and each partition, so the driving and receiving partitions of a bidirectional net are connected indirectly through back-to-back boundary elements.

The input to the D2A boundary element is the resolved value of the driving partition. The output of the A2D boundary element supplies the value of the receiving partition (see [Figure 9-6](#)). For example, if you display one of the signals on the net in the viewer, you will see the output of the A2D boundary element. The value of the driving partition is visible only at the input of the D2A boundary element.

Figure 9-6: Bidirectional Mixed-Signal Net



Pure Analog Interconnect

If an element of a mixed-signal net does not appear in explicit assignment statements anywhere in the model hierarchy, then the mixed-signal net is said to be a pure-analog interconnect. As there are no digital readers and no digital drivers, no boundary elements are required or inserted.

The pure interconnect, bidirectional net is the most common case, because it is the natural consequence of replacing digital components by their analog equivalents during system validation. A D2A boundary element will be inserted between the common analog node and the driving partition of the net.

The initial value of the root signal of the net is propagated to the D port of the D2A during initialization and remains constant throughout simulation. The circuit on the output of the D2A loads the common analog node. An A2D boundary element will be inserted between the common analog node and the receiving partition of the net. Reading references to signals on the net will see the output of the A2D. Care is required in defining the behavior of the D2A boundary element for a bidirectional interconnect. By default, the initial value of a VHDL std_logic signal is 'U', the initial value of a Verilog WIRE is 'Z', and the initial value of a Verilog TRI is 'X'. The D2A boundary element must be written to present an appropriate load to the common analog node when the initial value appears at its digital input. Ideally, the D2A should remain disconnected.

Analog Tunneling..... 9-7

Analog Tunneling

Considering a net viewed as a tree descending through the model hierarchy, the way in which all of the analog nodes on the net are connected together may appear like tunneling.

Note

The term "analog tunneling" may be misconstrued because it has nothing to do with the tunnel effect or the crossing of potential barriers.

In Verilog-AMS, “tunneling” is called “analog/digital segregation”.

The net is flattened, all of the digital inputs are gathered together and passed to the resolution function, and all of the analog nodes on the net and the D2A are shorted together. The single digital resolved value is passed through the D2A.

After the analog solution point is located, the A2D converts the analog result to a digital value. That value is provided as the resolved value to all of the digital outputs on the net.

Related Topics

Boundary Element Insertion
What is a Boundary Element?

Built-in or Custom Boundary Elements

You can use built-in boundary elements or custom boundary elements.

You can use two kinds of boundary elements in Questa ADMS:

- Built-in boundary elements, provided with Questa ADMS
Questa ADMS provides built-in A2D and D2A boundary elements, written in SPICE.
Built-in boundary elements are already defined, therefore, you need only to supply a name, to select the type (A2D or D2A) and provide a list of optional parameters, to configure the boundary element to match the electrical characteristics of your circuit.
For instructions on how to declare and insert built-in boundary elements, see [Boundary Element Insertion](#).
- Custom boundary elements
You can write custom boundary elements in VHDL-AMS or Verilog-AMS, compile them into an existing library, then declare and insert them in the same way as built-in boundary elements, see [Boundary Element Insertion](#).
You can also write custom boundary elements in Verilog-AMS and insert them using connectrules. See [Inserting Boundary Elements Using Connect Rules](#).
A typical boundary element contains a voltage source, a serial resistor (which can be null) and a capacitance in parallel (which can also be null). For typical cases, it is recommended that you use a SPICE built-in boundary element, because they are optimized. A custom boundary element will be interpreted as if it were any other kind of mixed-signal model.

Writing Custom VHDL-AMS Boundary Elements.....9-9

Writing Custom VHDL-AMS Boundary Elements

For A2D and D2A boundary elements, the VHDL-AMS model header must have a single port signal and a single port terminal. For bidirectional boundary elements, the VHDL-AMS model header must have a port signal of direction IN, a port signal of direction OUT or INOUT, and a single port terminal.

For A2D and D2A boundary elements, the VHDL-AMS model header must have:

- A single port signal:
 - IN for the input of a D2A boundary element
 - OUT or INOUT for the output of an A2D boundary element
- A single port terminal.

For bidirectional boundary elements, the VHDL-AMS model header must have three ports:

- A port signal of direction IN, which is considered as the input of the D2A portion of the bidirectional boundary element
- A port signal of direction OUT or INOUT, which is considered as the output of the A2D portion of the bidirectional boundary element
- A single port terminal

Example

The following is a VHDL-AMS D2A boundary element of type std_logic:

```

Library IEEE;
Use IEEE.std_logic_1164.all;
USE IEEE.electrical_systems.ALL;

ENTITY d2a_electrical IS
  GENERIC (vhi  : Real := 5.0;
           vlo   : Real := 0.0;
           trise : Real := 2.0e-9;
           tfall : Real := 2.0e-9);
  PORT (SIGNAL  inp  : IN std_logic;
        TERMINAL outp : Electrical);
END ENTITY d2a_electrical;

ARCHITECTURE a_d_m_s OF d2a_electrical IS
  SIGNAL vin : Real := vlo;
  QUANTITY vout ACROSS iout THROUGH outp;
BEGIN
  vin <= vlo WHEN inp = '0' ELSE vhi after 3ns;
  vout == vin'Ramp(trise, tfall);
END ARCHITECTURE a_d_m_s;

```

Related Topics

Boundary Element Insertion
What is a Boundary Element?

Boundary Element Insertion

This section describes how to manage boundary element insertion in Questa ADMS.

Inserting Boundary Elements..... 9-10

Controlling Boundary Element Assignment..... 9-11

Inserting Boundary Elements Using Connect Rules..... 9-15

Consideration of Continuous Disciplines..... 9-20

Cross-Domain Hierarchical References..... 9-22

Built-in A2D and D2A Boundary Elements..... 9-23

Real A2D Converter Tolerance..... 9-24

Special Supply Boundary Elements..... 9-25

Logical States with Built-In D2A Boundary Elements..... 9-26

Digital-to-Digital (Logic-Real) Boundary Elements..... 9-28

Inserting Boundary Elements

Boundary elements are inserted by declaring which type of boundary element to use then making the assignment for your boundary element within your design.

Restrictions and Limitations

- Implicit insertion of boundary elements using .DEFHOOK commands between terminals or signals and quantities in VHDL-AMS interface declarations are not supported. If boundary elements are required between port quantities and other objects, components converting such object values into a quantity (or reverse) must be declared manually in the description.

Procedure

1. Declare the boundary element using the .MODEL command:
 - To declare built-in (pre-defined) boundary elements, use:

```
.MODEL boundary_model_name A2D|D2A|D2D mode=type {parameter=value}  
+ [ANALOG=continuous_discipline]  
+ [break=fast|accurate]
```

- To declare user-defined boundary elements, use

```
.MODEL boundary_model hook [library_name:]design_entity
{parameter=value}
```

Note

You should place .MODEL and .DEFHOOK commands in a .cmd file for digital-on-top designs, or in a SPICE netlist for SPICE-on-top designs.

2. Assign the boundary element, using the .DEFHOOK command (can be placed inside or outside a subcircuit):

```
.DEFHOOK [scope] boundary_model {boundary_model}
```

Where scope can be:

INST=instance_name|MOD=model_name|NET=net_name PORT=port_name|GROUP=group_name|

Alternatively, you can use Verilog-AMS connect rules to perform boundary element assignment. See [Inserting Boundary Elements Using Connect Rules](#).

Results

When boundary elements are inserted, the net is separated into its analog and digital parts, but the net name does not change.

To avoid inserting boundary elements unnecessarily, D2A boundary elements are not inserted if there is no driver on the digital side of a D2A boundary.

A log file records the boundary elements that have been assigned; see [Boundary Elements Log File](#).

Controlling Boundary Element Assignment

You can restrict the application of boundary elements to specific parts of the design.

The parameters to the [.DEFHOOK](#) command enable you to restrict the application of A2D or D2A boundary elements to specific parts of the design. Additionally, the parameter

```
dtype = <library>:<package>.<floating_point_type>
```

allows the use of A2D or D2A boundary elements to be constrained to a particular user-defined floating-point type.

Questa ADMS respects a priority rule in the case where different boundary elements are found to be suitable for a given boundary, based on all of the .DEFHOOK statements that were specified. The priority levels are as follows, from highest to lowest:

1. **.DEFHOOK PDOMAIN=**

Use the PDOMAIN parameter to assign boundary elements for a particular power domain. This parameter is used in conjunction with the **.PDOMAIN** command.

2. **.DEFHOOK PORT=**

Use the PORT parameter to assign boundary elements for an identifiable port.

3. **.DEFHOOK NET=**

Use the NET parameter to assign boundary elements for an identifiable net. A single net may have many boundaries.

4. **.DEFHOOK INST=**

Use the INST parameter to assign boundary elements for a specified instance.

5. **.DEFHOOK MOD=**

Use the MOD argument to assign boundary elements for all instances of one model from a specified library.

6. **.DEFHOOK GROUP=**

Use the GROUP parameter to assign boundary elements for a previously defined group. This parameter is used in conjunction with the **.GROUP** command.

7. **.DEFHOOK**

Use the .DEFHOOK command without any parameters, other than the names of the boundary elements, to assign boundary elements to all A2D or D2A boundaries in the design.

Refer to **Boundary Element Commands** in the *Questa ADMS Command Reference* for detailed syntax descriptions of these .DEFHOOK expressions, and for their priorities when the parameter values contain wildcards.

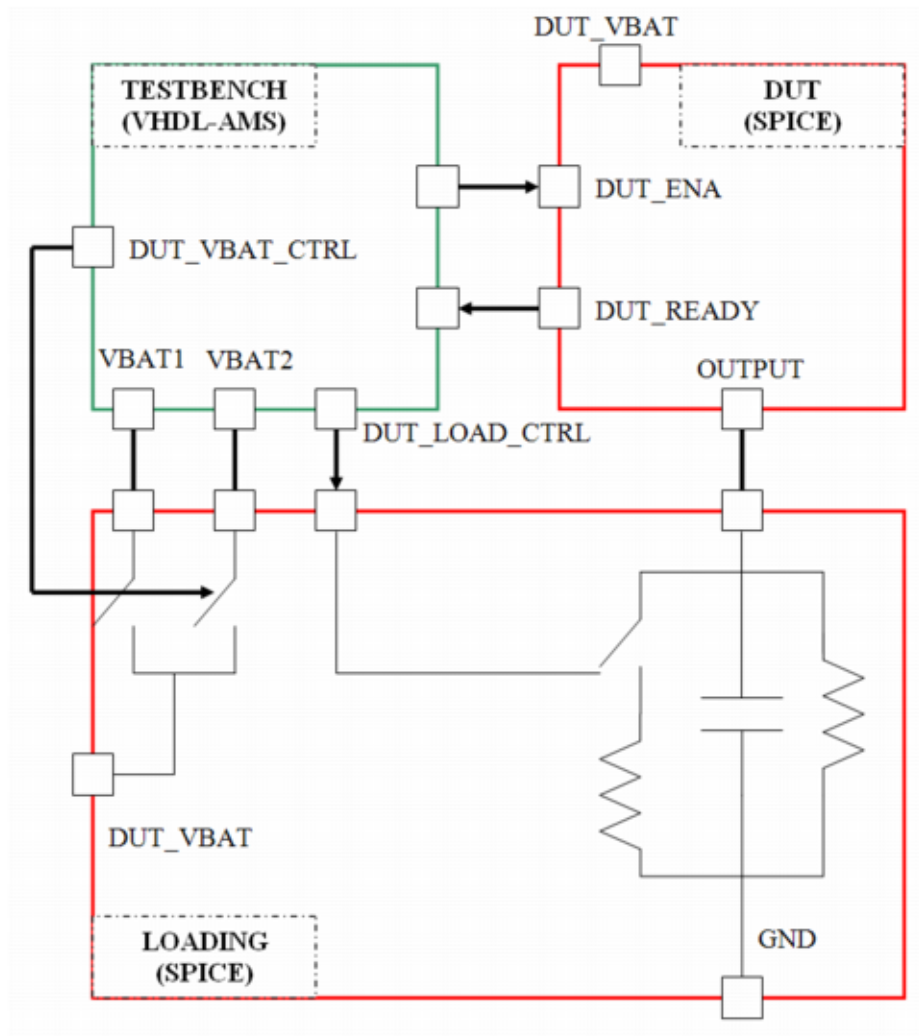
Example

The design shown in **Figure 9-7** contains an analog block (DUT) being simulated in a SPICE-on-top environment. A VHDL-AMS instance TESTBENCH is used to generate the controls for DUT (both analog and digital) and also a third block in SPICE providing the DUT's external components (load control and voltage supply).

- The TESTBENCH provides three control signals for the DUT: enable, switch control to select the voltage supply (either VBAT1 or VBAT2) and switch control to select the value of the output load resistor (either only one resistor or two resistors in parallel).
- The DUT generates a fixed output voltage as soon as it is enabled, and also a flag indicating that the output reached the expected value. This flag is fed back to the TESTBENCH for further control of the state machine.

- The LOADING switches between VBAT1 and VBAT2 to supply DUT, and also provides its loading components (tank capacitor and resistor).

Figure 9-7: Converters Example Design



All parts of the circuit apart from the control signals from the TESTBENCH are analog, which results in boundaries at the following locations:

- Mixed-signal net **DUT_ENA** (from TESTBENCH to DUT). In this case the digital description is driving—a D2A converter is required.
- Mixed-signal net **DUT_READY** (from DUT to TESTBENCH). In this case the digital description is receiving—an A2D converter is required.
- Mixed-signal net **DUT_VBAT_CTRL** (from TESTBENCH to LOADING). In this case the digital description is driving—a D2A converter is required.
- Mixed-signal net **DUT_LOAD_CTRL** (from TESTBENCH to LOADING). In this case the digital description is driving—a D2A converter is required.

All of the mixed-signal nets are unidirectional, hence only unidirectional converters are inserted.

Global Boundary Element Assignment Using .DEFHOOK

Assume that only global converters are required for the example, meaning that all conversions on the design can be done using the same parameters.

1. Two boundary elements are defined, one for analog-to-digital and one for digital-to-analog converters, each one with specific parameters.

```
.MODEL A2D_default A2D MODE=std_logic VTH1=0.6 VTH2=0.8 TX=10us
.MODEL D2A_default D2A MODE=std_logic
+ VLO=0.0 VHI=4.8 TRISE=10e-9 TFALL=10e-9
```

2. Boundary element assignment is done using the .DEFHOOK command without any parameters other than the name of the two groups of converters:

```
.DEFHOOK A2D_default D2A_default
```

Wherever boundary elements are required in the design at A2D or D2 A boundaries, one of these two boundary elements will be inserted.

This type of .DEFHOOK statement has the lowest priority of all. This means that it will be used anywhere in the circuit unless more specific converter assignments are specified. It is recommended to set the A2D threshold to the lowest possible value and setting the D2A output voltage to the greatest value possible.

Note that when a design requires boundary elements and no .DEFHOOK commands are specified, Questa ADMS will report an error at elaboration.

Specific Net Assignment Using .DEFHOOK NET=

Now assume that all of the switches inside the LOADING instance require a high voltage in order to be controlled (for example, 20 V). The rest of the circuit operates under a lower voltage range (0 V to 4.8 V) so there is no need to create default converters with such important values. In this case, a specific pair of converters can be created for use when converters are required around the LOADING instance. On other instances, the global boundary elements are still used.

```
.MODEL A2D_LOADING A2D MODE=std_logic VTH1=9.9 VTH2=10.1 TX=10us
.MODEL D2A_LOADING D2A MODE=std_logic
+ VLO=0.0 VHI=20.0 TRISE=50e-9 TFALL=50e-9
```


If only a few nets are likely to require these boundary elements, the .DEFHOOK NET= command can be used to manually insert boundary elements for each of them:

```
.DEFHOOK NET=DUT_VBAT_CTRL A2D_LOADING D2A_LOADING
.DEFHOOK NET=DUT_LOAD_CTRL A2D_LOADING D2A_LOADING
```

For this command, the net given must be compliant with Eldo net syntax, and must be the name of the root of the net.

Related Topics

- Inserting Boundary Elements Using Connect Rules
- Boundary Element Commands [Questa ADMS Command Reference]

Inserting Boundary Elements Using Connect Rules

Connect modules may be used in Verilog-AMS to insert boundary elements in your design to resolve mixed-signal interaction between modules. These modules may be manually inserted (by you) or automatically inserted (by the simulator) based on rules you provide.

The disciplines of mixed-nets are determined prior to the connect module insertion phase of elaboration. Connect module declarations with matching port discipline declarations and directions are instantiated to connect the continuous and discrete domains of the mixed net.

An instance of a connect module is automatically inserted when signals and ports with continuous (analog) and discrete (digital) disciplines are connected together. A connect module has one port of a continuous discipline and one of a discrete discipline. The continuous and discrete disciplines of the ports of the connect modules and their directions are used to determine the circumstances in which the module can be automatically inserted.

Note

Questa ADMS supports only one Verilog-AMS discrete discipline: the logic discipline (see also [Overriding the Default Disciplines of Built-in Boundary Elements](#)). In mixed-language design, the VHDL types real, integer, and std_logic from the package STD.STANDARD, Verilog-AMS wreal types and any user-defined enumeration types are treated as discrete disciplines for VHDL signals.

Refer to [Consideration of Continuous Disciplines](#) for details on how to define continuous disciplines.

Specifying Connect Rules..... 9-16

Connect Module Statements..... 9-16

Connect Specification Statements.....9-18

Specifying Connect Rules

A command file command or a command line option is needed if connect specifications have been defined using connectrules.

When connect specifications have been defined using connectrules for use in your design, you must specify which to use by one of the following methods:

- In the *.cmd* file use the *.CONNECTRULES* command:

```
.connectrules [<library_name>.<connectrules_name>
{[<library_name>.<connectrules_name>}
```

- On the command line, use the *vasim -connectrules* option:

```
vasim ... -connectrules [<library_name>.<connectrules_name> ...
{-connectrules [<library_name>.<connectrules_name>} ...
```

Note

The set of connectrules to be used need to be specified using one of these methods, otherwise no connectrules will be used.

Connect Module Statements

A connect statement specifies a connect module, its attributes, and additional selection criteria. The connect module defined in a connect statement will be used for mixed-signal connections that match the specified criteria.

The criteria takes into account the port direction and the disciplines of the signals connected to the port.

Connect Module Syntax

```
connectmodule_declaration ::=
    connectmodule module_identifier ( connectmod_port , connectmod_port ) ;
    [ module_items ]
    endmodule
connectmod_port ::=
    connectmod_port_identifier
```

Note

If two connect statements have identical selection criteria, an error will be reported.

The port disciplines define the default type of disciplines that shall be bridged by the connect module. The directional qualifiers of the discrete port determine the default scenarios where the module can be instantiated. The following combinations of directional qualifiers are supported for the continuous and discrete disciplines of a connect module:

Table 9-1: Directional Qualifiers

Continuous	Discrete
input	output
output	input
inout	inout

Examples

The following example can bridge:

- A mixed *input* port whose upper connection is compatible with discipline logic and whose lower connection is compatible with electrical, or
- A mixed *output* port whose upper connection is compatible with discipline electrical and whose lower connection is compatible with logic.

```
connectmodule d2a(in, out);
    input in;
    output out;
    logic in;
    electrical out;
    // insert connect module behavioral here
endmodule
```

The following example can bridge:

- A mixed *output* port whose upper connection is compatible with discipline logic and whose lower connection is compatible with electrical, or
- A mixed *input* port whose upper connection is compatible with discipline electrical and whose lower connection is compatible with logic.

```
connectmodule a2d(out, in);
    output out;
    input in;
    logic out;
    electrical in;
    // insert connect module behavioral here
endmodule
```

The following example can bridge any mixed port whose one connection is compatible with discipline logic and whose connection is compatible with electrical:

```
connectmodule bidir (out, in);
    inout out;
    inout in;
    logic out;
    electrical in;
    // insert connect module behavioral here
endmodule
```

Any number of connect modules can be defined. You can specify which are used in a design, using connect specification statements.

Connect Specification Statements

You can use connect specification statements to bridge given discrete and analog disciplines, override the connect module default disciplines and port directions, and resolve incompatible disciplines.

Verilog-AMS connect specification statements allow you to:

- Specify which connect module is used (including parameterization) to bridge given discrete and analog disciplines.
- Override the connect module default disciplines and port directions.
- Resolve incompatible disciplines (including VHDL-AMS natures which are treated as if they were names of Verilog-AMS disciplines).

In order to use SPICE built-in boundary elements between analog and logic/real, specify the connect modules \$A2D and \$D2A. In this case, the default SPICE built-in boundary elements are those of MODE=STD_LOGIC. However, all available built-in boundary elements can be specified using the MODE parameter value. The connect module auto-insertion statement enables you to specify the continuous discipline for which the connectmodule or boundary element is applicable.

Connect Specification Syntax

```
connectrules_declaration ::=
    connectrules connectrules_identifier;
    {connectrules_item}
endconnectrules

connectrules_item ::=
    connect_insertion | connect_resolution

connect_insertion ::=
    connect connectmodule_specification [connect_mode]
    [parameter_value_assignment] [connect_port_overrides];
```

```
connectmodule_specification ::=
    connectmodule_identifier | $a2d | $d2a |
    \[library_name.]entity_name[(architecture_name)]

connect_mode ::=
    merged | split
```

For the insertion of boundary elements under connect rules, the Verilog-AMS predefined discipline “electrical” is treated as identical to the VHDL-AMS nature IEEE.electrical_systems.electrical, and the Verilog-AMS predefined discipline “logic” as identical to the VHDL-AMS predefined type IEEE.Std_logic_1164.std_logic.

Examples: Using VHDL-AMS Connect Modules in Verilog-AMS

- In this example, the discipline and direction are supplied explicitly:

```
connect $a2d #( .vth1(2.5), .vth2(2.5), .vth(2.5))
input electrical, output logic;
```

- Suppose that a VHDL-AMS D2A boundary element has one port of type std_logic and one of nature electrical. The entity name is vamsd2a and the architecture name is v1_5. Because the discipline and direction are contained in the entity declaration, they need not be repeated in the connect rule:

```
connect \work.vamsd2a(v1_5) #(.vhi(5), .vlo(0), .trise(2n), .tfall(3n));
```

Because the name of the VHDL design entity takes the form of a Verilog extended name, the preceding “\” and the terminating white space are both required.

Next, assume that the architecture v1_5 is the only architecture of vamsd2a. Then this declaration is identical, but takes advantage of VHDL defaults:

```
connect \work.vamsd2a #(.vhi(5), .vlo(0), .trise(2n), .tfall(3n));
```

Finally, assume vamsd2a is in the working library. The declaration is:

```
connect vamsd2a #(.vhi(5), .vlo(0), .trise(2n), .tfall(3n));
```

The “\” is no longer required.

- In this example, two connect specifications are defined, that apply to a model containing ports of two user-defined disciplines, both compatible with electrical or voltage:

```
connect $a2d #(.vth1(2.0), .vth2(3.0)) input electrical_high, output
logic;
```

```
connect $a2d #(.vth1(1.1), .vth2(1.3)) input electrical_low, output
logic;
```

Related Topics

Consideration of Continuous Disciplines

Consideration of Continuous Disciplines

A mixed-signal net has a digital type or discrete discipline, and an analog continuous discipline associated with it. Upon elaboration, Questa ADMS will report any conflicting continuous disciplines in the design if no resolution exists.

A continuous discipline is either a Verilog-AMS continuous discipline or a VHDL-AMS nature. Within a design, you may choose to insert different boundary elements for connections with different continuous disciplines.

VHDL-AMS requires all natures on a net to be identical. In Verilog-AMS, the continuous discipline of a net must be resolved using the “resolveto” statement of the connectrules module if different continuous disciplines are used on the network.

Continuous disciplines can be attributed to ports of SPICE models, subcircuits or primitives using vasp -default_discipline or vamatch commands. When instantiating SPICE within Verilog-AMS, or for direct SPICE instantiation, the port_discipline attribute is used to associate disciplines to the SPICE port according to the syntax described in the *Verilog-AMS Language Reference Manual 2.4*, section E.3.2.1.

Specifying Continuous Disciplines.....	9-20
Overriding the Default Disciplines of Built-in Boundary Elements.....	9-22

Specifying Continuous Disciplines

For mixed-signal nets, you can specify a continuous discipline.

You can specify a continuous discipline for mixed-signal nets, as part of the boundary element declaration on the .MODEL statement, using the ANALOG=<continuous_discipline> argument:

Built-in Boundary Elements

```
.MODEL boundary_element_name A2D|D2A
+ MODE=BIT|X01Z|STD_VSRC|MVL4|STD_LOGIC|REAL|INTEGER
+ [ANALOG=continuous_discipline]
+ {parameter=value}
+ [BREAK=FAST|ACCURATE]
```

Where <continuous_discipline> is specified in VHDL-AMS as:

```
[logic_library.VHDL-AMS_package.]VHDL-AMS_nature
```

or in Verilog-AMS as:

```
Verilog-AMS_discipline /* Remark: simple identifier */
```

The set of continuous disciplines available for this operation is limited to those compiled and registered in libraries.

VHDL-AMS natures with no library and package names specified will be considered as IEEE natures.

User-Defined Boundary Elements

For user-defined boundary elements (VHDL-AMS design units or Verilog-AMS modules or connectmodules) this information is available in the boundary element definition:

```
.MODEL boundary_element_name HOOK [logic_library:]design_unit  
{parameter=value}
```

Where the design_unit can be:

- Verilog-AMS module name
- Verilog-AMS connectmodule name
- VHDL-AMS entity name [(VHDL-AMS architecture)]
- VHDL-AMS configuration name

Continuous Disciplines on SPICE Nets

By default, SPICE nets are considered empty of disciplines, and are compatible with all other disciplines. However, you can attribute specific continuous disciplines to ports of a SPICE subcircuit using:

```
vaspi -default_discipline continuous_discipline
```

In the case that the SPICE subcircuit was instantiated using vaspi, and a continuous discipline was already associated, the instantiation port_discipline attribute will override the port_discipline previously associated.

Overriding the Default Disciplines of Built-in Boundary Elements

By default, the built-in a2d and d2a boundary elements have the discrete discipline logic and continuous discipline electrical.

In the absence of any explicit connect port override, it is assumed that the two disciplines are logic and electrical.

For an electrical to logic connection, you may use the form that does not mention disciplines:

```
connect $a2d #(.VTH(2.5));
connect $d2a #(.trise(0.1e-9), .tfall(0.1e-9));
```

For another continuous discipline (for example, lowvolt) use the form that mentions both disciplines, with logic for the discrete discipline, and including the mode designations:

```
connect $a2d #(.VTH(1.3)) input lowvolt output logic;
connect $d2a #(.trise(0.1e-9), .tfall(0.1e-9), .vhi(1.3), .vlo(0.2)) output
lowvolt input logic;
```

Connect rules only have to be compiled after at least one module using the disciplines specified has been compiled. You may recompile the Verilog-AMS module, but disciplines must not have been modified. The connect rules do not have to be recompiled.

Related Topics

[Inserting Boundary Elements Using Connect Rules](#)

[Boundary Element Commands \[Questa ADMS Command Reference\]](#)

[Controlling Boundary Element Assignment](#)

[Cross-Domain Hierarchical References](#)

Cross-Domain Hierarchical References

When a hierarchical reference from a digital context refers to an electrical object, a boundary element of type logic (bit-to-electrical or electrical-to-bit) is inserted by default.

To handle wreal type or real variables in Verilog and SystemVerilog, you must read or write an analog hierarchical reference as a real net (real-to-electrical or electrical-to-real).

Because in Verilog and SystemVerilog, literal values do not represent an object's type (any value can be cast into the right one when needed) the digital port of the inserted boundary element cannot be defined by the context(s) of the hierarchical reference. The following rules are used to determine which kind of boundary element to insert:

- If there is only one kind of boundary element associated with the given net according to the .DEFHOOK rules (either logic or real) Questa ADMS uses it, and the corresponding type on the digital side is used.
- If both logic and real boundary elements are provided for the given net, the href= parameter must be specified on the .DEFHOOK command in order to select the correct boundary element:

```
.DEFHOOK [level] boundary_model_name {boundary_model_name}
[href = logic|real]
```

- If no boundary element is provided for a given net, an error will be reported.

Examples

- In this example, the href= parameter is used to specify that boundary elements of type real are to be inserted:

```
.DEFHOOK a2d_logic d2a_logic a2d_real d2a_real href=real
```

- In this example, when a net needs a boundary element for a hierarchical reference, because the boundary element to insert is following the rules provided by m, a hierarchical reference on this net will automatically insert a logic boundary element (this is the default behavior when href= is not specified):

```
.DEFHOOK model=m a2d_logic d2a_logic a2d_real d2a_real
```

Related Topics

[Boundary Element Commands \[Questa ADMS Command Reference\]](#)
[Controlling Boundary Element Assignment](#)

Built-in A2D and D2A Boundary Elements

You can use the predefined Eldo A2D and D2A boundary elements as bi-directional boundary elements, not only as uni-directional boundary elements.

Normally, Eldo boundary elements of type std_logic do not generate "X" values at DC. This is because these values are fed back into the analog part, and they could generate convergence problems. However, some models need to have an "X" value generated at DC from analog blocks. In these cases you can specify for this:

- By including the .OPTION LINKX command in the command (.cmd) file:

```
.OPTION LINKX
```

- By including the `-linkx` option when launching the simulator:

```
vasim -linkx
```

Related Topics

Controlling Boundary Element Assignment

Real A2D Converter Tolerance

The digital value of an A2D boundary element of type REAL (MODE=REAL) will be updated at time 0, during every DC computation quiescent phase, if the analog input changes by a certain amount, or if the direction of the input net voltage value changes by a certain amount.

When a built-in A2D boundary element of type REAL (MODE=REAL) is declared, the digital value will be updated under the following conditions:

- At time 0
- At each DC computation during quiescent phase
- If the analog input changes by an amount greater than EPS, plus or minus the value of `expr_tol` relative to the previous A2D change, and the time elapsed since that change is greater than `time_tol` (Figure 9-8).
- If the direction of the input net voltage value changes by an amount greater than `expr_tol` (Figure 9-9).

Figure 9-8: Controlling Analog Input Value Tolerance for an A2D of Mode REAL

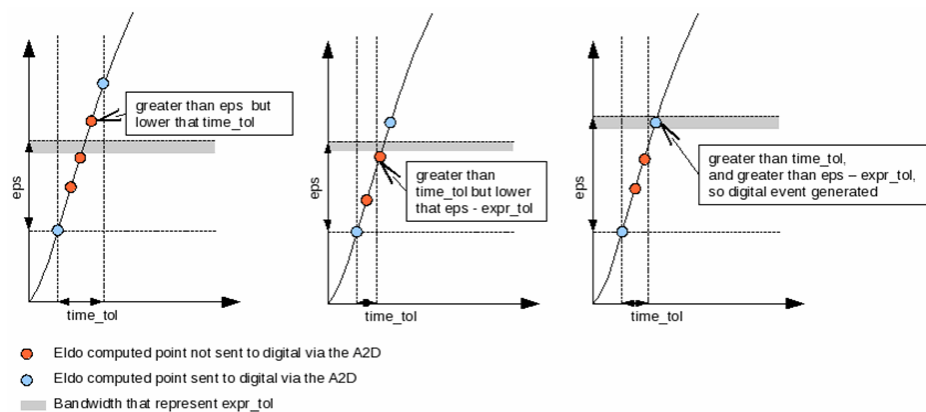
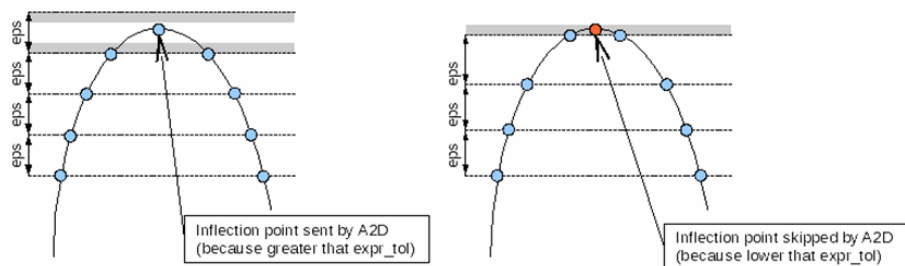


Figure 9-9: Controlling Input Net Direction Tolerance for an A2D of Mode REAL



Related Topics

Controlling Boundary Element Assignment

Special Supply Boundary Elements

When a SPICE subcircuit is powered by a digital power supply, a special type of digital-to-analog boundary element is required. The boundary element definition and assignment should be included in either the top-level SPICE netlist or the Questa ADMS command file (.cmd).

The syntax used to define the boundary element is:

```
.MODEL user_defined_model_name D2A mode=std_supply vhi=val vlo=val
```

The **.DEFHOOK** command can then be used to insert the boundary element onto the appropriate supply net.

Example

The following statement defines a boundary element called d2a_POWER of type STD_SUPPLY:

```
.MODEL d2a_POWER d2a mode=STD_SUPPLY vhi=1.8 vlo=0.0
```

The d2a_POWER boundary element is assigned to the vdd_pll and vss_pll nets using .DEFHOOK:

```
.DEFHOOK net=vdd_pll d2a_POWER
.DEFHOOK net=vss_pll d2a_POWER
```

Related Topics

Boundary Element Commands [Questa ADMS Command Reference]

Controlling Boundary Element Assignment

Logical States with Built-In D2A Boundary Elements

When the mode of a boundary element is STD_LOGIC (MVL9) or X01Z (RC) these modes emulate an IRC (or IGC) boundary element in Eldo. The boundary element consists of a current source in parallel with a conductance in parallel with a capacitor, with everything connected between the current node and the ground node.

Several IRC boundary elements can be placed on one node, and depending on the strength of the signals applied on each of them, the conflicts can be resolved with one of the modes. The values of this IGC vary depending on the state of the D2A boundary element. These different values are computed according to the values specified in the D2A statement; if no values are specified, the default values are used.

X State

For a logical state X, the values for conductance (GX), capacitance (CX), transition time (TX), and current source (IX), using the default values, are shown below with the formulae:

$$GX = \frac{RRISE^{-1} + RFALL^{-1}}{2} = 0.5S$$

$$CX = \frac{LOWCAP + HIGHCAP}{2} = 0\mu F$$

$$TX = \frac{TRISE + TFALL}{2} = 2ns$$

$$IX = \frac{(VHI + VLO)}{2} \times GX = 1.25A$$

When a logical state X is present on the digital port of the Eldo built-in D2A boundary element, using the default values, the voltage across the analog port will be 2.5 V.

W (Weak X) State

For a logical state W, which is also referred to as "Weak X", the values for conductance (GX), capacitance (CX), transition time (TX), and current source (IX), using the default values, are shown below with the formulae:

$$GX = \frac{WEAHIGHRES^{-1} + WEALOWRES^{-1}}{2} = 0.5S$$

$$CX = \frac{LOWCAP + HIGHCAP}{2} = 0\mu F$$

$$TX = \frac{TRISE + TFALL}{2} = 2ns$$

$$IX = \frac{(VHI + VLO)}{2} \times GX = 1.25A$$

Note

The capacitance (CX) and transition time (TX) are the same for both X and W states.

When a W (Weak X) state is present on the digital port of the Eldo built-in D2A boundary element, using the default values, the voltage across the analog port will be 2.5 V.

VHDL "U" State

The analog part on D2A outputs when the input signal is 'U' is identical to X input. This analog value will be:

- The previous value if the xeval parameter on the **.MODEL D2A** command is set to 'previous'.
- <value> if the xeval parameter on the **.MODEL D2A** command is set to <value>.
- $(v_{hi}+v_{lo})/2$ in other cases.

Z State

When a logical state Z is present on the digital port, the output resistance and capacitance will take the value of RZ and ZCAP which can be specified in the **.MODEL** command. If the parameters are not specified, the default values are used. Eldo will compute the analog voltage depending on the load connected across the analog node.

Z State Detection on A2D Nodes

To check the HiZ state on A2D nodes, you can specify the RZ threshold on the model card of the converter. This value takes priority over any other high impedance threshold specified on the **.HIZ/.HIZ_CFG** commands.

You can also check the node impedance of A2D converters by specifying the TYPE=A2D parameter on the **.HIZ**, **.DCHIZ** or **.HIZ_CFG** command and define a default threshold for those nodes (the RZ parameter on the model card takes priority).

By default, if a Z state arises, it is not sent to the digital part of the circuit. To activate Z state detection for all A2D nodes, specify the SEND_TO_DIGITAL parameter on the **.HIZ** or **.DCHIZ** command.

The value used to check the HiZ state is as follows (by order of priority):

- RZ on the model card if specified.
- The high impedance value (**.HIZ**, **.DCHIZ** or **.HIZ_CFG** command R parameter) if the node is also selected by the TYPE= parameter on the **.HIZ**, **.DCHIZ** or **.HIZ_CFG** command.
- 1e7 if no default value is specified.

V Source Boundary Element for Std_logic Nets

When interfacing a Verilog or VHDL Std_logic signal with analog nets, it is possible to manage conflicts of a Z state with a V source built-in boundary element of type FAST_STD_LOGIC. FAST_STD_LOGIC is a mixture of type BIT and pure STD_LOGIC; this boundary element can only handle 0, 1, X and Z states. Using this

type of boundary element can reduce the simulation time, because the analog nodes connected to the D2A will not be solved. The FAST_STD_LOGIC boundary element can be applied to A2D boundaries, in which case the boundary element is equivalent to a boundary element of type STD_LOGIC.

This type of boundary element can be used at any digital-analog connection, just like their slower counterparts. They provide a less physically detailed model of the D2A connection in exchange for a great improvement in performance.

A FAST_STD_LOGIC boundary element sends digital events to the analog simulator as PWL voltage sources. The PWL values are computed according to the characteristics of the STD_LOGIC boundary element.

A restriction exists: The analog load is not taken into account as is the case with Std_logic D2A boundary elements (note that, by default, the Std_logic boundary elements have an internal resistor value of 1ohm, so with such a value, for most of the circuits, the load has a low effect).

The V source built-in boundary element can be selected by setting the MODE of the built-in boundary element to FAST_STD_LOGIC or STD_VSRC (they are synonymous). It has the same set of parameters as the Eldo built-in boundary element of MODE=BIT.

Eldo checks whether there are one or more D2A boundary elements present that are not in a high-impedance (Z) state. If there are, Eldo will take into account all of the D2A signals on the node and will compute the resulting voltage value. This value will be imposed on the analog part as a voltage source would, which means that the voltage on that node is not an *unknown* to the system of equations, and this can make simulation faster (especially for Questa ADMS-ADiT). However, if there are only high-impedance states on the D2A node, Eldo will solve for that node.

Related Topics

[Boundary Element Commands \[Questa ADMS Command Reference\]](#)

Digital-to-Digital (Logic-Real) Boundary Elements

You can use digital-to-digital (D2D) built-in bidirectional boundary elements to connect ports of type real or wreal, to ports of type logic.

The .MODEL card for a D2D boundary element is different from A2D and D2A . Once the .MODEL has been declared, the normal mechanisms for [.DEFHOOK](#) insertion work as specified.

Digital-to-digital boundary elements are declared and inserted in the same way as A2D and D2A boundary elements, using the [.MODEL](#) and commands.

WREAL is the Verilog-AMS wreal, and REAL is the VHDL or Verilog real.

The real-valued side of a D2D can be either of the types wreal or real, indicated by the attribute parameter MODE=WREAL (the default) or MODE=REAL.

D2D Boundary Elements of Type MODE=WREAL.....	9-29
vasim -d2dmode.....	9-30
D2D Boundary Elements of Type MODE=REAL.....	9-35

D2D Boundary Elements of Type MODE=WREAL

You can use digital-to-digital (D2D) boundary elements in a network that mixes wreal and logic types.

For example, you can use D2D boundary elements if the contains cells that are configured with Verilog digital modules that may have logic or wreal ports. At some points “wire” interconnect is used to bridge connections between wreal and logic ports.

Use the following syntax to declare a D2D boundary element to connect wreal ports to logic ports:

```
.MODEL <model_name> D2D [mode=wreal] [vth1=<value>] [vth2=<value>]
[vlo=<value>] [vhi=<value>]
```

vth1 and vth2 are voltages representing the maximum '0' and the minimum '1' values respectively, when converting from real to bit. Any real value between vth1 and vth2 converts to 'X'.

vlo and vhi are voltages representing the converted values of 0 and 1, respectively, when converting from bit to real.

During D2D conversion, all of the logic drivers on the network are collected, passed through the standard Verilog resolution function, and presented to the logic input. All the wreal drivers on the network are collected, passed through the **wreal Resolution Function** (specified on the **vasim** command), and presented to the wreal input. The D2D generates a real number on the wreal output and a bit on the logic output. These are used as the effective values for all readers on the net of types wreal and logic, respectively.

If there are no drivers on the wreal signal then `wrealZState (Z) is assumed. If the driven value for logic is 'X' then the effective real value will be `wrealXState.

Note

The resolution function SUM (dedicated for current computation) is not supported on wreal nets that are connected to logic. If this function is called out on the command line when using a D2D boundary element, then an error message will be issued and the AVG resolution function will be used instead. This means that the wreal values on the net cannot represent currents that must be summed.

Refer to the tables in the topic **vasim -d2dmode** for details on how the effective real and logic values are calculated for D2D boundary elements with the attribute MODE=WREAL, and how the values behave according to the wreal resolution function setting.

vasim -d2dmode

Controls the behavior of D2D boundary elements of type MODE=WREAL.

Note
D2D boundary element modes have no effect on boundary elements of type MODE=REAL.

Syntax

```
vasim -d2dmode compatible | strict | relaxed
```

Arguments

- compatible
The real driver value is only compatible with the logic state that corresponds to it (not with X). If the real value falls between vth1 and vth2, then it is ignored. This is the default.
- strict
Only one driver is accepted. This means that only 1 of the two real and logic values that are non-Z and non-X get a non-X result. If the real value falls between vth1 and 2, it will be interpreted as X.
- relaxed
The real driver value is compatible with X or with the logic state that corresponds to its value. This mode is an intermediate case between the two modes above.

Description

The tables in this section describe how the effective real and logic values are calculated in each mode.

Compatible Mode

In compatible mode (vasim -d2dmode compatible) the driven real value is only compatible with the logic state that corresponds to it (not with X). In the tables below, value means the “driven” value of the wreal signal.

Effective real values are computed in compatible mode, as described in [Table 9-2](#):

Table 9-2: Effective Real Values in Compatible Mode

Driven Real Value	Logic 0	Logic 1	Z
`wrealXState	`wrealXState	`wrealXState	`wrealXState
`wrealZState	vlo	vhi	`wrealZState

Table 9-2: Effective Real Values in Compatible Mode (continued)

Driven Real Value	Logic 0	Logic 1	Z
value < vth1	`wrealXState when the wreal resolution function is DEFAULT. Otherwise, equal to value.	`wrealXState	value
vth1 ≤ value ≤ vth2	`wrealXState when the wreal resolution function is DEFAULT. Otherwise, equal to value.	`wrealXState when the wreal resolution function is DEFAULT. Otherwise, equal to value.	value
value > vth2	`wrealXState	`wrealXState when the wreal resolution function is DEFAULT. Otherwise, equal to value.	value

Resolved logic values are computed in compatible mode, as described in [Table 9-3](#)

Table 9-3: Effective Logic Values in Compatible Mode

Driven Real Value	Effective Logic Value ¹
`wrealXState	X
`wrealZState	Z
value < vth1	0
vth1 ≤ value ≤ vth2	0 for U, X, W, -, 1 and H 1 for 0 and L
value > vth2	1

Strict Mode

In strict mode (vasim -d2dmode strict) only one driver is accepted. This means that only 1 of the two real and logic values that are non-Z and non-X get a non-X result. In the tables below, value means the “driven” value of the wreal signal.

Effective real values are computed in strict mode, as described in [Table 9-4](#):

¹ U, X, W and - are under the 'X' category, 0 and L are under the 0 category, and 1 and H are under the '1' category.

Table 9-4: Effective Real Values in Strict Mode

Driven Real Value	Logic 0	Logic 1	Z
`wrealXState	`wrealXState	`wrealXState	`wrealXState
`wrealZState	vlo	vhi	`wrealZState
value < vth1	Dependent upon wreal resolution function as follows: • DEFAULT = `wrealXState • 4STATE=value • SUM=(value + vlo)/2 • AVG=(value + vlo)/2 • MIN=min(value, vlo) • MAX=(value, vlo)	`wrealXState	value
vth1 ≤ value ≤ vth2	`wrealXState	`wrealXState	value
value > vth2	`wrealXState	Dependent upon wreal resolution function as follows: • DEFAULT = `wrealXState • 4STATE=value • SUM=(value + vhi)/2 • AVG=(value + vhi)/2 • MIN=min(value, vhi) • MAX=(value, vhi)	value

Resolved logic values are computed in strict mode, as described in [Table 9-5](#)

Table 9-5: Effective Logic Values in Strict Mode

Driven Real Value	Effective Logic Value ²
`wrealXState	X
`wrealZState	Z
value < vth1	0
vth1 ≤ value ≤ vth2	X
value > vth2	1

Relaxed Mode

In relaxed mode (vasim -d2dmode relaxed) the real driver value is compatible with X or with the logic state that corresponds to its value. In the tables below, value means the “driven” value of the wreal signal.

Effective real values are computed in relaxed mode as described in [Table 9-6](#):

Table 9-6: Effective Real Values in Relaxed Mode

Driven Real Value	Logic 0	Logic 1	Z
`wrealXState	`wrealXState	`wrealXState	`wrealXState
`wrealZState	vlo	vhi	`wrealZState
value < vth1	Dependent upon wreal resolution function as follows: • DEFAULT = `wrealXState • 4STATE=value • SUM=(value + vlo)/2 • AVG=(value + vlo)/2 • MIN=min(value, vlo) • MAX=(value, vlo)	`wrealXState	value
vth1 ≤ value ≤ vth2	Dependent upon wreal resolution function as follows: • DEFAULT = `wrealXState	Dependent upon wreal resolution function as follows:	value

² U, X, W and - are under the 'X' category, 0 and L are under the 0 category, and 1 and H are under the '1' category.

Table 9-6: Effective Real Values in Relaxed Mode (continued)

Driven Real Value	Logic 0	Logic 1	Z
	4STATE=value - SUM=(value + vlo)/2 - AVG=(value + vlo)/2 - MIN=min(value, vlo) - MAX=(value, vlo)	- DEFAULT = `wrealXState 4STATE=value - SUM=(value + vlo)/2 - AVG=(value + vhi)/2 - MIN=min(value, vhi) - MAX=(value, vhi)	
value > vth2	`wrealXState	Dependent upon wreal resolution function as follows: - DEFAULT = `wrealXState 4STATE=value - SUM=(value + vhi)/2 - AVG=(value + vhi)/2 - MIN=min(value, vhi) - MAX=(value, vhi)	value

Effective logic values are computed in relaxed mode, as described in [Table 9-7](#)

Table 9-7: Effective Logic Values in Relaxed Mode

Driven Real Value	Effective Logic Value ³
`wrealXState	X
`wrealZState	Z
value < vth1	0
vth1 ≤ value ≤ vth2	0 for U, X, W, -, 1 and H 1 for 0 and L
value > vth2	1

³ U, X, W and - are under the 'X' category, 0 and L are under the 0 category, and 1 and H are under the '1' category.

D2D Boundary Elements of Type MODE=REAL

By default, D2D boundary elements are of type MODE=WREAL. However for SPICE-on-top designs, D2D boundary elements must be of type MODE=REAL. In this case, additional parameters are available to define values for X and Z.

You can use a D2D boundary element to connect a logic digital port to a real port. In this case, additional parameters are available to define real values representing the disconnect state X and the unknown state, Z.

```
.MODEL <model_name> D2D mode=real [vth1=<value>] [vth2=<value>]
[vlo=<value>] [vhi=<value>] [strength weak|strong] [resolution=real|logic]
[DIGHIZ=<value>] [DIGX=<value>] [XOUT=<value>] [ZOUT=<value>] [UOUT=<value>]
```

The resolution parameter is used in the case of bidirectional conversion. If resolution=real, the logic value is converted into real as described in [Table 9-8](#), the real resolution function is called with the two real values, and the logic value is then computed as described in [Table 9-9](#).

Table 9-8: Logic to Real Conversion for D2D Boundary Elements of Type MODE=REAL

Logic Value	Real Value
Logic = '1' or 'H'	Vhi
Logic = '0' or 'L'	Vlo
Logic = 'X' or 'W'	Xout
Logic = 'Z'	Zout
Logic = 'U' or '-'	Uout

Table 9-9: Real to Logic Conversion for D2D Boundary Elements of Type MODE=REAL

Value	Strong	Weak
rVal < vth1	'0'	'L'
rVal >= vth2	'1'	'H'
vth1 <= rVal < vth2	'X'	'W'
rVal = digHiZ	'Z'	'Z'
rVal = digX	'X'	'W'

Related Topics

[Using wreal](#)

Viewing Mixed-Signal Nets in Questa ADMS

When an instance containing boundary elements is selected in the Structure window, its mixed-signal nets are listed in the Objects window.

Figure 9-10 shows the Objects window, sorted by ascending Kind, as denoted by the up-arrow in the Kind column header. This has been done to bring the mixed-signal nets to the top of the list.

Figure 9-10: Objects Window Showing Mixed-Signal Nets

Name	Value	Kind	Mode
+ inr	0	Boundary	Internal
+ inp	0	Boundary	Internal
+ inl2	0	Boundary	Internal
+ inl1	0	Boundary	Internal
+ clock	0	Boundary	Internal

Expanding the list of nets shows the boundary elements as shown in **Figure 9-11**. The naming convention for boundary elements is:

```
\<net_name>#<direction>#<sig_base_type_name>#<xxx_name>\
```

Figure 9-11: Objects Window Showing Boundary Elements

Name	Value	Kind	Mode
- inr	0	Boundary	Internal
+ :top_spice:inr#defhook#0	U	Signal	D2A
- inp	0	Boundary	Internal
+ :top_spice:inp#defhook#0	U	Signal	D2A
- inl2	0	Boundary	Internal
+ :top_spice:inl2#defhook#0	U	Signal	D2A
- inl1	0	Boundary	Internal
+ :top_spice:inl1#hook#net0	U	Signal	D2A
- clock	0	Boundary	Internal
+ :top_spice:clock#hook#net0	U	Signal	A2D

Expanding the boundary elements shows the ports (**Figure 9-12**).

Figure 9-12: Ports

Name	Value	Kind	Mode
inr	0	Boundary	Internal
:top_spice:inr#defhook#0	U	Signal	D2A
:top_spice:y1:r	U	Signal	Out
inp	0	Boundary	Internal
:top_spice:inp#defhook#0	U	Signal	D2A
:top_spice:y1:p	U	Signal	Out
inl2	0	Boundary	Internal
:top_spice:inl2#defhook#0	U	Signal	D2A
:top_spice:y1:l2	U	Signal	Out
inl1	0	Boundary	Internal
:top_spice:inl1#hook#net0	U	Signal	D2A
:top_spice:y1:l1	U	Signal	Out
clock	0	Boundary	Internal
:top_spice:clock#hook#net0	U	Signal	A2D
:top_spice:y1:clock	U	Signal	In

The naming convention for ports is:

:<top level name>:<digital instance name>:<digital port name>

When a mixed-signal net is selected in the Objects window, the **Contributor Window** displays all active connectivity associated with it. Refer to **Interrogating Nets in the Contributor Window**.

See also: **Objects Window**.

Viewing Boundary Elements in the Structure Window.....	9-37
Plotting Mixed-Signal Nets.....	9-38
Naming Conventions for Boundary Elements.....	9-40
Boundary Elements Log File.....	9-42

Viewing Boundary Elements in the Structure Window

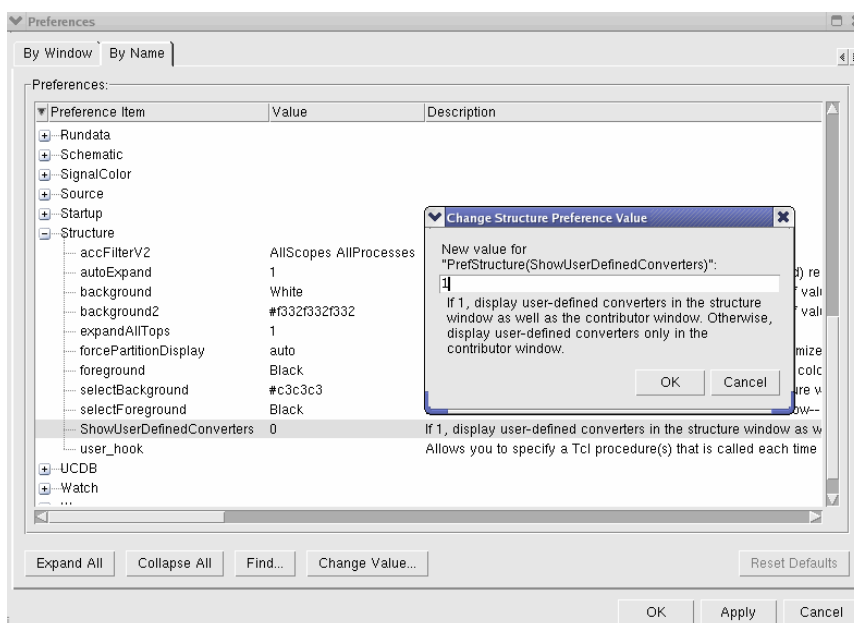
In order to debug user-defined A2D or D2A boundary elements, you can enable a preference which makes them visible in the Structure window. This can be used alongside the Contributor window to interrogate nets.

Procedure

- 1. Choose **Tools > Edit Preferences** and click the **By Name** tab
- 2. Expand the Structure preference item
- 3. Select ShowUserDefinedConverters and click **Change Value**

4. Change the value to 1 and click **OK**.
5. Click **OK** to exit the **Preferences Dialog Box**.

Figure 9-13: Changing Preference Value



Results

Now, when you right-click on a boundary element in the Contributor window and select **Goto region**, the instance of the relative boundary will be selected in the **Structure Window**, and the **Objects Window**.

You can also control visibility of user defined boundary elements in the Structure window, by setting the preference variable ShowUserDefinedConverters to 1.

Note

D2D boundary elements are not visible in the Structure window.

Related Topics

[Structure Window](#)

[Contributor Window](#)

[Interrogating Nets in the Contributor Window](#)

[Plotting Mixed-Signal Nets](#)

Plotting Mixed-Signal Nets

You can plot a mixed-signal net using the add wave command, using a .PLOT command, or by dragging and dropping from the Objects window to the Wave window.

When instructing to plot a mixed-signal net, either using the **add wave** command or by dragging and dropping from the **Objects Window** to the **Wave Window (Solido Waveform Analyzer)**, only the analog part of the selected net is plotted by default.

For example, if the following **add wave** command is used to plot a mixed-signal net b1, only the analog part of b1 is plotted:

```
add wave :test:b1
```

To plot the digital part of the net using the **add wave** command, specify the -s argument. For example:

```
add wave -s :test:b1
```

To plot the digital part of a mixed-signal net using a .PLOT command, specify the sg() argument. For example:

```
.PLOT tran sg(b1)
```

Refer to **.PLOT** in *Eldo Reference Manual* for more information.

Related Topics

[Naming Conventions for Boundary Elements](#)

[Viewing Mixed-Signal Nets in Questa ADMS](#)

[Requesting Plots of Mixed-Signal Simulation Results in a SPICE Netlist](#)

Naming Conventions for Boundary Elements

Boundary elements are identifiable by name when viewed in the Questa ADMS GUI. You can name boundary elements in order to access their internal elements if they are written in VHDL-AMS or Verilog-AMS.

The name is built as follows (this is an extended name):

```
\<net_name>#<direction>#<sig_base_type_name>#<xxx_name>\
```

- net_name

The name of the root net on which the boundary element is inserted. Boundary element and boundary net are in the same hierarchy. This is the name that appears in the [Boundary Elements Log File](#).

- direction

a2d, d2a, or bidir according to the boundary element.

- sig_base_type_name

Corresponds to the digital type of the boundary element. This is necessary because there may be more than one digital type associated to one boundary net, requiring more than one boundary element (one by digital type at least).

- xxx_name

An extra part of the boundary element name, required only in the following cases:

- When the boundary element has been inserted using a **.DEFHOOK MOD=** command. In this case, this name is the name of the model.
- When the boundary element has been inserted using a **.DEFHOOK GROUP=** command. In this case, this name is the name of the group.
- When the boundary element has been inserted using a **.DEFHOOK PDOMAIN=** command. In this case, this name is the name of the digital power that has been considered for this insertion.

If there is no digital power (which means that there is only one power, that is analog) this name does not exist.

- When the boundary element has been inserted using a **.DEFHOOK PORT=** (or **.HOOK PORT**) command. In this case, this name is the relative name of the port. This name is a hierarchical name starting where the boundary element has been inserted.
- If the mode is detailed (**.DETAILED** command) one boundary element is added for each OUT or INOUT port (generating a D2A boundary element). In this case, the name is the relative hierarchical name of this OUT or INOUT port (as above).

For the A2D boundary element, it follows the rules below, without exception as for the D2A.

If xxx_name is a hierarchical name, the hierarchy separator used is the dot ('.').

In addition, when one of those names contains some extended or escape identifiers, it is crypted as shown in [Table 9-10](#).

Table 9-10: Character Replacement in Names Containing Extended or Escape Identifiers

Original Character	Replacement
\	@b
<space>	@w
@	@@

This means that the crypted name is compatible with Verilog (because it has no white space) and also with VHDL/SPICE (because it has no single backslash) and thus may be used under these contexts (hierarchical name in Verilog for defparams for example, or in SPICE).

Related Topics

[Viewing Mixed-Signal Nets in Questa ADMS](#)

[Plotting Mixed-Signal Nets](#)

Boundary Elements Log File

A boundary elements log file, named `<cmd_filename>.conv`, is generated after elaboration.

The log file lists all the boundary elements that have been inserted in the design.

Format

```
NET basic_net
  [BIDIR]
  DtoA|AtoD|Bidir CONVERTER spice_model|connectrule_and_line VHDL-
  AMS|ELDO|VERILOG-AMS [logical_library.entity_or_module(.architecture)]
  [param_mapping]
    DEFHOOK file(line)|CONNECTRULE logical_library.connectrule line
    DISCIPLINE {logical_library.package.}continuous_discipline
    TYPE logical_library.package.type
    PORTS port {, port}
    WAVEFORM NAME waveform_name

END NET;
```

If boundary elements have been inserted or used as a consequence of the [Net Spy](#) commands `init_signal_spy()` and `signal_force()`, then data is added at the end of the A2D or D2A boundary element in the form:

```
FROM SIGNALSPY OR FORCE signal [,signal];
TO SIGNALSPY OR FORCE signal [,signal];
```

Parameters

- NET `<basic_net>`
The complete path name to the root of the net.
- [BIDIR] `<DtoA|AtoD|Bidir>`
The type of the boundary element, digital-to-analog, analog-to-digital or bidirectional. If the net is of a bidirectional nature (BIDIR), two boundary elements will be listed, one A2D and one D2A in parallel and in reverse direction (in staggered rows).
- CONVERTER `spice_model|connectrule_and_line VHDL-AMS|ELDO|VERILOG-AMS [logical_library.entity_or_module(.architecture)]`
The boundary element used and its details.
- [param_mapping]
Parameter values for user-defined boundary elements. Parameters for built-in boundary elements specified in the `.MODEL` command are not listed.
- DEFHOOK `file(line)|CONNECTRULE logical_library.connectrule line`

The origin of the boundary element insertion declaration. Can be either a .DEFHOOK statement within a command file, or a connectrules file. The file location and line number is given.

- DISCIPLINE

The terminal and nature of the net, with corresponding continuous discipline.

- TYPE

The discrete discipline of the net.

- PORTS

List of complete port signal path names.

Information about ports is grouped together when **.DETAILED** is not specified.

- WAVEFORM NAME

The waveform name of the net. When only one boundary element exists on the net, this will be the same as the name of the net <basic_net>.

Boundary Elements Examples

This section provides examples of working with boundary elements.

Boundary Elements Example: Cross Interface.....	9-43
Boundary Elements Example: Use of the .MODEL Command.....	9-44
Boundary Elements Examples: Net Spy.....	9-46

Boundary Elements Example: Cross Interface

You can set up a cross interface when instantiating SPICE from VHDL-AMS.

Procedure

1. Create an entity with which to associate the SPICE subcircuit.

If you want to connect SPICE nodes to digital signals, you must declare the ports as signals:

```
entity spice_ent is
port(terminal gnd, vana, rx, rxb, rxbus, rxbusb, rxsens : electrical;
      signal tx, txb, txinh : std_logic;
      terminal net40, net71 : electrical);
end spice_ent;
```

2. Declare a component and associated attributes in the VHDL-AMS design unit that is instantiating the SPICE subcircuit:

```

component spice_ent
  port (terminal gnd, vana, rx, rxb, rxbus, rxbusb, rxsens :
  electrical;
        signal tx, txb, txinh : std_logic;
        terminal net40, net71 : electrical);
end component;

attribute Eldo_device
of main_cb : component is Eldo_subckt;
attribute Eldo_subckt_name
of main_cb : component is "main";
attribute Eldo_file_name
of main_cb : component is "netlist.cir";

```

Results

Questa ADMS will insert boundary elements between the SPICE nets and the digital ports in the entity `spice_ent`.

Related Topics

Boundary Elements Examples

Defining Boundary Elements [Getting Started with Questa ADMS]

Boundary Elements Example: Use of the .MODEL Command

You can use the `.MODEL` command to define boundary elements.

In this example, an electrical node drives a digital input using the `STD_ULONGIC` states, a digital output using the `STD_ULONGIC` states drives an electrical node and an electrical voltage drives a `REAL` input. Boundary elements are defined for each of these interfaces using the `.MODEL` command, and are inserted using the `.DEFHOOK` command.

1. An electrical node drives a digital input using the `STD_ULONGIC` states. The boundary element `std_a2d` is defined with the parameters:

```

.MODEL std_a2d a2d MODE=STD_LOGIC STR=WEAK VTH1=0.6
+ VTH2=0.8 R=250K C=10f TX=0.3n

```

The characteristics of the electrical voltage are:

- Voltage for "0": 0V
- Voltage for "1": 1.5V

- State is "0" if the voltage is below: 0.6V
- State is "1" if the voltage is above: 0.8V

If the voltage remains between these two thresholds for less than 0.3ns, no "X" state is generated.

- Input capacitance of the gate: 10fF
- Input resistance of the gate: 250 k Ω
- The electrical voltage is considered as a WEAK digital output.

2. A digital output using the STD_ULONG states drives an electrical node. The boundary element (std_d2a) is defined with the parameters:

```
.MODEL std_d2a d2a MODE=STD_LOGIC VHI=1.5 VLO=0  
+ TRISE=0.3n TFALL=0.2n LOWCAP=10f RFALL=2.5k  
+ WEALOWRES=7.5k HIGHCAP=12f RRISE=3.5k WEAHIGHRES=10k  
+ ZCAP=15f RZ=200k
```

The characteristics of the electrical voltage are:

- Voltage for "0": 0V
- Voltage for "1": 1.5V
- Rise time of a digital signal: 0.3ns
- Fall time of a digital signal: 0.2ns
- Output capacitance of the gate at state "0" or "L": 10f
- Output resistance of the gate at state "0": 2.5k
- Output resistance of the gate at state "L": 7.5k
- Output capacitance of the gate at state "1" or "H": 12f
- Output resistance of the gate at state "1": 3.5k
- Output resistance of the gate at state "H": 10k
- Output capacitance of the gate at state "Z": 15f
- Output resistance of the gate at state "Z": 200k

3. An electrical voltage drives a REAL input. The analog-to-digital boundary element r_a2d of mode REAL, is defined with the parameters:

```
.MODEL r_a2d a2d MODE=REAL EPS=2m
```

You must define the minimum voltage difference considered as a digital change, using the parameter EPS, which in this example, is set to 2mV.

4. The .DEFHOOK command is now used to automatically insert the 3 boundary elements:

```
.DEFHOOK std_a2d std_d2a r_a2d
```

A2D, D2A, and bidirectional boundary elements will be automatically inserted between electrical and std_logic or its subtypes. However, only an A2D will be inserted between boundaries of type electrical and REAL. If a D2A or bidirectional boundary element of type REAL is needed by the description, an error will occur.

Related Topics

Boundary Elements Examples

Defining Boundary Elements [Getting Started with Questa ADMS]

Boundary Elements Examples: Net Spy

You can use Net Spy to add boundary elements.

Boundary elements automatically added or used by Net Spy commands `init_signal_spy()` and `signal_force()`, are identified in the log (.conv) file by FROM SIGNAL SPY and TO SIGNAL SPY statements.

Net Spy Example: A2D Terminal-to-Signal

The following VHDL code:

```
iss_t_s: init_signal_spy(":top: uut: source_ter", ":dest_sig_std", 1);
```

results in the following information being added to the log (.conv) file:

```
NET ADMS :top: uut: source_ter
NET ELDO UUT.SOURCE_TER :
  CONVERTER A2D_ELDO_BUILTIN_STD  ELDO
    DIRECTION AtoD
    NATURE IEEE:ELECTRICAL_SYSTEMS.ELECTRICAL
    TYPE IEEE:STD_LOGIC_1164.STD_LOGIC
    TO SIGNAL SPY OR FORCE :top:dest_sig_std;
END NET;
```

Net Spy Example: D2A Signal-to-Terminal

The following VHDL code:

```
iss_s_t: init_signal_spy(": uut: inst1: source_sig_bit", ":dest_ter_bit0", 1);
```


results in the following information being added to the log (.conv) file:

```
NET ADMS :top:dest_ter_bit0
NET ELDO DEST_TER_BIT0 :
  CONVERTER D2A_ELD0_BUILTIN_BIT  ELDO
  DIRECTION DtoA
  NATURE IEEE:ELECTRICAL_SYSTEMS.ELECTRICAL
  TYPE STD:STANDARD.BIT
  FROM SIGNAL SPY OR FORCE :top: uut:inst1:source_sig_bit;
END NET;
```

Related Topics

[Boundary Elements Examples](#)
[Net Spy](#)

10. Using Power Aware Simulation in Questa ADMS

This chapter describes how to use the Unified Power Format (UPF) in order to perform power-aware verification in Questa ADMS.

Questa ADMS Power Aware Simulation (PAsim).....	10-1
Connecting Power to Analog Power Ports.....	10-3
Example: Connecting UPF Power to Power Supply Pins in a SPICE Subcircuit.....	10-4
Example: Connecting UPF Power to Power Supply Ports of a Verilog-AMS Module.....	10-6
Example: Connecting UPF Power to Power Supply Ports in a VHDL-AMS Entity.....	10-7
Power Connect Elements.....	10-8
Power-Aware Signal Connect Elements.....	10-10
Plotting Nets of Type supply_net_type.....	10-13
Mixed-Signal Testbench with 2 Power Domains.....	10-14

Questa ADMS Power Aware Simulation (PAsim)

The increasing importance of power consumption in electronic systems creates a need for a model of active power management that supplements the digital register-transfer level (RTL) abstraction. The RTL abstraction incorporates the unstated assumption that sufficient power is always available for a system to operate correctly after simulation time zero.

The Unified Power Format (UPF) language provides the necessary notation for modeling active power control of digital RTL designs. In contrast, low-level electrical designs—traditionally coded in SPICE and, more recently, SPICE combined with the mixed-signal languages VHDL-AMS and Verilog-AMS—often model the power supply and power control explicitly. Electrical models are fundamentally dependent on appropriate power for correct operation, and in this they are unlike functional models at the RTL.

The Siemens Digital Industries Software power-aware simulation flow PAsim, uses the Unified Power Format (UPF) language to define backannotation of power supply switching information to a digital hierarchy that was designed without consideration of power. It enables you to specify the supply network, switches, isolation, retention and other aspects relevant to power management of an electronic system. The standard provides the low-power design specification to a design captured in HDL.

Power aware simulation in Questa ADMS is defined by the [IEEE Standard for Design and Verification of Low-Power Integrated Circuits](#), IEEE Std 1801-2013 (Revision of IEEE Std 1801-2009), May 29 2013.

Full details on the power aware flow can be found in the *Power Aware Simulation User's Manual*:

- UPF Commands and Reference
- Power Aware Default Settings

- Power Aware Simulation Command

Questa ADMS provides mixed-signal extensions to the Questa UPF flow, that allow you to:

- Attach UPF power supplies to analog power supply ports (the electrical power pins of analog and mixed-signal devices).

See [Connecting Power to Analog Power Ports](#).

- Insert power-aware connect elements (A2D and D2A boundary elements) that respond to changes in power switching as well as changes in signal value.

See [Power-Aware Signal Connect Elements](#).

Analog and mixed-signal elements use only explicit or automatic connections for UPF derived power, never implicit connection.

Limitations

When using power aware simulation, be aware of the following:

- Power aware simulation is only supported for digital-on-top designs. If the hierarchy includes a digital-analog-digital sandwich, then digital elements below the analog level are outside of PAsim control.
- It is mandatory to use the following options when compiling UPF input files using vopt:

```
-pa_enable=nonoptimizedflow
```

If this option is not used then the design will fail at elaboration time, and an error will be reported to the Transcript. Full details on the elaboration failure are written to the Questa *transcript.ms* file.

```
-pa_enable=anonupfobjects
```

This allows `supply_on()` calls to set the values of supply set functions from your test bench.

- The `UPF_pg_type` attribute can only have the values `primary_power` or `primary_ground`.
- In the digital description, all power ports declared using the type `supply_net_type` must be left open.
- It is not possible to create power connect elements using Verilog-AMS, because Verilog-AMS does not contain structure types.
- Any VHDL port of type `supply_net_type` and mode "in" must be provided with a default value. For example:

```
PORT (in_pwr : in supply_net_type := (OFF, (others => '0'))) ; ...
```

Related Topics

[Connecting Power to Analog Power Ports](#)
[Power-Aware Signal Connect Elements](#)

Connecting Power to Analog Power Ports

To connect power to analog power supply ports, you must provide a VHDL entity declaration, or a SystemVerilog module with ports of `supply_net_type`.

Suppose you begin with a power-aware design described by a homogeneous digital hierarchy and an accompanying UPF file. An instance of an AMS model (coded in SPICE, Verilog-AMS, or VHDL-AMS) with electrical ports will be substituted for an existing digital instance in the digital design. The AMS model has electrical ports that correspond one-for-one to the ports of the digital instance being replaced. The translation of information to and from the electrical ports should be transparent to the instantiating digital circuit. The translation should be sensitive to the power state of the UPF domains of the digital signals.

The AMS model has additional electrical ports that supply power to the analog sub-circuit within the model. The analog power supplies connected to the ports must be synchronized with the power state of the UPF power domain of the instance. Alternatively, the model may contain the root supply driver for a power function of the enclosing power domain—that is, the model acts as a producer rather than a consumer of power.

To connect power to analog power supply ports, you must provide a VHDL entity declaration, or a SystemVerilog module with ports of `supply_net_type`.

Figure 10-1: Digital Instance to be Replaced

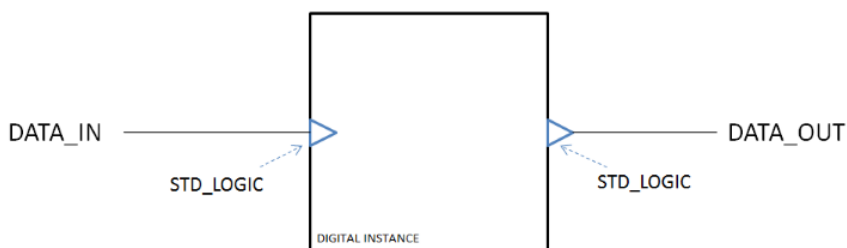
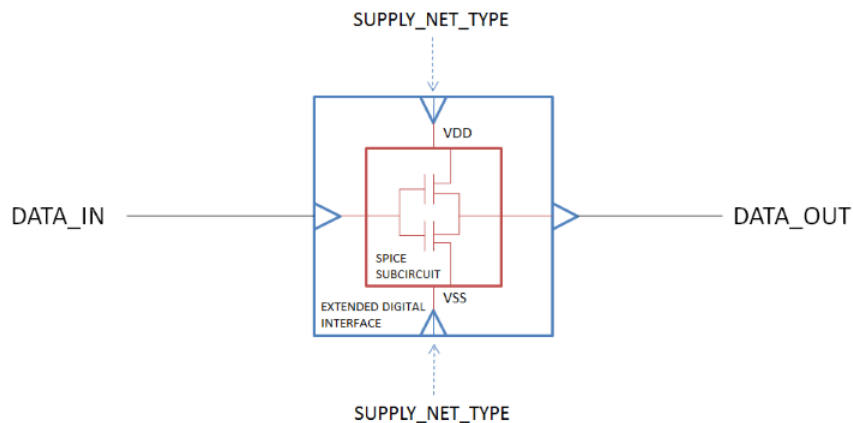


Figure 10-2: SPICE Subcircuit Replacing Digital Instance



Related Topics

[Example: Connecting UPF Power to Power Supply Pins in a SPICE Subcircuit](#)

[Example: Connecting UPF Power to Power Supply Ports of a Verilog-AMS Module](#)

[Example: Connecting UPF Power to Power Supply Ports in a VHDL-AMS Entity](#)

Example: Connecting UPF Power to Power Supply Pins in a SPICE Subcircuit

You can connect UPF power to power supply pins in a SPICE Subcircuit.

In this example, a digital instance (VHDL or Verilog) that is implicitly controlled by PAsim as described in an existing UPF file, will be replaced with a SPICE subcircuit. A VHDL entity declaration with ports of `supply_net_type` is created, and the connection is made by providing `UPF_pg_type` attributes.

Alternatively, you can add connect statements in the UPF file to connect to ports of type `supply_net_type`. Refer to Extended Power Aware HDL Model in the *Power Aware Simulation User's Manual*.

Procedure

1. Suppose the digital instance to be replaced has the following interface:

```
module inv(inp, outp);
```

or

```
entity inv is
  PORT (inp : in Std_logic;
        outp : out Std_logic);
```

The SPICE subcircuit that will replace the digital instance, has the following interface:

```
.subckt inv in out VDD VSS
```

2. Prepare an extended VHDL interface that includes power ports of `supply_net_type`, with `UPF_pg_type` attributes, as shown below in bold:

```
Library IEEE;
use ieee.upf.all, IEEE.Std_logic_1164.all;
entity inv is
  PORT (inp : in Std_logic;
        outp : out Std_logic;
        VDD : in supply_net_type:= (OFF, (others=>'0'));
        VSS : in supply_net_type:= (OFF, (others=>'0')));
  attribute UPF_pg_type of
    VDD : signal is "primary_power";
  attribute UPF_pg_type of
    VSS : signal is "primary_ground";
end;
```

Or a SystemVerilog module that contains:

```
import UPF::*;
module inv(inp, outp, VDD, VSS);
  (* UPF_pg_type = "primary_power" *)
  input supply_net_type VDD;
  (* UPF_pg_type = "primary_ground" *)
  input supply_net_type VSS;
  input inp;
  output reg outp;
endmodule
```

3. Compile the extended interface you just prepared, using the `-target_entity` option:

```
vacom -target_entity <filename>
```

or

```
valog -target_entity <filename>
```

4. Launch **vaspi** in interactive mode and use the **Interface Matcher** to map the ports of the SPICE subcircuit to the ports of the interface:

```
vaspi -interactive <digital_unit> <subckt_name>[@<file_name>]
```

Refer to [Mapping Ports Using the Interface Matcher](#) for instructions on how to map ports.

Related Topics

[Connecting Power to Analog Power Ports](#)

[Example: Connecting UPF Power to Power Supply Ports of a Verilog-AMS Module](#)

[Power Connect Elements](#)

[Example: Connecting UPF Power to Power Supply Ports in a VHDL-AMS Entity](#)

[Instantiating a SPICE Subcircuit from VHDL, VHDL-AMS or Verilog](#)

Example: Connecting UPF Power to Power Supply Ports of a Verilog-AMS Module

You can connect UPF power to the power supply ports of a Verilog-AMS Module.

In this example, an existing Verilog instance that is implicitly controlled by PAsim as described in an existing UPF file, will be replaced with Verilog-AMS description that have explicit power connections.

Procedure

1. Suppose the Verilog instance has this interface:

```
entity inv is
PORT (inp : in Std_logic;
      outp : out Std_logic);
```

2. Prepare the following Verilog-AMS description:

```
module inv(inp, outp, VDD, VSS);
  (* UPF_pg_type = "primary_power" *)
  input electrical VDD;
  (* UPF_pg_type = "primary_ground" *)
  input electrical VSS;
  input electrical inp;
  output electrical outp;
  ...
```

Electrical ports of Verilog-AMS modules with the UPF_pg_type attribute will be automatically connected to the designated UPF supply function within the power domain in which the instance is contained.

3. Compile the Verilog-AMS description using valog:

```
valog -work <new_library_name> <filename>
```


Use the -work options to compile the Verilog-AMS description in a different library. The new module has 4 ports. Because they have the same name, compiling in the same library will overwrite the original.

Related Topics

[Connecting Power to Analog Power Ports](#)

[Example: Connecting UPF Power to Power Supply Pins in a SPICE Subcircuit](#)

[Power Connect Elements](#)

[Example: Connecting UPF Power to Power Supply Ports in a VHDL-AMS Entity](#)

Example: Connecting UPF Power to Power Supply Ports in a VHDL-AMS Entity

You can connect UPF power to power supply ports in a VHDL-AMS Entity.

In this example, a digital instance will be replaced by a VHDL-AMS entity declaration.

Alternatively, you can add connect statements in the UPF file to connect to ports of type `supply_net_type`. Refer to Extended Power Aware HDL Model in the *Power Aware Simulation User's Manual*.

Procedure

1. Suppose the VHDL-AMS declaration has this interface:

```
Library IEEE;
use ieee.upf.all,
        IEEE.Electrical_systems.all;
entity inv is
    PORT (Terminal inp : Electrical;
          Terminal outp : Electrical;
          Terminal VDD : Electrical;
          Terminal VSS : Electrical);
end;
```

2. Prepare the following VHDL target entity:

```
Library IEEE;
use ieee.upf.all,
        IEEE.Electrical_systems.all;
use IEEE.std_logic_1164.all;
entity inv is
    PORT (Signal inp : in std_logic;
          Signal outp : out std_logic;
          Signal VDD : in supply_net_type:= (OFF, (others=>'0')));
          Signal VSS : in supply_net_type:= (OFF, (others=>'0')));
    attribute UPF_pg_type of
```

```
VDD : Signal is "primary_power";
attribute UPF_pg_type of
  VSS : Signal is "primary_ground";
end;
```

The digital wrapper port directions will dictate whether to source or sink the power supply from or to the VHDL-AMS unit.

3. Compile the VHDL target entity:

```
vcom <filename>
```

4. Compile the VHDL-AMS description using vacom -target_entity:

```
vacom -target_entity <filename>
```

5. Use either the entity form of component declaration or the component form of component declaration to make the instantiation.

The entity form is helpful if the VHDL instantiating the VHDL-AMS design entity uses the entity form of instantiation and cannot be changed because of design flow constraints.

Related Topics

[Connecting Power to Analog Power Ports](#)

[Example: Connecting UPF Power to Power Supply Ports of a Verilog-AMS Module](#)

[Example: Connecting UPF Power to Power Supply Pins in a SPICE Subcircuit](#)

[Power Connect Elements](#)

Power Connect Elements

A UPF net of type `supply_net_type` is connected to each electrical port using a Power-to-Electrical (P2E) or an Electrical-to-Power (E2P) power connect element.

Questa ADMS automatically attaches power connect elements to user-selected power ports at elaboration time. If an electrical port is sourcing power, Questa ADMS inserts a P2E power connect element. If an electrical port is sinking power, Questa ADMS inserts an E2P power connect element. You may connect input and output electrical ports to the same UPF net.

In a power connect element, the primary power value is defined by the state and the voltage between the power and ground. The primary ground is between the ground and the electrical reference. P2E and E2P power connect elements have 3 ports, one digital and two analog. The last analog port is considered to be the negative (or reference) power port.

The primary ground of a power domain is not the same as the electrical reference.

Power-to-Electrical (P2E) Power Connect Element Example

```

Library IEEE;
use ieee.upf.all,
        IEEE.Electrical_systems.all;

entity P2E is
  generic (default_OFF_voltage : Real := 0.0;
           default_UND_voltage : Real := 0.0;
           rising_time, falling_time : Real := 50.0e-12);

  port (upf_in : in supply_net_type;
        Terminal VDD, VSS : Electrical);
        -- VDD is the '+' terminal
        -- VSS is the '-' terminal where the
        -- power voltage is to be applied
end entity P2E;

```

Table 10-1 describes how the input value voltage (μV) of a P2E connect element is determined by the state value of the digital input port

Table 10-1: P2E Connect Element Input Value

State Value	Input Voltage
ON	upf_in
OFF	OFF
PARTIAL_ON	upf_in (if the previous state was not ON) or OFF (If the previous state was ON). This change is made according to the rising and falling times.
UNDETERMINED	UND

Electrical-to-Power (E2P) Power Connect Element Example

```

Library IEEE;
use ieee.upf.all,
        IEEE.Electrical_systems.all;

entity E2P is
  generic (Vth_off : Real := 2.0;
           Vth_on : Real := 3.0);

  port (signal upfout : out supply_net_type;
        terminal VDD: electrical;
        terminal VSS: electrical);
        -- VDD is the '+' terminal
        -- VSS is the '-' terminal where the
        -- power voltage is to be read
end entity E2P;

```

Table 10-2 describes how the voltage value of upf_out in an E2P connect element is related to the input voltage ($V_{VDD} - V_{VSS}$) as an integer in μV , regardless of the state.

Table 10-2: E2P Connect Element Primary Power Value

Input Voltage	upf_out State Value
< Vth_off	OFF
> Vth_on	ON
All other values	PARTIAL_ON

Inserting Power Connect Elements

Declare and insert power connect elements using the .MODEL and .DEFHOOK commands:

For P2E and E2P power connect elements:

```
.MODEL p2e hook mgc_ams:p2e(ams)
.MODEL e2p hook mgc_ams:e2p(ams)
.DEFH00K e2p p2e
```

For auto-calibrated power connect elements of type STD_LOGIC:

```
.MODEL auto_std_logic_d2a hook mgc_ams:logic2a(ams)
.MODEL auto_std_logic_a2d hook mgc_ams:a2logic(ams)
.DEFH00K auto_std_logic_d2a auto_std_logic_a2d
```

For auto-calibrated power connect elements of type REAL:

```
.MODEL auto_real_d2a hook mgc_ams:real2a(ams)
.MODEL auto_real_a2d hook mgc_ams:a2real(ams)
.DEFH00K auto_real_d2a auto_real_a2d
```

Related Topics

[Connecting Power to Analog Power Ports](#)
[Power-Aware Signal Connect Elements](#)

Power-Aware Signal Connect Elements

A power-aware signal connect element is the same as boundary element, except that it has 2 extra power ports. There are 4 ports in total: one signal, one terminal, primary power, and primary ground.

If an isolation or retention strategy has been defined around a digital instance, when power is taken from the digital side, it is taken according to these added elements.

Table 10-3 lists the built-in power-aware signal connect elements that are available in the library `mgc_ams`.

Table 10-3: Built-in Power Connect Elements in the `mgc_ams` Library

Power-Aware Signal Connect Element	Description
<code>a2logic</code>	Autocalibrated Electrical-To-Logic
<code>a2real</code>	Autocalibrated Electrical-To-Real
<code>logic2a</code>	Autocalibrated Logic-To-Electrical
<code>real2a</code>	Autocalibrated Real-To-Electrical
<code>ams</code>	

Examples

- D2A Std_logic

```
entity Std_D2A is
  generic (rising_time,
    falling_time : Real := 50.0e-12;
    Vx_as_previous : Boolean := TRUE;
    Vx : Real := 0.0;
    Voff : Real := 0.0);

  port (Terminal outp : Electrical,
    Signal inp : IN Std_logic,
    Signal power : IN supply_net_type,
    Signal ground : IN supply_net_type);

  attribute UPF_pg_type of
    power : Signal is "primary_power";

  attribute UPF_pg_type of
    ground : Signal is "primary_ground";

end entity Std_D2A;
```

`rising_time` and `falling_time` correspond to the time to switch from one voltage to another.

`Vx_as_previous`, when set to `TRUE` means that the output voltage for X is the voltage present before X occurs. `Vx` is used as a value for X when `Vx_as_previous` is set to `FALSE`.

Voff is the output voltage when the power supply is OFF or UNDERTERMINED. In the case of 1 or H, the output voltage is set to the supply voltage. In the case of 0 or L, it is set to the supply voltage. Otherwise, a HiZ state is considered in case of Z. Otherwise it is set to the X voltage for the other states.

- D2A Real

```
entity Real_D2A is
  generic (rising_time,
    falling_time : Real := 50.0e-12;
    Vx_as_previous : Boolean := TRUE;
    Vx : Real := 0.0;
    Voff : Real := 0.0);

  port (Terminal outp : Electrical,
    Signal inp : IN Real,
    Signal power : IN supply_net_type,
    Signal ground : IN supply_net_type);

  attribute UPF_pg_type of
    power : Signal is "primary_power";

  attribute UPF_pg_type of
    ground : Signal is "primary_ground";

end entity Real_D2A;
```

rising_time and falling_time correspond to the time to switch from one voltage to another.

Vx_as_previous, when set to TRUE means that the output voltage for X is the voltage present before X occurs. Vx is used as a value for X when Vx_as_previous is set to FALSE.

Voff is the output voltage when the power supply is OFF or UNDERTERMINED. In the case of Z, a HiZ state is considered. For the X state, it is set to the X voltage. Otherwise the digital voltage is considered.

- A2D Real

Real power connect elements are compatible with wreal types; X and Z values are correctly managed.

Tip

Further details can be found in the *Power Aware Simulation User's Manual*.

UPF Calibrated Signal Boundaries

Signals may be connected across the analog-to-digital boundary using power-aware connect elements that are dynamically calibrated using the UPF power supply information of the domains containing the digital drivers and receivers of the net.

A power-aware signal connect element must know what voltages correspond to digital '1', '0', and 'X'. The power specification in the UPF file specifies voltages for each power domain. The UPF voltages for the power domain determine the high and low voltages.

"Relaxed" and "Strict" Modes

The analog and digital sides of an analog-to-digital boundary may be in different power domains. You can specify whether the calibrating power domain is the domain of the digital or the analog side.

If power is taken from the analog side (known as "relaxed" mode) the simulation will run correctly and return reliable results, even if the power strategy is not accurate.

If power is taken from the digital side (known as "strict" mode) any mistakes in the power strategy will cause functional errors during simulation. If it is not possible to get power using strict mode, then relaxed mode is attempted.

There are two ways to specify strict or relaxed mode:

- A global option can be specified in the `.cir` or `.cmd` file:

```
.OPTION cvupf=strict | relaxed
```

- Use the `-cvupf` option when launching Questa ADMS:

```
vasim [-cvupf=strict | relaxed]
```

This overrides the SPICE option `.CVUPF`.

Ports with `UPF_pg_type` attributes will be automatically connected to the right power domain nets at elaboration time according the mode: strict or relaxed.

Verilog connect modules cannot be used to access power as digital signals (`supply_net_type` is not a Verilog-compatible type).

Related Topics

[Connecting Power to Analog Power Ports](#)
[Boundary Element Insertion](#)

Plotting Nets of Type `supply_net_type`

UPF nets of `supply_net_type` from VHDL may be plotted in Solido Waveform Analyzer as one bus of two 32-bit buses.

A UPF net from a VHDL-AMS region is represented as follows:

- For the first vector of 32 bits that corresponds to the voltage of the power in μV , Solido Waveform Analyzer considers it as a Real value in volts. Thus it can be compared to other voltages.
- For the second vector that corresponds to an integer value between 0 and 3 for the 4 states: ON, OFF, PARTIAL_ON and UNDETERMINED, Solido Waveform Analyzer represents it as an enum.

It is not possible to plot nets of supply_net_type from a SystemVerilog region.

Related Topics

Connecting Power to Analog Power Ports

Mixed-Signal Testbench with 2 Power Domains

You can test a mixed-signal design with components powered by 2 different power domains.

Figure 10-3 shows the design top, which is powered by the power domain PD_TOP. Instantiated in TOP are DFF2 and INV2 which are powered by a different power domain, PD_ECO. The Testbench switches PD_TOP and PD_ECO on and off, producing different outputs on Q1-Q4.

Figure 10-3: Mixed-Signal Testbench With 2 Power Domains

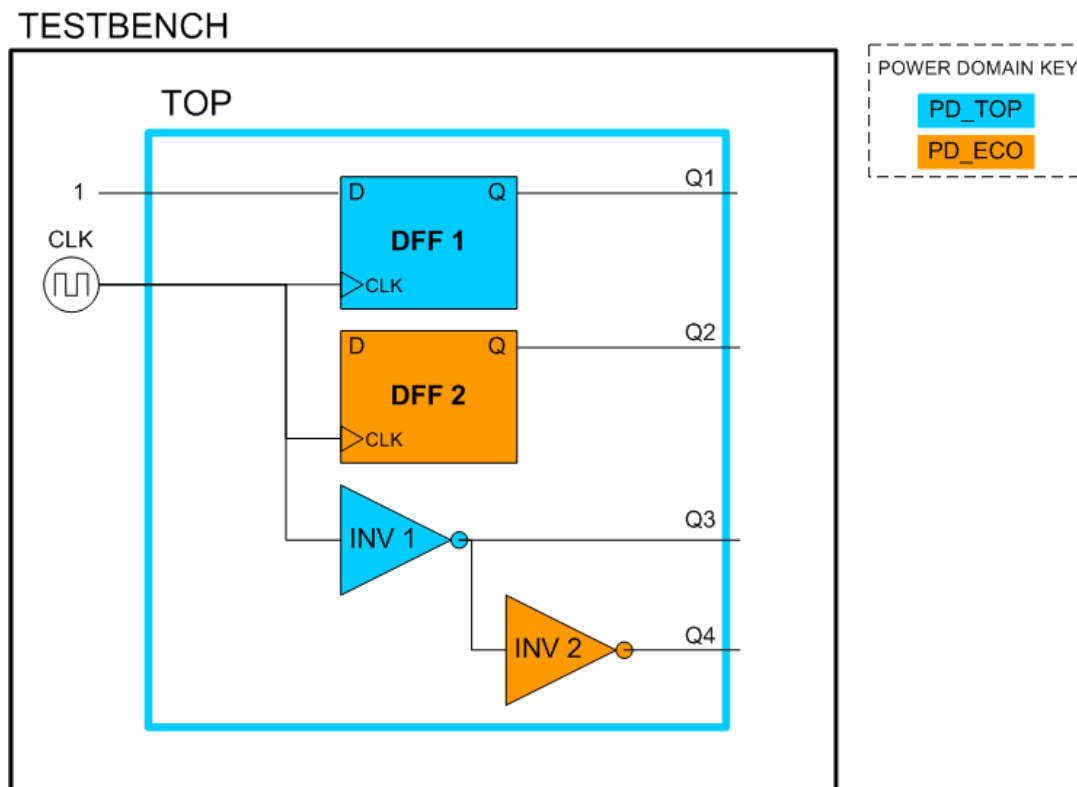
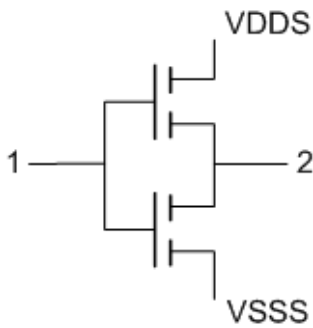


Figure 10-4: INV2



Prerequisites

- *inv.v* file:

```
`timescale 1ns / 1ns
module inv (inp, outp);
    parameter del = 2;
    input  inp;
    output outp;
    // not #del noti (outp, inp);
    assign #del outp = ~inp;
endmodule // invertor
```

- *inv_extended.sv* file:

```
import UPF::*;
module inv_extended(
    (*UPF_pg_type = "primary_power"*)
    input supply_net_type in_pwr,
    (*UPF_pg_type = "primary_ground"*)
    input supply_net_type in_gnd,
    input inp,
    output outp
);
    assign outp = ~inp;
endmodule
```

- *dff_nocell.v* file:

```
////////////////////////////////////
// Verilog library for ASIC Design Kit (ADK)
//
// 1.6 Initial Release
//
////////////////////////////////////
//%BEGIN dff
module dff (D, CLK, Q);
```

```

        input D, CLK;
        output reg Q;
    always @ (posedge CLK )
    // if (reset)
    // else
    begin
        Q = D;
    end
endmodule

//`celldefine
module dff_cell (D, CLK, Q, QB);
    input D, CLK;
    output Q, QB;
    reg viol_0;
    wire dummy_0, int_res_0;
    ////////////////////////////////////////////////////
    //                      FUNCTIONALITY                      //
    ////////////////////////////////////////////////////
    dff_err (dummy_0, CLK, D);
    dff_p (Q, viol_0, CLK,
           D, dummy_0);
    dff_p (int_res_0, viol_0, CLK,
           D, dummy_0);
    not (QB, int_res_0);
    ////////////////////////////////////////////////////
    //                      TIMING                      //
    ////////////////////////////////////////////////////
    specify
        specparam tpd_CLK_Q_posedge = 0;
        specparam tpd_CLK_QB_posedge = 0;
        specparam tsetup_D_CLK_noedge_posedge = 0;
        specparam thold_D_CLK_noedge_posedge = 0;
        specparam tpw_CLK_posedge = 0;
        specparam tpw_CLK_negedge = 0;
        (posedge CLK => (Q+:D)) = tpd_CLK_Q_posedge;
        (posedge CLK => (QB-:D)) = tpd_CLK_QB_posedge;
        $setup (posedge D, posedge CLK,
                tsetup_D_CLK_noedge_posedge, viol_0);
        $setup (negedge D, posedge CLK,
                tsetup_D_CLK_noedge_posedge, viol_0);
        $hold (posedge CLK, posedge D,
                thold_D_CLK_noedge_posedge, viol_0);
        $hold (posedge CLK, negedge D,
                thold_D_CLK_noedge_posedge, viol_0);
        $width (posedge CLK, tpw_CLK_posedge,
                0, viol_0);
        $width (negedge CLK, tpw_CLK_negedge,
                0, viol_0);
    endspecify
endmodule
//`endcelldefine
//%END dff
primitive dff_err (q, clk, d);
    output q;

```

```

    reg q;
    input clk, d;
    table
        (0x) ? : ? : 0;
        (1x) ? : ? : 1;
    endtable
endprimitive
primitive dff_p (q, v, clk, d, dummy);
    output q;
    reg q;
    input v, clk, d, dummy;
    table
        *   ?   ? ? : ? : x;
        ? (x1) 0 0 : ? : 0;
        ? (x1) 1 0 : ? : 1;
        ? (x1) 0 1 : 0 : 0;
        ? (x1) 1 1 : 1 : 1;
        ? (x1) ? x : ? : -;
        ? (bx) 0 ? : 0 : -;
        ? (bx) 1 ? : 1 : -;
        ? (x0) b ? : ? : -;
        ? (x0) ? x : ? : -;
        ? (01) 0 ? : ? : 0;
        ? (01) 1 ? : ? : 1;
        ? (10) ? ? : ? : -;
        ?   b   * ? : ? : -;
        ?   ?   ? * : ? : -;
    endtable
endprimitive

```

- *top.sv* file:

```

`timescale 1 ns/1 ns
module top(input CTRL_POWER_PD_PS, input d, input clk,output q1,
output q2, output q3,output q4 ) ;
    dff dff1 (d,clk,q1);
    dff dff2 (d,clk,q2);
    //dff dff3 (d,clk,q3);
    inv inv1 (clk, q3) ;
    inv_extended inv2 (.inp(q3), .outp(q4)) ;
    //inv inv3 (clk, q6) ;

    //analog driving digital PD
    //ana_driv_wrapper ana_driv_wrapper1 ();
initial
    begin
    end
endmodule

```

- *tb_twoPDs.sv* file:

```

`timescale 1 us/1 us
import UPF::*;
module tb;

```

```

logic clk;
logic q1,q2,q3,q4;
logic d;
reg CTRL_POWER_PD_PS;
top
top_inst(.CTRL_POWER_PD_PS(CTRL_POWER_PD_PS),.d(d),.clk(clk),.q1(q1
),.q2(q2),.q3(q3),.q4(q4));
initial forever #10 clk = !clk ;
initial
    begin
        clk = 0 ;
        d = 1;

        CTRL_POWER_PD_PS = 0;

        //from 0 to 100ns, All Power Domains OFF
        #100;
//test 1 : PD Top ONLY
        //from 100-200ns,Power Domain Top ON
        //assert(supply_on("top_inst/pwr_port", 5.0));
        assert(supply_on("/tb/top_inst/pwr_port", 5.0));
        assert(supply_on("/tb/top_inst/gnd_port", 0.0));
        #50;
        assert(supply_on("/tb/top_inst/pwr_port", 1.0));
        #20;
        assert(supply_on("/tb/top_inst/pwr_port", 5.0));
        #30;

        // from 200 to 300ns, Power Domain Top OFF
        assert(supply_off("/tb/top_inst/pwr_port"));
        assert(supply_off("/tb/top_inst/gnd_port"));
        #100;
//test 2 : PD ECO ONLY
        //from 300-400ns,Power Domain eco ON
        assert(supply_on("/tb/top_inst/pwr_port_eco", 5.0));
        assert(supply_on("/tb/top_inst/gnd_port_eco", 0.0));
        #50;
        assert(supply_on("/tb/top_inst/pwr_port_eco", 0.5));
        #20;
        assert(supply_on("/tb/top_inst/pwr_port_eco", 5.0));
        #30;

        // from 400 to 500ns, Power Domain eco OFF
        assert(supply_off("/tb/top_inst/pwr_port_eco"));
        assert(supply_off("/tb/top_inst/gnd_port_eco"));
        #100;

//test 3 : PD Top AND PD ECO
        // from 500-600ns,Power Domain Top ON
        assert(supply_on("/tb/top_inst/pwr_port"));
        assert(supply_on("/tb/top_inst/gnd_port"));
        #100;
        //from 500-600ns,Power Domain Top ON and Power Domain eco ON
        assert(supply_on("/tb/top_inst/pwr_port_eco", 5.0));

```

```

    assert(supply_on("/tb/top_inst/gnd_port_eco", 0.0));

    #50;
    assert(supply_on("/tb/top_inst/pwr_port_eco", 0.5));
    #20;
    assert(supply_on("/tb/top_inst/pwr_port_eco", 5.0));
    #30;

    // from 600-700ns, Power Domain Top ON and Power Domain eco OFF
    assert(supply_off("/tb/top_inst/pwr_port_eco"));
    assert(supply_off("/tb/top_inst/gnd_port_eco"));
    #100;
    assert(supply_off("/tb/top_inst/pwr_port"));
    assert(supply_off("/tb/top_inst/gnd_port"));
    #100;

    //assert(supply_on("top_inst/pwr_port_anadriv"));
    //assert(supply_on("top_inst/gnd_port_anadriv"));
    // waiting 100us for the analog source to drive PD ANADRIV
    #100;

    // at 1ms, PD ON
    assert(supply_on("/tb/top_inst/pwr_port_PS"));
    assert(supply_on("/tb/top_inst/gnd_port_PS"));
    #100;
//test 4 : turn ON/OFF PD SW from 900us to 1.2ms
CTRL_POWER_PD_PS = 1;
#100;
CTRL_POWER_PD_PS = 0;
#100;
    end
endmodule

```

Procedure

1. Create a library and compile the source code:

```

valib work
vlog inv.v
vlog inv_extended.sv
vlog dff_nocell.v
vlog top.sv
vlog tb_twoPDs.sv

```

2. The file *inv_extended.sv* provides the interface for the SPICE subcircuit *inv.ckt*. Use *vaspi ... -interface ...* to associate *inv.ckt* with the *inv* instance in *inv_extended*:

```

vaspi work.inv_extended inv@inv.ckt -interface
inv-inv_extended.assoc

```

3. Use the vopt command to perform optimizations, and declare the UPF file and Questa ADMS-specific options:

```
vopt top -pa_defertop -pa_upf test_twoPDs.upf -pa_genrpt
-pa_enable=nonoptimizedflow
```

UPF (*test_twoPDs.upf*) file:

```
upf_version 1.0

set_design_top top
set_scope .

#####
#pd_top Power Domain
#####

create_power_domain pd_top -include_scope
create_supply_port pwr_port -domain pd_top
create_supply_port gnd_port -domain pd_top
create_supply_net pwr_net -domain pd_top
create_supply_net gnd_net -domain pd_top
connect_supply_net pwr_net -ports { pwr_port }
connect_supply_net gnd_net -ports { gnd_port }
set_domain_supply_net pd_top -primary_power_net pwr_net
-primary_ground_net gnd_net

#####
#pd_eco Power Domain
#####

create_power_domain pd_eco -elements { dff2 inv2 }
create_supply_port pwr_port_eco -domain pd_eco
create_supply_port gnd_port_eco -domain pd_eco
create_supply_net pwr_net_eco -domain pd_eco
create_supply_net gnd_net_eco -domain pd_eco
connect_supply_net pwr_net_eco -ports { pwr_port_eco }
connect_supply_net gnd_net_eco -ports { gnd_port_eco }
set_domain_supply_net pd_eco -primary_power_net pwr_net_eco
-primary_ground_net gnd_net_eco
```

Command (*command.cmd*) file:

```
.global VDD VSS
.tran 800u 800u

.model D2A_STD D2A mode=std_logic
.model A2D_STD A2D mode=std_logic
.defhook D2A_STD A2D_STD

.model p2e hook mgc_ams:p2e(ams)
```

```
.model e2p hook mgc_ams:e2p(ams)
.defhook e2p p2e

.end
```

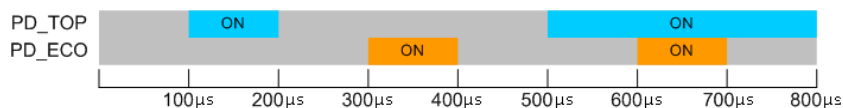
4. Run the simulation specifying the command file containing instructions for the simulation:

```
vasim tb -cmd command.cmd -pa -novopt -L mtiPA -pa_top /tb/top_inst
-do ms.do -ms -pa_allownovopt -outpath RES_MIXED_SIGNAL
```

5. A testbench, contained in the SystemVerilog file *tb_twoPDs.sv*, switches the power supply to the two power domains on and off in the following order:

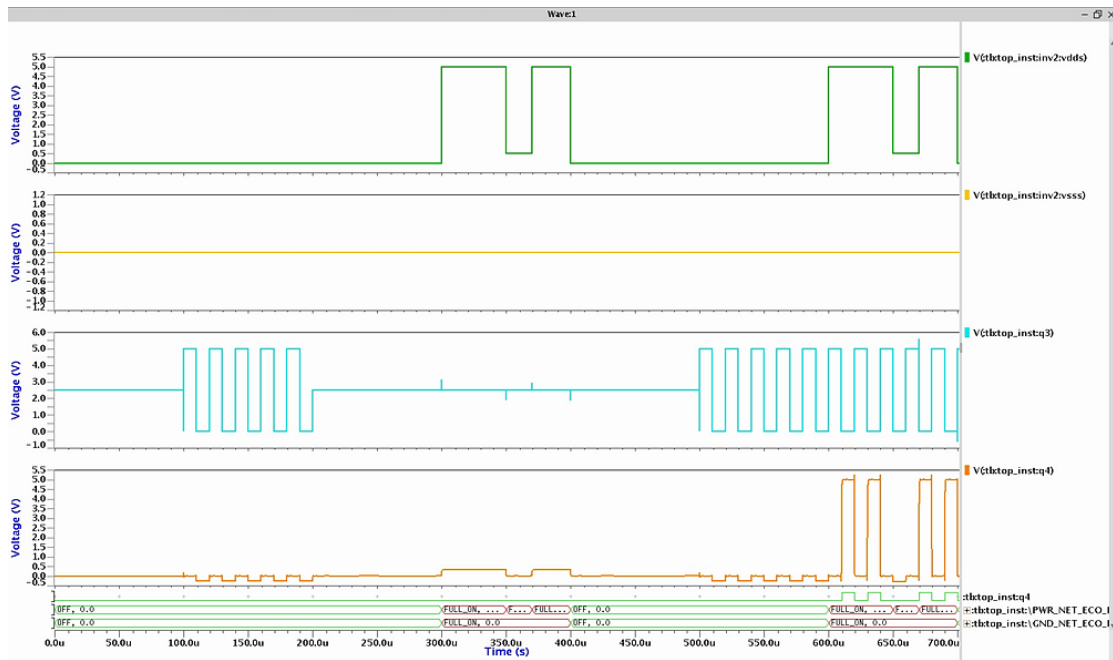
- From 0µs to 100µs: PD_TOP=OFF, PD_ECO=OFF
- From 100µs to 200µs: PD_TOP=ON, PD_ECO=OFF
- From 200µs to 300µs: PD_TOP=OFF, PD_ECO=OFF
- From 300µs to 400µs: PD_TOP=OFF, PD_ECO=ON
- From 400µs to 500µs: PD_TOP=OFF, PD_ECO=OFF
- From 500µs to 600µs: PD_TOP=ON, PD_ECO=OFF
- From 600µs to 700µs: PD_TOP=ON, PD_ECO=ON
- From 700µs to 800µs: PD_TOP=ON, PD_ECO=OFF

Figure 10-5: Power Domain Switching as Instructed by the Testbench



Results

INV2 is replaced by its SPICE equivalent and P2E converters are inserted.



Related Topics

[Questa ADMS Power Aware Simulation \(PAsim\)](#)

[Connecting Power to Analog Power Ports](#)

11. Using Questa ADMS with the Questa Verification Run Manager (VRM)

This chapter describes how to run Questa ADMS simulations under the control of the Questa Verification Run Manager (VRM).

Questa Verification Run Manager.....	11-1
Running Questa ADMS Simulations Under the VRM.....	11-1
Running Multiple Run Questa ADMS Simulations Under the VRM.....	11-2

Questa Verification Run Manager

The Questa Verification Run Manager (VRM) is a tool for compiling and running simulations under the control of a script, usually for regression purposes.

VRM runs are defined in one or more Run Manager Database (.rmdb) files. These are XML files. A .rmdb file typically contains one or more XML<...Script> tags. They define actions or jobs. There are several different types: <preScript>, <execScript>, <mergeScript>, <postScript>, <trriageScript>, <trendScript>, and <tplanScript>. They all have a common attribute called "launch" which specifies the script interpreter. Individual script commands are nested under the <...Script> ... </...Script> tags. between <command> ...</command> tags.

Refer to Verification Run Manager Overview in the *Questa Verification Run Manager User Guide* for more information.

Running Questa ADMS Simulations Under the VRM

You can run Questa ADMS simulations under the control of the Questa Verification Run Manager (VRM).

Refer to Verification Run Manager Overview in the *Questa Verification Run Manager User Guide* for more information.

Procedure

1. In the .rmdb file, set the launch attribute in the <...Script> tag to launch="vasim". This specifies vasim as the script interpreter.
2. Place Questa ADMS commands between <command>...</command> tags nested under the <...Script> ... </...Script> tags.
3. Save the .rmdb file as default.rmdb

4. Execute the VRM using

```
vrn -gui
```

Results

You can view the output from all simulations in the VRM GUI.

Examples

Here is an example of a .rmdb file to run a vasil script.

```
<?xml version="1.0" ?>
<rmdb version="1.0">
  <runnable name="mytest" type="task">
    <execScript launch="vasim">
      <command>file delete -force work</command>
      <command>vlib work</command>
      <command>vacom (%CONFDIR%)/code.vhdlams</command>
      <command>vasim -cmd (%CONFDIR%)/netlist.cir</command>
      <command>run -all</command>
      <command>quit -f</command>
    </execScript>
  </runnable>
</rmdb>
```

The builtin parameter CONFDIR points to the location of the RMDb file's parent folder.

Running Multiple Run Questa ADMS Simulations Under the VRM

You can run multiple run ADMS simulations from VRM. These perform one or more ADMS simulations as a single VRM job.

Refer to Verification Run Manager Overview in the *Questa Verification Run Manager User Guide* for more information.

Procedure

1. In the .rmdb file, set the launch attribute in the <...Script> tag to launch="vasim". This specifies vasil as the script interpreter.
2. Place Questa ADMS commands between <command>...</command> tags nested under the <...Script> ... </...Script> tags.
3. Save the .rmdb file is saved as default.rmdb

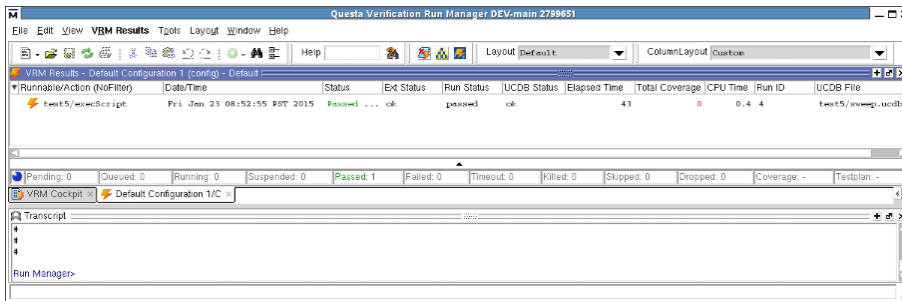
4. Execute the VRM using

```
vrunc -gui
```

Results

During the run, the Results window (Configuration tab) shows the status of the VRM job, along with statistics from the job run. The completion status of the job is shown in the Status column.

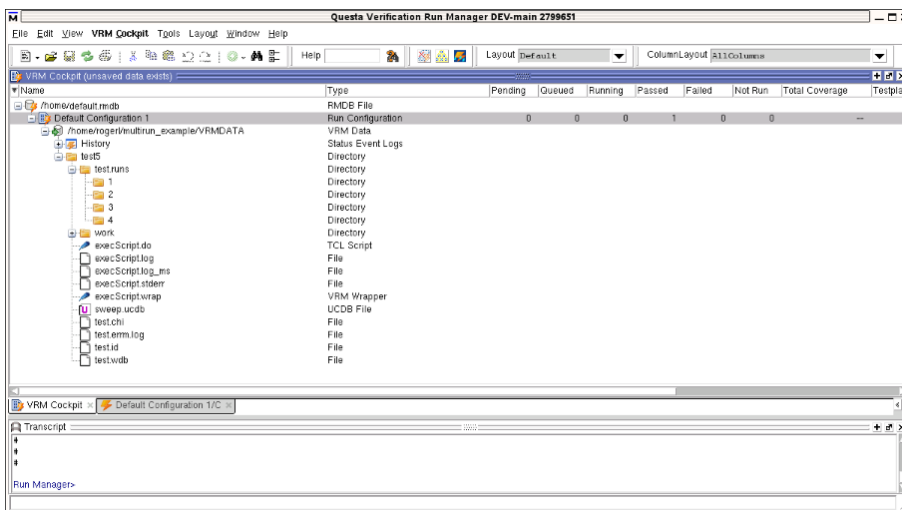
Figure 11-1: VRM Results Window



A VRM multiple run simulation only passes if none of the individual simulations fail. If the multiple run simulation shows a failed status, the Run ID column (not shown here) contains the sequence number of the first failed simulation run.

Following a multiple run simulation, you can view all simulation output and logs in the VRM Cockpit window.

Figure 11-2: VRM Cockpit Window



The VRMDATA entry contains all generated log files and simulation results. For multiple run simulations, there is a <top>.runs directory that contains the individual simulation results.

In multiple run simulations, `vasim` creates a directory for each individual simulation run. Individual runs of a multiple run simulation are sequentially numbered starting with 1. The name of the directory is the sequence number of the simulation run, under the alter block active, if any. If alter blocks are defined, then another level of subdirectories is created. These are named `Nominal`, `Alter_1`, `Alter_2`,... . The `Nominal` subdirectory contains the `Nominal` (unaltered) netlist results. Because alter names can contain illegal OS pathname characters, or can be empty, as in the case of an unnamed alter, they are mapped to a standard `Alter_n` name. A `README` file is created to map the actual alter block name to its corresponding directory.

As well as the VRM log files, the following output files are generated:

- A single JWDB file containing compound waveforms, allowing plotted signals from all the simulations to be viewed in Solido Waveform Analyzer.
- A unique UCDB file for each simulation, in a separate directory.
- A unique WLF for each simulation, in a separate directory.

Examples

Here is an example of a `.rmdb` file to run a multiple run `vasim` script.

```
<?xml version="1.0" ?>
<rmdb version="1.0" toprunnables="test5">
  <runnable name="test5" type="task">
    <parameters>
      <parameter name="ucdbfile">sweep.ucdb</parameter>
    </parameters>
    <execScript launch="vasim">
      <command>file delete -force work</command>
      <command>valib work</command>
      <command>vasim -cmd (%CONFDIR%)/test.cir -coverage -outpath
(%TASKDIR%)</command>
      <command>coverage save -onexit (%ucdbfile%)</command>
      <command>run -all</command>
      <command>quit -f</command>
    </execScript>
  </runnable>
</rmdb>
```

The `<execScript launch="vasim">` tag and the command tags nested in it create a work library, compile a VHDL file, and run the multiple run simulation. This is similar to a `vasim "do"` file.

The file `"netlist.cir"` referenced in the `vasim` command contains one or more sweep commands to specify the extent of the multiple run simulation.

The builtin parameter `TASKDIR` points to the location of the simulation run, under the `VRMDATA`.

The command coverage save -onexit (%ucdbfile%) placed before the command run -all means that a UCDB file is created for each individual simulation. (If it was placed after it, only one file, for the last individual simulation, would be created.)

12. Examples

Example data files are provided with Questa ADMS.

- Summary of the Questa ADMS Examples..... 12-2
- 12-Bit Analog to Digital Converter..... 12-3
- Mixed-Language Ring Oscillator..... 12-6
- C Code Encapsulation..... 12-9
- SPICE Inverter in a Top Verilog Design..... 12-11
- VHDL-AMS Description of a Phase Locked Loop..... 12-12
- Verilog-AMS Description of an Integrator..... 12-15
- Verilog-AMS Description of Phase Lock Loop..... 12-16
- Creating and Compiling a connectrules File..... 12-19
- SystemC and VHDL Description of a Flip-Flop Latch with a Top-Level SPICE Netlist..... 12-22
- Questa ADMS-ADiT Examples..... 12-24

Summary of the Questa ADMS Examples

The examples provided with Questa ADMS demonstrate a variety of mixed-signal modeling approaches. Each example includes a complete netlist of a circuit.

These examples may be found in the following location:

```
$MGC_AMS_HOME/examples/adms/
```

where MGC_AMS_HOME is the installation directory. To prepare and run the examples, copy the examples directory to your work area using the following command:

```
$ cp -r $MGC_AMS_HOME/examples/adms/* .
```

The following Questa ADMS examples are included in the package:

- **12-Bit Analog to Digital Converter**

A 12-Bit A2D converter model that demonstrates the simulation of a VHDL-AMS analog and mixed model, containing a VHDL instance.

- **Mixed-Language Ring Oscillator**

A mixed-signal ring oscillator is used to introduce the Verilog instance capability and the .DEFHOOK mechanism in Questa ADMS.

- **C Code Encapsulation**

C code encapsulation in VHDL-AMS.

- **SPICE Inverter in a Top Verilog Design**

A SPICE description of an ideal inverter instantiated in a Verilog design.

- **VHDL-AMS Description of a Phase Locked Loop**

VHDL-AMS structural description of a PLL model which displays the use of CommLib QuickStart VHDL-AMS models as building blocks.

- **Verilog-AMS Description of an Integrator**

Verilog-AMS description of an integrator.

- **Verilog-AMS Description of Phase Lock Loop**

Verilog-AMS mixed netlist description of a PLL. Demonstrates boundary element insertion, first using built-in boundary elements, then using Verilog-AMS connect rules.

- **SystemC and VHDL Description of a Flip-Flop Latch with a Top-Level SPICE Netlist**

SystemC-VHDL description of a Flip-Flop Latch with a Top-Level SPICE Netlist.

- **Questa ADMS-ADiT Examples**

Questa ADMS-ADiT with Verilog on top. A five-stage inverter chain with Verilog-on-top. The inverter chain is made up of Verilog and SPICE inverters.

- **Questa ADMS-ADiT with SPICE on Top**

Questa ADMS-ADiT with SPICE-on-top. A five-stage inverter chain with SPICE-on-top.

- **ADiT SPICE and Verilog Description of an Inverter Chain**

A SPICE-on-top design instantiating three SPICE inverters and two Verilog inverters. This example demonstrates how you can partition a SPICE instance to ADiT. Three top-SPICE designs are provided, each uses a different method of partitioning.

- **8-bit Adder Using AND, OR, and XOR VHDL VITAL Gates**

8-bit adder (structural description) using AND, OR and XOR VHDL VITAL gates or Verilog primitives.

- **8-bit Adder Using AND, OR, and XOR Verilog Primitives**

8-bit adder (structural description) using AND, OR, and XOR Verilog primitives.

- **Boundary Elements Insertion in a Mixed-Language Design**

Mixed-language, SPICE-on-top design demonstrating the automatic insertion of boundary elements between nodes of different types.

- **Verilog-AMS Amplifier with a Resolved wreal Input and an Electrical Output**

In this example, the resolution value of 2 digital drivers is input to a Verilog-AMS amplifier (with a gain of 2). The output is then transmitted through an electrical terminal.

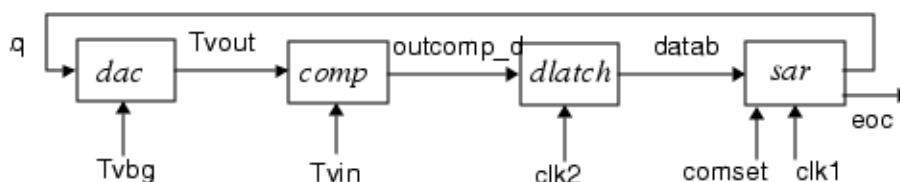
In addition, a set of behavioral VHDL-AMS, Verilog-AMS and Verilog-A models are provided. See [Modeling Examples](#).

12-Bit Analog to Digital Converter

You can demonstrate mixed-signal simulations, implicit library mapping, and replacement of design units.

The following example uses a 12-Bit successive-approximation analog to digital converter, as illustrated in [Figure 12-1](#)

Figure 12-1: ADC12 Example



The following Questa ADMS features are demonstrated in this example:

- Mixed-signal simulations in both interactive and batch mode (comment out the unrequired line in the *adc12_run* file).
- The use of implicit library mapping.
- The replacement of design units using the *.BIND* command.

Table 12-1: 12-Bit Analog to Digital Converter Example Files

Filename	Description
VHDL-AMS and VHDL Input Files	
<i>adc12-adms.vhd</i>	Top VHDL-AMS description using only VHDL-AMS instances
<i>adc12-adms-ms.vhd</i>	VHDL-AMS description using VHDL-AMS and VHDL instances
<i>adc12-all-adms.vhd</i>	VHDL-AMS description for Questa ADMS simulation
<i>adc12-all-ms.vhd</i>	VHDL description for pure digital simulation
<i>adc12-eldo.vhd</i>	Top VHDL-AMS description using SPICE subcircuits
<i>adc12-eldo-ms.vhd</i>	Top VHDL-AMS description using SPICE subcircuit and digital instances
<i>adc12-ms.vhd</i>	VHDL units simulated by Questa SIM
SPICE Input Files	
<i>adc12_tr.cir</i>	Pure analog description in SPICE
<i>adc12test_eldo_mixed_ms.cir</i>	Mixed description in SPICE
<i>adc12_dac.ckt</i>	DAC subcircuit written in SPICE
Command and Control Files	
<i>adc12test-ms-alone.do</i>	Questa SIM command file for pure VHDL simulation
<i>adc12test.do</i>	Questa ADMS command file for pure VHDL-AMS simulation
<i>adc12test-adms-ms.do</i>	Questa ADMS command file for “source_ms” example
<i>adc12test-ms-elab.do</i>	Questa SIM command file used in <i>adc12test-adms-ms.do</i>
<i>adc12test-ms-end.do</i>	Questa SIM command file used in <i>adc12test-adms-ms.do</i>
<i>adc12-ms.do</i>	Questa ADMS command file for <i>adc12test_eldo_mixed_ms.cir</i>

Table 12-1: 12-Bit Analog to Digital Converter Example Files (continued)

Filename	Description
<i>adc12_tr.do</i>	Questa ADMS command file for <i>adc12_tr.cir</i>
<i>adc12test.cmd</i>	SPICE command file for pure VHDL-AMS simulation
<i>adc12test_mixed_eldo.cmd</i>	SPICE command file
<i>adc12test_mixed_eldo_ms.cmd</i>	SPICE command file
<i>adc12test_mixed_eldo_ms_bind.cmd</i>	SPICE command file with .BIND syntax example
<i>adc12test_mixed_ms.cmd</i>	SPICE command file
<i>adc12test_mixed_ms_link.cmd</i>	SPICE command file
<i>adc12.pag</i>	Page layout file with simulation comparison
<i>modelsim.ini</i>	Questa SIM system initialization file
<i>modelsim.tcl</i>	Questa SIM GUI preferences file
<i>adc12_run</i>	Runs all the test cases of the example

Procedure

To run the *adc12* example, change to the directory *<your_work_area>/examples/adms/examples/adms/adc12*, then enter the following command:

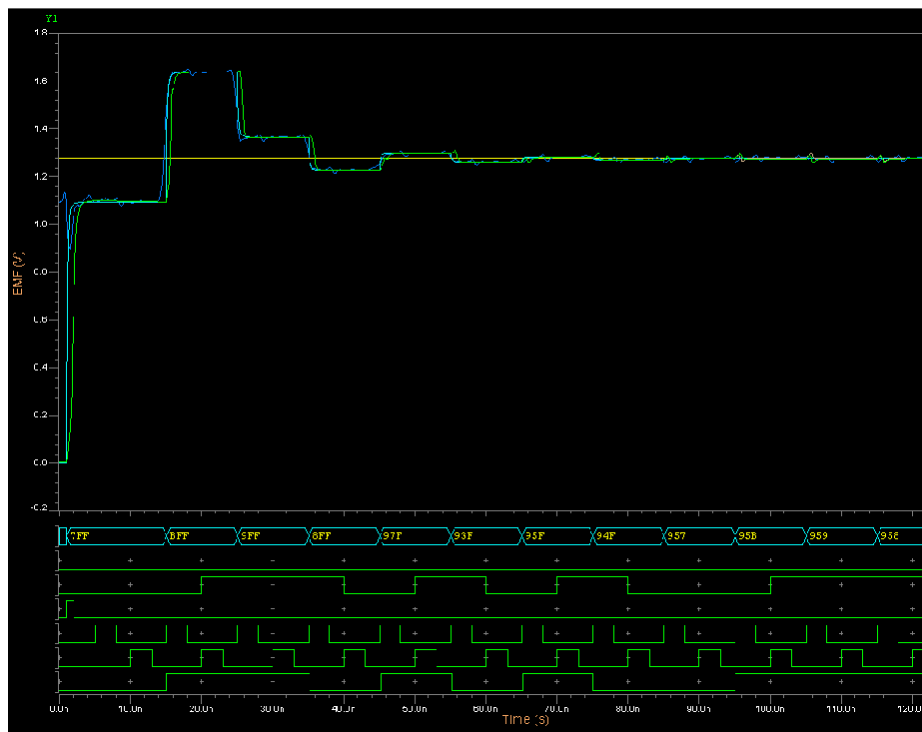
```
source adc12_run
```

A script to launch this example on a Windows platform is provided in the file, *run_adc12.bat*.

Results

The mixed simulation results visualized with the **Wave Window (Solido Waveform Analyzer)** are shown in **Figure 12-2**.

Figure 12-2: Analog and Digital Results



Related Topics

Examples

Tutorial 1: Simulating a Mixed-Signal Design in Interactive Mode [Getting Started with Questa ADMS]

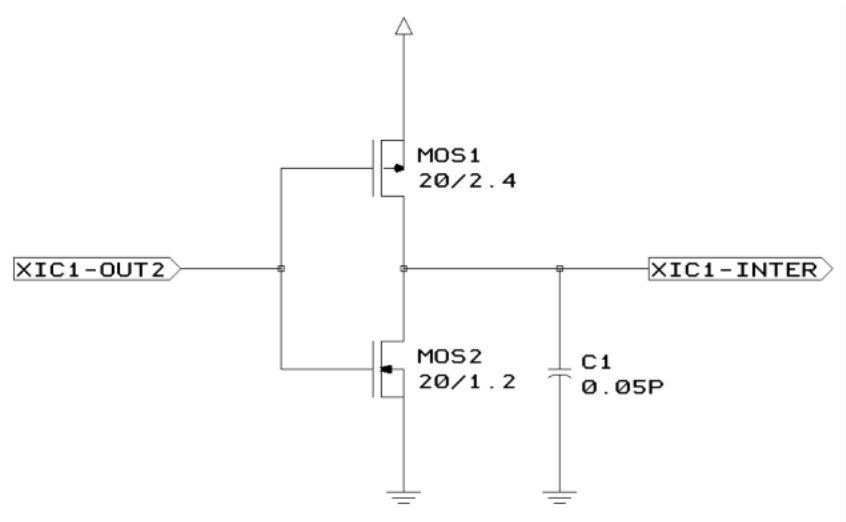
Mixed-Language Ring Oscillator

A ring oscillator model demonstrates how to mix different languages.

The ring oscillator is written using inverters in VHDL-AMS, Verilog and SPICE. The analog subcircuit *invana*, shown in [Figure 12-3](#) is instantiated in the SPICE file *oscmos_invana.ckt*:

```
xnot3 out2 out3 invana
```

Figure 12-3: Analog Subcircuit “invana”



A global .DEFHOOK mechanism is used to define the interface elements at the digital/analog boundaries:

```
.DEFHOOK a2d_eldo_bit d2a_eldo_bit
```

Table 12-2: Ring Oscillator Example Files

Filename	Description
VHDL-AMS and VHDL Input Files	
<i>converters.vhd</i>	VHDL-AMS models for Std_logic/electrical conversions
<i>econtrol.vhd</i>	VHDL-AMS description of the control source
<i>oscmos.vhd</i>	VHDL-AMS description for digital parts
<i>oscmos_adms.vhd</i>	VHDL-AMS description for the inverter
<i>oscmos_entity.vhd</i>	VHDL pure digital entity
<i>oscmos_modelsim.vhd</i>	VHDL pure digital architecture
Verilog Input File	
<i>oscmos.v</i>	Verilog Nand and inverter
SPICE Input Files	
<i>mixed_eldo_vhdlams_vhdl_verilog_defhook_eldo.cir</i>	Mixed description in SPICE: using VHDL-AMS and Verilog instances, and .DEFHOOK using predefined Eldo boundary elements
<i>oscmos_invana.ckt</i>	Contains analog subcircuit “invana”

Table 12-2: Ring Oscillator Example Files (continued)

Filname	Description
Command and Control Files	
<i>do_top_eldo.do</i>	Questa ADMS command file
<i>oscmos_run</i>	Runs all the test cases of the example

Procedure

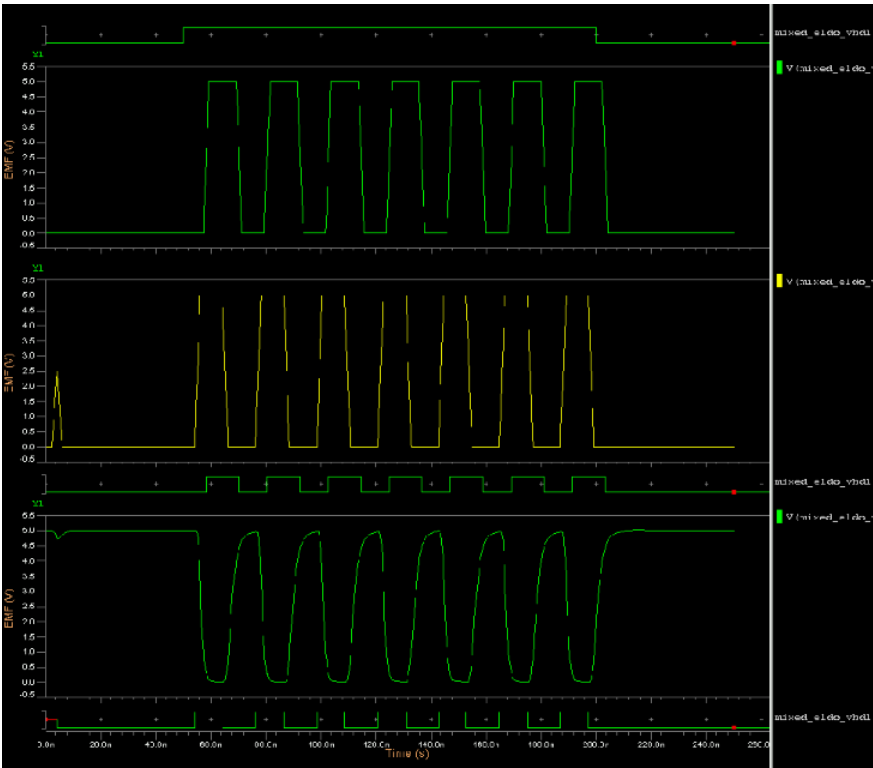
To run the oscmos example, change to the directory <your_work_area>/examples/adms/oscmos, then enter the following command:

```
source oscmos_run
```

Results

Figure 12-4 shows the simulation results, displayed in Solido Waveform Analyzer.

Figure 12-4: Analog and Digital Results



Related Topics

Controlling Boundary Element Assignment

C Code Encapsulation

Examples are provided that demonstrate how to instantiate C code in Questa ADMS.

The examples directory `c` contains a set of examples are provided that demonstrate how to instantiate C code in Questa ADMS.

Table 12-3: C Code Encapsulation Examples

Example Number <number>	Directory	Description
1.1	<i>c/example1.1</i>	The c code is encapsulated in a VHDL-AMS function with a SCALAR CONSTANT parameter of mode IN.
1.2	<i>c/example1.2</i>	The c code is encapsulated in a VHDL-AMS procedure with a SCALAR VARIABLE parameter of mode IN and a SCALAR VARIABLE parameter of mode OUT.
2.1	<i>c/example2.1</i>	The c code is encapsulated in: • AVHDL-AMS procedure with a SCALAR CONSTANT parameter of mode IN and a SCALAR VARIABLE parameter of mode OUT. • A VHDL-AMS procedure with a SCALAR CONSTANT parameter of mode IN and a SCALAR VARIABLE parameter of mode INOUT.
3.1	<i>c/example3.1</i>	The c code is encapsulated in a VHDL-AMS function with a 1-dimensional ARRAY CONSTANT parameter of mode IN.
3.2	<i>c/example3.2</i>	The c code is encapsulated in a VHDL-AMS procedure with a 1-dimension ARRAY VARIABLE parameter of mode IN and a 1-dimension ARRAY VARIABLE parameter of mode OUT.
4.1	<i>c/example4.1</i>	The c code is encapsulated in: • A VHDL-AMS procedure with a 1-dimension ARRAY CONSTANT parameter of mode IN and a 1-dimension ARRAY VARIABLE parameter of mode OUT. • A VHDL-AMS procedure with a 1-dimension ARRAY CONSTANT parameter of mode IN and a 1-dimension ARRAY VARIABLE parameter of mode INOUT.

Table 12-3: C Code Encapsulation Examples (continued)

Example Number <number>	Directory	Description
IO.1	<i>c/exampleIO.1</i>	The C function "table" is encapsulated in the VHDL-AMS function "read"; the VHDL-AMS function returns the number of data contained in the file "table.data".
IO.2	<i>c/exampleIO.2</i>	The C function "table" is encapsulated in the VHDL-AMS function "read"; the VHDL-AMS function returns the values of the data contained in the file "table.data".

Information about launching the scripts is provided in the "readme" files associated with each example. For example, *example1.1_run* on UNIX and *example1.1_run.bat* on Windows.

Prerequisites

- For C Code encapsulation, the LD_LIBRARY_PATH variable must be correctly set. In the provided examples, it is done using the following command:

```
LD_LIBRARY_PATH=`/bin/sh $MGC_AMS_HOME/com/ams_runtime_setup.ksh
LD_LIBRARY_PATH`

export LD_LIBRARY_PATH
```

On Linux systems, you can use:

```
export LD_LIBRARY_PATH=<path_name>
```

Procedure

- (Optional) To return the path of the AMS provided gcc compiler, enter the following command:

```
$MGC_AMS_HOME/com/ams_runtime_setup.ksh AMS_GCC_HOME
```

- To run an example, change to the directory *<your_work_area>/examples/adms/c/example<number>*, then enter the following command:

```
source example<number>_run
```

Where *<number>* is one of the numbers listed in [Table 12-3](#).

Related Topics

C Code Encapsulation
Examples

SPICE Inverter in a Top Verilog Design

You can instantiate a SPICE description of an ideal inverter in a digital Verilog design.

This example demonstrates how a SPICE description of an ideal inverter is instantiated in a digital Verilog design. The description of the boundary between the SPICE subcircuit and the Verilog top design is described using either a Verilog or VHDL-AMS file. The .DEFHOOK command will be used to insert A2D and D2A boundary elements.

To run the inverter example, go to the<your_work_area>/examples/adms/inverter directory and type:

```
source run_test_<selected_language>_description
```

Where <selected_language> is either verilog or vhdlams, allowing you to choose whether to use a Verilog or VHDL-AMS description of the boundary.

A script to launch this example on a Windows platform is provided in the file, *run_inverter.bat*.

Table 12-4: SPICE Inverter Example Files

Filename	Description
Verilog Input Files	
<i>top.v</i>	The Verilog description that defines the top of the design. This model contains the stimuli and the SPICE subcircuit instantiation.
<i>inverter.v</i>	A Verilog module, inv, that describes the boundary between the SPICE subcircuit and the Verilog-on-top description.
VHDL-AMS Input File	
<i>inv_entity.vhd</i>	Boundary between the inverter subcircuit (SPICE) and the Verilog-on-top module described in VHDL-AMS
SPICE Input File	
<i>inv.ckt</i>	CMOS inverter SPICE subcircuit.
Command and Control Files	
<i>dofile</i>	Questa ADMS and Questa SIM command file.

Table 12-4: SPICE Inverter Example Files (continued)

Filename	Description
<i>test.cmd</i>	The SPICE command file containing instructions for the simulation. The .DEFHOOK statement defines implicit A2D/D2A. A transient analysis is then carried out over 10 μ s.
<i>run_test_verilog_description</i>	Runs the test case using the Verilog description of the boundary between the inverter SPICE subcircuit and the Verilog-on-top module.
<i>run_test_vhdlams_description</i>	Runs the test case using the VHDL-AMS description of the boundary between the inverter SPICE subcircuit and the Verilog-on-top module.

Tip

For a step by step walk through of this example, refer to [Tutorial 4: Instantiating SPICE into a Verilog-on-Top Design](#) in *Getting Started with Questa ADMS*.

Related Topics**Examples**

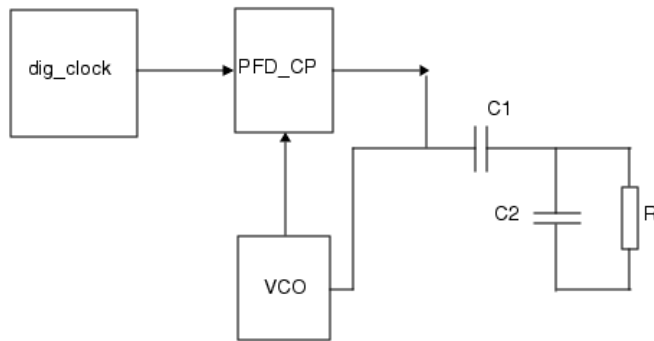
VHDL-AMS Description of a Phase Locked Loop

Questa ADMS provides a simple phase locked loop example that demonstrates CommLib QuickStart VHDL-AMS models, structural VHDL-AMS description, and SPICE-on-Top features.

The following example uses a structural VHDL-AMS description of a simple phase locked loop (PLL). It is built up from the following models, available in the AMS library located in *\$MGC_AMS_HOME/libraries/amslib*:

- VCO_d - Voltage Controlled Oscillator (VCO) with digital output
- PFD_CP - Phase Frequency Detector with charge pump
- dig_clock - Clock Generator

Figure 12-5 illustrates the models used to create the PLL and their connections.

Figure 12-5: Phase Locked Loop Block Diagram

The following Questa ADMS features are demonstrated in this example:

- CommLib QuickStart VHDL-AMS models
- Structural VHDL-AMS description
- SPICE-on-Top

Table 12-5: VHDL-AMS Phase Locked Loop Example Files

Filename	Description
PLL.vhd	Top VHDL-AMS file using VHDL-AMS instances
PLL.cir	PLL model instantiation and Loop filter implementation using SPICE primitives
PLL.do	Questa ADMS command file for PLL.cir
PLL_run	Runs all the test cases of the example

Procedure

To run the amslib_pll example, change to the directory `<your_work_area>/examples/adms/amslib_pll`, then enter the following command:

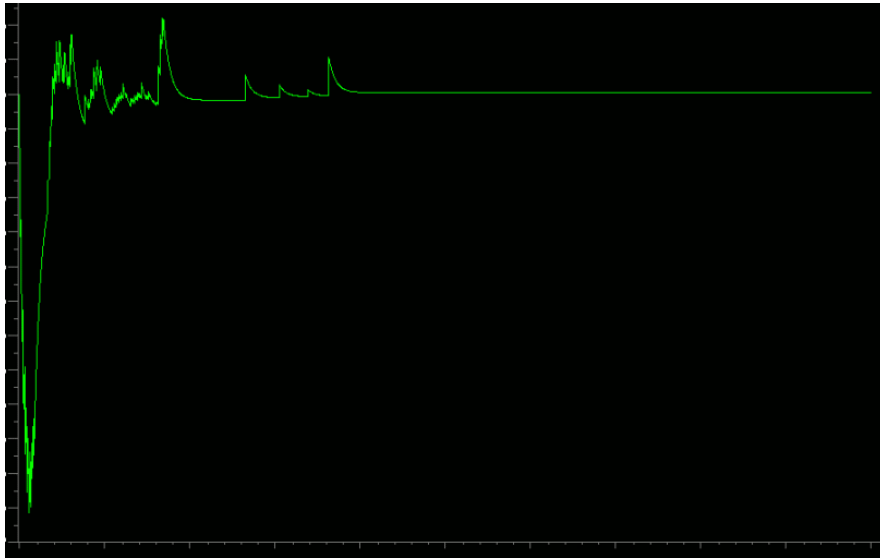
```
source PLL_run
```

Results

The simulation results visualized with the Solido Waveform Analyzer GUI are shown in the figures below: (output waveforms ctrl, Ref, VCO).

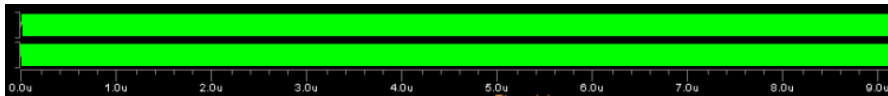
This is the voltage on the ctrl port. This port is the output of the PFD block and the input to the VCO_d block. This voltage reflects the phase error between the ref and vco signals.

Figure 12-6: CTRL Waveform



You can see the ref and vco signals. The PLL should synchronize the vco signal with the ref signal.

Figure 12-7: REF and VCO Digital Waveforms

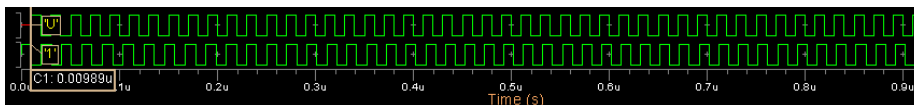


If you zoom into the digital results waveforms you will notice that at the start of the simulation the two signals vco and ref are out of phase but will become more and more synchronized as the value of $V(ctrl)$ becomes closer to zero.

Zoom into the period between 0 and 9 μs

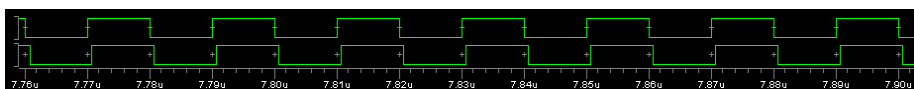
Figure 12-8 shows how ref and vco are originally completely out of phase:

Figure 12-8: REF and VCO Digital waveforms—Zoom 1



Zoom in to between 7.7 and 7.9 μs . **Figure 12-9** shows how the two signals are more synchronized later in the simulation (as $V(ctrl)$ approaches zero):

Figure 12-9: REF and VCO Digital waveforms—Zoom 2



Related Topics

Examples

Verilog-AMS Description of an Integrator

You can instantiate an integrator in a Verilog-AMS top design.

In the following example, a Verilog-AMS description of an integrator is instantiated in a Verilog-AMS top design. The integrator module described in the Verilog-AMS file *integrator.va* contains two ports of type “voltage”, one as input, the other as output. The integrator operation is performed through a branch contribution.

To run the integrator example, go to the<your_work_area>/examples/adms/examples/adms/integrator directory and type:

```
source run_integrator
```

A script to launch this example on a Windows platform is provided in the file, *run_integrator.bat*.

Table 12-6: Verilog-AMS Integrator Example Files

Filename	Description
<i>top.va</i>	Verilog-AMS description of the top module that instantiates the integrator module. Also includes the sinusoidal voltage stimuli defined in a behavioral description block.
<i>integrator.va</i>	Verilog-AMS description of the integrator
<i>dofile_integrator</i>	Questa ADMS command file containing the run -all command
<i>top.cmd</i>	SPICE command file. Instructs a transient analysis to be performed over 10ms
<i>run_integrator</i>	Compiles the Verilog-AMS source files and runs the simulation using the command files.

Related Topics

Examples

Verilog-AMS Description of Phase Lock Loop

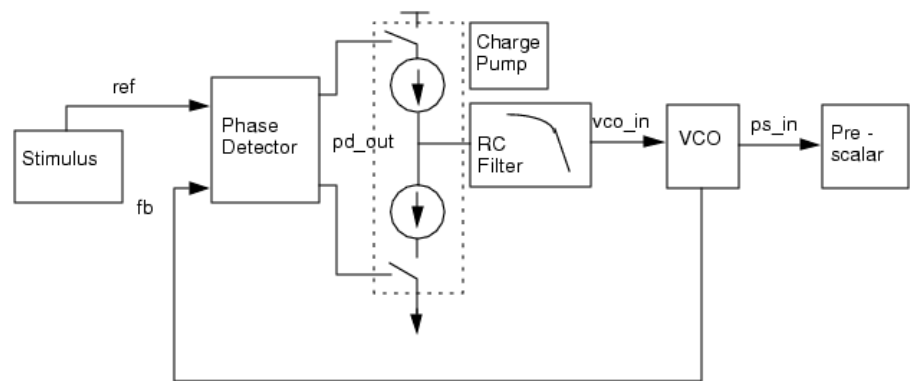
A SPICE-on-top example comprising a Verilog-D stimulus module connecting to a Verilog-AMS phase locked loop (PLL) is provided with Questa ADMS.

The following example uses a SPICE-on-top design, that instantiates a stimulus Verilog-D module connecting to a Verilog-AMS phase locked loop (PLL). All the source files are compiled, the simulation is run, and results are obtained for the voltage node FLT for the RC Filter. The second part of the example demonstrates how to use a Connect Rules file to redefine the boundary elements.

The Verilog-AMS PLL is a mixed netlist instantiating the following modules:

- Verilog-D Phase Detector
- Verilog-AMS Charge Pump
- Verilog-AMS RC Filter
- Verilog-AMS VCO
- Verilog-D Prescaler

Figure 12-10: Verilog-AMS Phase Locked Loop Example



The files used in this example are listed in the following table:

Table 12-7: Verilog-AMS Phase Locked Loop Example Files

Filename	Description
<i>stim.v</i>	Verilog-D description of stimulus
<i>pd.v</i>	Verilog-D description of phase detector
<i>cp.va</i>	Verilog-AMS description of charge pump
<i>lpf.va</i>	Verilog-AMS description of RC filter

Table 12-7: Verilog-AMS Phase Locked Loop Example Files (continued)

Filename	Description
<i>vco.va</i>	Verilog-AMS description of voltage-controlled oscillator (VCO)
<i>ps.v</i>	Verilog-D description of prescaler
<i>pll.va</i>	Verilog-AMS description of the phase lock loop
<i>pll.cmd</i>	SPICE command file for mixed simulation
<i>run_adms</i>	Questa ADMS command file for <i>pll.cmd</i>
<i>compile_all.csh</i>	Compilation file for all Verilog-AMS input files

Procedure

1. In a terminal window, run the file *compile_all.csh* to compile all the modules in the design:

```
source compile_all.csh
```

Notice how the *valog* command is a single compile command for both Verilog-D and Verilog-AMS modules.

2. Load the design, specifying the command file *pll.cmd*:

```
vasim -cmd pll.cmd
```

The command file *pll.cmd* contains the following SPICE netlist commands:

- .MODEL statements to define an A2D and a D2A boundary element:

```
.MODEL a2d_default A2D MODE=std_logic vth=2.5
.MODEL d2a_default D2A MODE=std_logic vlo=0.0 vhi=5.0
+ trise=1p tfall=1p
```

- A .DEFHOOK statement defines implicit A2D/D2A:

```
.DEFHOOK a2d_default d2a_default
```

- The following .MODEL statements define Verilog-AMS modules STIM_VERILOG and PLL_VERILOG:

```
.MODEL STIM_VERILOG MACRO LANG=veriloga LIB=pll_lib
.MODEL PLL_VERILOGA MACRO LANG=veriloga LIB=pll_lib
```

These models are then instantiated in the following Y instances:

```
Y_STIM STIM_VERILOG
+ PORT : REF

Y_PLL PLL_VERILOGA
+ PORT : REF FLT
```

REF and FLT are mapped ports of the instantiated models.

- A transient analysis is then carried out over 90µs, and the results are plotted:

```
.TRAN 0 90us
*.PLOT TRAN V(FLT)

.OPTION hmin=700ps

.end
```

3. In the Structure window, double-click the boundary elements log file, *pll.conv* to view the boundary elements that have been inserted:

```
NET :pll:y_pll:div_out
  BIDIR
  DtoA CONVERTER D2A_DEFAULT ELDO ["MODE = STD_LOGIC" "VLO = 0" "VH1
= 5" "TRISE = 1e-12" "TFALL = 1e-12"]
    DEFHOOK /home/examples/adms/verilogams_pll/pll.cmd(6)
    DISCIPLINE ELECTRICAL
    TYPE IEEE:STD_LOGIC_1164.STD_LOGIC
    PORTS :pll:y_pll:i_PS:ps_out;
    WAVEFORM NAME :pll:y_pll:div_out#defhook#bidir#0
  AtoD CONVERTER A2D_DEFAULT ELDO ["MODE = STD_LOGIC" "VTH = 2.5"]
    DEFHOOK /home/examples/adms/verilogams_pll/pll.cmd(6)
    DISCIPLINE ELECTRICAL
    TYPE IEEE:STD_LOGIC_1164.STD_LOGIC
    PORTS :pll:y_pll:i_PD:fb;
    WAVEFORM NAME :pll:y_pll:div_out#defhook#0
END NET;

NET :pll:y_pll:pd_out
  DtoA CONVERTER D2A_DEFAULT ELDO ["MODE = STD_LOGIC" "VLO = 0" "VH1
= 5" "TRISE = 1e-12" "TFALL = 1e-12"]
    DEFHOOK /home/examples/adms/verilogams_pll/pll.cmd(6)
    DISCIPLINE ELECTRICAL
    TYPE IEEE:STD_LOGIC_1164.STD_LOGIC
    PORTS :pll:y_pll:i_PD:pd_out;
    WAVEFORM NAME :pll:y_pll:pd_out
END NET;

NET :pll:y_pll:pll_out
  AtoD CONVERTER A2D_DEFAULT ELDO ["MODE = STD_LOGIC" "VTH = 2.5"]
    DEFHOOK /home/examples/adms/verilogams_pll/pll.cmd(6)
```



```

DISCIPLINE ELECTRICAL
TYPE IEEE:STD_LOGIC_1164.STD_LOGIC
PORTS :pll:y_pll:i_PS:ps_in;
WAVEFORM NAME :pll:y_pll:pll_out
END NET;

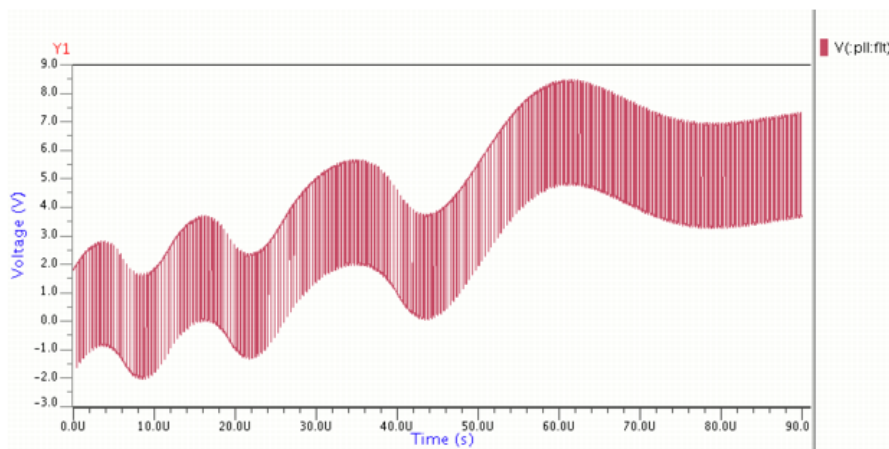
NET :pll:ref
  BIDIR
  DtoA CONVERTER D2A_DEFAULT ELDO ["MODE = STD_LOGIC" "VLO = 0" "VH1
= 5" "TRISE = 1e-12" "TFALL = 1e-12"]
  DEFHOOK /home/examples/adms/verilogams_pll/pll.cmd(6)
  DISCIPLINE ELECTRICAL
  TYPE IEEE:STD_LOGIC_1164.STD_LOGIC
  PORTS :pll:y_stim:ref;
  WAVEFORM NAME :pll:ref#defhook#bidir#0
  AtoD CONVERTER A2D_DEFAULT ELDO ["MODE = STD_LOGIC" "VTH = 2.5"]
  DEFHOOK /home/examples/adms/verilogams_pll/pll.cmd(6)
  DISCIPLINE ELECTRICAL
  TYPE IEEE:STD_LOGIC_1164.STD_LOGIC
  PORTS :pll:y_pll:i_PD:ref;
  WAVEFORM NAME :pll:ref#defhook#0
END NET;

```

4. Run the simulation by choosing **Simulate > Run > Run -All**.

Results

Figure 12-11: Results Using .DEFHOOK Command



When the simulation completes, the Wave Window (SOLIDO Waveform Analyzer) displays the plot for the voltage at node FLT for the RC filter ([Figure 12-10](#)). The plot shows the voltage varying with time.

Creating and Compiling a connectrules File

Connect rules can be used instead of the .MODEL command to select boundary elements.

In this part of the example, you will modify the SPICE command file (*pll.cmd*) remove the existing boundary element specification, and create a connect rules file to define the A2D and D2A interface.

When a connect rules file is used, an instance of a connect module will be automatically inserted when signals and ports with continuous (analog) and discrete (digital) disciplines are connected together. A connect specification contains the following information:

- The name of the boundary element to be inserted and its parameters
- Whether it is used for a D2A or and A2D boundary
- The names of the disciplines (one continuous, one discrete) at the boundary.

Procedure

1. Create a connectrules file in the Source Window. Choose **File > New > Source > Other** to open a blank file, and add the following lines:

```
connectrules myRules;
  connect $a2d #(.vth1(2.5), .vth2(2.5), .vth(2.5));
  connect $d2a #(.trise(1p), .tfall(1p), .vlo(0.0), .vhi(5));
endconnectrules
```

2. Select **File > Save As** to save the file in the current directory. Specify *connectrules.vams* as the filename.
3. Compile the file using valog in the Transcript Window.

```
valog connectrules.vams
```

4. Double-click the command file (*pll.cmd*) in the **Structure Window**, to open it for editing in the **Source Window**.

To ensure that you have write access, select **Source** from the main menu and uncheck the **Read Only** option.

- a. Comment out the following lines to remove the existing boundary element selections:

```
*.MODEL a2d_default A2D MODE=std_logic vth=2.5
*.MODEL d2a_default D2A MODE=std_logic vlo=0.0 vhi=5.0
*+ trise=1p tfall=1p

*.DEFHOOK a2d_default d2a_default
```

- b. Add the following line to specify the connectrules file you just created:

```
.CONNECTRULES myRules
```

- c. Save the command file.

5. Restart the simulation using the restart command:

```
restart
```

6. In the Structure window, double-click the boundary elements log file, *pll.conv* to view the boundary elements that have been inserted:

```
NET :pll:y_pll:div_out
  BIDIR
  DtoA CONVERTER model_d2a_std_myRules_1_connectrules ELD0 []
    CONNECTRULE myRules
    DISCIPLINE ELECTRICAL
    TYPE IEEE:STD_LOGIC_1164.STD_LOGIC
    PORTS :pll:y_pll:i_PS:ps_out;
    WAVEFORM NAME :pll:y_pll:div_out#defhook#bidir#0
  AtoD CONVERTER model_a2d_std_myRules_0_connectrules ELD0 []
    CONNECTRULE myRules
    DISCIPLINE ELECTRICAL
    TYPE IEEE:STD_LOGIC_1164.STD_LOGIC
    PORTS :pll:y_pll:i_PD:fb;
    WAVEFORM NAME :pll:y_pll:div_out#defhook#0
END NET;

NET :pll:y_pll:pd_out
  DtoA CONVERTER model_d2a_std_myRules_1_connectrules ELD0 []
    CONNECTRULE myRules
    DISCIPLINE ELECTRICAL
    TYPE IEEE:STD_LOGIC_1164.STD_LOGIC
    PORTS :pll:y_pll:i_PD:pd_out;
    WAVEFORM NAME :pll:y_pll:pd_out
END NET;

NET :pll:y_pll:pll_out
  AtoD CONVERTER model_a2d_std_myRules_0_connectrules ELD0 []
    CONNECTRULE myRules
    DISCIPLINE ELECTRICAL
    TYPE IEEE:STD_LOGIC_1164.STD_LOGIC
    PORTS :pll:y_pll:i_PS:ps_in;
    WAVEFORM NAME :pll:y_pll:pll_out
END NET;

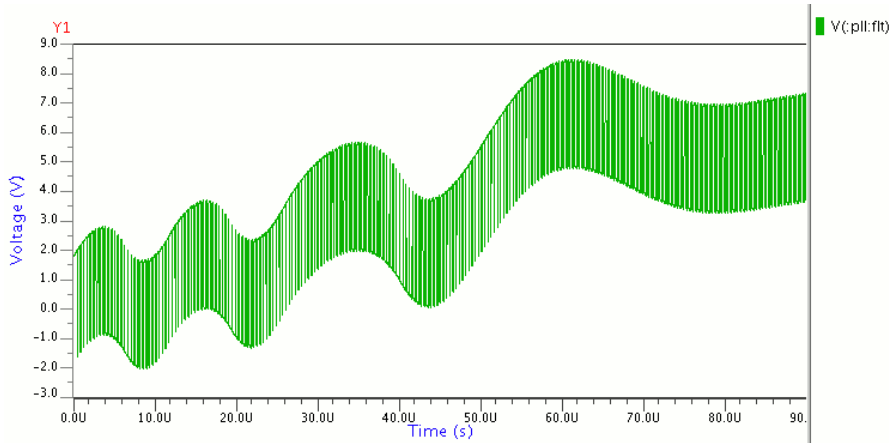
NET :pll:ref
  BIDIR
  DtoA CONVERTER model_d2a_std_myRules_1_connectrules ELD0 []
    CONNECTRULE myRules
    DISCIPLINE ELECTRICAL
    TYPE IEEE:STD_LOGIC_1164.STD_LOGIC
    PORTS :pll:y_stim:ref;
    WAVEFORM NAME :pll:ref#defhook#bidir#0
  AtoD CONVERTER model_a2d_std_myRules_0_connectrules ELD0 []
    CONNECTRULE myRules
    DISCIPLINE ELECTRICAL
    TYPE IEEE:STD_LOGIC_1164.STD_LOGIC
    PORTS :pll:y_pll:i_PD:ref;
```

```
WAVEFORM NAME :pll:ref#defhook#0
END NET;
```

7. Run the simulation by selecting **Simulate > Run > Run -All**.

The results obtained when replacing the boundary elements using connect rules are shown in [Figure 12-12](#). Note that they are very similar to those obtained using the .MODEL and .DEFHOOK commands ([Figure 12-11](#)).

Figure 12-12: Results Using Connect Rules



Related Topics

Examples

Connect Specification Statements

SystemC and VHDL Description of a Flip-Flop Latch with a Top-Level SPICE Netlist

A SystemC flip-flop latch example is provided with Questa ADMS.

Tip

Refer to SystemC Simulation in the *Questa SIM User's Manual* for more information.

The following example uses a SystemC design of a flip-flop latch that generates a signal sensitive to the positive edge of a clock. The flip-flop latch is instantiated in a VHDL unit.

Table 12-8: SystemC Flip-Flop Latch Example Files

Filename	Description
<i>clkgenerator.h</i>	SystemC clock generator description of the module ckgen.
<i>clkgenerator.cpp</i>	SystemC exported clock generator module. Exports ckgen using the SC_MODULE_EXPORT macro.
<i>tflipflop.h</i>	SystemC flip-flop latch description, containing the module tff.
<i>tflipflop.cpp</i>	SystemC exported flip-flop latch module. Exports tff using the SC_MODULE_EXPORT macro
<i>clkgenerator.vhd</i>	VHDL unit instantiating the SystemC clock generator. Contains the VHDL component declaration for instantiating ckgen. The component declaration and the instantiation of the clock generator, ckgen, is done in the VHDL model vhckgen(simp).
<i>tflipflop.vhd</i>	VHDL unit instantiating the SystemC flip-flop latch. Contains the VHDL component declaration for instantiating tff. The component declaration and the instantiation of the flip-flop, tff, is done in the VHDL model vhtff(simp).
<i>top_spice.cir</i>	Top-Level SPICE netlist instantiating the two VHDL units, vhckgen(simp) and vhtff(simp)
<i>dofile</i>	Questa ADMS and Questa SIM command file. Contains commands to plot waveforms and run the simulation.
<i>run.sh</i>	Questa ADMS command file. Compiles the source files and runs the simulation using <i>dofile</i> .
<i>README</i>	Description of the example

Procedure

To compile and run the systemc example, change to the directory `<your_work_area>/examples/adms/examples/systemc`, then enter the following command:

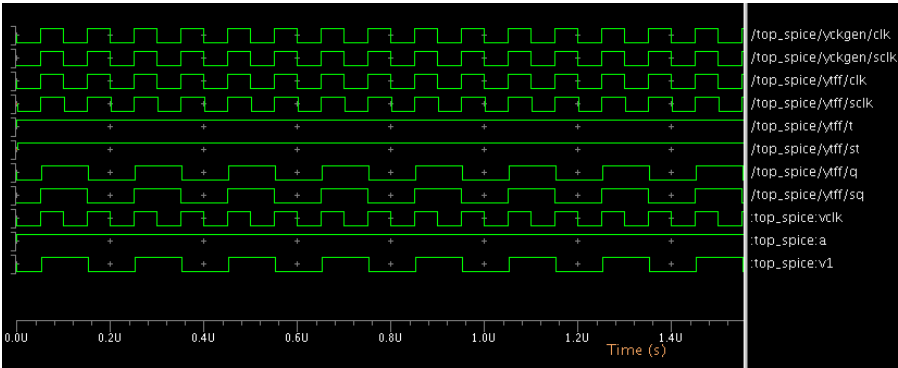
```
source run.sh
```

A script to launch this example on a Windows platform is provided in the file *run_systemc.bat*.

Results

The simulation output shows the flip-flop changing its output, vtff:q, on the rising edge of the clock, in effect dividing the clock frequency by 2. Signals at other ports are also displayed.

Figure 12-13: SystemC Example Simulation Results



Related Topics

Examples

Questa ADMS-ADiT Examples

This section demonstrates Questa ADMS-ADiT designs.

Inverter Chain with Verilog-on-Top.....12-24

Questa ADMS-ADiT with SPICE on Top.....12-25

ADiT SPICE and Verilog Description of an Inverter Chain..... 12-27

SPICE and Verilog Inverter Chain Example Source Files..... 12-31

8-bit Adder Using AND, OR, and XOR VHDL VITAL Gates..... 12-34

8-bit Adder Using AND, OR, and XOR Verilog Primitives..... 12-37

Boundary Elements Insertion in a Mixed-Language Design..... 12-38

Verilog-AMS Amplifier with a Resolved wreal Input and an Electrical Output.....12-41

Modeling Examples..... 12-43

Inverter Chain with Verilog-on-Top

A five-stage inverter chain example made up of Verilog and SPICE inverters is provided with Questa ADMS.

The design used in this example has Verilog-on-top.

Figure 12-14: Inverter Chain

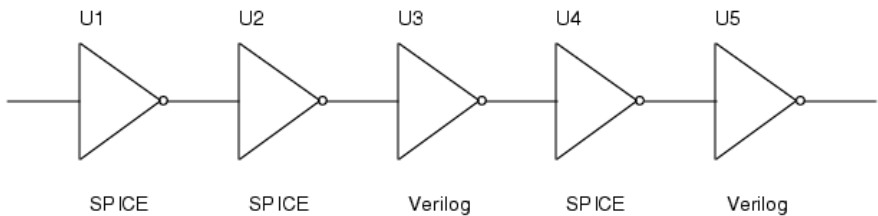


Table 12-9: Inverter Chain Example Files

Filename	Description
<i>admsadit_verilogontop.sp</i>	SPICE netlist containing the partitioning commands, subcircuit definitions, boundary element definitions, boundary element assignments and post processing and analysis commands.
<i>c018lv.l</i>	Library file called by SPICE .LIB command
<i>top.v</i>	Top-Level Verilog module
<i>inv.v</i>	Verilog inverter. Contains the Verilog <i>inv_spice</i> module and the SPICE inverter that will be associated to it.
<i>supply.v</i>	Verilog supply
<i>ams.do</i>	Questa ADMS do file. contains the run -all command, the command runs the simulation for 30ns as specified in the SPICE netlist.
<i>compile_n_run.sh</i>	Runs the example as a batch of commands

Procedure

To compile and run the dig-on-top example, change to the directory *<your_work_area>/examples/adms/examples/adit/dig-on-top*, then enter the following command:

```
source compile_n_run.sh
```

Related Topics

Examples

Questa ADMS-ADiT with SPICE on Top

A five stage inverter chain example with SPICE-on-top is delivered with Questa ADMS.

This example uses the five stage inverter chain with SPICE-on-top, shown in [Figure 12-14](#).

The files for this example are located in the following directory:

```
$MGC_AMS_HOME/examples/adms/adit/spice-on-top/
```

Once you have copied this directory into your work area, compile and run the spice-on-top example, by navigating to the directory:

```
<your_work_area>/examples/adms/examples/adit/spice-on-top
```

and type:

```
source compile_n_run.sh
```

Table 12-10: Inverter Chain With SPICE On Top Example Files

Filename	Description
<i>admsadit_spiceontop.sp</i>	SPICE netlist. Contains the partitioning commands, design hierarchy connectivity, subcircuit definitions, boundary element definitions, boundary element assignments and post processing and analysis commands.
<i>c018lv.l</i>	Library file called by SPICE .LIB command
<i>inv.v</i>	Verilog inverter inv_verilog
<i>ams.do</i>	Questa ADMS command do file. Contains the run -all command, the command runs the simulation for 30ns as specified in the SPICE netlist (on the .TRAN command)
<i>compile_n_run</i>	Runs the example as a batch of commands

The file *admsadit_spiceontop.sp* contains the following command:

```
.LIB 'c018lv.l' typ
.TEMP 27
.OPTION compat

*Top-level netlist connectivity:

X1 vdd vin s1 inv_spice
X2 vdd s1 s2 inv_spice
Y3 inv_verilog PORT: vdd s2 s3
X4 vdd s3 s4 inv_spice
Y5 inv_verilog PORT: vdd s4 ring_out

vvdd vdd 0 1.8
vin vin 0 pw1(0 0 10n 0 12n 1.8 30n 1.8 32n 0.0 60n 0.0 62n 1.8 150n 1.8)

.SUBCKT inv_spice vdd in out
m1 out in vdd vdd pch w=10e-6 l=0.18e-6
m2 out in 0 0 nch w=5e-6 l=0.18e-6
c0 out 0 1fF
.ENDS

*Model card to define the Verilog inverter model to be instantiated from
*SPICE:

.MODEL inv_verilog macro lang=verilog mod=inv_verilog

*ADMS A2D and D2A boundary elements. These are required to allow correct
```



```

*conversion from the analog domain to the digital domain and vice-versa:
.DEFHOOK a2d_def d2a_def
.MODEL d2a_def d2a mode=std_logic vhi=1.8 vlo=0.0
.MODEL a2d_def a2d mode=std_logic vth=0.9

*Post-processing and analysis commands:
.MEASURE tran freq1 TRIG v(ring_out) VAL=0.9 TD=20e-9 RISE=1
+ TARG v(ring_out) VAL=0.9 TD=20e-9 RISE=2
.MEASURE tran freq2 TRIG V(ring_out) VAL=0.9 TD=90e-9 RISE=1 TARG
+ v(ring_out) VAL=0.9 TD=90e-9 RISE=2
.plot tran v(ring_out) v(s1) v(s2) v(s3) v(s4) v(s5) v(s6) v(vdd) v(vin)

.TRAN 1n 30n
.END

```

The file *inv.v* contains the following commands:

```

`timescale 1 ns / 1 ns
module inv_verilog(vdd,in,out);
    input vdd,in;
    output out;
    assign #5 out = ~in;
endmodule

```

The file *ams.do* contains the following command:

```
run -all
```

Related Topics

Examples

ADiT SPICE and Verilog Description of an Inverter Chain

You can partition a SPICE instance to ADiT using netlist commands or the Questa ADMS GUI.

The circuit is a SPICE-on-top design instantiating three SPICE inverters and two Verilog inverters. The example demonstrates how you can partition a SPICE instance to ADiT using the Questa ADMS GUI or netlist commands.

Summary of Questa ADMS features used in this example

- SPICE inverters
- ADiT partitioning
- Verilog instance in SPICE

Two top-SPICE designs are provided, each uses a different method of partitioning.

Prerequisites

The files for this example are located in the following directory:

`$MGC_AMS_HOME/examples/adms/adit/partitioning`

Refer to [SPICE and Verilog Inverter Chain Example Source Files](#) for more information.

Procedure

1. To partition the instance using the Questa ADMS GUI, create a working library and compile the Verilog inverter:

```
valib work
vlog inverter.v
```

2. Load the design that performs ADiT partitioning through the Questa ADMS GUI, type the following `vasim` command:

```
vasim -cmd adit_gui.cir
```

3. Three SPICE subcircuits exist in the Top SPICE tree in the Structure window, XINV1_1, XINV1_3 and XINV1_5. The default simulator is Fast SPICE. Change them to ADiT using one of the following three methods. With the required instance selected so that it is highlighted:
 - a. Select the ADiT button  on the toolbar.
 - b. Right-click on the instance and select **Simulator > ADiT** from the pop up menu.
 - c. Choose **File > ADiT** from the menu bar.
4. Choose **Simulate > Run > Run-All** from the menu bar in the Questa ADMS Main window. The simulation runs to completion.
5. Alternatively, to partition the instance using ADiT markers, type the following `vasim` command:

```
vasim -cmd adit_marker.cir -do dofile_adms_adit
```

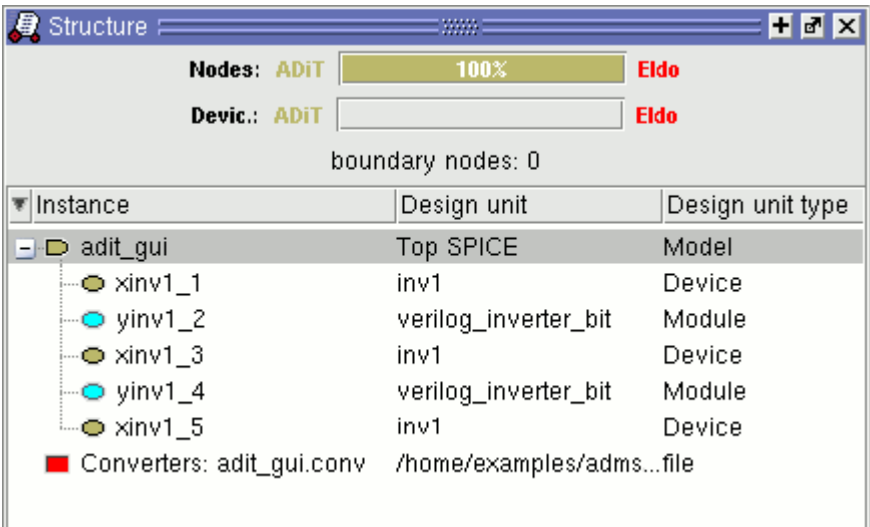
The simulation is run to completion. The results are exactly the same as when partitioning was done using the Questa ADMS GUI.

Prior to simulation, the Structure window displays the partitioning as 100% ADiT. However, this changes after simulation because the requested partitioning does not conform to the partitioning rules. If you expand Top SPICE in the Structure window, the XINV1_1, XINV1_3 and XINV1_5 instances are marked for processing by ADMS ADiT.

Results

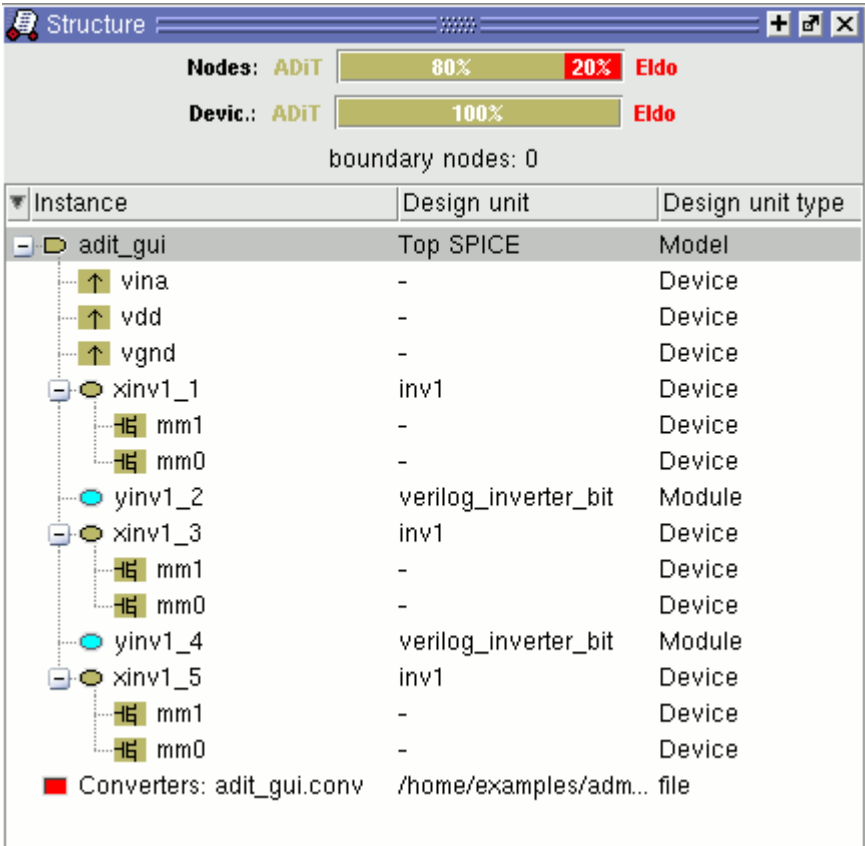
The initial partitioning shown in the Structure window is as [Figure 12-15](#). Note that before simulation, the Structure window displays the partitioning you requested. However, this may change after simulation if the requested partitioning does not conform to the partitioning rules.

Figure 12-15: Partitioning Before Simulation



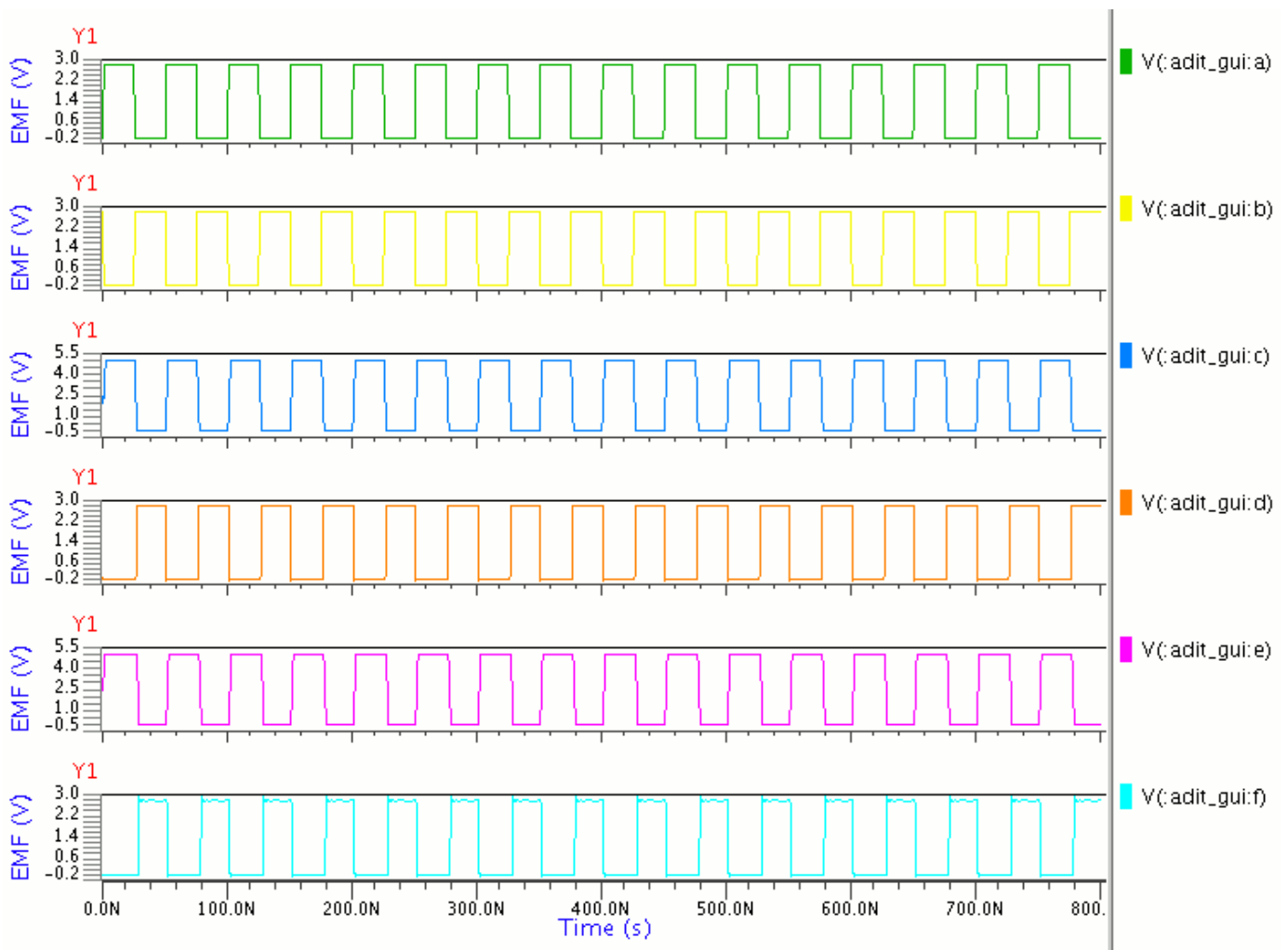
After simulation the partitioning shown in the Structure window changes to that as shown in [Figure 12-16](#).

Figure 12-16: Partitioning After Simulation



Simulation results are the same for each method of partitioning and are as shown in [Figure 12-17](#).

Figure 12-17: Simulation Results



Related Topics

Structure Window

SPICE and Verilog Inverter Chain Example Source Files

Files are provided for the example “ADiT SPICE and Verilog Description of an Inverter Chain”.

The files for the example [ADiT SPICE and Verilog Description of an Inverter Chain](#) are located in the following directory:

`$MGC_AMS_HOME/examples/adms/adit/partitioning`

Table 12-11: SPICE and Verilog Inverter Chain Example Source Files

Filename	Description
<i>adit_marker.cir</i>	Top SPICE description, partitioning performed using the adit marker.
<i>adit_gui.cir</i>	Top SPICE description, partitioning performed using the Questa ADMS GUI.
<i>spice_netlist.cir</i>	SPICE description containing model definitions and subcircuit definitions.
<i>inverter.v</i>	Verilog description of an inverter
<i>dofile_adms_adit</i>	Questa ADMS command file. Loads the Structure Window and Objects Window and runs the simulation.

adit_marker.cir

This netlist uses adit markers on the SPICE instances to select the partitioning. The netlist contains three instances of the SPICE inverter subcircuit INV and two instances of the Verilog inverter verilog_inverter_bit. The SPICE instances XINV1_1, XINV1_3, XINV1_5 are partitioned to ADiT as can be seen below:

```
XINV1_1 A B INV1 (adit)
YINV1_2 verilog_inv B C
XINV1_3 C D INV1 (adit)
YINV1_4 verilog_inv D E
XINV1_5 E F INV1 (adit)
.INCLUDE spice_netlist.cir
```

adit_gui.cir

This netlist does not specify any partitioning, the partitioning is left until you use the Questa ADMS GUI.

```
XINV1_1 A B INV1
YINV1_2 verilog_inv B C
XINV1_3 C D INV1
YINV1_4 verilog_inv D E
XINV1_5 E F INV1
.INCLUDE spice_netlist.cir
```

spice_netlist.cir

This is the SPICE netlist that contains the SPICE models, SPICE subcircuits, and so on. Excerpts from the netlist are shown here.

To instantiate the Verilog inverter, the module must be declared on the .MODEL command:

```
.MODEL verilog_inv macro lang=verilog mod=verilog_inverter_bit
```

A2D and D2A boundary elements are required because the design connects SPICE inverters to the Verilog inverter. The model declarations for the boundary elements are:

```
.MODEL a2d_std_logic a2d mode=std_logic  
.MODEL d2a_std_logic d2a mode=std_logic
```

The SPICE inverter subcircuit is:

```
.SUBCKT INV1 IN OUT  
MM1 OUT IN GND GND ENMM9 W=0.8u L=0.35u M=1  
MM0 OUT IN VDD VDD EPMM9 W=2.4u L=0.35u M=1  
.ENDS INV1  
  
VINA A 0 PULSE (0 2.8 0 1n 1n 24n 50n)
```

The built-in SPICE boundary elements are assigned using the .DEFHOOK command. This instructs Questa ADMS to automatically insert the specified boundary elements where either an analog to digital or a digital to analog boundary exists:

```
.DEFHOOK a2d_bit d2a_bit
```

The waveforms for each node of the inverter chain are plotted in the [Wave Window \(Solido Waveform Analyzer\)](#):

```
.PLOT tran V(A) V(B) V(C) V(D) V(E) V(F)
```

The netlist performs a transient analysis with a duration of 800ns:

```
.TRAN 1n 800n
```

inverter.v

This is the Verilog description of the inverter module. This module contains two ports, one input and one output:

```
module verilog_inverter_bit (inp, outp);  
    parameter del = 2;  
    input inp;  
    output outp;
```

```

    not #del noti (outp, inp);
endmodule // verilog_inverter_bit

```

8-bit Adder Using AND, OR, and XOR VHDL VITAL Gates

You can use AND, OR, and XOR VHDL VITAL gates to make an 8-bit adder.

The following example is an 8-bit adder (structural description) using AND, OR, and XOR VHDL VITAL gates.

```

SUM = A xor B xor CIN
COUT = (A and B) or ((A or B) and CIN)

```

Gates components to build the one-bit adder:

```

component or2
    generic(tpd_a_y, tpd_b_y: DelayTypeXX := 0 ns);
    port(a,b: in bit; y: out bit);
end component;

component and2
    generic(tpd_a_y, tpd_b_y: DelayTypeXX := 0 ns);
    port(a,b: in bit; y: out bit);
end component;

component xor2
    generic(tpd_a_y, tpd_b_y: DelayTypeXX := 0 ns);
    port(a,b: in bit; y: out bit);
end component;

```

According to the IEEE 1076.4 specification, VHDL generic name and SDF construct mapping are as follows:

- tpd_a_y:

```
(IOPATH a y (1:2:3))
```

- tpd_b_y:

```
(IOPATH b y (1:2:3))
```


Procedure

1. Create a library:

```
% valib work
```

2. Compile the models:

```
% vcom gates.vhd adder.vhd testadder.vhd
```

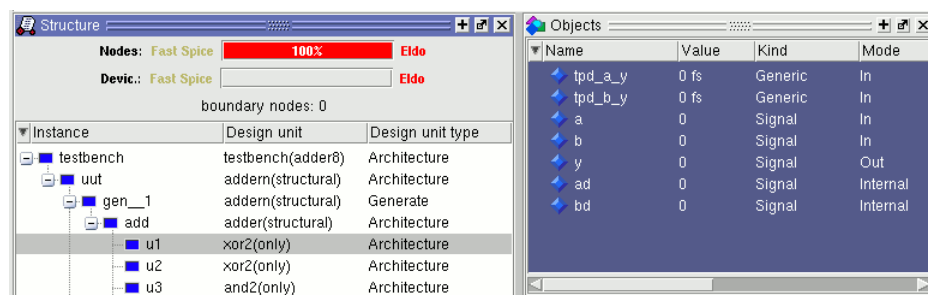
3. Simulate without SDF timing annotation:

```
% vasim testbench adder8
```

4. Run the following commands from the [Transcript Window](#):

```
view structure objects
add wave *
run 1000 ns
```

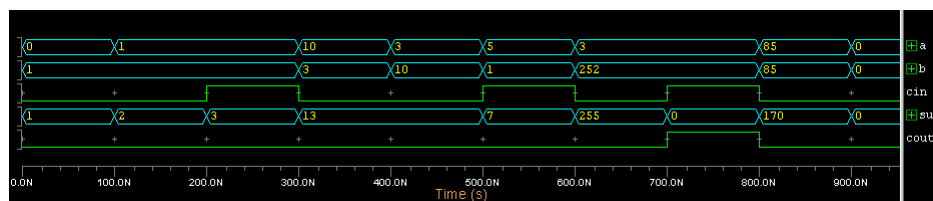
Figure 12-18: Structure and Objects Windows (VHDL) Without Propagation Delay



- The testbench is VHDL.
- Propagation delays (tpd_a_y and tpd_b_y) for gate instances (xor2, and2, and or2) are 0 fs (default).

Results without propagation delay, tpd = 0 fs (unsigned format):

Figure 12-19: Simulation Results—tpd=0ns



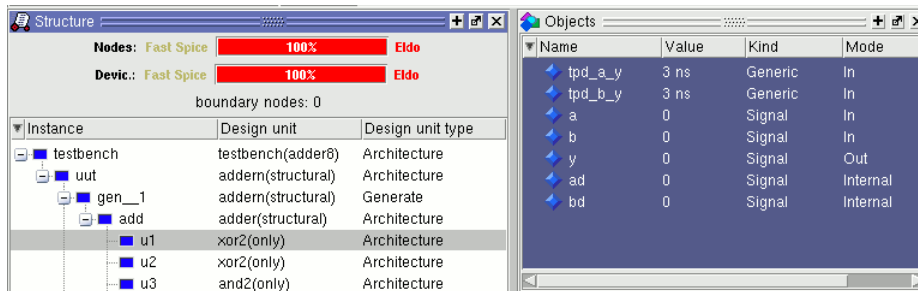
5. Simulate with SDF annotation using the *gates.sdf* file and max specification delay. Repeat the previous steps with timing backannotations:

```
% vasim testbench adder8 -sdfmax :testbench=gates.sdf
```

Note

If the PrefReuse(WaveConfig) variable in the Solido Waveform Analyzer configuration file is set to tcl or swd, the wave configuration is retained from the previous simulation, so there is no need to run the add wave * command again (doing so will result in duplicate wave forms in the Solido Waveform Analyzer viewer). Refer to [GUI Save and Reuse Preference Variables](#) for more information.

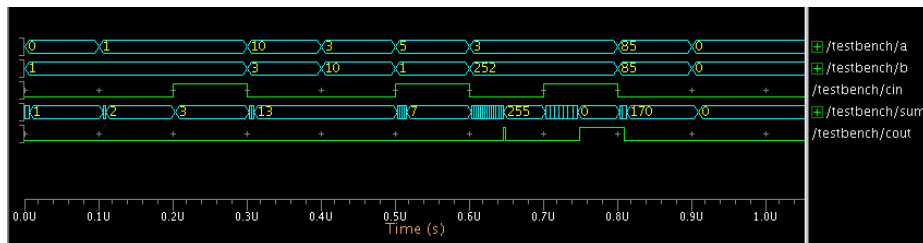
Figure 12-20: Structure and Objects Windows (VHDL) With Propagation Delay



- Propagation delays tpd_a_y and tpd_b_y for VHDL VITAL gates and2, or2, and xor2 are set to maximum delay specified in the SDF file gates.sdf (3 ns).

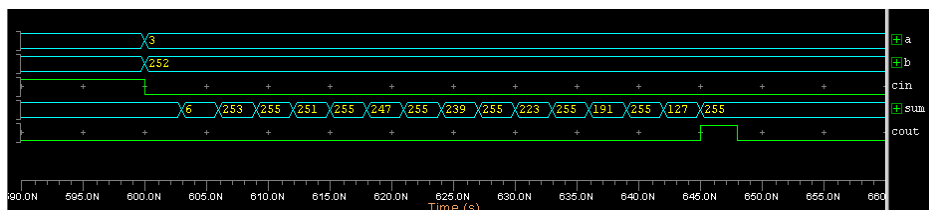
Results with max delay, tpd = 3ns (unsigned format):

Figure 12-21: Simulation Results—tpd=3ns



6. Click the Time axis and drag out a zoom range from 590 ns to 660 ns:

Figure 12-22: Simulation Results—tpd=3ns (zoom)



Related Topics

Examples

8-bit Adder Using AND, OR, and XOR Verilog Primitives

You can use AND, OR, and XOR Verilog primitives to make an 8-bit adder.

The following example is an 8-bit adder (structural description) using AND, OR, and XOR Verilog primitives.

```
SUM = A xor B xor CIN
COUT = (A and B) or ((A or B) and CIN)
```

SDF constructs are matched to Verilog constructs as follows:

```
xor u1 (y,a,b) ⇔ (DEVICE y (1:2:3))
```

Procedure

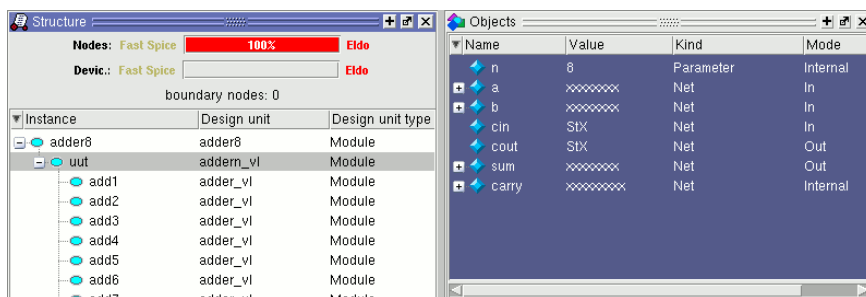
1. Compile the Verilog models for adder and adder8:

```
% vlog adder.v testadder_vl.v
```

2. Simulate without delay annotation:

```
% vasil adder8
```

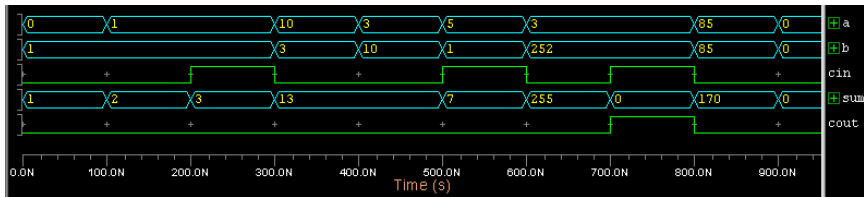
Figure 12-23: Structure and Objects Windows (Verilog)



- The testbench (testadder_vl) is a VHDL one with std_logic type to be able to map Verilog module addern_vl with Verilog type (X01Z).ls
- The adder is using Verilog primitives AND, XOR, and OR to build the adder.
- Default delay is 0 fs.

The results are the same as the VHDL description (unsigned format). Refer to [8-bit Adder Using AND, OR, and XOR VHDL VITAL Gates](#) for more information.

Figure 12-24: Simulation Results—tpd=0ns (Verilog)

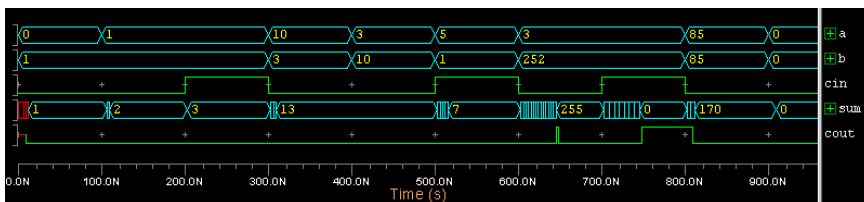


3. Simulate with maximum delay annotation from the *gates.sdf* file:

```
% vasil adder8 -sdfmax :adder8:uut=gates.sdf
```

Results with timing set to max delay, 3ns (unsigned format):

Figure 12-25: Simulation Results—tpd=3ns (Verilog)



You will notice some undefined state at the beginning due to propagation delay and `std_logic` type (equivalent to Verilog type).

Boundary Elements Insertion in a Mixed-Language Design

Questa ADMS automatically places a suitable boundary element between the two points when two nodes of different types are connected in a circuit.

This tutorial demonstrates that when two nodes of different types are connected in a circuit (analog terminal and digital port) Questa ADMS automatically places a suitable boundary element between the two points. The design used in this example is a mixed-language design with SPICE-on-top.

Both D-latches have been designed with the same characteristics although they are written in different languages: Verilog and VHDL. They each have an input `d` which is connected to the net `data_in`, and an output `q`, and a second output `qb`, which is the inverse of `q`. Both devices are driven simultaneously by the clock pulse. The comparator is a Verilog-AMS model.

- *vlog_test.cir* - Mixed description in SPICE using VHDL-AMS and Verilog-AMS instances, and `.DEFHOOK` using predefined boundary elements
- *dlatch_vlog.v* - Verilog D-latch

- *dlatch_bhv.vhd* - VHDL architecture for D-latch
- *ent_dlatch.vhd* - VHDL entity for D-latch
- *comp.va* - Verilog-AMS comparator

Procedure

1. From the UNIX command line, run the file *run.sh*, which creates and maps a library, compiles the models into the library, and starts the simulator:

```
vlib MODLIB
vmap work MODLIB

vlog comp.va
vlog dlatch_vlog.v
vcom ent_dlatch.vhd dlatch_bhv.vhd

vasim &
```

The Questa ADMS main window, and the [Start Simulation Dialog Box](#) open.

2. In the Start Simulation dialog box, select Eldo as the Top Design.
3. From the File menu select the file *vlog_test.cir*, and then click **Load**.

In this file, the `.MODEL` command is used to declare four built-in boundary elements of modes bit and `std_logic` for A2D and D2A conversion:

```
.MODEL a2d_eldo_std_logic a2d mode=std_logic
.MODEL d2a_eldo_std_logic d2a mode=std_logic
.MODEL a2d_eldo_bit a2d mode=bit
.MODEL d2a_eldo_bit d2a mode=bit
```

A global assignment is made for these boundary elements using the `.DEFHOOK` command:

```
.DEFHOOK a2d_eldo_std_logic d2a_eldo_std_logic
+ a2d_eldo_bit d2a_eldo_bit
```

4. From the [Structure Window](#), double-click the file *Converters: vlog_test.conv*. This loads the boundary elements log file into the [Source Window](#).

```
NET :vlog_test:data_in
  AtoD CONVERTER A2D_ELDO_STD_LOGIC ELDO ["MODE = STD_LOGIC"]
  DEFHOOK /home/demos/examples/adms/boundary_elements/
vlog_test.cir(33)
  DISCIPLINE IEEE:ELECTRICAL_SYSTEMS.ELECTRICAL
  TYPE IEEE:STD_LOGIC_1164.STD_LOGIC
  PORTS :vlog_test:y_1:d;
  WAVEFORM NAME :vlog_test:data_in#defhook#0
```

```

        AtoD CONVERTER A2D_ELD0_BIT ELD0 ["MODE = BIT"]
        DEFHOOK /home/demos/examples/adms/boundary_elements/
vlog_test.cir(33)
        DISCIPLINE IEEE:ELECTRICAL_SYSTEMS.ELECTRICAL
        TYPE STD:STANDARD.BIT
        PORTS :vlog_test:y_2:d;
        WAVEFORM NAME :vlog_test:data_in#defhook#1
END NET;

NET :vlog_test:clk
        AtoD CONVERTER A2D_ELD0_STD_LOGIC ELD0 ["MODE = STD_LOGIC"]
        DEFHOOK /home/demos/examples/adms/boundary_elements/
vlog_test.cir(33)
        DISCIPLINE IEEE:ELECTRICAL_SYSTEMS.ELECTRICAL
        TYPE IEEE:STD_LOGIC_1164.STD_LOGIC
        PORTS :vlog_test:y_1:clk;
        WAVEFORM NAME :vlog_test:clk#defhook#0
        AtoD CONVERTER A2D_ELD0_BIT ELD0 ["MODE = BIT"]
        DEFHOOK /home/demos/examples/adms/boundary_elements/
vlog_test.cir(33)
        DISCIPLINE IEEE:ELECTRICAL_SYSTEMS.ELECTRICAL
        TYPE STD:STANDARD.BIT
        PORTS :vlog_test:y_2:clk;
        WAVEFORM NAME :vlog_test:clk#defhook#1
END NET;

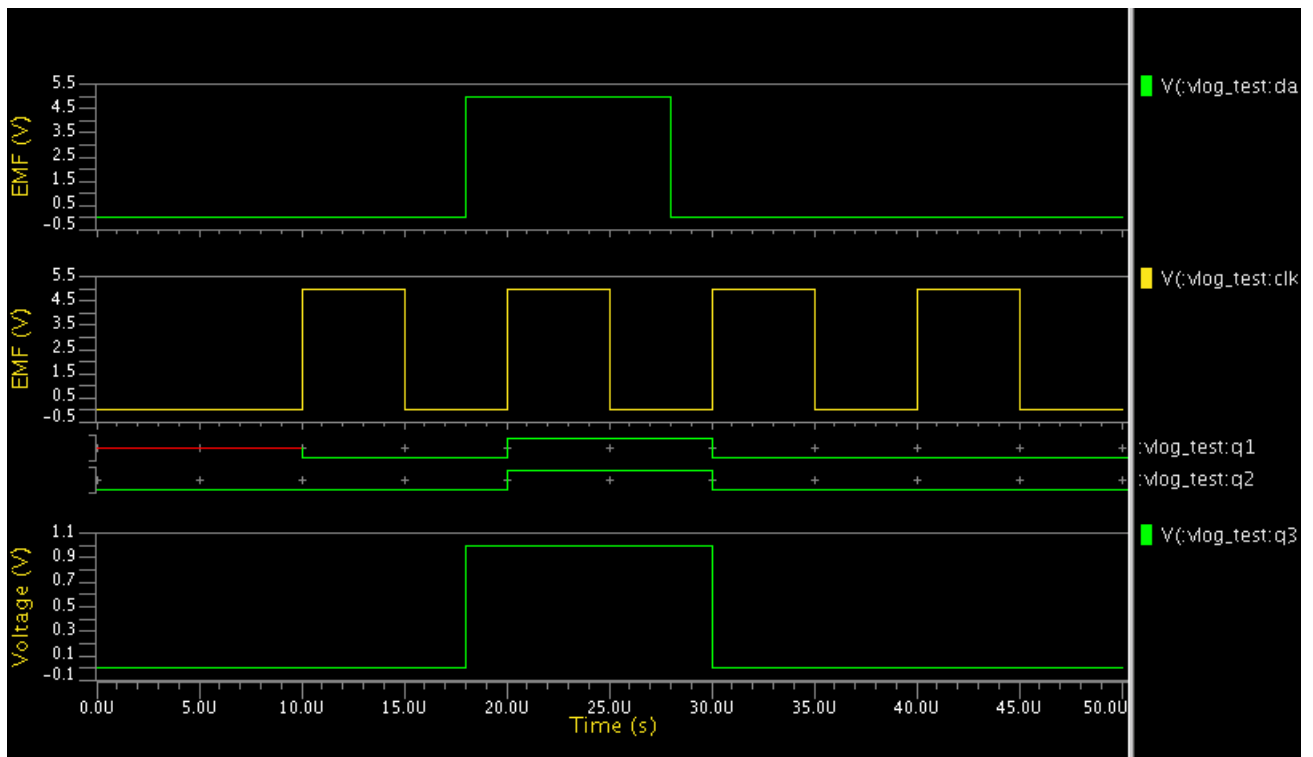
```

Note

The D2A boundary elements d2a_eldo_std_logic and d2a_eldo_bit, assigned by the **.DEFHOOK** command in *vlog_test.cir*, were not used, because for this design, no digital-to-analog conversion is required.

5. Choose **Simulate > Run > Run -All** to run the simulation. At the end of the simulation run, the **Wave Window (SOLIDO Waveform Analyzer)** displays the waveforms defined by the .PLOT commands in the netlist.

Figure 12-26: Simulation Results



Related Topics

[Boundary Elements](#)

[Examples](#)

[Boundary Elements Log File](#)

Verilog-AMS Amplifier with a Resolved wreal Input and an Electrical Output

A Verilog-AMS amplifier can input the resolution value of two digital drivers.

In the following example, the resolution value of two digital drivers is input to a Verilog-AMS amplifier. The two digital drivers are synchronized by a clock generated by the module, `clock_generator`. The digital values are sampled every clock cycle. The `-wreal_resolution` sum option is used to select the resolution function, which calculates and transmits the resolution value of the 2 drivers to the amplifier (with a gain of 2) and its output is transmitted through an electrical terminal.

Prerequisites

- The file `compile_all.csh` contains the following commands:

```
valib amp_lib
vlog src/clock_generator.v
```

```
vlog src/driver1.v
vlog src/driver2.v
valog src/amplifier.vams
valog src/top.vams
```

- The file *run_adms* contains the following command:

```
vasim -cmd amp.cmd -wreal_resolution sum top
```

- The command file *amp.cmd* contains the following instructions for the simulation:

```
.TRAN 0 14ns

.PLOT TRAN SG(in_voltage)
.PLOT TRAN V(out_voltage)

.END
```

Procedure

1. To compile the verilogams_amplifier example, change to the directory *<your_work_area>/examples/adms/verilogams_amplifier*, then enter the following command:

```
source compile_all.csh
```

2. To run the simulation, enter the following command:

```
source run_adms
```

3. When the Questa ADMS GUI finished loading the design, chose **Simulate > Run > Run -All** to run the simulation.

Results

The results show 2 objects:

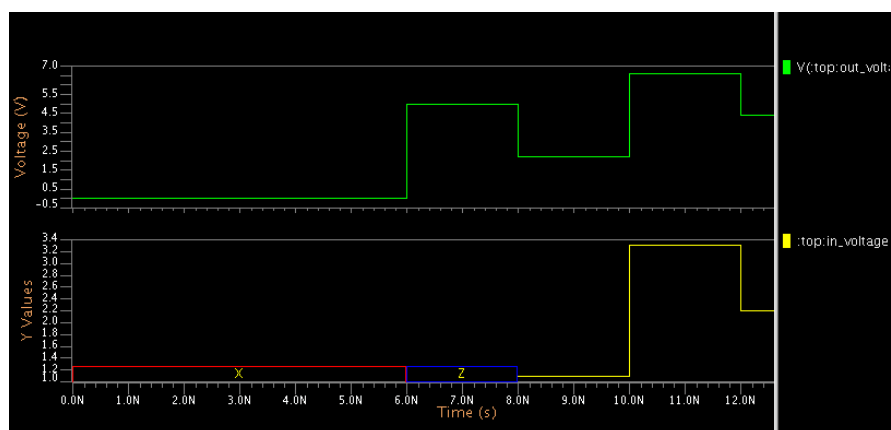
- *in_voltage*

The input signal of the amplifier which is a resolved value of the 2 digital drivers coming from D1:out and D2:out (the resolved value is the sum of the 2 wreal signals thanks to the usage of the "sum" resolution function). The input is a digital wreal signal.

- *out_voltage*

The output of the Verilog-AMS amplifier (the input voltage multiplied by the gain of 2), transmitted through an electrical terminal.

Figure 12-27: Simulation Results for in_voltage and out_voltage



Related Topics

[Using wreal](#)
[Examples](#)

Modeling Examples

Questa ADMS provides a set of behavioral VHDL-AMS, Verilog-AMS, and Verilog-A models for communications and multimedia applications. The models have graded levels of complexity.

The source code in the following location:

`$MGC_AMS_HOME/libraries/ams_lib`

Models are organized into the following categories or sub-libraries:

- A/D
- D/A
- Amplifiers/Comparators
- PLL
- Delta-Sigma
- Filters
- Control
- Functions
- DC-DC
- Modulators/Demodulators
- Digital

- Sources
- SerDes
- RF

You can modify any model(s) and re-run the compilation as required. A compilation script is provided to assist you with the generation of the library and compilation of the models.

For further information, see the appropriate documentation for the language of interest:

- [Modeling Examples for VHDL-AMS.pdf](#)
- [Modeling Examples for Verilog-AMS.pdf](#)
- [Modeling Examples for Verilog-A.pdf](#)

13. Net Spy

Net Spy subprograms and system tasks are used in Questa ADMS to access arbitrary digital or analog objects from any hierarchical region.

Introduction to Net Spy.....	13-1
Net Spy for Digital and Analog Objects.....	13-4
Net Spy for Analog Objects.....	13-29

Introduction to Net Spy

The Net Spy facilities in Questa ADMS provide a convenient extension Verilog hierarchical references.

In Verilog, you can use hierarchical notation to access to any signal from any other hierarchical Verilog module without having to route it via the interface. This means you can access the value of a signal in the design hierarchy from a testbench. The Net Spy facilities in Questa ADMS provide a convenient extension to the Verilog hierarchical reference, in order to allow a testbench to reference a signal in a VHDL, VHDL-AMS or Verilog-AMS model or reference a signal in a Verilog model through a VHDL level of hierarchy.

The Net Spy procedures and system tasks allow you to examine and modify the value of an object at any place in the design hierarchy. The reference uses a hierarchical name formulated according to the rules described in [Absolute and Relative Paths](#) and may occur in any HDL context. The target object may be a digital or analog signal within an HDL or SPICE region. Net Spy provides a generalization of the facility provided by the standard Verilog hierarchical reference.

With Net Spy, you can:

- Examine Questa ADMS analog objects:
 - VHDL-AMS terminals
 - Verilog-AMS nets of continuous domain
 - SPICE nodes
- Examine Questa ADMS digital objects:
 - VHDL and VHDL-AMS signals
 - Verilog, Verilog-AMS and SystemVerilog registers and nets
 - Eldo SPICE signals

See also:

- [Boundary Elements Examples: Net Spy.](#)
- [Util Package for VHDL-AMS.](#)

- Access to Analog Objects from SystemVerilog.

Net Spy Implementation in Questa ADMS.....13-2

Coverage and Limitations.....13-3

Net Spy Implementation in Questa ADMS

Different setup and Net Spy syntax are required for VHDL-AMS and Verilog-AMS designs.

VHDL and VHDL-AMS procedures are provided via the *Util* package within the *modelsim_lib* library. To access the procedures, add the following to the VHDL or VHDL-AMS code:

```
library modelsim_lib;  
use modelsim_lib.util.all;
```

The Verilog and Verilog-AMS tasks are available as built-in system tasks.

Call Only Once

The call for the VHDL-AMS procedure should be made as part of a block or architecture.

The call for the Verilog-AMS system task must be made in an *initial_step* global event control statement and must not be nested inside any conditional, loop or event control (other than an *initial_step*) statement.

Table 13-1 shows the VHDL-AMS procedures and their corresponding Verilog-AMS system tasks.

Table 13-1: VHDL-AMS Procedure and Equivalent Verilog-AMS System Task

Refer to:	VHDL-AMS and VHDL Procedure/Function	Verilog-AMS and Verilog System Task
init_signal_spy	init_signal_spy()	\$init_signal_spy()
enable_signal_spy	enable_signal_spy()	\$enable_signal_spy()
disable_signal_spy	disable_signal_spy()	\$disable_signal_spy()
signal_force	signal_force()	\$signal_force()
signal_release	signal_release()	\$signal_release()
init_terminal_short	init_terminal_short() ¹	\$init_terminal_short() ²
init_terminal_reference	init_terminal_reference() ¹	\$init_terminal_reference() ²

¹ Available only in VHDL-AMS.

² Available only in Verilog-AMS.

Table 13-1: VHDL-AMS Procedure and Equivalent Verilog-AMS System Task (continued)

Refer to:	VHDL-AMS and VHDL Procedure/Function	Verilog-AMS and Verilog System Task
init_terminal_contribution	init_terminal_contribution() ¹	\$init_terminal_contribution() ²
get_terminal_id	get_terminal_id()	N/A
get_terminal_across_value	get_terminal_across_value()	N/A

enable_signal_spy and disable_signal_spy are not available when the source and target objects are both analog.

Related Topics

[Coverage and Limitations](#)

[Net Spy for Digital and Analog Objects](#)

Coverage and Limitations

There are limitations on the portability and use of Net Spy in Questa ADMS.

Designed for Testbenches

Net Spy limits the portability of a model from one simulator to another. This means that a model containing Net Spy subprograms or tasks will only work in Questa ADMS (and Questa SIM), not other simulators. Therefore, Net Spy should only be used in testbenches where portability is less of a concern, and the need for such a tool is more applicable.

Supported Data Types

The init_signal_spy procedure can be used to spy digital or analog objects in any kind of Questa ADMS model: VHDL, VHDL-AMS, Verilog, Verilog-AMS, SystemVerilog and SPICE.

Table 13-2: Supported Data Types

Source --> Target Object	Scalar Type	Vector Type
disable_signal_spy Digital --> Digital	OK	OK ¹
Analog --> Analog	NO ²	NO ^{1,2}

¹ Not available for SPICE objects.

² Only available when called from an VHDL-AMS model in a concurrent procedure call.

Table 13-2: Supported Data Types (continued)

Source --> Target Object	Scalar Type	Vector Type
Digital --> Analog	OK	NO ³
Analog --> Digital	OK	NO ³

Note
The analog Net Spy commands `init_terminal_short`, `init_terminal_reference`, and `init_terminal_contribution` only support analog vectors from VHDL-AMS and Verilog-AMS models, not for an electrical bus of a SPICE subcircuit.

Related Topics

[Net Spy for Digital and Analog Objects](#)
[Net Spy for Analog Objects](#)

Net Spy for Digital and Analog Objects

There are procedures for examining digital and analog objects with Net Spy.

This section describes the following procedures:

<code>init_signal_spy</code>	13-5
<code>init_signal_spy</code> Example.....	13-8
<code>\$init_signal_spy</code> Example.....	13-9
Verilog-AMS Model Example.....	13-11
<code>init_signal_spy</code> Example with Top SPICE.....	13-12
<code>enable_signal_spy</code>	13-18
<code>disable_signal_spy</code>	13-20
<code>signal_force</code>	13-22
<code>signal_force</code> Example.....	13-24
<code>signal_release</code>	13-28

³ Vectors are not supported between A->D or D->A.

init_signal_spy

Mirrors the value of a Questa ADMS object onto an existing Questa ADMS object. This topic describes the VHDL/VHDL-AMS `init_signal_spy()` procedure and the Verilog/Verilog-AMS `$init_signal_spy()` task.

Syntax

VHDL/VHDL-AMS Syntax

```
init_signal_spy(<src_object>, <dest_object> [, <verbose>]
               [, <control_state>])
```

Verilog/Verilog-AMS Syntax

```
$init_signal_spy(<src_object>, <dest_object> [, <verbose>]
                [, <control_state>])
```

Arguments

- `src_object`

Required. Hierarchical name of a digital or analog object.

By default, `init_signal_spy` uses a colon (:) as a path separator. You can change this behavior by setting the `SignalSpyPathSeparator` variable (see [Questa SIM Simulation Control Variables](#)) in the `modelsim.ini` file. The path separator can be `;`, `/` or `..`. A full hierarchical path must begin with a `;`, `/` or `..`. The path must be contained within double quotes.

- `dest_object`

Required. Hierarchical name of the target digital or analog object.

- `verbose`

A message stating that the spy object's (`src_object`) value is mirrored onto the `dest_object` will be reported in the [Transcript Window](#).

- 0 — Does not report a message. Default.
- 1 — Reports a message.

- `control_state`

Determines whether the mirroring of values can be enabled/disabled and specifies the initial state when enabled. Subsequent control of whether the mirroring of values is enabled/disabled is handled by the [enable_signal_spy](#) and [disable_signal_spy](#) calls.

- -1

Turns off the ability to enable/disable and mirroring is enabled. Default.

- 0

Turns on the ability to enable/disable and initially disables mirroring.

- 1

Turns on the ability to enable/disable and initially enables mirroring.

Note

enable_signal_spy and disable_signal_spy are not available when the source and target objects are both analog.

Description

The concurrent procedure `init_signal_spy` is equivalent to the Questa SIM Signal Spy procedure, but is extended to also handle analog objects. Refer to Signal Spy in the *Questa SIM User's Manual* for more information.

It is possible to mirror an analog object to a digital object, and vice versa.

Digital Source and Digital Target

The `init_signal_spy` call only sets the value of the destination object—it does not drive or force the value. Any existing or subsequent drive or force of the destination signal, by some other means, will override the value that was set by `init_signal_spy`.

Analog Source and Analog Target

The `init_signal_spy` call is equivalent to an `init_terminal_reference` call with the default gain value of 1.0; see [init_terminal_reference](#).

Digital Source and Analog Target

The `init_signal_spy` call inserts a D2A built-in boundary element according to the boundary element rules described in [Boundary Element Insertion](#). Only SPICE built-in boundary elements are supported, user-defined boundary elements are not. When the source digital value changes, the corresponding analog value is mirrored onto the analog node.

Analog Source and Digital Target

The `init_signal_spy` call only sets the value of the destination object—it does not drive or force the value. It inserts an A2D boundary element according to the boundary element rules described in [Boundary Element Insertion](#). Only SPICE built-in boundary elements are supported, user-defined boundary elements are not. When the digital output value of the A2D boundary element changes, the corresponding analog value (determined by the code inside of the A2D boundary element) is mirrored onto the digital value.

How to Call init_signal_spy

Once `init_signal_spy` is called, any change on the source object will be mirrored on the target object until simulation ends (unless `enable_signal_spy` and `disable_signal_spy` calls are used).

The `init_signal_spy` is a concurrent procedure call in VHDL and VHDL-AMS. Like all concurrent procedures, it cannot be contained within a process. See [init_signal_spy Example](#).

For Verilog and Verilog-AMS tasks, you should place all `$init_signal_spy` tasks in a Verilog initial block; see [\\$init_signal_spy Example](#).

Boundary Element Log File

When boundary elements are automatically added or used by `init_signal_spy()` they are identified in the boundary element log file (`.conv`) by FROM SIGNAL SPY and TO SIGNAL SPY statements. For further information, see [Boundary Elements Log File](#).

Note

If the boundary elements log file (`.conv`) is updated during simulation, and is being viewed in the [Source Window](#), it may not be automatically updated. You must change the region in the [Structure Window](#) to refresh the data in the Source window.

Similarly, if the boundary elements log file is created during simulation, the Structure window will not update. The boundary elements log file must be opened directly from the Source window.

Limitations

- When mirroring the value of a SystemVerilog, Verilog or Verilog-AMS register/net onto a VHDL signal, the VHDL signal must be of type `bit`, `bit_vector`, `Std_logic`, or `Std_logic_vector`.
- Verilog and Verilog-AMS memories (arrays of registers) are not supported.
- The `control_state` is ignored when the source and destination are both analog objects.
- The `init_signal_spy()` procedure cannot be used to spy signals in a VHDL-AMS package.
- The `init_signal_spy()` procedure cannot be used to spy signals of type `complex` or `complex_vector`.

Related Topics

[init_signal_spy Example](#)

[\\$init_signal_spy Example](#)

[init_signal_spy Example with Top SPICE](#)

[enable_signal_spy](#)

[disable_signal_spy](#)

[Net Spy for Analog Objects](#)

init_signal_spy Example

You can use `init_signal_spy` to mirror a signal onto another signal.

In this example, the value of `:top: uut: inst1: source_sig` is mirrored onto `dest_sig` and a message is issued to the [Transcript Window](#). The ability to control the mirroring of values is turned on and `init_signal_spy` is initially enabled. The mirroring of values will be disabled when `enable_sig` transitions to 0 and enabled when `enable_sig` transitions to 1.

Procedure

1. Create a VHDL source file called `top_iss.vhd`, containing the following code:

```
library ieee, modelsim_lib;
use ieee.std_logic_1164.all;
use modelsim_lib.util.all;
entity top is
end entity top;

architecture only of top is
    signal dest_sig : std_logic;
begin
    spy_process : init_signal_spy(":top: uut: inst1: source_sig", "dest_sig",
    1);

    uut : block
        begin -- block uut
            inst1 : block
                signal source_sig : std_logic;
                begin -- block inst1
                    source_sig <= '1' after 10 ns, 'X' after 20 ns;
                end block inst1;
            end block uut;
        end architecture only;
```

2. Create a file called `top_iss.do` containing the following commands:

```
add wave :top: uut: inst1: source_sig
add wave :top: dest_sig

run -all
```

3. In the same directory, create a design library called `adms_lib`:

```
valib adms_lib
```

4. Compile the source code into the design library:

```
vacom top_iss.vhd
```

5. Launch the simulator using the command and do files:

```
vasim top -do top_iss.do &
```

The following message is written to the [Transcript Window](#), as the verbose flag has been set:

```
# Note: init_signal_spy [<path>/top_iss.vhd] : Mirroring the value
of ':top:uut:inst1:source_sig' onto ':top:dest_sig'.
```

Related Topics

[init_signal_spy](#)

[\\$init_signal_spy Example](#)

[init_signal_spy Example with Top SPICE](#)

[Writing Hierarchical Path Names](#)

\$init_signal_spy Example

You can use \$init_signal_spy to mirror a signal onto another signal.

In this example, the value of :top:uut:inst1:sig1 is mirrored onto :top:top_sig1. The mirroring is enabled on the rising of enable_reg using the \$enable_signal_spy() system task and disabled on the negative edge of enable_reg using the \$disable_signal_spy() system task.

Procedure

1. Create a Verilog source file *level.s.v* containing the following code:

```
`timescale 1ns/1ns
module level2();
    reg sig1;

    initial begin
        sig1 = 1'b0;
        forever #10 sig1 = !sig1;
    end
endmodule

`timescale 1ns/1ns
```

```
module level1();  
    \adms_lib.level2 inst1();  
endmodule
```

2. Create a Verilog source file *top_iss_ess_dss.v* containing the following code:

```
`timescale 1ns/1ns  
  
module top();  
    reg top_sig1;  
    reg enable_reg;  
    initial begin  
        $init_signal_spy(".top.uut.inst1.sig1", ".top.top_sig1", 1, 1);  
        enable_reg = 1'b0;  
        #35 enable_reg = !enable_reg;  
        #80 enable_reg = !enable_reg;  
    end  
    always@(posedge enable_reg) begin  
        $enable_signal_spy(".top.uut.inst1.sig1", ".top.top_sig1", 1);  
    end  
    always@(negedge enable_reg) begin  
        $disable_signal_spy(".top.uut.inst1.sig1", ".top.top_sig1", 1);  
    end  
    \adms_lib.level1 uut();  
endmodule
```

3. Create a command file *cmdfile.cmd* containing the specifications for the analysis:

```
* command file  
.TRAN 100ns 100ns
```

4. Create the file *top_iss_ess_dss.do* containing the following commands:

```
add wave :top:top_sig1  
add wave :top:enable_reg  
add wave :top:uut:inst1:sig1  
  
run 150 ns
```

5. In the same directory, create a design library called *adms_lib*:

```
valib adms_lib
```

6. Compile the source code into the design library:

```
valog levels.v  
vlog top_iss_ess_dss.v
```

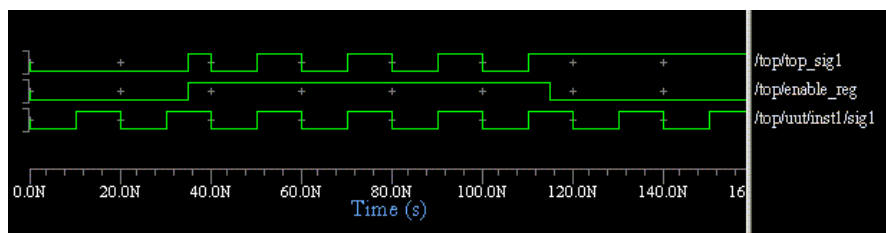
7. Launch the simulator using the command and do files:

```
vasim top -cmd cmdfile.cmd -do top_iss_ess_dss.do &
```

Results

When the simulation is complete the results are displayed in the **Wave Window (SOLIDO Waveform Analyzer)**, as shown in **Figure 13-1**:

Figure 13-1: init_signal_spy Example



The following messages are written to the **Transcript Window** if the verbose flag has been set:

```
# ** Note: init_signal_spy [<path>/top_iss_ess_dss.v(8)] : Mirroring the
value of ':top:uut:inst1:sig1' onto ':top:top_sig1'.

# ** Note: disable_signal_spy [<path>/top_iss_ess_dss.v(19)] : The signal
spy was disabled at time 0 fs.

# ** Note: enable_signal_spy [<path>/top_iss_ess_dss.v(16)] : The signal
spy was enabled at time 35000000 fs.

# ** Note: disable_signal_spy [<path>/top_iss_ess_dss.v(19)] : The signal
spy was disabled at time 115000000 fs.
```

Verilog-AMS Model Example

You can add an analog object to a Verilog model to create a Verilog-AMS model.

The source file `top_iss_ess_dss_ams.v` includes a dummy analog object, `gnd`.

Procedure

1. Create the source file `top_iss_ess_dss_ams.v` containing the following source code:

```
`timescale 1ns/1ns
`include "disciplines.h"
```

```

module top();
    reg top_sig1;
    reg enable_reg;

    electrical gnd; // so it is a Verilog-AMS model
    ground gnd;

    initial begin
        $init_signal_spy(".top.uut.inst1.sig1",".top.top_sig1",1,1);
        enable_reg = 1'b0;
        #35 enable_reg = !enable_reg;
        #80 enable_reg = !enable_reg;
    end
    always@(posedge enable_reg) begin
        $enable_signal_spy(".top.uut.inst1.sig1",".top.top_sig1",1);
    end
    always@(negedge enable_reg) begin
        $disable_signal_spy(".top.uut.inst1.sig1",".top.top_sig1",1);
    end
    \adms_lib.level1 uut();
endmodule

```

2. Compile the source code into the adms_lib design library you created in the previous example:

```

valog levels.v
valog top_iss_ess_dss_ams.v

```

3. Launch the simulator using the do file:

```

vasim top -do top_iss_ess_dss.do &

```

Results

The simulation gives the same results as for Verilog top design.

Related Topics

[init_signal_spy](#)
[init_signal_spy Example](#)
[init_signal_spy Example with Top SPICE](#)
[enable_signal_spy](#)
[disable_signal_spy](#)
[Writing Hierarchical Path Names](#)

init_signal_spy Example with Top SPICE

The value of a source signal can be mirrored onto a destination signal.

In this example, the value of :top:uut:inst1:source_sig will be mirrored onto dest_sig. A message is issued to the [Transcript Window](#). The ability to control the mirroring of values is turned on, and the init_signal_spy() procedure is initially enabled.

The mirroring of values will be disabled when enable_toggle changes to 0, and will be enabled when enable_toggle changes to 1.

Procedure

1. Create the source file *vhdlams_src_level.vhd* containing the following source code:

```
library ieee;
  use ieee.std_logic_1164.all;

entity src_level is
end entity src_level;

architecture vhdams of src_level is
  signal source_sig : std_logic := '0';
begin
  source_sig <= not source_sig after 10 ns;
end architecture vhdams;
```

2. Create the source file *vhdl_iss_level.vhd* containing the following source code:

```
library ieee, modelsim_lib;
use ieee.std_logic_1164.all;
use modelsim_lib.util.all;

entity iss_level is
end entity iss_level;

architecture vhd_eldo of iss_level is
  signal enable_toggle : bit := '0';
begin
  spy_process : process
  begin
    init_signal_spy(":\eldo-top_iss-vhdl\ysrc:source_sig",
                  ":\eldo-top_iss-vhdl\ydest:dest_reg[2]", 1, 1);
    init_signal_spy(":\eldo-top_iss-vhdl\ysrc:source_sig",
                  ":\eldo-top_iss-vhdl\ydest:dest_reg(0)", 1, 0);
    enable_toggle <= '1', '0' after 35 ns, '1' after 75 ns;
    wait;
  end process spy_process;

  spy_enable_disable : process
  begin
    l0 : loop
      wait on enable_toggle;
      if (enable_toggle = '1') then
        disable_signal_spy(":\eldo-top_iss-vhdl\ysrc:source_sig",
                          ":\eldo-top_iss-vhdl\ydest:dest_reg[2]", 1);
```

```

        enable_signal_spy(":\eldo-top_iss-vhdl\ysrc:source_sig",
                           ":\eldo-top_iss-vhdl\ydest:dest_reg(0)", 1);
    else
        enable_signal_spy(":\eldo-top_iss-vhdl\ysrc:source_sig",
                           ":\eldo-top_iss-vhdl\ydest:dest_reg[2]", 1);
        disable_signal_spy(":\eldo-top_iss-vhdl\ysrc:source_sig",
                            ":\eldo-top_iss-vhdl\ydest:dest_reg(0)", 1);
    end if;
end loop l0;
end process spy_enable_disable;
end architecture vhdl_eldo;

```

3. Create the source file *verilog_dest_level.v* (a Verilog bus) containing the following source code:

```

`timescale 1 ns/1 ns

module dest_level_verilog;
    reg [3:0] dest_reg;

    initial begin
        dest_reg <= 4'b0000;
    end // initial
endmodule

```

4. Create the netlist *eldo-top_iss-vhdl.cir* containing the following source code:

```

* File: eldo-top_iss-vhdl.cir

** VHDL-AMS model:
.MODEL src_level(vhdlams)    MACRO LANG=VHDL

** VHDL model:
.MODEL iss_level(vhdl_eldo) MACRO LANG=VHDLAMS

** Verilog model:
.MODEL dest_level_verilog    MACRO LANG=VERILOG

** Instances:
ysrc    src_level(vhdlams)
yiss    iss_level(vhdl_eldo)
ydest   dest_level_verilog

** Commands:
.TRAN 1n 100n

```

5. Create the file *eldo-top.do* containing the following instructions:

```

if [batch_mode] {
    add log -r *
} else {
    view structure objects source
}

```



```

    add wave -r *
}

run 100 ns

if [batch_mode] {
    quit -f
}

```

6. In the same directory, create a design library called adms_lib:

```
valib adms_lib
```

7. Compile the source code into the design library:

```

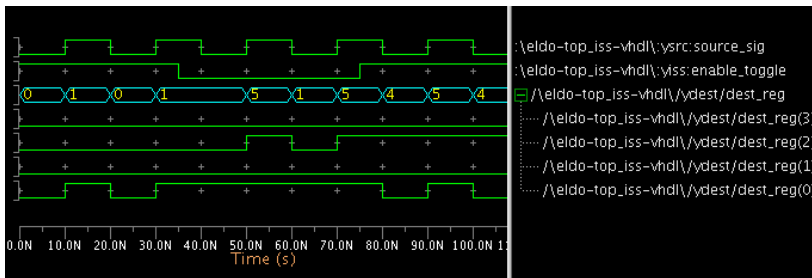
vacom vhdlams_src_level.vhd
vacom vhdl_iss_level.vhd
valog verilog_dest_level.v
vasim -cmd eldo-top_iss-vhdl.cir -do eldo-top.do &

```

Results

When the simulation is complete the results can be displayed in the [Wave Window \(Solido Waveform Analyzer\)](#), as shown in [Figure 13-2](#):

Figure 13-2: init_signal_spy Example with Top SPICE



The following messages are written to the [Transcript Window](#) if the verbose flag has been set:

```

# Note: (vasim - 2102) init_signal_spy : Mirroring the
value of ':\eldo-top_iss-vhdl\ysrc:source_sig' onto ':\eldo-top_iss-
vhdl\ydest:dest_reg(2)'.
# Time: 0 fs Iteration: 1 Instance: :\eldo-top_iss-vhdl:yiss
# File: <path>/vhdl_iss_level.vhd Model: ISS_LEVEL(VHDL_ELD0)
#
# Note: (vasim - 2102) init_signal_spy : Mirroring the
value of ':\eldo-top_iss-vhdl\ysrc:source_sig' onto ':\eldo-top_iss-
vhdl\ydest:dest_reg(0)'.
# Time: 0 fs Iteration: 1 Instance: :\eldo-top_iss-vhdl:yiss
# File: <path>/vhdl_iss_level.vhd Model: ISS_LEVEL(VHDL_ELD0)

```

```

#
# Note: (vasim - 2100) disable_signal_spy : The signal spy was
disabled. Between ':\eldo-top_iss-vhdl\ysrc:source_sig' onto ':\eldo-
top_iss-vhdl\ydest:dest_reg(2)'.
# Time: 0 fs Iteration: 1 Instance: :\eldo-top_iss-vhdl:yiss
# File: <path>/vhdl_iss_level.vhd Model: ISS_LEVEL(VHDL_ELD0)
#
# Note: (vasim - 2101) enable_signal_spy : The signal spy was
enabled. Between ':\eldo-top_iss-vhdl\ysrc:source_sig' onto ':\eldo-
top_iss-vhdl\ydest:dest_reg(0)'.
# Time: 0 fs Iteration: 1 Instance: :\eldo-top_iss-vhdl:yiss
# File: <path>/vhdl_iss_level.vhd Model: ISS_LEVEL(VHDL_ELD0)

...

# -----
# | Transient analysis starting |
# -----
#

...
# Starting analog time is 0.000000 ns
# Simulation time for "run -all" is 100.000000 ns

...
# Note: (vasim - 2101) enable_signal_spy : The signal spy was
enabled. Between ':\eldo-top_iss-vhdl\ysrc:source_sig' onto ':\eldo-
top_iss-vhdl\ydest:dest_reg(2)'.
# Time: 35 ns Iteration: 0 Instance: :\eldo-top_iss-vhdl:yiss
# File: <path>/vhdl_iss_level.vhd Model: ISS_LEVEL(VHDL_ELD0)
#
# Note: (vasim - 2100) disable_signal_spy : The signal spy was
disabled. Between ':\eldo-top_iss-vhdl\ysrc:source_sig' onto ':\eldo-
top_iss-vhdl\ydest:dest_reg(0)'.
# Time: 35 ns Iteration: 0 Instance: :\eldo-top_iss-vhdl:yiss
# File: <path>/vhdl_iss_level.vhd Model: ISS_LEVEL(VHDL_ELD0)
#
# Note: (vasim - 2100) disable_signal_spy : The signal spy was
disabled. Between ':\eldo-top_iss-vhdl\ysrc:source_sig' onto ':\eldo-
top_iss-vhdl\ydest:dest_reg(2)'.
# Time: 75 ns Iteration: 0 Instance: :\eldo-top_iss-vhdl:yiss
# File: <path>/vhdl_iss_level.vhd Model: ISS_LEVEL(VHDL_ELD0)
#
# Note: (vasim - 2101) enable_signal_spy : The signal spy was
enabled. Between ':\eldo-top_iss-vhdl\ysrc:source_sig' onto ':\eldo-
top_iss-vhdl\ydest:dest_reg(0)'.
# Time: 75 ns Iteration: 0 Instance: :\eldo-top_iss-vhdl:yiss
# File: <path>/vhdl_iss_level.vhd Model: ISS_LEVEL(VHDL_ELD0)

```

Related Topics

Net Spy for Digital and Analog Objects

[init_signal_spy](#)

[init_signal_spy Example](#)

[\\$init_signal_spy Example](#)

Writing Hierarchical Path Names

enable_signal_spy

Enables the associated `init_signal_spy` call. This topic describes the VHDL/VHDL-AMS `enable_signal_spy()` procedure and the Verilog/Verilog-AMS `$enable_signal_spy()` task.

Syntax

VHDL/VHDL-AMS Syntax

```
enable_signal_spy(<src_object>, <dest_object> [, <verbose>])
```

Verilog/Verilog-AMS Syntax

```
$enable_signal_spy(<src_object>, <dest_object> [, <verbose>])
```

Arguments

- `src_object`

Required. A full hierarchical path (or relative downward path with reference to the calling region) to a digital or analog Questa ADMS object. This path should match the path that was specified in the `init_signal_spy` call that is to be enabled.

By default, `enable_signal_spy` and `init_signal_spy` use a colon (:) as a path separator. You can change this behavior by setting the `SignalSpyPathSeparator` variable (see [Questa SIM Simulation Control Variables](#)) in the `modelsim.ini` file.

- `dest_object`

Required. A full hierarchical path (or relative downward path with reference to the calling region) to a digital or analog Questa ADMS object. This path should match the path that was specified in the `init_signal_spy` call that is to be enabled.

Note

When `enable_signal_spy()` has a source and destination that are both terminals, an error message is generated saying "not implemented".

- `verbose`

Specifies whether a message is reported in the transcript stating that an enable occurred and the simulation time that it occurred.

- 0 — Does not report a message. Default.
- 1 — Reports a message.

Description

The enable_signal_spy call enables the associated init_signal_spy call. The association between the enable_signal_spy call and the init_signal_spy call is based on specifying the same src_object and dest_object arguments to both. The enable_signal_spy call can only affect init_signal_spy calls that had their control_state argument set to "0" or "1". enable_signal_spy returns nothing.

Related Topics

[init_signal_spy](#)

[disable_signal_spy](#)

[Writing Hierarchical Path Names](#)

disable_signal_spy

Disables the associated `init_signal_spy` call. This topic describes the VHDL/VHDL-AMS `disable_signal_spy()` procedure and the Verilog/Verilog-AMS `$disable_signal_spy()` task.

Syntax

VHDL/VHDL-AMS Syntax

```
disable_signal_spy(<src_object>, <dest_object> [, <verbose>])
```

Verilog/Verilog-AMS Syntax

```
$disable_signal_spy(<src_object>, <dest_object> [, <verbose>])
```

Arguments

- `src_object`

Required. A full hierarchical path (or relative downward path with reference to the calling region) to a digital or analog Questa ADMS object. This path should match the path that was specified in the `init_signal_spy` call that is to be disabled.

By default, `disable_signal_spy` and `init_signal_spy` use a colon (:) as a path separator. You can change this behavior by setting the `SignalSpyPathSeparator` variable (see [Questa SIM Simulation Control Variables](#)) in the `modelsim.ini` file.

- `dest_object`

Required. A full hierarchical path (or relative downward path with reference to the calling region) to a digital or analog Questa ADMS object. This path should match the path that was specified in the `init_signal_spy` call that is to be disabled.

Note

When `disable_signal_spy()` has a source and destination which are both analog objects, an error message is generated saying "not implemented".

- `verbose`

Specifies whether a message is reported in the transcript stating that a disable occurred and the simulation time at which it occurred.

- 0 — Does not report a message. Default.
- 1 — Reports a message.

Description

The `disable_signal_spy` call disables the associated `init_signal_spy` call. The association between the `disable_signal_spy` call and the `init_signal_spy` call is based on specifying the same `src_object` and `dest_object` arguments to both. The `disable_signal_spy` call can only affect `init_signal_spy` calls where the `control_state` argument set to "0" or "1". `disable_signal_spy` returns nothing.

Related Topics

[init_signal_spy](#)

[enable_signal_spy](#)

[Writing Hierarchical Path Names](#)

signal_force

Forces the specified value onto an existing destination object. This topic describes the VHDL `signal_force()` procedure and the Verilog `$signal_force()` task.

Syntax

VHDL Syntax

```
signal_force(<dest_object>, <value>, <rel_time>, <force_type>,
             <cancel_period> [, <verbose>])
```

Verilog Syntax

```
$signal_force(<dest_object>, <value>, <rel_time>, <force_type>,
              <cancel_period> [, <verbose>])
```

Arguments

- `dest_object`

Required. A full hierarchical path (or relative downward path with reference to the calling region) to an existing Questa ADMS object. See [Absolute and Relative Paths](#).

By default, `signal_force` uses a colon (:) as a path separator. You can change this behavior by setting the `SignalSpyPathSeparator` variable (see [Questa SIM Simulation Control Variables](#)) in the `modelsim.ini` file. The path separator can be `;`, `/` or `..`. A full hierarchical path must begin with a `;`, `/` or `..`. The path must be contained within double quotes.

- `value`

Required. Specifies the value to which the `dest_object` is to be forced. The specified value must be appropriate for the type and enclosed within double quotes. `value` can be a real number, a sequence of character literals, or as a based number with a radix of 2, 8, 10 or 16. For example, the following values are equivalent for a signal of type `bit_vector` (0 to 3):

- 1111 — character literal sequence
- 2#1111 — binary radix
- 8#17 — octal radix
- 10#15 — decimal radix
- 16#F — hexadecimal radix

- `rel_time`

Specifies a time relative to the current simulation time for the force to occur. The default is 0.

- **force_type**

Specifies the type of force that will be applied.

For a VHDL procedure, the value must be one of the following;

- default — which is “freeze” for unresolved objects or “drive” for resolved objects
- deposit
- drive
- freeze.

For a Verilog task, the value must be one of the following;

- 0 — default, which is “freeze” for unresolved objects or “drive” for resolved objects
- 1 — deposit
- 2 — drive
- 3 — freeze

For a SystemC function, the value must be one of the following;

- 0 — default, which is “freeze” for unresolved objects or “drive” for resolved objects
- 1 — deposit
- 2 — drive
- 3 — freeze

See the **force** Tcl command for further details on force type.

- **cancel_period**

Cancels the signal_force command after the specified period of time units. Cancellation occurs at the last simulation delta cycle of a time unit.

For a VHDL procedure, a value of zero cancels the force at the end of the current time period. Default is -1 ms. A negative value means that the force will not be cancelled.

For a Verilog task, a value of zero cancels the force at the end of the current time period. Default is -1. A negative value means that the force will not be cancelled.

- **verbose**

Specifies whether a message is reported in the **Transcript Window** stating that the value is being forced on the dest_object at the specified time.

- 0 — Does not report a message. Default.

- 1 — Reports a message.

Description

The `signal_force()` call forces the specified value onto an existing destination object (`dest_object`). This enables you to force digital or analog objects at any level of hierarchy from within a Questa ADMS model (e.g. a testbench). `signal_force` returns nothing.

A `signal_force` works the same as the **force** command with the exception that you cannot issue a repeating force. The force will remain on the signal until a `signal_release`, a force or release command, or a subsequent `signal_force` is issued. `signal_force` can be called concurrently or sequentially in a process.

This command displays any signals using your radix setting (either the default, or as you specify) unless you specify the radix in the value you set.

When SPICE built-in boundary elements are automatically added or used by `signal_force()`, they are identified in the boundary element log file (`.conv`) by FROM force and TO force statements.

For further information, see [Boundary Elements](#) and [Boundary Elements Log File](#).

Limitations

The following limitations apply to the use of `signal_force`.

- You cannot force bits of a register; you can force only the entire register.
- Verilog memories (arrays of registers) are not supported.
- When a boundary element is needed because the `dest_object` argument is a analog object, the boundary element is chosen according the **.DEFHOOK** rules, or according to the Verilog-AMS **Connect Specification Statements**, if present.
- This command cannot be applied on signals of type `complex` or `complex_vector`.

Related Topics

[signal_force Example](#)

[init_signal_spy](#)

[signal_release](#)

[.DEFHOOK \[Questa ADMS Command Reference\]](#)

signal_force Example

Real, `std_logic` bit, and `bit_vector` can be used with `signal_force` and `signal_release`:

This example shows how real, `std_logic` bit, and `bit_vector` can be used with `signal_force` and `signal_release`.

Procedure

1. Create the source file *signal_force_values.vhd* containing the following source code:

```

library ieee, modelsim_lib;
use ieee.std_logic_1164.all;
use modelsim_lib.util.all;
use work.all;

entity e is
  port (
    i1, i2 : in real;
    o1 : out real;
    i3 : in std_logic;
    o3 : out std_logic;
    i4 : in bit_vector(0 to 1);
    o4 : out bit_vector(0 to 1);
    o5 : out bit_vector(0 to 1)
  );
end entity e;

architecture a of e is
begin
  o1 <= i1 * i2;
  o3 <= i3;
  o4 <= i4;
  o5 <= "00";
end architecture a;

entity etb is
end entity etb;

library ieee, modelsim_lib;
use ieee.std_logic_1164.all;
use modelsim_lib.util.all;
use work.all;

architecture bench of etb is
  signal i1, i2, o1 : real := 0.0;
  signal i3, o3 : std_logic;
  signal i4, o4, o5 : bit_vector(0 to 1);
begin
  u1 : entity work.e
    port map (i1 => i1, i2 => i2, o1 => o1, i3 => i3,
              o3 => o3, i4 => i4, o4 => o4, o5 => o5);

  process begin
    i1 <= 0.0;
    i2 <= 0.0;
    i3 <= 'X';
    i4 <= "11";

    wait for 10 ns;

```

```

i1 <= 1.1;
i2 <= 2.2;
signal_force(":etb:o3", "'1'"); -- character literal
i4 <= "01";

wait for 10 ns;
signal_force(":etb:o1", "5.3"); -- real value
signal_release(":etb:o3");
i4 <= "10";

wait for 10 ns;
signal_release(":etb:o1");
i3 <= 'U';
signal_force(":etb:o5", "11"); -- character literal sequence
signal_force(":etb:o4", "2#11"); -- based number with binary radix

wait for 10 ns;
i1 <= -3.3;
i2 <= 0.25;
i3 <= 'W';
signal_release(":etb:o4");

wait for 10 ns;
i1 <= 0.0;
i2 <= 0.0;
i3 <= 'L';
i4 <= "11";

wait for 10 ns;

wait;
end process;
end architecture bench;

```

2. Compile the source file *signal_force_values.vhd* into the library MYLIB and run the simulation:

```

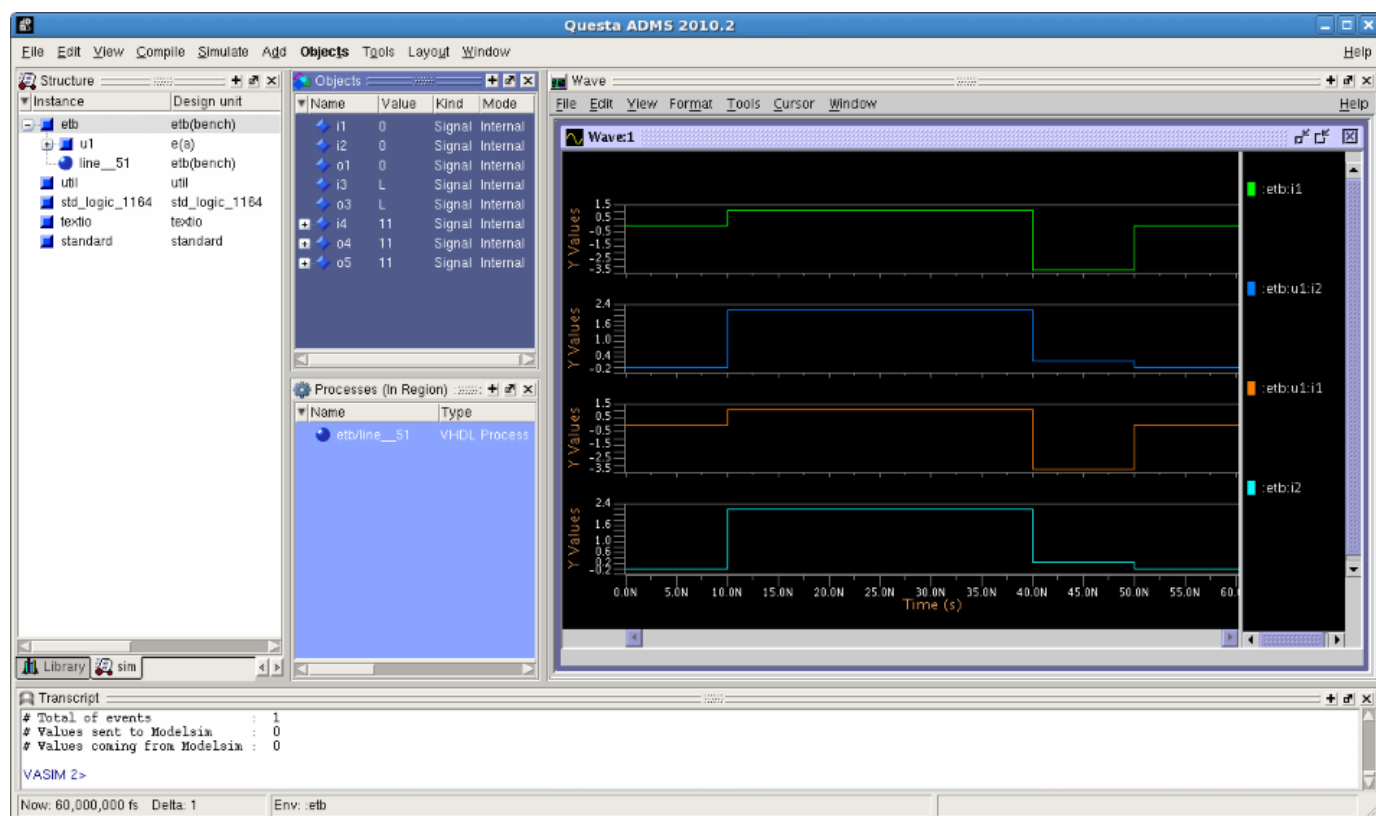
valib MYLIB
vcom signal_force_values.vhd
vasim etb -do "add wave -r :*; run -all"

```

Results

Figure 13-3 shows the simulation results.

Figure 13-3: Example of signal_force



Related Topics

[signal_force](#)

[signal_release](#)

signal_release

Releases any force that was applied to an existing Questa ADMS object. This enables you to release signals, registers or nets at any level of the design hierarchy. This topic describes the VHDL `signal_release()` procedure and the Verilog `$signal_release()` task.

Syntax

VHDL Syntax

```
signal_release(<dest_object> [, <verbose>])
```

Verilog Syntax

```
$signal_release(<dest_object> [, <verbose>])
```

Arguments

- `dest_object`

Required. A full hierarchical path (or relative downward path with reference to the calling region) to an existing Questa ADMS object. See [Absolute and Relative Paths](#).

By default, `signal_release` uses a colon (:) as a path separator. You can change this behavior by setting the `SignalSpyPathSeparator` variable (see [Questa SIM Simulation Control Variables](#)) in the `modelsim.ini` file. The path separator can be `:`, `/` or `..`. A full hierarchical path must begin with a `:`, `/` or `..`. The path must be contained within double quotes.

- `verbose`

Possible values are 0 or 1. Specifies whether you want a message reported in the [Transcript Window](#) stating that the signal is being released and the time of the release.

- 0 — Does not report a message. Default.
- 1 — Reports a message.

Description

The `signal_release()` call releases any force that was applied to an existing Questa ADMS object (called the `dest_object`). This enables you to release signals, registers or nets at any level of the design hierarchy from a Questa ADMS model (for example, a testbench).

A `signal_release` works the same as the [noforce](#) command. `signal_release` can be called concurrently or sequentially in a process. `signal_release` returns nothing.

Limitations

- You cannot release a bit or slice of a register; you can release only the entire register.
- When a boundary element is needed because the `dest_object` argument is an analog object, the boundary element is chosen according to the **.DEFHOOK** rules, or according to the Verilog-AMS **Connect Specification Statements**, if present.

Related Topics

[signal_force Example](#)

[init_signal_spy](#)

[signal_force](#)

[.DEFHOOK \[Questa ADMS Command Reference\]](#)

Net Spy for Analog Objects

Net Spy provides several functions for use with analog objects.

An analog object has two aspects that are represented by two quantities; the reference aspect (or across quantity, voltage for electrical), and the contribution aspect (or through quantity, current for electrical). It also implies an implicit equation, called Kirchhoff's Current Law (KCL) for electrical or extended KCL.

- The `init_terminal_short()` call short-circuits two analog objects together, or shorts an analog net with a digital signal (internals and ports).
- The `init_terminal_reference()` call mirrors the voltage from the source object onto the destination object.
- The `init_terminal_contribution()` call mirrors the current through the source object (contribution) onto the destination object as an equivalent voltage value.
- The `get_terminal_id()` function returns a reference ID for an analog object.
- The `get_terminal_across_value()` function returns the across quantity (reference) of the analog object that the specified reference ID refers to.

init_terminal_short.....	13-30
init_terminal_short Example.....	13-34
\$init_terminal_short Example.....	13-36
init_terminal_reference.....	13-39
init_terminal_reference Example.....	13-43
\$init_terminal_reference Example.....	13-45
init_terminal_contribution.....	13-48
init_terminal_contribution Example.....	13-51
\$init_terminal_contribution Example.....	13-53
get_terminal_id.....	13-56
get_terminal_across_value.....	13-58
get_terminal_across_value Example.....	13-59

init_terminal_short

Sets the value of two terminals, making a short-circuit. You can connect an analog net to another analog net, or to a digital net. The two terminals are shorted together and represent the same object.

Note

init_terminal_short cannot be used to connect 2 digital nets together.

Note

init_terminal_short returns nothing by default, but can be set to report messages message using the verbose argument.

Syntax

VHDL-AMS

```
init_terminal_short(<net_1>, <net_2> [, <verbose>])
```

Verilog-AMS

```
$init_terminal_short(<net_1>, <net_2> [, <verbose>])
```

Arguments

- net_1
Required. This is the name of the first object. You should provide a full hierarchical path (or relative downward path with reference to the calling region) to the analog object.

By default, init_terminal_short uses a colon (:) as a path separator. You can change this behavior by setting the SignalSpyPathSeparator variable (see [Questa SIM Simulation Control Variables](#)) in the *modelsim.ini* file. The path separator can be :, / or .. A full hierarchical path must begin with a :, / or .. The path must be contained within double quotes.
- net_2
Required. The name of the second analog object to be shorted to terminal_1. This argument follows the same syntax rules as terminal_1.
- verbose
Specifies whether you want a message to be reported in the [Transcript Window](#) stating that terminal_1 has been shorted to terminal_2.
 - 0 — Does not report a message. Default.

- 1 — Reports a message.

Description

If both objects are analog, they will share a single KCL function; the sum of the through (current for electrical) is the through quantity (contribution) for both. The resulting across quantity (voltage for electrical) will be the across quantity (reference) for both. If the first object is called terminal_1 and the second object is called terminal_2, then this is equivalent to the Eldo command:

```
.CONNECT <N1> <N2>
```

An analog net can be a SPICE node, a Verilog-AMS continuous discipline signal or a VHDL-AMS terminal (scalar, including a scalar element of a composite). A digital net can be a VHDL or VHDL-AMS entity or a Verilog or Verilog-AMS module. A digital net can be scalar (including a scalar element of a composite), a vector or a record.

When the init_terminal_short() procedure successfully shorts the two analog objects and the verbose flag has been set to 1, the following note will be generated:

```
# Note: init_terminal_short [<path>/correct_use.vhd] : Terminal
':top:term_in_package' and ':term' shorted.
```

If the specified path is invalid, the following error message will be generated:

```
# Error: init_terminal_short [<path>/try_bad_path.vhd] : Path is not
correct: ':top:a_wrong-path:terminal1' does not exist in the design.
# Time: 0fs Iteration: 0 Instance: :top
```

When the \$init_terminal_short() system task successfully shorts the two objects and the verbose flag is set to 1, the following note is generated:

```
# Note: init_terminal_short [<path>/top_its.v] : Terminal '.top.uut.term1'
and ':top:term2' shorted.
```

Call Only Once

The call for the VHDL-AMS procedure is performed during the elaboration stage, and can only be done as part of a block or architecture.

The call for the Verilog-AMS system task is performed during the elaboration stage, for this reason the call for this system task must take place in an initial_step global event control statement and must not be nested inside any conditional, loop or event control (other than an initial_step) statement.

How to Spy a Net in a VHDL-AMS Package

The `init_terminal_short()` procedure can be used to spy nets in VHDL-AMS packages. The path for the net must be specified, as shown below:

```
[:<lib>] :<package>:<net>
```

Note

The package must be loaded before trying to access it.

When the library name is not specified in the path, and there is more than one package with the same name in different libraries, Questa ADMS will use the first package that has been declared. When the procedure is successful and the verbose flag is set to 1, the following note is generated:

```
# Note: init_terminal_short [<path>/try_package.vhd] : Terminal
package_user:term_in_package' and ':term' shorted.
```

When the library name is specified it will appear in the generated note:

```
# Note: init_terminal_short [<path>/try_library_package.vhd] : Terminal
':adms2:package_user:term_in_package' and ':term' shorted.
```

SPICE Subckts

When `init_terminal_short` successfully shorts to the node in a SPICE subcircuit (Eldo or ADiT), the following note will be generated:

```
# Note: init_terminal_short [<path>/t1_vhdlams_t2_SPICE.vhd] : Terminal
':t1_vhdlams_t2_SPICE:yits:vhdlams_term' and ':top_eldo_node' shorted.
```

See Limitations (below) for use with Black-box mode for Eldo and ADiT.

It is possible to short to a node within an encrypted model or source with `init_terminal_short`, as long as you specify the full path to the node.

Grounds

`init_terminal_short` can be used to short a net to a ground (reference).

- In SPICE, "0" is the ground reference. Putting the string "0" as a parameter of `init_terminal_short` will short the net to ground.

- In packages, shortcuts to references defined in the "IEEE" library can be done in the same way as a terminal from a package. There are two ways to specify the parameter, as shown below:

```
" :<package_system>:<system>_ground"
" :ieee:<package_system>:<system>_ground"
```

Limitations

The following limitations apply to init_terminal_short.

- The init_terminal_short() procedure cannot be called as a sequential procedure. This procedure should only be used in a concurrent procedure call. If it is not, the following error message is displayed:

```
Only concurrent procedur calls are allowed for procedure
init_terminal_short
```

- The init_terminal_short() procedure cannot be used on a model that has been compiled with the -nodebug argument. If it is, the following error message is displayed:

```
#E Error: init_terminal_short [<path>/try_protected.vhd] : Path is not
correct: ':top:uut:term1'. Content of 'protected' instance not visible.
#E      Time: 0fs  Iteration: 0  Instance: :top
```

- init_terminal_short cannot access nodes of a subcircuit (both Eldo and ADiT) that is running in black-box mode. If you try to access these nodes, the following error message is displayed:

```
# Error: init_terminal_short [<path>/try_eldo_black_box.vhd]
: Path is not correct: ':xeldo_ckt_black_box:Xinv1:in'. Content of an
Eldo black box not visible.
#      Time: 0fs  Iteration: 0  Instance: :try_eldo_black_box:yits
```

- When connecting analog nets to digital nets, init_terminal_short cannot be used inside a user-defined boundary element. If you try to include it, an error message will be reported:

```
# ** Error: (vasim - 2111) init_terminal_short :
Between Signal ':top:\N1#A2D#STD_ULOGIC\:da_its' and terminal
':top:\N1#A2D#STD_ULOGIC\:inh_vdd' is ignored. It's not possible to use
such command in a user model converter model, because a converter could
be added between the two objects.
# +      Time: 0 ps  Iteration: 0  Region: :top:\N1#A2D#STD_ULOGIC\
# +      File: <SOURCE_DIR>/conv.v(33)  Model: E2L
```

Only SPICE built-in boundary elements are supported, user-defined boundary elements are not.

Related Topics

[init_terminal_short Example](#)
[\\$init_terminal_short Example](#)
[init_signal_spy](#)
[enable_signal_spy](#)
[disable_signal_spy](#)
[init_terminal_reference](#)
[init_terminal_contribution](#)
[get_terminal_id](#)
[get_terminal_across_value](#)

init_terminal_short Example

You can mirror the value of an analog object onto an analog target object.

In this example, the value of the analog object `:top:uut:term1` will be mirrored onto the analog target object `term2`.

Procedure

1. Create a file called `top_its.vhd` containing the following VHDL-AMS source code (code related to Net Spy is denoted in red):

```

library ieee, modelsim_lib;
use ieee.math_real.all;
use ieee.std_logic_1164.all;
use modelsim_lib.util.all;
use work.all;

library ieee;
use ieee.electrical_systems.all;

entity top is
end entity top;

architecture terminal_short of top is
    terminal term2: Electrical;
begin
    uut : block
        constant freq : real := 1.0e9 / 2; -- 2 ns
        constant amp : real := 1.0;
        terminal term1: Electrical;
        quantity v_term1 across i_term1 through term1;
    begin -- block uut
        v_term1 == amp * sin(2.0 * MATH_PI * freq * now);
    end block uut;

    its: init_terminal_short(":top:uut:term1", "term2", 1);

```

```
end architecture terminal_short;
```

2. In the same directory, create a command file called *top_itc.cmd* containing the instructions for the analysis:

```
*
.OPTION eps=100n
.TRAN 1ns 10ns
```

3. In the same directory, create a do file called *top_its.do* containing instructions to run the simulation and plot the results:

```
add wave :top:uut:term1
add wave :top:term2

run -all
```

4. In the same directory, create a design library called *adms_lib*:

```
valib adms_lib
```

5. Compile the VHDL-AMS source code into the design library:

```
vacom top_its.vhd
```

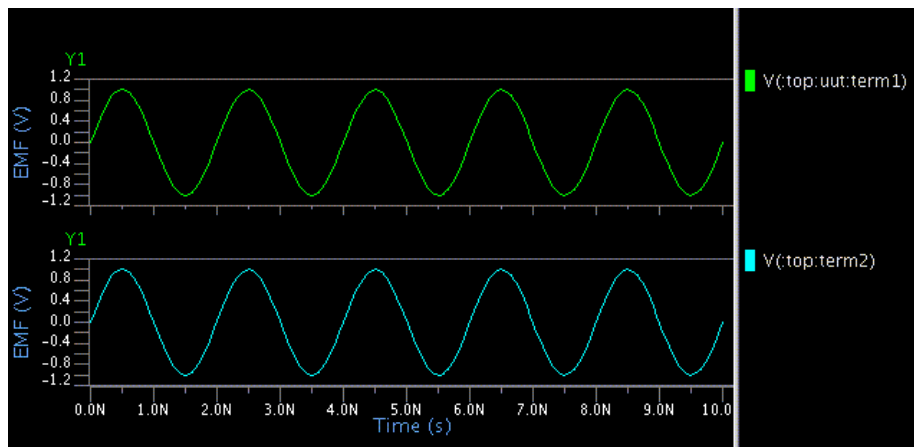
6. Launch the simulator using the command and do files:

```
vasim top -cmd top_its.cmd -do top_its.do &
```

Results

When the simulation is complete the results are displayed in the [Wave Window \(SOLIDO Waveform Analyzer\)](#), as shown in [Figure 13-4](#).

Figure 13-4: init_terminal_short Example



Related Topics

[\\$init_terminal_short Example](#)
[init_terminal_short](#)

\$init_terminal_short Example

The value of an analog object can mirror the voltage onto an analog target object.

In this example, the value of analog object uut.term1 will mirror the voltage onto analog target object term2.

Procedure

1. Create a file called *level.s.v* containing the following Verilog-AMS source code (code related to Net Spy is denoted in red):

```
'include "disciplines.h"
'define PI 3.14159265358979323846
module level1;
  electrical term1;
  parameter amp = 1.0;
  parameter freq = 1.0;
  analog begin
    V(term1) <+ amp * sin($realtime * 2.0 * 'PI * freq);
  end
endmodule
```

2. In the same directory, create a file called *top_its.v* containing the following Verilog-AMS source code (code related to Net Spy is denoted in red):

```
`include "disciplines.h"
module top;
    electrical term2;
    level1 uut();
    analog begin
        @(initial_step)
            $init_terminal_short("uut.term1", "term2", 1);
    end
endmodule
```

3. In the same directory, create a command file called *top_its.cmd* containing the instructions for the analysis:

```
*
.OPTION eps=100n
.TRAN 1ns 10ns
```

4. In the same directory, create a do file called *top_its.do* containing instructions to run the simulation and plot the results:

```
add wave :top:uut:term1
add wave :top:term2

run -all
```

5. In the same directory, create a design library called *adms_lib*:

```
valib adms_lib
```

6. Compile the Verilog-AMS source code into the design library:

```
valog levels.v
valog top_its.v
```

7. Launch the simulator using the command and do files:

```
vasim top -cmd top_its.cmd -do top_its.do &
```

The following note is written to the [Transcript Window](#) if the verbose flag has been set to 1:

```
# Note: init_terminal_short [<path>/top_its.v] :
Terminal '.top.uut.term1' and ':top:term2' shorted.
```

Related Topics

[init_terminal_short Example](#)

[init_terminal_short](#)

init_terminal_reference

Sets the value of the voltage at a destination analog object to the value of the voltage at a reference analog object. This topic describes the VHDL-AMS `init_terminal_reference()` procedure and the Verilog-AMS `$init_terminal_reference()` task.

Syntax

VHDL-AMS Syntax

```
init_terminal_reference(<ref_object>, <dest_object> [, <gain>]
                        [, <verbose>])
```

Verilog-AMS Syntax

```
$init_terminal_reference(<ref_object>, <dest_object> [, <gain>]
                        [, <verbose>])
```

Arguments

- `ref_object`

Required. The name of the analog object that will be taken as the reference. You should provide a full hierarchical path (or relative downward path with reference to the calling region) to the analog object.

By default, `init_terminal_reference` uses a colon (:) as a path separator. You can change this behavior by setting the `SignalSpyPathSeparator` variable (see [Questa SIM Simulation Control Variables](#)) in the `modelsim.ini` file. The path separator can be `:`, `/` or `..`. A full hierarchical path must begin with a `:`, `/` or `..`. The path must be contained within double quotes.

- `dest_object`

Required. The name of the destination analog object to which the reference will be applied. This argument follows the same syntax rules as `ref_object`.

- `gain`

The gain of the voltage between the objects `ref_object` and `dest_object`:

voltage at `dest_object` = gain(voltage of `ref_object`)

If omitted, the default value is 1.0.

- `verbose`

Specifies whether a message is reported in the [Transcript Window](#) during the elaboration phase.

- 0 — Does not report a message. Default.

- 1 — Reports a message.

Description

The `init_terminal_reference` call sets the value of the voltage at the destination analog object (`dest_object`), to the value of the voltage at the reference analog object (`ref_object`). This is equivalent to the Eldo command:

```
Exx <dest_object> 0 <ref_object> 0
* for a Voltage Controlled Voltage Source (VCVS)
```

The two analog objects can be any analog objects in the design. `init_terminal_reference` returns nothing by default, but can be set to report messages using the `verbose` argument.

When the `init_terminal_reference()` procedure successfully shorts the two analog objects, the following note will be generated:

```
# Note: init_terminal_reference [<path>/correct_use.vhd] : Terminal
':top:source' references ':dest' with gain = 1.0 .
```

If the analog object does not exist or the specified path is not a analog object, the following error messages will be generated:

```
# Error: init_terminal_reference [<path>/try_term_is_sig.vhd] : Object
':top:sig' is not a terminal.
#   Time: 0fs Iteration: 0 Instance: :top

# Error: init_terminal_reference [<path>/try_unknown_term.vhd] : Object
':top:unknown_terminal' is not a terminal.
#   Time: 0fs Iteration: 0 Instance: :top
```

If the specified path is invalid, the following error message will be generated:

```
# Error: init_terminal_reference [<path>/try_bad_path.vhd] : Path is not
correct: ':top:a_wrong-path:terminal1' does not exist in the design.
#   Time: 0fs Iteration: 0 Instance: :top
```

When the `$init_terminal_reference()` system task successfully shorts the two analog objects and the `verbose` flag is set to 1, then following note will be generated:

```
# Note: init_terminal_reference [<path>/top_itr.v] :
Terminal '.top.uut.source' references ':top:dest1' with gain= 2.
```

Call Only Once

The call for the VHDL-AMS procedure is performed during the elaboration stage, and can only be done as part of a block or architecture.

The call for the Verilog-AMS system task is performed during the elaboration stage, for this reason the call for this system task must take place in an `initial_step` global event control statement. It must not be nested inside any conditional, loop or event control (other than an `initial_step`) statement.

How to Spy a Net in a VHDL-AMS Package

The `init_terminal_reference()` procedure can be used to spy nets in VHDL-AMS packages. The path for the net must be specified, as shown below:

```
[:<lib>] :<package>:<net>
```

Note

The package must be loaded before trying to access it.

When the library name is not specified in the path, and there is more than one package with the same name in different libraries, Questa ADMS will use the first package that has been declared. When the procedure is successful the following note is generated:

```
# Note: init_terminal_reference [<path>/try_package.vhd] : Terminal
package_user:term_in_package' references ':term' with gain= 2.0 .
```

When the library name is specified and the procedure is successful, the following note is generated:

```
# Note: init_terminal_reference [<path>/try_library_package.vhd] : Terminal
':adms2:package_user:term_in_package' references ':term' with gain= 2.5 .
```

SPICE Subckts

When the `init_terminal_reference()` procedure successfully shorts to the node in a SPICE subckt (Eldo or ADiT), the following note will be generated:

```
# Note: init_terminal_reference [<path>/t1_vhdlams_t2_SPICE.vhd] : Terminal
':t1_vhdlams_t2_SPICE:yits:vhdlams_term' references ':top_eldo_node' with
gain= 3.0 .
```

See Limitations (below) for use with Eldo ADiT black-box mode.

It is possible to short to a node within an encrypted model or source with the `init_terminal_reference`, as long as you specify the full path to the node.

Grounds

The `init_terminal_reference()` procedure can be used to short a net to a ground (reference).

- In SPICE, "0" is the ground reference. Putting the string "0" as a parameter of the `init_terminal_reference()` procedure, will short the net to ground.
- In packages, shortcuts to references defined in the "IEEE" library can be done in the same way as a terminal from a package. There are two ways to specify the parameter, as shown below:

```
" :<package_system>:<system>_ground"
" :ieee:<package_system>:<system>_ground"
```

Limitations

The following limitations apply to the use of `init_terminal_reference`.

- The procedure `init_terminal_reference()` cannot be called as a sequential procedure. This procedure should only be used in a concurrent procedure call. If it is not, the following error message is displayed:

```
Only concurrent procedure calls are allowed for procedure
init_terminal_reference.
```

- `init_terminal_reference()` cannot be used on a model that has been compiled with the `-nodebug` argument. If it is, the following error message is generated:

```
#E Error: init_terminal_reference [<path>/try_protected.vhd] : Path is
not correct: ':top:uut:term1'. Content of 'protected' instance not
visible.
#E      Time: 0fs  Iteration: 0  Instance: :top
```

- `init_terminal_reference()` cannot access nodes of a subcircuit (both Eldo and ADiT) that is running in black-box mode. If you try to access these nodes, the following error message is generated:

```
# Error: init_terminal_reference [<path>/try_eldo_black_box.vhd]
: Path is not correct: ':xeldo_ckt_black_box:Xinv1:in'. Content of an
Eldo black box not visible.
#      Time: 0fs  Iteration: 0  Instance: :try_eldo_black_box:yits
```

- `init_terminal_reference()` cannot have as destination object a terminal with a source. If you try to do this, the following error message is generated:

```
# Error: (eldo - 27) Inductor/'voltage source' loop found
# Voltage loop made of 0 voltage sources only can be accepted
using .OPTION LOOPV0
# To downgrade this error to a warning use .OPTION
VOLTAGE_LOOP_SEVERITY = WARNING
# ->EITR_0 Y1.C1.E 0
# ->Y1.C1: voltage source between Y1.C1.E and 0
```

Related Topics

[init_terminal_reference Example](#)

[\\$init_terminal_reference Example](#)

[init_signal_spy](#)

[enable_signal_spy](#)

[disable_signal_spy](#)

[init_terminal_short](#)

[init_terminal_contribution](#)

[get_terminal_id](#)

[get_terminal_across_value](#)

init_terminal_reference Example

The value of a source can mirror the voltage onto a destination with a gain.

In this example, the value of `:top:uut:source` will mirror the voltage onto `:top:dest1` with 2.0 as gain and in `dest2` with a gain of 5.0.

Procedure

1. Create a file called `top_itr.vhd` containing the following VHDL-AMS source code (code related to Net Spy is denoted in red):

```
library ieee, modelsim_lib;
use ieee.math_real.all;
use ieee.std_logic_1164.all;
use modelsim_lib.util.all;
use work.all;

library ieee;
use ieee.electrical_systems.all;

entity top is
end entity top;
```

```

architecture terminal_reference of top is
  terminal dest1: Electrical;
  terminal dest2: Electrical;
begin
  uut : block
    constant freq : real := 1.0e9 / 2; -- 2 ns
    constant amp  : real := 1.0;
    terminal source: Electrical;
    quantity v_source across i_source through source;
  begin -- block uut
    v_source == amp * sin(2.0 * MATH_PI * freq * now);
  end block uut;

  itr1: init_terminal_reference(    ":top:uut:source",
                                   ":top:dest1", 2.0, 1);

  itr2: init_terminal_reference(    "uut:source",
                                   "dest2", 5.0, 1);

end architecture terminal_reference;

```

2. In the same directory, create a command file called *top_itr.cmd* containing the instructions for the analysis:

```

.OPTION eps=100n
.TRAN 1ns 10ns

```

3. In the same directory, create a do file called *top_itr.do* containing instructions to run the simulation and plot the results:

```

add wave :top:uut:source
add wave :top:d*

run -all

```

4. In the same directory, create a design library called *adms_lib*:

```

valib adms_lib

```

5. Compile the VHDL-AMS source code into the design library:

```

vacom top_itr.vhd

```

6. Launch the simulator using the command and do files:

```

vasim top -cmd top_itr.cmd -do top_itr.do &

```

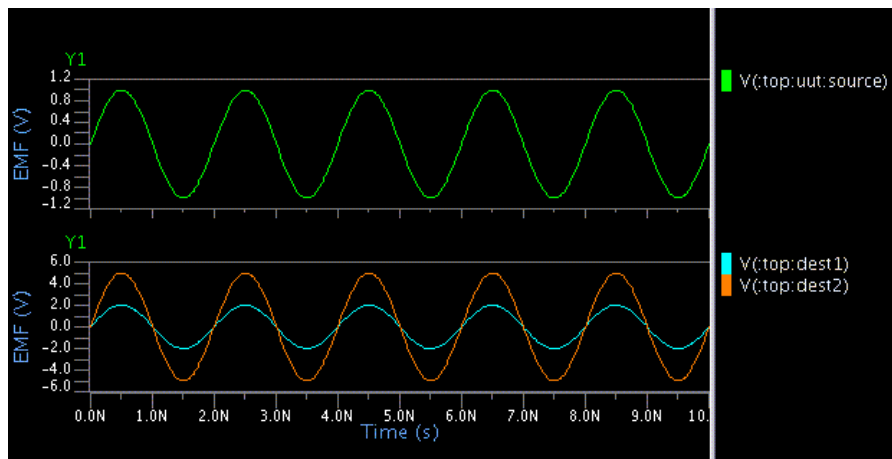
Results

The following note is written to the **Transcript Window** if the verbose flag has been set:

```
# Note: init_terminal_reference [<path>/top_itr.vhd] :  
Terminal ':top:uut:source' references ':top:dest2' with gain= 5 .  
# Note: init_terminal_reference [<path>/top_itr.vhd] :  
Terminal ':top:uut:source' references ':top:dest1' with gain= 2 .
```

When the simulation is complete the results can be displayed in the **Wave Window (SOLIDO Waveform Analyzer)**, as shown in **Figure 13-5**.

Figure 13-5: init_terminal_reference Example



Related Topics

[init_terminal_reference](#)
[\\$init_terminal_reference Example](#)

\$init_terminal_reference Example

The value of a source can mirror the voltage onto a destination with a gain.

In this example, the value of .top.uut.source will mirror the voltage onto dest1 with 2.0 as gain and in .dest2 with a gain of 5.0.

Procedure

1. Create a file called *level1.v* containing the following Verilog-AMS source code (code related to Net Spy is denoted in red):

```
'include "disciplines.h"
'define PI 3.14159265358979323846
module level1;
    electrical source;
    parameter amp = 1.0;
    parameter freq = 1.0;
    analog begin
        V(source) <+ amp * sin($realtime * 2.0 * 'PI * freq);
    end
endmodule
```

2. In the same directory, create a file called *top_itr.v* containing the following Verilog-AMS source code (code related to Net Spy is denoted in red):

```
`include "disciplines.h"
module top;
    electrical dest1;
    electrical dest2;
    level1 uut();
    analog begin
        @(initial_step) begin
            $init_terminal_reference(".top.uut.source", "dest1", 2, 1);
            $init_terminal_reference("uut:source", ".dest2", 5, 1);
        end
    end
endmodule
```

3. In the same directory, create a command file called *top_itr.cmd* containing the instructions for the analysis:

```
*
.OPTION eps=100n
.TRAN 1ns 10ns
```

4. In the same directory, create a do file called *top_itr.do* containing instructions to run the simulation and plot the results:

```
add wave :top:uut:source
add wave :top:dest1
add wave :top:dest2

run -all
```


5. In the same directory, create a design library called adms_lib:

```
valib adms_lib
```

6. Compile the Verilog-AMS source code into the design library:

```
valog levels.v  
valog top_itr.v
```

7. Launch the simulator using the command and do files:

```
vasim top -cmd top_itr.cmd -do top_itr.do &
```

The following note is written to the [Transcript Window](#) if the verbose flag has been set:

```
# Note: init_terminal_reference [<path>/top_itr.v] :  
Terminal ':top: uut:source' references '.dest2' with gain= 5.  
# Note: init_terminal_reference [<path>/top_itr.v] :  
Terminal '.top. uut.source' references ':top:dest1' with gain= 2.
```

Related Topics

[init_terminal_reference](#)
[init_terminal_short Example](#)

init_terminal_contribution

Sets the value of the voltage at the destination analog object to the value of the current (or contribution) of reference analog object. This topic describes the VHDL-AMS `init_terminal_contribution()` procedure and the Verilog-AMS `$init_terminal_contribution()` task.

Syntax

VHDL-AMS Syntax

```
init_terminal_contribution(<ref_object>, <dest_object> [, <gain>]
                          [, <verbose>])
```

Verilog-AMS Syntax

```
$init_terminal_contribution(<ref_object>, <dest_object> [, <gain>]
                          [, <verbose>])
```

Arguments

- `ref_object`

Required. The name of the analog object that will be taken as the reference. A full hierarchical path (or relative downward path with reference to the calling region) to a Questa ADMS analog object.

By default, `init_terminal_contribution` uses a colon (:) as a path separator. You can change this behavior by setting the `SignalSpyPathSeparator` variable (see [Questa SIM Simulation Control Variables](#)) in the `modelsim.ini` file. The path separator can be `:`, `/` or `..`. A full hierarchical path must begin with a `:`, `/` or `..`. The path must be contained within double quotes.

When mirroring vector terminals, either the VHDL or Verilog syntax can be used. For example, `'()` can be replaced with `'[]`, and the directions `'to` or `'downto` can be replaced with `'::`.

- `dest_object`

Required. The name of the Spice analog object, VHDL-AMS terminal, or Verilog-AMS analog signals object on which the current controlled voltage source is applied.

- `gain`

The gain between the current (or contribution) of the `ref_object` and voltage of the `dest_object`:

voltage of `dest_object` = gain(current of `ref_object`)

If omitted, the value is equal to 1.0.

- `verbose`

Default is 0 - no message. When set to a value other than 0, a message will be displayed during the elaboration phase.

Description

The `init_terminal_contribution` call sets the value of the voltage at the destination analog object to the value of the current (or contribution) of reference analog object. If the source is not an analog object, the resulting value is always 0.0 and in which case, it has no meaning. This is equivalent to the Eldo command:

```
Hxx <dest_object> 0 <ref_object> <gain>
* for a Current Controlled Voltage Source (CCVS)
```

The two analog objects that are given as parameters of this call can be any analog objects in the design. `init_terminal_contribution` returns nothing by default, but can be set to report messages using the `verbose` argument. When the `init_terminal_contribution()` procedure is successful and the `verbose` flag is set, then following note will be generated:

```
# Note: init_terminal_contribution [<SOURCE_DIR>/top.vhd] :
Terminal ':top:uut:source' references ':dest' contribution with gain= 2 .
```

If the analog object does not exist or the specified path is not a analog object, the following error messages will be generated:

```
# Error: init_terminal_contribution [<path>/try_term_is_sig.vhd] : Object
':top:sig' is not a terminal.
#   Time: 0fs Iteration: 0 Instance: :top

# Error: init_terminal_contribution [<path>/try_unknown_term.vhd] : Object
':top:unknown_terminal' is not a terminal.
#   Time: 0fs Iteration: 0 Instance: :top
```

If the specified path is invalid, the following error message will be generated:

```
# Error: init_terminal_contribution [<path>/try_bad_path.vhd] : Path is not
correct: ':top:a_wrong-path:terminal1' does not exist in the design.
#   Time: 0fs Iteration: 0 Instance: :top
```

When the `$init_terminal_contribution()` system task is successful in founding the two analog objects and the `verbose` flag is set, then following note is displayed:

```
# Note: init_terminal_contribution [<path>/top_itc.v] : Terminal
'.top.uut.source' references ':top:dest' contribution with gain= 2.
```

SPICE Subckts

When the `init_terminal_contribution()` procedure successfully connects to the interface node of a SPICE subckt (Eldo or ADiT), the following note will be generated:

```
# Note: init_terminal_contribution [<path>/t1_vhdlams_t2_SPICE.vhd] :
Terminal ':t1_vhdlams_t2_SPICE:yits:vhdlams_term' and ':top_eldo_node'
shorted.
```

It is possible to contribute to a node within an encrypted model or source with `init_terminal_contribution`, provided you specify the full path to the node.

Warning and Error Messages

If the source object is not a port terminal, the following message is displayed:

```
# Error: (vasim - 2123) init_terminal_contribution : Object
':top:source_term' is not a port terminal.
```

Or if the source object is a ground (reference):

```
# ** Error: (vasim - 2123) init_terminal_contribution : Object ':top:0' is
not a port terminal.
```

Limitations

The following limitations apply to VHDL-AMS [init_terminal_contribution](#):

- The VHDL-AMS `init_terminal_contribution()` procedure cannot be called as a sequential procedure. This procedure should only be used in a concurrent procedure call. If it is not, the following error message is displayed:

```
** Error: (vasim-2155) init_terminal_contribution : This procedure should
only be used in concurrent procedure call.
```

- The VHDL-AMS `init_terminal_contribution()` procedure cannot be used on a model that has been compiled with the `-nodebug` argument. If it is, the following error message is displayed:

```
# ** Error: (vasim-2141) init_terminal_contribution : The command is
ignored. A model in the path ':top:uut:r1:t2' is not visible as it is
'protected'.
```

- The `init_terminal_contribution()` procedure cannot be used where the source is a VHDL-AMS port terminal of a VHDL block.

Related Topics

[init_terminal_contribution Example](#)
[\\$init_terminal_contribution Example](#)
[init_terminal_reference](#)
[get_terminal_id](#)
[get_terminal_across_value](#)

init_terminal_contribution Example

An object can be equivalent to another object, with a gain.

In this example, the value of “:top:uut:r1:t2” will be equivalent to “:dest” with a gain of 2.0.

Procedure

1. Create a file called *resistor.vhd* containing the following code:

```
library ieee;
    use ieee.electrtical_systems.ALL;

entity resistor is
    generic (Res : real := 10.0);
    port (terminal ta, tb : electrical);
end entity resistor;

architecture vhdams of resistor is
    quantity vb across ib through ta to tb;
begin
    vb == Res * ib;
end architecture vhdams;
```

2. In the same directory, create a file called *top_itc.vhd* containing the following code:

```
library ieee, modelsim_lib;
    use ieee.math_real.all;
    use ieee.std_logic_1164.all;
    use modelsim_lib.util.all;
    use work.all;

library IEEE;
    use IEEE.electrical_systems.all;

entity top is
end entity top;

architecture terminal_contribution of top is
    terminal dest: Electrical;
    terminal top_source: Electrical;
```

```

constant res : real := 1.0;
constant offset : real := 10.0;

quantity                                v_top_source across
                                         i_top_source through top_source;
begin
  r1 : v_top_source == res * i_top_source - offset;

  uut : block
    port (terminal source : Electrical);
    port map (source => top_source);

    terminal t2: Electrical;
    quantity v_t2 across i_t2 through t2;

    constant freq : real := 1.0e9 / 2; -- 2 ns
    constant amp  : real := -8.0;
    quantity v_source across i_source through source;
  begin -- block uut
    v_source == amp * sin(2.0 * MATH_PI * freq * now);

    r1: entity work.resistor          generic map (1.0)
                                         port map(source, t2);
    v_t2 == 0.0;
  end block uut;

  itc: init_terminal_contribution(":top:uut:r1:t2",
                                ":dest", 2.0, 1);

end architecture terminal_contribution;

```

3. In the same directory, create a file called *top_itc.cmd*, containing the instructions for the analysis:

```

*
.OPTION eps=100n
.TRAN 1ns 10ns

```

4. In the same directory, create a do file called *top_itc.do* containing instructions to run the simulation and plot the results:

```

add wave :top:uut:r1:ta
add wave :top:uut:source :top:dest

run -all

```

5. In the same directory, create a design library called *adms_lib*:

```

valib adms_lib

```

6. Compile the VHDL-AMS source code into the design library:

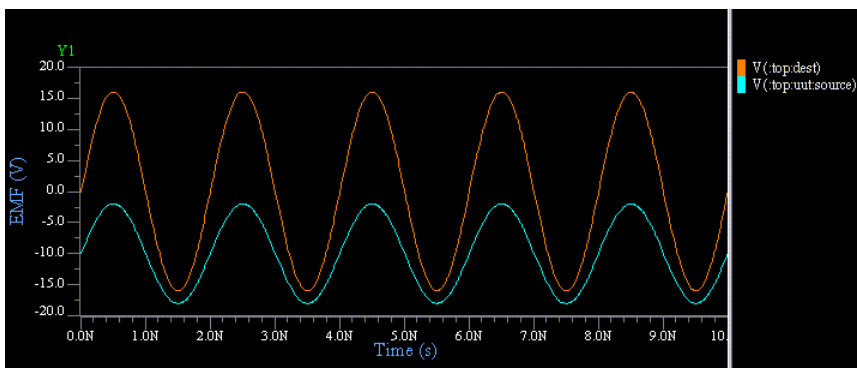
```
vacom resistor.vhd
vacom top_itc.vhd
```

7. Launch the simulator using the command and do files:

```
vasim top -cmd top_itc.cmd -do top_itc.do &
```

When the simulation is complete the results can be displayed in the **Wave Window (Solido Waveform Analyzer)**, as shown in **Figure 13-6**.

Figure 13-6: init_terminal_contribution Example



The following note is written to the **Transcript Window** if the verbose flag has been set:

```
# Note: init_terminal_contribution [<path>/top_itc.vhd] :
Terminal ':top:uut:source' references ':dest' contribution with gain=
2 .
```

Related Topics

[\\$init_terminal_contribution Example](#)

[init_terminal_contribution](#)

\$init_terminal_contribution Example

You can use \$init_terminal_contribution to set the value at a destination terminal with a gain of 2.

In this example, the value of "uut.source" will be equivalent to "dest" with gain of 2.

Procedure

1. Create a file called *level1.v* containing the following source code:

```
'include "disciplines.h"
#define PI 3.14159265358979323846
module level1 (source);
    inout source;
    electrical source;
    parameter amp = 8.0;
    parameter freq = 1.0 / 2.0;
    analog begin
        V(source) <+ amp * sin($realtime * 2.0 * 'PI * freq);
    end
endmodule
```

2. In the same directory, create a file called *top_itc.v* containing the following source code:

```
`include "disciplines.h"
module top;
    electrical top_source;
    electrical dest;
    parameter res = 1.0;
    parameter offset = 5.0;
    level1 uut(top_source);
    analog begin
        V(top_source) <+ res * I(top_source) - offset;
        @(initial_step) begin
            $init_terminal_contribution("uut.source", "dest", 2, 1);
        end
    end
end
endmodule
```

3. In the same directory, create a command file called *top_itc.cmd* containing the instructions for the analysis:

```
*
.OPTION eps=100n
.TRAN 1ns 10ns
```

4. In the same directory, create a do file called *top_itc.do* containing instructions to run the simulation and plot the results:

```
add wave :top:uut:source
add wave :top:dest
run -all
```


5. In the same directory, create a design library called adms_lib:

```
valib adms_lib
```

6. Compile the Verilog-AMS source code into the design library:

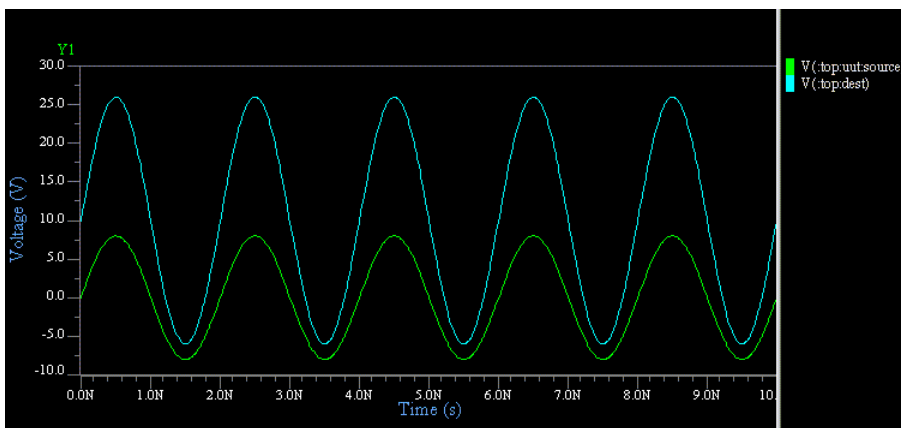
```
valog levels.v
valog top_itc.v
```

7. Launch the simulator using the command and do files:

```
vasim top -cmd top_itc.cmd -do top_itc.do &
```

When the simulation is complete the results can be displayed in the [Wave Window \(Solido Waveform Analyzer\)](#), as shown in [Figure 13-7](#).

Figure 13-7: init_terminal_contribution Example



Related Topics

[\\$init_terminal_reference Example](#)

[init_terminal_contribution](#)

get_terminal_id

Returns a reference ID for a Questa ADMS analog object. The function can be called from within a VHDL description or the digital part of a VHDL-AMS description.

Syntax

```
get_terminal_id(<ref_object> [, <verbose>])
```

Arguments

- **ref_object**

Required. This is the name of the analog object that will be referenced in the terminal reference id. A full hierarchical path (or relative downward path with reference to the calling region) to an analog object must be specified.

By default, `init_terminal_id` uses a colon (:) as a path separator. You can change this behavior by setting the `SignalSpyPathSeparator` variable (see [Questa SIM Simulation Control Variables](#)) in the `modelsim.ini` file. The path separator can be `;`, `/` or `..`. A full hierarchical path must begin with a `;`, `/` or `..`. The path must be contained within double quotes.

For vector terminals, either the VHDL or Verilog syntax can be used. For example, `'()` can be replaced with `'[]`, and the directions `'to` or `'downto` can be replaced with `'::`.

- **verbose**

Specifies whether you want a message reported in the [Transcript Window](#) stating that `terminal_name` has been referenced by the reference id.

- 0 — Does not report a message. Default.
- 1 — Reports a message.

Return Values

An integer value.

Description

To obtain the across value of the analog object with the returned reference ID, the [get_terminal_across_value](#) function must be used. An example of using the `get_terminal_id` function is shown in [get_terminal_across_value Example](#).

Call Only Once

The call for this function is performed during the elaboration/simulation stage, and can only be used within a declaration part or in a process.

Library Definition

To use the `get_terminal_id` function from within a VHDL description the following must be specified:

```
library MGC_AMS;  
use MGC_AMS.mixed_AD_util.all;
```

To use the `get_terminal_id` function from within a VHDL-AMS description the following must be specified:

```
library modelsim_lib;  
use modelsim_lib.util.all;
```

Related Topics

[init_signal_spy](#)

[enable_signal_spy](#)

[disable_signal_spy](#)

[init_terminal_short](#)

[init_terminal_reference](#)

[init_terminal_contribution](#)

[get_terminal_across_value](#)

get_terminal_across_value

Obtains the across value (voltage for electrical) of an analog object. The function can be called from within a VHDL description or the digital part of a VHDL-AMS description. The analog object is specified in the function `get_terminal_id`.

Syntax

```
get_terminal_across_value(<terminal_id>)
```

Arguments

- terminal_id
Required. This is the value of the reference terminal_id returned by the function [get_terminal_id](#).

Return Values

A real value.

Description

An example of using the `get_terminal_across_value` and `get_terminal_id` functions is shown in [get_terminal_across_value Example](#).

Call Only Once

The call for this function is performed during the simulation stage, and can only be used within a process.

Library Definition

To use the `get_terminal_across_value` function from within a VHDL description the following must be specified:

```
library MGC_AMS;  
use MGC_AMS.mixed_AD_util.all;
```

To use the `get_terminal_across_value` function from within a VHDL-AMS description the following must be specified:

```
library modelsim_lib;  
use modelsim_lib.util.all;
```

Related Topics

[get_terminal_across_value Example](#)

[get_terminal_id](#)

[Access to Analog Objects from SystemVerilog](#)

get_terminal_across_value Example

You can use `get_terminal_across_value` to obtain the value of a referenced analog object after using the `get_terminal_id` function to obtain the reference id of the analog object.

The following example obtains the reference id of the analog object `":top:b1:term1"` using the `get_terminal_id` function. The value of the referenced analog object is then obtained using the `get_terminal_across_value` function, and is assigned to the signal `term1_value`.

Procedure

1. Create a file called `top_gti_gtav.vhd` containing the following source code:

```
library ieee, disciplines, modelsim_lib;
use ieee.math_real.all;
use IEEE.electrical_systems.all;
use modelsim_lib.util.all;

entity top is
end entity top;

architecture ams of top is
    signal term1_value : Real := 0.0;
begin -- ams
b1: block
    constant freq : real := 1.0e8 / 2;
    constant amp : real := 1.0;
    constant offset : real := 2.0;

    terminal term1: Electrical;
    quantity v_term1 across i_term1 through term1;
begin -- block b1
    v_term1 == amp * sin(offset + (2.0 * MATH_PI * freq * now));
end block b1;

p1: process
    constant term1_id : integer := get_terminal_id(":top:b1:term1");
begin
    wait for 10 ns;
    l1 : for i in 1 to 250 loop
        term1_value <= get_terminal_across_value(term1_id);
        wait for 100 ps;
    end loop l1;
```

```

    wait for 20 ns;
    l2 : for i in 1 to 20 loop
        term1_value <= get_terminal_across_value(term1_id);
        wait for 1 ns;
    end loop l2;
    wait;
end process p1;
end architecture ams;

```

2. In the same directory, create a command file called *top_gti_gtav.cmd* containing the instructions for the analysis:

```

*
.OPTION eps=100n
.TRAN 100n 100n

```

3. In the same directory, create a do file called *top_gti_gtav.do* containing instructions to run the simulation and plot the results:

```

add wave :top:b1:term1 :top:term1_value
run -all

```

4. In the same directory, create a design library called *adms_lib*:

```

valib adms_lib

```

5. Compile the source code into the design library:

```

vacom top_gti_gtav.vhd

```

6. Launch the simulator using the command and do files:

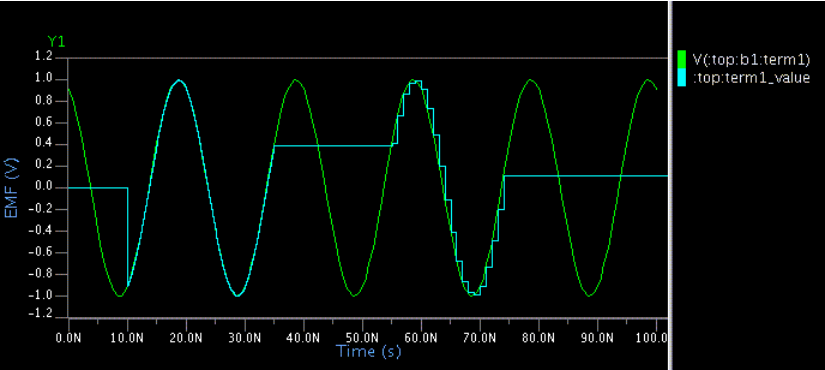
```

vasim top -cmd top_gti_gtav.cmd -do top_gti_gtav.do &

```

When the simulation is complete the results can be displayed in the [Wave Window \(Solido Waveform Analyzer\)](#), as shown in [Figure 13-8](#).

Figure 13-8: get_terminal_across_value Example



Related Topics

[get_terminal_across_value](#)

14. Value Change Dump (VCD) Files

You can use Value Change Dump (VCD) files to re-simulate specific design blocks as if they were connected to the whole of the parent design.

Overview of VCD Files.....	14-1
Creating an Extended VCD File.....	14-1
Extended VCD File Creation Example.....	14-3
Creating a Four-State VCD File.....	14-4

Overview of VCD Files

A VCD file is an ASCII file containing header information, variable definitions, and variable value changes. The VCD file format is specified in the IEEE 1364 standard.

VCD is in common use for Verilog designs, and is controlled by VCD system task calls in the Verilog source code. Questa ADMS provides simulator command equivalents for these system tasks and extends VCD support to VHDL designs; the Questa ADMS commands can be used on either VHDL or Verilog designs from any Questa SIM VHDL architecture or Verilog module.

In order to re-simulate a block of a design under the same conditions as if it was connected to the whole design using a VCD file, all of the ports in the VHDL or Verilog top design unit must be captured in a test bench. Once the ports are captured, the selected portion can only be re-simulated at the top level.

There are two separate flows for this process. Each creates different types of VCD file:

- Extended VCD files (see [Creating an Extended VCD File](#))

Variable changes are recorded in all states (U, X, 0, 1, Z, W, L, H and -), signal strength information, and port driver data. In this flow, the vcd dumptests command is used to write the file (performs both port splitting and signal dumping) and then it is loaded using vasim with the -vcdstim option.

This method is more accurate, however, much larger VCD files are generated.

- Four-State VCD files (see [Creating a Four-State VCD File](#))

Variable changes are recorded in states 0, 1, X and Z. In this flow, the splitio, vcd files, and vcd add commands are used to write the file, and then it is loaded using the vasim command with the -vcread option.

Creating an Extended VCD File

The extended VCD flow was introduced in Questa SIM/ModelSim versions 5.5 and later, and is the recommended flow. This flow produces an extended VCD file with variable changes in all states, signal strength information, and port driver data.

Use the following procedure to create an extended VCD file in Questa ADMS.

Prerequisites

- The top design must be VHDL or Verilog. The design must have been compiled using vcom.

Procedure

1. With the design loaded into the Questa ADMS GUI, specify the VCD file name and the objects to be added using the vcd dumpports command:

```
vcd dumpports [<options>] <object_name>
```

Run the simulation in the normal way.

If no file name is specified with the vcd dumpports command options, the data will be saved to the default file *dumpports.vhd* in the working directory.

Alternatively, the vcd dumpports command can be run in batch mode, by adding the following command to a do file:

```
# contents of the .do file  
vcd dumpports [<options>] <object_name>
```

If you use this method, invoke the simulation using the vasim command with the -do option:

```
vasim whole_design -do <do_file>
```

2. When the simulation has finished either restart or exit the program; this will close all of the VCD files that are currently open.

Results

Once all of the ports have been captured in a test bench, the selected portion of the design can be re-simulated at the top level using the VCD file as the stimulus. The new simulation has to be performed using the vasim command on the model model_name with the option -vcdstim.

```
vasim model_name -vcdstim <file_name>
```

Related Topics

[Extended VCD File Creation Example](#)

[vcd dumpports \[Questa ADMS Command Reference\]](#)

[Creating a Four-State VCD File](#)

Extended VCD File Creation Example

You can produce a VCD file that contains stimuli information for a particular block. You can then re-simulate the block using the saved stimuli information.

For example:

```
valib my_lib
vacom my_design.vhd -ms
vasim my_ent my_arch
vcd dumpports -file dump_extended.vcd -in -out :e:count1:*
view *
add wave *
run 100ns
quit -f
vacom test_bench -ms
vasim test_bench -vcdstim dump_extended.vcd
view *
add wave *
run 100ns
```

The design must be compiled using the Questa SIM compiler before it can be loaded in Questa ADMS. To do this, the following commands are used:

```
valib my_lib
vcom my_design.vhd
vacom my_design.vhd -ms
```

The following command will create a VCD file, which will contain the port driver data from the simulation. The -file option specifies the name of the VCD file that will contain the port information; this file will be stored in the working directory unless another location is specified. The -in option will record the data on ports of type IN. The -out option will record the data on ports of type OUT. :e:count1:* specifies the block in which the port information will be recorded. In this example, changes on the ports within count1 will be recorded.

```
vcd dumpports -file dump_extended.vcd -in -out :count1:*
```

The following are standard Questa ADMS commands; they will view all of the windows, add all of the nets in to the [Wave Window \(Solido Waveform Analyzer\)](#), run the simulation, and then quit Questa ADMS.

```
view *
add wave *
run 100ns
quit -f
```

The following command will load the test_bench, which instantiates the block count1 that is being re-simulated. The command can be run from the command line or from the Transcript window. The option -vcdstim specifies the name of the VCD file that contains all of the stimuli information.

```
vasim test_bench -vcdstim dump_extended.vcd
```

As before, add all of the signals into the Wave window (Solido Waveform Analyzer) and run the simulation. The same results are now visible in the Wave window as were visible when the block count1 was simulated with the whole design.

Related Topics

- Creating an Extended VCD File
- Four-State VCD File Example
- vcd dumptports [Questa ADMS Command Reference]

Creating a Four-State VCD File

This section describes how to produce a four-state VCD file that will record the variable changes in 0, 1, X and Z. Signal strength will not be recorded.

Four-State VCD files are less accurate than extended VCD files (see [Creating an Extended VCD File](#)).

Creating a VCD File..... 14-4

Re-simulation.....14-6

Four-State VCD File Example..... 14-6

Creating a VCD File

The generated VCD file will be located in the working directory. To create a VCD file, Questa SIM must be the top design.

Generating a VCD File in GUI Mode..... 14-4

Generating a VCD File in Batch Mode.....14-5

Generating a VCD File in GUI Mode

You can produce a VCD file using the Questa ADMS GUI.

Prerequisites

- The design must have been compiled using vcom.
- The design must have been loaded into the Questa ADMS GUI.

Procedure

1. With the design loaded, the driving contribution of a port can be recorded using the `splitio` command. By default the `splitio` command will only affect ports of type INOUT however, options can be specified to extend its use to ports of type OUT.

```
splitio [<options>] <signal_name>
```

For more information, and a full list of the options available, refer to [splitio](#) in the *Questa ADMS Command Reference*.

2. Specify the name of the VCD file with the `vcd files` command:

```
vcd files <file_name> [<options>]
```

For a full list of the options available, refer to [vcd files](#) in the *Questa ADMS Command Reference*.

3. Add the required signals to the file with the `vcd add` command:

```
vcd add [<options>] <item_name>
```

For a full list of the options available, refer to [vcd add](#) in the *Questa ADMS Command Reference*.

Note

In Questa ADMS the path separator for the <item_name> must to be a colon ":".

4. Run the simulation in the normal way.
5. Once the simulation has finished either restart or exit the program, this will close all of the VCD files that are currently open.

Generating a VCD File in Batch Mode

All of the commands can be contained within a single do file.

Note

Only a *four-state* VCD file can be opened with Solido Waveform Analyzer.

Prerequisites

- The design must have been compiled using `vcom`.

Procedure

1. Add the following command to a do file:

```
# contents of the .do file
splitio <signal_name>
vcd files <file_name> [<options>]
vcd add [<options>] <item_name>
```

2. Invoke the simulation using the vasim command with the -do option, as shown below.

```
vasim whole_design -do <do_file>
```

Re-simulation

You can re-simulate a block at the top level using a VCD file as the stimulus.

Before you can re-simulate a block, all of the block's ports must be captured in a test bench. You can then re-simulate the block at the top level using the VCD file as the stimulus.

The new simulation in Questa ADMS is performed using the vasim command on the model model_name with the option -vcdread:

```
vasim model_name -vcdread <file_name>
```

Related Topics

[Four-State VCD File Example](#)
[Creating an Extended VCD File](#)

Four-State VCD File Example

You can produce a VCD file and use the VCD file to simulate the selected portion of a design.

For example:

```
valib my_lib
vacom my_design.vhd -ms
vasim my_ent my_arch
splitio count1
vcd files dump.vcd -nomap -direction
vcd add -file dump.vcd -ports :e:count1:*
view *
add wave *
```

```
run 100ns
quit -f
vacom test_banch -ms
vasim test_bench -vcdread dump.vcd
view *
add wave *
run 100ns
```

The design must be compiled using the Questa SIM compiler before it can be loaded in Questa ADMS. To do this, the following commands are used:

```
valib my_lib
vcom my_design.vhd
vacom my_design.vhd -ms
```

The splitio command has been used without any options; this will only record the driving contribution of the ports of type INOUT.

```
splitio count1
```

The vcd files command specifies the filename *dump.vcd*, in which the information will be stored. This file will be placed in the working directory. The -direction option specifies that the port type recorded in the VCD header shall be one of the following: IN, OUT, or INOUT. The -nomap option (used only on a VHDL model) specifies signals of type std_logic; the values recorded in the VCD file will use the std_logic enumeration characters (U, X, 0, 1, Z, W, L, H, and -). The VCD file generated will be a four-state file; because of this, the states are only represented by a 0, 1, X, or Z. For more information refer to [vcd add](#) in the *Questa ADMS Command Reference*.

```
vcd files dump.vcd -direction -nomap
```

The vcd add command is used to add the required items to a VCD file. The -file option specifies the name of the VCD file to which the information will be written. Only port changes will be recorded, which is specified by the -ports option.

```
vcd add -file dump.vcd -ports :count1:*
```

The following are standard Questa ADMS commands; they will view all of the windows, add all of the nets into the wave window, run the simulation, and then quit Questa ADMS.

```
view *
add wave *
run
quit -f
```

The following command will load the specified test_bench which instantiates the block count1 that is being re-simulated. The command has been run from the UNIX command line. The option -vcdread specifies the name of the file that will contain all of the stimulus information.

```
vasim test_bench -vcdread dump.vcd
```

As before, add all of the signals into the [Wave Window \(Solido Waveform Analyzer\)](#) and run the simulation. The same results are now visible in Solido Waveform Analyzer as were visible when the block count1 was simulated with the whole design.

Related Topics

[Extended VCD File Creation Example](#)
[Creating a Four-State VCD File](#)

15. C Code Encapsulation

You can encapsulate C code in Questa ADMS. You need to follow a procedure when importing C functions.

This chapter also describes the calling of C functions directly from VHDL-AMS by using the VHDL-AMS predefined mechanisms to import foreign code.

C Code Conventions.....	15-1
Use of C Functions in VHDL-AMS.....	15-2
C Code Encapsulation Procedure in Questa ADMS.....	15-3

C Code Conventions

C code must be de-correlated from the way that the solver computes and resolves the equations.

Because C code is called from VHDL-AMS, the C function has to have the same properties as a VHDL-AMS subprogram:

- No side effect, in other words, the behavior of a subprogram is the same for the same set of parameters.
- No assumption is made about the number of calls which will be made on these functions to compute one simulation point.
- The values of the parameters may be independent of those at a previous call.
- It is not possible to get VHDL-AMS complex objects (as quantities or signals for example, only their values are allowed).

Organization of the C Code

Due to the number of calls of the C function, it is important to free all the memory allocated within this function.

The use of static values to memorize points from another call is not possible. For example, the time variation of one parameter can not be computed inside a function. The time variation must be computed by Questa ADMS and set as a parameter of the C function.

The parameters may be of either scalar or composite type and the specification can be found in the section [Correspondence Between C Type and VHDL-AMS Type](#).

If the C function is encapsulated in a VHDL-AMS function, the C function can return either scalar or composite values, the type of the return value is specified in [Correspondence Between C Type and VHDL-AMS Type](#).

Related Topics

[Use of C Functions in VHDL-AMS](#)

[C Code Encapsulation Procedure in Questa ADMS](#)

[C Code Encapsulation](#)

[C Template and Reference Tables](#)

Use of C Functions in VHDL-AMS

All the C functions to be encapsulated are declared in VHDL-AMS as VHDL-AMS subprograms.

The name of the VHDL-AMS declared subprogram may be different from its C name. The parameters of the VHDL-AMS declared subprogram are given in the same order as they are declared in the C function.

The type of the VHDL-AMS return value, if it exists, is linked to the return type of the C function. See the specification in the section [Correspondence Between C Type and VHDL-AMS Type](#).

FOREIGN Attribute

The FOREIGN attribute indicates that a VHDL-AMS declared subprogram is a foreign C function, and enables you to specify how to find such a function.

The FOREIGN attribute is defined as a string and contains the following data:

- A label corresponding to its use: "ADMS" has been chosen
- The C name of the function.

Examples

```
ADMS: C_function
```

The C file must be compiled with the provided gcc compiler using the -fPIC (Position Independent Code) option. Using this option, the code may be directly linked with the simulator or loaded when required during the simulation.

```
gcc -c -g -fPIC C_function.c -o C_function.o
```

Tip

To return the path of the AMS provided gcc compiler, run the following command:

```
$MGC_AMS_HOME/com/ams_runtime_setup.ksh AMS_GCC_HOME
```

Library Path Environment Variable

For C Code encapsulation, the LD_LIBRARY_PATH variable must be correctly set. This is done using the following command:

```
LD_LIBRARY_PATH=`/bin/sh $MGC_AMS_HOME/com/ams_runtime_setup.ksh
LD_LIBRARY_PATH`

export LD_LIBRARY_PATH
```

Methodology

The declaration of encapsulated foreign C functions is only restricted to packages. The declaration of the C function and the Attribute Specification differs, in that only the C functions and no other declarations are put in the same declaration package header.

An identical package can contain a collection of different foreign C functions. A FOREIGN attribute has to be attached to each corresponding VHDL-AMS declaration.

Example

The package contains two subprograms: a function vhdL_function1, and a procedure vhdL_proc2.

```
#####
PACKAGE encapsulation IS
FUNCTION vhdL_function1(arg : Real) RETURN Real;
ATTRIBUTE foreign OF vhdL_function1 : FUNCTION IS "ADMS : c_function1";
-- c_function1 is the name of the corresponding C function

PROCEDURE vhdL_proc2(arg1 : IN Real; arg2 : OUT Real);
ATTRIBUTE foreign OF vhdL_proc2 : PROCEDURE IS "ADMS : c_function2"
-- c_function2 is the name of the corresponding C function
#####
```

Related Topics

[C Code Encapsulation Procedure in Questa ADMS](#)

[C Code Conventions](#)

[C Code Encapsulation](#)

[C Template and Reference Tables](#)

C Code Encapsulation Procedure in Questa ADMS

When the declaration of foreign C functions is located in a VHDL-AMS package header and has been compiled with Questa ADMS using vacom, it is necessary to fill the C template of the package. This establishes the link between the VHDL-AMS functions and their corresponding C functions.

C Template in Questa ADMS.....	15-4
Correspondence Between C Type and VHDL-AMS Type.....	15-7
Predefined Macros in Questa ADMS.....	15-7

C Template in Questa ADMS

The C template is the C image of the package that contains the declaration of the C functions. You must provide information about the C functions to be encapsulated and the corresponding VHDL-AMS functions.

Caution

It is important that the C type of the C image of the parameters of the VHDL-AMS subprograms fits the C type of the parameters of the C subprograms to be encapsulated.

This C template will be compiled and linked with the compiled end-user's code in order to build the *.so file corresponding to the package for the delivered platforms.

Caution

The new *.so file has to be placed in the `_OS/$AMS_VCO` directory of the current working library.

The new *.so file has the same name as the VHDL-AMS package name that contains the declarations of the encapsulated foreign C functions.

Example

If the name of the package is defined as follows:

```
PACKAGE encapsulation IS...
```

The examples described in the [General C Template in Questa ADMS](#) and [C Template and Reference Tables](#) give more information about the compilation commands.

The C template has the following format:

```
#####
#include "macro.h"

/*****
/*      MANUAL MODIFICATIONS      */
/*****
/* the file containing the prototype of the C encapsulated */
/* functions has to be included */

#include <C_function_prototype_file>
/*****
```

```

/* END OF MANUAL MODIFICATIONS */
/*****

extern PROC_CAST current_active_process;
RG_DECL;
VPTR_DECL;
LOOP_I_DECL;
LOOP_DRIVER_DECL;
static void elab_desent();
static int dblines[0]={};

/*****
/*      MANUAL MODIFICATIONS      */
/*****

/* Each VHDL-AMS subprogram has a corresponding C */
/* predeclaration <TYPE_FUNC_Image> has to be replaced by the */
/* C type corresponding to the type of VHDL-AMS subprogram */
/* "name_of_VHDL_subprogram__i": */
/* the correspondence is defined in the */
/* General C Template in
   Questa ADMS section in the appendix */
/* there are as many predeclarations as there are VHDL-AMS */
/* subprograms */

static <TYPE_FUNC_Image> <name_of_VHDL_subprogram__1> ();
static <TYPE_FUNC_Image> <name_of_VHDL_subprogram__2> ();
.
.
/*****/
/* END OF MANUAL MODIFICATIONS */
/*****/

/*****/
/*      MANUAL MODIFICATIONS      */
/*****/
/* Table of the names of C image of the VHDL-AMS subprograms */
/* It is mandatory that the order of these subprograms is the */
/* same as the order of the declarations of the VHDL-AMS */
/* subprograms in the package */

CODE_SUPRGS_DECL=
{(CODE_CAST) <name_of_VHDL_subprogram__1>, (CODE_CAST)
<name_of_VHDL_subprogram__2>, ..};

/*****/
/* END OF MANUAL MODIFICATIONS */
/*****/

PKGI_DECL(0);
ELB_CODE_CAST elab_funcs=
{elab_desent,0,(void*)code_suprgs,NULL};
static void elab_desent(int* ENVIR_CAST,int selector){
switch(selector){
case -1: /* Analog Code and Subprogram Definition */
break;

```

```

case 0: /* block: design entity */
ALLOC_RG;
break;
default:
/* Error Management */
break;
}
}

/*****
/*      MANUAL MODIFICATIONS      */
*****/
/* body of the C image of the VHDL-AMS subprogram*/
/* the following body has to be defined for each VHDL-AMS */
/* subprogram. <TYPE_FUNC_Image> has to be replaced by the */
/* C type corresponding to the type of the VHDL-AMS subprogram*/
/* <Param_MACRO>: the name of the macros corresponding to the */
/* parameters of the C Image of the VHDL-AMS subprogram, */
/* defined in the tables in the section on */
/* Predefined Macros in the appendix */
/* <declarative_MACROi> (varm, i) : a declarative macro which */
/* does the correspondence between <Param_MACRO> type and the */
/* type of the parameters of the C encapsulated functions, */
/* defined in the tables in the section on */
/* Predefined Macros in the appendix */
/* Funci_C : call of the C encapsulated function */
/* <assignment_MACROi>(varj, i)> and <alias_MACROi>(var, i)>:*/
/* correspondence between the parameter of the C function and */
/* the VARIABLE parameter of the VHDL-AMS subprogram */

static <TYPE_FUNC_Image> <name_of_VHDL_subprogram__i> /(<Param_MACRO>
(0),...,<Param_MACRO>(i), PKG_DECL){
    declarative_MACRO01(var2, i);
    .
    .
    <declarative_MACROi>(vark, i)>;
    .
    .
    <alias_MACROi>(var, i)>
    .
    .
    TYPE_var1 var;
    .
    .
    <TYPE_var> varl;
    .
    .
    Funci_C (varj,...,vark);

    <assignment_MACROi>(varj, i)>;
} /* end <name_of_VHDL_subprogram__i> */
/*****
/*      END OF MANUAL MODIFICATIONS      */
*****/

```

```
/* end of C template */g
#####
```

Related Topics

[Correspondence Between C Type and VHDL-AMS Type](#)
[Predefined Macros in Questa ADMS](#)
[C Code Encapsulation](#)

Correspondence Between C Type and VHDL-AMS Type

There is a correspondence between VHDL-AMS objects and C types.

Two cases are distinguished between:

- Scalar objects

The correspondence between VHDL-AMS scalar objects and C types is described in [Table D-1](#) of the [C Template and Reference Tables](#) appendix.

- 1-dimensional array objects

The correspondence between VHDL-AMS 1-dimensional array objects and C types is described in [Table D-2](#) of the [C Template and Reference Tables](#) appendix.

Note

The C type corresponding to the INT VHDL-AMS is a LONG LONG.

The tables in the [Correspondence Between C Type and VHDL-AMS Type](#) section of the appendices are used to specify the C image type of the VHDL-AMS subprograms associated with the C subprograms to be encapsulated.

Related Topics

[Predefined Macros in Questa ADMS](#)
[C Code Encapsulation Procedure in Questa ADMS](#)

Predefined Macros in Questa ADMS

Macros have been predefined to help you fill the C template.

All the predefined macros are defined in the *macro.h* file. This file is included in the C template. There are four types of macros:

- <param_MACROi>

Defines the C image type of the parameter's VHDL-AMS subprograms. This macro contains one parameter which corresponds to the position of the VHDL-AMS parameter, and takes the value 0...n-1 (n is the total number of parameters).

- **<declarative_MACROi>**

Establishes the correspondence between the C function parameter type and the mode IN parameter type of the VHDL-AMS subprogram C image. This macro has two parameters: the first is the C function parameter name, and the second is the position of the corresponding VHDL-AMS parameter.

- **<alias_MACROi>**

Gives the correspondence between the C function parameter type and the 1-dimensional array VARIABLE parameter type of the VHDL-AMS subprogram C image. This macro has two parameters: the first is the name of the C function parameter, and the second is the position of the corresponding VHDL-AMS parameter.

- **<assignment_MACROi>**

Establishes the correspondence between the C function parameter type and the SCALAR VARIABLE parameter type of the VHDL-AMS subprogram C image. This macro has two parameters: the first is the name of the C function parameter, and the second is the position of the corresponding VHDL-AMS parameter.

Depending on the dimension and the mode of the VHDL-AMS parameters, four cases are distinguished:

- **Scalar Parameters of Mode IN**
- **Scalar Parameters of Mode OUT/INOUT**
- **1-Dimensional Array Parameters of Mode IN**
- **1-Dimensional Array Parameters of Mode OUT/INOUT**

Three object types: CONSTANT, VARIABLE and SIGNAL also have to be distinguished.

Caution

All the predefined macros have, amongst their parameters, an index which corresponds to the VHDL-AMS subprogram parameter position. This index takes the value 0 to n-1 for each type of object, (n is the total number of CONSTANT, VARIABLE, and SIGNAL parameter types).

Scalar Parameters of Mode IN

CONSTANT Parameters

By default, the parameters of a VHDL-AMS function are of mode IN.

You must define:

- A <param_MACROi> macro: this parameter macro contains one parameter which corresponds to the position of the VHDL-AMS parameter, and takes the value 0...n-1 (n is the total number of parameters).
- A <declarative_MACROi> macro: this declarative macro has two parameters. The first is the C function parameter name and the second is the position of the corresponding VHDL-AMS parameter.

The two macros are defined in [Table D-3](#) of the [C Template and Reference Tables](#) appendix.

Note

The VHDL-AMS type INTEGER occasionally needs to be casted in advance to enable its usage. This is the reason for declarative macros using cast on type being defined as FRGN_DECL_CINT_CON_INT.

Example VHDL-AMS function:

```
VHDL_Compute(v_in : Real, v_out : Real) RETURN Real;
```

Corresponding C encapsulated function:

```
double C_Compute(double vin);
```

C template:

```
Static double VHDL_Compute (FRGN_PARAM_CON_REAL(0), PKG_DECL){
    FRGN_DECL_CON_REAL(vin,0);
    return C_Compute(vin);
}
```

FRGN_PARAM_CON_REAL(0) is the parameter macro.

FRGN_DECL_CON_REAL(vin,0) is the declarative macro.

Example VHDL-AMS function (macro using cast on type):

```
VHDL_Compute(n : INTEGER; rp1 : REAL)
```

Corresponding C encapsulated function:

```
double C_Compute(int *n, double rp1)
```

Note

The VHDL-AMS type INTEGER needs to be casted in advance in order for it to be used by the C function. This is the reason for the declarative macro FRGN_DECL_CINT_CON_INT being used and not FRGN_DECL_CON_INT.

VARIABLE Parameters

You must define:

- A <param_MACROi> macro: this parameter macro contains one parameter which corresponds to the position of the VHDL-AMS parameter, and takes the value 0...n-1 (n is the total number of parameters).
- A <declarative_MACROi> macro: this declarative macro has two parameters. The first is the name of the C function, and the second is the position of the corresponding VHDL-AMS parameter.

The two macros are defined in [Table D-4](#) of the [C Template and Reference Tables](#) appendix.

Note

The VHDL-AMS type INTEGER sometimes needs to be casted in advance to enable it to be used. This is the reason for the declarative macros using cast on type being defined as FRGN_DECL_CINT_CON_INT.

Example VHDL-AMS procedure:

```
VHDL_Compute(VARIABLE v_in :IN Real, v_out : OUT Real);
```

Corresponding C encapsulated function:

```
void C_Compute(double v_in, double *vout);
```

C template:

```
Static double VHDL_Compute (FRGN_PARAM_VAR_REAL(0),
FRGN_PARAM_VAR_OUT_REAL(1), PKG_DECL) {
    FRGN_DECL_VAR_REAL(vin,0);
    double v_out;
    C_Compute(v_in, &v_out);
    ASSIGN_VAR_OUT(1, v_out);
}
```

FRGN_PARAM_VAR_REAL(0) is the parameter macro.

FRGN_DECL_VAR_REAL(vin,0) is the declarative macro.

In this example, the parameter `v_out` of the VHDL-AMS procedure is a SCALAR VARIABLE parameter of mode OUT. This case is handled in the “VARIABLE Parameters” section below.

SIGNAL Parameters

This functionality is not implemented in Questa ADMS 11.1.

Scalar Parameters of Mode OUT/INOUT

VARIABLE Parameters

The two modes OUT and INOUT are handled in the same way.

You must define:

- A `<param_MACROi>` macro: this parameter macro contains one parameter which corresponds to the position of the VHDL-AMS parameter, and takes the value 0...n-1 (n is the total number of parameters).
- A `<assignment_MACROi>` macro: this assignment macro has two parameters. The first is the name of the C function parameter, and the second is the position of the corresponding VHDL-AMS parameter.

This macro has to be placed after the C function call.

The two macros are defined in [Table D-6](#) of the [C Template and Reference Tables](#) appendix.

The cast on type is not allowed for the modes OUT or INOUT.

Example VHDL-AMS procedure:

```
VHDL_Compute(v_in : IN Real, v_out : OUT Real)
```

Corresponding C encapsulated function:

```
void C_Compute(double vin, double *vout)
```

C template:

```
Static void VHDL_Compute(FRGN_PARAM_CON_REAL(0),
FRGN_PARAM_VAR_OUT_REAL(0), PKG_DECL) {
    FRGN_DECL_CON_REAL(vin, 0);
    double vout;
    C_Compute(vin, &vout);
    FRGN_ASSIGN_VAR_OUT(vout, 0);
}
```

`FRGN_PARAM_VAR_OUT_REAL(0)` is the declarative macro.

FRGN_ASSIGN_VAR_OUT(v_out,0) is the assignment macro.

Note

You must declare the parameters in the C template which are modified by the C function.

SIGNAL parameters

This functionality is not implemented in Questa ADMS.

1-Dimensional Array Parameters of Mode IN

CONSTANT Parameters

You must define:

- A <param_MACROi> macro: this parameter macro contains one parameter which corresponds to the position of the VHDL-AMS parameter, and takes the value 0...n-1 (n is the total number of parameters).
- A <declarative_MACROi> macro: this declarative macro has two parameters. The first is the name of the C function parameter and the second is the position of the corresponding VHDL-AMS parameter.

The two macros are defined in [Table D-8](#) of the [C Template and Reference Tables](#) appendix. The distinction is made between constrained and unconstrained vectors.

You can also have access to the size, first index, last index and the direction of the vector by using the following macros:

- GET_CON_LENGTH(i): indicates the size of the vector
- GET_CON_FIRST(i): gives the first index of the vector
- GET_CON_LAST (i): provides the last index of the vector
- GET_CON_DIR(i): gives the direction of the vector. The return value of 0 gives the "downto" direction and the return value of 1 gives the "to" direction.

Note

All these macros return an integer.

Example VHDL-AMS function:

```
VHDL_Compute(n : INTEGER; rp1 : Real_vector)
```

Corresponding C encapsulated function:

```
double C_Compute(int n, char param1[10])
```

C template:

```
Static double VHDL_Compute (FRGN_PARAM_CON_INT(0),
FRGN_PARAM_CON_ARR(1), PKG_DECL) {
    FRGN_DECL_CINT_CON_INT(n,0)
    FRGN_DECL_CCHAR_CON_ARR_ENUM(param1, 1);
    int first = GET_CON_FIRST(1);
    return C_Compute(n, &param1);
}
```

FRGN_PARAM_CON_ARR(1) is the parameter macro.

FRGN_DECL_CCHAR_CON_ARR_ENUM(param1, 1) is the declarative macro.

GET_CON_FIRST(1) gives the first index of the vector rp1.

VARIABLE Parameters

You must define:

- A <param_MACROi> macro: this parameter macro contains one parameter which corresponds to the position of the VHDL-AMS parameter, and takes the value 0...n-1 (n is the total number of parameters).
- A <alias_MACROi> macro: this alias macro has two parameters. The first is the name of the C function parameter, and the second is the position of the corresponding VHDL-AMS parameter.

This alias macro has to put before the call of the C function.

The two macros are defined in [Table D-9](#) of the [C Template and Reference Tables](#) appendix. The distinction is made between constrained and unconstrained vectors.

You can also have access to the size, first index, last index and direction of the vector by using the following macros:

- GET_VAR_LENGTH(i): indicates the size of the vector
- GET_VAR_FIRST(i): gives the first index of the vector
- GET_VAR_LAST (i): provides the last index of the vector
- GET_VAR_DIR(i): gives the direction of the vector. The return value of 0 gives the "downto" direction and the return value of 1 gives the "to" direction.

Note

All these macros return an integer.

Example VHDL-AMS procedure:

```
VHDL_Compute(VARIABLE v_in : IN Real_vector, v_out : OUT Real_vector)
```

Corresponding C encapsulated function:

```
void C_Compute(double *v_in, double *vout)
```

C template:

```
Static void VHDL_Compute (FRGN_PARAM_VAR_UARR(0),
FRGN_PARAM_VAR_OUT_UARR(1), PKG_DECL) {
    FRGN_ALIAS_VAR_ARR_REAL(vin,0);
    FRGN_ALIAS_VAR_OUT_ARR_REAL(vout, 1);
    int size = GET_VAR_LENGTH(0);
    C_Compute(vin, vout);
}
```

FRGN_PARAM_VAR_UARR(0): parameter macro.

FRGN_ALIAS_VAR_ARR_REAL(vin, 0): declarative macro.

GET_VAR_LENGTH(0): gives the size of the vector v_in.

In this example, the parameter v_out of the VHDL-AMS procedure is a 1-DIMENSIONAL ARRAY VARIABLE parameter of mode OUT. This case is handled in the “VARIABLE Parameters” section below.

SIGNAL Parameters

This functionality is not implemented in Questa ADMS.

1-Dimensional Array Parameters of Mode OUT/INOUT

VARIABLE Parameters

The two modes, OUT and INOUT, are handled in the same way.

You must define:

- A <param_MACROi> macro: this parameter macro contains one parameter which corresponds to the position of the VHDL-AMS parameter, and takes the value 0...n-1 (n is the total number of parameters).
- A <alias_MACROi> macro: this alias macro has two parameters. The first parameter is the name of the parameter of the C function and the second parameter is the position of the corresponding VHDL-AMS parameter.

The alias macro has to be placed in front of the call for the C function.

The two macros are defined in [Table D-11](#) of the [C Template and Reference Tables](#) appendix. The distinction is made between constrained and unconstrained vectors.

You can also have access to the size, first index, last index and direction of the vector by using the following macros:

- GET_VAR_LENGTH(i): indicates the size of the vector
- GET_VAR_FIRST(i): gives the first index of the vector
- GET_VAR_LAST (i): provides the last index of the vector
- GET_VAR_DIR(i): gives the direction of the vector. The return value of 0 gives the "downto" direction and the return value of 1 gives the "to" direction.

Note

All these macros return an integer.

Example VHDL-AMS procedure:

```
VHDL_Compute(v_in : IN Real_vector, v_out : OUT Real_vector)
```

Corresponding C encapsulated function:

```
void C_Compute(double vin[2], double *vout)
```

C template:

```
Static void VHDL_Compute (FRGN_PARAM_CON_ARR(0),
FRGN_PARAM_VAR_OUT_UARR(0), PKG_DECL) {
    FRGN_DECL_CON_REAL(vin, 0);
    FRGN_ALIAS_VAR_OUT_ARR_REAL(vout, 0);
    int size = GET_VAR_LENGTH(0);
    C_Compute(vin, vout);
}
```

FRGN_PARAM_VAR_OUT_UARR(0): parameter macro.

FRGN_ALIAS_VAR_OUT_ARR_REAL(vout, 0): declarative macro.

GET_VAR_LENGTH(0): gives the size of the vector v_out.

SIGNAL Parameters

This functionality is not implemented in Questa ADMS.

Note

If the function returns a vector object, it is necessary to use an additional declarative macro to allocate memory. You must provide the number of elements of the vector object:

```
SET_RANGE_WITH_LENGTH(ret, nb_elements)
```

Example

- This is the case of a return vector that contains 10 elements:

```
SET_RANGE_WITH_LENGTH(ret, 10)
```

Related Topics

[C Code Encapsulation](#)

[C Template and Reference Tables](#)

16. MathWorks Integration

This chapter describes how MathWorks integrates with Questa ADMS.

Mathworks Integration Overview.....	16-1
MATLAB Testbench Configuration.....	16-3
MATLAB Component Configuration.....	16-5
Simulink Cosimulation Configuration.....	16-7

Mathworks Integration Overview

Integration between Questa ADMS/Questa SIM and MATLAB®/Simulink® is possible using HDL Verifier™.

The integration provides a fast bidirectional link between Questa ADMS and the MathWorks MATLAB¹ and Simulink¹ products for direct hardware design verification and cosimulation. The integration of these tools enables you to apply each product to the tasks it does best:

- Questa ADMS — Analog and mixed-signal modeling in HDL and SPICE
- MATLAB — Numerical computing, algorithm development, and visualization
- Simulink — Simulation of system-level designs and complex models.

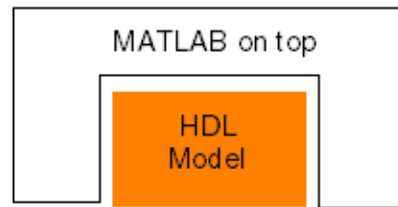
HDL Verifier provides MATLAB and Simulink on-top configurations, and aVHDL/Verilog on-top configuration, with a MATLAB M-file as a leaf cell.

HDL Verifier v3.1 is supported on Linux (32-bit) or Linux (64-bit). Complete documentation on HDL Verifier is available at <http://www.mathworks.co.uk/>.

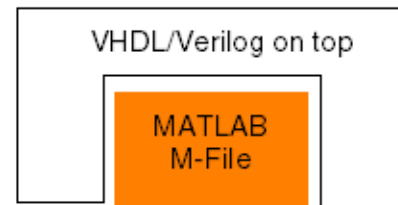
¹ MATLAB, Simulink, and Real-Time Workshop are registered trademarks of The MathWorks, Inc.

Figure 16-1: Possible EDA Simulator Link Simulation Configurations

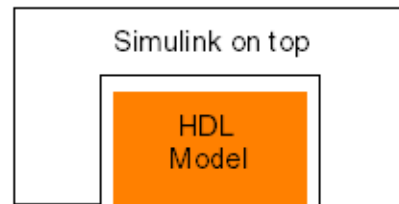
MATLAB Testbench Configuration:



MATLAB Component Configuration:



Simulink Cosimulation Configuration:



Environment Variables

Ensure that the following environment variables are set correctly:

- \$MGC_AMS_HOME

Specifies the installation path of AMS products. See [Setting Up the User Environment](#) in the *Solido Simulation Suite Installation Guide*.

- \$MATLAB

Specifies the installation path of MATLAB.

- \$LIB_MHDLS

Specifies the Simulink server used. Set this variable if you want to force a specific version. For example, \$LIB_MHDLS could contain:

```
'/ams/tools/mathworks/v2013b/x86_64/toolbox/edalink/extensions/modelsim/linux64/
liblfmhdls_gcc412.so')
```

- \$LIB_MHDLCL

Specifies the MATLAB Client used. Set this variable if you want to force a specific version. For example, \$LIB_MHDL_C could contain:

```
'/ams/tools/mathworks/v2013b/x86_64/toolbox/edalink/extensions/modelsim/linux64/  
liblfmhdlc_gcc412.so')
```

Notice that the letter c is used instead of s, in *liblfmhdlc_gcc412.so*'

Related Topics

[MATLAB Testbench Configuration](#)

[MATLAB Component Configuration](#)

[Simulink Cosimulation Configuration](#)

MATLAB Testbench Configuration

MATLAB provides stimulus to an HDL model and processes the simulation results.

Figure 16-2: MATLAB Testbench Configuration

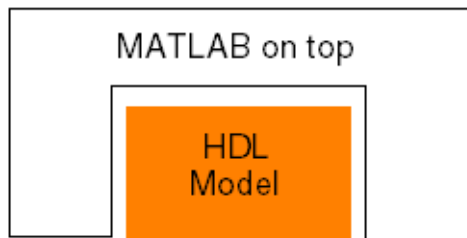
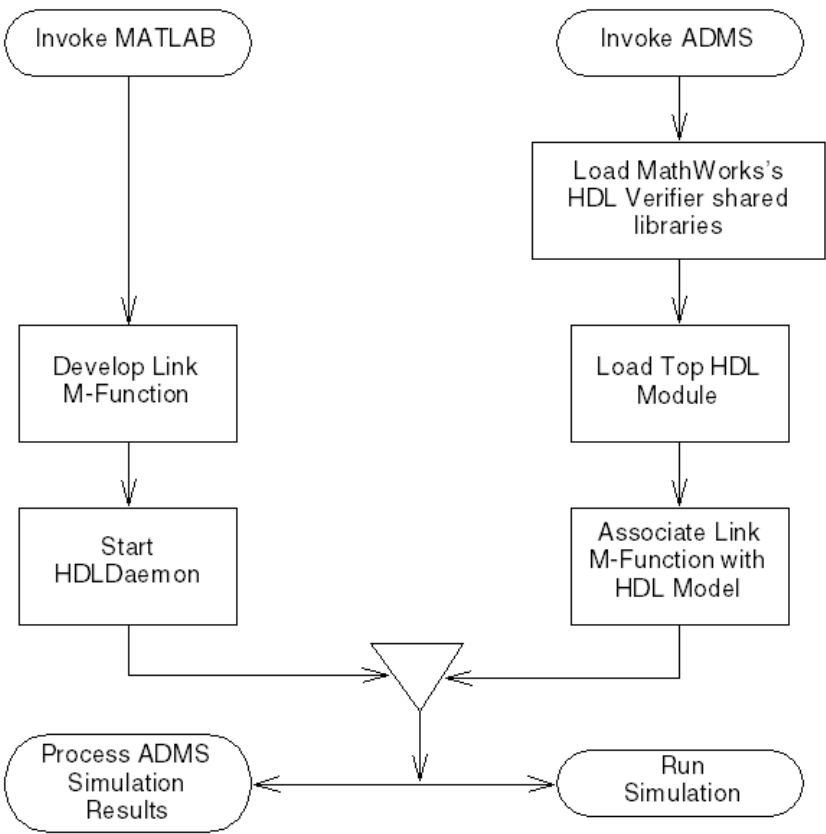


Figure 16-3: MATLAB Testbench Configuration Setup Flow



Procedure

- 1. Start MATLAB from a shell; add path to m files.
- 2. For Questa SIM, follow the steps in [Table 16-1](#).

Table 16-1: MATLAB Testbench Configuration with Questa SIM

Step	MATLAB	Questa SIM
1	hdldaemon	
2	vsim	
3		vsimmatlab work.sinesrcr
4		matlabtb sinesrcr
5		run 2us
6	hdldaemon kill	

For Questa ADMS, start Questa ADMS from a shell using the command:

```
vasim work.sinesrcr -foreign
      "matlabclient $MATLAB/toolbox/modelsim/$PLTFORM/matlablink.so"
```

then follow the steps in [Table 16-2](#).

Table 16-2: MATLAB Testbench Configuration with Questa ADMS

Step	MATLAB	Questa ADMS
1	hdldaemon	
2		ms matlabtb sinesrcr
3		run 2us
4	hdldaemon kill	

Related Topics

- [MATLAB Component Configuration](#)
- [Simulink Cosimulation Configuration](#)

MATLAB Component Configuration

MATLAB M-code describes the behavior of a component within an HDL hierarchy.

Figure 16-4: MATLAB Component Configuration

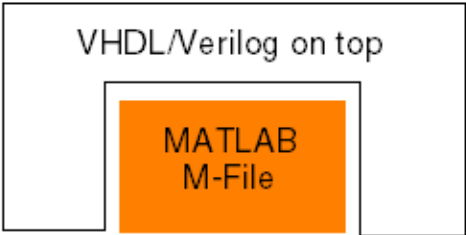
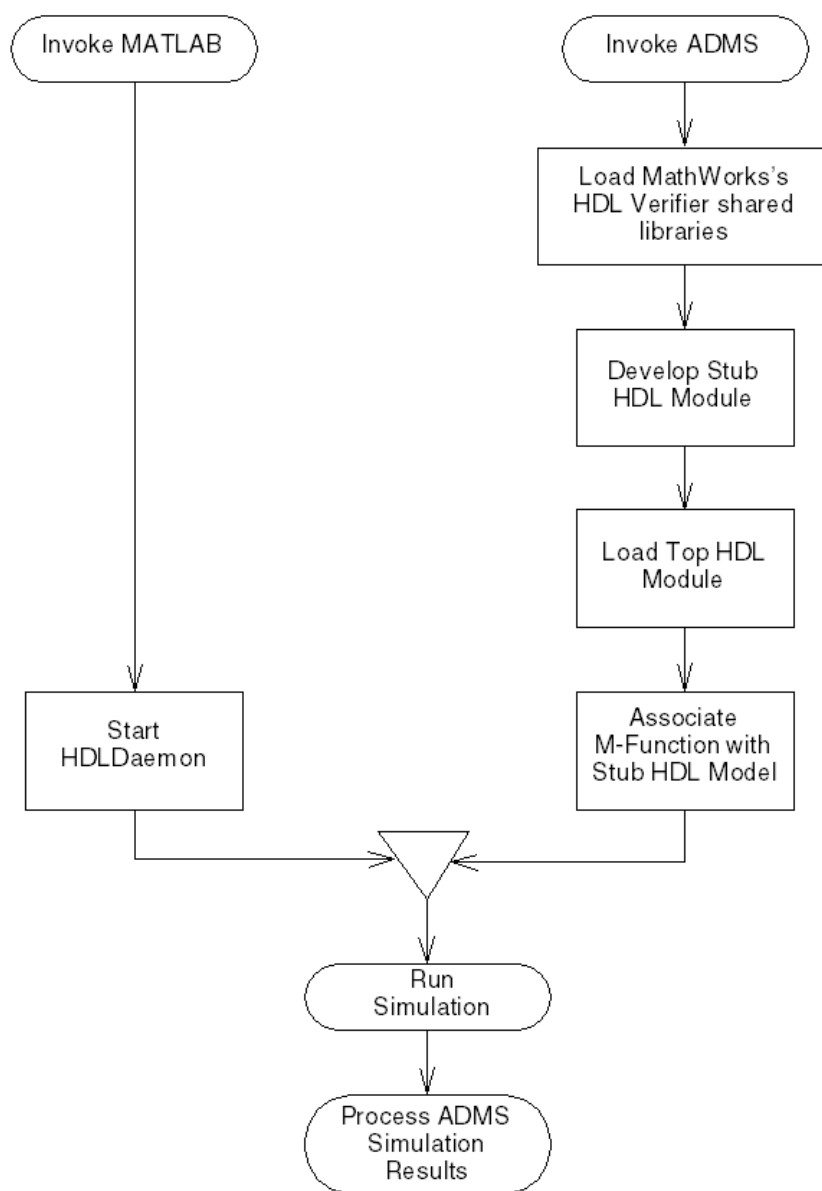


Figure 16-5: MATLAB Component Configuration Setup Flow

Procedure

1. Start MATLAB from a shell; add path to m files.
2. For Questa SIM, follow the steps in [Table 16-3](#).

Table 16-3: MATLAB Component Configuration with Questa SIM

Step	MATLAB	Questa SIM
1	hdldaemon	
2	vsim	

Table 16-3: MATLAB Component Configuration with Questa SIM (continued)

Step	MATLAB	Questa SIM
3		vsimmatlab work.top
4		matlabcp top.i2 -mfunc square
5		run 2us
6	hdldaemon kill	

For Questa ADMS, start Questa ADMS from a shell using the command:

```
vasim work.top -foreign
"matlabclient $MATLAB/toolbox/modelsim/$PLATFORM/matlablink.so"
```

then follow the steps in [Table 16-4](#).

Table 16-4: MATLAB Component Configuration with Questa ADMS

Step	MATLAB	Questa ADMS
1	hdldaemon	
2		ms matlabcp top.i2 -mfunc square
3		run 2us
4	hdldaemon kill	

Related Topics

[MATLAB Testbench Configuration](#)
[Simulink Cosimulation Configuration](#)

Simulink Cosimulation Configuration

The Simulink configuration uses a HDL cosimulation block from the Simulink blockset.

Figure 16-6: Simulink Cosimulation Configuration

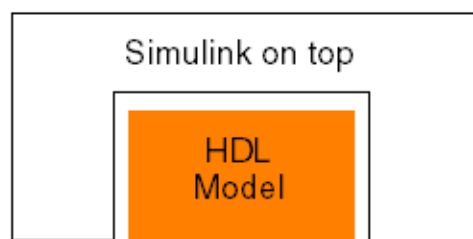
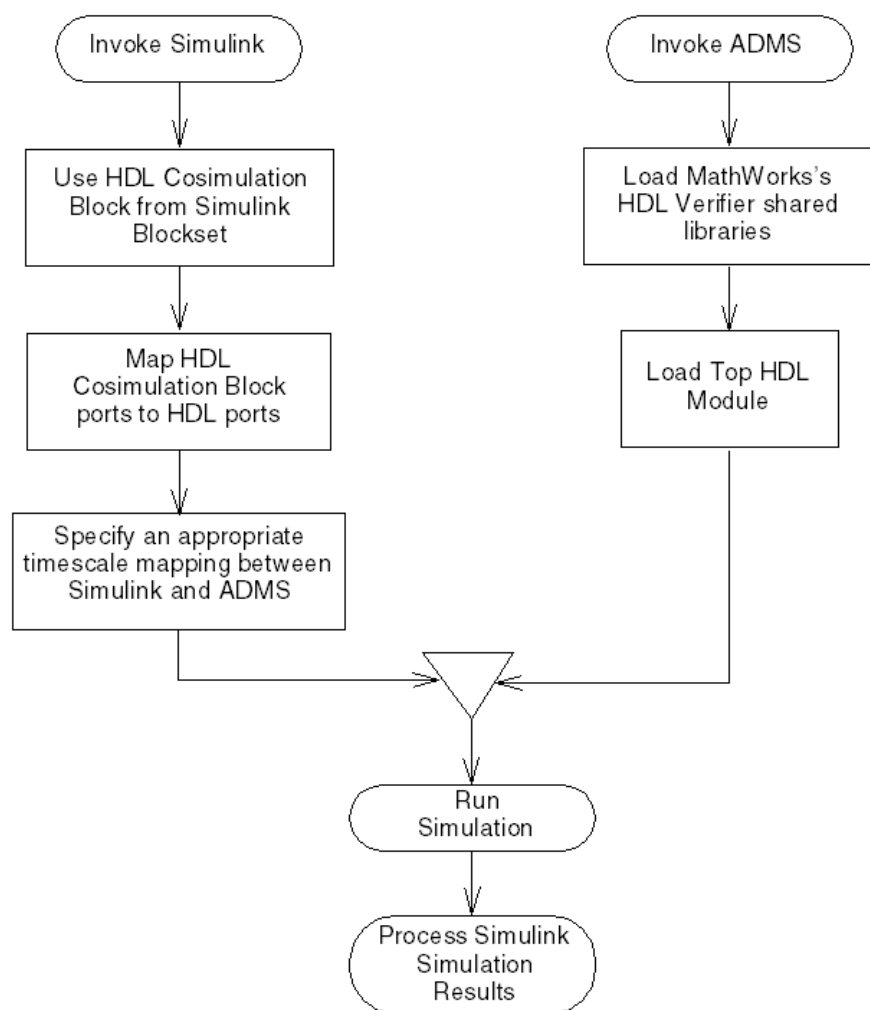


Figure 16-7: Simulink Cosimulation Configuration Setup Flow



Procedure

1. Use the Simulink GUI to instantiate a specific block for cosimulation with Questa SIM/Questa ADMS. Choose **Simulink Library > > Link for ModelSim > HDL Cosimulation**.
2. For Questa SIM, follow the steps in [Table 16-5](#).

Table 16-5: Simulink Cosimulation Configuration with Questa SIM

Step	Simulink	Questa SIM
1	Map HDL Cosimulation Block Ports to HDL Model Ports	
2	Configure HDL Cosimulation Block Timescale Mapping	
3	vsim	
4		vsimsimulink work.sinesrcr
5	Run Simulink Simulation Session	

For Questa ADMS, follow the steps in [Table 16-6](#).

Table 16-6: Simulink Cosimulation Configuration with Questa ADMS

Step	Simulink	Questa ADMS
1	(Use Simulink GUI to) Map HDL Cosimulation Block Ports to HDL Model Ports	
2	(Use Simulink GUI to) Configure HDL Cosimulation Block Timescale Mapping	
3		vasim work.sinesrcr -foreign "simlinkserver \$MATLAB/ toolbox/modelsim/\$PLTFORM/ simulinklink.so"
4	Run Simulink Simulation Session	

Related Topics

[MATLAB Testbench Configuration](#)

[MATLAB Component Configuration](#)

17. Dialog and Field Reference

This chapter describes the windows and dialog boxes in the Questa ADMS graphical user interface.

For a more general overview information on using and navigating the GUI, see [Questa ADMS GUI Overview](#).

Assertions Window.....	17-2
Contributor Window.....	17-7
CurrentFlow Window.....	17-9
Design Options Dialog Box.....	17-12
Eldo Commands Dialog Box.....	17-13
File Breakpoint Dialog Box.....	17-18
Force Selected Signal Dialog Box.....	17-20
Interface Matcher.....	17-23
Library Window.....	17-28
Locals Window.....	17-30
Modify Breakpoints Dialog Box.....	17-32
Modulated Steady State Dialog Box.....	17-34
Objects Window.....	17-36
Preferences Dialog Box.....	17-39
Processes Window.....	17-42
Questa ADMS > Questa Import Library Dialog Box.....	17-44
Questa > Questa ADMS Import Library Dialog Box.....	17-47
Restart Dialog Box.....	17-49
Reuse Previous Configuration Dialog Box.....	17-50
Rundata Window.....	17-51
Runtime Options Dialog Box.....	17-52
Source Window.....	17-55
Start Simulation Dialog Box.....	17-58
Stat Analog Kernel Window.....	17-68
Stat Mixed-Signal Activity Window.....	17-70
Stat Summary Window.....	17-72
Structure Window.....	17-74
Transcript Window.....	17-77
Wave Window (Solido Waveform Analyzer).....	17-80
Toolbars.....	17-81

Assertions Window

To access do one of the following:

- Choose **View > Coverage > Assertions**.
- Use the following command:

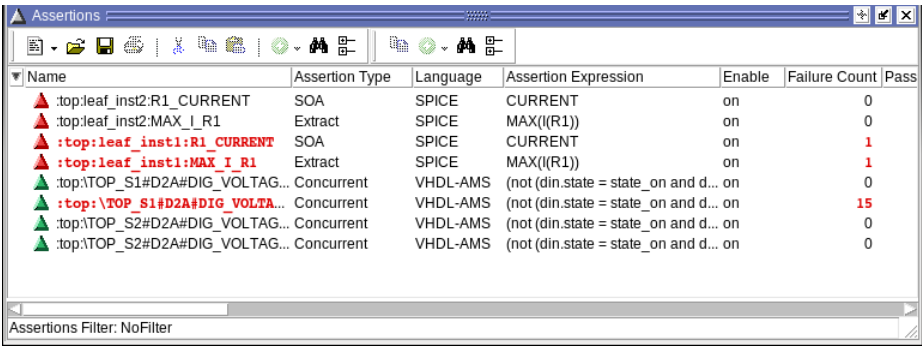
```
view assertions
```

Lists all embedded and external assertions that were successfully compiled and simulated during the current session. This dialog box is updated whenever the simulator pauses, either at a break or at the end of simulation.

Note

When assertions fire with failure messages, the Assertions window displays the name and failure count in red, both during simulation and in post-simulation mode.

Figure 17-1: Assertions Window



Characteristics

Table 17-1: Assertions Window Contents

Column Title	Description
Name	Identifies the assert directive you specified in the assertion code.
Assertion Type	Indicates the assertion type: Immediate or Concurrent. For SPICE language: SOA, EXTRACT, or MEAS.
Language	Identifies the HDL used to write the assertion.
Enable	Identifies whether failure checking is active for the assertion.

Table 17-1: Assertions Window Contents (continued)

Column Title	Description
Failure Count	Total number of times the assertion has failed in the current simulation.
Pass Count	The total number of times the assertions has passed in the current simulation. Only enabled if the simulator is invoked with either the -assertcover or the -assertdebug option for vasim, or if either the AssertionCover or AssertionDebug environment variable is set to 1.
Active Count	The number of active assertion attempts at the current time. Only enabled if the simulator is invoked with the -assertdebug option or the environment variable AutoExclusionsDisable is set to 1. Empty (-) for AMS assertions.
Memory	Tracks the current memory used by the assertion. Empty (-) for AMS assertions.
Peak Memory	The peak memory used by the assertion. Empty (-) for AMS assertions.
Peak Memory Time	Indicates the simulation run time at which the peak memory usage occurred. Empty (-) for AMS assertions.
Cumulative Threads	The cumulative thread count for the assertion. Empty for AMS assertions.
ATV	Indicates the status of assertion thread viewing: on or off. Only enabled if the simulator is invoked with the -assertdebug option or the environment variable AssertionDebug is set to 1. Disabled for AMS assertions.
Assertion Expression	Displays the actual assertion expression. See Usage Notes below.
Included	Indicates whether the Assertion is included in (check mark) or excluded from (X mark) aggregate statistics and reports.
Attempt Count	The number of times the assertion has been attempted. This is the number of assertion clocks for clocked assertions, or the number of passes and fails for unclocked assertions. Only enabled if the simulator is invoked with either the -assertcover or the -assertdebug option for vasim, or if either the AssertionCover or AssertionDebug environment variable is set to 1. Empty (-) for AMS assertions.
Design Unit	Identifies the design unit to which the assertion is bound.
Design Unit Type	Identifies the HDL type of the design unit.

Table 17-1: Assertions Window Contents (continued)

Column Title	Description
Disable Count	The number of assertion attempts that have been disabled either because an abort expression (PSL) or a disable if expression (SVA) became true. Only enabled if the simulator is invoked with either the -assertcover or the -assertdebug option for vsim, or if either the AssertionCover or AssertionDebug environment variable is set to 1. Empty (-) for AMS assertions.
EOS Note	Identifies when assertion directives are active at the end of simulation (EOS): on or off. Refer to the assertion active command. If the assert directive is strong, the EOS Note column will report both the "active at end of simulation" note along with a strong error message. Will be "off" for AMS assertions.
Failure Limit	The number of times the simulator will respond to a failure event on an assertion. Will be "unlimited" for AMS assertions.
Failure Log	Indicates whether the logging of failure messages to the transcript is enabled or disabled.
Formal Status	Indicates formal analysis has been performed. Empty for AMS assertions.
Formal Radius	Indicates that the property has been verified to a depth of x number of cycles in the formal analysis. Shown as positive integer. Data appears in this column only during post-process analysis (Coverage View mode). Empty (0) for AMS assertions.
FPSA Actions	Displays a matrix of information relating the current action for each possible state: Failure, Pass, Start, and Antecedent. The field will always show four letters that indicate the current action: • C - Continue • B - Break • E - Exit • S - Subroutine Call The order of the letters relates to the state: F - Failure, P - Pass, S - Start, A - Antecedent. For example, if you see CCSB, you can derive the following: Failure state = Continue, Pass state = Continue, Start state = Subroutine Call, Antecedent state = Break.
Pass Log	Indicates whether the logging of pass messages to the transcript is enabled or disabled. Only enabled if the simulator

Table 17-1: Assertions Window Contents (continued)

Column Title	Description
	is invoked with the -assertdebug option or the environment variable AssertionDebug is set to 1. Empty (-) for AMS assertions.
Peak Active Count	The peak simultaneously active thread count that has occurred up to the current time. Peak Active Count will also be shown in reports created with the vcover report command. Only enabled if the simulator is invoked with the -assertdebug option or the environment variable AssertionDebug is set to 1. Empty (-) for AMS assertions.
Vacuous Count	The number of assertion attempts that have succeeded vacuously, that is, if the left hand side of an implication is false on a clock edge. Only enabled if the simulator is invoked with either the -assertcover or the -assertdebug option for vasim, or if either the AssertionCover or AssertionDebug environment variable is set to 1. Empty (-) for AMS assertions.

Usage Notes

SPICE assertions will only be listed in this window after a run command.

Some columns are not visible by default, to enable them, right-click on a column heading and choose Change Column Visibility.

Assertion Expression —

For VHDL-AMS, the assertion expression will be:

```
assert(<normalized expression>)
```

The normalized expression is a simplified version of the expression found in the source code:

- Comments are removed.
- Multiple lines are concatenated.
- Strings are in lowercase.
- The length is limited to 1000 characters, any part of the string beyond this limit is replaced by "...".

For SPICE .EXTRACT commands, the assertion expression displayed is the assertion being tested. For example, MAX(V(A)) is displayed for the following SPICE command:

```
.extract tran label=max_va max(v(a)) ubound = 3.0
```

For SPICE SOA and .MEAS commands, the assertion expression displayed will be the label (when present in the SPICE command) or the same expression as that given in the .chi file when there is no label.

Names of signals used inside a VHDL-AMS assertion are not visible in the Assertion window. In comparison, for a pure digital VHDL assertion, all signals used within the assertion are visible in the Assertion window. However, plotting a VHDL-AMS assertion displays all the signals used to express the assertion (as elements of the assertion in the Wave window). In addition, the assertion expression appears in the Assertion Expression field of the Assertion window, and the source code of the assertion can be displayed by double-clicking on the assertion.

Related Topics

[Capturing VHDL-AMS Assertion Data in Interactive Mode](#)

[Capturing SPICE Extractions in a UCDB](#)

Contributor Window

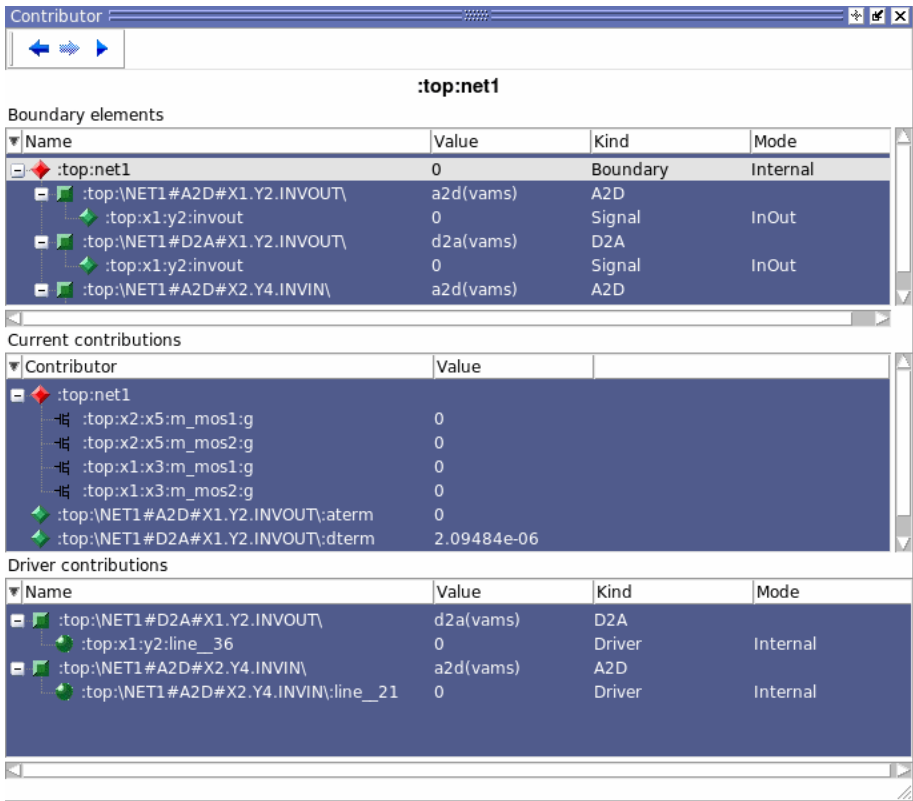
To access do one of the following:

- Choose **View > Contributor**.
- Right-click on an object in the **Objects Window** and choose View Contributor.
- Enter the following command into the **Transcript Window**:

```
view contributor
```

Displays the active connectivity associated with the root name of a selected object (an internal net or port, digital or analog). Used to interrogate the connectivity and read contribution values at an analog-to-digital or digital-to-analog boundary when troubleshooting your design.

Figure 17-2: Contributor Window



Characteristics

Table 17-2: Contributor Window Contents

Field	Description
Boundary elements	Lists the mixed-signal interconnectivity of nets, signals, analog nodes and A2D/D2A boundaries found on the highest parent

Table 17-2: Contributor Window Contents (continued)

Field	Description
	of the mixed-signal net currently selected in the Objects Window .
Current contributions	Shows the analog current contributions to the selected mixed-signal net. There is a menu to enable and set a Current Threshold. When the Current Threshold is enabled, current contributors with currents below the set threshold value will be hidden.
Driver contributions	Displays the names and values of each of the digital drivers on the selected net. The information here is similar to that reported using the drivers command.
 Go to this node	Highlights the location of the node selected in the Contributor Window, in the Structure and Objects windows.
 Go back one context	Scrolls backwards through the recent node selections
 Go forward one context	Scrolls forwards through the recent node selections

Usage Notes

When undocked, the Go to this node  and scroll buttons   are located on the Contributor window panel. When docked, they will be located on the toolbar.

It is not possible to access the following in the Contributor window:

- Some digital vector nets, because of time optimizations in Questa SIM. A warning will be issued when this occurs.
- SystemVerilog vectors of vectors.
- SystemVerilog elements of structs.

Related Topics

[drivers \[Questa ADMS Command Reference\]](#)

[tracei \[Questa ADMS Command Reference\]](#)

[Interrogating Nets in the Contributor Window](#)

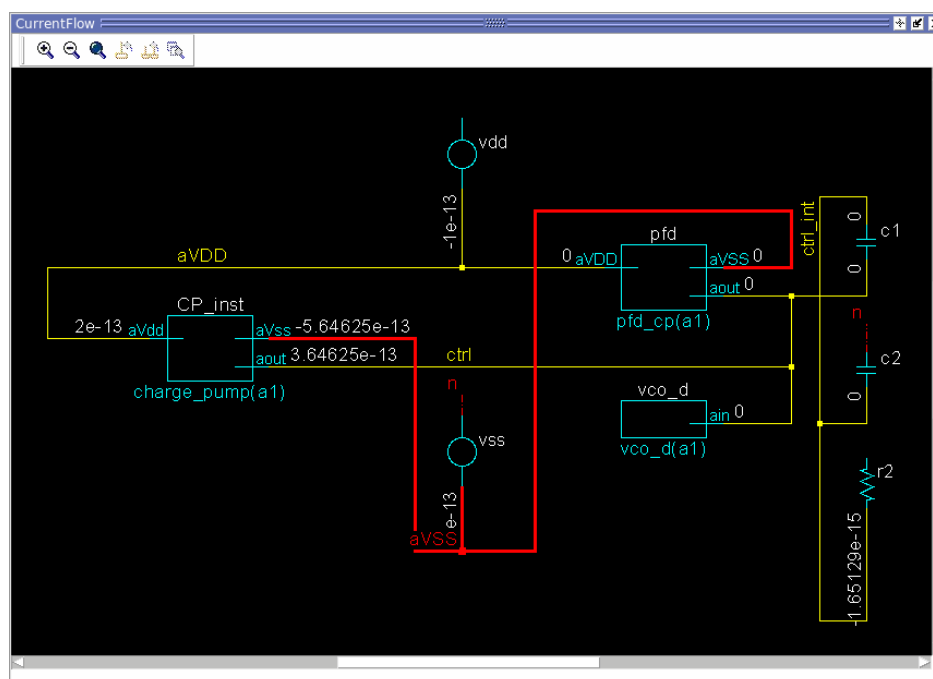
CurrentFlow Window

To access:

- Right-click on an analog current contribution in the Current contributions pane of the **Contributor Window** and choose **View > currentflow**

Displays a schematic of all current contributions of the selected net.

Figure 17-3: CurrentFlow Window



Objects

Table 17-3: CurrentFlow Window Actions

Action	Description
Double-click on the red dashed line beside a pin	Draws all current contributions for this pin, in red.
Single-click on a pin	Selects the corresponding region in the Structure Window and selects the pin in the Objects Window .
Single-click on a net	Selects the corresponding region in the Structure Window and the net in the Objects Window .
Single-click on an instance	Selects the corresponding region in the Structure Window .

Table 17-3: CurrentFlow Window Actions (continued)

Action	Description
Right-click on a selected net and choose Remove Selected Net	Removes the selected net from the schematic.
Right-click on the CurrentFlow Window and choose Zoom In	Zooms in.
Right-click on the CurrentFlow Window and choose Zoom Out	Zooms out.
Right-click on the CurrentFlow Window and choose Zoom Full	Zooms full (zooms to fit).
Right-click on the CurrentFlow Window and choose Clear	Clears the CurrentFlow window.
Click and drag to the lower-right	Zooms in.
Click and drag to the upper-right	Zooms out.
Click and drag to the lower-left	Zooms to fit (zooms full).
	When the window is undocked, zooms in.
	When the window is undocked, zooms out.
	When the window is undocked, zooms full (zooms to fit).

Usage Notes

The schematic in the window grows as new contributions are displayed. To clear the schematic, right-click and choose **Clear**. Or to remove a selected net, right-click on the selected net and choose **Remove Selected Net**.

Contributions are not shown if they are below the Current Threshold value set in the Contributions field in the [Contributor Window](#).

Current values are always displayed in amps.

A negative current value on a pin indicates positive current flowing out.

In a big network it can take a long time to compute all the node current flow values. In this situation a busy icon is shown. It is not possible to interrupt this computation.

The current flow value may fail to update if the Current Threshold value is changed or if the user re-runs the simulation by calling a new 'run' command.

Related Topics

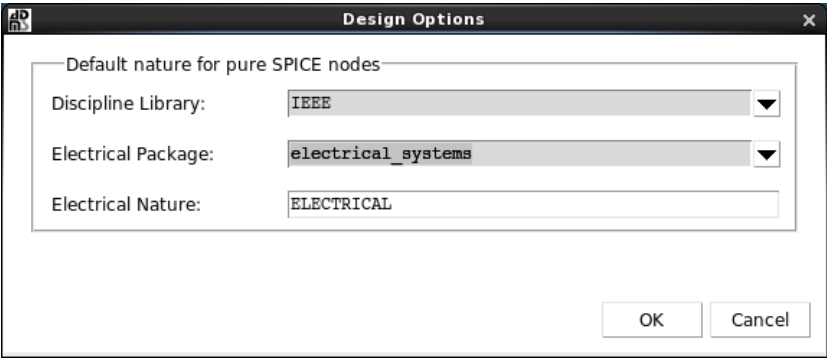
[Interrogating Nets in the Contributor Window](#)
[Contributor Window](#)

Design Options Dialog Box

To access: choose **Simulate > Design...**

Used to set default nature options for SPICE nodes in the design. This information is required in order to make SPICE nodes compatible with VHDL-AMS natures. SPICE parameter values are automatically allocated the VHDL STD Real type.

Figure 17-4: Design Options Dialog Box



Fields

Table 17-4: Design Options Dialog Box Contents

Name	Descriptions
Discipline Library	Allows selection of a Disciplines library
Electrical Package	Allows selection of an electrical package
Electrical Nature	Allows a label for electrical nature to be added

Related Topics

[Working with SPICE Netlists](#)

[Mapping Ports Using the Interface Matcher](#)

Eldo Commands Dialog Box

To access: click the **New** button on the [Figure 17-30](#) of the [Start Simulation Dialog Box](#).

Allows the specification of Eldo equivalent commands for use during simulation. Specified at the time of loading a design using the Start Simulation dialog box.

Note

Click the **More** button to show all options.

Figure 17-5: Eldo Commands Dialog Box - All Options

The screenshot shows the 'ELDO Commands' dialog box. At the top, the title bar reads 'ELDO Commands'. Below it, the 'File name' field contains 'hierarchy-test.cmd'. The 'Analysis' section has four options: 'OP' (unchecked), 'AC' (unchecked), 'Transient' (checked), and 'ModSST' (unchecked). The 'AC' option has 'Min frequency: 1' and 'Max frequency: 1e6'. The 'Transient' option has 'Simulation time: 10ns' and 'Print period: 100'. The 'ModSST' option has 'Simulation time: 10ns' and 'Print period: 100'. A 'Less' button is located below these options. A warning message states: 'Warning: All options below require advanced knowledge in analog simulation'. Below the warning, the 'AC Analysis: ASP per interval' is set to '10', with radio buttons for 'Linear' (selected), 'Decade', and 'Octave'. The 'Accuracy: eps' is set to '5e-3'. The 'Integration method' has radio buttons for 'Trapezoidal' (selected), 'Backward Euler', and 'Gear'. The 'Gear level' has radio buttons for '1', '2' (selected), '3', '4', '5', and '6'. The 'Simulation step: Hmin' is set to '1e-12', and 'Simulation step: Hmax' is an empty field. The 'Additional Options' field is also empty. At the bottom, there are 'Save' and 'Cancel' buttons.

File name:

Analysis

☐ OP

☐ AC Min frequency: Max frequency:

☒ Transient Simulation time: Print period:

☐ ModSST Simulation time: Print period:

Warning: All options below require advanced knowledge in analog simulation

AC Analysis: ASP per interval ☒ Linear ☐ Decade ☐ Octave

Accuracy: eps

Integration method: ☒ Trapezoidal ☐ Backward Euler ☐ Gear

Gear level: ☐ 1 ☒ 2 ☐ 3 ☐ 4 ☐ 5 ☐ 6

Simulation step: Hmin Simulation step: Hmax

Additional Options:

Fields

Table 17-5: Eldo Commands Dialog Box Contents

Name	Description
File name	Enter a name for the new CMD file should be created. The file will be created in the current working directory.
Analyses	
OP	Instructs the tool to include a DC operating point analysis of the design in the simulation run. See the description of the .OP command in the <i>Eldo Reference Manual</i> for more information.
AC	Instructs the tool to include an AC small-signal analysis of the design in the simulation run. See the description of the .AC command in the <i>Eldo Reference Manual</i> for more information.
Min frequency	Specifies the frequency at which to start the simulation (fstart). This option is only available if the AC option is selected.
Max frequency	Specifies the frequency at which to end the simulation (fstop). This option is only available if the AC option is selected.
Transient	Instructs the tool to include a transient analysis of the design in the simulation run. See the description of the .TRAN command in the <i>Eldo Reference Manual</i> for more information.
Simulation time	The transient analysis duration, in seconds. This can be specified as a parameter or as an expression. Note that the simulation time is based on TSTART being zero. This option is only available if the Transient option is selected.
Print period	The time interval used for the printing or plotting of the transient analysis results, in seconds. Also used to compute a default maximal internal timestep value in case the circuit does not contain any signals (no PWL/SIN, and so on), which is often the case in oscillator circuits. Can be specified as a parameter or as an expression. This option is only available if the Transient option is selected.
ModSST	Instructs the tool to include a modulated steady-state analysis of the design in the simulation run, supporting RF simulation. See the description of the MODSST command in the <i>Eldo RF User's Manual</i> for more information. Upon selecting this option, the Modulated Steady State Dialog Box is displayed.
Simulation time	The MODSST analysis duration, in seconds. This can be specified as a parameter or as an expression. This option is only available if the ModSST option is selected.
Print period	The time interval used for the printing or plotting of the MODSST analysis results, in seconds. Also used to compute a default maximal

Table 17-5: Eldo Commands Dialog Box Contents (continued)

Name	Description
	internal timestep value in case the circuit does not contain any signals, which is often the case in oscillator circuits. Can be specified as a parameter or as an expression. This option is only available if the ModSST option is selected.
More/Less	Toggles the dialog box between basic and advanced modes. The button label changes as applicable for the current mode.
Advanced Options	
AC Analysis: ASP per interval	Specifies the Analog Solution Point—the simulation interval of a kind defined by the chosen simulation Type: Linear - linear variation (the default setting). This Type supports negative frequencies. Decade - logarithmic variation. Octave - octave variation. This option is only available if the AC option is selected.
Accuracy: eps	Specifies the internal simulator accuracy. Refer to Global Tuning of the Accuracy - EPS in the <i>Eldo User's Manual</i> for more information.
Tuning	Specifies the set of tuning parameters to control accuracy: fast, standard, accurate, vhigh. Refer to Global Tuning of the Accuracy - TUNING in the <i>Eldo User's Manual</i> for more information
Integration method	Specifies the approximation scheme used to remove time-derivatives from the original circuit equations. Select from: • Trapezoidal - the default • Backward Euler • Gear - when selected, an additional option to specify the Gear level from 1 to 6 is available. Refer to the topic Speed and Accuracy in Eldo in the <i>Eldo User's Manual</i> for a thorough discussion of the implications of each integration method.
Simulation step: Hmin	Specifies the minimum internal timestep.
Simulation step: Hmax	Specifies the maximum internal timestep.
Additional Options	Allows further Eldo options to be specified beyond those covered by the dialog box controls above. Refer to the Simulator and Control

Table 17-5: Eldo Commands Dialog Box Contents (continued)

Name	Description
	Options topic of the <i>Eldo Reference Manual</i> for a complete list of available options.

Related Topics

- [Start Simulation Dialog Box](#)
- [Questa ADMS Command File](#)
- [Running an Interactive Simulation](#)

File Breakpoint Dialog Box

To access: Choose **Tools > Breakpoints** to display the **Modify Breakpoints Dialog Box**, then click **Modify**.

Used to modify properties of breakpoints.

Note
Breakpoints can be set in the source code using the [Source Window](#).

Figure 17-6: File Breakpoint Dialog Box

The screenshot shows a standard Windows-style dialog box titled "File Breakpoint". It features a blue title bar with a close button (X) on the right. The main area contains several input fields and buttons. At the top is a "Breakpoint Label" text box. Below it is a "File" section with a text box containing "adc12-ms.vhd" and a "Browse..." button to its right. Underneath are two side-by-side fields: "Line" with the value "74" and "Instance Name" with an empty text box. Below these is a "Breakpoint Condition" text box. At the bottom of the main area is a large "Breakpoint Commands" text area. At the very bottom of the dialog are "OK" and "Cancel" buttons.

Fields

Table 17-6: File Breakpoint Dialog Box Contents

Field	Description
Breakpoint Label	An optional label for the breakpoint
File	The source file in which the breakpoint exists
Line	The line number on which the breakpoint exists

Table 17-6: File Breakpoint Dialog Box Contents (continued)

Field	Description
Instance Name	The full pathname to an instance that sets a SystemC breakpoint so it applies only to that specified instance
Breakpoint Condition	Used to specify one or more conditions that determine whether the breakpoint is observed. Condition expressions must be enclosed within quotation marks (" "). If the condition is true, the simulation stops at the breakpoint. If false, the simulation bypasses the breakpoint. A condition cannot refer to a VHDL variable (only a signal).
Breakpoint Command	Specifies one or more commands to be run at the breakpoint.

Related Topics

[Setting File-Line Breakpoints](#)

[Modifying File-Line Breakpoints](#)

[bp \[Questa ADMS Command Reference\]](#)

Force Selected Signal Dialog Box

To access:

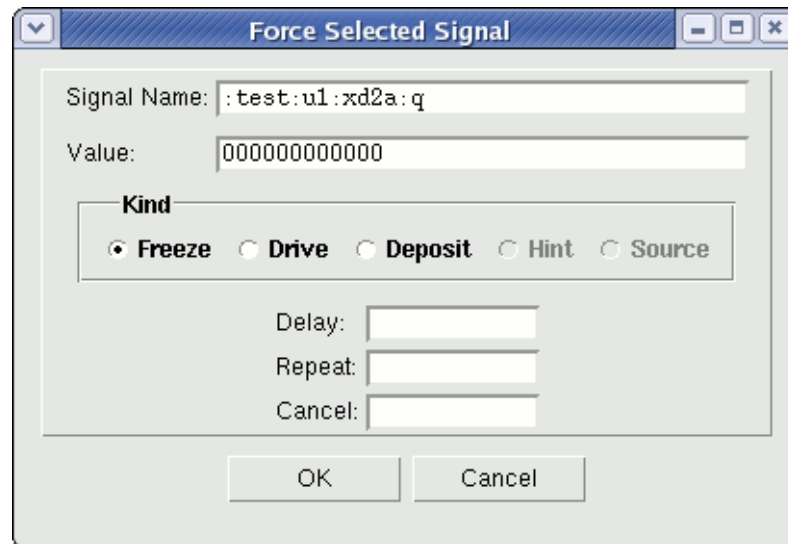
- With the **Objects Window** active, choose **Objects > Force**
- Right-click on an object in the **Objects Window** and choose **Modify > Force**

Used to apply a stimulus to the selected signal, terminal, quantity or boundary element.

Description

The options on this dialog box are different depending on whether the object is a signal or a net. for signals, it is formatted in signal mode, as shown in [Figure 17-7](#); for terminals, quantities and boundary elements, it is formatted in net mode as shown in [Figure 17-8](#).

Figure 17-7: Force Selected Signal Dialog Box - Signal Mode



The screenshot shows a dialog box titled "Force Selected Signal". It contains the following fields and controls:

- Signal Name:** A text field containing the value `:test:u1:xd2a:q`.
- Value:** A text field containing the value `000000000000`.
- Kind:** A group box containing five radio buttons: **Freeze** (selected), **Drive**, **Deposit**, **Hint**, and **Source**.
- Delay:** A text field.
- Repeat:** A text field.
- Cancel:** A text field.
- Buttons:** **OK** and **Cancel** buttons at the bottom.

Figure 17-8: Force Selected Signal Dialog Box - Net Mode

Fields

Table 17-7: Force Selected Signal Dialog Box Contents

Name	Description
Signal Name / Net Name	The name of the signal/terminal/quantity being forced. This defaults to the name of the object selected when the dialog box was launched.
Value / Eldo Parameters	This field is named based on whether a signal/quantity (Value) or terminal (Eldo Parameters) is selected when the dialog box is launched: - Value - Overwrite the initial value. A value can be specified in radices other than decimal by using the form: base#value. For example, 16#EE specifies the hexadecimal value EE. - Eldo Parameters - Enter the parameters of the source as an Eldo string (i.e. like the SPICE command in the .cmd or .cir file) excluding the name of the source and the names of the terminals.
Kind	Specifies the type of force option to use. The options available depend on the object type you are forcing: - Freeze - (signal) Freezes the signal at the specified value until it is forced again or until it is unforced. - Drive - (signal) Sets the signal to the specified value. The value remains until there is a subsequent driver transaction,

Table 17-7: Force Selected Signal Dialog Box Contents (continued)

Name	Description
	or until the signal or net is forced again, or until it is unforced. • Deposit - (signal) Sets the signal to the specified value. The value remains until there is a subsequent driver transaction, or until the signal or net is forced again, or until it is unforced. • Hint - (terminal/boundary element) Force the terminal/boundary element with the specified source. • Source - (terminal/quantity) Force the terminal/quantity with the specified value.
Delay	The time at which the force value is applied. The time is relative to the current time unless an absolute value is specified by preceding the value with the @ character. If no time units are specified, the default is the resolution units selected at simulation startup. A zero-delay force causes the change to occur in the current, rather than the next, simulation delta cycle.
Repeat	The time—relative to the current time—at which the force command is repeated. A repeating force command forces a value before other, non-repeating force commands that occurred in the same step.
Cancel	The time—relative to the current time—at which the force is cancelled. Cancellation occurs at the last simulation delta cycle of a time unit. A value of 0 cancels the force at the end of the current time period.

Usage Notes

Note that a force on a terminal may only occur at the beginning of an analog solution point (ASP) and will be used as a hint to start iterations to compute the values at this ASP. Use the **run** -next BeginofASP command.

Related Topics

[force \[Questa ADMS Command Reference\]](#)
[Objects Window](#)

Interface Matcher

To access: Enter the following at the command line:

```
vaspi -interactive [<options>]
```

```
[<digital_unit> <subckt_name>[@<file_name>]]
```

Used to make interface associations between a SPICE subcircuit and a digital design unit, by mapping analog pins to digital ports, either automatically or manually. This creates an interface association file (.assoc) in the current working directory.

Description

If you provide unit names and options on the command line, the corresponding fields are populated in the Interface Matcher Unit Selection window.

Figure 17-9: Interface Matcher Selection Window

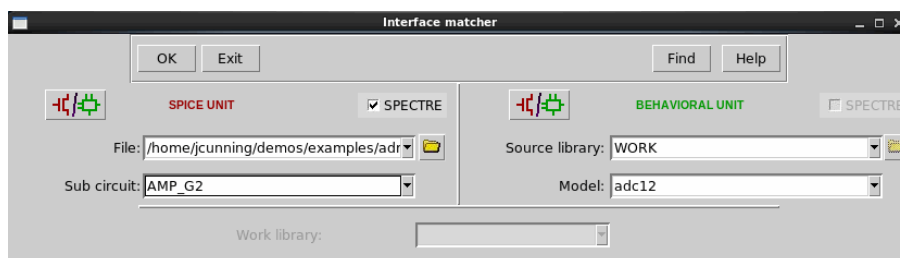
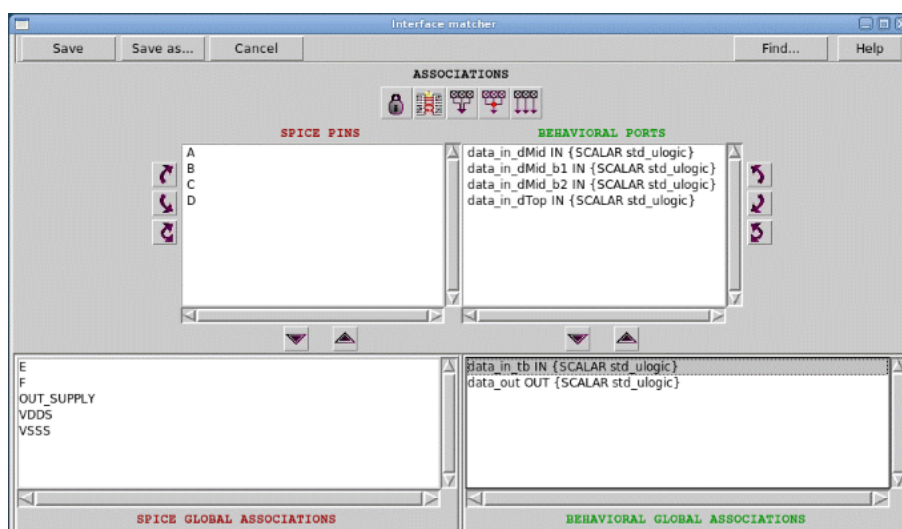


Figure 17-10: Interface Matcher Dialog Box



Fields

Table 17-8: Interface Matcher Dialog Box

Icon	Name	Description
Associations		
	List Lock	Use this button to toggle the list lock on and off. When list lock is on, both the analog and digital lists move together. When list lock is off, the lists are free to move independently.
	Automatically Map	Attempts to map associated ports automatically by matching up pins and ports with similar names, and displays them in order in the association lists. This can also be specified on the command line using the option -by_name when invoking the Interface Matcher. Any pins/ports that the automatic mapping was unable to map are displayed in the Global Associations sections.
	Analog Association	Create an analog association to map an array.
	Interconnection	Interconnect pins/ports to map them to another unique pin/port.
	Split	Split an association or interconnection previously created.
	SPICE PINS	Lists the current pin mapping associations. Analog pins in this list are mapped with the digital ports they are adjacent to in the BEHAVIORAL PORTS list.
	BEHAVIORAL PORTS	Lists the current digital port mapping associations. Digital ports in this list are mapped with the analog pins they are adjacent to in the SPICE PINS list.
	SPICE GLOBAL ASSOCIATIONS	Displays SPICE pins that are either associated globally or declared as unconnected. They can be added or removed from this field by using the up/down arrows located above the section.
	BEHAVIOURAL GLOBAL ASSOCIATIONS	Displays behavioral ports that are either associated globally or declared as unconnected. They can be added or removed from this field by using the up/down arrows located above the section.

Related Topics

[Mapping Ports Using the Interface Matcher](#)
[Associating Design Units with vamatch](#)

vaspi [Questa ADMS Command Reference] Interface Association File (.assoc)

Supply Connections Dialog Box

To access: Open by double-clicking a Spice pin either from SPICE PINS or SPICE GLOBAL ASSOCIATIONS in the Interface Matcher dialog box.

Defines the association between the supply Spice pin and the Power Domain Connection.

Figure 17-11: Supply Connections Dialog Box

Supply connections

Selected port: P_IN

Object to map description:

Mode

☒ in

☐ out

☐ inout

UPF

Power domain connection name

P_IN

UPF pg_type

Value

Global net

Discipline

RESET

OK

Fields

Table 17-9: Supply Connections Dialog Box

Field	Description
Mode	The port direction: • in (default)

Table 17-9: Supply Connections Dialog Box (continued)

Field	Description
	• out • inout (not available for a UPF supply port)
Power domain connection name	The name of the Power Domain Connection. This is automatically filled with the Spice supply port name.
UPF pg_type	A UPF pg_type (optional). The possible values are: • primary_power • primary_ground • pwell • nwell • deepnpwell • deepnwell
Value	The initial state and voltage. Possible values of the initial state are: • off • undetermined • partial_on • full_on
Global net	The name of a global Spice or HDL node to match the Spice port against the HDL ports. The format for the name of a global HDL node is: <pre><pathSeparator>[<library>.<package><pathSeparator><signal></pre> If you do not specify <i><library></i>, all packages loaded in the design will be searched.
Discipline	A discipline is attached to a Spice port to determine the bound array element at elaboration time.

Related Topics

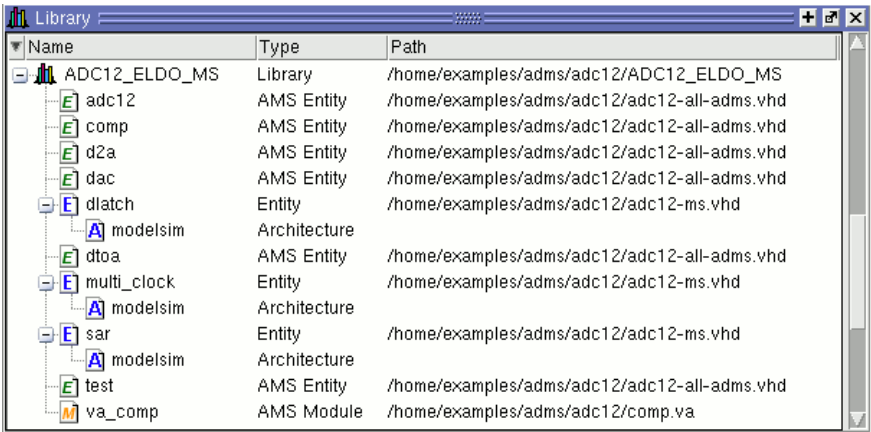
Interface Matcher

Library Window

To access: choose **View > Library**.

Displays the libraries in the current project and their contents.

Figure 17-12: Library Window



Characteristics

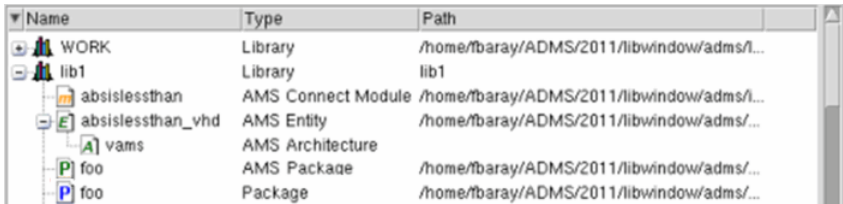
Table 17-10: Library Window Contents

Name	Description
Name	A hierarchical tree view showing, at the first level, the library name, then primary units, then secondary units. Colored icons are used to quickly identify items, see Table 3-3 in GUI Icons and Their Meanings .
Type	The type of each item shown in the tree.
Path	The source file or directory path of each item shown in the tree.

Usage Notes

In a library, two packages with the same name may exist, one VHDL and one VHDL-AMS. The VHDL-AMS version may contain analog declarations that cannot be used in a digital context. A VHDL model will use the digital version of the package and a VHDL-AMS model will use the analog version.

Figure 17-13: Multiple Packages in the Library Window



Related Topics

[Design Libraries](#)

[Creating a Design Library](#)

[Viewing Library Contents](#)

[Refreshing Libraries](#)

Locals Window

To access: Choose **View > Locals** or use the command: view locals

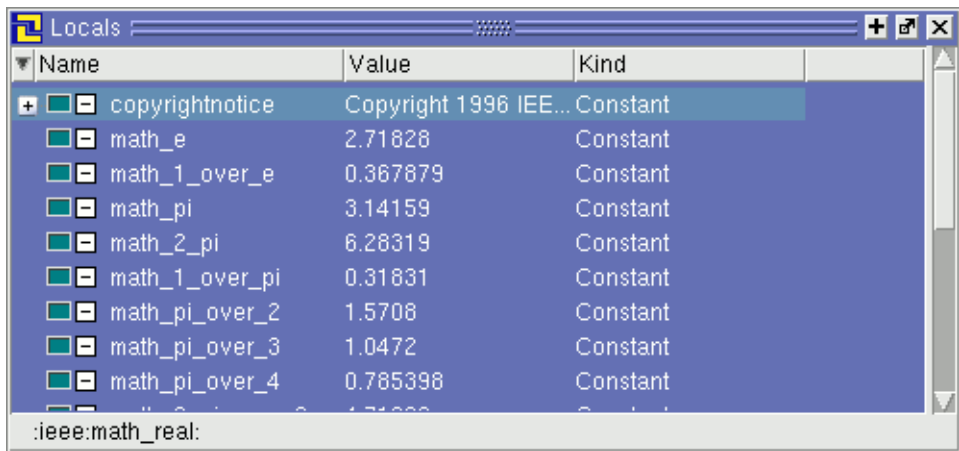
Lists the names of VHDL-AMS variables, generics and constants within the selected process, followed by the current value(s) associated with each name.

Description

When a Questa SIM region has been selected in the **Structure Window**, the Locals window is automatically updated to display the correct content.

The pathname (design structure) of the current process is displayed at the bottom of the Locals window. The design structure can remain hidden, using the -nodebug option of the vacom command and the Questa SIM vcom command.

Figure 17-14: Locals Window



Characteristics

Table 17-11: Locals Window Contents

Name	Description
Locals List	
Name	The names of the immediately visible data objects. Color conventions indicate which parts are managed by which language. The names of any VHDL-AMS composite types (arrays) are shown in a hierarchical fashion. Hierarchy is indicated with plus (expandable), minus (expanded), and blank (single level) boxes.

Table 17-11: Locals Window Contents (continued)

Name	Description
Value	The current value(s) associated with each name
Kind	The type of variable

Usage Notes

Although the constants (param keyword) of Verilog-A models computed by ADiT are displayed, the variables are not. Instead of the value, the following string appears:

```
<ADiT: N/A>
```

In Questa ADMS the names and values of variables appear in the Locals window only after the DC point is computed. For ADiT/Verilog-A, the name appears after the DC point is computed, but the value is listed as "N/A".

Related Topics

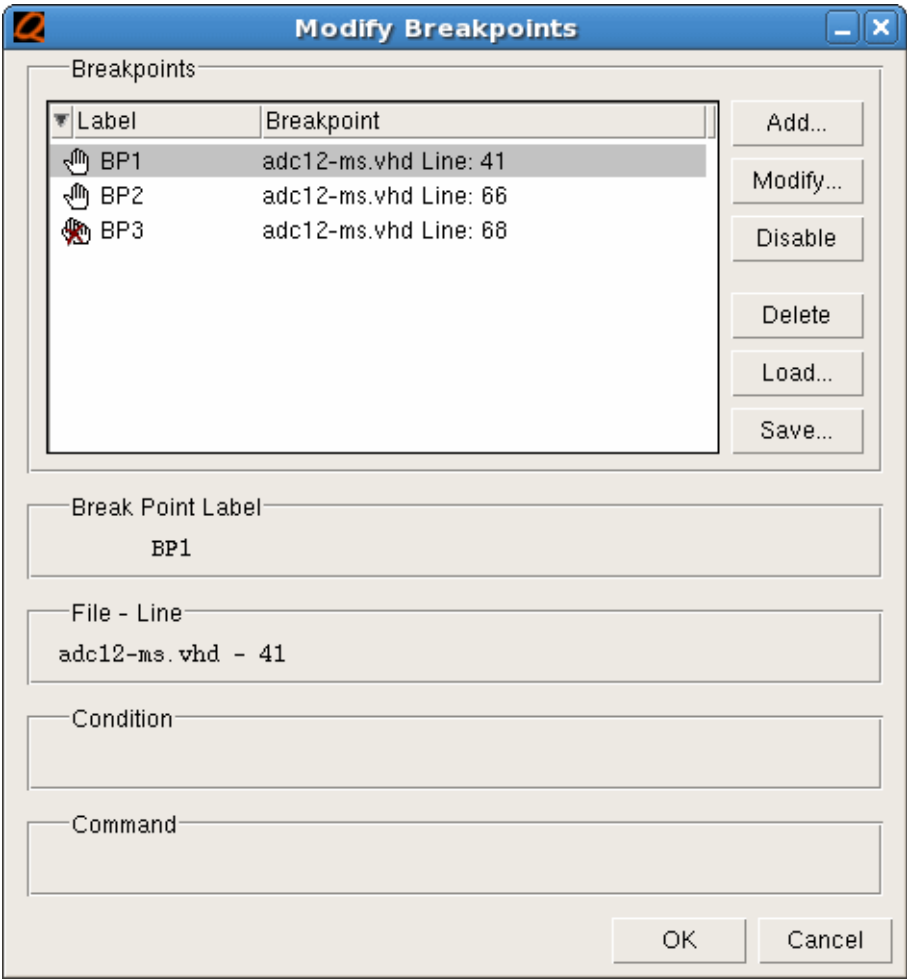
[Examining Design Files in the Source Window](#)
[GUI Icons and Their Meanings](#)

Modify Breakpoints Dialog Box

To access: Choose **Tools > Breakpoints** or right-click on a breakpoint marker in the **Source Window** and choose **Edit all Breakpoints**

Used to add, modify, disable, delete, load and save breakpoints for debugging your source code.

Figure 17-15: Modify Breakpoints Dialog Box



Fields

Table 17-12: Modify Breakpoints Dialog Box Contents

Field	Description
Label	The user-defined label for the breakpoint
Breakpoint	The breakpoint’s location in the source code (filename and line number).
Add	Used to add a new breakpoint based on either:

Table 17-12: Modify Breakpoints Dialog Box Contents (continued)

Field	Description
	• A signal or signal value • A file and line number
Modify	Opens the File Breakpoint Dialog Box to edit values
Disable	Disables the breakpoint selected in the list
Delete	Deletes the breakpoint selected in the list
Load	Allows a previously saved do file containing breakpoint information to be loaded.
Save	Saves a do file containing breakpoint information

Related Topics

[File Breakpoint Dialog Box](#)

[Setting File-Line Breakpoints](#)

[Modifying File-Line Breakpoints](#)

[Saving and Loading Breakpoints](#)

[bp \[Questa ADMS Command Reference\]](#)

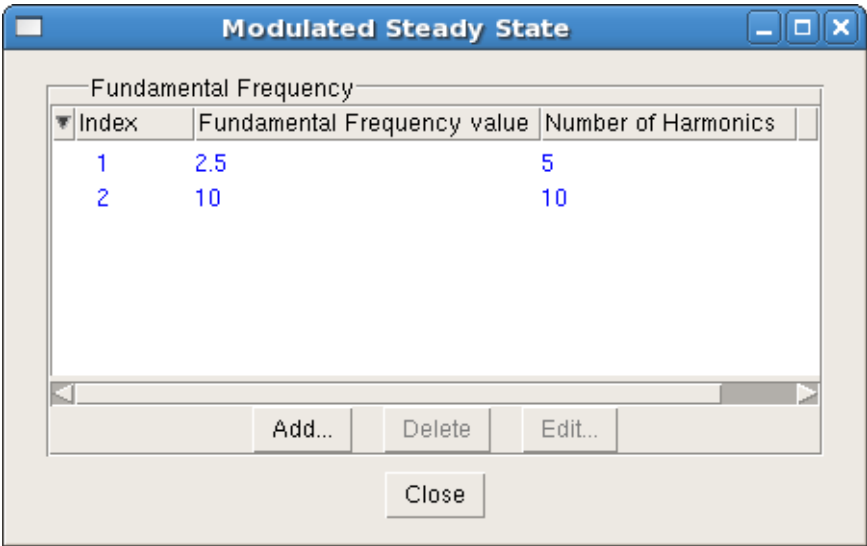
[bd \[Questa ADMS Command Reference\]](#)

Modulated Steady State Dialog Box

To access: Choose the ModSST option on the [Eldo Commands Dialog Box](#).

Used to specify the fundamental frequencies and number of harmonics for a MODSST simulation. The Fundamental Frequency list is blank when the dialog box is first displayed.

Figure 17-16: Modulated Steady State Dialog Box



Fields

Table 17-13: Modulated Steady State Dialog Box Contents

Name	Description
Index	Specifies the integer suffix applied to the fund and nharm parameters when the CMD file is generated. For example, an index of one results in parameters fund1 and nharm1 being generated.
Fundamental Frequency value	Specifies the value for the fund parameter at this Index.
Number of Harmonics	Specifies the value for the nharm parameter at this Index.
Add	Used to add a new line. Enter values for Index, Fundamental Frequency value and Number of Harmonics in the entry dialog box and click OK.
Delete	Used to remove a line from the list.
Edit	Used to edit the values of a line.

Related Topics

[Eldo Commands Dialog Box](#)

[Running an Interactive Simulation](#)

[Start Simulation Dialog Box](#)

Objects Window

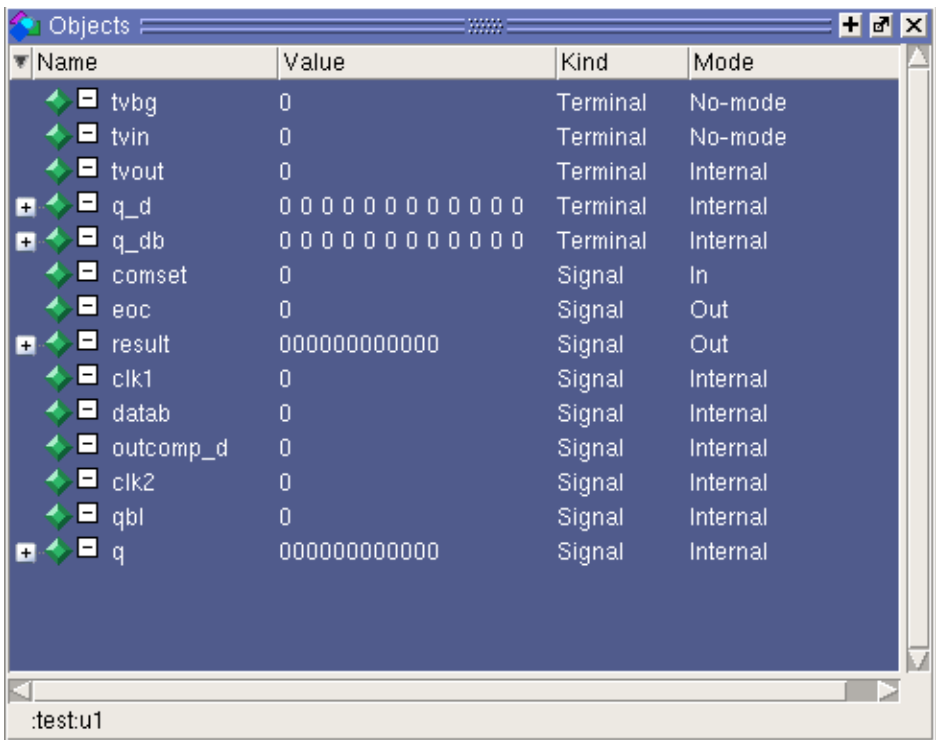
To access: Choose **View > Objects**, or use the command: view objects

Displays the digital, analog and mixed-signal objects within the selected region.

Description

When a Questa SIM region has been selected in the **Structure Window**, the Objects window is automatically updated to display the correct content.

Figure 17-17: Objects Window



Characteristics

Table 17-14: Objects Window Contents

Name	Description
Objects List	
Name	The design object name. Aliases are only visible in the Objects window. For example when you select an alias to be plotted, only the source of the alias will appear selected. Only the source name of the alias will appear in

Table 17-14: Objects Window Contents (continued)

Name	Description
	the corresponding window. Implicit Verilog-AMS branches are listed inside brackets, separated with commas and no spaces: (<net1>,<net2>).
Value	The design object's value.
Kind	The design object type. The following types are supported: • Boundary, for mixed-signal nets where A2D, D2A or bidirectional boundary elements are attached. These are expandable to show boundary elements (first level of expansion), then ports (second level of expansion). • Signal, for a purely digital port. • Quantity. A quantity represents continuous time or frequency waveforms that may only take floating-point values. • Terminal, for VHDL-AMS terminals, Verilog-AMS analog nets or SPICE nodes. The value that is displayed is the reference value of the terminal. • Branch, for both implicit and declared Verilog-AMS branches.
Mode	The following modes are identified for each item listed: • In, Out or Inout, for ports of direction IN, OUT or INOUT, whatever they can be: Signal, Quantities or Verilog-AMS analog nets (Terminals). • No-mode, for ports that have no directions (like VHDL-AMS terminals or SPICE pins) • Internal, for explicitly VHDL-AMS, Verilog-AMS or Verilog declared objects or SPICE nodes. Quantities are not tagged as internal (see below). • Across, Through or Free, for internal Quantities. • A2D, for the boundary nets where only an A2D boundary element is attached. • D2A, for the boundary nets where only a D2A boundary element is attached. • Bidir, for the boundary nets where only a bidirectional boundary element is attached.

Usage Notes

The Objects window uses the Questa ADMS GUI color conventions to indicate which parts are managed by which language, see [GUI Icons and Their Meanings](#).

Objects that have been added to the [Wave Window \(Solido Waveform Analyzer\)](#) are labelled with a W:



Objects which have been logged using the **add log** command are labelled with an L:



Analog or mixed-signal objects can be filtered out of the window by type. Check or uncheck the options under the **View > Filter** menu to control what is visible in the Objects window. You can also use **View > Sort** to sort analog or mixed-signal objects in ascending or descending order.

Note

Digital objects cannot be filtered or sorted in the Objects window using these menu items.

Verilog-AMS named events are not visible in the Objects Window.

Related Topics

[Filtering and Sorting Objects in the Objects Window](#)

[Adding Signals to the Solido Waveform Analyzer Wave Window](#)

[Contributor Window](#)

Preferences Dialog Box

To access: Choose **Tools > Edit Preferences**.

Used to change colors, fonts, and the settings of Tcl variables.

Description

The dialog box is organized into two tab groups:

- **By Window**
Used to change colors and fonts used by GUI windows.
- **By Name**
Used to change the settings of any Tcl variable.

Figure 17-18: Preferences Dialog Box - By Window Tab

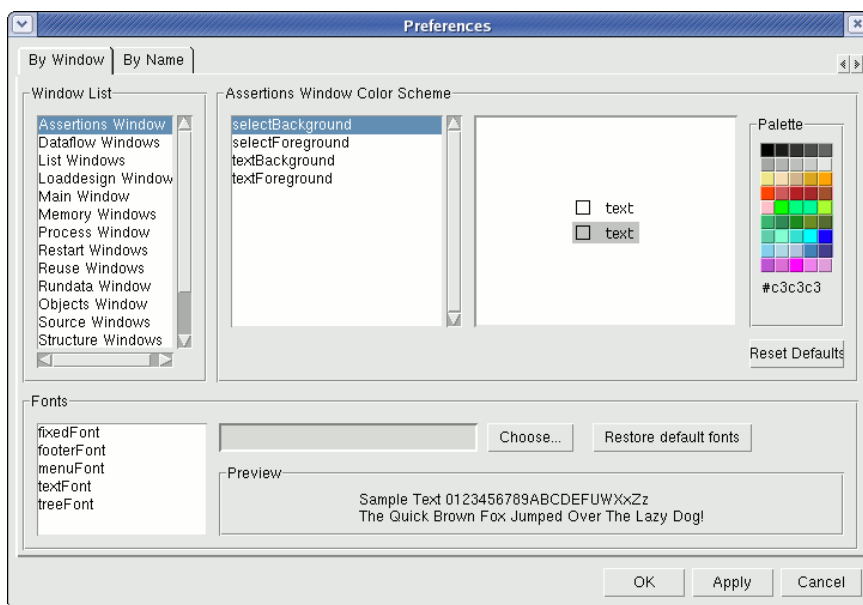
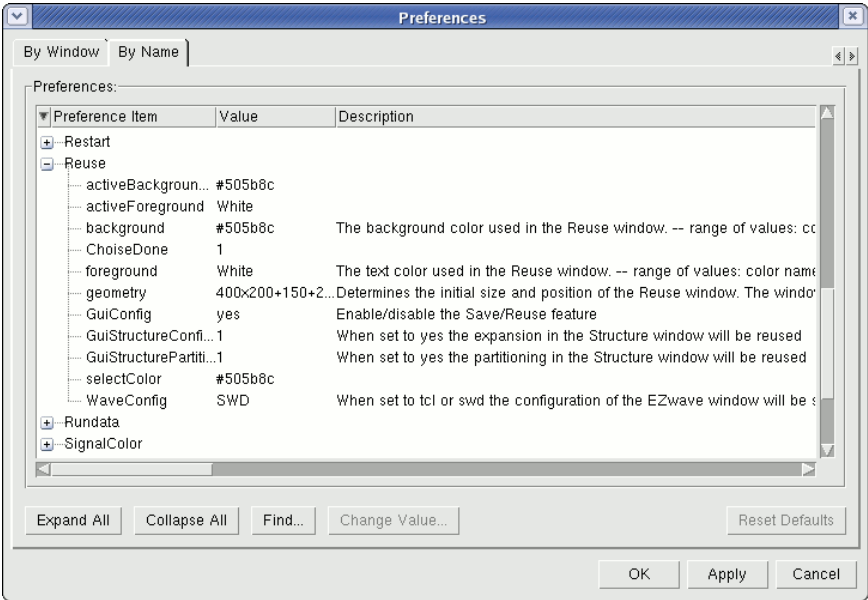


Figure 17-19: Preferences Dialog Box - By Name Tab



Fields

Table 17-15: Preferences Dialog Box - By Window Tab Contents

Field	Description
Window List	Lists all the available GUI windows
<Window Name> Color Scheme	Allows window elements to be selected.
Palette	Allows color selection
Reset Defaults	Clears the currently selected color scheme and restores the window's default color settings
Fonts	Lists the current font types available in the selected window.
Choose	Allows a different font to be chosen for the selected window font type.
Restore default fonts	Clears the currently selected font scheme and restores the window's default font settings.

Table 17-16: Preferences Dialog Box - By Name Tab Contents

Field	Description
Preference Item	Expandable list of preferences
Value	The current value of the preference

Table 17-16: Preferences Dialog Box - By Name Tab Contents (continued)

Field	Description
Description	A description of the preference
Expand All	Expand all items in the preferences list in order to display all Preferences
Collapse All	Collapses all items in the preferences list
Find	Opens a search box to search for preferences
Change Value	Allows a new value for the preference to be entered
Reset Defaults	Used to restore the default settings of a preference category

Related Topics

[Changing GUI Preferences](#)

[GUI Preference Variables](#)

Processes Window

To access: Choose **View > Process** or use the command: view processes

Displays a list of processes and indicates the pathname of the instance in which the process is located.

Description

The window can be viewed in two modes:

- Active Processes - these are all the processes that are scheduled to run during the current simulation cycle.
- Processes in Region - these are any processes that exist in the region that is selected in the **Structure Window**. This is the default view of the Processes window.

Figure 17-20: Processes Window - Active Processes

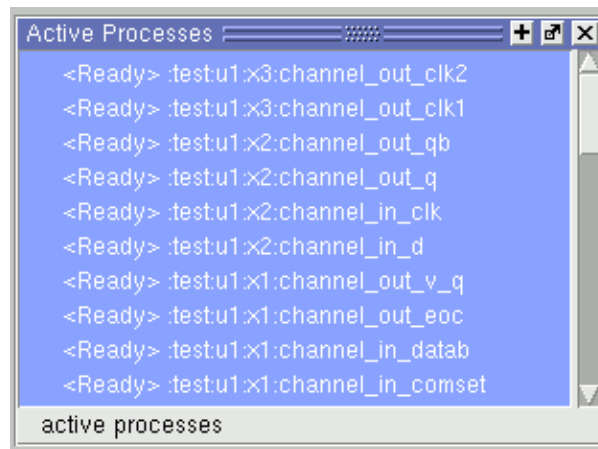
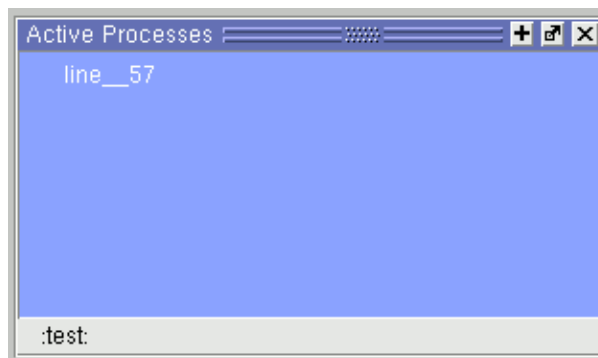


Figure 17-21: Processes Window - Processes in Region



Use either the Process menu or the **Process Toolbar** to switch between process modes.

Objects

Table 17-17: Processes Window Contents

Name	Description
Active Processes List	Lists all processes scheduled to run in the current simulation cycle (Active mode) or all processes in the selected region (Processes in Region Mode). Selecting a process in Active mode results in the following windows updating to display more information: • The Locals Window displays the VHDL-AMS variables of the selected process. • The Source Window shows the associated source code of the selected process.
Status Bar	In Processes in Region mode, the status bar displays the name of the region selected in the Structure Window - the region for which processes are being displayed. In Active mode, the status bar simply reads active processes.

Related Topics

[view \[Questa ADMS Command Reference\]](#)

Questa ADMS > Questa Import Library Dialog Box

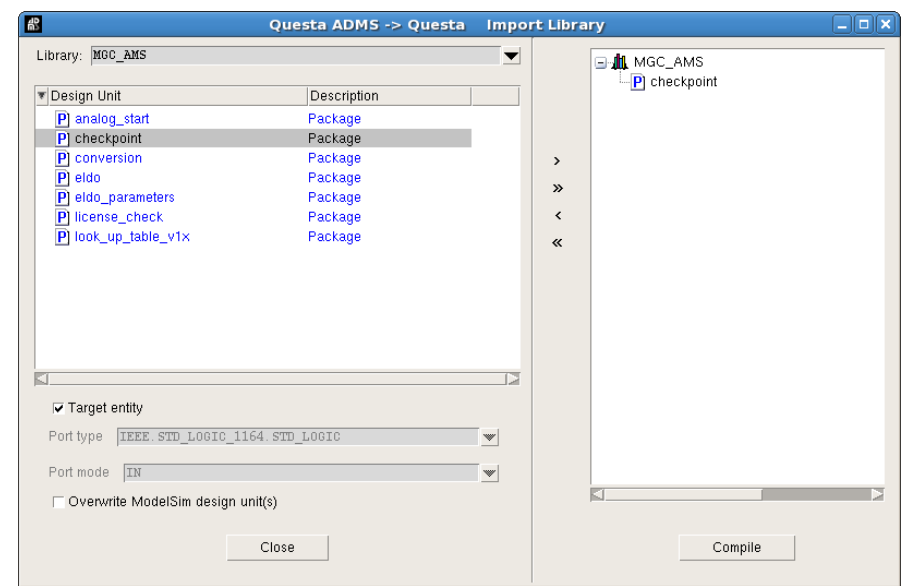
To access: Enter the command `import_adms` into the command line prompt or the [Transcript Window](#).

Used to transfer mixed-signal design units to a Questa SIM library. You can instantiate transferred design units inside a Questa SIM VHDL or Verilog description.

Note
Although this functionality is available, it is not necessary to use it, as Questa ADMS design units are automatically visible from Questa SIM.

Description

Figure 17-22: Questa ADMS > Questa Import Library Dialog Box



You can also run `import_adms` in batch mode using the `-c` option; specifying the required design unit to transfer design units.

Fields

Table 17-18: Questa ADMS > Questa Import Library Dialog Box Contents

Field	Description
Library	Specifies the Questa ADMS library. When this field is chosen, the list of design units is populated with those that are not Questa SIM design units or modules (as is the case for <code>import_ms</code>), but analog or mixed-signal design units. The selection of the source

Table 17-18: Questa ADMS > Questa Import Library Dialog Box Contents (continued)

Field	Description
	Questa ADMS library also selects the destination Questa SIM library which is linked to that Questa ADMS library.
> Import Selection	Moves selected items from the list of design units on the left into the "items to be transferred" list on the right
>> Import All	Moves all of the items from the list of design units on the left into the "items to be transferred" list on the right
< Remove Selection	Removes any selected items from the "items to be transferred" list
<< Remove All	Removes all items from the "items to be transferred" list
Target Entity	Specifies that a target entity with the same name as the source Questa ADMS entity already exists in the current Questa SIM library (and has already been compiled into this library). This information is used to determine how the analog ports will be replaced during the import. When this option is selected the "Port type" and "Port Mode" fields will be unavailable.
Port type	Specifies a port type to determine how the analog ports will be replaced during the import: • IEEE.STD_LOGIC_1164.STD_LOGIC (default) • STD.STANDARD.BIT • STD.STANDARD.REAL • STD.STANDARD.INTEGER To specify a different port type, you can enter it into the Port type field in the following format: <library_name>.<package_name>.<type_name>
Port mode	Specifies the mode that all analog ports will be converted to: IN (default), OUT or INOUT.
Overwrite ModelSim design unit(s)	Forces any existing Questa SIM design units to be overwritten by imported units.
Compile	Transfers the items in the "items to be transferred" list to Questa ADMS. The dialog box remains open so it is possible to select another library and continue.
Close	Closes the dialog box

Usage Notes

Select items from the list of design units and use the transfer buttons (>, >>, <, <<) to move them into the **items to be transferred** list on the right. > and < will move only the selected items between the lists; >> and << will move all items from one list to the other. When you click **Compile**, the items to be transferred are transferred to Questa SIM. The dialog box is not closed so it is possible to select another library and continue. Otherwise, click **Close** to exit the dialog box.

Related Topics

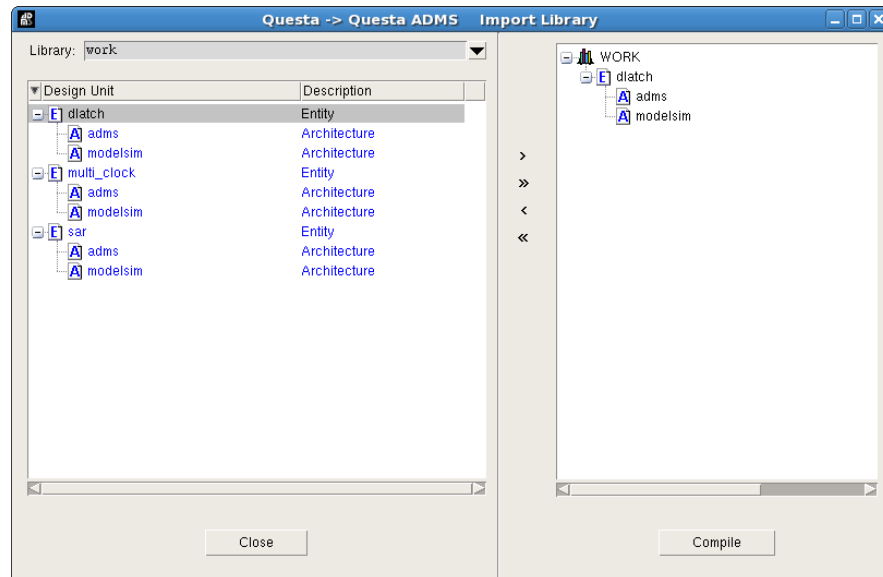
[import_adms \[Questa ADMS Command Reference\]](#)

Questa > Questa ADMS Import Library Dialog Box

To access: Enter the command `import_ms` into the command line prompt or the [Transcript Window](#)

Used to transfer digital design units into Questa ADMS. For use when the required Questa SIM design units do not fulfill the necessary conditions for automatic import.

Figure 17-23: Questa ADMS Import Library Dialog Box



Fields

Table 17-19: Questa > Questa ADMS Import Library Dialog Box Contents

Field	Description
Library	Specifies the Questa ADMS library (same effect as the <code>-lib</code> option). When this field is chosen, the list of design units is populated with all the items present in the associated Questa SIM library.
> Import Selection	Moves selected items from the list of design units on the left into the "items to be transferred" list on the right.
>> Import All	Moves all of the items from the list of design units on the left into the "items to be transferred" list on the right.
< Remove Selection	Removes any selected items from the "items to be transferred" list.
<< Remove All	Removes all items from the "items to be transferred" list.

Table 17-19: Questa > Questa ADMS Import Library Dialog Box Contents (continued)

Field	Description
Compile	Transfers the items in the “items to be transferred” list to Questa ADMS. The dialog box remains open so it is possible to select another library and continue.
Close	Closes the dialog box.

Related Topics

[Importing Design Units Manually](#)
[import_ms \[Questa ADMS Command Reference\]](#)

Restart Dialog Box

To access: Choose **Simulate > Run > Restart** or use the command: restart

Used to reload the design elements and reset the simulation time to zero.

Figure 17-24: Restart Dialog Box



Note

If no waves or logs have been created, the restart will run without opening the dialog box.

Fields

Table 17-20: Restart Dialog Box Contents

Name	Description
List Format	When enabled, the contents of the List window are reloaded following the restart.
Wave Format	When enabled, the Solido Waveform Analyzer Wave window configuration is reloaded following the restart, overriding any request from the command file.
Breakpoints	When enabled, any breakpoints set in the design are reloaded following the restart.
Logged Signals	When enabled, the contents of the JWDB database are reloaded following the restart.

Related Topics

GUI Preference Variables

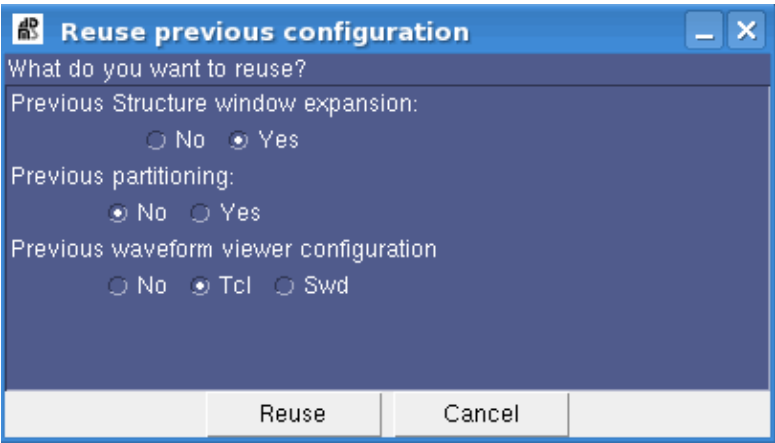
[restart \[Questa ADMS Command Reference\]](#)

Reuse Previous Configuration Dialog Box

This dialog box is displayed after the design has been elaborated when the PrefReuse(GuiConfig) variable (see [GUI Preference Variables](#)) is set to ask.

Prompts you to confirm the settings of the save/reuse variables.

Figure 17-25: Reuse Previous Configuration Dialog Box



Fields

Table 17-21: Reuse Previous Configuration Dialog Box Contents

Field	Description
Previous Structure window expansion	Loads the design with the previous Structure Window configuration
Previous partitioning	Loads the design with the previous partitioning options
Previous waveform viewer configuration	Loads the design with the previous Solido Waveform Analyzer configuration
Reuse	Loads the design with the chosen previous configuration options
Cancel	Loads the design without using the previous configuration

Related Topics

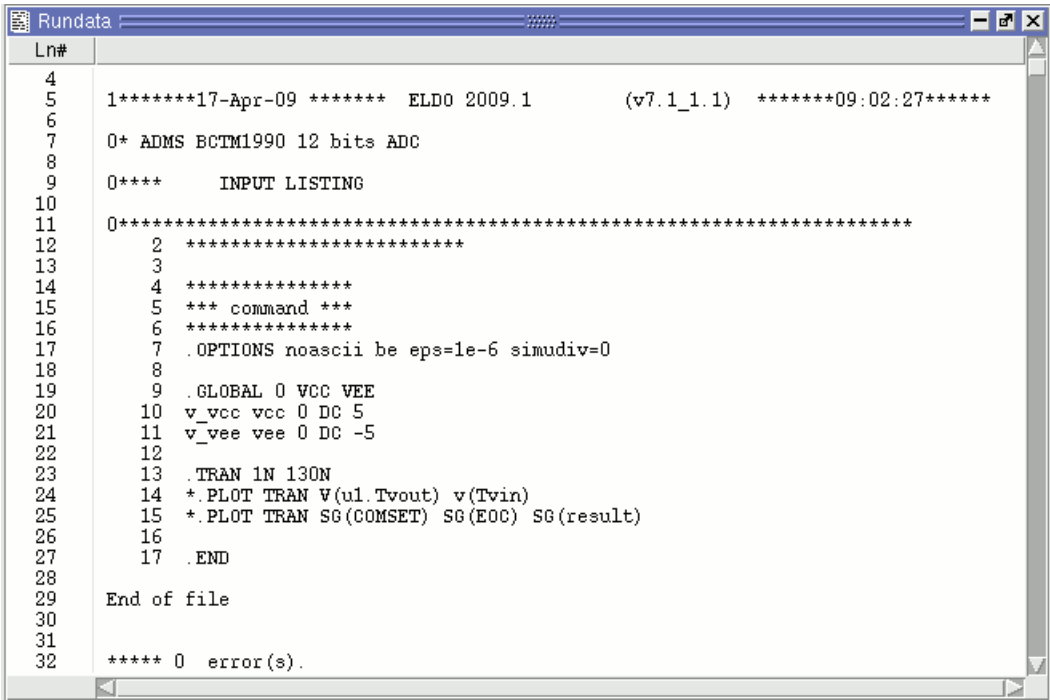
[Save/Reuse of Questa ADMS GUI Configuration](#)
[GUI Preference Variables](#)

Rundata Window

To access: Choose **View > Rundata**

Displays the *.chi* results file. This can be done after an AC analysis has been requested, and the *.PRINT* command specified.

Figure 17-26: Rundata Window



Objects

Table 17-22: Rundata Window Contents

Object	Description
Ln#	The line number.

Usage Notes

You can copy and paste text between the Rundata window and the [Transcript Window](#). Select the text to copy, then paste it into the Transcript window with the middle mouse button. Alternatively, you can use the cut, copy, and paste window toolbar options of the Rundata and Transcript windows.

Related Topics

Viewing Simulation Results

Runtime Options Dialog Box

To access: **Simulate > Runtime Options**

Specifies how the simulator behaves when messages, errors, or failures occur during the current simulation. You can also use the Runtime Options dialog box to specify how the simulator behaves at the end of the simulation.

Figure 17-27: Runtime Options Dialog Box—Message Severity Tab

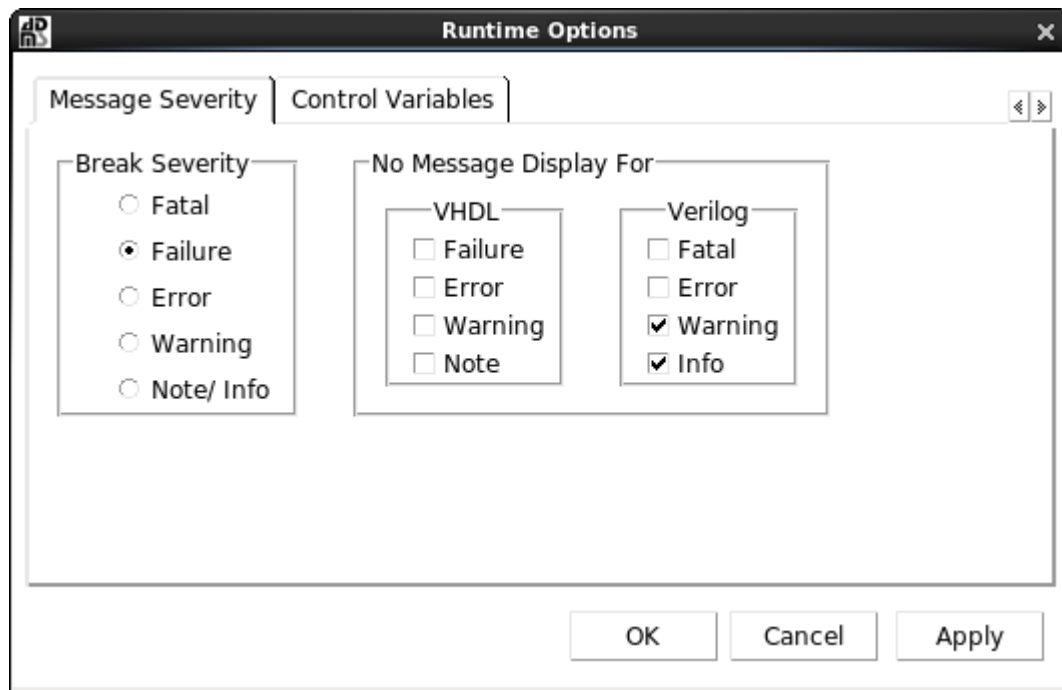
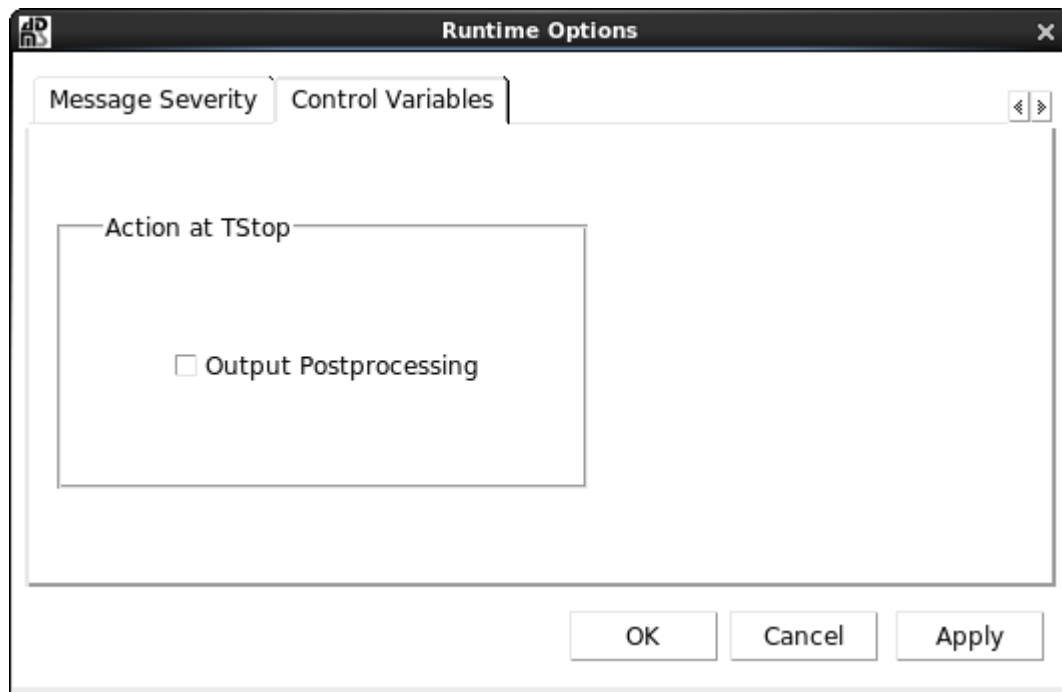


Figure 17-28: Runtime Options Dialog Box—Control Variables Tab

**Note**

The Runtime Options dialog box changes settings for the current simulation only.

Fields

Table 17-23: Runtime Options Dialog Box Fields

Field	Description
Message Severity Tab	
Break Severity	The severity level that will stop simulation. By default, a severity level of Failure causes a simulation break. To change this default permanently, edit the BreakOnAssertion variable in the <i>modelsim.ini</i> file. Refer to BreakOnAssertion in the <i>Questa SIM User's Manual</i> for more information.
No Message Display For VHDL/Verilog	Disables message logging at certain levels of severity. Multiple selections are possible.
Control Variables Tab	
Output Postprocessing	Execute the output postprocessing command when the simulation reaches TStop. To enable this option permanently, set the DisplayOutputPostprocessingAtTStop variable to 1 in the <i>modelsim.ini</i> file.

Usage Notes

Assertions that appear within an instantiation or configuration port map clause conversion function will not stop the simulation regardless of the severity level of the assertion.

Related Topics

[Interactive Debug](#)

[System Initialization File \(modelsim.ini\)](#)

Source Window

To access, do one of the following:

- Double-click on an instance in the [Structure Window](#)
- Right-click on an instance in the Structure Window and choose **View Declaration**
- With an instance selected in the Structure Window, choose **Structure > View Declaration**
- Use the command:

```
view source
```

Used to view, edit, compile, and simulate VHDL-AMS, Verilog-AMS, SPICE, VHDL or Verilog source code.

Description

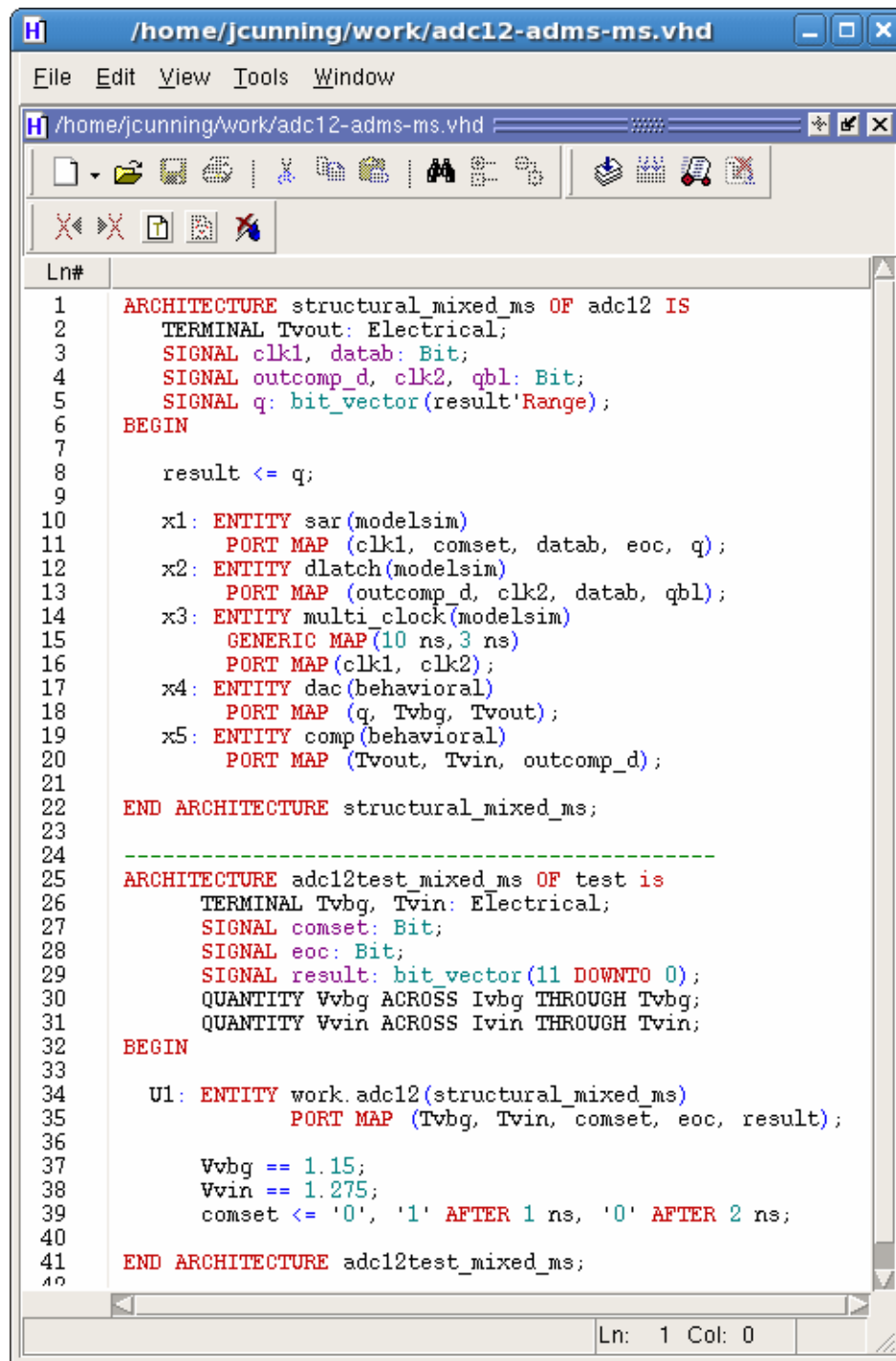
You can access the *.cmd* file in the Source window, by selecting it in the [Structure Window](#) or using the menu selection: **File > Open**.

Any Eldo subcircuit design or *.cir* file can be viewed as any VHDL-AMS design entity only by selecting the corresponding instance in the [Structure Window](#).

By default, the Source window displays your source code with line numbers. You may also see the following graphic elements:

- Red line numbers — denotes executable lines, where you can set a breakpoint.
- Blue text — denotes executable lines where you cannot set a breakpoint.
- Blue arrow—denotes the currently active line or a process that you have selected in the [Processes Window](#).
- Red ball in line number column — denotes file-line breakpoints; gray ball denotes breakpoints that are currently disabled.
- Blue flag in line number column — denotes line bookmarks
- Language Templates pane — displays templates for writing HDL code. Refer to [Accessing Language Templates](#) for more information.

Figure 17-29: Source Window



Objects

- Open
Select a source file to open.

- **Show Line Numbers**
Shows/Hides the line numbers next to the lines of code.
- **Show Language Templates**
Shows/Hides the language template in the left hand tab of the Source window.
- **Show Source Annotation**
Only available for digital source files.
- **Read Only**
Sets the source file to read-only mode. To edit a source file, make sure this option is not set.

Usage Notes

The Source window includes an search feature that enables you to perform an incremental search for specific code. To activate the search bar, choose **Edit > Find** from the menus or click the Find icon in the [Standard Toolbar](#).

The appearance of the Source window can be customized, see [Changing GUI Preferences](#) for details.

Related Topics

[Recompiling Design Units in the Source Window](#)

[Examining Design Files in the Source Window](#)

[Setting File-Line Breakpoints](#)

[view \[Questa ADMS Command Reference\]](#)

Start Simulation Dialog Box

To access, do one of the following:

- Start the simulator using the **vasim** command without specifying a design name as an argument.
- Choose **Simulate > Start Simulation...** without a design loaded.
- With the **Structure Window** active, choose **File > Open** or click the  icon.

Used to specify settings and options before a simulation.

Description

The controls on this dialog box are organized across five tabs:

- [Figure 17-30](#)
- [Figure 17-31](#)
- [Figure 17-32](#)
- [Libraries Tab](#)
- [Figure 17-33](#)

Design Tab

Used to select the required compiled design units and command file for the current simulation.

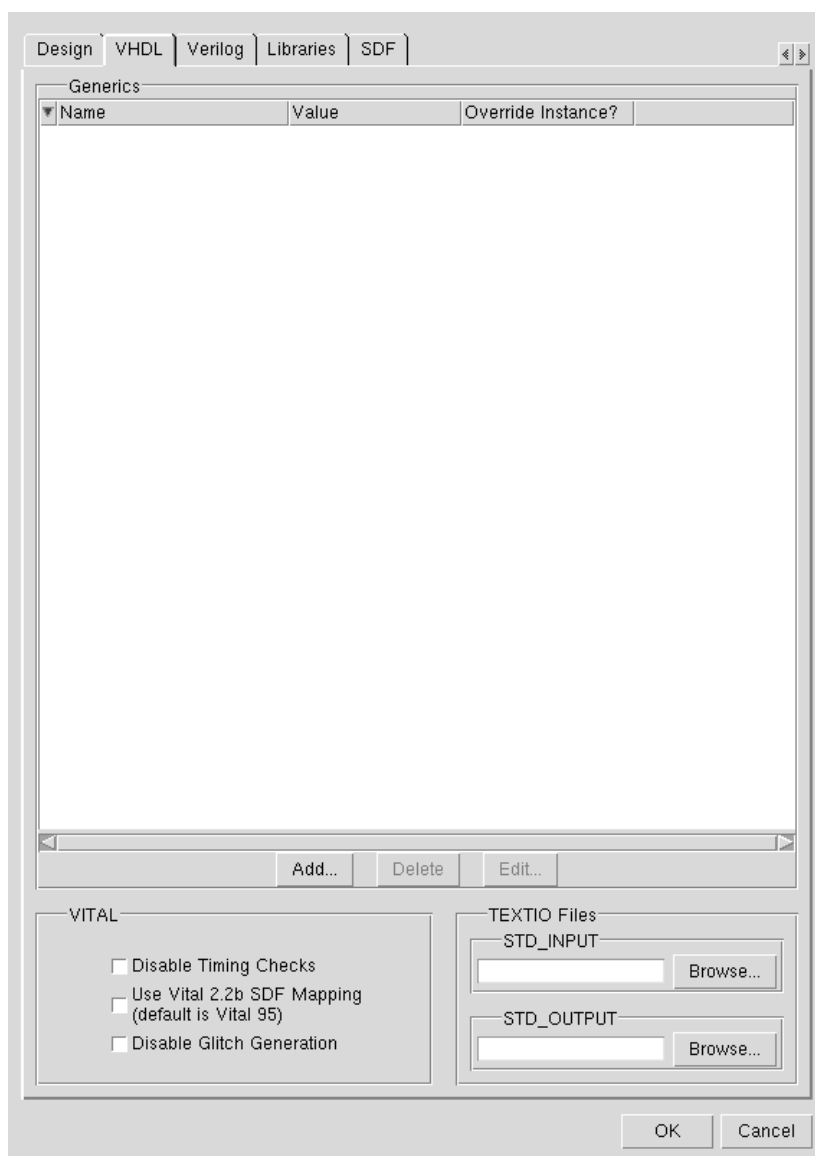
Figure 17-30: Start Simulation Dialog Box—Design Tab



VHDL Tab

Used to specify VHDL-specific options. To access, click the **VHDL** tab on the [Start Simulation Dialog Box](#).

Figure 17-31: Start Simulation Dialog Box—VHDL Tab



Verilog Tab

Used to specify Verilog-specific options for the current design being loaded. To access, click the **Verilog** tab on the [Start Simulation Dialog Box](#).

Figure 17-32: Start Simulation Dialog Box—Verilog Tab

Design | VHDL | **Verilog** | Libraries | SDF

Delay Selection

☐ min ☒ typ ☐ max

Pulse Options

☐ Disable pulse error and warning messages (+no_pulse_msg)

Rejection Limit % (+pulse_r)

Error Limit % (+pulse_e)

Other Options

☐ Enable Hazard Checking (-hazards)

☐ Disable Timing Checks in Specify Blocks (+notimingchecks)

User Defined Arguments (+<plusarg>)

OK Cancel

Libraries Tab

Used to specify libraries in which to search for the design units to be simulated. To access, click the **Libraries** tab on the [Start Simulation Dialog Box](#).

SDF Tab

Used to specify Standard Delay Format file timing data to be applied to regions for the simulation. To access, click the **SDF** tab on the [Start Simulation Dialog Box](#).

Figure 17-33: Start Simulation Dialog Box—SDF Tab



Fields

Table 17-24: Design Tab Contents

Name	Description
Top Design	Specifies the top of the design: • Compiled-HDL for VHDL-AMS, Verilog-AMS, VHDL or Verilog • Spice for SPICE-on-top designs
Incremental saving	Loads incremental saving mode when simulating the design. Incremental saving mode is still available following a restart, but if a new design is loaded, this option will be disabled. This equivalent to launching Questa ADMS using vasim with the option -isaving. Refer to Incremental Saving Mode for more information.

Table 17-24: Design Tab Contents (continued)

Name	Description
Reuse EZwave configurations	Specifies that Solido Waveform Analyzer configurations will be reused following a restart when the PrefReuse(GuiConfig) variable (see GUI Preference Variables) is set to Yes. If it is set to No, enabling this option will allow waves shown for the previous run to be displayed, but any changes to the settings (such as cursors or zooms) are lost. Waves are displayed if the added wave symbol appears against the wave name in the Restart Dialog Box .
Outpath	Specifies a location to which the results of the simulation will be saved, overriding the default (the current working library). This is equivalent to launching Questa ADMS using vasim with the option -outpath. Either enter a path directly into the field or click Browse and navigate to a directory.
Library	Specifies a library from which to choose a design unit to load. When the Top Design selection is Compiled HDL, the contents of the selected library are displayed in the Design Unit Selection field.
Design Unit Selection	Lists the contents of the library specified in the Library field. Used to select one top-level entity or configuration to be simulated. Available only when the Top Design selection is Compiled HDL.
Command File/ Eldo Design	Specifies the command file(.cmd) or netlist file (.cir) which contains the required simulation commands. When the top of the design is Eldo, this section is labeled Eldo Design. When the top of the design is Compiled HDL, it is labeled Command File. You can filter this field, as described in Design Tab.
Edit	Opens the selected command file for editing in a text editor.
New	Allows a new command file to be created and configured using the Eldo Commands Dialog Box .
User QuestaSim Arguments	Questa SIM options can be specified in this field.

Table 17-25: VHDL Tab Contents

Name	Description
Generics	Lists all generic parameters specified for the current simulation. Three options are available beneath the list: • Add - opens a dialog box to specify the name and value(s) of a generic parameter. • Delete - removes the selected generic from the list.

Table 17-25: VHDL Tab Contents (continued)

Name	Description
	Edit - opens a dialog box with the selected generic's properties displayed for editing.
Vital	
Disable Timing Checks (+notimingchecks)	Disables timing checks generated by VITAL models.
Use Vital 2.2b SDF Mapping (-vital2.2b)	Selects SDF mapping for VITAL 2.2b (default is Vital95).
Disable Glitch Generation (-noglitch)	Disables VITAL glitch generation.
TEXTIO files	
STD_INPUT (-std_input <filename>)	Specifies the file to use for the VHDL textio STD_INPUT file. Use the Browse button to locate a file within your directories.
STD_OUTPUT (-std_output <filename>)	Specifies the file to use for the VHDL textio STD_OUTPUT file. Use the Browse button to locate a file within your directories.

Table 17-26: Verilog Tab Contents

Name	Description
Delay Selection	Select the delay timing method to use from: min (+mindelays), typ (+typdelays) and max (+maxdelays).
Pulse Options	
Disable pulse error and warning messages (+no_pulse_msg)	Disables path pulse error warning messages.
Rejection Limit (+pulse_r)	Sets module path pulse rejection limit as a percentage of path delay. +pulse_r/<percent>
Error Limit (+pulse_e)	Sets module path pulse error limit as percentage of path delay. +pulse_e/<percent>
Other Options	
Enable Hazard Checking (-hazards)	Enables hazard checking in Verilog modules.

Table 17-26: Verilog Tab Contents (continued)

Name	Description
Disable Timing Checks in Specify Blocks (+notimingchecks)	Disables the timing check system tasks (\$setup, \$hold,...) in specify blocks.
User Defined Arguments (+<plusarg>)	Specifies user defined arguments. Arguments must be preceded with "+", so that they are accessible by the Verilog PLI routine mc_scan_plusargs.

Table 17-27: Libraries Tab Contents

Name	Description
Search Libraries (-L)	Specifies the libraries in which Questa ADMS should look for design units instantiated from Verilog.
Search Libraries First (-Lf)	As Search Libraries, but these libraries are searched before 'uselib.
Add	Opens the Select Library dialog box, from which you can select a library to add to the list.
Modify	Used to edit a library selected in the list.
Delete	Used to delete a library selected in the list.

Table 17-28: Start Simulation Dialog Box - SDF Tab

Name	Description
SDF Files	Lists selected SDF Files, the region to which it applies and the delay type selected.
Add	Opens a dialog box from which you can select an SDF file to add to the list and select a region and delay type.
Delete	Used to delete a file association selected in the list.
Modify	Used to modify a file association selected in the list.
SDF Options	
Disable SDF warnings	-sdfnowarn Select to disable warnings from the SDF reader.
Reduce SDF errors to warnings	-sdfnoerror

Table 17-28: Start Simulation Dialog Box - SDF Tab (continued)

Name	Description
	Change SDF errors to warnings so that the simulation can continue if Questa ADMS encounters an error during SDF backannotation.
Multi-Source Delay	-multisource_delay <sdf_option> Select a delay type from max, min or latest. Controls how multiple PORT or INTERCONNECT constructs that terminate at the same port are handled. By default, the Module Input Port Delay (MIPD) is set to the latest value encountered in the SDF file. Alternatively, you may choose the min or max of the values.

Usage Notes

Design Tab

Dynamic filters are available to help you find the required command file. The filters available are as follows:

- *.cmd (default)
- *.cir
- *<entity>*
- *<entity>*<architecture>*
- *<architecture>*<entity>*

These filters help you to find the command file associated with the design unit selected (you only have to list the command files with the same names as the design entity being simulated). These filters are only appropriate if you already selected a design unit in the previous Design Unit Selection area.

VHDL Tab

Click **Add** on the VHDL Tab to define a new generic parameter.

- Generic Name -g <Name=Value>

The name of the generic parameter. Enter it as it appears in the VHDL source (case is ignored).

- Value

Specifies a value for all generics in the design with the given name (above) that have not received explicit values in generic maps (such as top-level generics and generics that would otherwise receive their default value). Value is an appropriate value for the declared data type of the generic parameter. No spaces are allowed in the specification (except within quotes) when specifying a string value.

- Override Instance-specific Values -G <Name=Value>

Select to override generics that received explicit values in generic maps. The name and value are specified as above. The use of this switch is indicated in the Override Instance column of the Generics list.

SDF Tab

Click **Add** to specify a Standard Delay Format file from which to extract timing data for the simulation.

- SDF File [<region>] = <sdf_filename>)

Specifies the SDF file to use for annotation. Use the **Browse** button to locate a file within your directories.

- Apply to region

Specifies the design region to use with the selected SDF options.

- Delay Selection (-sdfmin | -sdftyp | -sdfmax)

Dropdown menu selects delay timing (min, typ or max) to be used from the specified SDF file.

Related Topics

[Running an Interactive Simulation](#)

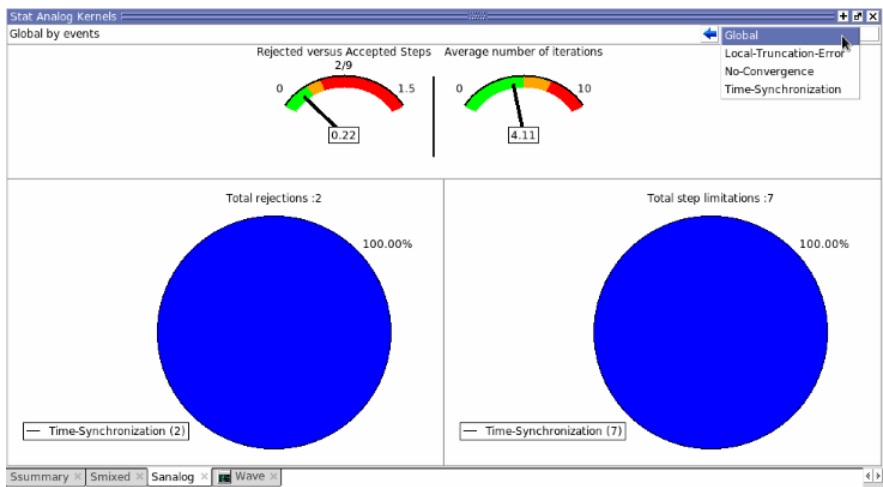
[Design Libraries](#)

Stat Analog Kernel Window

To access: Choose **View > Stat > Analog Kernels**, or use the command `view sanalog`.

Displays statistical information about analog kernel activity.

Figure 17-34: Stat Analog Kernels Window



Note

The contents of this window are updated each time the simulation pauses.

Objects

Table 17-29: Stat Analog Kernel Window Contents

Field	Description
Rejected vs Accepted Steps	The ratio between rejected and accepted steps. Value ranges from 0 to infinite. The counter display colors indicate the value as follows: - Green - below 1/3 - Orange - between 1/3 and 1/2 - Red - greater than 1/2
Average number of iterations	The average number of newton iterations per step. Value ranges from 0 to infinite. The counter display colors indicate the value as follows: - Green - below 5 - Orange - between 5 and 7

Table 17-29: Stat Analog Kernel Window Contents (continued)

Field	Description
	Red - greater than 7
Matrix sparsity (%)	For designs with more than 1000 nodes, a counter showing the matrix density level is visible. The counter display colors indicate the value. For designs with 100 to 10000 nodes: - Green - below 10% - Red - greater than 10% For designs with more than 10000 nodes: - Green - below 99% - Red - greater than 99%
Total Rejections	A graph showing the distribution of local truncation error, no convergence and time synchronization errors in all rejections.
Total Step Limitations	A graph showing the distribution of local truncation error, no convergence and time synchronization in all step limitations.

Usage Notes

Use the button to filter the results by the error type: Local Truncation Error, No Convergence or Time Synchronization.

Related Topics

[Viewing Statistics in Interactive Mode](#)

[Statistics File](#)

[Simulation Statistics](#)

[Stat Mixed-Signal Activity Window](#)

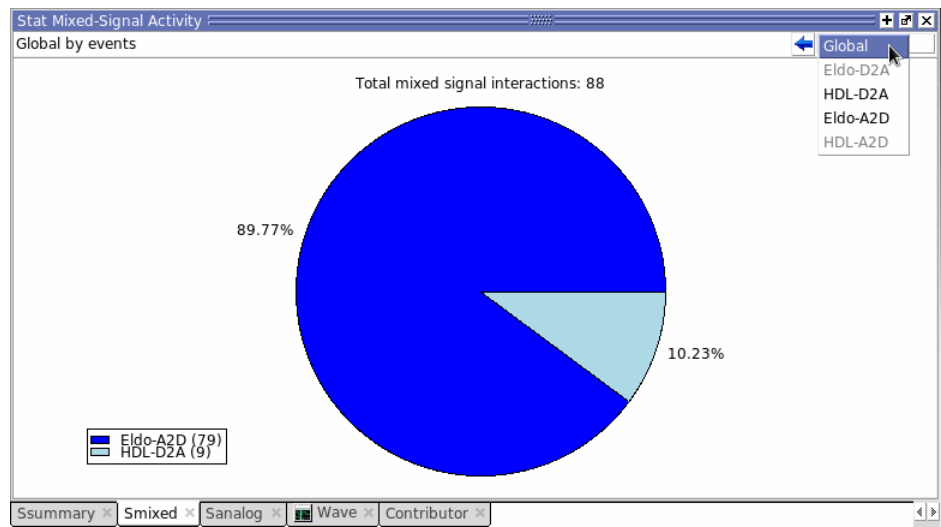
[Stat Summary Window](#)

Stat Mixed-Signal Activity Window

To access: Choose **View > Stat > Stat Mixed-Signal Activity** or use the command: view smixed.

Displays statistical information about interactions between analog and digital.

Figure 17-35: Stat Mixed-Signal Activity Window



Note
The absolute values are shown in the key, and the total number of interactions is shown in the graph legend. The contents of this window are updated each time the simulation pauses.

Objects

Table 17-30: Stat Mixed-Signal Activity Window Contents

Field	Description
Global	Displays a graph showing the distribution of the events that cause mixed-signal interactions.
Eldo-D2A	Displays a graph showing the distribution of Eldo D2A Built-in boundary elements.
HDL-D2A	Displays a graph showing the distribution of HDL model break statements
Eldo-A2D	Displays a graph showing the distribution of Eldo A2D Built-in boundary elements.
HDL-A2D	Displays a graph showing the distribution of VHDL-AMS Q'above and Verilog-AMS @Cross statements.
	Used to return to the global chart when a node-related chart is being viewed.

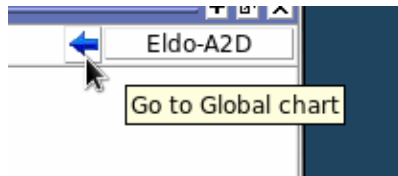
Usage Notes

Click on a category in the global chart to display a node-related chart, showing the distribution of the 10 most active nodes in the design (regarding events).

In the node-related chart, click on a node name to select it as the current region. It will be highlighted in the Objects window.

To return to the global chart, select the blue arrow next to the graph selection button.

Figure 17-36: Go to Global Chart Button



When hierarchical node names are too long, they are truncated in the display. Hover over a name to view the full name.

Right-click on a node name and choose **View Declaration** to view the source in the Source window.

Related Topics

[Viewing Statistics in Interactive Mode](#)

[Statistics File](#)

[Simulation Statistics](#)

Stat Summary Window

To access: Choose **View > Stat > Stat Summary**, or use the command: view summary

Provides a summary of statistical elaboration and simulation information, in graphical format.

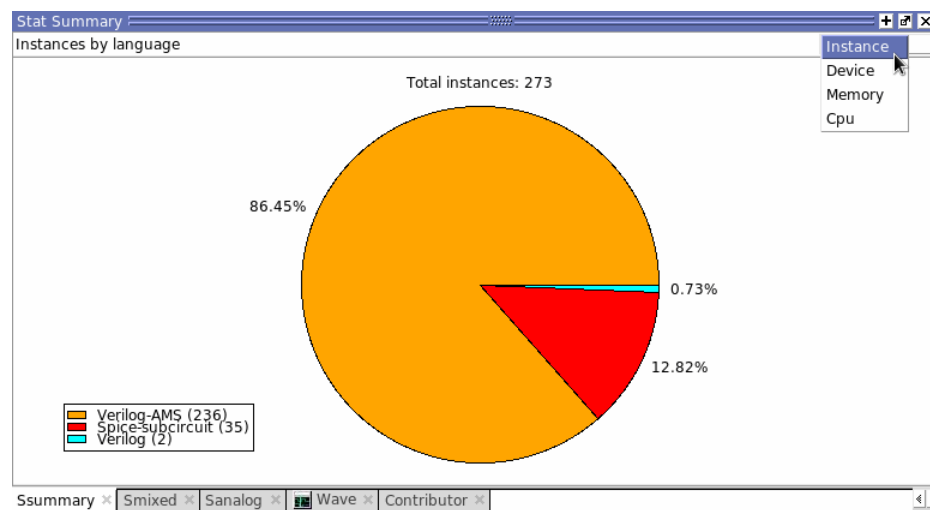
Description

Click the label in the top right corner to switch between graphs, using the dropdown list.

- Instances by language
- Distribution of SPICE devices by kind
- Distribution of memory allocation
- Cumulative CPU time by phase: elaboration, DC, AC, TRAN

The absolute values are shown in the key, and the total number of instances and devices is shown in the graph legend. The contents of this window are updated each time the simulation pauses.

Figure 17-37: Stat Summary Window



Objects

Table 17-31: Stat Summary Window Contents

Field	Description
Instance	Displays a graph showing a breakdown by language of the total number of instances in the design.
Device	Displays a graph showing a breakdown by device type, of the total number of devices in the design: • Resistors • Capacitors • Grounded Capacitors • Voltage Sources • HDL Devices
Memory	Displays a graph showing a breakdown of memory allocation to the amount of memory (in kilobytes) used by the following: • The Questa ADMS kernel (including Eldo and Questa SIM) during each phase of the simulation run • Questa ADMS GUI during each phase of the simulation run • Solido Waveform Analyzer and the JWDB server during the elaboration phase
Cpu	Displays the total cumulative CPU time by phase (elaboration, DC, AC, TRAN)

Related Topics

[Simulation Statistics](#)

[Statistics File](#)

[Viewing Statistics in Interactive Mode](#)

[Stat Mixed-Signal Activity Window](#)

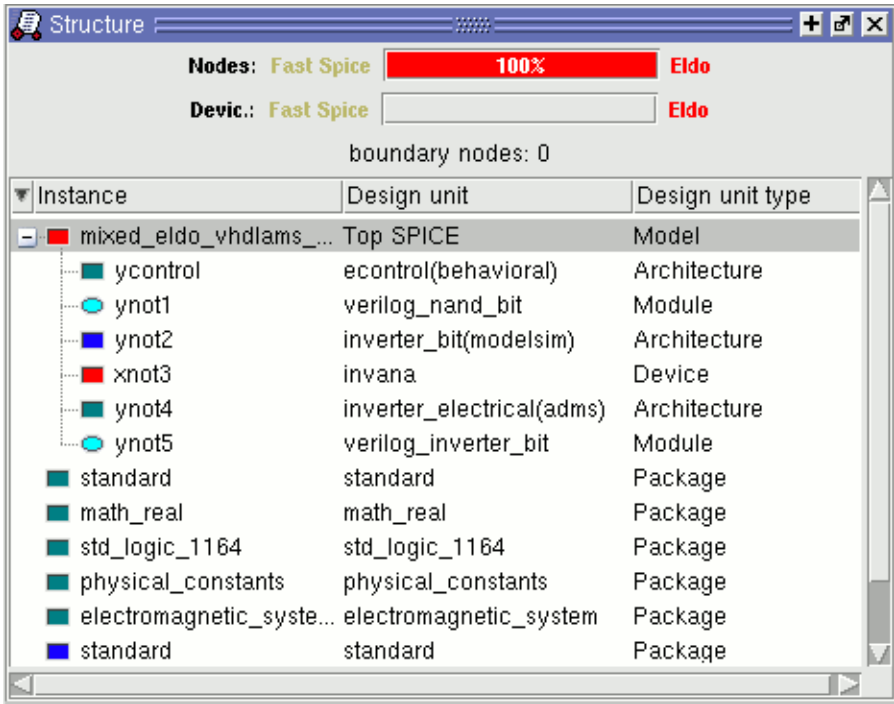
[Stat Analog Kernel Window](#)

Structure Window

To access: Choose **View > Structure** or use the command: view structure

Shows a hierarchical view of the design entities in the active simulation. This includes all regions in the current design, as well as the sub-instances.

Figure 17-38: Structure Window



Note
When a region is selected, the other windows are automatically updated to display the corresponding content.

Characteristics

Table 17-32: Structure Window Contents

Name	Description
Partitioning	
Nodes	Displays the percentage of nodes partitioned for ADiT (Fast SPICE) and Eldo.
Devices	Displays the percentage of devices partitioned for ADiT (Fast SPICE) and Eldo.

Table 17-32: Structure Window Contents (continued)

Name	Description
Boundary Nodes	Displays the number of boundary elements in the design.
Structure List	
Instance	Lists all elements in the design, structured hierarchically and ordered, by default, by name. For a key to the colors and symbols in the hierarchies, refer to GUI Icons and Their Meanings .
Design Unit	Displays the design unit from which the element is instantiated.
Design Unit Type	Displays the category of design unit, such as Module, for example.

Usage Notes

Selecting a region in the Structure window is equivalent to setting a region with the [env](#) command. the selected region becomes the current region, and the [Objects Window](#), [Source Window](#) and [Locals Window](#) are updated to display the information for that region. The [Processes Window](#) is also updated; the Processes window will in turn update the [Locals Window](#) if a process is selected.

By default, the Structure window suppresses the display of implicit wire processes. To enable this, set the variable PrefMain(HideImplicitWires) to 0 either using the [Preferences Dialog Box](#), or by using the set command in the [Transcript Window](#). On the Preferences dialog box, this option can be found by expanding the Main object on the By Name tab.

The design structure can remain hidden if required; see the -nodebug option of the [vacom](#) command in and the Questa SIM vcom command.

Note

Packages that are referenced by both Questa SIM (VHDL) and Questa ADMS (VHDL-AMS) will appear twice in the Structure Window.

Instance Names in the Structure Window

Instance names displayed in the [Structure Window](#) consist of the following parts:

- VHDL-AMS or VHDL instance names:

```
instantiation_label : entity_name ( architecture_name )
```

instantiation_label indicates the label assigned to the instance in the instantiation statement, entity_name indicates the name of the entity that has been instantiated, and architecture_name indicates the name of the architecture associated with the entity.

- Verilog-AMS Instance Names

```
instantiation_label : module_name
```

instantiation_label indicates the label assigned to the instance in the instantiation statement and module_name indicates the name of the Verilog-AMS module that has been instantiated.

- Verilog Instance Names:

```
instantiation_label : module_name
```

instantiation_label indicates the label assigned to the instance in the instantiation statement and module_name indicates the name of the Verilog module that has been instantiated.

- SPICE Instance Names

```
instantiation_label : subckt_name
```

instantiation_label indicates the label assigned to the instance in the instantiation statement and subckt_name indicates the name of the SPICE subcircuit that has been instantiated.

Note

The Structure window always displays instance names in lower case; this is graphical only.

Related Topics

[Running an Interactive Simulation](#)

[GUI Icons and Their Meanings](#)

[Filtering Objects by Type in the Structure Window](#)

Transcript Window

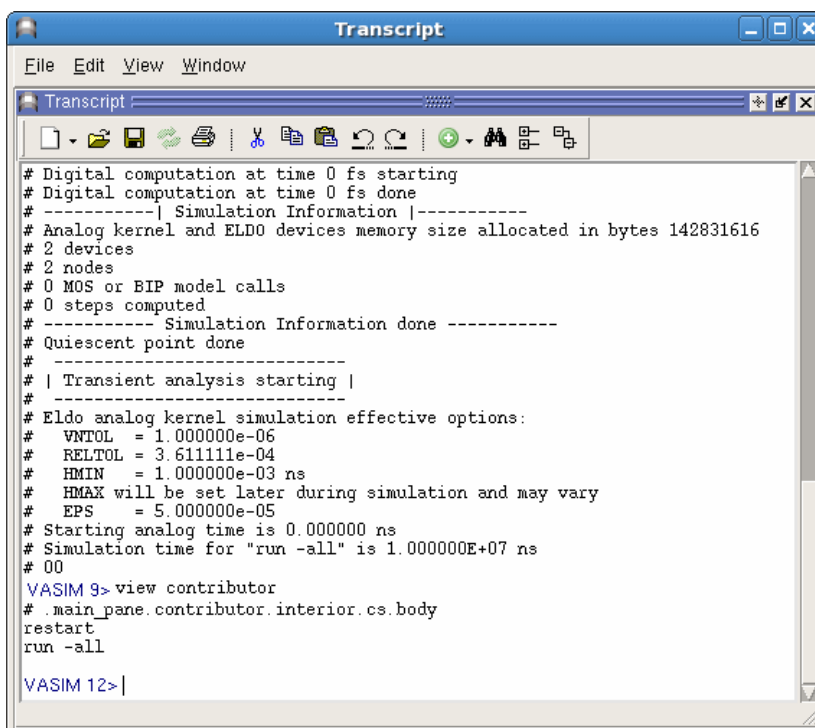
To access: Choose **View > Transcript**.

Maintains a running history of commands that are invoked and messages that occur as you work with Questa ADMS. Displayed automatically when Quest ADMS is launched.

Description

Maintains a running history of commands that are invoked and messages that occur as you work with Questa ADMS. When a simulation is running, the Transcript displays a VASIM prompt, allowing you to enter command line commands from within the graphic interface.

Figure 17-39: Transcript Window



Objects

All options are offered as sub-selections of the Transcript menu option, available when the Transcript window is active.

Table 17-33: Transcript Window Menu Contents

Name	Description
Adjust Font Scaling...	Controls how fonts appear for your display environment.
Transcript File	The name used when saving the transcript file, which contains all the text in the current transcript file. The default is <i>transcript</i> .
Command History...	Enables you to specify the default name used when saving command history information. This file is saved at the same time as the transcript file.
Save File...	Enables you to specify the default name used when selecting File > Save transcript As .
Saved Lines...	The number of lines of text saved in the Transcript window. Setting this value to zero (0) saves all lines. The default is 5000.
Line Prefix...	The character(s) that precedes the lines in the transcript. The default is # .
Update Rate...	The length of time, in ms, between transcript refreshes. The default is 797.
Questa ADMS Prompt...	The string used for the command line prompt. The default is "VASIM>".
VASIM Prompt...	The string used for the simulation prompt. The default is "VASIM [history nextid]>".
Paused Prompt...	The string used for when the simulation is paused. The default is "VASIM (paused)>".
Drag and Drop Preferences	Controls drag and drop behavior when a file of type "File Type" is dragged from a file explorer window to a "Location" window. • Location — Project, Transcript or Wave • File Type — Select the file type • Action — Open, Execute, Add to Project, Custom • Custom — Enables you to enter a custom action

Usage Notes

The simulator variable `VerboseMode` controls the number of notes and messages that are written to the Transcript window. When `VerboseMode=1`, all notes and messages are displayed in the Transcript window. When `VerboseMode=0` (the default setting) only important messages are displayed. Warnings and error messages are always displayed. Refer to [Simulator Control Variables](#) for information on setting variables.

Related Topics

[Running Commands from the Transcript Window](#)

[Manually Saving the Transcript File](#)

[vasim \[Questa ADMS Command Reference\]](#)

Wave Window (SOLIDIS Waveform Analyzer)

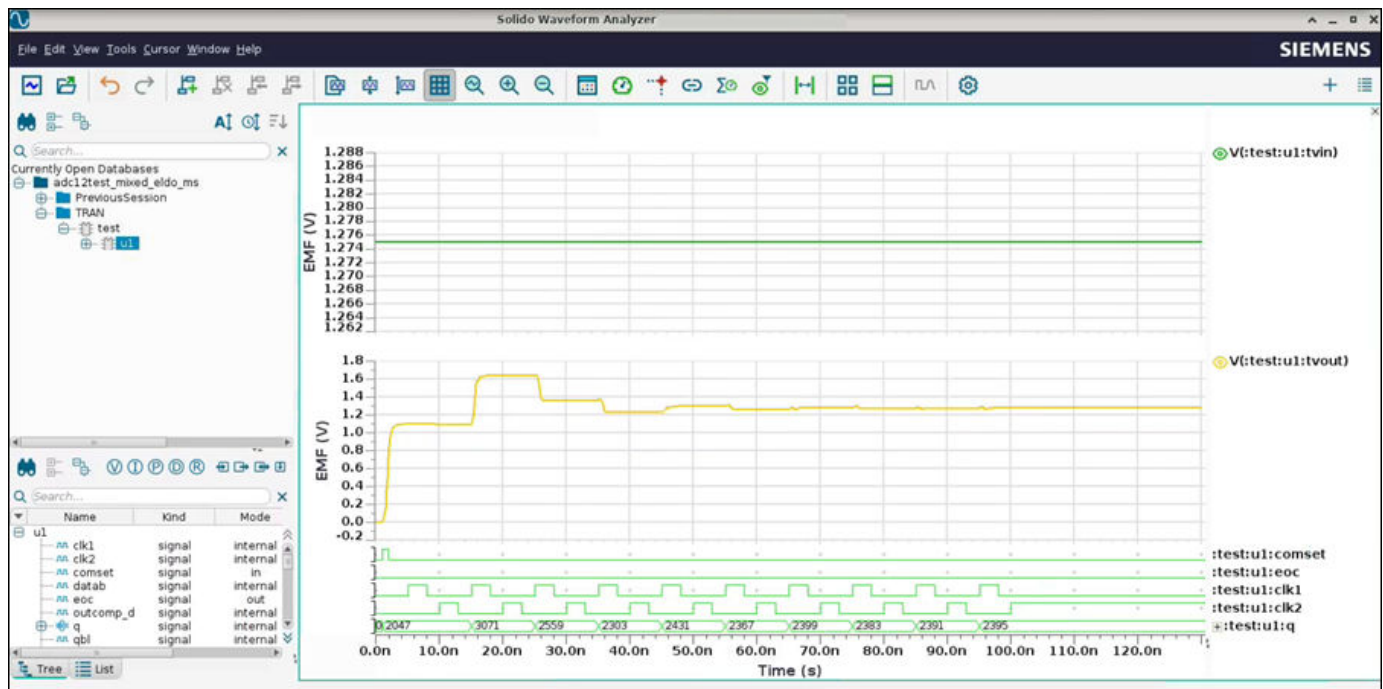
To access: Choose **View > Wave > Questa ADMS** or enter the following command into the **Transcript Window**: view wave

Questa ADMS integrates the Solido Waveform Analyzer wave viewer, which is used to view the results of your simulation as waveforms.

Description

Analog and digital waves can be viewed at the same time, as shown in **Figure 17-40**. Results are saved into Joint Waveform Database Format (JWDB), with the filename extension *.wdb*. You can save the database along with your graph windows at any time for later viewing. Displayed waveforms can also be printed locally, and exported to PDF.

Figure 17-40: Solido Waveform Analyzer Wave Window



The Solido Waveform Analyzer viewer uses an advanced graphical user interface that supports viewing multiple waveforms and databases through:

- A **Waveform List** displaying the database in either a hierarchical (tree) format or flat (list) format. To access, choose the Solido Waveform Analyzer menu item **View > Waveform List**.
- The ability to add waveforms to multiple graph windows.
- A tabbed workspace for organizing windows.

By default, the Wave window is not docked in the Questa ADMS main application window. When the variable `DockEZwaveInQuestaAdms` in the `modelsim.ini` file is set to 1, the Wave window is docked in the Questa ADMS main application window.

Objects

Refer to [Solido Waveform Analyzer Viewer Interface](#) in *Solido Waveform Analyzer User's and Reference Manual*.

Usage Notes

Digital waves are not always dependent on time, for example, in a DC sweep they can be dependent on temperature.

Units can be viewed in Solido Waveform Analyzer. These can be declared using the VHDL-AMS `SYMBOL` attribute in the package that declares the nature (e.g. `IEEE.ELECTRICAL_SYSTEMS`), or the Verilog-AMS nature attribute units (e.g. `disciplines.vams`).

Waveform List

The Waveform List resides on the left-hand side of the Solido Waveform Analyzer window and displays all of the currently open databases and the individual waveforms contained within them.

When the simulator setup variable `AutoSaveInPreviousSession` (see [Simulator Control Variables](#)) in the `modelsim.ini` file is set to *yes*, the folder called `PreviousSession` is shown in the Waveform List. This folder contains the waveform data from the previous simulation. This enables you to compare the simulation results from the previous session with the simulation results from the current session. By default, no data from a previous run is saved, meaning no `PreviousSession` folder is created in the JWDB file.

By default the display of Solido Waveform Analyzer Waveform List is controlled within the Solido Waveform Analyzer settings. To hide the Solido Waveform Analyzer Waveform List within the Questa ADMS preferences, set `HideEZwaveWaveformList` to 1.

Related Topics

[Adding Signals to the Solido Waveform Analyzer Wave Window](#)
[System Initialization File \(modelsim.ini\)](#)
[.ez.do File Limitations](#)
[Solido Waveform Analyzer Error Handling](#)

Toolbars

The main window contains a toolbar frame that displays context-specific toolbars.

The following sections describe the toolbars and their associated buttons.

Standard Toolbar..... 17-83

Compile Toolbar..... 17-85

Process Toolbar..... 17-86

Simulate Toolbar..... 17-87

Source Toolbar..... 17-89

AC Tool Toolbar..... 17-90

Standard Toolbar

To access: Choose **Window > Toolbars > Standard**

Offers common functions that apply to most windows.

Objects

Table 17-34: Standard Toolbar Buttons

Button	Name	Shortcuts	Description
	New File	Menu: File > New > Source	Opens a new Source text file
	Open/Start Simulation	Menu: File > Open	
	Save	Menu: File > Save	Saves the contents of the active window or, saves the current wave window display and signal preferences to a macro file (DO file).
	Print	Menu: File > Print	Opens the Print dialog box
	Cut	Menu: File > Cut Hotkey: CTRL-X	-
	Copy	Menu: File > Copy Hotkey: CTRL-C	-
	Paste	Menu: Edit > Paste Hotkey: CTRL-V	-
	Undo	Menu: Edit > Undo Hotkey: CTRL-Z	-
	Redo	Menu: Edit > Redo Hotkey: CTRL-Y	-
	Find	Menu: Edit > Find Hotkey: CTRL-S (UNIX) or CTRL-F (Windows)	Opens the Search bar.

Table 17-34: Standard Toolbar Buttons (continued)

Button	Name	Shortcuts	Description
	Collapse All	Menu: Edit > Expand > Collapse All	Collapses the entire hierarchy
	Expand All	Menu: Edit > Expand > Expand All	Expands the entire hierarchy

Compile Toolbar

To access: Choose **Window > Toolbars > Compile**.

Provides access to compile and simulation actions.

Objects

Table 17-35: Compile Toolbar Buttons

Button	Name	Shortcuts	Description
	Compile	Command: vcom or vlog Menu: Compile > Compile	Opens the Compile Source Files dialog box.
	Compile All	Command: vcom or vlog Menu: Compile > Compile All	Compiles all files in the open project. The library must be compiled with valib -type directory.
	Simulate	Command: vasim Menu: Simulate > Start Simulation	Opens the Start Simulation Dialog Box .
	Break	Menu: Simulate > Break Hotkey: Break	Stop a compilation, elaboration, or the current simulation run.

Process Toolbar

To access: available at the top of the **Processes Window** when undocked.

Sets the viewing mode of the Processes window.

Objects

Table 17-36: Process Toolbar Buttons

Button	Name	Shortcuts	Description
	View Active Processes	Menu: Process > Active	Only active processes are shown
	View Processes in Region	Menu: Process > In Region	Only show processes in the active region are shown
	View Processes for the Design	Menu: Process > Design	Processes in the design are shown
	View Processes Hierarchy	Menu: Process > Hierarchy	Process hierarchy is shown

Simulate Toolbar

This toolbar is not detachable.

Provides various tools for controlling your active simulation.

Objects

Table 17-37: Simulate Toolbar Buttons

Button	Name	Shortcuts	Description
	Compile	none	Not available
	Restart	Menu: Simulate > Run > Restart	Restarts the current simulation from time zero; see also restart command
	Run	Menu: Simulate > Run > Run	Equivalent to run command
	Run Length	none	Specifies length of the simulation Run, for: Run > Run , invoked from pull down menu; the run command, invoked in the Transcript Window ; the Run button, invoked from the toolbar
	Continue Run	Menu: Simulate > Run > Continue	run -continue
	Run All	Menu: Simulate > Run > Run -All	run -all
	Step	Menu: Simulate > Run > Step	step
	Step Over	Menu: Simulate > Run > Step-Over	step -over
	Break	Menu: Simulate > Break	For digital: run -next. For analog: run -nextASP
	QuestaSim GUI	none	Open/Close the Questa SIM GUI ¹ . Note that when you close the Questa SIM GUI, any undocked Questa SIM windows will remain visible. They can be closed by docking them.

¹ Note, this button acts as a toggle, but if you quit the Questa SIM GUI, using File > Quit, the Questa ADMS GUI will also close.

Table 17-37: Simulate Toolbar Buttons (continued)

Button	Name	Shortcuts	Description
			All the outputs of Questa SIM are directed to the Questa ADMS GUI.

Source Toolbar

To access: Choose **Window > Toolbars > Source**.

Offers options applicable to the Source window.

Objects

Table 17-38: Source Toolbar Buttons

Button	Name	Shortcuts	Description
	Previous Zero Hits	None	Jump to previous line with zero coverage.
	Next Zero Hits	None	Jump to next line with zero coverage.
	Show Language Templates	Menu: Source > Show Language Templates	Display language templates in the left hand side of every open source file.
	Source Annotation	Menu: Source > Show Annotation	Allows Debugging with Source Annotation in every open source file.
	Clear Bookmarks	Menu: Source > Clear Bookmarks	Removes any bookmarks in the active source file.

Related Topics

Source Window

AC Tool Toolbar

This toolbar is displayed at the top of the **Objects Window** in the case of an AC analysis or both AC and Transient analyses.

Controls the type of plot when performing an add wave or add log command.

Objects

Table 17-39: AC Tool Toolbar Buttons

Button	Description
V	The transient value of the specified nets will be waved or logged
dB	AC magnitude in DB of the specified analog nets will be waved or logged. Ignored for digital signals
M	AC magnitude of the specified analog nets will be waved or logged. Ignored for digital signals
ϕ	AC phase of the specified analog nets will be waved or logged. Ignored for digital signals
R	AC Real part of the specified analog nets will be waved or logged. Ignored for digital signals
I	AC Imaginary part of the specified analog nets will be waved or logged. Ignored for digital signals

The kind of wave or log can be selected differently for any net.

Note

When only a transient analysis is requested, the add wave or add log commands directly wave and log the transient values of the objects without the need of any toolbar.

Related Topics

[Questa ADMS GUI Overview](#)

A. Predefined VHDL-AMS Packages Supplied with Questa ADMS

This appendix describes the VHDL-AMS packages that are supplied with Questa ADMS.

Table A-1: Predefined VHDL-AMS Packages

Package Name	Description	Library
STANDARD	Basic VHDL-AMS standard package	STD
TEXTIO	Contains declarations of types and procedures that support I/O operations on TEXT files.	STD
MATH_REAL	VHDL declarations containing common real constants, common real functions, and real transcendental functions.	IEEE
MATH_COMPLEX	VHDL declarations containing common complex constants, common complex functions, and complex transcendental functions.	IEEE
STD_LOGIC_1164	Defines a standard used to describe the interconnection of data types in VHDL modeling.	IEEE
NUMERIC_BIT (from file <i>mti_numeric_bit.vhd</i>)	Defines numeric types (SIGNED and UNSIGNED numbers in vector form) and arithmetic functions for use with synthesis tools. The base element type is type BIT. The left-most bit is treated as the most significant bit. Signed vectors are represented in two's complement form. This package contains overloaded arithmetic operators on the SIGNED and UNSIGNED types. The package also contains useful type conversions functions, clock detection functions, and other utility functions.	IEEE
NUMERIC_STD (from file <i>mti_numeric_std.vhd</i>)	Defines numeric types (SIGNED and UNSIGNED numbers in vector form) and arithmetic functions for use with synthesis tools. The base element type is type STD_LOGIC. The left-most bit is treated as the most significant bit. Signed vectors are represented in two's complement form. This	IEEE

Table A-1: Predefined VHDL-AMS Packages (continued)

Package Name	Description	Library
	package contains overloaded arithmetic operators on the SIGNED and UNSIGNED types. The package also contains useful type conversions functions.	
STD_LOGIC_ARITH (from file <i>mti_std_logic_arith.vhd</i>)	A set of arithmetic, conversion, and comparison functions for SIGNED, UNSIGNED, SMALL_INT, INTEGER, STD_ULONGIC, STD_LOGIC, and STD_LOGIC_VECTOR.	IEEE
STD_LOGIC_MISC (from file <i>mti_std_logic_misc.vhd</i>)	Defines supplemental types, subtypes, constants, and functions for the STD_LOGIC_1164 package.	IEEE
STD_LOGIC_SIGNED (from file <i>mti_std_logic_signed.vhd</i>)	A set of signed arithmetic, conversion, and comparison functions for STD_LOGIC_VECTOR.	IEEE
STD_LOGIC_UNSIGNED (from file <i>mti_std_logic_unsigned.vhd</i>)	A set of unsigned arithmetic, conversion, and comparison functions for STD_LOGIC_VECTOR.	IEEE
STD_LOGIC_TEXTIO (from file <i>mti_std_logic_textio.vhd</i>)	This package overloads the procedures for READ and WRITE for use with STD ULOGIC, STD ULOGIC VECTOR, STD LOGIC VECTOR and for Hexadecimal and Octal values.	IEEE
VITAL_TIMING (from file <i>mti_timing_p_2000.vhd</i> or <i>mti_timing_p_2.2b.vhd</i>)	VITAL ASIC Modeling Specification package, containing VITAL timing objects.	IEEE
VITAL_PRIMITIVES (from file <i>mti_prmtvs_p_2000.vhd</i> or <i>mti_prmtvs_p_2.2b.vhd</i>)	Defines standard types, constants, functions and procedures for use in developing ASIC models. Specifically a set of logic primitives are defined.	IEEE
VITAL_MEMORY	VITAL ASIC Modeling Specification package, containing VITAL memory objects.	IEEE

Table A-1: Predefined VHDL-AMS Packages (continued)

Package Name	Description	Library
(from file <i>MTI_mti_memory_p_2000.vhd</i> or <i>mti_prmtvs_p_2.2b.vhd</i>)		
MATERIAL_CONSTANTS	Defines a set of physical constants without default values. The end-user defines the values, however, the names are standardized. According to the developers the rationale for this is that, for example, properties of materials are measured and subject to variation according the application context, environmental conditions, and assumptions of individual experiments. • PHYS_EPS_SI — Relative permittivity of silicon • PHYS_EPS_SIO2 — Relative permittivity of silicon dioxide • PHYS_E_SI — Young's Modulus for silicon (in Pascals) • PHYS_E_SIO2 — Young's Modulus for silicon dioxide (in Pascals) • PHYS_E_POLY — Young's Modulus for polysilicon (in Pascals) • PHYS_NU_SI — Poisson's Ratio for silicon <100-orientation> • PHYS_NU_POLY — Poisson's Ratio for polysilicon <100-orientation> • PHYS_RHO_POLY — Density of polysilicon • PHYS_RHO_SIO2 — Density of silicon-dioxide • AMBIENT_TEMPERATURE — Environmental constant • AMBIENT_PRESSURE — Environmental constant • AMBIENT_ILLUMINANCE — Environmental constant	IEEE_ENV

Table A-1: Predefined VHDL-AMS Packages (continued)

Package Name	Description	Library
STD_LOGIC_ARITH (from file <i>std_arit.vhd</i>)	This package allows synthesis of the 1164 package. It adds the capability of SIGNED/UNSIGNED math.	ARITHMETIC
ARITHMETIC (from file <i>syn_ari.vhd</i>)	A set of arithmetic, conversion, and comparison functions for SIGNED, UNSIGNED, and MVL7_VECTOR.	SYNOPSYS
TYPES (from file <i>syn_type.vhd</i>)	Defines the types, logic functions, truth tables, definitions for wired signals, and conversion functions for the Synopsys Standard Logic library.	SYNOPSYS
ATTRIBUTES (from file <i>syn_attributes.vhd</i>)	Defines the attributes associated with the Synopsys VHDL System Simulator and the HDL compiler. The simulator specific attributes are built into the analyzer, so this source should not be analyzed on the Synopsys VHDL System Simulator. It is provided for reference and portability to other systems.	SYNOPSYS
FUNDAMENTAL_CONSTANTS	Some physical constants	IEEE
ELECTRICAL_SYSTEMS	Electromagnetic nature definitions	IEEE
THERMAL_SYSTEMS	Thermal nature definition	IEEE
MECHANICAL_SYSTEMS	Kinematic and rotational nature definitions	IEEE
FLUIDIC_SYSTEMS	Fluidic nature definitions.	IEEE
ANALOG_START	Starts the analog solver computation	MGC_AMS
CHECKPOINT	Saves the state of the simulation	MGC_AMS
ASPDTECT	A signal that toggles at the end of each accepted analog solution point (ASP); each time step and DC point, but not AC.	MGC_AMS
CONVERSION	Time-to-Real and Real-to-Time type conversion functions	MGC_AMS
ELDO	Required to instantiate an Eldo subcircuit within a VHDL-AMS description	MGC_AMS
ELDO_PARAMETERS	Enables SPICE parameters to be used within a VHDL-AMS description	MGC_AMS

Table A-1: Predefined VHDL-AMS Packages (continued)

Package Name	Description	Library
LOOK_UP_TABLE_V1X	Enables the Verilog-AMS system function \$table to be accessed from within a VHDL-AMS description	MGC_AMS
MIXED_AD_UTIL	Enables the value of a terminal to be accessed from within a digital context.	MGC_AMS

TEXTIO Packages.....	A-5
LOOK_UP_TABLE_V1X.....	A-16

TEXTIO Packages

Enables human-readable text input from a declared source within a VHDL-AMS file during simulation.

This section demonstrates the basic use of the standard TEXTIO package from the STD library and the IEEE STD_LOGIC_TEXTIO package from the IEEE library.

Standard TEXTIO Package.....	A-5
File Declaration.....	A-6
Using STD_INPUT and STD_OUTPUT.....	A-6
Example of How to Make Output Data Available Before Simulation Ends.....	A-7
Writing Strings and Aggregates.....	A-8
IEEE STD_LOGIC_TEXTIO Package.....	A-10
Multiple Data Types.....	A-11
ASPDETECT.....	A-14
absdelta.....	A-15

Standard TEXTIO Package

This package contains declarations of types and procedures that support both input and output operations on TEXT files.

To access the routines in the TEXTIO package, the following syntax has to be added before the entity or architecture declaration in the VHDL-AMS code:

```
use STD.TEXTIO.all;
```

File Declaration

When a file is declared within an architecture, process, or package, the file is opened when you invoke the simulator and it is subsequently closed when you end the simulation.

Note

If you declare a file within a subprogram, the file is opened when you call the subprogram. The file is closed when the subprogram returns.

The syntax of a file declaration is:

```
file identifier_list : TEXT [file_open_information];
```

where file_open_information is:

```
file_open_information ::= [open file_open_kind_expression] is  
file_logical_name;  
type file_open_kind is (read_mode, write_mode, append_mode);
```

- **read_mode** — Resulting access mode is read-only.
- **write_mode** — Resulting access mode is write-only.
- **append_mode** — Resulting access mode is write-only; information is appended to the end of the existing file.

file_logical_name is:

```
file_logical_name ::= string_expression;
```

- **file_logical_name** — Must be an expression of the predefined type STRING.

Note

Only file declarations and files of type TEXT are supported by Questa ADMS.

Using STD_INPUT and STD_OUTPUT

The TEXTIO package contains two file declarations.

The two file declarations are:

```
file input
file output
```

These can be defined in the following way:

```
file input : TEXT open read_mode is "STD_INPUT";
file output : TEXT open write_mode is "STD_OUTPUT";
```

- STD_INPUT

This is a file_logical_name that refers to characters that are entered interactively from the keyboard and not from a file.

- STD_OUTPUT

This is a file_logical_name that refers to text that is displayed on the screen and not written to a file.

Example of How to Make Output Data Available Before Simulation Ends

You can use macros to make data available before simulation ends.

The following example shows how you can use the file_open and file_close macros to make data available before simulation ends:

```
LIBRARY ieee, std;
use std.textio.all;
use ieee.std_logic_1164.all;
use ieee.std_logic_textio.all;

entity simple_example is
port (input : string (5 downto 1));
end entity simple_example;

architecture adms OF simple_example is
file F : text;                                --declaration of F
begin

process (input) is
variable L : line;
begin

file_open(F, "output.dat", write_mode);      --open F in write mode

if input'event then
write(L, string'("Input new value is : "));
write(L, input);
writeline(F, L);
```

```

end if;

file_close(F);           --close F
end process;
end architecture adms;

```

Compare the above with the following example, where the data only becomes available when you have exited from the simulation:

```

library ieee, std;
use std.textio.all;
use ieee.std_logic_1164.all;
use ieee.std_logic_textio.all;

entity simple_example is
port (input : string (5 downto 1));
end entity simple_example;

architecture adms of simple_example is
begin

process (input) is
file F : text open write_mode is "output.dat"; --single line declaration
variable L : line;
begin

if input'event then
write(L, string("Input new value is : "));
write(L, input);
writeline(F, L);
end if;

end process;
end architecture adms;

```

Writing Strings and Aggregates

A common error in VHDL-AMS source code occurs when a WRITE procedure is called and the argument is of type STRING or BIT_VECTOR but the type is not declared. In the standard TEXTIO package, the WRITE procedure is an additional procedure for the types STRING and BIT_VECTOR.

If the following syntax is compiled, an error will occur:

```
WRITE(L, "0101")
```

The error produced is:

```
[Error] Procedure call WRITE is ambiguous;
```

The error is produced because a type has not been declared. To overcome the error, call the procedure using one of the following:

```
WRITE(L,string'("0101"));
```

or

```
WRITE(L,bit_vector'("0101"));
```

Example

The following example reads the text from the *input.dat* file and writes it to the *output.dat* file:

```
entity simple_textio is
end entity simple_textio;

use STD.TEXTIO.all;

architecture adms of simple_textio is
begin
    copy: process
        file copy_input  : text open read_mode is "input.dat";
        file copy_output : text open write_mode is "output.dat";
        variable test_line : line;

        begin
            -- copy the contents of the input file into the output file
            while not endfile(copy_input) loop
                readline(copy_input, test_line);
                if (test_line /= null) then
                    writeline(copy_output, test_line);
                end if;
            end loop;
            file_close(copy_input);
            file_close(copy_output);
            report "input.dat copied into output.dat.";
            wait;
        end process;
    end architecture adms;
```

Load the model in Questa ADMS, and run for say 100 ns. The data is copied from *input.dat* into *output.dat*.

Note

As soon as you load the model via the [Start Simulation Dialog Box](#), an empty *output.dat* file is created. Only after running for a simulation time is the data copied into the output file.

IEEE STD_LOGIC_TEXTIO Package

This package overloads the procedures for READ and WRITE for use with the following types; STD ULOGIC, STD ULOGIC VECTOR, STD LOGIC VECTOR and for Hexadecimal and Octal values.

To access the routines defined in the STD_LOGIC_TEXTIO package, the following syntax has to be added before the entity or architecture declaration in the VHDL-AMS code:

```
library IEEE;
use IEEE.std_logic_textio.all;
```

Example

This example reads a *std_ulogic* vector which is written in a Hexadecimal form from the file *input.dat*, then converts it to *bit* vector and writes it to the file *output.dat*. i.e. if the file *input.dat* contains the Hexadecimal data F, then 1111 will be written to the file *output.dat*.

```
use std.TEXTIO.all;
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.std_logic_textio.all;

ENTITY simple_std_logic_textio IS
END ENTITY simple_std_logic_textio;

ARCHITECTURE adms OF simple_std_logic_textio IS
  BEGIN
    TESTING : PROCESS
      file F_IN : TEXT open READ_MODE is "input.dat";
      file F_OUT : TEXT open WRITE_MODE is "ouput.dat";
      variable L : LINE;
      variable SUV : STD_ULOGIC_VECTOR(1 to 4);
      BEGIN
        READLINE(F_IN, L);
        HREAD(L, SUV);
        WRITE(L, SUV);
        WRITELINE(F_OUT, L);

        wait;
      END PROCESS TESTING;
END ARCHITECTURE adms;
```

Multiple Data Types

Data can be read from files that contain several data types.

The following example reads in data from the file *input_all_kind.txt* which contains a line of data that consists of several data types: `std_ulogic`, `std_ulogic_vector`, `std_logic`, `std_logic_vector`, `bit`, `bit_vector`, `character`, `integer`, `boolean`, `real`, `time`, and `string`. The Standard and IEEE TEXTIO procedures `READLINE(..)` and `READ(..)` are used for reading this input file.

The design then writes the data read from the input file to the output file *output_all_kind.txt* by using the Standard and IEEE TEXTIO procedures `WRITELINE(..)` and `WRITE(..)`.

The data that is read in from the file *input_all_kind.txt* and written to the file *output_all_kind.txt* is shown below:

```
X10LHWH0L1110118TRUE1.253400e+14It is my string73000 ns1
```

The data type and the associated data that is read in is shown in [Table A-2](#).

Table A-2: Data and Data Type

Data Type	Value
STD_ULOGIC	X
STD_ULOGIC_VECTOR	10LH
STD_LOGIC	W
STD_LOGIC_VECTOR	H0L1
BIT_VECTOR	1101
CHARACTER	ESC
INTEGER	18
BOOLEAN	TRUE
REAL	125.34e12
STRING	It is my string
TIME	73us
BIT	1

See also The TextIO Package in the *Questa SIM User's Manual*.

Example

The syntax used to read the data in from the file *input_all_kind.txt* and write it to the file *output_all_kind.txt* is shown below:

```

LIBRARY ieee, std;
USE IEEE.electrical_systems.ALL,
ieee.std_logic_1164.ALL,
ieee.std_logic_arith.ALL,
ieee.math_real.ALL;
USE std.TEXTIO.ALL;
USE ieee.std_logic_textio.ALL;

ENTITY allkind_IO IS
END allkind_IO;

ARCHITECTURE adms OF allkind_IO IS

  SIGNAL s_std_ulogic : STD_ULOGIC := '0';
  SIGNAL s_std_ulogic_vector : STD_ULOGIC_VECTOR(3 DOWNTO 0) := "0000";
  SIGNAL s_std_logic : STD_LOGIC := '0';
  SIGNAL s_std_logic_vector : STD_LOGIC_VECTOR(3 DOWNTO 0) := "0000";
  SIGNAL s_bit_vector : BIT_VECTOR(3 DOWNTO 0) := "0000";
  SIGNAL s_character : CHARACTER := NUL;
  SIGNAL s_integer : INTEGER := 0;
  SIGNAL s_boolean : BOOLEAN := FALSE;
  SIGNAL s_real : REAL := 0.0;
  SIGNAL s_string : STRING(1 TO 14) := (others => NUL);
  SIGNAL s_time : TIME := 0 us;
  SIGNAL s_bit : BIT := '0';

BEGIN

  TESTING: PROCESS
    file FI: TEXT OPEN READ_MODE is "input_all_kind.txt";
    file FO: TEXT OPEN WRITE_MODE is "output_all_kind.txt";
    VARIABLE LI, LO : LINE ;
    VARIABLE v_std_ulogic : STD_ULOGIC;
    VARIABLE v_std_ulogic_vector : STD_ULOGIC_VECTOR(3 DOWNTO 0);
    VARIABLE v_std_logic : STD_LOGIC;
    VARIABLE v_std_logic_vector : STD_LOGIC_VECTOR(3 DOWNTO 0);
    VARIABLE v_bit_vector : BIT_VECTOR(3 DOWNTO 0) ;
    VARIABLE v_character : CHARACTER ;
    VARIABLE v_integer : INTEGER ;
    VARIABLE v_boolean : BOOLEAN ;
    VARIABLE v_real : REAL ;
    VARIABLE v_string : STRING(1 TO 14) ;
    VARIABLE v_time : TIME ;
    VARIABLE v_bit : BIT ;

  BEGIN
    while not endfile(FI) loop
      READLINE (FI, LI);
      WAIT FOR 1ns;

```



```

    READ(LI, v_std_ulogic );
    WAIT FOR 1ns;
    READ(LI, v_std_ulogic_vector );
    WAIT FOR 1ns;
    READ(LI, v_std_logic );
    WAIT FOR 1ns;
    READ(LI, v_std_logic_vector );
    WAIT FOR 1ns;
    READ(LI, v_bit_vector );
    WAIT FOR 1ns;
    READ(LI, v_character );
    WAIT FOR 1ns;
    READ(LI, v_integer );
    WAIT FOR 1ns;
    READ(LI, v_boolean );
    WAIT FOR 1ns;
    READ(LI, v_real );
    WAIT FOR 1ns;
    READ(LI, v_string );
    WAIT FOR 1ns;
    READ(LI, v_time );
    WAIT FOR 1ns;
    READ(LI, v_bit );
    WAIT FOR 1ns;
end loop;

s_std_ulogic <= v_std_ulogic ;
s_std_ulogic_vector <= v_std_ulogic_vector ;
s_std_logic <= v_std_logic ;
s_std_logic_vector <= v_std_logic_vector ;
s_bit_vector <= v_bit_vector ;
s_character <= v_character ;
s_integer <= v_integer ;
s_boolean <= v_boolean ;
s_real <= v_real ;
s_string <= v_string ;
s_time <= v_time ;
s_bit <= v_bit ;

WAIT FOR 1ns;
-- Output Routines for Standard Types
WRITE(L0, s_std_ulogic );
WRITE(L0, s_std_ulogic_vector );
WRITE(L0, s_std_logic );
WRITE(L0, s_std_logic_vector );
WRITE(L0, s_bit_vector );
WRITE(L0, s_character);
WRITE(L0, s_integer);
WRITE(L0, s_boolean);
WRITE(L0, s_real);
WRITE(L0, s_string);
WRITE(L0, s_time);
WRITE(L0, s_bit);
WRITELINE(F0, L0);

wait;

```

```

END PROCESS TESTING;

END ARCHITECTURE adms;

```

ASPDETECT

ASPDETECT is a signal that changes state at the end of each accepted analog solution point (ASP). An ASP is a transient step or a DC point, not an AC solution. The value of the ASPDETECT signal has no meaning; only the change is significant.

At time 0, each computed DC generates an event. If no DC is performed (UIC) no event is generated at time 0.

It is accessible from VHDL, VHDL-AMS and Verilog-AMS contexts.

From VHDL and VHDL-AMS Context

This signal is a Bit signal named ASPDETECT and is defined in the package ASPDETECT, in the library MGC_AMS.

To use ASPDETECT from a VHDL or VHDL-AMS context, specify the following:

```

library mgc_ams;
use mgc_ams.aspdetect.all;

```

The definition of ASPDETECT is provided in the file *mgc_aspdetect.vhd* and is as follows:

```

package aspdetect is
begin
  signal aspdetect : Bit := '0';
end package aspdetect;

```

From Verilog-AMS Context

From Verilog-AMS, this signal is an event function named \$aspdetect. It can only be part of a digital portion of Verilog-AMS.

absdelta

Generates events during initialization and in response to changes on its value argument.

Syntax

```
absdelta '(' value ',' delta
          [ ',' time_tol
            [ ',' expr_tol ] ] ')'
```

Arguments

- **value**
A dynamic expression.
- **delta**
May be a dynamic expression. During the first iteration of each new time step delta is computed and remains constant until the next time step begins.

If the computed value is negative, the absolute value is used and a warning is issued.

If delta is zero then an event is generated at the end of each analog solution point.
- **time_tol**
A static expression. If the computed value is negative, the absolute value is used and a warning is issued. Can take the string value NULL to instruct use of the default value.
- **expr_tol**
A static expression. If the computed value is negative, the absolute value is used and a warning is issued. Can take the string value NULL to instruct use of the default value.

Description

The events are generated as follows:

- At each DC computation.
- During the first digital simulation cycle at time 0.
- When value changes more than one delta relative to the previous event. time_tol and expr_tol provide tolerances in order to find when the event occurs.
- When value changes direction (the amount of change must be more than expr_tol).

This means that even if a delta is not reached and the wave is changing direction, the extremum (local maximum or minimum) generates an event according both tolerances.

Related Topics

ASPDETECT

LOOK_UP_TABLE_V1X

The Verilog-AMS system function \$table can be accessed from within a VHDL-AMS description using the package LOOK_UP_TABLE_V1X defined in the MGC_AMS library. The data can be extracted either from ASCII files or from parameter arrays.

Each system function operates in DC, AC and transient analyses, and returns a single real value. The method for calculating the interpolated value must be specified, the methods supported are:

- 1D - linear (default) or spline
- 2D - linear or y_spline (default)
- 3D - linear or y_spline (default)

Examples

- The following shows two examples of how to access the table-system function from within a VHDL-AMS description. The method has been omitted, therefore, the default method, linear interpolation, will be used:

```
entity resistance is
    port(a, b Electrical);
end entity resistance;
-----
library MGC_AMS;
    use MGC_AMS.look_up_table_v1x.all;
architecture measured_ex1 of resistance is
    quantity Vab across Iab through a to b;
    constant x : Real_vector := (0.0, 1.0, 2.0, 3.0, 5.0, 10.0, 20.0);
    constant f_x : Real_vector := (0.0, 0.2, 0.4, 0.6, 1.0, 2.0, 4.0);
begin
    Iab == table1a(Vab, x, f_x); -- default linear method
end architecture measured_ex1
-----
library MGC_AMS;
    use MGC_AMS.look_up_table_v1x.all;
architecture measured_ex2 of resistance is
    quantity Vab across Iab through a to b;
begin
    Iab == table1f(Vab, "imp,dat"); -- default linear method
end architecture measured_ex2
-----
#Input data file inp.dat
x f(x)
0.0 1.0 # first pair
1.0 1.0 # second pair
```

```
2.0 1.1 # third pair
3.0 2.0 # fourth pair
3.5 4.0 # fifth pair
5.0 4.9 # sixth pair
6.1 5.0 # seventh pair
```

Related Topics

Table-Based Interpolation and Lookup System Functions

B. VHDL-AMS Subset Definition

The current implementation of VHDL-AMS in Questa ADMS supports a subset of the features of the language defined by IEEE VHDL standards 1076-1987, 1076-1993, 1076-2002, and 1076-2008.

Note
For summary information, see also [VHDL-AMS Quick Reference.pdf](#).

This appendix describes in detail the areas of the language where support is limited in Questa ADMS. VHDL-AMS syntax shown in red is not supported in Questa ADMS. Syntax shown in amber is supported with limitations.

Note
This appendix does not provide a full description of the VHDL-AMS language. For this, refer to the *1076.1-1999 IEEE Standard VHDL Analog and Mixed-Signal Extensions Language Reference Manual*, available from the IEEE website (<http://standards.ieee.org>).

Design Entities and Configurations..... B-2

Subprograms and Packages.....B-7

Types and Natures..... B-11

Declarations.....B-14

Specifications.....B-21

Names.....B-24

Expressions.....B-25

Sequential Statements..... B-26

Concurrent Statements..... B-30

Identifiers..... B-36

Predefined Attributes..... B-37

Simultaneous Statements..... B-38

Design Entities and Configurations

In VHDL-AMS, a design entity represents a portion of a hardware design that has well-defined inputs and outputs and performs a well-defined function. A configuration can be used to describe how design entities are put together to form a complete design.

A design entity may represent an entire system, a subsystem, a board, a chip, a macro-cell, a logic gate, or any level of abstraction in between.

This topic describes the limitations in Questa ADMS for [Entity Declarations](#), [Architecture Bodies](#) and [Configuration Declarations](#).

Entity Declarations

Entity declarations define the interface between a given design entity and the environment in which it is used. It may also specify declarations and statements that are part of the design entity. The syntax for an entity declaration is shown below. The items shown in

red

are not supported in Questa ADMS. The items shown in

amber

are supported with limitations.

Syntax

```
entity_declaration ::=
    entity identifier is
        entity_header
        entity_declarative_part
    [ begin
        entity_statement_part ]
    end [ entity ] [ entity_simple_name ] ;

entity_declarative_part ::=
    { entity_declarative_item }

entity_declarative_item ::=
    subprogram_declaration
    | subprogram_body
    | type_declaration
    | subtype_declaration
    | constant_declaration
    | signal_declaration
    | shared_variable_declaration
```



```

| file_declaration
| alias_declaration
| attribute_declaration
| attribute_specification
| disconnection_specification
| step_limit_specification
| use_clause
| group_template_declaration
| group_declaration
| nature_declaration
| subnature_declaration
| quantity_declaration
| terminal_declaration

```

Limitations on Entity Declarations

- **alias_declaration**
Object aliases of signals and terminals (only scalar declarations or elements of a composite declaration of these objects) are supported.
 - Aliases of file objects, quantity objects and composite objects are not supported.
 - Non-object aliases are not supported.
- **attribute_declaration** and **attribute_specification**
These are allowed in an entity declaration, but access to attributes is not supported.
- Ports of mode LINKAGE are not supported.
- Concurrent procedure calls are not supported in the entity statement part:

```

entity_statement_part ::=
    { entity_statement }

entity_statement ::=
    concurrent_assertion_statement
    | passive_concurrent_procedure_call
    | passive_process_statement

```

Architecture Bodies

Architecture bodies specify the relationships between the inputs, outputs, quantities, and terminals of a design entity and may be expressed in terms of structure, dataflow, equations, or behavior. The syntax for an architecture body is shown below. The items shown in

red

are not supported in Questa ADMS. The items in

amber

are supported with limitations.

Syntax

```
architecture_body ::=
    architecture identifier of entity_name is
        architecture_declarative_part
    begin
        architecture_statement_part
    end [ architecture ] [ architecture_simple_name ] ;

architecture_declarative_part ::=
    { block_declarative_item }

block_declarative_item ::=
    subprogram_declaration
    | subprogram_body
    | type_declaration
    | subtype_declaration
    | constant_declaration
    | signal_declaration
    | shared_variable_declaration
    | file_declaration
    | alias_declaration
    | component_declaration
    | attribute_declaration
    | attribute_specification
    | configuration_specification
    | disconnection_specification
    | step_limit_specification
    | use_clause
    | group_template_declaration
    | group_declaration
    | nature_declaration
    | subnature_declaration
    | quantity_declaration
    | terminal_declaration
```

Limitations on Architecture Bodies

- **type_declaration**
File type declarations are not supported.
- **alias_declaration**
Object aliases of signals and terminals (only scalar declarations or elements of a composite declaration of these objects) are supported.

- Aliases of file objects, quantity objects and composite objects are not supported.
- Non-object aliases are not supported.
- attribute_declaration and attribute_specification

These are allowed in an entity declaration, but access to attributes is not supported.

Configuration Declarations

Configuration declarations specify the binding of design entities to component instances. They appear in the same declarative part of the block as the corresponding component instances. The syntax for a configuration declaration is shown below. The items shown in

red

are not supported in Questa ADMS. The items in

amber

are supported with limitations.

Syntax

```
configuration_declaration ::=
    configuration identifier of entity_name is
        configuration_declarative_part
        block_configuration
    end [ configuration ] [ configuration_simple_name ] ;

configuration_declarative_part ::=
    { configuration_declarative_item }

configuration_declarative_item ::=
    use_clause
    | attribute_specification
    | group_declaration

block_configuration ::=
    for block_specification
        { use_clause }
        { configuration_item }
    end for ;

block_specification ::=
    architecture_name
    | block_statement_label
    | generate_statement_label [ ( index_specification ) ]

index_specification ::=
```

```

    discrete_range
    | static_expression

configuration_item ::=
    block_configuration
    | component_configuration

component_configuration ::=
    for component_specification
        [ binding_indication ; ]
        [ block_configuration ]
    end for ;

```

Limitations on Configuration Declarations

- Configuration declarations are not supported at the border between analog/mixed-signal and digital.
- binding_indication

A binding indication specification can only be an entity name defined with entity_aspect.

```

binding_indication ::=
    [ use entity_aspect ]
    [ generic_map_aspect ]
    [ port_map_aspect ]

```

A special syntax is required for the entity_aspect of a configuration when it specifies a nested configuration from VHDL (or when a VHDL-AMS object contains a nested VHDL configuration). You must use the reserved word entity in place of the reserved word configuration:

```

entity_aspect ::=
    entity entity_name [ ( architecture_identifier ) ]
    | configuration configuration_name
    | entity configuration_name
    | open

```

- Open bindings (deferred design entity identification) are not supported.
- block_configuration is not allowed if the language of the library unit is different from the language of the enclosing configuration.

Related Topics

[Subprograms and Packages](#)

[Declarations](#)

[Specifications](#)

Subprograms and Packages

This section describes the limitations in Questa ADMS for subprograms and packages.

The items shown in

red

are not supported in Questa ADMS. The items in

amber

are supported with limitations.

Subprogram Declarations

A subprogram declaration declares a procedure or a function, as indicated by the appropriate reserved word. The declaration of a subprogram is optional. In the absence of such a declaration, the subprogram specification of the subprogram body acts as the declaration. The syntax for a subprogram declaration and a subprogram body is shown below.

Subprogram Declaration Syntax

```
subprogram_declaration ::=
    subprogram_specification ;

subprogram_specification ::=
    procedure designator [ ( formal_parameter_list ) ]
    | [ pure | impure ] function designator [ ( formal_parameter_list ) ]
      return type_mark

designator ::= identifier | operator_symbol

operator_symbol ::= string_literal
```

Subprogram Body Syntax

```
subprogram_body ::=
    subprogram_specification is
      subprogram_declarative_part
    begin
      subprogram_statement_part
    end [ subprogram_kind ] [ designator ] ;

subprogram_declarative_part ::=
    { subprogram_declarative_item }
```

```

subprogram_declarative_item ::=
    subprogram_declaration
    | subprogram_body
    | type_declaration
    | subtype_declaration
    | constant_declaration
    | variable_declaration
    | file_declaration
    | alias_declaration
    | attribute_declaration
    | attribute_specification
    | use_clause
    | group_template_declaration
    | group_declaration

subprogram_statement_part ::=
    { sequential_statement }

subprogram_kind ::= procedure | function

```

Limitations on Subprogram Bodies

- **alias_declaration**
Object aliases of signals and terminals (only scalar declarations or elements of a composite declaration of these objects) are supported.
 - Aliases of file objects, quantity objects and composite objects are not supported.
 - Non-object aliases are not supported.
- **attribute_declaration** and **attribute_specification**
These are allowed in an entity declaration, but access to attributes is not supported.
- wait statements are not supported in the **subprogram_statement_part**.
- A use clause may not appear in a function declarative part.
- Signals declared in an enclosing declarative region of a given procedure cannot appear on the left hand side of a signal assignment within the procedure. This limitation does not apply to signal ports of the procedure.

Package Declarations

A package declaration defines the interface to a package. The syntax for the package declarative part of a package declaration is shown below.

Syntax

```

package_declaration ::=
    package identifier is
    package_declarative_part

```

```

end [ package ] [ package_simple_name ] ;

package_declarative_part ::=
  { package_declarative_item }

package_declarative_item ::=
  subprogram_declaration
  | type_declaration
  | subtype_declaration
  | constant_declaration
  | signal_declaration
  | shared_variable_declaration
  | file_declaration
  | alias_declaration
  | component_declaration
  | attribute_declaration
  | attribute_specification
  | disconnection_specification
  | use_clause
  | group_template_declaration
  | group_declaration
  | nature_declaration
  | subnature_declaration
  | terminal_declaration

```

Limitations on Package Declarations

- **type_declaration**
File type declarations are only allowed in TEXTIO packages for text type (STD.TEXTIO.TEXT).
- **alias_declaration**
Object aliases of signals and terminals (only scalar declarations or elements of a composite declaration of these objects) are supported.
 - Aliases of file objects, quantity objects and composite objects are not supported.
 - Non-object aliases are not supported.
- **attribute_declaration** and **attribute_specification**
These are allowed in a package declaration, but access to attributes is not supported.

Package Bodies

A package body defines the bodies of subprograms and the values of deferred constants declared in the interface to the package. The syntax for a package body is shown below.

Syntax

```

package_body ::=
  package body package_simple_name is
    package_body_declarative_part

```

```

    end [ package body ] [ package_simple_name ] ;

package_body_declarative_part ::=
    { package_body_declarative_item }

package_body_declarative_item ::=
    subprogram_declaration
    | subprogram_body
    | type_declaration
    | subtype_declaration
    | constant_declaration
    | shared_variable_declaration
    | file_declaration
    | alias_declaration
    | use_clause
    | group_template_declaration
    | group_declaration

```

Limitations on Package Bodies

- type_declaration
File type declarations are only allowed in TEXTIO packages for text type (STD.TEXTIO.TEXT).
- alias_declaration
Object aliases of signals and terminals (only scalar declarations or elements of a composite declaration of these objects) are supported.
 - Aliases of file objects, quantity objects and composite objects are not supported.
 - Non-object aliases are not supported.

Related Topics

Declarations

Types and Natures

This section describes the areas where the VHDL-AMS language is limited in Questa ADMS for various types.

Composite Types

Composite types are used to define collections of values. These include both arrays of values (collections of values of a homogeneous type) and records of values (collections of values of potentially heterogeneous types).

An array object is a composite object consisting of elements that have the same subtype. The name for an element of an array uses one or more index values belonging to specified discrete types. The value of an array object is a composite value consisting of the values of its elements.

In Questa ADMS, composite types with up to three levels are supported:

- A record of arrays
- An array of records
- A record of records
- An array of arrays.

The lowest level is in all cases is an element of a scalar type. The elements of a resolved composite type must be scalar.

The syntax for a composite type definition is shown below. The items shown in

amber

are supported with limitations in Questa ADMS.

Syntax

```
composite_type_definition ::=
    array_type_definition
    | record_type_definition

array_type_definition ::=
    unconstrained_array_definition | constrained_array_definition
unconstrained_array_definition ::=
    array ( index_subtype_definition { , index_subtype_definition } )
    of element_subtype_indication

constrained_array_definition ::=
    array index_constraint of element_subtype_indication
```

```

index_subtype_definition ::= type_mark range <>
index_constraint ::= ( discrete_range { , discrete_range } )
discrete_range ::= discrete_subtype_indication | range
record_type_definition ::=
    record
        element_declaration
        { element_declaration }
    end record [ record_type_simple_name ]
element_declaration ::=
    identifier_list : element_subtype_definition ;
identifier_list ::= identifier { , identifier }
element_subtype_definition ::= subtype_indication

```

Limitations on Composite Types

- array_type_definition

In Questa ADMS, only vectors are allowed in array objects. The following items are not supported:

- Multi-dimension array types and multi-dimensional array aggregates
- Arrays of arrays greater than 2 levels of depth
- Arrays of arrays of records
- The type of a function return cannot be an array of array type.

- record_type_definition

Questa ADMS supports record objects within VHDL-AMS and at the VHDL/VHDL-AMS boundary. Constants and generic constants, variables and signals can be of a record type, although the following restrictions apply to the types of the field of a record in VHDL-AMS:

- Elements of records can be scalars, array or records, however, the elements of a nested array cannot themselves be records.
- A record type can contain a maximum of 4095 elements
- A record type cannot contain an array of records
- A record type cannot contain globally static elements (locally static elements are allowed)
- Resolved records may only be associated as a whole. Partial sub-element associations are not accepted.
- Record nature declarations are not supported.

Access Types

Access type definitions are allowed in TEXTIO packages for line types only (STD.TEXTIO.LINE).

In addition, the following limitations apply:

- Indexed variables are not supported for access types.
- Incomplete type declarations are not supported:

```
incomplete_type_declaration ::= type identifier ;
```

- The procedure for deallocation of objects is not supported.

File Types

File type declarations are only allowed in TEXTIO packages for text types (STD.TEXTIO.TEXT).

Protected Types

Protected types are not supported.

Related Topics

[Declarations](#)

Declarations

The VHDL-AMS language defines several kinds of entities that are declared explicitly or implicitly by declarations.

The items shown in

amber

are supported with limitations in Questa ADMS.

```
declaration ::=
  type_declaration
  | subtype_declaration
  | object_declaration
  | interface_declaration
  | alias_declaration
  | attribute_declaration
  | component_declaration
  | group_template_declaration
  | group_declaration
  | entity_declaration
  | configuration_declaration
  | subprogram_declaration
  | package_declaration
  | nature_declaration
  | subnature_declaration
```

The following sections describe the areas where the VHDL-AMS language is limited in Questa ADMS for [Subtype Declarations](#), [Object Declarations](#), [Interface Declarations](#), [Alias Declarations](#), [Attribute Declarations](#), [Group Template Declarations](#), [Group Declarations](#) and [Nature Declaration](#).

The items shown in

red

are not supported in Questa ADMS.

Subtype Declarations

The syntax for a subtype declaration is shown below.

Syntax

```
subtype_declaration ::=
  subtype identifier is subtype_indication ;
```

```

subtype_indication ::=
    [ resolution_function_name ] type_mark [ constraint ]
    [ tolerance_aspect ]

type_mark ::=
    type_name
    | subtype_name

constraint ::=
    range_constraint
    | index_constraint

tolerance_aspect ::=
    tolerance_string_expression

```

Limitations on Subtype Declarations

- tolerance_aspect

In a subtype declaration, tolerance_aspect is accepted by Questa ADMS without error, but this functionality is not supported.

Object Declarations

An object declaration declares an object of a specified type. The syntax for an object declaration is shown below.

Syntax

```

object_declaration ::=
    constant_declaration
    | signal_declaration
    | variable_declaration
    | file_declaration
    | terminal_declaration
    | quantity_declaration

constant_declaration ::=
    constant identifier_list : subtype_indication [ := expression ] ;

signal_declaration ::=
    signal identifier_list : subtype_indication [ signal_kind ]
    [ := expression ] ;

signal_kind ::= register | bus

variable_declaration ::=
    [ shared ] variable identifier_list : subtype_indication [ :=
expression ] ;

file_declaration ::=
    file identifier_list : subtype_indication [ file_open_information ] ;

```

```

file_open_information ::= [ open file_open_kind_expression ] is
file_logical_name

file_logical_name ::= string_expression

terminal_declaration ::=
    terminal identifier_list : subtype_indication ;

quantity_declaration ::=
    free_quantity_declaration
    | branch_quantity_declaration
    | source_quantity_declaration

free_quantity_declaration ::=
    quantity identifier_list : subtype_indication [ := expression ] ;

branch_quantity_declaration ::=
    quantity [ across_aspect ] [ through_aspect ] terminal_aspect ;

source_quantity_declaration ::=
    quantity identifier_list : subtype_indication source_aspect ;

across_aspect ::=
    identifier_list [ tolerance_aspect ] [ := expression ] across

through_aspect ::=
    identifier_list [ tolerance_aspect ] [ := expression ] through

terminal_aspect ::=
    plus_terminal_name [ to minus_terminal_name ]

source_aspect ::=
    spectrum magnitude_simple_expression , phase_simple_expression
    | noise power_simple_expression

```

Limitations on Signal Declarations

- signal_kind

If signal_kind appears in a signal declaration, then the signals so declared are guarded signals of the kind indicated. Guarded signals are not supported in signal declarations, therefore signal_kind is not supported.

- subtype_indication

Declaring a resolved signal using a resolution function and an unresolved type is not supported in signal declarations. A resolved signal is a signal that has a resolution function associated with it, either in the signal declaration, or in the subtype declaration used to declare the signal.

Limitation on Variable Declarations

- Shared variable declarations are not supported in Questa ADMS.

Limitation on File Declarations

- When specifying `file_logical_name`, only locally static logical names are supported. Globally static logical names are not supported.

Limitations on Quantity Declarations

- Records or array of records are not supported in the free quantity declaration part.
- In a quantity declaration, `tolerance_aspect` is accepted by Questa ADMS without error, but this functionality is not supported.
- Noise source quantities are not supported.

Eldo noise sources may be instantiated in an Eldo `.ckt` file and included in a VHDL-AMS design. The magnitude and phase expressions in a spectrum source must be static (neither quantities nor the frequency function is allowed).

Interface Declarations

An interface declaration declares an interface object of a specified type. The syntax for an interface declaration is shown below.

Syntax

```
interface_declaration ::=
    interface_constant_declaration
    | interface_signal_declaration
    | interface_variable_declaration
    | interface_file_declaration
    | interface_terminal_declaration
    | interface_quantity_declaration

interface_constant_declaration ::=
    [constant] identifier_list : [ in ] subtype_indication [ :=
static_expression ]

interface_signal_declaration ::=
    [signal] identifier_list : [ mode ] subtype_indication [ bus ] [ :=
static_expression ]

interface_variable_declaration ::=
    [variable] identifier_list : [ mode ] subtype_indication [ :=
static_expression ]

interface_file_declaration ::=
    file identifier_list : subtype_indication

mode ::= in | out | inout | buffer | linkage

interface_terminal_declaration ::=
    terminal identifier_list : subnature_indication
```

```
interface_quantity_declaration ::=
    quantity identifier_list : [ in | out ] subtype_indication [ :=
    static_expression ]
```

Limitations on Interface Declarations

The following items are not supported in interface declarations:

- Guarded signals
- Resolved signals of unresolved subtype

A resolved signal is a signal that has a resolution function associated with it, either in the signal declaration, or in the subtype declaration used to declare the signal.
- Ports of mode linkage.
- Conversion functions within a formal association list are limited in Questa ADMS as follows:
 - Not supported as an actual of port map: expression, type conversion, function call, open.
 - Not supported as an actual of generic map: type conversion, function call, open.
 - Not supported as a formal of port map: type conversion, function call.
 - Gathered partial associations are not supported as an actual of parameter map.
 - Unassociated formal ports are not supported.

Alias Declarations

An alias declaration declares an alternate name for an existing named entity. Object aliases of signals and terminals (only scalar declarations or elements of a composite declaration of these objects) are supported in Questa ADMS.

Syntax

```
alias_declaration ::=
    alias alias_designator [ : alias_indication ] is name [ signature ] ;
alias_designator ::= identifier | character_literal | operator_symbol
alias_indication ::= subtype_indication | subnature_indication
```

Limitations on Alias Declarations

- Aliases of file objects, quantity objects and composite objects are not supported in Questa ADMS.
- Non-object aliases are not supported in Questa ADMS.

Attribute Declarations

The declaration of attributes is allowed in Questa ADMS, but access to attributes is not supported.

```
attribute_declaration ::=
    attribute identifier: type_mark ;
```

Group Template Declarations

A group template declaration defines the allowable classes of named entities that can appear in a group. Group template declarations are not supported in Questa ADMS.

```
group_template_declaration ::=
    group identifier is ( entity_class_entry_list ) ;

entity_class_entry_list ::=
    entity_class_entry { , entity_class_entry }

entity_class_entry ::= entity_class [ <> ]
```

Group Declarations

A group declaration declares a group - a named collection of named entities. Group declarations are not supported in Questa ADMS.

```
group_declaration ::=
    group identifier : group_template_name ( group_constituent_list ) ;

group_constituent_list ::= group_constituent { , group_constituent }

group_constituent ::= name | character_literal
```

Nature Declaration

A nature declaration declares a nature and defines the across and through types of the nature.

Syntax

```
nature_declaration ::=
    nature identifier is nature_definition ;

nature_definition ::=
    scalar_nature_definition | composite_nature_definition

subnature_declaration ::=
    subnature identifier is subnature_indication ;
```

```
subnature_indication ::=
    nature_mark [ index_constraint ]
    [ tolerance string_expression across string_expression through ]
nature_mark ::=
    nature_name | subnature_name
```

Limitations on Nature Declarations

- Array nature declaration is restricted in Questa ADMS as described in [Composite Types](#).
- Record nature declaration is not supported.

Related Topics

[Design Entities and Configurations](#)

Specifications

A specification associates additional information with a named entity that has been previously declared.

There are four kinds of specifications: [Attribute Specifications](#), [Configuration Specifications](#), [Disconnection Specifications](#) and [Step Limit Specifications](#).

Attribute Specifications

An attribute specification associates a user-defined attribute with one or more named entities and defines the value of that attribute for those entities. In Questa ADMS, attribute specifications are allowed, but access to attributes is not supported.

```
attribute_specification ::=
    attribute attribute_designator of entity_specification is expression ;
```

Configuration Specifications

A configuration specification associates binding information with component labels representing instances of a given component declaration. The instantiation list identifies those component instances with which binding information is to be associated. In Questa ADMS, the specification of binding indications within a configuration specification is limited.

The syntax for a configuration specification is shown below. Items shown in

red

are not supported in Questa ADMS. The items shown in

amber

are supported with limitations.

Syntax

```
configuration_specification ::=
    for component_specification binding_indication;

component_specification ::=
    instantiation_list : component_name

instantiation_list ::=
    instantiation_label { , instantiation_label }
    | others
    | all
```

```

binding_indication ::=
    [ use entity_aspect ]
    [ generic_map_aspect ]
    [ port_map_aspect ]

entity_aspect ::=
    entity entity_name [ ( architecture_identifier) ]
    | configuration configuration_name
    | open

```

Limitations on Configuration Specifications

- Binding indications are used in configuration specifications to associate instances of a component declaration with a particular design entity. In Questa ADMS, a binding indication specification can only be an entity name defined with entity_aspect.
- Open bindings (deferred design entity identification) are not supported.

Disconnection Specifications

Disconnection specifications are not supported.

```

disconnection_specification ::=
    disconnect guarded_signal_specification after time_expression;

guarded_signal_specification ::=
    guarded_signal_list : type_mark

signal_list ::=
    signal_name { , signal_name }
    | others
    | all

```

Step Limit Specifications

Step limit specifications are not supported.

```

step_limit_specification ::=
    limit quantity_specification with real_expression ;

quantity_specification ::=
    quantity_list : type_mark

quantity_list ::=
    quantity_name { , quantity_name }
    | others
    | all

```

Related Topics

[Design Entities and Configurations
Declarations](#)

Names

A selected name is used to denote a named entity whose declaration appears either within the declaration of another named entity or within a design library.

```
selected_name ::= prefix . suffix  
  
suffix ::=  
    simple_name  
    | character_literal  
    | operator_symbol  
    | all
```

In Questa ADMS, the prefix of a selected name cannot be a function call.

Related Topics

[Expressions](#)

Expressions

Limitations apply to operators and operands in Questa ADMS.

- The concatenation operator & is not supported for arrays of records.
- The literal null is only supported for access types of STD.TEXTIO.LINE.

```
literal ::=
    numeric_literal
    | enumeration_literal
    | string_literal
    | bit_string_literal
    | null

numeric_literal ::=
    abstract_literal
    | physical_literal
```

- Allocators are not supported:

```
allocator ::=
    new subtype_indication
    | new qualified_expression
```

Related Topics

Sequential Statements

Sequential Statements

Sequential statements define algorithms for the execution of a subprogram or process. Sequential statements run in the order in which they appear.

The syntax for a sequential statement is shown below. The items shown in

amber

are supported with limitations in Questa ADMS.

Syntax

```
sequence_of_statements ::=
    { sequential_statement }

sequential_statement ::=
    wait_statement
  | assertion_statement
  | report_statement
  | signal_assignment_statement
  | variable_assignment_statement
  | procedure_call_statement
  | if_statement
  | case_statement
  | loop_statement
  | next_statement
  | exit_statement
  | return_statement
  | null_statement
  | break_statement
```

This topic describes the limitations in Questa ADMS limitations for [Wait Statements](#), [Signal Assignment Statements](#), [Variable Assignment Statements](#) and [Break Statements](#).

The items shown in

red

are not supported in Questa ADMS.

Wait Statements

The wait statement causes the suspension of a process statement or a procedure. The syntax for a wait statement is shown below.

Syntax

```
wait_statement ::=
    [ label : ] wait [ sensitivity_clause ] [ condition_clause ]
    [ timeout_clause ] ;

sensitivity_clause ::= on sensitivity_list
sensitivity_list ::= signal_name { , signal_name }
condition_clause ::= until condition
condition ::= boolean_expression
timeout_clause ::= for time_or_real_expression
```

Limitations on Wait Statements

- wait statements are not supported in procedure bodies.
- The use of aggregates in wait statements is not supported.

Signal Assignment Statements

A signal assignment statement modifies the projected output waveforms contained in the drivers of one or more signals. The syntax for a signal assignment statement is shown below.

Syntax

```
signal_assignment_statement ::=
    [ label : ] target <= [ delay_mechanism ] waveform ;

delay_mechanism ::=
    transport
    | [ reject time_expression ] inertial

target ::=
    name
    | aggregate

waveform ::=
    waveform_element { , waveform_element }
    | unaffected

waveform_element ::=
    value_expression [ after time_expression ]
    | null [ after time_expression ]
```

Limitations on Signal Assignment Statements

- Inside procedures, the target must be a parameter; target aggregates are not supported.

- Unaffected waveforms are not supported.
- Null waveform elements are not supported.
- No check is made that the value of the waveform element belongs to the subtype of the signal.

Variable Assignment Statements

A variable assignment statement replaces the current value of a variable with a new value specified by an expression. The named variable and the right-hand side expression must be of the same type.

```
variable_assignment_statement ::=
    [ label : ] target := expression ;
```

Limitations on Variable Assignment Statements

- Variable assignments with non-null values are not supported for access types.
- Target aggregates are not supported.
- No check is made that the value of the expression belongs to the subtype of the variable.

Break Statements

The execution of a break statement notifies the analog solver that it must determine the discontinuity augmentation set for the next analog solution point. It may also specify reset values for quantities. The effect is conditional if the statement includes a condition.

Syntax

```
break_statement ::=
    [ label : ] break [ break_list ] [ when condition ] ;

break_list ::= break_element { , break_element }

break_element ::= [ break_selector_clause ] quantity_name => expression

break_selector_clause ::= for quantity_name use
```

Limitations on Break Statements

- The break list of the break statement is not supported. Therefore, it is not possible to specify initial conditions for initializing in the quiescent state (DC) nor to specify initial conditions (other than the defaults) after a discontinuity. Execution of a break statement signals the simulation kernel that re-initialization for the next continuous interval is required. The full VHDL-AMS re-initialization algorithm is only approximated.
- When a break statement is specified and not used, or a break statement is required and has not been specified, errors or warnings may be reported in the following situations:

- When a composite signal is defined in a simultaneous statement with an index or a slice, the signal also appears in a simultaneous statement, and no break statement has been found for that signal.
- When implicit signals that depend on expressions S'Delayed(exp), S'Stable(exp), and Q'Above(exp) are used in simultaneous statements or break statements, and exp is neither a literal nor a generic (no expressions with operators or function calls).

Related Topics

Concurrent Statements

Concurrent Statements

Limitations apply to concurrent statements in Questa ADMS.

This topic describes the limitations for:

- **Block Statements**
- **Process Statements**
- **Concurrent Procedure Call Statements**
- **Concurrent Assertion Statements**
- **Concurrent Signal Assignment Statements**
- **Generate Statements**
- **Concurrent Break Statements**

The items shown in red

red

are not supported in Questa ADMS. The items shown in

amber

are supported with limitations.

Block Statements

A block statement defines an internal block representing a portion of a design. Blocks may be hierarchically nested to support design decomposition. The syntax for a block statement is shown below.

Syntax

```
block_statement ::=
    block_label :
        block [ ( guard_expression ) ] [ is ]
            block_header
            block_declarative_part
        begin
            block_statement_part
        end block [ block_label ] ;

block_header ::=
    [ generic_clause
    [ generic_map_aspect ; ] ]
```

```

    [ port_clause
    [ port_map_aspect ; ] ]

block_declarative_part ::=
    { block_declarative_item }

block_declarative_item ::=
    subprogram_declaration
    | subprogram_body
    | type_declaration
    | subtype_declaration
    | constant_declaration
    | signal_declaration
    | shared_variable_declaration
    | file_declaration
    | alias_declaration
    | component_declaration
    | attribute_declaration
    | attribute_specification
    | configuration_specification
    | disconnection_specification
    | step_limit_specification
    | use_clause
    | group_template_declaration
    | group_declaration
    | nature_declaration
    | subnature_declaration
    | quantity_declaration
    | terminal_declaration

block_statement_part ::=
    { architecture_statement }

```

Limitations on Block Statements

- **type_declaration**
File type declarations are not supported.
- **alias_declaration**
Object aliases of signals and terminals (only scalar declarations or elements of a composite declaration of these objects) are supported.
 - Aliases of file objects, quantity objects and composite objects are not supported.
 - Non-object aliases are not supported.
- **attribute_declaration** and **attribute_specification**
These are allowed, but access to attributes is not supported.
- Guarded expressions are not supported.

Process Statements

A process statement defines an independent sequential process representing the behavior of some portion of the design. The syntax for a process statement is shown below.

Syntax

```
process_statement ::=
    [ process_label : ]
    [ postponed ] process [ ( sensitivity_list ) ] [ is ]
    process_declarative_part
    begin
    process_statement_part
    end [ postponed ] process [ process_label ] ;

process_declarative_part ::=
    { process_declarative_item }

process_declarative_item ::=
    subprogram_declaration
    | subprogram_body
    | type_declaration
    | subtype_declaration
    | constant_declaration
    | variable_declaration
    | file_declaration
    | alias_declaration
    | attribute_declaration
    | attribute_specification
    | use_clause
    | group_type_declaration
    | group_declaration

process_statement_part ::=
    { sequential_statement }
```

Limitations on Process Statements

- type_declaration
File type declarations are not supported.
- alias_declaration
Object aliases of signals and terminals (only scalar declarations or elements of a composite declaration of these objects) are supported.
 - Aliases of file objects, quantity objects and composite objects are not supported.
 - Non-object aliases are not supported.
- attribute_declaration and attribute_specification
These are allowed, but access to attributes is not supported.

Concurrent Procedure Call Statements

A concurrent procedure call statement represents a process containing the corresponding sequential procedure call statement.

```
concurrent_procedure_call_statement ::=
    [ label : ] [ postponed ] procedure_call ;
```

In Questa ADMS, concurrent procedure calls are only supported in Net Spy procedures with static parameters. Concurrent procedure calls for Net Spy on analog objects are not supported for dynamic parameters.

Concurrent Assertion Statements

A concurrent assertion statement represents a passive process statement containing the specified assertion statement.

```
concurrent_assertion_statement ::=
    [ label : ] [ postponed ] assertion ;
```

In Questa ADMS, postponed concurrent assertions are not supported.

Concurrent Signal Assignment Statements

A concurrent signal assignment statement represents an equivalent process statement that assigns values to signals. The syntax for a concurrent signal assignment statement is shown below.

Syntax

```
concurrent_signal_assignment_statement ::=
    [ label : ] [ postponed ] conditional_signal_assignment
    | [ label : ] [ postponed ] selected_signal_assignment

conditional_signal_assignment ::=
    target <= options conditional_waveforms ;

selected_signal_assignment ::=
    with expression select
        target <= options selected_waveforms ;

options ::= [ guarded ] [ delay_mechanism ]
```

Limitations on Concurrent Signal Assignment Statements

- Postponed concurrent signal assignments are not supported
- The option guarded is not supported

- Target aggregates are not supported.
- No check is made that the value of the waveform element belongs to the subtype of the signal.

Generate Statements

A generate statement provides a mechanism for iterative or conditional elaboration of a portion of a description. The syntax for a generate statement is shown below.

Syntax

```
generate_statement ::=
  generate_label :
    generation_scheme generate
    [ { block_declarative_item }
    begin ]
    { architecture_statement }
    end generate [ generate_label ] ;

block_declarative_item ::=
  subprogram_declaration
  | subprogram_body
  | type_declaration
  | subtype_declaration
  | constant_declaration
  | signal_declaration
  | shared_variable_declaration
  | file_declaration
  | alias_declaration
  | component_declaration
  | attribute_declaration
  | attribute_specification
  | configuration_specification
  | disconnection_specification
  | step_limit_specification
  | use_clause
  | group_template_declaration
  | group_declaration
  | nature_declaration
  | subnature_declaration
  | quantity_declaration
  | terminal_declaration

generation_scheme ::=
  for generate_parameter_specification
  | if condition

label ::= identifier
```

Limitations on Generate Statements

- alias_declaration

Object aliases of signals and terminals (only scalar declarations or elements of a composite declaration of these objects) are supported.

- Aliases of file objects, quantity objects and composite objects are not supported.
- Non-object aliases are not supported.
- `attribute_declaration` and `attribute_specification`

These are allowed, but access to attributes is not supported.

- `architecture_statement`

Must correspond to generate statements, to entity instantiations, or to component instantiations.

Concurrent Break Statements

The concurrent break statement represents a process containing a break statement.

```
concurrent_break_statement ::=
    [ label : ] break [ break_list ] [ sensitivity_clause ]
    [ when condition ];
```

The break list in a concurrent break statement is not supported in Questa ADMS.

Related Topics

Simultaneous Statements

Identifiers

Identifiers are used as names, and also as reserved words.

```
identifier ::= basic_identifier | extended_identifier
```

A basic identifier consists only of letters, digits, and underlines. Extended identifiers may contain graphic characters.

Questa ADMS does not support the following characters in extended identifiers:

```
@, (space), "
```

Related Topics

Names

Predefined Attributes

Predefined attributes denote values, functions, types, and ranges associated with various kinds of named entities.

The following predefined attributes are limited in Questa ADMS:

- T'IMAGE is not supported for types defined in subprograms.
- The predefined attributes E'SIMPLE_NAME, E'INSTANCE_NAME and E'PATH_NAME are only supported for constants, quantities, signals, terminals, files, labels and variables declared in an entity or architecture. The following prefixes are not supported:
 - entity, architecture, configuration, literal, units, group, nature, subnature, type, subtype, function, procedure, package, component.

Not supported for objects declared inside functions or procedures.

- Attributes of composite port quantities are not supported.

The following predefined attributes are not supported in Questa ADMS:

- S'DRIVING
- S'DRIVING_VALUE
- T'CONTRIBUTION
- T'TOLERANCE
- Q'TOLERANCE

Related Topics

Declarations

Simultaneous Statements

Simultaneous statements express explicit differential and algebraic equations. Together with implicit equations, simultaneous statements constrain the values of the quantities of a model.

The syntax for a simultaneous statement is shown below. Items in

red

are not supported in Questa ADMS. The items shown in

amber

are supported with limitations.

Syntax

```
simultaneous_statement_part ::=
    { simultaneous_statement }

simultaneous_statement ::=
    simple_simultaneous_statement
    | simultaneous_if_statement
    | simultaneous_case_statement
    | simultaneous_procedural_statement
    | simultaneous_null_statement
```

Limitations on Simple Simultaneous Statements

- In a simple simultaneous statement, `tolerance_aspect` is accepted by Questa ADMS without error, but this functionality is not supported.

```
simple_simultaneous_statement ::=
    [ label : ] simple_expression == simple_expression [ tolerance_aspect
    ] ;
```

- Equations with the same `simple_expression` on both sides are not supported, for example, the following is not allowed:

```
q == q
```

Limitation on Simultaneous if Statements

A simultaneous if statement selects for evaluation one of the enclosed simultaneous statement parts depending on the value of one or more conditions.

```
simultaneous_if_statement ::=  
  [ if_label : ]  
    if condition use  
      simultaneous_statement_part  
    { elsif condition use  
      simultaneous_statement_part }  
  [ else  
    simultaneous_statement_part ]  
  end use [ if_label ] ;
```

In Questa ADMS, the else clause is mandatory, because simultaneous if statements must be “balanced”. The statement lists of the alternative parts of a given simultaneous if statement must all generate the same number of characteristic expressions, regardless of conditions. For example, if the statement lists contain only simple simultaneous statements with scalar expressions, each alternative part must have the same number of simple simultaneous statements.

Related Topics

[Concurrent Statements](#)

C. Verilog-AMS Subset Definition

The current implementation of Verilog-AMS in Questa ADMS is limited to a subset of the Accellera Verilog-AMS 2.4 standard.

The implementation of Verilog-AMS in Questa ADMS does not contain all of the features of the language defined by the Accellera¹ Verilog-AMS Language Reference Manual (LRM). In addition, some parts of the Verilog-AMS language have been extended further in Questa ADMS.

This appendix describes the areas of the Verilog-AMS language where support is limited in Questa ADMS, and the areas where support is extended beyond the LRM definition.

Tip

The Accellera *Verilog-AMS Language Reference Manuals* are available from:

<http://www.accellera.org/downloads/standards/v-ams/>

Verilog-AMS Characteristics in Questa ADMS..... C-1

Verilog-AMS Extensions..... C-3

Reserved Keywords..... C-8

Table-Based Interpolation and Lookup System Functions.....C-8

Verilog-AMS Characteristics in Questa ADMS

Some parts of the Verilog-AMS language have been extended further in Questa ADMS.

In comparison with the full Verilog-AMS language, the following characteristics exist in Questa ADMS:

- A design file can mix Verilog-AMS and Verilog-D modules. It must be compiled using the **valog** command.

Verilog-AMS Case Sensitivity

By default, Questa ADMS does not support case sensitive Verilog-AMS models. Errors will be reported at elaboration in the following cases:

- In a Verilog-AMS region, when two instance names differ only in case
- In a Verilog-AMS region, when two electrical names differ only in case
- In a Verilog-AMS region, when two branch names differ only in case.

¹ Accellera Organization Inc.

- In a Verilog-AMS region, when two wire names differ only in case, and when these two wires are both resolved into electrical.

A `vasim` command line option enables the SPICE solver case sensitivity:

```
vasim -eldoopt -case
```

When this is used, errors will no longer be displayed and the Verilog-AMS models as well as the SPICE netlist become case sensitive.

Verilog-AMS Limitations in Questa ADMS

- The only discrete discipline supported is the built-in logic discipline.
- The `$param` system function call only supports numerical values.
- Arrays of instances are not supported
- Accessing continuous nets and variables in digital continuous assignments is not supported.
- Parameters of array types are not supported in:
 - Digital blocks
 - Digital blocks of mixed modules
 - Pure digital modules.

They are supported in the interface of mixed modules and can be used within analog blocks.

- Analog functions can have in, inout and/or output arguments. However all the terms in the analog expression must be in the analog domain.
- The analog function identifier in a `$limit` function call must be one of the built-in functions `fetlim` or `pnjlim`. It cannot be a user-defined function (*Verilog-AMS Language Reference Manual, section 9.17.3*).
- A digital net or a register cannot have a discrete discipline.
- The seed for random functions must be a digital variable in a digital block or an analog variable in an analog block.
- Only the opening modes "read", "write append" and "write replace" are supported on `$fopen`.
- Minimum, typical and maximum delay expressions are not supported in analog blocks and Verilog-AMS parameters.
- The parent nature of a nature declaration must not be an inherited nature. , Discipline parent for nature is not supported, i.e. an inherited nature; for example:
- If a nature is given a parent nature in its nature declaration, it cannot be in the form:

```
<discipline>.potential
```


- The following construction for a derivative operator is not supported:

```
ddx(ddx(expr, unknown_quantity))
```

- String variables are supported with the following limitations:
 - \$sscanf and \$fscanf of %s are not supported
 - Arrays of strings and member access (i.e., reading or writing a character in a string) are not supported
 - When assigning to string variables in the GUI, you cannot assign content that contains a comma or a space.
- Named events are supported with the following limitations:
 - Only scalar named events are supported
 - Hierarchical references to named events are not supported.
- Generate constructs are not supported (*Verilog-AMS Language Reference Manual, section 6.6*).

Limitations on Ports

- Concatenated port declarations are not supported.
- Modules instantiated from Verilog-AMS must not contain unnamed ports. Be aware that putting the slice in the port list makes the port “unnamed”.

Related Topics

[Verilog-AMS Extensions](#)

[Working with Verilog-AMS](#)

[Reserved Keywords](#)

[Table-Based Interpolation and Lookup System Functions](#)

Verilog-AMS Extensions

The support of Verilog-AMS in Questa ADMS has been extended further than that explicitly defined in the Verilog-AMS Language Reference Manual.

- SPICE models (.MODEL), subcircuits (.SUBCKTS) and primitives may be directly instantiated from Verilog-AMS. The models, subcircuits and parameters must be included in the command file.
- The \$simparam() function queries the simulator for the value of a simulation parameter. The built-in simulation parameters are:

ABSTOL, CHGTOL, GMIN, GDEV, ITERATION, RELTOL, SCALE, SHRINK, SIMULATORVERSION, SIMULATORSUBVERSION, SOURCESCALEFACTOR, TNOM, TEMP, VNTOL.

For example:

```
$simparam("TEMP")
```

Refer to [Access to Simulation Parameter Values](#) for more information.

- The *m* factor can be used on SPICE primitives and subcircuits in Verilog-AMS. It is used in Verilog-AMS as a Verilog-AMS parameter. For example:

```
mresistor #(.m(3)) Rm (.pport(b2), .nport(b1))
```

where *mresistor* is the SPICE subcircuit, and *Rm* is the instance name.

- The multiplicity factor attributes *mult*, *passed_mfactor* and *inherited_mfactor* are supported in Verilog-AMS to place multiple instances of Verilog-AMS modules, SPICE primitives and SPICE subcircuits in parallel. For example:

```
resistor #(.r(10k)) (* passed_mfactor= 2; *) R1  
(.p(pport), .n(nport));  
resistor #(.r(10k)) (* mult= 2; *) R1 (.p(pport), .n(nport));
```

where *resistor* is the SPICE primitive, and *R1* is the instance name.

- The *vdc* and *idc* ideal sources are supported as SPICE primitives in Verilog-AMS (see [Table 4-1](#)). For example:

```
vdc #(.dc(5), .mag(1), .phase(0)) vs1 (.pport(a), .nport(b));
```

Untyped Wire and Vector Wire

When the type (analog, digital or mixed signal) is omitted from the definition of a either a scalar or vector wire, Questa ADMS will resolve the wire to either a terminal, a signal or a mixed signal connection. Questa ADMS will resolve the wire using its connectivity. For example, if the wire is only connected to an analog terminal the wire will be resolved to a terminal.

Example

The following example demonstrates how Questa ADMS will resolve each index of an untyped wire vector, i.e. the wire vector has not been defined as an analog, digital or mixed-signal connection, when they are connected to either an analog or digital port. This example uses a top-level Verilog-AMS module that instantiates a Verilog module and a Verilog-A module. In the top Verilog-AMS module the vector wire *x* is not defined as an analog or digital connection, the vector wire is connected to ports B1 and B2 of the Verilog-A module *anaB*. The vector wire is also connected to ports A1 and A2 of the Verilog module *digA*. The first and second index of the vector wire *x* will be resolved as analog terminals, and the third and fourth index of the vector wire *x* will be resolved as digital signals. The syntax for the modules are shown below:

- Top module (Verilog-AMS):

```
`include "disciplines.vams"

module toto;
    wire [4:1] x;
    digA inst1 (x[4], x[3]);
    anaB inst2 (x[2], x[1]);
endmodule
```

- Pure analog module (Verilog-A):

```
module anaB (B1, B2);
    output B1, B2;
    electrical B1, B2;

    analog begin
        V(B1) <+ 5.0;
        V(B2) <+ 0;
    end
endmodule
```

- Digital module (Verilog):

```
// Digital module

module digA (A1, A2);
    output A1;
    input A2;
    reg A1;

    always
        #1 A1 = A2;

endmodule
```

Untyped Ports and Port Vector

When the type (analog, digital or mixed signal) is omitted from the definition of either a scalar port or port vector, Questa ADMS will resolve the port to either a terminal, a signal or a mixed-signal connection. Questa ADMS will resolve the port using its connectivity; for example if the port is connected to an analog terminal the port will be resolved to an analog port.

Example

The following example demonstrates how Questa ADMS will resolve a port vector that has not been defined as an analog, digital or mixed-signal connection. This example uses a top-level Verilog-AMS module that instantiates a Verilog module and a Verilog-A module. In the top Verilog-AMS module the port vector *x* is not defined as an analog or digital connection, the port vector is connected to ports *B1* and *B2*

of the Verilog-A module anaB. The port vector is also connected to ports A1 and A2 of the Verilog module digA. The first and second index of the port vector x will be resolved as analog terminals, and the third and fourth index of the port vector x will be resolved as digital signals. The syntax for the modules are shown below:

- Top module (Verilog-AMS):

```
`include "disciplines.vams"

module toto (x);
    input [4:1] x;
    digA inst1 (x[4], x[3]);
    anaB inst2 (x[2], x[1]);
endmodule
```

- Pure analog module (Verilog-A):

```
module anaB (B1, B2);
    output B1, B2;

    analog begin
        V(B1) <+ 5.0;
        V(B2) <+ 0;
    end
endmodule
```

- Digital module (Verilog):

```
// Digital module

module digA (A1, A2);
    output A1;
    input A2;
    reg A1;

    always
        #1 A1 = A2;
endmodule
```

SystemVerilog Assertion Declarations

Questa ADMS supports extensions to the System Verilog sub-language of Verilog-AMS, by adding grammar constructs to access voltages and electrical objects. This facilitates the declaration of **SystemVerilog Assertions**.

The following limitations exist:

- Increment/decrement operators don't exist in Verilog-AMS (*i++* or *i--*). Therefore the syntax shown in *italic* is not supported:

```
sequence_match_item_31 ::=
    operator_assignment_87
    | inc_or_dec_expression
    | subroutine_call
```

- Neither associative dimensions or queues are available in Verilog-AMS. Therefore the syntax shown in *italic* is not supported:

```
unsized_dimension
| unpacked_dimension
| associative_dimension
| queue_dimension
```

- Neither variable dimensions nor classes are available in Verilog-AMS. Therefore the syntax shown in *italic* is not supported:

```
variable_decl_assignment ::=
    variable_identifier { variable_dimension } [ = expression ]
    | dynamic_array_variable_identifier unsized_dimension
      { variable_dimension } [ = dynamic_array_new ]
    | class_variable_identifier [ = class_new ]
```

Related Topics

[SystemVerilog Assertions](#)

[Working with Verilog-AMS](#)

[Verilog-AMS Characteristics in Questa ADMS](#)

Reserved Keywords

Reserved keywords may not be used in Verilog-AMS code.

automatic	cell	config
design	endconfig	endgenerate
generate	genvar	incdir
include	instance	liblist
library	localparam	noshowcancelled
pulsestyle_onevent	pulsestyle_ondetect	showcancelled
signed	unsigned	use
uwire		

Table-Based Interpolation and Lookup System Functions

Questa ADMS supports Verilog-AMS interpolation and lookup functions. The interpolation and lookup functions are designed to operate on parametrically swept data.

Overview of the \$table Functions..... C-8

\$table_model.....C-11

\$table1f..... C-12

\$table1a.....C-14

\$table2f..... C-15

\$table2a.....C-16

\$table3f..... C-18

\$table3a.....C-19

Overview of the \$table Functions

You can use the Verilog-AMS \$table system functions to calculate a smoothed, interpolated value from a static table of discrete points.

The data table itself may be expressed as a memory or array but it can also be contained in an external file. The independent variable may lie within the range of the table or outside of it. If the independent variable lies outside of the table, a value is extrapolated. A \$table() function can be used in any expression where a real value is allowed.

Note

The functions \$table1f, \$table1a, \$table2f, \$table2a, \$table3f, and \$table3a are supported for legacy use only. For new designs, the use of **\$table_model** is recommended.

ASCII Tables Format

The table-based routines \$table1f, \$table2f and \$table3f require the data to be tabulated in an ASCII data file. This data file should have a specific format.

An ASCII data file of n columns assumes that the first $n-1$ columns are independent variables, while the last column is the dependent one. Therefore an ASCII data file of two columns assumes that the first column is the independent variable, while the second column is the dependent one. An ASCII data file of three columns assumes that the first and second columns are the first and second independent variables, and the third one is assumed to be the dependent variable. The parsing algorithm assumes the following about these data files:

- Empty lines in the ASCII tabulated data file are ignored.
- Columns are separated with white spaces (tabs or spaces).
- Comments are preceded with the hash character "#".
- It is optional to have the first line in the file as a header, but it has to be commented using a hash "#".
- A valid data line may start with a white space.

An ASCII data file of three or more columns consists of sections. Each section has a constant first independent variable value. The first column is identical along all rows of the same section. The parsing algorithm assumes the following about the three or more column data files:

- The number of rows in all sections is a constant (4 in the example below).
- The number of sections equals the number of distinct first independent variable values.
- The second independent variable values (in the second column) are identical from one section to the other (-1.2, -1.3, -1.9, -2.6 in the \$table2f example below).

Examples:

- Data file for **\$table1f** system function is composed of two columns with the following format:

```
#sample data
#Independent Variable
# Dependent Variable
0.10    3.30
0.20    3.31
0.30    3.32
0.40    3.33
0.50    66.6
0.60    70.6
```

```
0.70    -100.3
0.80     0.0
0.90     1.0
1.00     2.0
```

- Data file for [\\$table2f](#) system function is composed of three columns with the following format:

```
#sample data
#Independent Variable 1
#      Independent Variable 2
#      Dependent Variable
#first section
0.1    -1.2    12.0
0.1    -1.3     8.0
0.1    -1.9    5.54
0.1    -2.6    98.1

#second section
0.2    -1.2   -52.0
0.2    -1.3    78.2
0.2    -1.9    54.0
0.2    -2.6   -54.0

#third section
0.3    -1.2    87.21
0.3    -1.3   -8.021
0.3    -1.9   -87.0
0.3    -2.6    32.37
```

- Data file for [\\$table3f](#) system function is composed of four columns with the following format:

```
#sample data
#Independent Variable 1
#      Independent Variable 2
#      Independent Variable 3
#      Dependent Variable
#first section
1      1      1      3
1      1      2      5
1      1      4      6
1      2      1      7
1      2      3      5
1      3      1      4
1      3      2      6
1      3      4      7
#second section
2      1      1      4
2      1      3      6
2      1      4      7
2      2      1      8
2      2      3      6
2      3      1      5
```


2	3	2	7
2	3	5	8

Related Topics

[LOOK_UP_TABLE_V1X](#)

\$table_model

\$table_model is a multidimensional interpolation and lookup function, designed to operate on parametrically swept data, commonly generated by most analog simulators.

This type of data is generated when simulating a system while varying (sweeping) a parameter across a range. Data dimensionality increases when parameter sweeps are nested. While the samples are those of a multidimensional function, sample generation via parametric sweeping leads to a simple recursive interpolation and extrapolation process defined by the \$table_model function.

Tip

Environment variables may be used in \$table functions.

To locate the source file, \$table_model looks in the directory relative to the simulation run, and if the file is not found there, it searches the directory relative to the Verilog source.

Limitations

- Ignoring independent variables in the input data (I in control string) is not supported
- Selecting the dependent variable column (<number> in control string) is not supported.
- When a call to \$table_model is made from the digital part of a Verilog-AMS module, you must use a file as the input for interpolation (array input data is not supported).

Related Topics

[Verilog-AMS Extensions](#)

[Working with Verilog-AMS](#)

\$table1f

Returns the interpolated or extrapolated value corresponding to the given value of a single independent variable. The mapping between the dependent variable and the independent variable is specified in an ASCII file.

Syntax

```
$table1f(x_var, path, method)
```

Parameters

- **x_var**
Independent real variable value.
- **path**
ASCII file path.
- **method**
Integer flag for interpolation method: 1 for linear, 3 for natural cubic spline; if method does not equal 1 or 3, an error is issued.

Description

See [LOOK_UP_TABLE_V1X](#).

Examples

The following is an example of a voltage-controlled oscillator (VCO) described in Verilog-AMS. The value of $V(t1)$ is extrapolated/interpolated from the data listed in the file *inp.dat* using the value of $V(in)$.

```
module vco1(vin, vout, t1);
    inout vin, vout, t1;
    electrical vin, vout, t1;
    parameter real amp = 1.0;
    parameter real center_freq = 2.5e3;
    parameter real vco_gain = 1.0e4;
    real phase;
    analog begin
        V(t1) <+ $table1f(V(vin), "inp.dat", 3);
        phase = idt(center_freq + vco_gain*V(t1));
        V(vout) <+ amp*abs(sin(6.28*phase));
    end
endmodule

#Input data file inp.dat
```

```
y_var f(y_var)
0.0 1.0 # first pair
1.0 1.0 # second pair
2.0 1.1 # third pair
3.0 2.0 # fourth pair
3.5 4.0 # fifth pair
5.0 4.9 # sixth pair
6.1 5.0 # seventh pair
```

When $V(\text{in})$ is 0.0, $V(\text{t1})$ will be assigned the value 1.0 defined by the first pair in the input data file *inp.dat*, when $V(\text{in})$ is 1.0, $V(\text{t1})$ is also 1.0 defined by the second pair.

Related Topics

[Overview of the \\$table Functions](#)

\$table1a

Returns the interpolated or extrapolated value corresponding to the given value of a single independent variable. The mapping between the dependent variable and the independent variable is specified using arrays.

Syntax

```
$table1a(x_var, x_array, y_array, method)
```

Parameters

- **x_var**
Real variable used to determine value in x_array.
- **x_array**
Parameter array of real values.
- **y_array**
Parameter array of real values representing fct(x_array).
- **method**
Integer flag for interpolation method: 1 for linear, 3 for natural cubic spline; if method does not equal 1 or 3, an error is issued.

Description

See [LOOK_UP_TABLE_V1X](#).

\$table2f

Returns the interpolated or extrapolated value corresponding to the given values of two independent variables. The mapping between the dependent variables and the independent variable is specified in an ASCII file.

Syntax

```
$table2f(x_var, y_var, path, method)
```

Parameters

- **x_var**
First independent real variable value.
- **y_var**
Second independent real variable value.
- **path**
ASCII file path.
- **method**
Integer flag for interpolation method: 1 for linear, 2 for quadratic spline; if method does not equal 1 or 2, an error is issued.

Description

See [LOOK_UP_TABLE_V1X](#).

Related Topics

[Overview of the \\$table Functions](#)

\$table2a

Returns the interpolated or extrapolated value corresponding to the given values of two independent variables. The mapping between the dependent variables and the independent variable is specified using arrays.

Syntax

```
$table2a(x_var, y_var, x_array, y_array, z_array, method)
```

Parameters

- **x_var**
First independent real variable value used to determine value in x_array.
- **y_var**
Second independent real variable value used to determine value in y_array.
- **x_array**
First independent variable parameter array of real values.
- **y_array**
Second independent variable parameter array of real values.
- **z_array**
Dependent variable array.
- **method**
Integer flag for interpolation method: 1 for linear, 2 for quadratic spline, if method does not equal 1 or 2, then an error is issued.

Description

See [LOOK_UP_TABLE_V1X](#).

Examples

The following is an example of a voltage-controlled oscillator (VCO) described in Verilog-AMS. The value of V(t2) is extrapolated/interpolated from the data specified on the parameters x2, y2, d2, and the value of V(in) and V(vdd).

```
module vco2(in, out, vdd, t2);
    in in, vdd;
    out out;
    inout t2;
```

```

electrical in, out, vdd, t2;
parameter real amp=1.0,center_freq=2.5e3, vco_gain=1.0e4;
real phase;
parameter real x2[0:13]={1.0, 1.0, 1.0,
                          2.0, 2.0,
                          3.0, 3.0, 3.0,
                          4.0, 4.0, 4.0,
                          5.0, 5.0, 5.0};
parameter real y2[0:13]={3.5, 4.5, 5.0,
                          4.0, 5.0,
                          3.3, 3.8, 4.8,
                          3.5, 4.3, 4.9,
                          3.8, 4.0, 5.0};
parameter real d2[0:13]={1.0e6, 1.1e6, 1.2e6,
                          2.0e6, 2.1e6,
                          3.0e6, 3.1e6, 3.2e6,
                          3.5e6, 3.6e6, 3.7e6,
                          4.0e6, 4.1e6, 4.2e6};

analog begin
    V(t2) <+ $table2a(V(in),V(vdd), x2,y2,d2, 1);
    phase = idt(center_freq + vco_gain*V(t2));
    V(out) <+ amp*abs(sin(`M_TWO_PI*phase));
end
endmodule

```

When V(in) is 1.0, and V(vdd) is 3.5, V(t2) will be assigned the value 1.0×10^6 .

\$table3f

Returns the interpolated or extrapolated value corresponding to the given values of three independent variables. The mapping between the dependent variables and the independent variable is specified in an ASCII file.

Syntax

```
$table3f(w_var, x_var, y_var, path, method)
```

Parameters

- w_var
First independent real variable value.
- x_var
Second independent real variable value.
- y_var
Third independent real variable value.
- path
ASCII file path.
- method
Integer flag for interpolation method: 1 for linear, 2 for quadratic spline; if method does not equal 1 or 2, an error is issued.

Description

See [\\$table_model](#).

Related Topics

[Overview of the \\$table Functions](#)

\$table3a

Returns the interpolated or extrapolated value corresponding to the given values of three independent variables. The mapping between the dependent variables and the independent variable is specified using arrays.

Syntax

```
$table3a(w_var, x_var, y_var, w_array, x_array, y_array, z_array, method)
```

Parameters

- **w_var**
First independent real variable value.
- **x_var**
Second independent real variable value.
- **y_var**
Third independent real variable value.
- **w_array**
First independent variable parameter array of real values.
- **x_array**
Second independent variable parameter array of real values.
- **y_array**
Third independent variable parameter array of real values.
- **z_array**
Dependent variable array.
- **method**
Integer flag for interpolation method: 1 for linear, 2 for quadratic spline; if method does not equal 1 or 2, an error is issued.

Description

See [\\$table_model](#).

D. C Template and Reference Tables

You can compile C templates.

- General C Template in Questa ADMS..... D-2
- Correspondence Between C Type and VHDL-AMS Type..... D-4
- Predefined Macros.....D-5
- macro.h File.....D-9
- Encapsulated C Function Example..... D-12

General C Template in Questa ADMS

The general C template shows the structure of the C template.

```
#####
#include "macro.h"

/*****
/*      MANUAL MODIFICATIONS      */
*****/
/* the file containing the prototype of the C encapsulated */
/* functions has to be included */

#include <C_function_prototype_file>
/*****
/* END OF MANUAL MODIFICATIONS      */
*****/

extern PROC_CAST current_active_process;
RG_DECL;
VPTR_DECL;
LOOP_I_DECL;
LOOP_DRIVER_DECL;
static void elab_desent();
static int dblines[0]={};

/*****
/*      MANUAL MODIFICATIONS      */
*****/
/* A C predeclaration corresponds to each VHDL-AMS */
/* subprogram */

static <TYPE_FUNC_Image> <name_of_VHDL_subprogram__1> ();
static <TYPE_FUNC_Image> <name_of_VHDL_subprogram__2> ();
.
.
/*****
/* END OF MANUAL MODIFICATIONS      */
*****/

/*****
/*      MANUAL MODIFICATIONS      */
*****/
/* Table of VHDL-AMS subprograms C image names */
/* It is mandatory that the order of these subprograms is */
/* the same as the order of the declarations of the */
/* VHDL-AMS subprograms in the package */
CODE_SUPRGS_DECL=
{ (CODE_CAST) <name_of_VHDL_subprogram__1>, (CODE_CAST)
<name_of_VHDL_subprogram__2>, .. };

/*****
/* END OF MANUAL MODIFICATIONS      */
*****/
```

```

PKG_DECL(0);
ELB_CODE_CAST elab_funcs=
{elab_desent,0,(void*)code_suprgs,NULL};
static void elab_desent(int* ENVIR_CAST,int selector){
    switch(selector){
        case -1: /* Analog Code and Subprogram Definition */
            break;
        case 0: /* block: design entity */
            ALLOC_RG;
            break;
        default:
            /* Error Management */
            break;
    }
}
/*****
/*      MANUAL MODIFICATIONS      */
*****/
/* body of the VHDL-AMS subprogram C image */
/* the following body has to be defined for each VHDL-AMS */
/* subprogram */

static <TYPE_FUNC_Image> <name_of_VHDL_subprogram__i> /(<Param_MACRO>
(0),...,<Param_MACRO>(i), PKG_DECL){
    declarative_MACROi(var2, i);
    .
    .
    <declarative_MACROi>(vark, i);
    .
    .
    <alias_MACROi>(var, i);
    .
    .
    TYPE_var1 var;
    .
    .
    <TYPE_var>  varl;
    .
    .
    Funci_C (varj,...,vark);

    <assignment_MACROi>(varj, i);

} /* end <name_of_VHDL_subprogram__i> */
/*****
/* END OF MANUAL MODIFICATIONS      */
*****/

/* end of C template */
#####

```

Correspondence Between C Type and VHDL-AMS Type

VHDL-AMS scalar object types and 1-dimensional array object types correspond to C types.

Table D-1: Case of scalar object

VHDL-AMS FUNCTION TYPE	C Image TYPE: <TYPE_FUNC_Image>
REAL	double
INT	long long
PHYS	long long
ENUM	long

Table D-2: Case of 1-dimensional array object

VHDL-AMS FUNCTION TYPE	C Image TYPE: <TYPE_FUNC_Image>
1-dimensional array of REAL	double *
INT	long long *
PHYS	long long *
ENUM (<256)	char *
ENUM (>256)	long *

Predefined Macros

Questa ADMS provides macros for use in the C template.

Table D-3: Case of CONSTANT SCALAR Parameters of Mode IN

VHDL-AMS TYPE	C TYPE	param macro: <Param_MACRO>	declarative macro: <declarative_MACROi>
REAL	double	FRGN_PARAM_CON_REAL	FRGN_DECL_CON_REAL
INT	long long	FRGN_PARAM_CON_INT	FRGN_DECL_CON_INT
INT	long	FRGN_PARAM_CON_INT	FRGN_DECL_CINT_CON_INT
PHYS	long long	FRGN_PARAM_CON_PHYS	FRGN_DECL_CON_PHYS
PHYS(time)	double	FRGN_PARAM_CON_PHYS	FRGN_DECL_CDOUBLE_CON_TIME
ENUM	long	FRGN_PARAM_CON_ENUM	FRGN_DECL_CON_ENUM

Table D-4: Case of VARIABLE SCALAR Parameters of Mode IN

VHDL-AMS TYPE	C TYPE	param macro: <Param_MACRO>	declarative macro: <declarative_MACROi>
REAL	double	FRGN_PARAM_VAR_REAL	FRGN_DECL_VAR_REAL
INT	long long	FRGN_PARAM_VAR_INT	FRGN_DECL_VAR_INT
INT	long	FRGN_PARAM_VAR_INT	FRGN_DECL_CINT_VAR_INT
PHYS	long long	FRGN_PARAM_VAR_PHYS	FRGN_DECL_VAR_PHYS
PHYS(time)	double	FRGN_PARAM_VAR_PHYS	FRGN_DECL_CDOUBLE_VAR_TIME
ENUM	long	FRGN_PARAM_VAR_ENUM	FRGN_DECL_VAR_ENUM

Table D-5: Case of SIGNAL SCALAR Parameters of Mode IN

This functionality is not implemented in the current release of Questa ADMS

Table D-6: Case of VARIABLE SCALAR Parameters of Mode OUT/INOUT

VHDL-AMS TYPE	C TYPE	param macro: <Param_MACRO>	declarative macro: <declarative_MACROi>
REAL	double	FRGN_PARAM_VAR_OUT_REAL	FRGN_ASSIGN_VAR_OUT
INT	long long	FRGN_PARAM_VAR_OUT_INT	FRGN_ASSIGN_VAR_OUT
PHYS	long long	FRGN_PARAM_VAR_OUT_PHYS	FRGN_ASSIGN_VAR_OUT
ENUM	long	FRGN_PARAM_VAR_OUT_ENUM	FRGN_ASSIGN_VAR_OUT

Table D-7: Case of SIGNAL SCALAR Parameters of Mode OUT/INOUT

This functionality is not implemented in the current release of Questa ADMS

Table D-8: Case of CONSTANT 1-DIMENSION ARRAY Parameters of Mode IN

VHDL-AMS TYPE	C TYPE	param macro: <Param_MACRO>	declarative macro: <declarative_MACROi>
constr. REAL	double *	FRGN_PARAM_CON_ARR	FRGN_DECL_CON_ARR_REAL
unconstr. REAL	double *	FRGN_PARAM_CON_UARR	FRGN_DECL_CON_ARR_REAL
constr. PHYS	long long *	FRGN_PARAM_CON_ARR	FRGN_DECL_CON_ARR_PHYS
unconstr. PHYS	long long *	FRGN_PARAM_CON_UARR	FRGN_DECL_CON_ARR_PHYS
constr. INT	long long	FRGN_PARAM_CON_ARR	FRGN_DECL_CON_ARR_INT
unconstr. INT	long long *	FRGN_PARAM_CON_UARR	FRGN_DECL_CON_ARR_INT
constr. ENUM	char *	FRGN_PARAM_CON_ARR	FRGN_DECL_CCHAR_CON_ARR_ENUM
unconstr. ENUM < 256 char.	char *	FRGN_PARAM_CON_UARR	FRGN_DECL_CCHAR_CON_ARR_ENUM
constr. ENUM	long *	FRGN_PARAM_CON_ARR	FRGN_DECL_CINT_CON_ARR_ENUM
unconstr. ENUM > 256 char.	long *	FRGN_PARAM_CON_UARR	FRGN_DECL_CINT_CON_ARR_ENUM

The distinction between constrained "constr." 1-dimension arrays and unconstrained "unconstr." 1-dimension arrays has been made

Table D-9: Case of VARIABLE 1-DIMENSION ARRAY Parameters of Mode IN

VHDL-AMS TYPE	C TYPE	param macro: <Param_MACRO>	declarative macro: <declarative_MACROi>
constr. REAL	double *	FRGN_PARAM_VAR_ARR	FRGN_ALIAS_VAR_ARR_REAL
unconstr. REAL	double *	FRGN_PARAM_VAR_UARR	FRGN_ALIAS_VAR_ARR_REAL
constr. PHYS	long long *	FRGN_PARAM_VAR_ARR	FRGN_ALIAS_VAR_ARR_PHYS
unconstr. PHYS	long long *	FRGN_PARAM_VAR_UARR	FRGN_ALIAS_VAR_ARR_PHYS
constr. INT	long long *	FRGN_PARAM_VAR_ARR	FRGN_ALIAS_VAR_ARR_INT
unconstr. INT	long long *	FRGN_PARAM_VAR_UARR	FRGN_ALIAS_VAR_ARR_INT
constr. ENUM	char *	FRGN_PARAM_VAR_ARR	FRGN_ALIAS_CCHAR_VAR_ARR_ENUM
unconstr. ENUM < 256 char.	char *	FRGN_PARAM_VAR_UARR	FRGN_ALIAS_CCHAR_VAR_ARR_ENUM
constr. ENUM	long *	FRGN_PARAM_VAR_ARR	FRGN_ALIAS_CINT_VAR_ARR_ENUM
unconstr. ENUM > 256 char.	long *	FRGN_PARAM_VAR_UARR	FRGN_ALIAS_CINT_VAR_ARR_ENUM

The distinction between constrained “constr.” 1-dimension arrays and unconstrained “unconstr.” 1-dimension arrays has been made:

Table D-10: Case of SIGNAL 1-DIMENSION ARRAY Parameters of Mode IN

This functionality is not implemented in the current release of Questa ADMS

Table D-11: Case of VARIABLE 1-DIMENSION ARRAY Parameters of Mode OUT/INOUT

VHDL-AMS TYPE	C TYPE	param macro: <Param_MACRO>	declarative macro: <declarative_MACROi>
constr. REAL	double *	FRGN_PARAM_VAR_OUT_ARR	FRGN_ALIAS_VAR_OUT_ARR_REAL
unconstr. REAL	double *	FRGN_PARAM_VAR_OUT_UARR	FRGN_ALIAS_VAR_OUT_ARR_REAL
constr. PHYS	long long *	FRGN_PARAM_VAR_OUT_ARR	FRGN_ALIAS_VAR_OUT_ARR_PHYS
unconstr. PHYS	long long *	FRGN_PARAM_VAR_OUT_UARR	FRGN_ALIAS_VAR_OUT_ARR_PHYS

Table D-11: Case of VARIABLE 1-DIMENSION ARRAY Parameters of Mode OUT/INOUT (continued)

VHDL-AMS TYPE	C TYPE	param macro: <Param_MACRO>	declarative macro: <declarative_MACROi>
constr. INT	long long *	FRGN_PARAM_VAR_OUT_ARR	FRGN_ALIAS_VAR_OUT_ARR_INT
unconstr. INT	long long *	FRGN_PARAM_VAR_OUT_UARR	FRGN_ALIAS_VAR_OUT_ARR_INT
constr. ENUM	char *	FRGN_PARAM_VAR_OUT_ARR	FRGN_ALIAS_CCHAR_VAR_OUT_ARR_ENUM
unconstr. ENUM < 256 char.	char *	FRGN_PARAM_VAR_OUT_UARR	FRGN_ALIAS_CCHAR_VAR_OUT_ARR_ENUM
constr. ENUM	long *	FRGN_PARAM_VAR_OUT_ARR	FRGN_ALIAS_CINT_VAR_OUT_ARR_ENUM
unconstr. ENUM > 256 char.	long *	FRGN_PARAM_VAR_OUT_UARR	FRGN_ALIAS_CINT_VAR_OUT_ARR_ENUM

The distinction between constrained "constr." 1-dimension arrays and unconstrained "unconstr." 1-dimension arrays has been made.

Table D-12: Case of SIGNAL 1-DIMENSION ARRAY Parameters of Mode OUT/INOUT

This functionality is not implemented in the current release of Questa ADMS

macro.h File

The *macro.h* file contains predefined macros for use in the C template.

```
#####
#include <vams.h>

#define FRGN_PARAM_CON_REAL(i) \
DECL_FUNC_CON_P_REAL(i)
#define FRGN_PARAM_CON_INT(i) \
DECL_FUNC_CON_P_INT(i)
#define FRGN_PARAM_CON_PHYS(i) \
DECL_FUNC_CON_P_PHYS(i)
#define FRGN_PARAM_CON_ENUM(i) \
DECL_FUNC_CON_P_ENUM(i)
#define FRGN_PARAM_VAR_REAL(i) \
DECL_FUNC_VAR_REAL(i)
#define FRGN_PARAM_VAR_INT(i) \
DECL_FUNC_VAR_INT(i)
#define FRGN_PARAM_VAR_PHYS(i) \
DECL_FUNC_VAR_PHYS(i)
#define FRGN_PARAM_VAR_ENUM(i) \
DECL_FUNC_VAR_ENUM(i)
#define FRGN_PARAM_VAR_OUT_REAL(i) \
DECL_FUNC_VAR_REAL_PT(i)
#define FRGN_PARAM_VAR_OUT_INT(i) \
DECL_FUNC_VAR_INT_PT(i)
#define FRGN_PARAM_VAR_OUT_PHYS(i) \
DECL_FUNC_VAR_PHYS_PT(i)
#define FRGN_PARAM_VAR_OUT_ENUM(i) \
DECL_FUNC_VAR_ENUM_PT(i)
#define FRGN_PARAM_CON_ARR(i) \
DECL_FUNC_CON_P_ARR(i)
#define FRGN_PARAM_CON_UARR(i) \
DECL_FUNC_CON_P_UARR(i)
#define FRGN_PARAM_VAR_ARR(i) \
DECL_FUNC_VAR_ARR_P(i)
#define FRGN_PARAM_VAR_UARR(i) \
DECL_FUNC_VAR_UARR_P(i)
#define FRGN_PARAM_VAR_OUT_ARR(i) \
DECL_FUNC_VAR_ARR_PT(i)
#define FRGN_PARAM_VAR_OUT_UARR(i) \
DECL_FUNC_VAR_UARR_PT(i)
#define FRGN_DECL_CON_REAL(var,i) \
double var = READ_FUNC_CON_P_REAL(i)
#define FRGN_DECL_CON_INT(var,i) \
long long var = READ_FUNC_CON_P_INT(i)
#define FRGN_DECL_CINT_CON_INT(var,i) \
int var = (int)READ_FUNC_CON_P_INT(i)
#define FRGN_DECL_CON_PHYS(var,i) \
long long var = READ_FUNC_CON_P_PHYS(i)
#define FRGN_DECL_CDDOUBLE_CON_TIME(var,i) \
double var = (double)READ_FUNC_CON_P_PHYS(i) * 1.0e-15
#define FRGN_DECL_CON_ENUM(var,i) \
```

```

long var = READ_FUNC_CON_P_ENUM(i)
#define FRGN_DECL_VAR_REAL(var,i) \
double var = READ_FUNC_VAR_REAL(i)
#define FRGN_DECL_VAR_INT(var,i) \
long long var = READ_FUNC_VAR_INT(i)
#define FRGN_DECL_CINT_VAR_INT(var,i) \
int var = (int)READ_FUNC_VAR_INT(i)
#define FRGN_DECL_VAR_PHYS(var,i) \
long long var = READ_FUNC_VAR_PHYS(i)
#define FRGN_DECL_CDOUBLE_VAR_TIME(var,i) \
double var = (double)READ_FUNC_VAR_PHYS(i) * 1.0e-15
#define FRGN_DECL_VAR_ENUM(var,i) \
long var = READ_FUNC_VAR_ENUM(i)
#define FRGN_ASSIGN_VAR_OUT(name,i)\
*variable##i=name
#define FRGN_DECL_CON_ARR_REAL(var,i) \
double * var = READ_FUNC_CON_P_ARR(i)
#define FRGN_DECL_CON_ARR_PHYS(var,i) \
long long * var = READ_FUNC_CON_P_ARR(i)
#define FRGN_DECL_CCHAR_CON_ARR_ENUM(var,i) \
char * var = READ_FUNC_CON_P_ARR(i)
#define FRGN_DECL_CINT_CON_ARR_ENUM(var,i) \
long * var = READ_FUNC_CON_P_ARR(i)
#define FRGN_DECL_CON_ARR_INT(var,i) \
long long * var = READ_FUNC_CON_P_ARR(i)
#define FRGN_ALIAS_VAR_OUT_ARR_REAL(var,i) \
double * var = * (double **) variable##i
#define FRGN_ALIAS_VAR_OUT_ARR_PHYS(var,i) \
long long * var = * (long long **) variable##i
#define FRGN_ALIAS_CCHAR_VAR_OUT_ARR_ENUM(var,i) \
char * var = * (char **) variable##i
#define FRGN_ALIAS_CINT_VAR_OUT_ARR_ENUM(var,i) \
long * var = * (long **) variable##i
#define FRGN_ALIAS_VAR_OUT_ARR_INT(var,i) \
long long * var = * (long long **) variable##i
#define FRGN_ALIAS_VAR_ARR_REAL(var,i) \
double * var = (double *) variable##i
#define FRGN_ALIAS_VAR_ARR_PHYS(var,i) \
long long * var = (long long *) variable##i
#define FRGN_ALIAS_CCHAR_VAR_ARR_ENUM(var,i) \
char * var = (char *) variable##i
#define FRGN_ALIAS_CINT_VAR_ARR_ENUM(var,i) \
long * var = (long *) variable##i
#define FRGN_ALIAS_VAR_ARR_INT(var,i) \
long long * var = (long long *) variable##i
#define GET_CON_LENGTH(i) \
ARRAY_LENGTH_B(con_range_##i)
#define GET_CON_FIRST(i) \
ARRAY_LEFT_B_RANGE(con_range_##i)
#define GET_CON_LAST(i) \
ARRAY_RIGHT_B_RANGE(con_range_##i)
#define GET_CON_DIR(i) \
ARRAY_ASCENDING_B(con_range_##i)
#define GET_VAR_LENGTH(i) \
ARRAY_LENGTH_B(var_range_##i)
#define GET_VAR_FIRST(i) \

```

```
ARRAY_LEFT_B_RANGE(var_range__##i)
#define GET_VAR_LAST(i) \
ARRAY_RIGHT_B_RANGE(var_range__##i)
#define GET_VAR_DIR(i) \
ARRAY_ASCENDING_B(var_range__##i)
#####
```

Encapsulated C Function Example

The sample encapsulated C function has a scalar parameter of mode IN and returns a scalar object.

Files

The C function has been defined in the *C_function_ex.c* file:

```
#####
#include "C_prototype_ex.h"
double C_Compute(double v_in) {
    double v_output;
    v_output = 2.0 * v_in;
    return v_output;
}
#####
```

The prototype of the C function is specified in the *C_prototype_ex.h* file:

```
#####
double C_Compute(double);
#####
```

The C code is called from a VHDL-AMS function declared in a package called ENCAPSULATION. This package is defined in the *encapsulation_package_ex.vhd* file:

```
#####
PACKAGE ENCAPSULATION IS
FUNCTION VHDL_Compute (v_in : REAL) RETURN Real;
ATTRIBUTE Foreign OF VHDL_Compute : FUNCTION IS "ADMS : C_Compute";
END PACKAGE ENCAPSULATION;
#####
```

The C image of the package filled by the end-user is given in the *Template_n_functions_ex.c* file:

```
#####
/* C Template corresponding to the C image of the package */
/* ENCAPSULATION : */
/* link between the C image of the VHDL-AMS functions */
/* and the C functions to be encapsulated */
/* The total number of VHDL-AMS functions is supposed to */
/* be n */
#include "macro.h"
/*****/
/*          MANUAL MODIFICATIONS          */
/*****/
```

```

/*****
/* the file containing the prototype of the C encapsulated*/ /* functions
has to be included */
#include "C_prototype_ex.h"
/*****
/* END OF MANUAL MODIFICATIONS      */
/*****

extern PROC_CAST current_active_process;
RG_DECL;
VPTR_DECL;
LOOP_I_DECL;
LOOP_DRIVER_DECL;
static void elab_desent();
static int dblines[0]={};
/*****
/*      MANUAL MODIFICATIONS      */
/*****
/* A C predeclaration corresponds to each VHDL-AMS function */ /*
"Funci_VHDL" */
/* <TYPE_FUNC_Image> has to be replaced by the C type */
/* corresponding to the type of the VHDL-AMS function */
/* "TYPE_Funci_VHDL" : */
static double VHDL_COMPUTE ();
/*****
/* END OF MANUAL MODIFICATIONS      */
/*****

/*****
/*      MANUAL MODIFICATIONS      */
/*****

/* Table of the names of C image of the VHDL-AMS functions */
/* It is mandatory that the order of these subprograms is */
/* the same as the order of the declarations of the */
/* VHDL-AMS subprograms in the package */
CODE_SUPRGS_DECL=
{(CODE_CAST)VHDL_COMPUTE};
/*****
/* END OF MANUAL MODIFICATIONS      */
/*****

PKGI_DECL(0);
ELB_CODE_CAST elab_funcs=
{elab_desent,0,(void*)code_suprgs,NULL};
static void elab_desent(int* ENVIR_CAST,int selector){
    switch(selector){
        case -1: /* Analog Code and Subprogram Definition */
            break;
        case 0: /* block: design entity */
            ALLOC_RG;
            break;
        default:
            /* Error Management */
            break;
    }
}

```

```

}
/*****
/*      MANUAL MODIFICATIONS      */
*****/
/* body of the C image of the VHDL-AMS functions */
static double VHDL_COMPUTE (FRGN_PARAM_CON_REAL(0),PKG_DECL) {
    FRGN_DECL_CON_REAL(v_in, 0);
    return C_Compute(v_in);
} /* end VHDL_COMPUTE */
/*****
/* END OF MANUAL MODIFICATIONS      */
*****/

/* end of C template */
#####

```

The C function is called in the following model defined in the *test.vhd* file.

```

#####
USE WORK.ENCAPSULATION.ALL;
ENTITY result IS
END;
ARCHITECTURE one OF result IS
CONSTANT v_input : REAL := 2.0;
QUANTITY v_output : REAL;
BEGIN
    computing : v_output == VHDL_Compute(v_input);
END ARCHITECTURE one;
#####

```

Compiling Commands

The compiling commands of the different files are as follows:

- User C code to *C_function_ex.c* file:

```
gcc -c -g -fPIC C_function_ex.c -o C_function_ex.o
```

- VHDL-AMS package to *encapsulation_package_ex.vhd*:

```
vacom encapsulation_package_ex.vhd
```

- For Linux 32-bit platforms:
 - C template compilation:

```
gcc -fPIC -std=gnu99 -Wno-implicit -Wno-strict-aliasing
-DSS5_RELEASE=8 -o0 -DNDEBUG -I $MGC_AMS_HOME/include/.internal
```



```
-D__EXTENSIONS__ -D__EXT_VHDLA__ -DVHDLANALOG -D__EXT_ELDO__
-I $MGC_AMS_HOME/include Template_n_functions_ex.c -c -o
Template_n_functions_ex.o
```

- *ENCAPSULATION.so* building:

```
gcc -Wl,-G -nostdlib C_function_ex.o Template_n_functions_ex.o -o /
user_path/<WORKING_LIBRARY>/_OS/$AMS_VCO/ENCAPSULATION.so -lgcc
```

- For Linux 64-bit platforms:

- C template compilation:

```
gcc -fPIC -std=gnu99 -Wno-implicit -Wno-strict-aliasing
-DSS5_RELEASE=8 -o0 -DNDEBUG -I $MGC_AMS_HOME/include/.internal
-D__EXTENSIONS__ -D__EXT_VHDLA__ -DVHDLANALOG -D__EXT_ELDO__
-I $MGC_AMS_HOME/include Template_n_functions_ex.c -c -o
Template_n_functions_ex.o
```

- *ENCAPSULATION_64.so* building:

```
gcc -Wl,-G -nostdlib C_function_ex.o Template_n_functions_ex.o -o /
user_path/<WORKING_LIBRARY>/_OS/$AMS_VCO/ENCAPSULATION_64.so -lgcc
```

- VHDL-AMS model to *test.vhd* file:

```
vacom test.vhd
```


E. Tips and Techniques

This appendix provides information and examples that have evolved from answers to questions received by technical support.

Zero-delay Loop Issues.....	E-1
Structural vs. Behavioral Descriptions.....	E-2
Using Intermediate Quantities.....	E-5
Handling of VHDL-AMS Assertions.....	E-6
Reduction of Analog Output File Size.....	E-7
Plotting Waveforms.....	E-7
Workarounds.....	E-10
Macros for Simulator Version Identification.....	E-21

Zero-delay Loop Issues

Simulations use steps that advance simulated time, and steps that do not advance simulated time. Steps that do not advance simulated time are called delta cycles or simply deltas. Deltas are used when signal assignments are made with zero time delay.

If a large number of deltas occur without advancing time, it is usually a symptom of an infinite zero-delay loop in the design. In order to detect the presence of these loops, Questa ADMS defines a limit, the *iteration limit*, on the number of successive deltas that can occur. When the number of zero-delay deltas exceeds the value of the iteration limit minus the value of the `IterationLimitDeltaDebug` simulator setup variable (see [Simulator Control Variables](#)), Questa ADMS will add a watch point to any activity in Questa ADMS or Questa SIM to help you to debug zero-delay loops until the iteration limit is reached (in this case, an error is displayed). For more information refer to the [watch](#) command.

The iteration limit default value is 5000. If you receive an iteration limit warning, first increase the iteration limit and try to continue simulation. You can set the iteration limit by specifying the `vasim` command with the option `-iteration_limit <nb>`. This is taken into account on both the Questa ADMS and Questa SIM sides. This option will override (but not modify) the value specified in the `modelsim.ini` file.

If the problem persists, look for zero-delay loops. Run the simulation and look at the source code when the error occurs. Two common causes are a loop that has no exit, or a series of gates with zero delay where the outputs are connected back to the inputs.

The following cases are examples of problems that might exist:

1. The design may require many deltas to be stable according to the number of non-postponed processes in the cycle and their activity. Therefore, the designer can increase the iteration limit to 10000 for example.
2. The design may iterate through a zero-delay feedback loop.

3. The signal propagation delays of the models may be less than the digital simulation resolution, i.e. a timescale problem.

Cases 2 and 3 are the result of an incorrect design. In this case, changing the iteration limit value will not make any difference.

Structural vs. Behavioral Descriptions

cell(l0) is a 16 x16 matrix of scalar elements.

There are two ways of describing cell(10):

- A structural description:
 - 16 lines of 16 cells.
 - Each cell has 4 ports and 4 equations, and as a result there are 196 instantiations of the basic cell(l0).
- A behavioral description:
 - All is in one entity (architecture) called mod_mat16(behav). This design entity has:
 - ◊ 16 vector equations of 16 elements.
 - ◊ 34 scalar equations. Therefore, it has 230 equations for this instance of mod_mat16(behav).

Suppose that the optimization for the vector equation is OK.

Case 1

The optimization is OK. Just one of the four equations is in the matrix, plus 4 ports. 5 lines are in the matrix. In addition, no call to user functions are in the matrix, therefore, the Jacobian is optimized (sparse matrix) and formally computed. It has 10 non-zero elements in the matrix. For each cell, the matrix has 5 new lines maximum (some nets may be shared by different models) and 10 non-zero elements.

- In total, there are 980 lines for 1,960 non-zero elements. There is about 1 kB of memory per element in the matrix. The required memory is about 2 MB.

Because there are no calls to any user functions, the number of calls for each instance on the model per iteration is equal to 1, computing 5 characteristic expressions (CEs) and 10 Jacobian values (partials). (15 scalar computations (SCs) are calculated.) The model is converging well. (There is 1 iteration per step.)

- There are 15 scalar computations per model and per iteration. 2,940 SCs are needed per step (196 x 15 x 1 SCs).
- 2 MB of memory, 2,940 SCs per step.

Case 2

The optimization is OK. The 16 vector equations (16 elements per equations) are on the matrix, plus 34 ports. 230 lines are required in the matrix. Because of the use of user-defined functions, there is no optimization in the Jacobian. The result is that the matrix is not a sparse matrix and that the matrix is computed numerically.

- Non-zero elements in the matrix: $230 \times 230 = 52,900$.
- In total there are 230 lines for 52,900 non-zero elements. There is 1 kB of memory per element in the matrix. The required memory is about 53 MB.

Because there are calls to any user functions, the Jacobian is computed numerically. The number of calls for each instance of the model per iteration is equal to `nb_lines + 1`, thus 231 calls per iteration, computing 230 characteristic expressions (CEs) and 0 Jacobian values (partials). This means that 230 scalar computations (SCs) must be computed.

The model is not converging as well as the model in case 1. This is because of the large number of floating-point operations (non-sparse matrix). This may generate a lot of numerical noise and an increase of the number of iterations. In the example, there are 5% more iterations per step.

- There are 231×230 scalar computations per model and per iteration, and about $1 \times 53,130 \times 1.05$ SCs (55,786 SCs) per step. 53 MB of memory are required for the 55,786 SCs per step.

Global conclusion in the case where the vector equation statement is optimized

Table E-1: Memory Usage With Vector Optimization

	Memory	SCs/step
Structural	2 MB	2,940
Behavioral	53 MB	55,786

Behavioral description is 20 times slower.

The optimization is not efficient. This has an impact on the structural description.

- Non-sparse matrix for the model, numerical computation of the Jacobian, all the equations on the Matrix:
 - There are 8 additional lines in the matrix instead of 5.
 - It has 64 elements in the Jacobian, instead of having 10 elements.
 - There are $8 + 1$ calls of the model per iteration, 8 CEs per call. $9 \times 8 = 72$ SCs per iteration.
 - Convergence is almost the same (1 iteration per step):
 - ◊ Memory is $196 \times 64 \times 1k = 12.5$ MB (2 MB if optimized).
 - ◊ Speed is $196 \times 72 \times 1 = 14,112$ iterations (4.8 times slower than when optimized).

But it has a less important impact on the behavioral description.

- Non-sparse matrix for the model, numerical computation of the Jacobian, all the equations on the Matrix:
 - It has 264 lines in the matrix instead of having 230.
 - Instead of having 230 x 230 elements in the Jacobian, it has $264 \times 264 = 69,696$ elements.
 - There are $264 + 1$ calls of the model per iteration, 264 CEs per call; $265 \times 264 = 69,960$ SCs per iteration.
 - Convergence is almost the same (1.05 iterations per step):
 - ◊ Memory is $1 \times 69,696 \times 1k = 69.5$ MB (55 MB if optimized).
 - ◊ Speed $1 \times 69,960 \times 1.5 = 73,458$ iteration (1.3 times slower than when optimized).

Global conclusion in the case where the vector equation statement is not optimized

Table E-2: Memory Usage Without Vector Optimization

	Memory	SCs/step
Structural	12.5 MB	14,112
Behavioral	69.5 MB	73,458

Behavioral Description is five times slower.

All times given consider only the model calls, not the matrix computation. There are 6 times more elements in the matrix.

The simulations give:

- Structural - 7 seconds.
- Behavioral - 3 minutes, 17 seconds (47 times slower).

The matrix computation takes 9.4 times more CPU time.

Conclusion

It is perceived that writing a behavioral description of structural model gives better simulation time. This is the case only if the level of the description is higher in the behavioral part than in the structural one (OpAmp in transistor or using few equations). If the level is the same between the structural and the behavioral part, the number of generated equations will be the same in both cases.

In case one, therefore, optimization should be used on only a few equations (structural model). The optimization will be easy and the number of lines in the matrix will be reduced. In the other case (behavioral description), optimizing the set of equations will be difficult. Equations and the manipulation

of matrices and vectors, using user-defined functions will cancel the formal computation of the Jacobian. This means:

- It will have $nb + 1$ calls of the model for each iteration (where nb is the number of scalar equations that are put on the matrix).
- It will generate a non-sparse portion of the matrix equal to $nb \times nb$. (A huge amount of memory may still be needed even in the case of a well done optimization.)

Just a simple memory diagnostic for 64×64 matrix:

- Optimized vector equation:
 - $64 \times 64 + 2 \times 64 = 4,224$ equations.
 - $4,224 \times 4,224 = 17,842,176$ elements in the matrix.
 - At about 1kB per element, 17 GB are required.

Note

1 kilobyte is not the exact size needed by the elements in the Matrix. It is used to illustrate the way in which the memory increases in the examples.

Optimization of the code in the case of the structural description

To avoid the effect of the vector equation, two new functions `init_or_val()` and `init_or_dot()` have been introduced that return the correct value according to the DOMAIN signal. They reduce the number of equations in the matrix to 5 instead of 8 and the number of elements in the Matrix to 25 instead of 64. The resulting simulation time is 9 seconds instead of 17 seconds (2 times faster).

Using Intermediate Quantities

The presence of intermediate quantities can affect simulation time.

Problem: two close descriptions are not simulated with the same time (one is done in 5 minutes and the second in 8 minutes). There should be a greater difference according to the differences in the descriptions.

The same description in HDL-A is done in 2,510 steps and the same in Questa ADMS is done in 7,801 steps.

- The first claim was not reproduced, both simulations took 7 minutes 22 seconds and 7 minutes 28 seconds.
- The second point could (using same numbers of steps) which gave a big difference in the simulation time with HDL-A only taking 23 seconds.

After analysis of the description, it appears that there are two main problems:

- An vector equation statement (not much of a problem here).
- A vector quantity and then a vector equation (this is the major problem).

By removing the vector quantity (just an intermediate quantity) and replacing it with its expression in the main equation, the simulation time of both Questa ADMS descriptions become 8 seconds which is 3 times faster than HDL-A.

In conclusion, there is no problem adding intermediate quantities to give more readability in the description, as long as these intermediate quantities can be optimized.

This is not the case however:

- When using an vector equation statement.
- When intermediate quantities are vector quantities.

In addition, increasing the quantity a little, will not affect the simulation severely. By adding 50 or more equations (a vector of 50 equations) to only one equation in the model, the simulation time will be greatly affected. Removing these un-needed intermediate quantities gives a simulation time gain from 2 minutes 20 seconds to 8 seconds.

Handling of VHDL-AMS Assertions

By default, when a digital VHDL assertion fails, the simulation stops if the severity is "FAILURE".

The threshold severity can be changed by setting the BreakOnAssertion variable of the [vsim] section of the *modelsim.ini* file.

Note

From AMS2010.1 onwards, Questa ADMS no longer stops a simulation when the severity is "ERROR". Questa ADMS now behaves in the same way as Questa SIM. As a consequence, ADMS testcases that were supposed to stop at the first assertion "ERROR" should be changed either by changing the severity to "FAILURE" in the VHDL description or by setting the variable BreakOnAssertion to 2.

Refer to BreakOnAssertion in the *Questa SIM User's Manual* for more information.

Related Topics

Questa SIM Simulation Control Variables

Reduction of Analog Output File Size

You can specify that the simulator only has to solve analog data when it has changed sufficiently. This significantly reduces the stored size of the analog output file in long simulations or simulations with many logged nets.

A delta value controls the minimum amount that analog data must change by.

Procedure

1. With a design loaded in interactive mode, undock the **Objects Window** to make the Delta controls visible on its toolbar.
2. Select the design object for which you want to specify a delta value.
3. Click the **Delta** button to activate the field to the right, then enter the required delta value.

Related Topics

[Undocking and Docking Windows](#)

Plotting Waveforms

This section describes issues and workarounds related to plotting mixed-signal waveforms in Solido Waveform Analyzer.

.ez.do File Limitations.....	E-7
Solido Waveform Analyzer Error Handling.....	E-8
Plot of Electro Mechanical Force (EMF) Type Waveforms.....	E-9
Net Activity Profiler.....	E-10

.ez.do File Limitations

Limitations apply when loading Solido Waveform Analyzer configuration files (.ez.do).

Be aware of the following:

- When multiple nets are plotted in the same row in the **Wave Window (Solido Waveform Analyzer)**, they may be plotted in separate rows when the Solido Waveform Analyzer configuration.ez.do file is loaded. This occurs if the nets have different types of Y-axis (for example, magnitude, voltage, and so on).
- The.ez.do file does not support waveforms from an RF analysis.

- The following limitation applies. When a plot of a current going through some pins of an Eldo predefined device, for example:

```
.PLOT tran i(rl)
```

the plot is performed successfully. Upon restarting the simulation, the following error message is delivered:

```
# ** Error: No object matching ':mytop:rl'
```

where mytop corresponds to the top of the design.

This means that the wave will not be present in Solido Waveform Analyzer at restart. But if the same *.cir* or *.cmd* file has been run at restart, or if the plot command is still present within the new *.cmd* or *.cir* file, the wave is present in the JWDB file and has to be displayed manually.

Related Topics

Viewing Simulation Results

Solido Waveform Analyzer Error Handling

The JWDB (Solido Waveform Analyzer) server may encounter a disk space issue.

If there is a disk space issue:

- In batch mode, a message related to the space issue is output and simulation stops.
- In GUI mode, simulation is paused and a message informs you how much disk space is needed to save the *.wdb* file and gives you the choice to retry or abort. At this point you have the opportunity to free up some disk space.
 - Click **Retry** to continue the simulation (assuming you have created some disk space).
 - Click **Abort** to stop the simulation. The database is not saved and a message is output to the [Transcript Window](#).

For other errors encountered by Solido Waveform Analyzer or JWDB, Questa ADMS exits and, depending on which had the error, a log file named *swave_error.log* or *jwdb_error.log* is generated.

Where possible, Questa ADMS catches the SIGPIPE signals and displays a general purpose message (this may occur when the device runs out of memory).

In addition:

- For cases of increased use of RAM, the JWDB server will issue a warning as soon as the used RAM goes above a threshold.

At this time, Questa ADMS creates a checkpoint, so if the simulation stops later because of a leak of RAM, you can restart from this point after changing system settings to allow simulation to continue under better operating conditions.

- If there are many processes requiring memory, the JWDB server may be killed. In these cases, Questa ADMS will create a checkpoint.
- A save (checkpoint) is done automatically in case of computer resource warnings or errors. Each automatic save deletes the previous one, except if the last one issues with an error.

Related Topics

Viewing Simulation Results

Plot of Electro Mechanical Force (EMF) Type Waveforms

When running mixed-signal simulations, plotting voltage in Solido Waveform Analyzer may provide either Voltage (V) or EMF (V) values on the Y axis of the plot.

This is shown in [Figure E-1](#).

Figure E-1: Plot of EMF Type Waveform



Verilog-AMS internal nodes are represented as Voltage(V), whereas VHDL-AMS quantities, VHDL-AMS internal terminals and SPICE internal terminals are represented as EMF(V). Depending on its connection to upper nodes, an analog port of any region type can be represented as Voltage (V) or EMF (V).

Related Topics

Viewing Simulation Results

Net Activity Profiler

You can access information about the time-step to assist debugging performance issues.

Time-step information is accessible through the DBG(TStep) waveform defined as:

$$20 * \log(TStep(T))$$

where TStep(T) corresponds to the value of the time step at time T (the time-step immediately preceeding time T).

This waveform can be requested either from the SPICE command file:

```
.PLOT tran DBG(TStep)
```

or by using the Tcl commands:

```
add wave -dbg TStep
add log -dbg TStep
```

Note

When using Questa ADMS Premier, you can enable the display of more information inside the DBG(Tstep) curve, by including the **.FORMAT_STEP_LIMITERS** command inside the netlist. If not by specifying **.FORMAT_STEP_LIMITERS**, some information in the waveform DBG(tstep) will be missing with Premier if it is plotted explicitly. Information about time step limited by Premier engine in the waveform DBG(tstep) is not relevant.

Workarounds

Several issues have known workarounds.

VHDL String Representation..... E-11

UNIX Limitation When Instantiating a SPICE Entity..... E-11

Simulation Display Time Limitation..... E-12

HMIN Option Convergence..... E-12

Differences Between HDL-A and Questa ADMS Using across'Dot..... E-13

Computing Huge Values for Quantities..... E-13

Different Results are Generated With Two Equivalent Equations.....	E-13
Difference in Default Quantities.....	E-13
Model Writing Style Effect on Results.....	E-14
Case-sensitivity for Verilog Modules.....	E-14
Different Results for Equations Containing Functions.....	E-14
Laplace Transform Function.....	E-14
Converters Limitation.....	E-15
Specifying the Number of Warnings Displayed.....	E-15
File Declarations in TEXTIO.....	E-16
Aggregate Operand.....	E-18
ABS Operator.....	E-18
VHDL-AMS Parent Instantiating a Verilog-AMS Child.....	E-18
Verilog Parent Instantiating a VHDL-AMS Child.....	E-18
Unnamed Ports in Modules Instantiated From Verilog-AMS.....	E-19
Questa SIM in Restore Mode With Anti-Hacking Linux OS.....	E-19
FFT Analysis on Pure Digital Signals.....	E-20
Miscellaneous Workarounds.....	E-20

VHDL String Representation

Do not use brace (curly bracket) characters "{" and "}" in a VHDL string. These characters are reserved keywords for the GUI based on the TCL package.

UNIX Limitation When Instantiating a SPICE Entity

When using the vacom -link option, for example:

```
vaspi ADMS_entity ckt@file.ckt
vacom -ams -link ADMS_entity(ckt)
```

You need to include "" as follows:

```
vacom -ams -link "ADMS_entity(ckt)"
```

However, this command only works if the ADMS_entity already exists. If it does not exist, use:

```
vacom -link "inv_spice" -ams
vacom -link "inv_spice(inv)" -ams
```

When instantiating this ADMS_entity in a VHDL-AMS file, you need to set the instantiation (for example OV: ENTITY work.inv_spice) even if 'use work.all' is set in the VHDL-AMS file. However an error will occur if you set the instantiation using:

```
OV: inv_spice
```

and you will get the following error message:

```
20:      OV: inv_spice
      ^^^^^^^^^^^
[Error] Illegal name INV_SPICE: expecting component or procedure name
instead of entity_decl
```

Simulation Display Time Limitation

When there is no event in Questa ADMS (full design in Questa SIM), the simulation runs normally. However, at the end of simulation, the displayed time can be, for example, 9,223,372,036,854,775,807 fs. All simulation results are correct, but you need to zoom in to see them.

HMIN Option Convergence

When the value of hmin is important and the Modified Lock Step algorithm (the one fixed by default) is used, exceptionally, Analog/Digital synchronization step conflict may occur during the simulation. Questa ADMS points out these convergence problems with the following error message:

```
#Error: Analog/digital synchronization step conflict.
Use .OPTION mixedstep=locked
```

These conflicts can be bypassed in the following ways:

- by decreasing or removing hmin option with the Modified Lock Step algorithm, or
- by switching to the Locked Step algorithm by using the option mixedstep=locked as the error message suggests.

This option has to be added:

- in the command file (extension.*cmd*) if it exists
- in the circuit file (extension.*cir*) if it exists
- in a new command file if no command file or no circuit file already exists as follows:

in the Start Simulation dialog box, select **New**, then **More** and add mixedstep=locked in the Additional Options command space.

Differences Between HDL-A and Questa ADMS Using across'Dot

This occurs sometimes when a 'Dot is applied on an across quantity of a port terminal.

Workaround: In this case, declare two intermediate free quantities (vout2 and dvout) and add the two following equations (assume that the across quantity is named vout):

```
0.0 == vout - vout2;
0.0 == dvout - vout2'Dot;
```

and use dvout instead of vout'Dot in the description. Questa ADMS uses backward Euler, so adding the .OPTION be command in HDL-A will result in an improved match for waves.

Computing Huge Values for Quantities

When an equation is computing too large a value for a quantity ($1.0e14$), the simulation may give a bad value, usually 0.0.

Try to normalize large values in order to not have a too big a dispersion in the quantity values.

Different Results are Generated With Two Equivalent Equations

The following problem is based upon a testcase. In this case, it was found that the following equation gave strange results around $1e11$ Hz, (phase discontinuity and gain bump):

```
Voa == Voff + vdif'Ltf(...)
```

However, the following equation gave good results:

```
Voa - Voff == vdif'Ltf(...)
```

This is because at the frequency where the phase and gain discontinuity occur, `vdif'Ltf(...)` is extremely small compared to `Voff`, and `vdif'Ltf(...)` is truncated when added to `Voff` because of finite floating number accuracy.

Difference in Default Quantities

In VHDL-AMS, quantities are initialized by default to 0.0 whereas real signals are initialized to `real'left` ($-1.0e38$ in Questa ADMS). When developing models, common mistakes can be made and developers should be aware of this difference.

Model Writing Style Effect on Results

Some models may seem to be running correctly, however they run differently depending on their environment. Make sure your models follow these modeling rules:

- The set of simultaneous statements should not be linearly independent
- Do not add a dummy pin to “solve” a problem of mismatch between the number of equations and unknowns
- Avoid having too big a difference (order of magnitude) between values of variables in an equation

Analyze your modeling style very carefully when in the following situations:

- One instance of the model gives certain results, but duplicating the instance with the same stimuli as the first instance gives different results
- Model behavior might become sensitive to equation order

Usually, such erroneous behavior is notified by DC, or even Transient convergence problems, or by warning about “non-invertible matrices.”

Case-sensitivity for Verilog Modules

You can compile a Verilog module in any mixture of case, and instantiate it in any (different) mixture of case, Questa ADMS will retrieve the correct module. The only limitation is when there are two modules which differ only by their letter case: Questa ADMS then generates an error message, whereas Questa SIM would accept such a case. There is no need to compile in Questa SIM with -u.

Different Results for Equations Containing Functions

It should be noted that when a function is used inside an equation, the derivatives are computed numerically, whereas for an equation not using a function, analytical derivatives are computed. This can lead to slight differences in results at low accuracy.

Laplace Transform Function

In VHDL-AMS, the laplace transform function is provided by the 'ltf() attribute.

It is possible to have the denominator (or the numerator) equal to a constant. However, there is a problem because of the way vectors are written. In VHDL, the following two expressions are equivalent:

```
1.0
```

and

```
(1.0)
```

Therefore, (1.0) cannot represent a vector, it is a real (scalar) value. A vector having only one element has to be represented in VHDL by:

```
(0 => 1.0)
```

Otherwise, a vector having two elements can be used (in this particular case) if the second element is NULL:

```
(1.0, 0.0)
```

Therefore, in the 'ltf() attribute, both previous literals can be used.

Converters Limitation

When a boundary element is associated with a group of models (through .GROUP and .DEFHOOK group commands), the current limitation is that these models have to be compiled in the working library. If they are compiled in any other library, they will be ignored and an error message will be issued that the models are not found.

Specifying the Number of Warnings Displayed

Questa ADMS (Eldo) reports four types of node connection faults as follows:

```
Warning 107: node "xxx": Less than two connections.
Warning 108: node "xxx": This node is a floating gate.
Warning 113: node "xxx": Not connected to any element.
This node is removed from the netlist.
Warning 252: OBJECT "xxx": Self-connected object not created.
```

You can limit the number of times each type of node connection fault is reported using the following syntax:

.OPTION MSGNODE=val

If MSGNODE=0 then all connection fault warnings are displayed. By default, MSGNODE is set to 3, which means Questa ADMS displays each type of connection fault for the first three nodes on which the fault is detected. If the number of nodes at which a fault is detected exceeds the number specified by this option, then the following warning message is issued:

Warning 29: Set .OPTION MSGNODE=0 to receive all such warnings

File Declarations in TEXTIO

In VHDL-AMS, a globally static logical file name is not supported in this version of the tool. Note: a locally static file name *is* supported, see example below. As an example, the following description is disallowed:

```
-- globally static logical file name
library std;
use std.TEXTIO.all;
ENTITY globally_static_file IS
  GENERIC(file_name : STRING := "");
END ENTITY globally_static_file;
ARCHITECTURE adms OF globally_static_file IS
BEGIN
  TESTING: PROCESS
    file          F      : TEXT OPEN READ_MODE is file_name;
    BEGIN
      REPORT "globally static logical file name : " & file_name;
    wait;
  END PROCESS TESTING;
END ARCHITECTURE adms;

USE WORK.ALL;
ENTITY globally_static_file_test IS
END ENTITY globally_static_file_test;
ARCHITECTURE adms OF globally_static_file_test IS
  BEGIN
    inst1 : ENTITY globally_static_file(adms)
      GENERIC MAP (file_name => "data_int.in");
  END ARCHITECTURE adms;
```

Below is an example based on the use of a locally static file which is allowed:

```
-- locally static logical file name
library std;
use std.TEXTIO.all;
```

```

ENTITY locally_static_file IS
END locally_static_file;

ARCHITECTURE adms OF locally_static_file IS
BEGIN
  TESTING: PROCESS
    file          F      : TEXT OPEN READ_MODE is "input_file.txt";
    variable      L      : LINE;
    variable      v      : INTEGER;
  BEGIN
    READLINE (F, L);
    READ (L, v);
    REPORT "value of v in locally static logical file name : " &
integer'image(v);
    wait;
  END PROCESS TESTING;
END adms;

```

Workaround: A workaround to allow globally static logical file names is based on the use of the implicit function *file_open(...)*. For example, the following description is allowed:

```

-- workaround globally static logical file name
library std;
use std.TEXTIO.all;
ENTITY globally_static_file_workaround IS
  GENERIC(file_name : STRING := "");
END globally_static_file_workaround;
ARCHITECTURE adms OF globally_static_file_workaround IS
BEGIN
  TESTING: PROCESS
    -- file          F      : TEXT OPEN READ_MODE is file_name;
    file          F      : TEXT;
    variable v : integer;
    variable L : line;
  BEGIN
    file_open(F, file_name, READ_MODE);
    READLINE(F, L);
    READ(L, v);
    REPORT "v = " & integer'image(v) severity note;
    REPORT "workaround for globally static filename" severity note;

    wait;
  END PROCESS TESTING;
END ARCHITECTURE adms;

USE WORK.ALL;
ENTITY globally_static_file_workaround_test IS
END ENTITY globally_static_file_workaround_test;
ARCHITECTURE adms OF globally_static_file_workaround_test IS
  BEGIN
    inst1 : ENTITY globally_static_file_workaround(adms)
      GENERIC MAP (file_name => "data_int.in");
  END ARCHITECTURE adms;

```

Aggregate Operand

In a VHDL-AMS aggregate operation, an element association with the choice others is not supported if the expression is a function. For example:

```
constant c : vect_type := (others => f);
```

where c is a vector and f is a function, is not allowed.

Workaround: Replace the choice others by the call of the function for each element.

ABS Operator

For ADMS RF, using the ABS operator may result in some convergence problems.

VHDL-AMS Parent Instantiating a Verilog-AMS Child

It is not possible to instantiate Verilog-AMS from VHDL-AMS. A module compiled with valog can be instantiated from VHDL-AMS only if it is pure digital code; if it contains an analog block or a mixed-signal netlist, and if you attempt to instantiate it from VHDL-AMS, an error message will be returned informing you that the module cannot be found. The same module can be instantiated from VHDL-D, Verilog-D, SPICE or Verilog-AMS without problems.

Workaround: Wrap the Verilog-AMS block in a SPICE wrapper.

Verilog Parent Instantiating a VHDL-AMS Child

When instantiating a VHDL-AMS model in a Verilog model, the instance name in the Verilog module must be in upper case, for example the following entry will cause an error during elaboration because "rom" is lower case:

```
module top-verilog;
  child-vhdlams u1 ( .rom(romsz), .AVDD(ramsz) );
endmodule
```

Workaround: Instantiate the name in upper case, i.e. "ROM".

Unnamed Ports in Modules Instantiated From Verilog-AMS

Module instantiated from Verilog-AMS must not have unnamed ports. For example, putting the slice in the port list makes the port “unnamed”. For example, the following is allowed:

```
module verilog_allowed_in_adms ( port_vector[7:0] );
  inout [7:0] port_vector;
endmodule // verilog_allowed_in_adms
```

The following is not allowed:

```
module verilogsams_not_allowed_in_adms ( port_vector[7:0] );
inout [7:0] port_vector;
  analog begin end // This makes it a Verilogams module
endmodule // verilogsams_not_allowed_in_adms
```

If this is given, the following error message is reported:

```
...
Compiling Module Declaration verilogsams_not_allowed_in_adms
-----
In file a.v line      1:
  module verilogsams_not_allowed_in_adms ( port_vector[7:0] );
                                ^
[Error] Indexed and sliced port reference not yet supported
Verilog-ams subset : Unsupported
Error: (valog) Compilation: Analysis failed.
```

However, if you remove the list of ports information [7:0], this entry is allowed:

```
module verilogsams_allowed_in_adms ( port_vector );
  inout [7:0] port_vector;
  analog begin end // This makes it Verilogams module
endmodule // verilogsams_allowed_in_adms
```

Questa SIM in Restore Mode With Anti-Hacking Linux OS

The following is a known defect in Questa SIM v6.5.

On some Red Hat Linux Operating System versions the “-restore” feature occasionally fails. This is because of the memory allocation security (anti-hacking) feature of Linux. Red Hat Enterprise Linux release v.3 update3 was the first version to have this security feature. In these Linux releases two consecutive program invocations do not get the same memory allocation footprint. For the “-restore” feature the simulator relies on having the same memory allocation footprint. Users are advised to retry this feature a few times as

further attempts can be successful. In recent Linux versions, an override for this anti-hacking feature is provided; use it at your own discretion.

FFT Analysis on Pure Digital Signals

It is possible to perform an FFT analysis on a pure digital signal if it is of a numerical type: BIT, INTEGER, REAL or BOOLEAN.

However, for STD_LOGIC, it is not possible to perform FFT analysis or EXTRACT on this kind of signal. A workaround is to connect an analog object to the signal port in order to insert a converter.

Miscellaneous Workarounds

The following describes additional miscellaneous workarounds:

- For multiple run analyses, output of the waveform data is in the form of compound waveforms. A known problem is that some compound waveforms may be displayed more than once. Also, add log/wave does not work, .PROBE/.PLOT commands must be used instead.
- Vector ports with their range defined using functions do not work properly: *.info is not generated correctly. Such modules cannot be instantiated in Verilog(-D) modules.
- Linux platforms only: when compiling a Verilog-AMS module with a pure digital part (for example an initial block), the following message may appear:

```
sh: line 1: /usr/bin/mkdir: No such file or directory
```

Workaround: Create a symbolic link "mkdir" under /usr/bin/ that refers to /bin/mkdir, for example perform the following command as system administrator:

```
ln -s /bin/mkdir /usr/bin/mkdir
```

- Spaces are not allowed in entity names in certain situations (this is mainly applicable to the Windows platform release).
- Non-convergence can happen when values are outside the mathematical validity range [-1,1] of the ACOS and ASIN functions (IEEE.MATH_REAL VHDL package). A checker will be available in a future release.

Workaround: Encapsulate ASIN and ACOS into a VHDL-AMS user function with an assert to check the range of the argument.

- A SystemVerilog module with a port of a real array type cannot be instantiated from VHDL-AMS.
- Eldo options or commands contained in a library are only visible to Eldo when they are included in the .SUBCKT command processed by **vaspi**. This is because vaspi extracts from the library only the

section that corresponds to the .SUBCKT command, and nothing else. If you want options or commands present in a library file to be visible to Eldo, use the Eldo .LIB or .INCLUDE commands.

- Subcircuits referenced in an Eldo .PARAM statement cannot contain Y instances.
- Eldo Interactive Mode (see [Eldo Interactive Mode](#) in the *Eldo User's Manual*) is not supported by Questa ADMS. If the .eil command is found in a netlist, an error is reported.
- When a node is named VDD, VSS, VEE, VCC, or GND, and this node is not connected to a power supply, a warning message will be reported.

Macros for Simulator Version Identification

The `ADVANCE\_MS and `MODEL_TECH values can be used to return the versions of Questa ADMS and Questa SIM respectively used during simulation. These macros are implicitly defined in both vlog and valog.

Verilog-AMS Example

```
module sim_ident;

analog @(initial_step) begin
`ifdef ADVANCE_MS
    $display("The macro ADVANCE_MS is defined as %s", `ADVANCE_MS) ;
`else
    $display("The macro ADVANCE_MS is NOT defined") ;
`endif
`ifdef MODEL_TECH
    $display("The macro MODEL_TECH is defined as %s", `MODEL_TECH) ;
`else
    $display("The macro MODEL_TECH is NOT defined") ;
`endif
end

endmodule
```

The following is an example of the output at the start of a simulation run, where <version> shows the version of Questa ADMS/Questa SIM:

```
# The macro ADVANCE_MS is defined as Siemens Digital Industries Software
<version>
# The macro MODEL_TECH is defined as <version>
```

Related Topics

[valog \[Questa ADMS Command Reference\]](#)

F. Improved Diagnostics for Certain Erroneous Models

Certain models give rise to errors during simulation. For instance, a singularity may occur during the search for the quiescent point, or the design may have a node with no DC path to ground.

Overview of Singularities..... F-1

No DC Path to Ground..... F-2

Voltage Loop..... F-4

Inconsistent Equations..... F-5

Multiple Solutions..... F-6

Numerical Noise..... F-7

Overview of Singularities

The analog kernel of Questa ADMS computes a solution to the system of equations describing the behavior of a design. When Questa ADMS detects that the system has no solution or that it has multiple solutions, the simulator reports the singularity then stops.

A singularity may occur during computation of the quiescent point (the SPICE “DC” point) or during transient simulation. The singularity may be caused by a design mistake, such as an unconnected component or a sign error. For example, the following system of model equations has no solution:

- 1. $X + Y = 4.0$
- 2. $X + Z = 2.0$
- 3. $Y - Z = -2.0$

There is a sign error in the third equation. With the correct third equation $Y + Z = -2.0$, the solution of the system is $X=4.0, Y=0.0, Z=-2.0$.

Questa ADMS can detect some design configurations that would lead to a singularity during simulation. Refer to **No DC Path to Ground** for more information.

You can use the option NOMATSING to enable the simulation to continue if a singularity occurs during computation of the quiescent point. However, if a singularity occurs, the computed quiescent point will be incorrect and the results of any subsequent transient simulation may not be physically reasonable. To prevent an inaccurate solution, you should ensure that the equations generated by your design do not produce a singularity.

See also **.OPTION NOMATSING** in *Eldo Reference Manual*.

No DC Path to Ground

If a node has no DC path to ground, Questa ADMS cannot determine the voltage at the node during computation of the quiescent point.

A node has no DC path to ground if the design's branch equations do not specify a voltage and current relationship between the node and the ground node. Each branch equation may be built into an Eldo primitive, may be a VHDL-AMS through quantity and its associated simultaneous statement, or may be a Verilog-AMS branch contribution statement.

By itself, a capacitor does not provide a DC path to ground. Questa ADMS reports a node with no DC path to ground if the only path from the node to ground is through an Eldo primitive capacitor, or through a VHDL-AMS or Verilog-AMS branch equation that acts like a capacitor.

Similarly, a current source does not provide a DC path to ground. Questa ADMS reports a node with no DC path to ground if the only path from the node to ground is through a current source, or through an Eldo primitive current source, or through a VHDL-AMS or Verilog-AMS statement that acts like a current source.

For certain design configurations, Questa ADMS is able to detect nodes with no DC path to ground before starting the quiescent point computation. For example, the following design contains a VHDL-AMS current source that has no load in the top-level SPICE circuit:

```
--test_module.vhd
LIBRARY IEEE;
USE IEEE.electrical_systems.ALL;
ENTITY test_module IS
    PORT (terminal inp, outp : electrical);
END ENTITY test_module;

-- Provides a current source between outp and ground
ARCHITECTURE current_source OF test_module IS
    QUANTITY vin ACROSS iin THROUGH inp;
    QUANTITY vout ACROSS iout THROUGH outp;
    CONSTANT Rin : real := 1.0e3;
BEGIN
    iout == 1.0;
    vin == Rin*iin;
END ARCHITECTURE current_source;

*test_module.cir
.MODEL test_module(current_source) macro lang=vhdlams lib=test_lib
Y1 test_module(current_source) PORT: 1 2

V1 1 0 1
R1 1 3 1
C1 3 0 12p

.DC
.END
```

For this design, Questa ADMS reports error eldo-112 in the transcript before starting the quiescent point computation:

```
** Error: (eldo-112)  NODE "2": No DC path to ground.
+   .OPTION RGNDI =<value> can be set to force a grounded resistor on this
node.
```

You can correct the problem by adding a component that sinks current at node 2. You can also correct the problem by setting the RGNDI option. Refer to [.OPTION RGNDI](#) in *Eldo Reference Manual* for more information.

Tip

Nodes with no DC path to ground can lead to convergence issues. If you know that the design does not have convergence issues, use option HDL_DCPATH_SEVERITY to change error eldo-112 to a warning and continue the simulation:

```
.OPTION HDL_DCPATH_SEVERITY=WARNING
```

If Questa ADMS is unable to detect nodes with no DC path to ground before starting the quiescent point computation, Questa ADMS reports warning eldo-127 in the transcript during the quiescent point computation. For example, the following design contains a Verilog-AMS module that can be switched to produce no DC path to ground during a simulation:

```
//test_switch.va
`include "disciplines.h"

module test_switch(cntl,in,out);
    inout cntl,in,out;
    electrical cntl,in,out;
    parameter real Rc = 1000;

    analog begin
        if (V(cntl) > 2.5) begin
            I(in,out)<+ V(in,out) / 1000;
        end else begin
            I(in,out) <+ 0.0;
        end
    end
endmodule

*test_switch.cir
.VERILOG test_switch.va
.MODEL test_switch MACRO LANG=VERILOGA
Y1 test_switch PORT: cntl in out

Vcntl cntl 0 PWL(0 -5.0 1u -5.0 1.001u 5 2u 5 2.001u -5 3u -5)
Iin in 0 DC -1m
Rout out 0 1k
```

```
.TRAN 3u 3u
.PLOT TRAN V(cntl)
.PLOT TRAN V(out)
.END
```

For this design, Questa ADMS reports warning eldo-127 in the transcript during the quiescent point computation:

```
** Warning: (eldo-127)   NODE "IN":Possibly no DC path on that node..
```

Questa ADMS then reports a convergence issue and stops the simulation:

```
** Error: (vasim-1355) Analog DC computation will not converge, computation
aborted : Transient simulation cannot begin.
```

The problem is that the Verilog-AMS module test_switch either connects or disconnects the nodes in and out, and is set up to initially disconnect them. When the voltage at cntl is 5V, test_switch connects in and out. However, when the voltage at cntl is -5V, test_switch disconnects in and out.

You can correct the problem by changing the definition of the voltage source Vcntl so that the voltage at cntl is initially 5V and there is no initial disconnection in the design. This enables Questa ADMS to successfully compute the quiescent point.

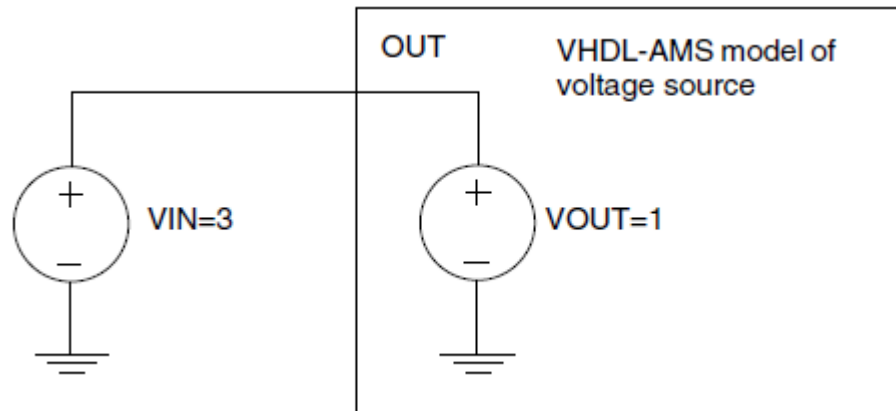
Note

You cannot necessarily correct the problem using the option UIC. If UIC is specified, the error will not be reported because it is specific to DC analysis, but there will still be convergence issues if the voltage at the problematic node affects computation of the first transient point.

Voltage Loop

If a VHDL-AMS voltage source and a SPICE voltage source are connected to ground and the same net, a "Voltage loop found" error is issued just before starting the quiescent point computation.

A voltage loop occurs when two voltage sources of different values are inserted in parallel between the same two nodes.



This yields the following pair of equations:

$$V_2 - V_1 = A$$

$$V_2 - V_1 = B$$

If A and B are different, the system has no solution and the simulator reports a singularity.

Here is the source code corresponding to the schematic:

```
--voltage_source.vhd
LIBRARY IEEE;
    USE IEEE.electrical_systems.ALL;
ENTITY VOLTAGE_SOURCE IS
    PORT (TERMINAL OUTP: electrical);
END ENTITY VOLTAGE_SOURCE;
ARCHITECTURE A OF VOLTAGE_SOURCE IS
    QUANTITY VOUT ACROSS IOUT THROUGH OUTP;
    BEGIN
        VOUT == 1.0;
    END ARCHITECTURE A;
--end voltage_source.vhd

*voltage_source.cir
.MODEL VOLTAGE_SOURCE(A) MACRO LANG=VHDLAMS LIB=LIB
Y1 VOLTAGE_SOURCE(A) PORT: OUT
VIN OUT 0 3
.TRAN 10N 10N
.PLOT TRAN V(OUT)
.END
```

Inconsistent Equations

A singularity can occur when one or more equations derived from a model cannot be solved in DC.

Take as an example the model *integrator.vhd*, which describes a simple integrator:

```
-- integrator.vhd
LIBRARY IEEE;
USE IEEE.electrical_systems.ALL;
ENTITY EQUATION_SYSTEM IS
    PORT (TERMINAL INP, OUTP: ELECTRICAL);
END ENTITY EQUATION_SYSTEM;
ARCHITECTURE A of EQUATION_SYSTEM IS
    QUANTITY VIN ACROSS INP;
    QUANTITY VOUT ACROSS IOUT THROUGH OUTP;
    QUANTITY INTEGRAL: REAL;
BEGIN
    INTEGRAL'DOT == VIN; --EQUATION(1)
    VOUT == INTEGRAL; --EQUATION(2)
END ARCHITECTURE A;
end integrator.vhd

*integrator.cir
.MODEL EQUATION_SYSTEM(A) MACRO LANG=VHDLAMS LIB=LIB
Y1 EQUATION_SYSTEM(A) PORT: IN OUT
VIN IN 0 PWL (0 5 5u 5 5.001u 0 40u 0)
.TRAN 40u 40u
.PLOT TRAN V(IN)
.PLOT TRAN V(OUT)
.END
```

The derivative of the free quantity INTEGRAL is set equal to the input voltage in EQUATION(1). However, during the search for the quiescent point all derivatives are held at zero. If the input voltage is not also zero, the equation has no solution. Questa ADMS will report a singularity.

The same sort of singularity can be the result of an inconsistency between two equations, or between one equation and a combination of other equations. To illustrate this case, take the example of *multiple_solution.vhd* (see below) and replace equation (3) by:

$YV - ZV == -2$

Multiple Solutions

If a design has multiple solutions in DC, Questa ADMS will suggest a solution from among them according to initial conditions. Then the simulation will be stopped and a message reported that a singularity has been found.

Independent of the DC computed, the option NOMATSING allows the simulation to continue.

In the following example, the three equations are not linearly independent:

```
-- multiple_solution.vhd
LIBRARY IEEE;
    USE IEEE.electrical_systems.ALL;
ENTITY MULTIPLE_SOLUTIONS IS
END ENTITY MULTIPLE_SOLUTIONS;
ARCHITECTURE A OF MULTIPLE_SOLUTIONS
    TERMINAL X, Y, Z: ELECTRICAL;
    QUANTITY XV ACROSS XI THROUGH X;
    QUANTITY YV ACROSS YI THROUGH Y;
    QUANTITY ZV ACROSS ZI THROUGH Z;
BEGIN
    XV + YV == 4.0; --EQUATION(1)
    XV + ZV == 2.0; --EQUATION(2)
    YV - ZV == 2.0; --EQUATION(3)
END ARCHITECTURE A;
--end multiple_solution.vhd

*multiple_solution.cmd
.TRAN 10N 10N
```

EQUATION(3) is a linear combination of EQUATION(1) and EQUATION(2)—the difference. The system has an infinite number of solutions because the number of independent equations is less than the number of unknowns. The simulator will report a singularity.

The problem is obvious in this trivial example. However in a real design, there will be many more than three equations. The only safeguard is to be careful in formulating equations derived from a physical model.

Numerical Noise

Every current source must have a load, but an arbitrary load may not be sufficient. The value of the load must be realistic.

As the conductance of the load decreases, the voltage across the load becomes more insensitive to changes in the current. The simulator uses iterative numerical techniques with finite precision. If the load is small enough, the change in voltage in response to a change in current seem to disappear altogether—it gets lost in the numerical noise. Convergence on a solution takes many iterations to obtain the required accuracy or convergence cannot be obtained at all.

The following example illustrates this case. A current source I1 flows through four resistors. Two of them have large resistances. Even though the system of equations is complete and consistent, numerical noise leads the simulator to report a singularity.

```
* numerical_noise.cir
I1 1 0 1
```

```
R1 1 2 1  
R2 2 3 1e20  
R3 3 4 1e20  
R4 4 0 1  
.DC  
.END
```


G. Interface Association File (.assoc)

When associating a SPICE subcircuit to a digital behavioral model, you can use an association file (.assoc) to associate subcircuit ports to the header of the corresponding digital model.

To associate a subcircuit to a digital behavioral model, use the **vaspi** or **vacom -spice** commands.

.assoc File..... G-2

.assoc File

An association file may be created explicitly, or automatically generated using the [Interface Matcher](#).

When associating a SPICE subcircuit to a digital behavioral model, an association file can be used to associate subcircuit ports to the header of the digital corresponding model. If such a file is not provided, the mapping by position is taken by default.

By default, the association file will be saved to the location from which the [Interface Matcher](#) was launched. The saved location is reported in the terminal window when generation is complete, along with a full description of all performed processes.

If a previously generated association file exists with the same name, the new file will overwrite the existing file. Use the **Save as** button to specify a different name and location of the association file under which it is to be saved.

Note

If any internal errors occur during the [Interface Matcher](#) wizard session, an error file named `tcl_script.error` is generated, and saved in the same directory from which the Interface Matcher was launched.

See also Mapping Data Type in the *Questa SIM User's Manual*.

Format

The association file is structured in three sections:

- Header section
Lists the different units that will be associated.
- Port Association section
Contains the entire normal mapping.
- Special Units 1 and 2 sections
Used when there are not the same number of ports for both units.

Any of the four sections can be empty, except the Header section. It is important that all ports in both sides (SPICE side and digital model side) are listed, otherwise an error message is displayed at elaboration.

In the Association file, words in **Bold** face correspond to keywords. Keywords and SPICE port names are case insensitive. However, digital port names for Verilog models are case sensitive. For Verilog names, writing them in lowercase is accepted if there is only one port name with the same character, disregarding the case. CR means Carriage Return (new line).

Comments are accepted in the association file as follows:

- at the beginning of a line starting with the '#' character and finishing at the end of the current line;

anywhere in a line, proceeding a '#' character, until the end of the line.

Header Section

The first section in the Association file contains the header file. It lists the different units that will be associated; the header of one of the units is associated to the upper level (parent), and the other unit is the child. The order of the associated units is not important.

```
header_section ::=
    '[' Header ']' CR
    Unit 1 ':' unit_type unit_name
    Unit 2 ':' unit_type unit_name
unit_type ::=
    VHDL
    | VHDL-AMS
    | Verilog
    | Verilog-AMS
    | SPICE
unit_name ::=
    behavioral_unit_name
    | spice_unit_name
behavioral_unit_name ::=
    [ logical_library_name ':' ]
    primary_unit_name
spice_unit_name ::=
    spice_subckt_name [ '@' file_path_name ]
```

Port Association Section

The second section contains the entire normal mapping. Normal mapping is a Unit 1 port associated to a Unit 2 port; in a digital model substituted by a SPICE subcircuit, the association is between the ports of the entity and the ports of the subcircuit. The syntax is defined as follows:

```
port_association_section ::=
    '[' Port association ']' CR
    [ port_association_list ]
port_association_list ::=
    By_position CR
    | By_name CR
    | port_association_elt { port_association_elt }
port_association_elt ::=
    unit_1_port_name '='>
    unit_2_port_name CR
port_name ::=
    port_name
    | aggregate
```

```
aggregate ::=
    '(' port_name { ',' port_name } ')'
```

The port association section begins with the flag [Port association] and is followed by a succession of associations, one line per association.

There are two predefined default mechanisms, one providing the association by position, and one providing the association by name (same port names for both units; if not the case, an error message is returned). If one of the default mechanisms is chosen, it is not possible to provide any special sections (the two other sections must be empty).

If any default mechanism is chosen, both units must have the same number of scalar elements. If this is not the case, an error message is returned.

When the association is provided (no default mechanisms), the association elements must be provided one per line. An association is composed of a pair of a Unit 1 element and its associated Unit 2 part. Here two cases may occur:

- The association is combining two scalar elements. In this case, the association is simple, one port is associated to another.
- The association is combining two composite elements. In this case, it is important to associate at least one complete composite port to either another composite port having the same length or a collection of elements (scalar or not). It is not allowed to associate two aggregates.

In the second case, the scalar number of ports must be the same for the two elements of the association. The association is done by order at this level: left element (vector'left in VHDL) of the complete port to the first element in the aggregate list. For example: consider a VHDL with a vector *v* which should match with three SPICE scalars: *s1*, *s2*, *s3*. The .assoc file contains the line ***v* => (*s1 s2 s3*)**.

- If *v* is an ascending vector, for instance `std_logic_vector(1 to 3)` then the association is equivalent to:

```
v(1) => s1
v(2) => s2
v(3) => s3
```

- If *v* is a descending vector, for instance `std_logic_vector(3 downto 1)` then the association is equivalent to:

```
v(3) => s1
v(2) => s2
v(1) => s3
```

Note that different ports of one unit can be connected to the same port of the other unit. Thus some ports may appear several times in this list.

Only ports of the unit connected to the parent in the design may be connected to several ports of the child, and not the reverse.

Special Units 1 and 2 Sections

The third and fourth sections contain the special units 1 and 2 mapping. These sections are important because, in many cases, there are not the same number of ports for both units. For example, for SPICE units, ports that are supplying power must be considered. The syntax is defined as follows:

```
special_unit_<i>_section ::=
    '[' Unit <i> special  ']' CR
    [ special_association_list ]
special_association_list ::=
    special_port_association_elt
    { special_port_association_elt }
special_port_association_elt ::=
    unit_i_port_name '=>' [ global_net ] CR
global_analog_net ::=
    global_SPICE_net_name
    | global_VHDL(-AMS)_net_name
global_VHDL(-AMS)_net_name ::=
    [ logical_lib_name ':' ]
    package_name '.' net_name
```

These sections only allow you to associate Unit <i> ports to:

- Global SPICE declarations (.global declaration in the .cmd file or .cir file, at the top of the description, not within subcircuits).

In this case, the name of the global SPICE net is the same as that in the SPICE .global card. For example, for the following SPICE declaration:

```
.global vdd vss
```

VDD or VSS should be used for the name of the corresponding global SPICE net.

- VHDL-AMS terminal declarations in VHDL-AMS packages (global VHDL-AMS terminal declarations).

In this case, the name of the form is:

```
Lib: Pkg .Net
```

The library is provided using its logical name. If omitted, the working library (at simulation time) will be considered.

- VHDL or VHDL-AMS digital signal declarations in VHDL or VHDL-AMS packages (global VHDL or VHDL-AMS signal declarations).
- Open associations, by not specifying anything after the '=>' symbol.

Parameters

Header Section

- **unit_type**
Specifies the language of the behavioral model or subcircuit.
- **unit_name**
Specifies the analog subcircuit or behavioral model name.
- **logical_library_name**
Specifies the library in which the digital model is located. If omitted, the working library will be used.
- **file_path_name**
Specifies the file in which the subcircuit is contained. If omitted, the subcircuit will be taken from the *.cmd* or *.cir* file.

Note

For a VHDL or VHDL-AMS unit, only the entity is requested because the header is only part of the entity.

Examples

A digital model is compiled in the SPICELIB library:

```
entity model is
    port (inp : Std_logic;
          q, qb : Std_logic;
          outp : out Std_logic);
end entity model;
```

and its corresponding implementation in SPICE (in the *.cmd* file):

```
.SUBCKT spice_model e s control vdd vss
...
.ENDS
```

There are not the same number of ports in both models, therefore an association file is required, as follows:

```
# File model.assoc
[Header]
Unit 1: VHDL spicelib:model
Unit 2: SPICE spice_model
[Port association]
```

```
qb => control
inp => e
outp => s
[Unit 1 special]
q =>                                # nothing connected to this port
[Unit 2 special]
vdd => vdd                          # port vdd connected to global vdd
vss => vss
```

A mapping table can be used with vaspi as shown below:

```
vaspi -interface model.assoc
<digital_unit> <subckt_name>[@<file_name>]
```

-interface is used for providing the interface association file.

vaspi can directly generate a mapping table with a default-mapping name:

```
vaspi -by_name <digital_unit> <subckt_name>[@<file_name>]
```

This is the default mechanism. For orthogonality, an option is also provided to access this mode for vaspi to directly generate a mapping table with a default-position name:

```
vaspi -by_position <digital_unit> <subckt_name>[@<file_name>]
```

Related Topics

[Instantiating a SPICE Subcircuit from VHDL, VHDL-AMS or Verilog](#)

[vaspi \[Questa ADMS Command Reference\]](#)

[Interface Matcher](#)

H. Statistics File Reference

This appendix provides a reference for the content reported in the statistics (.stat) file, with a typical example of the output.

For information on how to use the data, see [Simulation Statistics](#).

The statistics file is divided into three main sections:

Definition Parameters.....	H-2
Elaboration Parameters.....	H-4
Simulation Parameters.....	H-6
Statistics File Example.....	H-8

Definition Parameters

The statistics report contains various definition parameters.

- General Design Info
 - Design: List of Top Level Design Names (for multiple top designs)
 - Machine: Machine and Platform information
 - Starting time: Starting simulation time and date record
 - Command: Report of the command line used to launch the simulation
 - Working directory: The location into which digital source files and simulation output files (unless a different location was specified using the `-outpath` argument in the launch command) are placed.
 - System initialization files: The location of the *modelsim.ini* and *eldo.ini* files.
 - Output files: The files containing the simulation results.
 - Digital Simulation Resolution: The time resolution specified for the simulation.
- Summarized Elaboration Info
 - Hierarchical layers distinguished: Listing of language-distinguished hierarchical layers, used to detect the different descriptions used in the design
 - Maximum hierarchical levels: Depth of the design
 - A table showing the number of models in the design, and the number of instances of each, listed by language type.
 - A table showing the CPU time for the simulation versus the elapsed time.
- Summarized Simulation Info
 - Requested simulation time:
 - Overall simulation CPU usage:
 - Average processors used: (for multithreaded simulations)
 - A global report about the CPU time invested in diverse aspects, CPU time (or Threads CPU Time for multithreaded simulations) and Elapsed time.
 - ◊ DC/Quiescent and AC: Time needed for DC/Quiescent and AC calculations
 - ◊ Transient: Time needed for transient simulation
 - ◊ Analog only (DC/AC/TRAN): Analog kernel invested time

For a Questa ADMS/ADiT simulation, analog CPU time is split into Eldo only and ADiT only.
 - ◊ Digital only (DC/TRAN): Digital kernel invested time
 - ◊ Total: Total CPU time and Elapsed time summary
 - Mixed-signal interaction

A table containing the following:

- ◇ D2A (any breaks) lists the number of events and rejections
- ◇ A2D (Eldo-built-in conv) lists the number of events and rejections
- ◇ A2D (Vhdl-Ams Q'above) lists the number of events and rejections (non-effective and effective).
Note that these do not include Q'Slew rejections, they are included in the Rejected time steps table under Simulation: Analog kernel (see [Simulation Parameters](#)).
- ◇ The Total number of events

Note

Limitation: Data for multithreaded simulations that use ADiT is not reported.

Elaboration Parameters

The statistics report contains various elaboration parameters.

- **Elaboration: Packages**

This is a list of HDL packages that are actually used in the design via a “use clause” (for example, use work.pack.all); this is not a list of all the compiled packages. The list is in two columns:

- Name: the name of the package, in the format <library_name>.<package_name>
- Engine: either Questa ADMS or QuestaSim, denoting which engine has handled the package.

- **Elaboration: Models**

A table listing the models inside the design.

- Design Unit: The model name, in the format <entity_name>(<arch_name>) or <module_name>
- Language: The model language (SPICE, VHDL-AMS, VHDL, Verilog-AMS, Verilog, and so on)
- Logical Library: In the case of models, the logical name of the library where the model is compiled
- Nb. of Inst: The number of instances of the model in the design

- **Converters: A sub-table listing the boundary element models used inside the design.**

- Model Name: The name of the boundary element model
- Type: HDL model or built-in/Eldo boundary elements
- Direct: The direction (A2D or D2A) of the boundary element
- Nb. of Inst: The number of instances (for automatic insertion) of each boundary element in the design
- Parameters: available for the Eldo D2A and A2D built-in converters.

- **A table listing the number of Nodes and Devices in Eldo**

- Number of Nodes (totalled) broken down into:
 - ◊ Nodes
 - ◊ Stimulus nodes
- Number of Devices (totalled) broken down into:
 - ◊ Number of resistors
 - ◊ Number of capacitors
 - ◊ Number of voltage sources
 - ◊ Number of current sources
 - ◊ Number of diodes
 - ◊ Number of MOS

- ◊ Number of ADMS analog devices (includes Y instances for VHDL-AMS and Verilog-AMS models but not CFAS models)
- Memory Allocated:
 - A table detailing the amount of memory (in kilobytes) used by the Questa ADMS kernel (including Eldo and Questa SIM) and the Questa ADMS GUI during each phase of the simulation run.
 - A table detailing the amount of memory (in kilobytes) allocated and used by Solido Waveform Analyzer and the JWDB server during the elaboration phase.

To view a graphical representation of this information in interactive mode, select View on the **Stat Summary Window**. Use the selection button to choose **Memory** from the dropdown list.

Note

The elaboration report excludes information for third party simulators such as Simulink¹ and MATLAB¹.

¹ Simulink and MATLAB are registered trademarks of The MathWorks, Inc.

Simulation Parameters

The statistics report contains various simulation parameters.

- Simulation: Mixed-Signal activity
 - Top 10 A2D boundary elements

The ten most active A2D boundary nets are listed together with the total number of events each was involved with. The number of all rejections on each net is reported as well as the number of rejections that cause the simulator to significantly increase its number of time steps (effective rejections). For non-effective rejections the time step is gradually relaxed, whereas for effective rejections the simulator continues to take very tight time steps until it can gradually relax the time step again. The reported number of effective rejections is not a subset of the number of (non-effective) rejections.

Note

10 is the default value for the number of most active boundary elements to be listed, but you can specify a different number using the BoundaryStatListNb variable (see [Simulator Control Variables](#)).

- Top 10 D2A boundary elements

The ten most active D2A boundary nets are listed together with the total number of events each was involved with. The number of all rejections on each net is reported as well as the number of effective rejections.

- Top 10 D2A Events

The following types of D2A events are listed in a table, each with its path and position in the source file:

- ◊ Break Statement
- ◊ All attributes that generate a break statement implicitly (for example, S'Ramp Break, S'Slew Break, Q'ZOH Break, Q'ZTF Break)
- ◊ disable_signal_spy Break
- ◊ noforce or signal_release Break
- ◊ Checkpoint Break
- ◊ Posedge, Negedge or Any Edge Event Break

- Top 10 A2D Events

The following types of events are listed in a table, each with its path and position in the source file:

- ◊ Cross events
- ◊ Q'Above

- Simulation: Digital kernel

- A table listing the type and number of Events:

- ◊ The number of events handled by Questa ADMS only
- ◊ The total number of events in the digital kernel
- Simulation: Analog kernel

A table listing statistics for Eldo/Questa ADMS (not Fast-SPICE):

 - Newton/Jacobian Order: the lower the number, the easier the system is to solve for.
 - Terms in matrix: the lower the number, the easier the system is to solve for.
 - Matrix Sparsity(%): percentage of zero terms in the matrix to solve for, the larger the number, the easier the system is to solve for.
 - Newton iterations: also regarded as the number of iterations effectuated on the analog kernel side to solve for the accepted time steps during the simulation
 - Average number of Newton iterations: equivalent to the number of Newton iterations/Number of accepted time steps. It provides a rough average estimation for the number of iterations needed to solve for one time step. A number greater than or equal to 3.0 shows that the design has some difficulty to converge because of high non-linearity or a bad description.
 - Accepted time steps: total number of time steps accepted during the simulation
 - Rejected time steps: total number of time steps rejected during the simulation
 - ◊ due to local truncation error, for further explanation refer to [Time Step Control](#) in the *Eldo User's Manual*
 - ◊ due to Newton
 - ◊ due to mixed-signal interaction, see the Mixed-Signal interaction table above
 - ◊ Evaluation of active devices: number of MOS, DIODE, BJT, JFET devices: number of evaluations (calls) of the models by Eldo.
- Simulation: Reject, synchro, ...
 - Rejections listed on pure analog nets together with corresponding causes such as time synchronization, HMAX and LTE.
 - Contributions from design units listed as a percentage of the total units in the design.
- ADIT Infos

This section lists all data coming from Fast-SPICE, if any: DC time, transient time, Eldo => Fast-SPICE interaction. This data will be integrated into the preceding Analog tables in a future release.

Statistics File Example

The example *.stat* file contains information about the design, elaboration, and simulation.

```

ADMS v5.5_1.1 Wed Sep 21 14:59:44 GMT 2011
ELDO 11.2 (64 bits) Wed Oct 26 11:10:53 GMT 2011
Solido Wave 11.2 Production Thu Oct 27 08:11:15 GMT 2011

*****
*           General Design Info           *
*****

Design name: read
Machine: abc-rhel45-opt-01 Linux 2.6.9-55.ELsmp #1 SMP Fri Apr 20 16:36:54 EDT 2007 x86_64
Starting time: 10:05:40 11/02/2011
Command: /my_source_directory/read.cir -lib TWIGO_AMS -t 1fs -statfile /my_working_directory/statfile.stat
-do /my_working_directory/read.do -eldoopt '-outpath /my_working_directory/TWIGO_AMS_RES '

Working directory: /my_working_directory

System initialization files:
modelsim.ini
/home/me/elido.ini

Output files:
/my_working_directory/TWIGO_AMS_RES/read.chi
/my_working_directory/TWIGO_AMS_RES/read.wdb (4 waveforms)
/my_working_directory/TWIGO_AMS_RES/read.conv

Digital Simulation Resolution: 1fs

*****
*           Summarized Elaboration Info           *
*****

Hierarchical layers distinguished:
SPICE (ELDO)
    Vhdl-Ams (Questa ADMS)

Maximum hierarchical levels: 9

Number of packages: 6

-----+-----
Model type    | N° of models | N° of instances|
-----+-----+-----
Vhdl-Ams      |           11|           22|
-----+-----+-----
Spice subckt  |           70|          3793|
-----+-----+-----

-----+-----
CPU time      | Elapsed time
-----+-----

```



```

0h 0mn 3s 0ms | 0h 0mn 20s 0ms
-----+-----

*****
*      Summarized Simulation Info      *
*****

Requested simulation time: 2 ns
Overall simulation CPU usage: 30%

-----+-----+-----
Items          | CPU time      | Elapsed time
-----+-----+-----
DC/Quiescent and AC | 0h 0mn 2s 0ms | 0h 0mn 8s 0ms
Transient         | 0h 0mn 5s 0ms | 0h 0mn 5s 0ms
-----+-----+-----
Total           | 0h 0mn 7s 0ms | 0h 0mn 13s 0ms
-----+-----+-----
Analog only (DC/AC/TRAN) | 0h 0mn 5s 0ms | 0h 0mn 5s 0ms
Digital only (DC/TRAN)   | 0h 0mn 2s 0ms | 0h 0mn 8s 0ms
-----+-----+-----

Or for Multi-threaded Simulations:

Requested simulation time: 1 us
Overall simulation CPU usage: 199%
Average processors used: 3

-----+-----+-----
Items          | Threads CPU time | Elapsed time
-----+-----+-----
DC/Quiescent and AC | 0h 0mn 20s 0ms | 0h 0mn 10s 0ms
Transient         | 0h 7mn 16s 0ms | 0h 3mn 38s 0ms
-----+-----+-----
Total           | 0h 7mn 36s 0ms | 0h 3mn 49s 0ms
-----+-----+-----
Analog only (DC/AC/TRAN) | 0h 7mn 33s 0ms | 0h 3mn 46s 0ms
Digital only (DC/TRAN)   | 0h 0mn 3s 0ms | 0h 0mn 2s 0ms
-----+-----+-----

Mixed signal interaction:
-----+-----+-----+-----
Type of Events      | Events | All Rej. | Rejection reducing timestep significantly
-----+-----+-----+-----
D2A(any breaks)    | 10 | 1 | 0
A2D(Elco-Built-in-conv) | 1 | 0 | 0
A2D(Vhdl-Ams-Q'above) | 122 | 15 | 2
-----+-----+-----+-----
Total              | 133 | 16 | 2
-----+-----+-----+-----

*****
*      Elaboration: Packages      *
*****
-----+-----+

```

Used packages		Engine	
STD.standard		Questa	ADMS
IEEE.math_real		Questa	ADMS
IEEE.std_logic_1164		Questa	ADMS
IEEE.FUNDAMENTAL_CONSTANTS		Questa	ADMS
IEEE.ELECTRICAL_SYSTEMS		Questa	ADMS
MGC_AMS.conversion		Questa	ADMS

* Elaboration: Models *			

Design Unit	Language	Logical Library	Nb. of Inst
sensetemp(functional)	Vhdl-Ams	TWIGO_AMS	1
vreadreg(functional)	Vhdl-Ams	TWIGO_AMS	1
ypgen(functional)	Vhdl-Ams	TWIGO_AMS	1
yrgen(functional)	Vhdl-Ams	TWIGO_AMS	1
proglload(functional)	Vhdl-Ams	TWIGO_AMS	8
sense(functional)	Vhdl-Ams	TWIGO_AMS	4
sswxuemc(functional)	Vhdl-Ams	TWIGO_AMS	1
vgatesw(functional)	Vhdl-Ams	TWIGO_AMS	1
hvdetector(functional)	Vhdl-Ams	TWIGO_AMS	2
gatedrv(functional)	Vhdl-Ams	TWIGO_AMS	1
vpdreg(functional)	Vhdl-Ams	TWIGO_AMS	1
memflash_top2_g65	Spice	-	1
barra2bis2_g61	Spice	-	1
ivhd_g1	Spice	-	141
en50bs3ju	Spice	-	1244
ep50bs3ju	Spice	-	1171
virefgen_m01_g18	Spice	-	1
mydsp_g19	Spice	-	1
ydec_g7	Spice	-	1
alberopass_g4	Spice	-	4
ynpass_g2	Spice	-	32
alberopassp_g5	Spice	-	4
ynpassp_g3	Spice	-	32
senseprog_g8	Spice	-	4
refsensegen_g6	Spice	-	4
intrif_g20	Spice	-	1
nr2hd_g9	Spice	-	67
rcrd_sw_g10	Spice	-	4
ynmpredec_g21	Spice	-	1
ymdec_g14	Spice	-	4
ao8hd_g11	Spice	-	6
yndec_g15	Spice	-	8
ao7hd_g12	Spice	-	8
nd2hd_g13	Spice	-	42
logic_b2bis_g22	Spice	-	1
or2hd_g16	Spice	-	19
or3hd_g17	Spice	-	2
bfhdp_g62	Spice	-	18
barra1bis_tc02_g63	Spice	-	1

command_g48	Spice	-		1
an2hd_g25	Spice	-		11
an3hd_g26	Spice	-		1
nr3hd_g27	Spice	-		10
ivhdp_g23	Spice	-		15
nd3hd_g24	Spice	-		14
cdonwpo1m1	Spice	-		26
wrsdelay_mim4_g28	Spice	-		1
progdelay_mim4_g29	Spice	-		1
impulsoreset_mim4_g30	Spice	-		2
flip_mim4_g31	Spice	-		2
tmode_g49	Spice	-		1
tdlatchx8_g33	Spice	-		6
tdlatch_g32	Spice	-		48
tdlatcgen_g34	Spice	-		1
decod_g50	Spice	-		1
negdec_g38	Spice	-		1
bit1prog_x1x2x4_g39	Spice	-		1
fd2qhd_g35	Spice	-		3
fd4qhd_g36	Spice	-		1
mux21hd_g37	Spice	-		4
dumdec_g40	Spice	-		1
vpddec_g41	Spice	-		1
hvdec_g42	Spice	-		1
command2_g51	Spice	-		1
rigind_g44	Spice	-		16
ivhdx4_g43	Spice	-		16
disgen_g45	Spice	-		1
rigin_g46	Spice	-		1
riginn_g47	Spice	-		1
matrice_sel_g64	Spice	-		1
matrice_g60	Spice	-		1
mathalfc_g58	Spice	-		1
matbitc_g56	Spice	-		4
matcol_g54	Spice	-		128
cellp_g53	Spice	-		128
fep	Spice	-		128
matcolc_g55	Spice	-		128
cell_g52	Spice	-		128
fee	Spice	-		128
mathalf_g59	Spice	-		1
matbit_g57	Spice	-		4

Converters

Model Name	Type	Direct.	Nb. of Inst.	Parameters
D2A_STD_LOGIC_DEFAULT	built-in/ELD0	D2A	6	MODE = STD_LOGIC VHI = 5.100000e+00 VLO = 1.000000e-01 TRISE = 1.000000e-10 TFALL = 1.000000e-10 RZ = 1.000000e+09

```
|ZCAP      = 1.000000e-12
|RRISE     = 1.200000e+00
|RFALL     = 1.200000e+00
|LOWCAP    = 1.000000e-12
|HIGHCAP   = 1.000000e-12
|WEAHIGHRES= 1.100000e+00
|WEALOWRES = 1.100000e+00
|XEVAL     = PREVIOUS

-----+-----+-----+-----+-----+
A2D_STD_LOGIC_DEFAULT|built-in/ELD0| A2D |          1 |
|MODE      = STD_LOGIC
|VHIREF    = NN2
|VLOREF    = NN1
|VTH1REL   = 2.600000e-01
|VTH2REL   = 7.600000e-01
|VTHREL    = 5.100000e-01
|TX        = 1.000000e-01
|R         = 1.000000e+09
|C         = 1.000000e-12
|RZ        = 1.000000e+09
|STR       = STRONG

-----+-----+-----+-----+-----+

-----+-----+
Items | Eldo |
-----+-----+
Nodes (total) | 1271|
-----+-----+
Nodes| 1244|
Stimulus nodes| 27|
-----+-----+
Devices (total) | 3091|
-----+-----+
Resistors| 2|
Capacitors| 335|
Voltage sources| 26|
Current sources| 1|
Diodes| 50|
MOS| 2655|
ADMS analog devices| 22|
-----+-----+

*****
*      Memory Allocated      *
*****

-----+-----+-----+-----+-----+
| Elaboration | DC cycle 1 | DC cycle 2 | Simulation
-----+-----+-----+-----+-----+
Questa ADMS kernel | 536628 kB | 593164 kB | 593084 kB | 596132 kB
-----+-----+-----+-----+-----+
Questa ADMS GUI | 128484 kB | 139740 kB | 139740 kB | 140800 kB
-----+-----+-----+-----+-----+

Solido Wave/jwdb server:
-----+-----+-----+-----+-----+
```

```

memory allocated | memory being used
-----+-----
          123840 kB|          59472 kB
-----+-----

```

```

*****
*   Simulation: Mixed-Signal activity   *
*****

```

Top 10 A2D boundary elements

```

-----+-----+-----+-----
          Net          | Events | All Rej. | Rejection reducing timestep significantly
-----+-----+-----+-----
:read:xi1:xibarra2:sapre |      1 |      0 |      0
-----+-----+-----+-----
Total |      1 |      0 |      0
-----+-----+-----+-----

```

Top 10 D2A events

```

-----+-----+-----+-----
          Net / Context | Events | All Rej. | Rejection reducing timestep significantly
-----+-----+-----+-----
BREAK STATEMENT      |      3 |      0 |      0
region: :read:xi1:xibarra2:ximydsp:xisnsprl_0:yi358 model: sense(functional) file: /my_working_directory/
MODELS/sense.vhd(130:1)
-----+-----+-----+-----
BREAK STATEMENT      |      3 |      0 |      0
region: :read:xi1:yivpdreg model: vpdreg(functional) file: /my_working_directory/MODELS/
vpdreg2.vhd(155:1)
-----+-----+-----+-----
BREAK STATEMENT      |      1 |      1 |      0
region: :read:xi1:xibarra2:yi109 model: sswxuemc(functional) file: /my_working_directory/MODELS/
sswxuemc2.vhd(134:1)
-----+-----+-----+-----
BREAK STATEMENT      |      1 |      0 |      0
region: :read:xi1:xibarra2:ximydsp:xisnsprl_2:yi358 model: sense(functional) file: /my_working_directory/
MODELS/sense.vhd(130:1)
-----+-----+-----+-----
sm0'ramp              |      1 |      0 |      0
region: :read:xi1:xibarra2:ximydsp:xicoldec:yiynge model: yrgen(functional) file: /my_working_directory/
MODELS/yrgen2.vhd(89:16)
-----+-----+-----+-----
sn0'ramp              |      1 |      0 |      0
region: :read:xi1:xibarra2:ximydsp:xicoldec:yiynge model: yrgen(functional) file: /my_working_directory/
MODELS/yrgen2.vhd(107:16)
-----+-----+-----+-----
Total                |     10 |      1 |      0
-----+-----+-----+-----

```

Top 10 A2D events

```

-----+-----+-----+-----
          Net / Context | Events | All Rej. | Rejection reducing timestep significantly
-----+-----+-----+-----

```

vdatoxd'above		1		1		1	
region: :read:xi1:xibarra2:ximydsp:xisnsprl_2:yi358						model: sense(functional)	file: /my_working_directory/
MODELS/sense.vhd(105:53)							
-----+-----+-----+-----							
vdatoxd'above		1		1		1	
region: :read:xi1:xibarra2:ximydsp:xisnsprl_2:yi358						model: sense(functional)	file: /my_working_directory/
MODELS/sense.vhd(104:53)							
-----+-----+-----+-----							
vdatoxd'above		1		1		1	
region: :read:xi1:xibarra2:ximydsp:xisnsprl_2:yi358						model: sense(functional)	file: /my_working_directory/
MODELS/sense.vhd(102:57)							
-----+-----+-----+-----							
vdatoxd'above		1		1		1	
region: :read:xi1:xibarra2:ximydsp:xisnsprl_3:yi358						model: sense(functional)	file: /my_working_directory/
MODELS/sense.vhd(105:53)							
-----+-----+-----+-----							
vdatoxd'above		1		1		1	
region: :read:xi1:xibarra2:ximydsp:xisnsprl_3:yi358						model: sense(functional)	file: /my_working_directory/
MODELS/sense.vhd(104:53)							
-----+-----+-----+-----							
vdatoxd'above		1		1		1	
region: :read:xi1:xibarra2:ximydsp:xisnsprl_3:yi358						model: sense(functional)	file: /my_working_directory/
MODELS/sense.vhd(102:57)							
-----+-----+-----+-----							
vdatoxd'above		1		1		1	
region: :read:xi1:xibarra2:ximydsp:xisnsprl_3:yi358						model: sense(functional)	file: /my_working_directory/
MODELS/sense.vhd(101:57)							
-----+-----+-----+-----							
v_ynl-7_12'above		1		1		1	
region: :read:xi1:xibarra2:ximydsp:xicoldec:yiynge						model: yrgen(functional)	file: /my_working_directory/
MODELS/yrgen2.vhd(133:42)							
-----+-----+-----+-----							
v_ynl-6_11'above		1		1		1	
region: :read:xi1:xibarra2:ximydsp:xicoldec:yiynge						model: yrgen(functional)	file: /my_working_directory/
MODELS/yrgen2.vhd(129:42)							
-----+-----+-----+-----							
...		...		...		...	
-----+-----+-----+-----							
Total		122		15		2	
-----+-----+-----+-----							

* Simulation: Digital kernel *							

-----+-----							
Type of Events		Events					
-----+-----							
Questa ADMS only		188					
-----+-----							
Total		188					
-----+-----							

```
*****
*   Simulation: Analog kernel   *
*****
```

```
-----+-----
|      Eldo      |
-----+-----
Newton/Jacobian Order|      1324
-----+-----
Terms in matrix|      16914
-----+-----
Matrix Sparsity(%)| 99.035127
-----+-----
Newton iterations|      160
Average number of Newton iterations| 3.076923
-----+-----
Accepted time steps|      52
-----+-----
Rejected time steps|      18
  due to local truncation error|      1
  due to mixed signal interaction|     17
-----+-----
Evaluation of active devices|    358105
-----+-----
```

```
*****
*   Simulation: Reject, synchro, ...   *
*****
```

Rejection due to Local-Truncation-Error
1e+02% (1/1) due to Node XI1.XIBARRA2.XIMYDSP.CRADICER[1]

Rejection due to time-synchronization
24% (4/17) due to Object XI1.YIVPDREG
18% (3/17) due to Object XI1.XIBARRA2.XIMYDSP.XICOLDEC.YIYNGEN
12% (2/17) due to Object XI1.XIBARRA2.YI109
5.9% (1/17) due to Digital engine
5.9% (1/17) due to Object XI1.XIBARRA2.XIMYDSP.XISNSPRL_2.YI50
5.9% (1/17) due to Object XI1.XIBARRA2.XIMYDSP.XISNSPRL_3.YI358
Other items not dumped because 7.000000e+01% of total number reached

Step limitation due to fast variations of signals
25% (2/8) due to Node XI1.XIBARRA2.XIMYDSP.CRADICER[0]
25% (2/8) due to Node XI1.XIBARRA2.XIMYDSP.CRADICER[1]
12% (1/8) due to Node XI1.XIBARRA2.XIMYDSP.RADICER[3]
12% (1/8) due to Node XI1.XIBARRA2.XIMYDSP.RADICER[1]
Other items not dumped because 7.000000e+01% of total number reached

Step limitation due to time-synchronization
73% (8/11) due to Digital engine
Other items not dumped because 7.000000e+01% of total number reached

Summary of contribution per X instance
35% (13/37) due to Top circuit
22% (8/37) due to XI1.XI1.XIBARRA2.XI1.XIBARRA2.XIMYDSP

```
11% (4/37) due to XI1
8.1% (3/37) due to XI1.XI1.XIBARRA2
Other items not dumped because 7.000000e+01% of total number reached
```

Related Topics

Simulation Statistics

Example: Analyzing Simulation Issues Using the Statistics File

[vasim \[Questa ADMS Command Reference\]](#)

[stat \[Questa ADMS Command Reference\]](#)

Third-Party Information

Details on open source and third-party software that may be included with this product are available in the `<your_software_installation_location>/legal` directory.